

# Contents

|                                                   |               |
|---------------------------------------------------|---------------|
| <b>vLLM 源码结构分析—总览导航</b>                           | <b>19</b>     |
| 全局知识图谱                                            | 19            |
| 一、项目概述                                            | 20            |
| 1.1 基本信息                                          | 20            |
| 1.2 核心价值主张                                        | 20            |
| 1.3 生态地位                                          | 20            |
| 二、整体架构图                                           | 21            |
| 各层职责速查                                            | 22            |
| 三、版本演进路线: v0 → v1 迁移策略                            | 22            |
| 3.1 迁移背景                                          | 22            |
| 3.2 向后兼容机制                                        | 23            |
| 3.3 v0 vs v1 核心差异                                 | 24            |
| 四、目录结构全景图                                         | 24            |
| 五、快速导航索引                                          | 29            |
| 六、关键技术特性一览表                                       | 32            |
| 七、数据流总览                                           | 33            |
| 关键数据结构流转                                          | 34            |
| 八、设计哲学总结                                          | 34            |
| 8.1 配置驱动 (Configuration-Driven)                   | 34            |
| 8.2 注册表模式 (Registry Pattern)                      | 35            |
| 8.3 策略模式 (Strategy Pattern)                       | 35            |
| 8.4 关注点分离 (Separation of Concerns)                | 35            |
| 8.5 异步优先 (Async-First)                            | 35            |
| 8.6 渐进式复杂度 (Progressive Complexity)               | 36            |
| 8.7 平台抽象 (Platform Abstraction)                   | 36            |
| 附录: 关键文件速查                                        | 36            |
| <br><b>vLLM 架构总览</b>                              | <br><b>37</b> |
| 总体架构图                                             | 37            |
| 一、分层架构详解                                          | 38            |
| 1.1 第 1 层: 服务层 (entrypoints/)                     | 38            |
| 1.2 第 2 层: 引擎层 (engine/ + v1/engine/)             | 38            |
| 1.3 第 3 层: 调度核心 (v1/core/sched/)                  | 39            |
| 1.4 第 4 层: 执行器层 (v1/executor/)                    | 40            |
| 1.5 第 5 层: 模型执行器 (model_executor/)                | 41            |
| 1.6 第 6 层: 内核层 (csrc/ + kernels/)                 | 42            |
| 二、v0 → v1 架构演进                                    | 42            |
| 2.1 演进总览                                          | 42            |
| 2.2 v0 LLMEngine → v1 包装策略                        | 42            |
| 2.3 EngineCore 解耦设计                               | 43            |
| 2.4 v1 LLMEngine: 向后兼容的门面                         | 44            |
| 2.5 v1 核心改进点总结                                    | 45            |
| 2.6 向后兼容策略                                        | 46            |
| 三、核心数据结构                                          | 46            |
| 3.1 请求生命周期数据结构                                    | 46            |
| 3.2 Request (内部请求表示)                              | 46            |
| 3.3 EngineInput → EngineCoreRequest → Request 转换链 | 47            |
| 3.4 输出数据结构体系                                      | 49            |
| 3.5 FinishReason 状态机                              | 50            |
| 四、设计原则                                            | 50            |
| 4.1 配置驱动 (Configuration-Driven)                   | 50            |
| 4.2 注册表模式 (Registry Pattern)                      | 51            |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 4.3 策略模式 (Strategy Pattern)                                | 52        |
| 4.4 工厂模式 (Factory Pattern)                                 | 53        |
| 4.5 其他重要设计模式                                               | 53        |
| 五、关键交互时序                                                   | 54        |
| 5.1 完整推理请求生命周期                                             | 54        |
| 5.2 多进程架构下的 IPC 流程                                         | 54        |
| 六、目录导航速查                                                   | 55        |
| 七、扩展阅读指引                                                   | 55        |
| <b>vLLM 配置系统 (Config System) 深度分析</b>                      | <b>55</b> |
| 目录                                                         | 56        |
| 一、VllmConfig 主配置类                                          | 57        |
| 1.1 设计理念: 聚合容器模式                                           | 57        |
| 1.2 完整子配置字段清单                                              | 57        |
| 1.3 Config Hash 与缓存机制                                      | 58        |
| 1.4 __post_init__ 验证流程                                     | 59        |
| 二、各子配置详解                                                   | 59        |
| 2.1 ModelConfig —模型架构与 dtype                               | 59        |
| 2.2 CacheConfig —KV 缓存管理                                   | 61        |
| 2.3 ParallelConfig —并行执行                                   | 62        |
| 2.4 SchedulerConfig —调度策略                                  | 63        |
| 2.5 DeviceConfig —设备检测                                     | 64        |
| 2.6 LoadConfig —权重加载                                       | 64        |
| 2.7 LoRAConfig —LoRA 适配器                                   | 65        |
| 2.8 QuantizationConfig —量化方案                               | 66        |
| 2.9 CompilationConfig —torch.compile 与 CUDA Graphs         | 67        |
| 2.10 SpeculativeConfig —推测解码                               | 68        |
| 2.11 ObservabilityConfig —可观测性                             | 69        |
| 2.12 OffloadConfig / ReasoningConfig / StructuredConfigs 等 | 70        |
| 三、配置验证与合并逻辑                                                | 71        |
| 3.1 model_validator 方法                                     | 71        |
| 3.2 device 配置如何影响其他配置                                      | 71        |
| 四、环境变量覆盖机制                                                 | 72        |
| 4.1 CUDA 兼容性路径设置                                           | 72        |
| 4.2 PyTorch 环境变量预设                                         | 72        |
| 4.3 PyTorch 版本特定 Monkey-Patch                              | 72        |
| 五、OptimizationLevel (O0-O3) 各级别详解                          | 73        |
| 5.1 OptimizationLevel 枚举定义                                 | 73        |
| 5.2 各级别详细配置                                                | 73        |
| 5.3 OptimizationLevel 应用机制                                 | 75        |
| 5.4 Performance Mode 交互                                    | 75        |
| 5.5 各级别对比总览                                                | 76        |
| 附录: 配置类汇总速查表                                               | 76        |
| <b>vLLM 引擎核心 (Engine Core) 深度分析</b>                        | <b>77</b> |
| 目录                                                         | 78        |
| 一、LLMEngine 类                                              | 78        |
| 1.1 初始化流程                                                  | 78        |
| 1.2 核心方法分析                                                 | 79        |
| 1.3 与 EngineCoreClient 的交互                                 | 81        |
| 二、AsyncLLM 异步引擎                                            | 82        |
| 2.1 异步架构设计                                                 | 82        |
| 2.2 高级 API 封装                                              | 83        |
| 2.3 Output Handler 后台循环                                    | 85        |

|                                       |            |
|---------------------------------------|------------|
| 三、EngineCore 解耦执行循环                   | 86         |
| 3.1 EngineCore 基类                     | 86         |
| 3.2 EngineCoreProc 进程封装               | 87         |
| 3.3 核心调度-执行循环                         | 88         |
| 3.4 数据流详解                             | 89         |
| 四、InputProcessor 请求预处理                | 90         |
| 核心职责                                  | 91         |
| process_inputs() 主方法                  | 91         |
| assign_request_id() 一请求 ID 分配         | 93         |
| 五、OutputProcessor 输出后处理               | 93         |
| 架构概览                                  | 93         |
| 核心数据结构                                | 93         |
| process_outputs() 一核心处理方法             | 95         |
| Stream Interval 控制                    | 96         |
| 六、Detokenizer 异步反 Tokenization        | 96         |
| 类层次结构                                 | 96         |
| 工厂方法: 自动选择策略                          | 97         |
| FastIncrementalDetokenizer 一快速路径      | 98         |
| Stop String 检测机制                      | 98         |
| 七、Coordinator 多节点协调                   | 100        |
| 架构角色                                  | 100        |
| DPCoordinator 类                       | 100        |
| DPCoordinatorProc 一核心协调逻辑             | 101        |
| Wave 协议机制                             | 102        |
| 八、异常处理机制                              | 103        |
| 异常传播路径                                | 103        |
| 异常在 AsyncLLM.generate() 中的处理          | 103        |
| Error Propagation 一广播到所有请求            | 104        |
| EngineCore 进程内的错误处理                   | 104        |
| 附录: 处理管线完整数据流图                        | 104        |
| 各阶段数据格式转换总结                           | 105        |
| <b>vLLM 调度系统深度分析</b>                  | <b>106</b> |
| 调度系统架构总览                              | 106        |
| 一、Scheduler 类核心逻辑                     | 107        |
| 1.1 类定义与核心属性                          | 107        |
| 1.2 schedule() 方法完整逻辑                 | 107        |
| 1.3 请求状态管理                            | 109        |
| 1.4 请求抢占机制                            | 110        |
| 二、Continuous Batching 原理              | 110        |
| 2.1 静态批处理 vs Continuous Batching      | 110        |
| 2.2 Chunked Prefill 实现                | 111        |
| 2.3 Prefill-Decode 混合调度               | 111        |
| 三、调度策略细节                              | 111        |
| 3.1 SchedulingPolicy 枚举               | 111        |
| 3.2 FCFS (First-Come-First-Served) 策略 | 111        |
| 3.3 PRIORITY 策略                       | 112        |
| 3.4 双策略调度权衡                           | 112        |
| 3.5 抢占与恢复机制                           | 113        |
| 四、SchedulerOutput 数据结构                | 113        |
| 4.1 类定义                               | 113        |
| 4.2 NewRequestData 结构                 | 114        |
| 4.3 CachedRequestData 结构              | 114        |
| 4.4 下游消费流程                            | 114        |

|                                           |            |
|-------------------------------------------|------------|
| 五、AsyncScheduler 异步调度变体                   | 115        |
| 5.1 类定义                                   | 115        |
| 5.2 核心差异                                  | 115        |
| 5.3 关键方法重写                                | 115        |
| 5.4 异步调度优势                                | 116        |
| 六、调度决策流程图                                 | 116        |
| 七、关键配置参数                                  | 117        |
| 7.1 SchedulerConfig 核心参数                  | 117        |
| 7.2 参数验证逻辑                                | 117        |
| 八、总结                                      | 117        |
| <b>PagedAttention 核心创新深度分析</b>            | <b>118</b> |
| 定位                                        | 118        |
| 核心价值总览                                    | 118        |
| 一、问题背景：传统 KV Cache 的内存困境                  | 119        |
| 1.1 KV Cache 的本质                          | 119        |
| 1.2 传统方案的三大缺陷                             | 119        |
| 二、PagedAttention 设计思想                     | 119        |
| 2.1 操作系统虚拟内存类比                            | 119        |
| 2.2 Block（页）的概念与设计                        | 120        |
| 2.3 逻辑地址 → 物理地址映射（BlockTable）             | 121        |
| 2.4 Block Pool（物理内存池）                     | 123        |
| 三、关键实现深度解析                                | 124        |
| 3.1 KVCacheManager：统一入口与协调者               | 124        |
| 3.2 BlockPool：内存池的精细管理                    | 128        |
| 3.3 SingleTypeKVCacheManager：特定注意力类型的精细管理 | 130        |
| 3.4 MultiGroupBlockTable：多组 BlockTable 管理 | 133        |
| 3.5 KVCacheInterface：接口规范体系               | 134        |
| 四、内存共享机制（Memory Sharing）                  | 135        |
| 4.1 Copy-on-Write (COW) 语义                | 135        |
| 4.2 Beam Search 共享                        | 136        |
| 4.3 Parallel Sampling 共享                  | 137        |
| 五、Prefix Caching 公共前缀自动识别与复用              | 137        |
| 5.1 核心原理                                  | 137        |
| 5.2 自动检测公共前缀                              | 138        |
| 5.3 缓存命中/未命中处理                            | 139        |
| 5.4 与调度器的集成                               | 140        |
| 六、实际效果数据与分析                               | 141        |
| 6.1 内存节省的具体场景                             | 141        |
| 6.2 综合性能提升总结                              | 143        |
| 6.3 Block 分配流程完整数据流                       | 143        |
| 6.4 核心数据结构关系图                             | 144        |
| 七、总结与展望                                   | 146        |
| 7.1 PagedAttention 的核心优势总结                | 146        |
| 7.2 设计哲学                                  | 146        |
| 7.3 对行业的影响                                | 146        |
| 7.4 未来发展方向                                | 146        |
| 参考资料                                      | 147        |
| 核心源码文件                                    | 147        |
| 相关论文                                      | 147        |
| <b>vLLM 注意力后端实现详细分析</b>                   | <b>147</b> |
| 定位                                        | 147        |
| 架构总览                                      | 147        |

|                                           |     |
|-------------------------------------------|-----|
| 一、注意力后端选择器                                | 149 |
| 1.1 核心入口: <code>get_attn_backend()</code> | 149 |
| 1.2 缓存机制                                  | 149 |
| 1.3 Mamba 后端选择器                           | 150 |
| 二、FlashAttention 后端                       | 150 |
| 2.1 后端定义与能力声明                             | 150 |
| 2.2 KV Cache 布局                           | 150 |
| 2.3 Metadata Builder                      | 150 |
| 2.4 核心前向传播                                | 151 |
| 2.5 KV Cache 更新                           | 152 |
| 三、FlashInfer 后端                           | 152 |
| 3.1 后端概述                                  | 152 |
| 3.2 KV Cache 形状差异                         | 153 |
| 3.3 双路径架构: Native vs TRT-LLM              | 153 |
| 3.4 TRT-LLM 路由决策                          | 154 |
| 3.5 NVFP4 量化处理                            | 154 |
| 3.6 DCP 支持                                | 154 |
| 3.7 Fast Plan for CUDA Graph              | 154 |
| 四、MLA (Multi-head Latent Attention)       | 155 |
| 4.1 MLA 概述                                | 155 |
| 4.2 MLA 后端家族                              | 155 |
| 4.3 FlashInfer MLA 实现                     | 155 |
| 4.4 MLA Forward 流程                        | 155 |
| 4.5 MLA Sparse 变体                         | 156 |
| 4.6 MLA Prefill 选择器                       | 156 |
| 五、Triton 注意力                              | 156 |
| 5.1 后端定义                                  | 156 |
| 5.2 Per-Token-Head 量化支持                   | 156 |
| 5.3 统一 Kernel 入口                          | 157 |
| 5.4 2D/3D Kernel 自适应选择                    | 157 |
| 5.5 RoPE 融合支持                             | 157 |
| 六、ROCm 后端—AMD GPU 支持                      | 158 |
| 6.1 后端定义                                  | 158 |
| 6.2 双 Kernel 策略                           | 158 |
| 6.3 Chunked Prefill + Paged Decode        | 158 |
| 6.4 AIter 集成                              | 159 |
| 七、CPU 注意力—CPU fallback 实现                 | 159 |
| 7.1 后端定义                                  | 159 |
| 7.2 ISA 自适应选择                             | 159 |
| 7.3 SDPA 混合策略                             | 160 |
| 7.4 CPU Attention 执行流程                    | 160 |
| 八、其他特殊注意力                                 | 160 |
| 8.1 Mamba/Mamba2 注意力 (SSM)                | 160 |
| 8.2 Linear Attention                      | 161 |
| 8.3 Short Conv Attention                  | 161 |
| 8.4 GDN (Gated DeltaNet) Attention        | 161 |
| 九、注意力操作层 (底层算子)                           | 162 |
| 9.1 Ops 目录结构                              | 162 |
| 9.2 Paged Attention 核心                    | 162 |
| 9.3 Chunked Prefill + Paged Decode        | 162 |
| 9.4 Triton Prefill Attention              | 163 |
| 9.5 Triton Unified Attention              | 163 |
| 9.6 Merge Attention States                | 163 |
| 9.7 DCP AllToAll 通信                       | 163 |

|                                             |            |
|---------------------------------------------|------------|
| 9.8 DeepSeek-V4 专用操作                        | 164        |
| 十、后端选择决策流程                                  | 164        |
| 十一、Registry 系统                              | 165        |
| 11.1 后端注册表                                  | 165        |
| 11.2 动态覆盖机制                                 | 165        |
| 十二、性能优化要点总结                                 | 165        |
| 12.1 各后端适用场景                                | 165        |
| 12.2 关键优化技术                                 | 166        |
| 附录：关键文件索引                                   | 166        |
| <b>07 模型执行器层 (Model Executor) 全貌分析</b>      | <b>166</b> |
| 一、ModelRegistry 模型注册表机制                     | 167        |
| 1.1 核心数据结构                                  | 167        |
| 1.2 注册流程: Lazy Loading                      | 167        |
| 1.3 模型解析流程                                  | 168        |
| 1.4 子进程 inspect 机制                          | 168        |
| 二、支持的模型架构全景 (200+)                          | 168        |
| 2.1 文本生成模型 (Text Generation / Decoder-only) | 168        |
| 2.2 嵌入模型 (Embedding Models)                 | 170        |
| 2.3 多模态模型 (Multimodal Models)               | 170        |
| 2.4 推测解码 / Speculative Decoding 模型          | 171        |
| 三、模型接口定义体系                                  | 171        |
| 3.1 核心接口: VllmModel                         | 171        |
| 3.2 文本生成接口: VllmModelForTextGeneration      | 172        |
| 3.3 Pooling 接口: VllmModelForPooling         | 172        |
| 3.4 扩展能力接口一览 (interfaces.py)                | 172        |
| 四、主要模型家族深入分析                                | 173        |
| 4.1 LLaMA 系列—最广泛的基础架构                       | 173        |
| 4.2 DeepSeek V2/V3 —MoE + MLA (多头潜在注意力)     | 175        |
| 4.3 Qwen2/Qwen3/Qwen3.5 —国产大模型代表            | 176        |
| 4.4 Mistral / Gemma —开源主流架构                 | 176        |
| 4.5 MoE 模型—专家混合架构                           | 177        |
| 4.6 多模态模型—视觉语言模型                            | 177        |
| 五、Transformer 层组件详解                         | 178        |
| 5.1 Attention 层                             | 178        |
| 5.2 MLP / FeedForward 层                     | 179        |
| 5.3 RMSNorm / LayerNorm                     | 179        |
| 5.4 RoPE 位置编码                               | 179        |
| 六、自定义操作注册 (Custom Op)                       | 180        |
| 6.1 CustomOp —基础设施                          | 180        |
| 6.2 注册机制                                    | 180        |
| 6.3 后端分发机制                                  | 181        |
| 6.4 启用/禁用控制                                 | 181        |
| 6.5 PluggableLayer —可插拔层                    | 181        |
| 七、模型注册与加载完整流程                               | 181        |
| 八、架构总结                                      | 182        |
| 8.1 设计亮点                                    | 182        |
| 8.2 模型执行器的角色定位                              | 182        |
| <b>vLLM Worker 与执行框架分析</b>                  | <b>183</b> |
| 定位                                          | 183        |
| 一、Executor 抽象基类                             | 184        |
| 1.1 核心职责                                    | 184        |
| 1.2 类定义与关键属性                                | 184        |

|                                       |     |
|---------------------------------------|-----|
| 1.3 Executor 选择策略                     | 184 |
| 1.4 统一接口定义                            | 185 |
| 二、UniProcExecutor 单进程执行器              | 186 |
| 2.1 适用场景                              | 186 |
| 2.2 实现细节                              | 186 |
| 2.3 ExecutorWithExternalLauncher      | 188 |
| 三、MultiProcExecutor 多进程执行器            | 188 |
| 3.1 适用场景                              | 188 |
| 3.2 进程架构                              | 188 |
| 3.3 初始化流程详解                           | 189 |
| 3.4 进程间通信机制 (MessageQueue)            | 190 |
| 5. WorkerProc 内部实现                    | 191 |
| 3.6 健康监测机制                            | 192 |
| 四、RayExecutorV2 Ray 分布式执行器            | 193 |
| 4.1 适用场景                              | 193 |
| 4.2 类继承关系                             | 193 |
| 4.3 初始化流程 (10 步)                      | 193 |
| 4.4 RayWorkerProc 实现                  | 196 |
| 4.5 健康监测 (基于 ObjectRef)               | 196 |
| 五、WorkerBase Worker 抽象基类              | 197 |
| 5.1 设计目标                              | 197 |
| 5.2 核心接口定义                            | 197 |
| 5.3 关键抽象方法                            | 197 |
| 5.4 execute_model() 接口详解              | 198 |
| 5.5 WorkerWrapperBase 代理类             | 198 |
| 六、GPUWorker GPU 工作进程                  | 199 |
| 6.1 概述                                | 199 |
| 6.2 初始化流程                             | 199 |
| 6.3 设备初始化                             | 200 |
| 6.4 模型加载                              | 201 |
| 6.5 内存 Profiling                      | 201 |
| 6.6 模型执行                              | 202 |
| 6.7 AsyncIntermediateTensors 异步通信     | 203 |
| 6.8 休眠/唤醒机制                           | 203 |
| 6.9 编译与预热                             | 204 |
| 七、GPUModelRunner 模型运行器                | 205 |
| 7.1 概述                                | 205 |
| 7.2 核心依赖与导入                           | 205 |
| 7.3 输入批处理 (InputBatch)                | 205 |
| 7.4 Micro-batching 策略 (UBatching)     | 206 |
| 7.5 模型前向传播编排                          | 207 |
| 7.6 Sampling 执行                       | 208 |
| 7.7 KV Cache 写入                       | 208 |
| 八、CPUWorker / CPUModelRunner CPU 执行路径 | 208 |
| 8.1 CPUWorker 实现                      | 208 |
| 8.2 设备初始化差异                           | 209 |
| 8.3 限制                                | 209 |
| 九、XPUWorker 其他硬件支持                    | 209 |
| 9.1 XPUWorker (Intel GPU)             | 209 |
| 9.2 硬件支持矩阵                            | 210 |
| 十、输入批处理与优化                            | 211 |
| 10.1 InputBatch 架构                    | 211 |
| 10.2 Padding 优化策略                     | 211 |
| 10.3 MultiGroupBlockTable             | 211 |

|                                                    |            |
|----------------------------------------------------|------------|
| 10.4 Micro-batching 流水线 . . . . .                  | 212        |
| 十一、Executor 选择与请求处理流程 . . . . .                    | 212        |
| 11.1 Executor 选择决策树 . . . . .                      | 212        |
| 11.2 典型请求处理流程 . . . . .                            | 213        |
| 11.3 性能关键路径 . . . . .                              | 214        |
| 总结 . . . . .                                       | 214        |
| 架构亮点 . . . . .                                     | 214        |
| 关键代码路径 . . . . .                                   | 214        |
| <b>vLLM KV Cache 系统完整分析 . . . . .</b>              | <b>214</b> |
| 目录 . . . . .                                       | 215        |
| 一、KV Cache Manager 层次 . . . . .                    | 216        |
| 1.1 KVCacheManager — 顶层抽象 . . . . .                | 216        |
| 1.2 SingleTypeKVCacheManager — 单类型基类 . . . . .     | 217        |
| 1.3 具体管理器子类 . . . . .                              | 218        |
| 1.4 EncoderCacheManager — 编码器缓存 . . . . .          | 219        |
| 1.5 层次关系总览 . . . . .                               | 220        |
| 二、KV Cache Coordinator 多 GPU 缓存协调 . . . . .        | 221        |
| 2.1 三种 Coordinator 变体 . . . . .                    | 221        |
| 2.2 HybridKVCacheCoordinator — 固点算法 . . . . .      | 221        |
| 2.3 Coordinator 工厂函数 . . . . .                     | 222        |
| 三、KV Cache 接口定义 . . . . .                          | 222        |
| 3.1 KVCacheConfig — 全局配置 . . . . .                 | 222        |
| 3.2 KVCacheSpec 层次体系 . . . . .                     | 223        |
| 3.3 UniformTypeKVCacheSpecs . . . . .              | 224        |
| 四、KV Cache Metrics 缓存使用统计 . . . . .                | 224        |
| 4.1 架构设计 . . . . .                                 | 224        |
| 4.2 核心实现 . . . . .                                 | 225        |
| 五、KV Offload CPU 卸载 . . . . .                      | 225        |
| 5.1 整体架构 . . . . .                                 | 225        |
| 5.2 OffloadingManager 抽象接口 . . . . .               | 226        |
| 5.3 CPUOffloadingManager 实现 . . . . .              | 226        |
| 5.4 缓存策略 . . . . .                                 | 227        |
| 5.5 FilterReusedOffloadingManager — 复用门控 . . . . . | 227        |
| 5.6 Worker 侧传输引擎 . . . . .                         | 227        |
| 六、Simple KV Offload . . . . .                      | 228        |
| 6.1 架构对比 . . . . .                                 | 228        |
| 6.2 SimpleCPUOffloadScheduler . . . . .            | 229        |
| 6.3 SimpleCPUOffloadWorker . . . . .               | 229        |
| 6.4 Metadata 定义 . . . . .                          | 229        |
| 七、KV Transfer 跨节点传输 . . . . .                      | 230        |
| 7.1 KVConnectorBase_V1 核心接口 . . . . .              | 230        |
| 7.2 连接器注册与发现 . . . . .                             | 231        |
| 7.3 支持的连接器类型 . . . . .                             | 231        |
| 7.4 Disaggregated Prefill  workflow . . . . .      | 232        |
| 八、Block Table 管理 . . . . .                         | 232        |
| 8.1 BlockTable . . . . .                           | 232        |
| 8.2 MultiGroupBlockTable . . . . .                 | 233        |
| 8.3 Slot Mapping Kernel . . . . .                  | 233        |
| 九、前缀缓存 Prefix Caching . . . . .                    | 233        |
| 9.1 查找最长缓存命中 . . . . .                             | 234        |
| 9.2 各 Manager 的查找策略对比 . . . . .                    | 234        |
| 9.3 FullAttentionManager 详细流程 . . . . .            | 234        |
| 9.4 SlidingWindowManager 详细流程 . . . . .            | 234        |



|                                                    |            |
|----------------------------------------------------|------------|
| 9.5 缓存命中后的处理路径                                     | 235        |
| 9.6 BlockPool 中的缓存数据结构                             | 235        |
| 9.7 与调度器的集成                                        | 235        |
| 十、数据流全景图                                           | 235        |
| 附录: 关键文件索引                                         | 237        |
| <b>10. 采样与生成策略分析</b>                               | <b>237</b> |
| 总体架构概览                                             | 238        |
| 一、SamplingParams 采样参数详解                            | 238        |
| 1.1 核心参数定义与默认值                                     | 238        |
| 1.2 Penalty 惩罚机制参数                                 | 239        |
| 1.3 Stop Sequences 与 Token 控制                      | 240        |
| 1.4 Logprobs 相关参数                                  | 240        |
| 1.5 其他重要参数                                         | 240        |
| 1.6 参数验证逻辑                                         | 241        |
| 1.7 采样类型判断                                         | 241        |
| 二、Sampler 采样核心                                     | 241        |
| 2.1 整体处理流程                                         | 241        |
| 2.2 forward() 主入口方法                                | 242        |
| 2.3 apply_logits_processors() - Logits Processor 链 | 243        |
| 2.4 sample() 核心采样方法                                | 243        |
| 2.5 Temperature 应用逻辑                               | 244        |
| 2.6 Logprobs 计算与收集                                 | 245        |
| 2.7 特定 Token ID 的 Logprobs 收集 (高效模式)               | 245        |
| 三、RejectionSampler 拒绝采样                            | 246        |
| 3.1 核心概念                                           | 246        |
| 3.2 forward() 主流程                                  | 246        |
| 3.3 rejection_sample() 核心算法                        | 247        |
| 3.4 Triton Kernel 实现细节                             | 248        |
| 3.5 Recovered Token 采样                             | 250        |
| 3.6 apply_sampling_constraints()                   | 250        |
| 四、ParallelSampling 并行采样                            | 251        |
| 4.1 ParentRequest 数据结构                             | 251        |
| 4.2 初始化逻辑                                          | 251        |
| 4.3 子请求参数生成                                        | 252        |
| 4.4 子请求信息获取                                        | 252        |
| 4.5 输出聚合与完成检测                                      | 252        |
| 4.6 统计信息记录                                         | 253        |
| 五、Logprobs 计算与追踪                                   | 253        |
| 5.1 数据结构定义 (v0 层)                                  | 253        |
| 5.2 LogprobsProcessor (v1 Engine 层)                | 254        |
| 5.3 UTF-8 替换字符修正                                   | 256        |
| 5.4 Top-Logprobs 实现机制                              | 257        |
| 六、ThinkingBudgetState 推理预算控制                       | 258        |
| 6.1 核心功能                                           | 258        |
| 6.2 初始化                                            | 258        |
| 6.3 Per-Request 状态初始化                              | 259        |
| 6.4 状态更新逻辑                                         | 259        |
| 6.5 Think State 更新核心逻辑                             | 260        |
| 6.6 Logits 强制应用                                    | 261        |
| 七、Speculative Decode Metadata 推测解码元数据              | 261        |
| 7.1 数据结构定义                                         | 261        |
| 7.2 字段说明与用途                                        | 262        |
| 7.3 Dummy Metadata 工厂方法                            | 262        |

|                                      |            |
|--------------------------------------|------------|
| 7.4 在 RejectionSampler 中的使用          | 263        |
| 八、完整采样数据流图                           | 264        |
| 8.1 从 Logits 到 Token 的端到端流程          | 264        |
| 8.2 Speculative Decoding 采样流程        | 265        |
| 8.3 关键数值常量                           | 265        |
| 九、性能优化要点                             | 265        |
| 9.1 内存优化                             | 265        |
| 9.2 计算优化                             | 265        |
| 9.3 缓存优化                             | 266        |
| 十、总结                                 | 266        |
| <b>vLLM 多模态处理管线分析</b>                | <b>266</b> |
| 目录                                   | 267        |
| 一、MultiModalRegistry 多模态处理器注册表       | 268        |
| 1.1 注册机制                             | 268        |
| 1.2 处理器创建流程                          | 269        |
| 1.3 模态支持检查                           | 269        |
| 1.4 缓存管理                             | 270        |
| 1.5 Dummy Inputs 生成                  | 270        |
| 二、媒体连接器 Media Connectors             | 270        |
| 2.1 基类 MediaIO                       | 270        |
| 2.2 Image Connector — 图像加载与预处理       | 271        |
| 2.3 Audio Connector — 音频处理           | 271        |
| 2.4 Video Connector — 视频帧提取          | 272        |
| 2.5 统一接口 MediaConnector              | 273        |
| 三、处理器 Processors                     | 274        |
| 3.1 处理上下文 InputProcessingContext     | 274        |
| 3.2 BaseProcessingInfo — 模型特定信息      | 274        |
| 3.3 BaseMultiModalProcessor — 核心处理基类 | 275        |
| 3.4 Prompt Update 机制                 | 276        |
| 3.5 HF Processor 集成                  | 277        |
| 3.6 缓存感知处理                           | 277        |
| 3.7 Dummy Inputs Builder             | 278        |
| 3.8 ProcessorInputs — 输入数据结构         | 278        |
| 3.9 EncDecMultiModalProcessor        | 279        |
| 四、输入解析                               | 279        |
| 4.1 MultiModalDataParser — 数据归一化引擎   | 279        |
| 4.2 各模态解析逻辑                          | 280        |
| 4.3 数据项类层次                           | 281        |
| 4.4 嵌入验证                             | 282        |
| 五、特征提取管线                             | 282        |
| 5.1 视觉编码器                            | 282        |
| 5.2 音频编码器                            | 282        |
| 5.3 Encoder Budget Management        | 282        |
| 六、支持的多模态模型                           | 284        |
| 6.1 LLaVA-NeXT                       | 284        |
| 6.2 Qwen2-VL / Qwen-VL 系列            | 284        |
| 6.3 Pixtral                          | 285        |
| 七、Assets 系统                          | 285        |
| 7.1 基础设施                             | 285        |
| 7.2 图像资源                             | 286        |
| 7.3 音频资源                             | 286        |
| 7.4 视频资源                             | 287        |
| 八、端到端数据流                             | 287        |

|                               |            |
|-------------------------------|------------|
| 关键设计原则总结                      | 288        |
| <b>vLLM 量化方案全面分析</b>          | <b>288</b> |
| 目录                            | 289        |
| 一、量化方案矩阵                      | 290        |
| 在线量化 (Online Quantization)    | 290        |
| 所有已注册量化方法                     | 290        |
| 二、FP8 量化                      | 291        |
| 2.1 架构概述                      | 291        |
| 2.2 Fp8Config 核心配置            | 291        |
| 2.3 量化 Key 体系                 | 292        |
| 2.4 离线 FP8 Linear 方法          | 292        |
| 2.5 在线 FP8 量化                 | 293        |
| 2.6 FP8 MoE 方法                | 293        |
| 2.7 MXFP8 (MicroScaling FP8)  | 293        |
| 三、INT8 / INT4 量化 (GPTQ / AWQ) | 293        |
| 3.1 GPTQ 量化                   | 293        |
| 3.2 AWQ 量化                    | 295        |
| 3.3 AWQ → Marlin 格式转换         | 295        |
| 四、GGUF 格式                     | 296        |
| 4.1 概述                        | 296        |
| 4.2 支持的量化类型                   | 296        |
| 4.3 GGUF GEMM 操作              | 296        |
| 4.4 GGUF MoE 支持               | 296        |
| 4.5 GGUF 工具函数                 | 297        |
| 五、NVFP4 Blackwell 4-bit 浮点    | 297        |
| 5.1 概述                        | 297        |
| 5.2 数据结构                      | 297        |
| 5.3 量化 Key 定义                 | 297        |
| 5.4 NVFP4 内核后端                | 297        |
| 5.5 NVFP4 工具函数                | 298        |
| 六、MXFP4 / MXFP8 块缩放浮点         | 298        |
| 6.1 MXFP4 (MicroScaling FP4)  | 298        |
| 6.2 MXFP8 (MicroScaling FP8)  | 298        |
| 七、Marlin 高性能 4-bit 内核         | 299        |
| 7.1 概述                        | 299        |
| 7.2 支持的量化类型                   | 299        |
| 7.3 Marlin 约束条件               | 299        |
| 7.4 Marlin 权重预处理              | 299        |
| 7.5 Marlin Workspace          | 300        |
| 7.6 Marlin GEMM 调用            | 300        |
| 7.7 W4A8 扩展: INT8 / FP8 激活量化  | 300        |
| 八、Machete 新一代量化内核             | 300        |
| 8.1 概述                        | 300        |
| 8.2 Machete 约束                | 300        |
| 8.3 MacheteLinearKernel       | 301        |
| 8.4 Marlin vs Machete 对比      | 301        |
| 九、量化线性层与 GEMM 操作              | 301        |
| 9.1 内核选择体系                    | 301        |
| 9.2 ScaledMM 内核层次             | 302        |
| 9.3 Mixed Precision 内核层次      | 302        |
| 十、量化配置体系                      | 303        |
| 10.1 基类层次                     | 303        |
| 10.2 配置类清单                    | 303        |

|                                            |            |
|--------------------------------------------|------------|
| 10.3 参数类型体系                                | 304        |
| 10.4 自动格式检测与覆盖                             | 304        |
| 十一、量化方案选择决策树                               | 304        |
| 十二、量化数据流全景                                 | 305        |
| 典型数据流示例                                    | 306        |
| 附录：关键源码路径索引                                | 307        |
| <b>vLLM 分布式计算分析</b>                        | <b>308</b> |
| 目录                                         | 309        |
| 一、并行策略                                     | 309        |
| 1.1 Tensor Parallelism (TP) —张量并行          | 310        |
| 1.2 Pipeline Parallelism (PP) —流水线并行       | 310        |
| 1.3 Data Parallelism (DP) —数据并行            | 310        |
| 1.4 Expert Parallelism (EP) —专家并行 (MoE 场景) | 311        |
| 并行策略选择流程                                   | 311        |
| 二、并行状态管理                                   | 312        |
| 2.1 GroupCoordinator 通信组协调器                | 312        |
| 2.2 并行组初始化流程                               | 313        |
| 2.3 NCCL 通信域建立                             | 314        |
| 2.4 StatelessGroupCoordinator 无状态协调器       | 315        |
| 三、设备通信器 Device Communicators               | 315        |
| 3.1 基类 DeviceCommunicatorBase              | 315        |
| 3.2 CudaCommunicator —CUDA 通信器             | 316        |
| 3.3 PyNcclCommunicator —PyNCL NCCL 包装      | 318        |
| 3.4 CustomAllreduce —自定义 AllReduce         | 318        |
| 3.5 QuickAllReduce —快速 AllReduce (ROCm)    | 320        |
| 3.6 MessageQueue —共享内存广播                   | 320        |
| 3.7 Ray/CPU/XPU 通信器                        | 321        |
| 3.8 All2AllManager —MoE 全对全通信              | 321        |
| 四、数据传输机制                                   | 322        |
| 4.1 KV Transfer —跨节点 KV 缓存传输               | 322        |
| 4.2 Weight Transfer —权重分发                  | 323        |
| 4.3 EC Transfer —纠删码传输                     | 324        |
| 五、高级特性                                     | 324        |
| 5.1 Elastic EP —弹性专家并行                     | 324        |
| 5.2 EPLB —弹性负载均衡                           | 326        |
| 5.3 Stateless Coordinator —无状态协调器          | 327        |
| 5.4 NIXL Utils —NIXL 工具集                   | 327        |
| 六、通信流程总览                                   | 328        |
| 典型请求处理中的通信模式                               | 328        |
| AllReduce 路径选择决策                           | 328        |
| 七、文件索引                                     | 329        |
| <b>vLLM LoRA 适配器系统深度解析</b>                 | <b>329</b> |
| 定位                                         | 329        |
| 一、LoRA 概述                                  | 330        |
| 1.1 Low-Rank Adaptation 原理                 | 330        |
| 二、LoRAConfig 配置参数                          | 331        |
| 2.1 关键配置项                                  | 331        |
| 2.2 配置校验逻辑                                 | 331        |
| 2.3 Hash 计算                                | 332        |
| 三、LoRAModel 模型封装                           | 332        |
| 3.1 数据结构                                   | 332        |
| 3.2 从 Tensor 构建                            | 333        |

|                                                   |            |
|---------------------------------------------------|------------|
| 3.3 从本地 Checkpoint 加载 . . . . .                   | 333        |
| 3.4 Clone 与查询接口 . . . . .                         | 333        |
| 四、LoRAWeights 权重管理 . . . . .                      | 334        |
| 4.1 LoRALayerWeights 单层权重 . . . . .               | 334        |
| 4.2 PackedLoRALayerWeights 打包权重 . . . . .         | 334        |
| 4.3 Dummy 权重创建 . . . . .                          | 335        |
| 五、LoRA Request 请求级绑定 . . . . .                    | 335        |
| 5.1 数据结构 . . . . .                                | 335        |
| 5.2 关键设计决策 . . . . .                              | 336        |
| 六、LoRA Layer 层实现 . . . . .                        | 336        |
| 6.1 类继承体系总览 . . . . .                             | 337        |
| 6.2 BaseLayerWithLoRA 抽象基类 . . . . .              | 337        |
| 6.3 BaseLinearLayerWithLoRA Linear 公共基类 . . . . . | 338        |
| 6.4 ColumnParallelLinearWithLoRA . . . . .        | 339        |
| 6.5 RowParallelLinearWithLoRA . . . . .           | 340        |
| 6.6 FusedMoEWithLoRA . . . . .                    | 341        |
| 6.7 LogitsProcessorWithLoRA . . . . .             | 342        |
| 6.8 ReplicatedLinearWithLoRA . . . . .            | 342        |
| 6.9 VocabParallelEmbeddingWithLoRA . . . . .      | 342        |
| 七、Punica Wrapper GPU LoRA 计算库集成 . . . . .         | 343        |
| 7.1 架构层次 . . . . .                                | 343        |
| 7.2 PunicaWrapperBase 一元数据中心 . . . . .            | 343        |
| 7.3 PunicaWrapperGPU Triton 实现 . . . . .          | 344        |
| 7.4 PunicaWrapperCPU PyTorch Op 实现 . . . . .      | 346        |
| 7.5 PunicaWrapperXPU Intel XPU 实现 . . . . .       | 346        |
| 八、LoRA Resolver 适配器解析 . . . . .                   | 347        |
| 8.1 抽象接口 . . . . .                                | 347        |
| 8.2 注册中心 . . . . .                                | 347        |
| 九、WorkerManager Worker 级管理 . . . . .              | 347        |
| 9.1 架构角色 . . . . .                                | 347        |
| 9.2 WorkerLoRAManager 基础管理器 . . . . .             | 348        |
| 9.3 LRUCacheWorkerLoRAManager LRU 缓存管理器 . . . . . | 349        |
| 十、动态加载/卸载机制 . . . . .                             | 350        |
| 10.1 完整请求处理流程 . . . . .                           | 350        |
| 10.2 Slot-Based 权重管理 . . . . .                    | 351        |
| 10.3 LoRAMapping Token 到 Adapter 的映射 . . . . .    | 351        |
| 10.4 性能优化要点 . . . . .                             | 351        |
| 附录: 关键文件索引 . . . . .                              | 351        |
| <b>vLLM 服务接口层 (API Layer) 源码分析 . . . . .</b>      | <b>352</b> |
| 目录 . . . . .                                      | 353        |
| 一、OpenAI 兼容 API . . . . .                         | 353        |
| 1.1 FastAPI 服务器入口 . . . . .                       | 353        |
| 1.2 路由注册体系 . . . . .                              | 355        |
| 1.3 核心端点定义 . . . . .                              | 355        |
| 1.4 流式响应 SSE 实现 . . . . .                         | 356        |
| 1.5 Orca Metrics . . . . .                        | 356        |
| 二、Anthropic API . . . . .                         | 357        |
| 2.1 协议模型 protocol.py . . . . .                    | 357        |
| 2.2 路由定义 api_router.py . . . . .                  | 357        |
| 2.3 消息格式转换 serving.py . . . . .                   | 358        |
| 三、gRPC Server . . . . .                           | 359        |
| 架构概要 . . . . .                                    | 359        |
| 关键特性 . . . . .                                    | 359        |

|                                          |            |
|------------------------------------------|------------|
| 四、CLI 入口                                 | 360        |
| 4.1 命令分发架构                               | 360        |
| 4.2 各命令详解                                | 360        |
| 五、Batch Serving 离线批处理                    | 361        |
| 整体流程                                     | 361        |
| 请求/响应模型                                  | 362        |
| 端点注册表机制                                  | 362        |
| 文件 I/O                                   | 363        |
| 六、SageMaker 集成                           | 363        |
| 路由设计                                     | 363        |
| 关键特性                                     | 363        |
| 七、MCP Tool Server 模型上下文协议工具服务            | 363        |
| 架构设计                                     | 364        |
| ToolServer 抽象基类                          | 364        |
| MCPToolServer 一远程 MCP 工具连接器              | 364        |
| DemoToolServer 一内置演示工具                   | 364        |
| 八、Pooling 端点 嵌入池化接口                      | 365        |
| 目录结构                                     | 365        |
| 工厂函数—factories.py                        | 366        |
| 支持的任务类型映射                                | 366        |
| 九、LLM 高级 API                             | 366        |
| 类概览                                      | 366        |
| 核心方法体系                                   | 367        |
| 9.1 generate() 一文本生成                     | 367        |
| 9.2 encode() 一通用池化                       | 368        |
| 9.3 chat() 一对话生成                         | 368        |
| 9.4 异步流水线模式                              | 368        |
| 9.5 束搜索 beam_search()                    | 368        |
| 9.6 权重传输 (RL 训练支持)                       | 369        |
| 十、SSL/TLS 支持                             | 369        |
| SSLCertRefresher                         | 369        |
| 十一、请求日志与访问过滤                             | 369        |
| UvicornAccessLogFilter                   | 369        |
| 配置生成                                     | 370        |
| 十二、API 请求路由与处理流程总览                       | 370        |
| 中间件处理顺序                                  | 371        |
| EngineClient 与 Serving 层的关系              | 371        |
| <b>vLLM CUDA/C++ 内核层深度分析</b>             | <b>372</b> |
| 目录                                       | 372        |
| 一、构建系统                                   | 373        |
| 1.1 整体架构概述                               | 373        |
| 1.2 Python 绑定层 (vllm/kernels/)           | 373        |
| 1.3 torch_utils 扩展编译方式                   | 374        |
| 1.4 平台导入机制                               | 375        |
| 二、注意力内核                                  | 375        |
| 2.1 注意力后端架构                              | 375        |
| 2.2 FlashInfer 后端详解                      | 376        |
| 2.3 MLA (Multi-Head Latent Attention) 内核 | 377        |
| 2.4 注意力后端选择策略                            | 378        |
| 三、缓存内核                                   | 379        |
| 3.1 reshape_and_cache_kernel - KV 缓存写入核心 | 379        |
| 3.2 NVFP4 KV Cache 支持                    | 380        |
| 3.3 KV Cache 量化格式支持                      | 381        |

|                                                |            |
|------------------------------------------------|------------|
| 四、激活函数内核                                       | 381        |
| 4.1 RMSNorm 和 FusedAddRMSNorm                  | 381        |
| 4.2 RoPE (Rotary Position Embedding) 内核        | 382        |
| 4.3 SiLU * FP8 融合操作                            | 382        |
| 五、量化内核                                         | 383        |
| 5.1 量化内核架构总览                                   | 383        |
| 5.2 AWQ (Activation-aware Weight Quantization) | 383        |
| 5.3 Marlin - 高性能 4-bit GEMM Engine             | 384        |
| 5.4 Machete - 新一代 CUTLASS GEMM                 | 385        |
| 5.5 GGUF 反量化内核                                 | 386        |
| 5.6 CUTLASS 扩展内核                               | 387        |
| 5.7 ScaledMM - 缩放矩阵乘法框架                        | 387        |
| 六、MoE 内核                                       | 388        |
| 6.1 MoE 架构概述                                   | 388        |
| 6.2 Mixtral-style MoE 路由器                      | 389        |
| 6.3 WNA16 (4-bit MoE) 量化                       | 390        |
| 6.4 MXFP8 MoE 线性层                              | 390        |
| 七、通信内核                                         | 390        |
| 7.1 Custom AllReduce - 自定义全归约                  | 390        |
| 7.2 CU Memory Allocator - CUDA 内存管理器           | 391        |
| 八、采样内核                                         | 392        |
| 8.1 Top-k / Top-p 采样器                          | 392        |
| 8.2 TopKTopPSampler - 核心采样 Kernel              | 393        |
| 九、平台特定内核                                       | 394        |
| 9.1 ROCm (AMD GPU) 内核                          | 394        |
| 9.2 CPU 内核                                     | 395        |
| 9.3 Intel XPU 内核                               | 396        |
| 十、内核分类与调用链总览                                   | 396        |
| 10.1 完整内核分类图                                   | 396        |
| 10.2 典型调用链示例                                   | 397        |
| 10.3 性能优化策略总结                                  | 398        |
| 10.4 关键文件索引                                    | 398        |
| 总结                                             | 398        |
| 架构特点                                           | 399        |
| 性能优化核心                                         | 399        |
| 未来方向                                           | 399        |
| <b>vLLM 编译与运行时优化分析</b>                         | <b>399</b> |
| 一、torch.compile 集成                             | 400        |
| 1.1 Dynamo + Inductor 编译流水线原理                  | 400        |
| 1.2 CustomOp 自定义算子注册                           | 401        |
| 1.3 编译后端选择                                     | 401        |
| 二、编译 Passes                                    | 402        |
| 2.1 Pass 管理架构                                  | 402        |
| 2.2 InductorPass 基类                            | 403        |
| 2.3 VllmInductorPass 增强                        | 403        |
| 2.4 FX 工具函数                                    | 404        |
| 三、CUDA Graphs                                  | 404        |
| 3.1 架构总览                                       | 404        |
| 3.2 CUDAGraphWrapper 一核心包装器                    | 405        |
| 3.3 Piecewise CUDAGraphs 一子图捕获 (推荐模式)          | 406        |
| 3.4 Full CUDAGraphs 一整体捕获                      | 407        |
| 3.5 Encoder CUDA Graphs 一视觉编码器图捕获              | 407        |
| 四、Partition Rules 图分割规则                        | 408        |

|                                       |            |
|---------------------------------------|------------|
| 五、Codegen 代码生成                        | 409        |
| 六、Caching 编译缓存                        | 409        |
| 6.1 VllmSerializableFunction          | 409        |
| 6.2 StandaloneCompiledArtifacts       | 410        |
| 6.3 反序列化与重建                           | 410        |
| 6.4 缓存键因子                             | 411        |
| 七、Monitor 编译监控                        | 411        |
| 7.1 monitor_torch_compile             | 411        |
| 7.2 CUDA Graph 捕获合法性验证                | 412        |
| 八、Counter 性能计数                        | 412        |
| 九、BaseStaticGraph 静态图基类               | 413        |
| 十、PiecewiseBackend 分段后端               | 413        |
| 10.1 核心职责                             | 413        |
| 10.2 双模式初始化                           | 414        |
| 10.3 Shape 分发                         | 414        |
| 10.4 具体参数创建                           | 414        |
| 10.5 序列化                              | 415        |
| 十一、Triton Utils Triton 编译工具           | 415        |
| 11.1 importing - Triton 可用性检测         | 415        |
| 11.2 jit_monitor - JIT 编译监控           | 416        |
| 11.3 allocation - Triton 内存分配器        | 416        |
| 十二、OptimizationLevel O0-O3 含义详解       | 416        |
| 12.1 各级别总览                            | 416        |
| 12.2 详细配置对照表                          | 417        |
| 12.3 条件启用函数详解                         | 418        |
| 12.4 默认值                              | 418        |
| 附录: 编译流水线完整 Mermaid 图                 | 418        |
| A.1 从模型到可执行函数的完整流程                    | 418        |
| A.2 CUDA Graph 捕获与回放详细流程              | 419        |
| A.3 编译 Pass 执行数据流                     | 420        |
| <b>vLLM 结构化输出约束分析文档</b>               | <b>420</b> |
| 定位                                    | 420        |
| 一、StructuredOutputsConfig 配置          | 421        |
| 源码位置                                  | 421        |
| 核心配置类                                 | 421        |
| 关键源码片段                                | 421        |
| 二、后端实现（四种后端逐一深入分析）                    | 422        |
| 类型系统基础                                | 422        |
| 2.1 XgrammarBackend - 高性能语法约束         | 423        |
| 2.2 OutlinesBackend - 基于 FSM 的约束引擎    | 426        |
| 2.3 LMFormatEnforcerBackend - 轻量级格式强制 | 429        |
| 2.4 GuidanceBackend - Guidance 库集成    | 431        |
| 三、支持的结构类型对比                           | 433        |
| 3.1 各后端支持矩阵                           | 433        |
| 3.2 详细说明                              | 434        |
| 四、Request 级别配置                        | 435        |
| StructuredOutputRequest 类             | 435        |
| 五、工具函数与辅助模块                           | 437        |
| 5.1 Utils 工具函数                        | 437        |
| 六、结构化输出处理流程                           | 440        |
| 流程详解                                  | 440        |
| 七、后端选择策略与建议                           | 442        |
| Auto 模式选择逻辑                           | 442        |



|                                                               |            |
|---------------------------------------------------------------|------------|
| 性能特征对比 . . . . .                                              | 442        |
| 兼容性注意事项 . . . . .                                             | 442        |
| 八、关键设计模式总结 . . . . .                                          | 442        |
| 8.1 抽象工厂模式 . . . . .                                          | 442        |
| 8.2 Strategy 模式 . . . . .                                     | 443        |
| 8.3 Observer 模式 (Bitmask 更新) . . . . .                        | 443        |
| 8.4 Lazy Initialization & Async Compilation . . . . .         | 443        |
| 九、扩展指南 . . . . .                                              | 443        |
| 添加新的结构化输出后端 . . . . .                                         | 443        |
| 十、常见问题排查 . . . . .                                            | 444        |
| 问题 1: ValueError - Unsupported JSON Schema features . . . . . | 444        |
| 问题 2: Regex does not have universal start state . . . . .     | 444        |
| 问题 3: LM Format Enforcer 不支持 Speculative Decoding . . . . .   | 444        |
| 问题 4: Grammar 编译超时 . . . . .                                  | 444        |
| 附录: 核心数据结构关系图 . . . . .                                       | 445        |
| 参考资源 . . . . .                                                | 446        |
| 源码文件索引 . . . . .                                              | 446        |
| 外部依赖 . . . . .                                                | 447        |
| <b>vLLM 推测解码 (Speculative Decoding) 加速技术分析</b>                | <b>447</b> |
| 一、推测解码原理 . . . . .                                            | 447        |
| 1.1 Draft Model + Target Model 架构 . . . . .                   | 447        |
| 1.2 Rejection Sampling 算法 . . . . .                           | 448        |
| 1.3 典型加速比 2-4x 分析 . . . . .                                   | 448        |
| 1.4 与普通自回归解码对比 . . . . .                                      | 448        |
| 二、SpeculativeConfig 配置 . . . . .                              | 449        |
| 2.1 核心参数 . . . . .                                            | 449        |
| 2.2 N-gram 配置 . . . . .                                       | 449        |
| 2.3 Suffix Decoding 配置 . . . . .                              | 449        |
| 2.4 Rejection Sampling 配置 . . . . .                           | 449        |
| 2.5 支持的方法类型汇总 . . . . .                                       | 450        |
| 2.6 自动检测逻辑 . . . . .                                          | 450        |
| 三、Draft Model 接口体系 . . . . .                                  | 450        |
| 3.1 抽象基类: SpecDecodeBaseProposer . . . . .                    | 450        |
| 3.2 核心 propose() 方法 . . . . .                                 | 451        |
| 3.3 set_inputs_first_pass() 输入构建 . . . . .                    | 451        |
| 3.4 权重共享机制 . . . . .                                          | 451        |
| 3.5 DraftModelProposer . . . . .                              | 452        |
| 四、推测解码方法详解 . . . . .                                          | 452        |
| 4.1 EAGLE Early Exit Layer . . . . .                          | 452        |
| 4.2 Medusa 多头预测 . . . . .                                     | 453        |
| 4.3 N-gram Proposer (CPU 版) . . . . .                         | 454        |
| 4.4 N-gram GPU Proposer (GPU 加速版) . . . . .                   | 455        |
| 4.5 DFlash 推测解码 . . . . .                                     | 457        |
| 4.6 Suffix Decoding (后缀解码) . . . . .                          | 458        |
| 4.7 Gemma4 MTP (Multi-Token Prediction) . . . . .             | 459        |
| 4.8 MLP Speculator (MLP 草稿模型) . . . . .                       | 460        |
| 4.9 LLM Base Proposer (基于 LLM 的草稿) . . . . .                  | 461        |
| 五、Extract Hidden States (隐藏状态提取) . . . . .                    | 461        |
| 5.1 用途 . . . . .                                              | 461        |
| 5.2 核心特性 . . . . .                                            | 461        |
| 5.3 propose() 行为 . . . . .                                    | 462        |
| 六、Metrics (指标体系) . . . . .                                    | 462        |
| 6.1 SpecDecodingStats . . . . .                               | 462        |

|                                              |            |
|----------------------------------------------|------------|
| 6.2 日志指标                                     | 463        |
| 6.3 Prometheus 指标                            | 463        |
| 七、辅助模块                                       | 463        |
| 7.1 utils.py — Triton Kernels 与工具函数          | 463        |
| 7.2 metadata.py — SpecDecodeMetadata         | 464        |
| 八、推测解码完整 workflow                            | 464        |
| 九、方法选型指南                                     | 465        |
| 十、关键源码文件索引                                   | 466        |
| <b>vLLM 可观测性分析: Metrics &amp; Monitoring</b> | <b>466</b> |
| 一、指标收集系统                                     | 467        |
| 1.1 性能指标—perf.py (MFU / 带宽估算)                | 467        |
| 1.2 统计数据—stats.py                            | 469        |
| 1.3 Prometheus 导出端点—prometheus.py            | 471        |
| 1.4 指标读取接口—reader.py                         | 472        |
| 1.5 指标工具函数—utils.py                          | 472        |
| 二、日志系统                                       | 473        |
| 2.1 StatLogger 工厂体系—loggers.py               | 473        |
| 2.2 Ray 日志适配—ray_wrappers.py                 | 475        |
| 三、StatLoggerFactory / StatLoggerManager 详解   | 476        |
| 3.1 类型别名体系                                   | 476        |
| 3.2 插件加载机制                                   | 476        |
| 3.3 StatLoggerManager 初始化流程                  | 477        |
| 四、OpenTelemetry 集成                           | 477        |
| 4.1 OTel 初始化—otel.py                         | 477        |
| 4.2 自动化 Instrumentation                      | 478        |
| 4.3 Trace Context 传播                         | 478        |
| 4.4 Span 属性标准—tracing/utils.py               | 479        |
| 五、Profiling 性能分析                             | 479        |
| 5.1 Profiler 包装器体系—wrapper.py                | 479        |
| 5.2 逐层性能分析—layerwise_profile.py              | 480        |
| 5.3 Profiler 工具函数—utils.py                   | 482        |
| 5.4 NVTX Hooks —nvtx_pytorch_hooks.py        | 482        |
| 六、Usage Context & Usage Lib                  | 483        |
| 6.1 使用情况追踪—usage_lib.py                      | 483        |
| 七、Observability Config                       | 484        |
| 7.1 配置定义—observability.py                    | 484        |
| 八、指标收集与导出架构全景                                | 485        |
| 核心数据流路径                                      | 486        |
| 附录: 关键文件索引                                   | 487        |
| <b>vLLM 设计模式汇总与架构洞察</b>                      | <b>487</b> |
| 一、设计模式清单 (9 种核心模式)                           | 488        |
| 1. 策略模式 (Strategy Pattern)                   | 488        |
| 2. 工厂模式 (Factory Pattern)                    | 489        |
| 3. 观察者模式 (Observer Pattern)                  | 491        |
| 4. 注册表模式 (Registry Pattern)                  | 493        |
| 5. 装饰器模式 (Decorator Pattern)                 | 496        |
| 6. 适配器模式 (Adapter Pattern)                   | 496        |
| 7. 建造者模式 (Builder Pattern)                   | 497        |
| 8. 原型模式 (Prototype Pattern)                  | 498        |
| 9. 模板方法模式 (Template Method Pattern)          | 499        |
| 二、代码组织原则                                     | 501        |
| 2.1 分层职责分离 (六层架构)                            | 501        |

|                                        |     |
|----------------------------------------|-----|
| 2.2 接口抽象 (Protocol/ABC 大量使用)           | 502 |
| 2.3 配置驱动开发 (28+ 配置模块控制行为)              | 502 |
| 2.4 错误处理层次                             | 503 |
| 三、架构优势与权衡                              | 503 |
| 3.1 性能 vs 灵活性的权衡                       | 503 |
| 3.2 抽象开销 vs 可维护性的平衡                    | 503 |
| 3.3 同步 vs 异步的选择                        | 503 |
| 四、可扩展性设计                               | 504 |
| 4.1 如何添加新模型                            | 504 |
| 4.2 如何添加新硬件后端                          | 504 |
| 4.3 如何添加新量化方案                          | 505 |
| 4.4 如何添加新注意力后端                         | 505 |
| 五、性能优化策略总结                             | 506 |
| 5.1 六大核心优化技术                           | 506 |
| 5.2 各优化技术的设计模式支撑                       | 507 |
| 5.3 性能优化的分层实现                          | 507 |
| 六、与竞品对比视角 (架构层面)                       | 508 |
| 6.1 vs HuggingFace Transformers        | 508 |
| 6.2 vs Text Generation Inference (TGI) | 508 |
| 6.3 vLLM v0 → v1 架构演进                  | 508 |
| 七、未来发展方向推断                             | 509 |
| 7.1 架构层面                               | 509 |
| 7.2 从代码结构推测的具体方向                       | 509 |
| 7.3 长期架构挑战                             | 510 |
| 附录: 设计模式快速参考                           | 510 |
| 总结                                     | 510 |

## vLLM 源码结构分析—总览导航

定位: 本文档是 vLLM 源码分析系列 (共 22 篇) 的总览导航入口与宏观概览, 旨在为读者建立对 vLLM 整体架构的全局认知地图。建议首次阅读时按顺序浏览本文档的全部章节, 之后可作为快速查阅索引使用。

### 全局知识图谱

#### mindmap

```
root((vLLM<br/> 源码全景))
  项目概述
    PagedAttention
    Continuous Batching
    高吞吐低延迟推理引擎
  六层架构
    服务层 entrypoints/
      OpenAI API 兼容
      Anthropic API
      CLI 入口
    引擎层 engine/ & v1/engine/
      LLMEngine (v1)
      AsyncLLMEngine
      EngineCore / EngineCoreProc
    调度核心 v1/core/sched/
      SchedulerInterface
      RequestQueue
```

```

    Chunked Prefill
    执行器层 v1/executor/
        UniprocExecutor
        MultiprocExecutor
        RayExecutor
    模型执行器 v1/worker/
        GPUModelRunner
        CPUModelRunner
        WorkerBase
    内核层 kernels/ & compilation/
        Attention Kernels
        Custom Ops
        torch.compile Passes
    关键技术
        KV Cache 管理
        Speculative Decoding
        LoRA 动态适配
        多模态支持
        分布式并行
    配置体系 config/
        VllmConfig 聚合
        20+ 子配置项
    Optimization Level 00-03

```

---

## 一、项目概述

### 1.1 基本信息

| 属性   | 值                                                                                                       |
|------|---------------------------------------------------------------------------------------------------------|
| 项目名称 | vLLM                                                                                                    |
| 定位   | 高吞吐、内存高效的大语言模型（LLM）推理与服务引擎                                                                              |
| 开源协议 | Apache-2.0                                                                                              |
| 核心口号 | <i>"a high-throughput and memory-efficient inference engine for LLMs"</i> (见 <code>__init__.py</code> ) |
| 版本管理 | 通过 <code>version.py</code> → <code>_version.py</code> 管理，开发版显示 <code>dev</code>                         |

### 1.2 核心价值主张

vLLM 的核心竞争力源于两项奠基性技术创新：

- **PagedAttention**: 将操作系统虚拟内存的分页（Paging）思想引入注意力计算中的 KV Cache 管理。将 KV Cache 按固定大小的 Block 组织，实现非连续内存分配，极大减少内存碎片和浪费。
- **Continuous Batching**（连续批处理）：突破传统静态批处理的限制，在一个迭代步（iteration step）内动态地混排处于不同阶段（prefill / decode）的请求，实现 GPU 利用率的最大化。

### 1.3 生态地位

vLLM 已成为 LLM 推理服务的事实标准之一：- 提供 **OpenAI** 兼容 **API** 服务端(`entrypoints/openai/api_server.py`)  
 - 支持 **Anthropic** 兼容协议(`entrypoints/anthropic/`) - 支持 **200+** 模型架构 (见 `model_executor/models/` 目录下的模型文件) - 支持 多模态（视觉、音频、视频）、**LoRA**、**Speculative Decoding**、前缀缓存、分布式并行（TP/PP/DP/EP）等企业级特性

## 二、整体架构图

vLLM 采用清晰的六层分层架构，自上而下依次为：

```
graph TB
    subgraph L6[" 第六层: 服务层 — entrypoints/"]
        direction TB
        A6_1["OpenAI API Server<br/>api_server.py"]
        A6_2["Anthropic API<br/>serving.py"]
        A6_3["CLI / serve.py<br/>launch.py"]
        A6_4["LLM 高级封装 <br/>entrypoints/llm.py :: LLM"]
    end

    subgraph L5[" 第五层: 引擎层 — engine/ & v1/engine/"]
        direction TB
        A5_1["AsyncLLMEngine<br/> 异步引擎封装"]
        A5_2["LLMEngine ~v1.engine.llm_engine.LLMEngine<br/> 引擎总控 (已迁移至 v1)"]
        A5_3["EngineCore / EngineCoreProc<br/> 引擎内核 + ZMQ 进程通信"]
        A5_4["EngineCoreClient<br/> 引擎内核客户端"]
        A5_5["InputProcessor / OutputProcessor<br/> 输入输出处理"]
        A5_6["Detokenizer<br/> 反 Tokenize"]
    end

    subgraph L4[" 第四层: 调度核心 — v1/core/sched/"]
        direction TB
        A4_1["SchedulerInterface<br/> 调度器抽象接口"]
        A4_2["Scheduler<br/> 核心调度器实现"]
        A4_3["AsyncScheduler<br/> 异步调度器"]
        A4_4["RequestQueue<br/> 请求队列管理"]
        A4_5["chunked_prefill<br/> 分块预填充"]
    end

    subgraph L3[" 第三层: 执行器层 — v1/executor/"]
        direction TB
        A3_1["Executor 抽象基类 <br/>abstract.py"]
        A3_2["UniprocExecutor<br/> 单进程执行器"]
        A3_3["MultiprocExecutor<br/> 多进程执行器"]
        A3_4["RayExecutor / RayExecutorV2<br/>Ray 分布式执行器"]
    end

    subgraph L2[" 第二层: 模型执行器 — v1/worker/"]
        direction TB
        A2_1["WorkerBase<br/>Worker 抽象基类"]
        A2_2["GPUWorker / GPUModelRunner<br/>GPU 模型运行器"]
        A2_3["CPUWorker / CPUModelRunner<br/>CPU 模型运行器"]
        A2_4["GPUInputBatch / UBatchWrapper<br/> 输入批次封装"]
        A2_5["LoRA Model Runner Mixin<br/>LoRA 适配混合"]
        A2_6["KV Connector Mixin<br/>KV Cache 连接器"]
    end

    subgraph L1[" 第一层: 内核层 — kernels/ & compilation/"]
        direction TB
        A1_1["Attention Backends<br/>FlashInfer / FlashAttn / XFormers"]
    end
```

```

A1_2["Custom Ops<br/>RMSNorm / SiLU / RotaryEmb / QuantFP8"]
A1_3["torch.compile Passes<br/>Inductor / Fusion / Graph Partition"]
A1_4["CUDA Graphs<br/>Piecewise & Full CUDAGraph"]
A1_5["Triton Kernels<br/> 自定义 Triton 算子"]
A1_6["csrc/<br/>C++/CUDA 扩展内核"]
end

L6 -->|"HTTP/gRPC 请求"| L5
L5 -->|"add_request()"| L4
L4 -->|"schedule() → SchedulerOutput"| L3
L3 -->|"execute_model()"| L2
L2 -->|"model forward()"| L1

style L6 fill:#e3f2fd,stroke:#1565c0,stroke-width:2px
style L5 fill:#fff3e0,stroke:#e65100,stroke-width:2px
style L4 fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px
style L3 fill:#fce4ec,stroke:#c62828,stroke-width:2px
style L2 fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px
style L1 fill:#ffffde,stroke:#f9a825,stroke-width:2px

```

各层职责速查

| 层级              | 核心目录                          | 核心职责                                  | 关键类/模块                                                |
|-----------------|-------------------------------|---------------------------------------|-------------------------------------------------------|
| <b>L6</b> 服务层   | entrypoints/                  | 对外提供 API 服务、CLI 入口、高级 SDK 封装          | LLM, ApiServer, OpenAIServing                         |
| <b>L5</b> 引擎层   | engine/, v1/engine/           | 引擎生命周期管理、请求编排、进程通信                    | LLMEngine, EngineCore, EngineCoreProc, AsyncLLMEngine |
| <b>L4</b> 调度核心  | v1/core/sched/                | 请求排队、策略调度、Chunked Prefill、KV Cache 分配 | Scheduler, SchedulerInterface, RequestQueue           |
| <b>L3</b> 执行器层  | v1/executor/                  | 多进程/多节点分布式执行策略                        | UniprocExecutor, MultiprocExecutor, RayExecutor       |
| <b>L2</b> 模型执行器 | v1/worker/                    | 模型加载、前向传播、采样、CUDA Graph 管理            | GPUModelRunner, WorkerBase, GPUWorker                 |
| <b>L1</b> 内核层   | kernels/, compilation/, csrc/ | 底层算子、注意力后端、编译优化、CUDA/C++ 扩展           | FlashInfer, CustomOps, InductorPasses                 |

### 三、版本演进路线: v0 → v1 迁移策略

#### 3.1 迁移背景

vLLM 经历了一次重大的架构重构——从 v0 架构迁移到 v1 架构。这次重构的核心目标是:

1. 解耦 **Engine Core**: 将引擎的核心调度循环 (**EngineCore**) 从对外接口中分离出来, 支持跨进程部署
2. 引入 **ZMQ** 进程通信: 通过 ZeroMQ 实现 EngineCore 与前端的高效 IPC
3. 支持 **Data Parallel (DP)**: 原生支持数据并行和弹性扩缩容 (Elastic EP)
4. 统一 **Async Scheduling**: 内置异步调度能力

### 3.2 向后兼容机制

vLLM 在迁移过程中采用了精巧的向后兼容策略，确保现有用户代码无需修改即可升级：

(1) **LLMEngine** 别名重定向 在 `engine/llm_engine.py` 中，原来的 `LLMEngine` 类已被替换为一个简单的别名重定向：

```
from vllm.v1.engine.llm_engine import LLMEngine as V1LLMEngine
LLMEngine = V1LLMEngine # type: ignore
"""The `LLMEngine` class is an alias of [vllm.v1.engine.llm_engine.LLMEngine]()."""
```

这意味着所有 `from vllm import LLMEngine` 的导入都会自动指向 v1 版本。

(2) `__getattr__` 延迟导入 在 `__init__.py` 中，使用了 Python 3.7+ 的 `__getattr__` 延迟导入机制：

```
def __getattr__(name: str) -> typing.Any:
    from importlib import import_module
    if name in MODULE_ATTRS:
        module_name, attr_name = MODULE_ATTRS[name].split(":")
        module = import_module(module_name, __package__)
        return getattr(module, attr_name)
    else:
        raise AttributeError(f"module {__package__} has no attribute {name}")
```

配合 `MODULE_ATTRS` 字典映射表（第 16-39 行），实现了：- 按需加载：只有当用户实际访问某个属性时才触发对应模块的导入 - 零成本兼容：不增加启动时的导入开销 - 统一入口：`from vllm import LLMEngine, SamplingParams, LLM` 等写法全部生效

(3) **v1** 引擎内部结构 v1 引擎的核心组件关系如下：

```
classDiagram
    class LLMEngine {
        +engine_core_client: EngineCoreClient
        +input_processor: InputProcessor
        +output_processor: OutputProcessor
        +add_request()
        +step()
        +abort_request()
    }

    class EngineCoreClient {
        +core_engine_type: type
        +add_request()
        +abort_requests()
        +step()
        +shutdown()
    }

    class EngineCore {
        +scheduler: SchedulerInterface
        +model_executor: Executor
        +structured_output_manager
        +add_request()
        +step() tuple~dict, bool~
        +abort_requests()
        +shutdown()
    }
```

```

}

class EngineCoreProc {
    +input_queue: Queue
    +output_queue: Queue
    +run_busy_loop()
    +process_input_sockets()
    +process_output_sockets()
}

class DPEngineCoreProc {
    +dp_group
    +step_counter
    +current_wave
    +run_busy_loop()
    +_has_global_unfinished_reqs()
}

LLMEngine --> EngineCoreClient : owns
EngineCoreClient ..> EngineCore : manages (in-proc)
EngineCoreClient ..> EngineCoreProc : manages (multi-proc)
EngineCore <|-- EngineCoreProc : extends with ZMQ
EngineCoreProc <|-- DPEngineCoreProc : extends for MoE DP

```

### 3.3 v0 vs v1 核心差异

| 维度                        | v0 架构                | v1 架构                                           |
|---------------------------|----------------------|-------------------------------------------------|
| 引擎位置                      | engine/llm_engine.py | v1/engine/llm_engine.py +<br>v1/engine/core.py  |
| 调度器位置                     | core/scheduler.py    | v1/core/sched/scheduler.py                      |
| 进程模型                      | 单进程为主                | 支持 Uniproc / Multiproc / Ray / DP               |
| 通信方式                      | 直接方法调用               | ZMQ IPC (msgpack 序列化)                           |
| 调度模式                      | 同步调度                 | 支持同步 + 异步调度 (Async Scheduling)                  |
| <b>KV Cache 管理</b>        | CacheEngine          | v1/core/kv_cache_manager.py +<br>Hybrid Manager |
| <b>Speculative Decode</b> | 内嵌于 Scheduler        | v1/spec_decode/ 独立模块化                           |
| <b>DP 支持</b>              | 不支持                  | 原生 Data Parallel + Elastic EP                   |

## 四、目录结构全景图

以下是 /workspace/vllm 的主要目录树及其职责说明:

```

vllm/                                # 项目根目录
__init__.py                          # 包入口: MODULE_ATTRS + __getattr__ 延迟导入
version.py / _version.py             # 版本号管理
envs.py                              # 环境变量集中定义
env_override.py                      # 环境变量覆盖机制

config/                              # 配置体系 (20+ 子配置聚合)
    vllm.py                          # VllmConfig: 全局配置聚合容器 (Pydantic model)
    model.py                         # ModelConfig: 模型名称、架构、dtype、量化

```



|                          |                                                      |
|--------------------------|------------------------------------------------------|
| cache.py                 | # CacheConfig: KV Cache 大小、block_size、prefix caching |
| parallel.py              | # ParallelConfig: TP/PP/DP/EP 并行配置                   |
| scheduler.py             | # SchedulerConfig: 调度策略、max_num_seqs/tokens          |
| device.py                | # DeviceConfig: 设备类型 (cuda/xpu/cpu)                  |
| load.py                  | # LoadConfig: 权重加载格式、下载路径                            |
| attention.py             | # AttentionConfig: 注意力后端选择                           |
| lora.py                  | # LoRAConfig: LoRA 适配器配置                             |
| speculative.py           | # SpeculativeConfig: 推测解码配置                          |
| compilation.py           | # CompilationConfig: torch.compile + CUDAGraph       |
| kernel.py                | # KernelConfig: 自定义算子优先级                             |
| quantization/            | # 各量化方案的配置 (AWQ/GPTQ/FP8 等)                          |
| ...                      | # reasoning/offload/profiler/observability 等         |
| engine/                  | # 引擎层 (v0 兼容入口)                                      |
| llm_engine.py            | # LLMEngine → v1 LLMEngine 的别名重定向                    |
| async_llm_engine.py      | # AsyncLLMEngine 异步引擎                                |
| arg_utils.py             | # EngineArgs / AsyncEngineArgs 参数解析                  |
| protocol.py              | # 引擎协议定义                                             |
| v1/                      | # V1 架构 (当前主力架构)                                     |
| engine/                  | # 引擎核心                                               |
| llm_engine.py            | # LLMEngine: 对外总控接口                                  |
| core.py                  | # EngineCore: 引擎内核 (调度+执行主循环)                        |
| core_client.py           | # EngineCoreClient: 内核客户端 (IPC 代理)                   |
| async_llm.py             | # AsyncLLMEngine: 异步引擎                               |
| coordinator.py           | # DPCoordinator: 数据并行协调器                             |
| input_processor.py       | # InputProcessor: 请求预处理                              |
| output_processor.py      | # OutputProcessor: 响应后处理                             |
| detokenizer.py           | # Detokenizer: 流式反 Tokenize                          |
| tensor_ipc.py            | # TensorIpcReceiver: 张量跨进程传输                         |
| utils.py                 | # 引擎工具类                                              |
| core/                    | # 调度核心                                               |
| sched/                   | # 调度器                                                |
| scheduler.py             | # Scheduler: FIFO/双调度的核心实现                           |
| interface.py             | # SchedulerInterface: 调度器抽象接口                        |
| async_scheduler.py       | # AsyncScheduler: 异步调度器                              |
| request_queue.py         | # RequestQueue: 请求排队与优先级                             |
| output.py                | # SchedulerOutput: 调度输出数据结构                          |
| kv_cache_manager.py      | # KV Cache 管理器                                       |
| kv_cache_coordinator.py  | # KV Cache 跨节点协调器                                    |
| block_pool.py            | # Block 内存池管理                                        |
| encoder_cache_manager.py | # 编码器缓存管理                                            |
| executor/                | # 执行器 (分布式策略)                                        |
| abstract.py              | # Executor: 执行器抽象基类                                  |
| uniproc_executor.py      | # UniprocExecutor: 单进程执行器                            |
| multiproc_executor.py    | # MultiprocExecutor: 多进程执行器                          |
| ray_executor.py          | # RayExecutor: Ray 部署执行器                             |
| ray_executor_v2.py       | # RayExecutorV2: Ray 执行器 v2                          |
| worker/                  | # 模型执行器 (Worker)                                     |
| worker_base.py           | # WorkerBase: Worker 抽象基类                            |
| gpu_worker.py            | # GPUWorker: GPU Worker                              |

```

cpu_worker.py          # CPUWorker: CPU Worker
gpu_model_runner.py    # GPUModelRunner: GPU 模型前向传播
cpu_model_runner.py    # CPUModelRunner: CPU 模型前向传播
gpu_input_batch.py     # GPUInputBatch: GPU 输入批次封装
gpu_ubatch_wrapper.py  # UBatchWrapper: Micro-batch 封装
ubatching.py          # Micro-batching 调度逻辑
block_table.py         # Block Table 管理
workspace.py          # 工作空间 (显存分配)
lora_model_runner_mixin.py # LoRA 模型适配
kv_connector_model_runner_mixin.py # KV 连接器适配
encoder_cudagraph.py   # 编码器 CUDA Graph

spec_decode/          # 推测解码
  eagle.py            # EAGLE 推测解码
  medusa.py           # MEDUSA 推测解码
  ngram_proposer.py   # N-Gram 草稿模型
  draft_model.py       # Draft Model 管理
  ...

structured_output/    # 结构化输出
  backend_outlines.py # Outlines 后端
  backend_lm_format_enforcer.py # LMFEnforcer 后端
  backend_xgrammar.py # XGrammar 后端
  backend_guidance.py # Guidance 后端

sample/               # 采样模块
  sampler.py           # 采样器 (softmax/top-k/top-p)
  rejection_sampler.py # 拒绝采样 (Speculative Decode 用)
  metadata.py          # 采样元数据

metrics/              # 可观测性指标
  stats.py             # 统计数据结构
  prometheus.py        # Prometheus 指标导出
  loggers.py           # 日志记录器
  perf.py              # 性能计数器

attention/            # 注意力后端选择
  backend.py           # 后端枚举与选择逻辑
  selector.py          # 选择器实现

kv_offload/           # KV Cache 卸载
  simple_kv_offload/   # 简单 KV 卸载实现

outputs.py            # ModelRunnerOutput 等输出数据结构
request.py            # Request / RequestStatus 数据结构
kv_cache_interface.py # KVCacheConfig 接口定义
cudagraph_dispatcher.py # CUDA Graph 分发器

model_executor/       # 模型执行框架
  models/              # 200+ 模型架构实现
    llama.py           # LLaMA 系列
    deepseek_v2.py     # DeepSeek-V2/V3 (MLA)
    qwen3.py           # Qwen3 系列
    mistral.py         # Mistral 系列

```

```

    gpt_neox.py          # GPT-NeoX
    interfaces.py        # 模型接口抽象 (ForCausalLM 等)
    interfaces_base.py   # 基础接口定义
    registry.py          # ModelRegistry: 模型注册表
    ...                 # 其余 170+ 模型文件
    custom_op.py         # 自定义算子注册与管理
    parameter.py         # 参数管理 (sharding 等)
    utils.py            # 模型工具函数
    warmup/             # 模型预热 (kernel warmup)

multimodal/            # 多模态支持
    registry.py         # MultiModalRegistry: 多模态注册中心
    image.py / audio.py / video.py # 各模态输入处理
    processing/         # 处理流水线
        processor.py    # 处理器基类
        inputs.py       # 输入数据结构
        context.py      # 处理上下文
    media/              # 媒体类型定义
    cache.py            # 多模态特征缓存
    hasher.py          # 输入哈希 (用于缓存 key)

lora/                  # LoRA 适配器系统
    lora_model.py       # LoRA 模型管理
    lora_weights.py     # LoRA 权重管理
    model_manager.py    # LoRA 模型管理器
    layers/             # LoRA 层实现
        base.py         # LoRA 层基类
        column_parallel_linear.py # 列并行线性层+LoRA
        row_parallel_linear.py   # 行并行线性层+LoRA
        fused_moe.py     # MoE + LoRA 融合
    punica_wrapper/     # Punica GPU 库封装 (高效 LoRA 计算)
    request.py          # LoRARequest 请求数据

distributed/           # 分布式通信
    parallel_state.py   # 并行状态 (TP/PP/DP group 管理)
    device_communicators/ # 设备通信后端
        base_device_communicator.py # 通信基类
        cuda_communicator.py        # CUDA 通信 (NCCL/PYNCLL)
        custom_all_reduce.py         # 自定义 AllReduce
        flashinfer_all_reduce.py     # FlashInfer AllReduce
        all2all.py                   # All2All 通信
    kv_transfer/         # KV Cache 跨节点传输
        kv_connector/      # KV 连接器工厂与实现
    elastic_ep/          # 弹性并行 (Elastic EP)
        elastic_state.py   # 弹性状态机
        elastic_execute.py # 弹性执行逻辑
    weight_transfer/     # 权重传输 (RLHF 训练场景)
    stateless_coordinator.py # 无状态协调器

entrypoints/           # 服务入口
    llm.py              # LLM 类 (面向用户的 Python SDK)
    api_server.py        # API Server 主入口
    openai/              # OpenAI 兼容服务
        api_server.py     # OpenAI API Server

```

```

    cli_args.py          # OpenAI CLI 参数
anthropic/              # Anthropic 兼容服务
cli/                    # 命令行入口
    main.py             # vllm 命令入口
    serve.py            # vllm serve 命令
pooling/                # Pooling 任务 (embedding/classification)
mcp/                    # MCP (Model Context Protocol) 支持

compilation/            # 编译优化体系
    compiler_interface.py # torch.compile 接口封装
    backends.py          # 编译后端选择
    cuda_graph.py        # CUDA Graph 捕获与管理
    piecewise_backend.py  # Piecewise CUDA Graph 后端
    base_static_graph.py  # 静态图基础
    decorators.py        # @support_torch_compile 装饰器
    counter.py           # 编译计数器
    caching.py           # 编译结果缓存
    codegen.py           # 代码生成
    wrapper.py           # 编译包装器
    passes/              # 编译优化 Pass
        pass_manager.py  # Pass 管理器
        inductor_pass.py # Inductor Pass
        vllm_inductor_pass.py # vLLM 自定义 Inductor Pass
        fx_utils.py      # FX 图工具

kernels/                # 自定义算子内核
    __init__.py / vllm_c.c # C++/CUDA 扩展入口
    aiter_ops.py         # AITer 算子 (ROCm 平台)
    xpu_ops.py           # Intel XPU 算子
    oink_ops.py          # Oink 算子
    triton/              # Triton 自定义内核
    helion/               # Helion 配置

tokenizers/             # Tokenizer 系统
    hf.py                # HuggingFace Tokenizer
    registry.py          # TokenizerRegistry
    fasttokens.py        # FastTokenizer
    ...                  # 特殊 tokenizer (DeepSeek/Grok/Qwen-VL 等)

renderers/              # 输入渲染器 (Prompt 构建)
    base.py              # 渲染器基类
    hf.py                # HuggingFace 渲染器
    registry.py          # 渲染器注册表
    deepseek_v32.py      # DeepSeek-V3/V32 特殊渲染
    ...

reasoning/              # 推理 (Reasoning) 解析
    abs_reasoning_parsers.py # 解析器抽象基类
    deepseek_r1_reasoning_parser.py # DeepSeek-R1 推理解析
    ...                  # 其他模型的推理解析器

tool_parsers/           # 工具调用 (Function Calling) 解析
    abstract_tool_parser.py # 工具解析器基类
    openai_tool_parser.py  # OpenAI 格式解析

```

|                     |                                        |
|---------------------|----------------------------------------|
| ...                 | # 各厂商工具调用解析                            |
| platforms/          | # 平台抽象层                                |
| interface.py        | # 平台接口定义                               |
| cuda.py             | # NVIDIA CUDA 平台                       |
| rocm.py             | # AMD ROCm 平台                          |
| xpu.py              | # Intel XPU 平台                         |
| cpu.py              | # CPU 平台                               |
| tpu.py              | # Google TPU 平台                        |
| transformers_utils/ | # Transformers 工具                      |
| config.py           | # HF Config 工具                         |
| tokenizer.py        | # Tokenizer 工具                         |
| configs/            | # 各模型 HF Config 映射                     |
| processors/         | # 图像处理器映射                              |
| inputs/             | # 输入处理                                 |
| __init__.py         | # PromptType, TextPrompt, TokensPrompt |
| engine.py           | # EngineInput 数据结构                     |
| llm.py              | # LLM 输入数据结构                           |
| preprocess.py       | # 预处理管道                                |
| outputs/            | # 输出数据结构                               |
| __init__.py         | # RequestOutput, CompletionOutput 等    |
| ...                 |                                        |
| utils/              | # 通用工具库                                |
| torch_utils.py      | # PyTorch 工具                           |
| mem_utils.py        | # 内存管理工具                               |
| hashing.py          | # 哈希工具 (用于 prefix caching)             |
| serial_utils.py     | # 序列化工具 (msgpack)                      |
| network_utils.py    | # 网络/ZMQ 工具                            |
| profiler.py         | # 性能分析工具                               |
| ...                 |                                        |
| benchmarks/         | # 性能基准测试                               |
| profiler/           | # 层级性能分析器                              |
| tracing/            | # OpenTelemetry 集成                     |
| logging_utils/      | # 日志工具                                 |
| plugins/            | # 插件系统                                 |
| device_allocator/   | # 设备内存分配器 (CuMem)                      |
| parser/             | # 输出解析器                                |
| usage/              | # 使用统计                                 |
| assets/             | # 静态资源                                 |
| third_party/        | # 第三方依赖                                |
| ir/                 | # 中间表示 (IR)                            |
| csrc/               | # C++/CUDA 源码                          |

---

## 五、快速导航索引

以下表格链接到本系列的其余 21 个分模块文档，按推荐阅读顺序排列：

| 序号 | 文档名            | 文件路径                        | 核心内容                                                                         | 建议阅读顺序 |
|----|----------------|-----------------------------|------------------------------------------------------------------------------|--------|
| 01 | 架构总览           | 01_architecture_overview.md | 六层架构 v0→v1 演进、核心数据结构、设计原则                                                    | 第 1 篇  |
| 02 | 配置系统           | 02_config_system.md         | VllmConfig 及 20+ 子配置、环境变量覆盖、OptimizationLevel 00-03                          | 第 2 篇  |
| 03 | 引擎核心           | 03_engine_core.md           | LLMEngine / AsyncLLM / EngineCore / InputProcessor / OutputProcessor         | 第 3 篇  |
| 04 | 调度器            | 04_scheduler.md             | Scheduler.schedule() 完整逻辑、Continuous Batching、Chunked Prefill、抢占机制           | 第 4 篇  |
| 05 | PagedAttention | 05_pagedattention.md        | 页式 KV Cache 管理、Block-Pool/BlockTable、Copy-on-Write、Prefix Caching            | 第 5 篇  |
| 06 | 注意力后端          | 06_attention_backend.md     | FlashAttention / FlashInfer / MLA / Triton / ROCm / CPU 后端选择与实现              | 第 6 篇  |
| 07 | 模型执行器          | 07_model_executor.md        | ModelRegistry (200+ 模型)、模型接口、主要模型家族、Transformer 层组件                          | 第 7 篇  |
| 08 | Worker 与执行器    | 08_worker_and_executor.md   | UniProc/MultiProc/Ray Executor、Worker-Base/GPUWorker/GPUModelRunner          | 第 8 篇  |
| 09 | KV 缓存系统        | 09_kv_cache.md              | KVCacheManager 层次、Of-fload/Transfer/Prefix Caching、Block Table 管理            | 第 9 篇  |
| 10 | 采样系统           | 10_sampling.md              | SamplingParams / Sampler 核心 / RejectionSampler / ParallelSampling / Logprobs | 第 10 篇 |
| 11 | 多模态处理          | 11_multimodal.md            | MultiModalRegistry、图像/音频/视频管线、CLIP/SigLIP/Whisper 编码器                        | 第 11 篇 |

| 序号 | 文档名     | 文件路径                               | 核心内容                                                                | 建议阅读顺序 |
|----|---------|------------------------------------|---------------------------------------------------------------------|--------|
| 12 | 量化支持    | 12_quantization.md                 | FP8 / INT4 / GPTQ / AWQ / GGUF / NVFP4 / Marlin / Machete 量化方案      | 第 12 篇 |
| 13 | 分布式计算   | 13_distributed.md                  | TP / PP / DP / EP 并行策略、通信原语、Elastic EP / EPLB                       | 第 13 篇 |
| 14 | LoRA 系统 | 14_lora.md                         | LoRA Layer 实现、Punica Wrapper、动态加载/卸载、Resolver 解析                    | 第 14 篇 |
| 15 | API 服务层 | 15_api_layer.md                    | OpenAI / Anthropic / gRPC API、Batch Serving、MCP Tool Server、Pooling | 第 15 篇 |
| 16 | CUDA 内核 | 16_cuda_kernels.md                 | 注意力/缓存/激活/量化/MoE/通信/采样内核、CPU/ROCm 平台内核                              | 第 16 篇 |
| 17 | 编译优化    | 17_compilation_and_optimization.md | CUDA Graphs (Piecewise+Full) / Inductor Passes / Triton Utils       | 第 17 篇 |
| 18 | 结构化输出   | 18_structured_output.md            | grammar / outlines / lm-format-enforcer / guidance 四种后端实现           | 第 18 篇 |
| 19 | 推测解码    | 19_speculative_decoding.md         | EA-GLE / Medusa / N-gram / DFlash / Suffix Decoding / Gemma4 草稿方法   | 第 19 篇 |
| 20 | 监控指标    | 20_metrics_and_monitoring.md       | Profiling 指标 / 日志系统 / OpenTelemetry / Profiling (Kineto/NVTX)       | 第 20 篇 |
| 21 | 设计模式    | 21_design_patterns.md              | 四种设计模式、可扩展性设计、性能优化策略、竞品对比与未来方向                                      | 第 21 篇 |

阅读路径建议：- 快速上手路径：01 → 02 → 07 → 15（了解如何从配置到 API 调用）- 深度研究路径：01 → 03 → 04 → 05 → 06 → 07 → 08 → 12（深入理解核心算法）- 工程实践路径：01 → 07 → 14 → 15 → 18 → 20（关注功能集成与运维）

## 六、关键技术特性一览表

| 特性名                                  | 核心位置                                                              | 原理一句话                                                | 影响一句话                                            |
|--------------------------------------|-------------------------------------------------------------------|------------------------------------------------------|--------------------------------------------------|
| <b>PagedAttention</b>                | v1/core/block_pool.py,<br>v1/core/kv_cache_manager.py             | 将 KV Cache 按固定大小 Block 分页存储, 类似 OS 虚拟内存管理            | 将 GPU 显存利用率从 ~20% 提升至 ~90%+, 支持更大并发量             |
| <b>Continuous Batching</b>           | v1/core/sched/scheduler.py                                        | 每个 iteration 动态混排 prefill 和 decode 请求, 消除 batch 边界等待 | 将 TTFT (首字延迟) 和 TPOT (每词延迟) 同时优化到极致              |
| <b>Chunked Prefill</b>               | v1/core/sched/scheduler.py                                        | 将 long prompt 拆分为多个 chunk 分批处理, 避免长请求阻塞短请求           | 保证短请求的延迟 SLA, 提升整体公平性和吞吐                         |
| <b>Prefix Caching</b>                | v1/core/kv_cache_manager.py                                       | 对相同前缀的请求复用已计算的 KV Cache Block                        | 对重复前缀场景 (如 system prompt) 实现接近零开销                |
| <b>CUDA Graphs</b>                   | compilation/cuda_graph.py,<br>v1/cudagraph_dispatcher.py          | 捕获 kernel 执行图为可重放对象, 消除 Python/CUDA launch 开销        | decode 阶段延迟降低 50%+, 高吞吐场景收益显著                    |
| <b>torch.compile</b>                 | compilation/compiler_interpreter.py,<br>compilation/passes/       | 通过 TorchDynamo + Inductor 对模型进行 JIT 编译和算子融合          | 减少 kernel launch 次数和内存 I/O, 融合 Norm/Act/Quant 算子 |
| <b>Speculative Decoding</b>          | v1/spec_decode/                                                   | 用小草案模型 (draft model) 生成候选 token, 大目标模型并行验证           | 在保持完全一致性的前提下, 加速 2-4 倍                           |
| <b>LoRA 动态适配</b>                     | lora/                                                             | 运行时动态注入/卸载 LoRA 适配器权重, 共享基础模型                        | 单实例服务数百个微调模型, 大幅降低部署成本                           |
| <b>Attention Backend 选择</b>          | v1/attention/backend.py<br>selector.py                            | 根据 GPU 架构和模型特性自动选择最优注意力后端                            | 自动适配 FlashInfer/FlashAttention/XFormers 等        |
| <b>异步调度 (Async Scheduling)</b>       | v1/core/sched/async_scheduler.py                                  | 将采样操作与模型前向传播解耦, 重叠执行                                 | 减少每个 iteration 的气泡, 提升 GPU 利用率                   |
| <b>Data Parallel (DP)</b>            | v1/engine/core.py<br>(DPEngineCoreProc),<br>distributed/          | 多副本间通过 all-reduce 协调请求波次 (wave), MoE 场景下路由专家         | 线性扩展吞吐量, 支持弹性伸缩容                                 |
| <b>Pipeline Parallelism (PP)</b>     | v1/worker/worker_base.py,<br>sequence.py<br>(IntermediateTensors) | 将模型层切分到多个 GPU, 阶段间通过 IntermediateTensors 传递          | 支持超大模型 (70B+) 在有限 GPU 上部署                        |
| <b>Tensor Parallelism (TP)</b>       | model_executor/<br>distributed/device_communicator.py             | 将单层矩阵运算切分到多卡, 通过 AllReduce 同步                        | 减少单卡显存需求, 几乎无损的线性加速                              |
| <b>KV Cache Offload</b>              | v1/kv_offload/<br>config/kv_transfer.py                           | 将不活跃序列的 KV Cache 卸载到 CPU 或远端内存                       | 支持超长上下文和超大并发, 降低 GPU 显存压力                        |
| <b>Multi-Modal Support</b>           | multimodal/                                                       | 统一的输入处理 pipeline, 将图像/音频/视频编码为特征向量注入模型               | 一个引擎同时服务文本和多模态任务                                 |
| <b>Structured Output</b>             | v1/structured_output/                                             | 通过 grammar bitmask 约束采样过程, 保证输出符合 JSON Schema        | 实现可靠的工具调用和结构化数据提取                                |
| <b>Elastic EP (弹性并行)</b>             | distributed/elastic_ep/                                           | 运行时动态增减 DP 副本数, 无状态协调器管理扩缩容                          | 根据负载自动调整资源, 云原生场景下降本增效                           |
| <b>Optimization Levels (-O0~-O3)</b> | config/vllm.py<br>(OPTIMIZATION_LEVEL_*)                          | 四级预设优化级别, 控制编译/CUDAGraph/fusion 的组合                  | 用户可在启动速度和推理性能间灵活权衡                               |



| 特性名                            | 核心位置                            | 原理一句话                                               | 影响一句话                                         |
|--------------------------------|---------------------------------|-----------------------------------------------------|-----------------------------------------------|
| <b>Hybrid KV Cache Manager</b> | v1/core/kv_cache_coordinator.py | 同时管理 full-attention 和 sliding-window/Mamba 类型 cache | 统一管理混合注意力模型（如 Jamba、Hybrid-Transformer-Mamba） |

## 七、数据流总览

以下 Mermaid 图展示了一个 HTTP 请求从接收到返回响应的完整数据流：

```

flowchart LR
    subgraph Client ["客户端"]
        R1["HTTP POST<br/>/v1/completions<br/> 或 /v1/chat/completions"]
    end

    subgraph L6_Entry ["服务层 entrypoints/openai/"]
        R2["OpenAIServing<br/> 路由分发"]
        R3["请求校验 &<br/> 参数解析"]
        R4["构建 RequestOutput<br/> 回调 Future"]
    end

    subgraph L5_Engine ["引擎层 v1/engine/"]
        R5["LLMEngine.add_request()<br/>→ EngineCoreClient.add_request()"]
        R6["InputProcessor.preprocess()<br/>Tokenize + Multimodal Encode"]
        R7["EngineCore.step()<br/> 调度循环"]
    end

    subgraph L4_Sched ["调度核心 v1/core/sched/"]
        R8["Scheduler.schedule()<br/>ChunkedPrefill 策略"]
        R9["Block 分配 &<br/>Copy-on-Write"]
        R10["构建 SchedulerOutput<br/>→ scheduled_chunks"]
    end

    subgraph L3_Exec ["执行器层 v1/executor/"]
        R11["Executor.execute_model()<br/> 序列化 & 发送"]
    end

    subgraph L2_Worker ["模型执行器 v1/worker/"]
        R12["GPUModelRunner.execute_model()<br/> 输入组装"]
        R13["模型 Forward<br/>Attention + FFN + ..."]
        R14["Sampling<br/>softmax → top-k/p → sample"]
        R15["返回 ModelRunnerOutput<br/>hidden_states + logits"]
    end

    subgraph L1_Kernel ["内核层 kernels/ + compilation/"]
        R16["FlashInfer Attention<br/>+ CustomOps<br/>+ CUDA Graph"]
    end

    subgraph ResponsePath ["响应回传路径"]
        R17["scheduler.update_from_output()<br/> 更新 KV Cache / 生成 token"]
        R18["OutputProcessor.process()<br/> 组装输出"]
        R19["Detokenizer.detokenize()<br/>token ID → 文本"]
        R20["流式 SSE / 非流式 JSON<br/> 返回给客户端"]
    end

```

```

end

Client -->|" HTTP Request"| L6_Entry
L6_Entry --> R2 --> R3 --> R4
R4 -->|" add_request()"| L5_Engine
R5 --> R6
R6 -->|" EngineCoreRequest (ZMQ/msgpack)"| R7
R7 -->|" schedule()"| L4_Sched
R8 --> R9 --> R10
R10 -->|" SchedulerOutput"| L3_Exec
R11 -->|" execute_model()"| L2_Worker
R12 --> R13 --> R14 --> R15
R13 -->|" kernel launch"| L1_Kernel
R15 -->|" ModelRunnerOutput"| L4_Sched
L4_Sched -->|" EngineCoreOutputs"| L5_Engine
L5_Engine --> ResponsePath
R17 --> R18 --> R19 --> R20
R20 -->|" HTTP Response"| Client

style Client fill:#e3f2fd,stroke:#1565c0
style L6_Entry fill:#fff3e0,stroke:#e65100
style L5_Engine fill:#fce4ec,stroke:#c62828
style L4_Sched fill:#e8f5e9,stroke:#2e7d32
style L3_Exec fill:#f3e5f5,stroke:#7b1fa2
style L2_Worker fill:#fffde7,stroke:#f9a825
style L1_Kernel fill:#ffebee,stroke:#d32f2f
style ResponsePath fill:#e0f2f1,stroke:#00796b

```

关键数据结构流转

```

用户输入 (prompt/text)
  ↓ InputProcessor.preprocess()
EngineCoreRequest (request_id, prompt_token_ids, mm_features, sampling_params, ...)
  ↓ EngineCore.add_request()
Request (internal, with block_hashes, grammar_state, ...)
  ↓ Scheduler.schedule()
SchedulerOutput (num_scheduled_tokens, scheduled_chunks, ...)
  ↓ Executor.execute_model() → GPUModelRunner.execute_model()
ModelRunnerOutput (logits, hidden_states, sampled_token_ids, ...)
  ↓ Scheduler.update_from_output()
EngineCoreOutputs (outputs per request, finished_requests, ...)
  ↓ OutputProcessor.process()
CompletionOutput / RequestOutput (text, token_ids, logprobs, finish_reason, ...)
  ↓ Detokenizer + SSE/JSON encode
HTTP Response (流式/非流式)

```

## 八、设计哲学总结

### 8.1 配置驱动 (Configuration-Driven)

核心理念：所有行为通过配置对象驱动，避免硬编码散落各处。

vLLM 的 `VllmConfig` 是一个 Pydantic model，聚合了 **20+** 子配置项 (`ModelConfig`, `CacheConfig`, `ParallelConfig`, `SchedulerConfig`, `CompilationConfig` 等)。每个子配置负责一个维度的决策：

- 为什么这样做：配置集中使得：
  - 新增特性只需新增配置字段，无需修改核心逻辑
  - 配置间的交叉验证集中在 `__post_init__` 中（`VllmConfig.__post_init__` 有 500+ 行验证逻辑）
  - `compute_hash()` 方法基于配置自动计算唯一哈希，用于编译缓存失效判定
  - 四级优化级别（O0-O3）通过 `OPTIMIZATION_LEVEL_TO_CONFIG` 字典一键切换全部编译选项

## 8.2 注册表模式 (Registry Pattern)

核心理念：通过全局注册表实现可扩展的组件发现与绑定。

vLLM 中广泛使用的注册表包括：

| 注册表                             | 位置                                             | 用途                     |
|---------------------------------|------------------------------------------------|------------------------|
| <code>ModelRegistry</code>      | <code>model_executor/models/registry.py</code> | 模型架构 → 实现类的映射          |
| <code>MultiModalRegistry</code> | <code>multimodal/registry.py</code>            | 模态类型 → 处理器的映射          |
| <code>TokenizerRegistry</code>  | <code>tokenizers/registry.py</code>            | 模型 → Tokenizer 实现的映射   |
| <code>RendererRegistry</code>   | <code>renderers/registry.py</code>             | 模型 → Prompt 渲染器的映射     |
| <code>StatLoggerFactory</code>  | <code>v1/metrics/loggers.py</code>             | 日志类型 → Logger 实现的映射    |
| Scheduler 选择                    | <code>config/scheduler.py</code>               | 配置 → 调度器实现类的映射         |
| Executor 选择                     | <code>v1/executor/abstract.py</code>           | backend 名称 → 执行器实现类的映射 |

为什么这样做：第三方开发者可以通过装饰器 `@register_xxx("name")` 注册新组件，零修改核心代码即可扩展 vLLM 支持新的模型架构/模态/平台。

## 8.3 策略模式 (Strategy Pattern)

核心理念：将算法族封装为可互换的策略对象。

典型应用：

- **Attention Backend**: `v1/attention/backend.py` 定义了多种注意力后端（FlashInfer, FlashAttention, RoCmFlashAttn, XFormers 等），根据设备能力和模型需求在运行时选择。
- **Executor Backend**: `v1/executor/` 中 `UniprocExecutor` / `MultiprocExecutor` / `RayExecutor` 互为策略变体，通过 `Executor.get_class(vllm_config)` 工厂方法选择。
- **KV Cache Manager**: 不同类型的模型（full-attn / sliding-window / Mamba / hybrid）使用不同的 KV Cache 管理策略。
- **Speculative Decode**: `v1/spec_decode/` 下 EAGLE / Medusa / N-Gram 等不同推测策略共享统一的验证接口。

为什么这样做：策略可以在运行时切换（甚至通过配置热更新），便于 A/B 测试和渐进式优化。

## 8.4 关注点分离 (Separation of Concerns)

核心理念：EngineCore（调度 + 执行循环）与 EngineCoreClient（IPC 代理）的解耦。

v1 架构最显著的设计决策是将 **EngineCore** 作为纯计算内核：- 它不知道自己运行在哪个进程中（同进程 / 子进程 / Ray Actor）- 它只通过 **SchedulerInterface** 和 **Executor** 抽象交互 - 进程通信（ZMQ socket / Ray call）由 **EngineCoreProc** 包装层处理

为什么这样做：- 同一套调度逻辑可以复用在单机、多机、Ray 集群等多种部署模式 - 单元测试可以直接构造 **EngineCore** 进行测试，无需模拟网络 - 未来可以替换 IPC 机制（如改为 gRPC）而不影响核心逻辑

## 8.5 异步优先 (Async-First)

核心理念：默认启用异步调度，最大化 GPU 利用率。

v1 引入了 **Async Scheduling** (`config/scheduler.py` 中的 `async_scheduling` 配置项), 其核心思想是: - 将采样 (sampling) 操作从模型前向传播的关键路径中解耦 - 在模型执行当前 batch 时, 调度器同时准备下一个 batch - 通过 **Future** 对象协调两个并发的操作

为什么这样做: 传统同步调度在每个 iteration 中存在「调度 → 执行 → 采样 → 更新」的串行瓶颈, 异步调度将这些步骤部分重叠, 减少了 GPU 空闲时间。

8.6 渐进式复杂度 (Progressive Complexity)

核心理念: 提供从简单到复杂的多种优化级别。

vLLM 的 -O 优化级别 (`config/vllm.py`) 体现了这一哲学:

| 级别  | 编译              | CUDAGraph      | Fusion                     | 适用场景           |
|-----|-----------------|----------------|----------------------------|----------------|
| -O0 | 关闭              | NONE           | 无                          | 最快启动、调试        |
| -O1 | Dynamo+Inductor | Piecewise      | Norm/Act                   | 快速启动 + 基础加速    |
| -O2 | 全量编译            | Full+Piecewise | 全部融合 (含 SP/AllReduce)      | 默认选项、平衡性能      |
| -O3 | 全量编译            | Full+Piecewise | 全部融合 + FlashInfer autotune | 最大性能、可接受较长启动时间 |

为什么这样做: 用户可以根据自己的延迟/启动时间偏好灵活选择, 而不需要理解底层每一个优化开关。

8.7 平台抽象 (Platform Abstraction)

核心理念: 通过 Platform 接口屏蔽硬件差异。

`platforms/` 目录定义了统一的平台接口 (`current_platform`), 各平台 (CUDA / ROCm / XPU / TPU / CPU) 实现各自的: - 设备能力查询 (`get_device_capability()`) - 注意力后端选择 - 自定义算子注册 - 内存分配策略 - 编译配置调整 (`apply_config_platform_defaults()`)

为什么这样做: 确保 vLLM 的核心算法代码与硬件无关, 新增硬件支持只需实现 Platform 接口。

附录: 关键文件速查

| 文件                                             | 角色     | 一句话说明                                                                                              |
|------------------------------------------------|--------|----------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>                       | 包入口    | <code>MODULE_ATTRS</code> + <code>__getattr__</code> 延迟导入, 向后兼容的枢纽                                 |
| <code>engine/llm_engine.py</code>              | 兼容层    | <code>LLMEngine = V1LLMEngine</code> 别名重定向, v0→v1 迁移的见证                                            |
| <code>v1/engine/core.py</code>                 | 引擎内核   | <code>EngineCore</code> / <code>EngineCoreProc</code> / <code>DPEngineCoreProc</code> , 调度 + 执行主循环 |
| <code>v1/engine/llm_engine.py</code>           | 引擎总控   | <code>LLMEngine</code> v1 版本, 管理 <code>EngineCoreClient</code> + <code>IO Processor</code>         |
| <code>config/vllm.py</code>                    | 配置中枢   | <code>VllmConfig</code> 聚合 20+ 子配置, 500+ 行验证逻辑                                                     |
| <code>v1/core/sched/scheduler.py</code>        | 调度器    | <code>Chunked Prefill</code> / 双调度 / 请求排队核心实现                                                      |
| <code>v1/worker/gpu_model_runner.py</code>     | 模型运行器  | 模型前向传播、CUDA Graph 执行、采样触发                                                                          |
| <code>v1/executor/abstract.py</code>           | 执行器基类  | <code>Executor</code> 抽象接口, 定义 <code>execute_model/shutdown</code> 等契约                             |
| <code>model_executor/models/register.py</code> | 模型注册表  | <code>@ModelRouter.register("name")</code> 装饰器的定义                                                  |
| <code>entrypoints/llm.py</code>                | 高级 SDK | <code>LLM</code> 类, 面向最终用户的 Python API                                                             |

| 文件                        | 角色         | 一句话说明                                            |
|---------------------------|------------|--------------------------------------------------|
| sequence.py               | 数据结构       | IntermediateTensors 用于 Pipeline Parallel 阶段间数据传递 |
| compilation/cuda_graph.py | CUDA Graph | CUDA Graph 捕获、缓存、分派的核心逻辑                         |

## vLLM 架构总览

定位：本文档从架构层面深度分析 vLLM 源码，建立全局认知。涵盖六层分层架构、v0→v1 演进、核心数据结构与设计原则。

### 总体架构图

```
graph TB
    subgraph L6 ["第 6 层：内核层 csrc/"]
        CUDA["CUDA Kernels<br/>cutlass/flashinfer/triton"]
        CustomOps["Custom Ops<br/>vllm._custom_ops"]
    end

    subgraph L5 ["第 5 层：模型执行器 model_executor/"]
        ModelReg["Model Registry<br/>150+ 模型架构"]
        ModelRunner["ModelRunner<br/>GPU/CPU/XPU/TPU"]
        QuantBackend["量化后端 <br/>FP8/GPTQ/AWQ/NVFP4"]
    end

    subgraph L4 ["第 4 层：执行器层 v1/executor/"]
        UniProc["UniProcExecutor<br/>单进程"]
        MultiProc["MultiprocExecutor<br/>多进程"]
        RayExec["RayExecutor<br/>Ray 分布式"]
        WorkerBase["WorkerBase<br/>Worker 抽象"]
    end

    subgraph L3 ["第 3 层：调度核心 v1/core/sched/"]
        Scheduler["Scheduler<br/>调度算法"]
        AsyncSched["AsyncScheduler<br/>异步调度"]
        ReqQueue["RequestQueue<br/>请求队列"]
        BlockPool["BlockPool<br/>KV Cache 块池"]
    end

    subgraph L2 ["第 2 层：引擎层 engine/ + v1/engine/"]
        EngineCore["EngineCore<br/>核心引擎循环"]
        LLMEngine["LLMEngine<br/>兼容包装"]
        InputProc["InputProcessor<br/>输入处理"]
        OutputProc["OutputProcessor<br/>输出处理"]
        Detokenizer["Detokenizer<br/>解码器"]
    end

    subgraph L1 ["第 1 层：服务层 entrypoints/"]
        OpenAIAPI["OpenAI API Server"]
        AnthropicAPI["Anthropic API"]
        ChatCompletion["Chat Completion"]
        Embedding["Embedding/Pooling"]
        BatchServing["Batch Serving"]
    end
```

```

end

L1 -->|"HTTP/gRPC"| L2
L2 -->|"EngineCoreRequest"| L3
L3 -->|"SchedulerOutput"| L4
L4 -->|"collective_rpc"| L5
L5 -->|"CUDA Kernel"| L6

style L6 fill:#fff3e0,stroke:#e65100,color:#000
style L5 fill:#e8f5e9,stroke:#2e7d32,color:#000
style L4 fill:#fce4ec,stroke:#c62828,color:#000
style L3 fill:#f3e5f5,stroke:#7b1fa2,color:#000
style L2 fill:#e1f5fe,stroke:#01579b,color:#000
style L1 fill:#e0f2f1,stroke:#00695c,color:#000

```

## 一、分层架构详解

### 1.1 第 1 层：服务层（entrypoints/）

职责：对外提供 API 接口，负责协议转换、请求路由与负载均衡。

| 子模块            | 职责                                           | 关键文件                      |
|----------------|----------------------------------------------|---------------------------|
| openai/        | OpenAI 兼容 API<br>(chat/completion/embedding) | api_server.py, serving.py |
| anthropic/     | Anthropic Messages API<br>兼容                 | api_router.py             |
| pooling/       | Embedding /<br>Classification / Scoring      | embed/io_processor.py     |
| cli/           | CLI 入口<br>(serve/benchmark)                  | serve.py                  |
| grpc_server.py | gRPC 服务入口                                    | grpc_server.py            |

接口定义：服务层通过 llm.py 将 HTTP 请求转化为对 LLMEngine 的调用，使用 protocol.py 定义数据契约。

### 1.2 第 2 层：引擎层（engine/ + v1/engine/）

职责：编排输入预处理、调度执行、输出后处理的完整流水线。

核心组件

- **LLMEngine** (v1): 面向用户的引擎接口，负责：
  - 输入转换 (InputProcessor): EngineInput → EngineCoreRequest
  - 输出转换 (OutputProcessor): EngineCoreOutputs → RequestOutput
  - 统计日志 (StatLoggerManager)
  - LoRA 管理
- **EngineCore**: 解耦后的核心引擎，包含：
  - 模型执行器管理 (model\_executor)
  - 调度器管理 (scheduler)
  - KV Cache 初始化与管理
  - 核心步进循环 (step() / step\_with\_batch\_queue())
  - 多模态缓存管理 (mm\_receiver\_cache)
  - 结构化输出管理 (structured\_output\_manager)

- **EngineCoreProc**: 基于 ZMQ 的进程内通信封装, 支持:
  - 后台进程运行 EngineCore
  - Socket IO 线程 (input/output)
  - 握手协议 (handshake)
  - Data Parallel 协调 (DPEngineCoreProc)
  - Elastic EP 扩缩容

层间数据流

```

flowchart LR
    subgraph Frontend[" 前端 (服务层)"]
        HTTPReq["HTTP Request"]
    end

    subgraph Engine[" 引擎层"]
        EI["EngineInput<br/>(TokensInput/EmbedsInput)"]
        ECR["EngineCoreRequest<br/>(msgspec Struct)"]
        ECO["EngineCoreOutputs<br/>(msgpack 序列化)"]
        RO["RequestOutput<br/>(用户可见)"]
    end

    subgraph Core["EngineCore"]
        Req["Request<br/>(内部表示)"]
        SO["SchedulerOutput"]
        MRO["ModelRunnerOutput"]
    end

    HTTPReq -->|"render"| EI
    EI -->|"InputProcessor.process_inputs()"| ECR
    ECR -->|"add_request()"| Req
    Req -->|"schedule()"| SO
    SO -->|"execute_model()"| MRO
    MRO -->|"update_from_output()"| ECO
    ECO -->|"OutputProcessor"| RO

    style Frontend fill:#e0f2f1,stroke:#00695c,color:#000
    style Engine fill:#e1f5fe,stroke:#01579b,color:#000
    style Core fill:#f3e5f5,stroke:#7b1fa2,color:#000
  
```

### 1.3 第 3 层: 调度核心 (v1/core/sched/)

职责: 决定每个 step 中各请求的 token 数量, 管理 KV Cache 分配。

| 组件                        | 文件                 | 职责                                                        |
|---------------------------|--------------------|-----------------------------------------------------------|
| <b>SchedulerInterface</b> | interface.py       | 调度器抽象基类, 定义 schedule/add_request/update_from_output 等核心方法 |
| <b>Scheduler</b>          | scheduler.py       | 默认调度器实现, FCFS + 连续 batching                               |
| <b>AsyncScheduler</b>     | async_scheduler.py | 异步调度实现, 支持 scheduling 与 execution 重叠                      |
| <b>RequestQueue</b>       | request_queue.py   | 优先级队列, 支持 priority/arrival_time 排序                        |
| <b>BlockPool</b>          | block_pool.py      | KV Cache 物理块分配器                                           |

| 组件                    | 文件                  | 职责                              |
|-----------------------|---------------------|---------------------------------|
| <b>KVCacheManager</b> | kv_cache_manager.py | KV Cache 逻辑管理, 含 prefix caching |

关键调度决策流程:

```
# 来自 interface.py L52-L75
def schedule(self) -> "SchedulerOutput":
    """Schedule the requests to process in this scheduling step.

    The scheduler produces a dictionary of {req_id: num_tokens}
    that specifies how many tokens to process for each request.
    num_tokens can be:
    - prompt token count for new requests (prefill)
    - 1 for auto-regressive decoding
    - somewhere between for chunked prefills / speculative decoding
    """
```

#### 1.4 第 4 层: 执行器层 (v1/executor/)

职责: 管理分布式 worker 进程, 屏蔽单机/多机/Ray 差异。

##### Executor 类层次

```
classDiagram
class Executor {
    <<abstract>>
    +get_class(vllm_config) Executor$
    +execute_model(scheduler_output) ModelRunnerOutput
    +collective_rpc(method, args, kwargs) list
    +initialize_from_config(kv_cache_configs)
    +determine_available_memory() list~int~
    +shutdown()
}

class UniProcExecutor {
    +_init_executor()
    +collective_rpc()
}

class MultiprocExecutor {
    +_init_executor()
    +collective_rpc()
}

class RayDistributedExecutor {
    +_init_executor()
    +collective_rpc()
}

class RayExecutorV2 {
    +_init_executor()
    +collective_rpc()
}
```



```

class ExecutorWithExternalLauncher {
    +_init_executor()
}

Executor <|-- UniProcExecutor
Executor <|-- MultiprocExecutor
Executor <|-- RayDistributedExecutor
Executor <|-- RayExecutorV2
Executor <|-- ExecutorWithExternalLauncher

```

工厂方法——`Executor.get_class()` 根据 `distributed_executor_backend` 配置选择具体实现:

```

# abstract.py L48-L92
@staticmethod
def get_class(vllm_config: VllmConfig) -> type["Executor"]:
    distributed_executor_backend = parallel_config.distributed_executor_backend
    if isinstance(distributed_executor_backend, type):
        executor_class = distributed_executor_backend # 用户自定义
    elif distributed_executor_backend == "ray":
        executor_class = RayExecutorV2 or RayDistributedExecutor
    elif distributed_executor_backend == "mp":
        executor_class = MultiprocExecutor
    elif distributed_executor_backend == "uni":
        executor_class = UniProcExecutor
    elif distributed_executor_backend == "external_launcher":
        executor_class = ExecutorWithExternalLauncher
    return executor_class

```

### 1.5 第 5 层: 模型执行器 (`model_executor/`)

职责: 加载模型权重、构建计算图、执行前向传播。

| 子模块                               | 职责                                                                               |
|-----------------------------------|----------------------------------------------------------------------------------|
| <code>models/</code>              | 150+ 模型架构实现 (Llama/Qwen/Gemma/Mistral 等), 通过 <code>registry.py</code> 注册         |
| <code>models/interfaces.py</code> | 模型能力接口定义 ( <code>supports_multimodal / supports_pp / is_attention_free</code> 等) |
| <code>kernels/</code>             | 自定义 CUDA/Triton kernel 封装                                                        |
| <code>layers/</code>              | 通用算子层 (Linear / Attention / RMSNorm 等)                                           |
| <code>warmup/</code>              | 模型 warmup 逻辑                                                                     |

**Worker** 类型:

| Worker 类                    | 用途           | 文件位置                             |
|-----------------------------|--------------|----------------------------------|
| <code>GPUModelRunner</code> | GPU 上模型执行主逻辑 | <code>gpu/model_runner.py</code> |
| <code>CPUModelRunner</code> | CPU 推理       | <code>cpu_model_runner.py</code> |
| <code>XPUModelRunner</code> | Intel XPU    | <code>xpu_model_runner.py</code> |
| <code>TPUModelRunner</code> | Google TPU   | <code>tpu_input_batch.py</code>  |

1.6 第 6 层：内核层（csrc/ + kernels/）

职责：高性能 GPU kernel 实现。

| 组件             | 技术         | 用途                                            |
|----------------|------------|-----------------------------------------------|
| FlashAttention | flash-attn | 高效注意力计算                                       |
| FlashInfer     | flashinfer | Paged Attention / 解码优化                        |
| Cutlass MLA    | CUTLASS    | DeepSeek MLA 注意力                              |
| Triton Kernels | Triton     | 自定义融合 kernel（attention / rms_norm / silu_mul） |
| Custom Ops     | C++/CUDA   | FP8 量化、AllReduce 融合等                          |

二、v0 → v1 架构演进

2.1 演进总览

```
graph TD
    subgraph V0 ["v0 架构 (已废弃)"]
        V0Engine["LLMEngine<br/> 单体类"]
        V0Scheduler["Scheduler<br/> 同步调度"]
        V0Worker["Worker<br/> 直接 RPC"]
    end

    subgraph V1 ["v1 架构 (当前)"]
        V1Public["LLMEngine<br/> 兼容门面 (llm_engine.py)"]
        V1CoreClient["EngineCoreClient<br/>IPC 抽象"]
        V1Core["EngineCore<br/> 解耦核心 (core.py)"]
        V1CoreProc["EngineCoreProc<br/>ZMQ 进程封装"]
        V1Scheduler["SchedulerInterface<br/> 可插拔调度"]
        V1Executor["Executor<br/> 分布式抽象"]
    end

    V0Engine -->|"重构"| V1Public
    V0Scheduler -->|"抽象为 Interface"| V1Scheduler
    V0Worker -->|"统一为 Executor"| V1Executor

    V1Public --> V1CoreClient
    V1CoreClient -->|"inproc / zmq"| V1Core
    V1Core --> V1Scheduler
    V1Core --> V1Executor

    V1Core -. "继承" .-> V1CoreProc

    style V0 fill:#ffebee,stroke:#c62828,color:#000
    style V1 fill:#e8f5e9,stroke:#2e7d32,color:#000
```

2.2 v0 LLMEngine → v1 包装策略

关键发现：当前 vllm/engine/llm\_engine.py 仅是一个 重导出别名：

```
# vllm/engine/llm_engine.py (全文, 仅 6 行)
from vllm.v1.engine.llm_engine import LLMEngine as V1LLMEngine
```

```
LLMEngine = V1LLMEngine # type: ignore
"""The `LLMEngine` class is an alias of vllm.v1.engine.llm_engine.LLMEngine."""
```

这意味着 **v0** 版本已被完全移除，当前代码库中不存在旧的 **LLMEngine** 实现。所有调用方使用的 **LLMEngine** 实际上都是 **v1** 版本。

## 2.3 EngineCore 解耦设计

EngineCore 是 v1 架构的核心创新，实现了以下解耦：

初始化流程

```
sequenceDiagram
    participant LLM as LLMEngine
    participant ECC as EngineCoreClient
    participant EC as EngineCore
    participant EX as Executor
    participant SCH as Scheduler

    LLM->>ECC: make_client(multiprocess_mode)
    ECC->>EC: __init__(vllm_config, executor_class)
    EC->>EX: executor_class(vllm_config)
    EC->>EX: determine_available_memory()
    EC->>EX: get_kv_cache_specs()
    EC->>EC: _initialize_kv_caches()
    EC->>SCH: Scheduler(vllm_config, kv_cache_config)
    EC->>EX: initialize_from_config(kv_cache_configs)
    Note over EC: EngineCore 就绪
```

核心步进循环 `EngineCore.step()` 是引擎的心跳：

```
# core.py L402-L431
def step(self) -> tuple[dict[int, EngineCoreOutputs], bool]:
    """Schedule, execute, and make output.

    Returns tuple of outputs and a flag indicating whether the model
was executed.
    """

    # 1. 检查是否有待处理请求
    if not self.scheduler.has_requests():
        return {}, False

    # 2. 调度决策：决定每个请求处理多少 token
    scheduler_output = self.scheduler.schedule()

    # 3. 异步执行模型前向传播
    future = self.model_executor.execute_model(scheduler_output, non_block=True)

    # 4. 获取结构化输出的 grammar bitmask
    grammar_output = self.scheduler.get_grammar_bitmask(scheduler_output)

    # 5. 等待模型输出 & 采样
    with self.log_error_detail(scheduler_output), \
        self.log_iteration_details(scheduler_output):
        model_output = future.result()
```

```

        if model_output is None:
            model_output = self.model_executor.sample_tokens(grammar_output)

# 6. 处理异步 abort
self._process_aborts_queue()

# 7. 更新调度器状态并生成输出
engine_core_outputs = self.scheduler.update_from_output(
    scheduler_output, model_output
)

return engine_core_outputs, scheduler_output.total_num_scheduled_tokens > 0

```

**Pipeline Parallelism 优化: batch\_queue** 当 `max_concurrent_batches > 1` 时 (即 `PP > 1`), `EngineCore` 使用 `step_with_batch_queue()` 替代 `step()`:

- 使用双端队列 `deque` 缓冲多个 batch
- 优先填满 **batch queue** (消除 pipeline bubble)
- 阻塞等待最早完成的 batch 结果
- 支持 deferred sampling (结构化输出 + speculative decoding 场景)

## 2.4 v1 LLMEngine: 向后兼容的门面

v1 LLMEngine 作为公共 API 门面, 职责包括:

组合模式

```

# llm_engine.py L90-L111 (精简)
class LLMEngine:
    def __init__(self, vllm_config, executor_class, log_stats, ...):
        # 1. Renderer: 处理 chat template / multimodal inputs
        self.renderer = renderer_from_config(self.vllm_config)

        # 2. InputProcessor: EngineInput → EngineCoreRequest
        self.input_processor = InputProcessor(self.vllm_config, renderer)

        # 3. OutputProcessor: EngineCoreOutputs → RequestOutput
        self.output_processor = OutputProcessor(
            renderer.tokenizer,
            log_stats=self.log_stats,
            stream_interval=...,
        )

        # 4. EngineCoreClient: 通过 IPC 访问 EngineCore
        self.engine_core = EngineCoreClient.make_client(
            multiprocess_mode=multiprocess_mode,
            asyncio_mode=False,
            ...
        )

```

**add\_request 流水线**

```

# llm_engine.py L209-L285 (精简)
def add_request(self, request_id, prompt, params, ...):
    # 1. 输入预处理: EngineInput → EngineCoreRequest

```

```

request = self.input_processor.process_inputs(
    request_id, prompt, params, ...
)

# 2. n>1 时 fan-out 子请求 (beam search)
if n > 1:
    for idx in range(n):
        child_request = copy(request)
        child_request.sampling_params = child_params
        self.output_processor.add_request(child_request, ...)
        self.engine_core.add_request(child_request)
else:
    self.output_processor.add_request(request, ...)
    self.engine_core.add_request(request)

```

step 流水线

```

# llm_engine.py L287-L325 (精简)
def step(self) -> list[RequestOutput | PoolingRequestOutput]:
    # 1. 从 EngineCore 获取原始输出
    outputs = self.engine_core.get_output()

    # 2. 后处理: EngineCoreOutputs -> RequestOutput
    processed_outputs = self.output_processor.process_outputs(
        outputs.outputs, engine_core_timestamp=outputs.timestamp, ...
    )

    # 3. 中止 stop string 触发的请求
    self.engine_core.abort_requests(processed_outputs.reqs_to_abort)

    # 4. 记录统计信息
    if self.logger_manager is not None:
        self.logger_manager.record(...)

    return processed_outputs.request_outputs

```

## 2.5 v1 核心改进点总结

| 改进维度  | v0                       | v1                                    |
|-------|--------------------------|---------------------------------------|
| 核心循环  | 单体<br>LLMEngine.step()   | EngineCore.step() 解耦                  |
| 进程模型  | 同步多进程                    | 可选 in-proc/multiproc/ZMQ              |
| 调度器   | 硬编码 Scheduler            | SchedulerInterface 可插拔                |
| 执行器   | RayWorkerWrapper 固定      | Executor 抽象 + 工厂方法                    |
| 序列表示  | Sequence / SequenceGroup | 简化为 Request (无 group 概念)              |
| 数据序列化 | pickle                   | msgspec Struct (零拷贝友好)                |
| 异步调度  | 不支持                      | AsyncScheduler (scheduling execution) |

| 改进维度          | v0  | v1                                |
|---------------|-----|-----------------------------------|
| Data Parallel | 不支持 | DPEngineCoreProc + DP Coordinator |
| Elastic EP    | 不支持 | 完整扩缩容支持                           |

## 2.6 向后兼容策略

虽然 v0 已被移除，但 vLLM 通过以下机制保持 API 兼容性：

1. 模块级别名: `vllm.engine.llm_engine.LLMEngine` 直接指向 v1 实现
2. 属性透传: `self.model_executor = self.engine_core.engine_core.model_executor` (v0 兼容访问路径，见 `llm_engine.py:L124`)
3. 接口签名保留: `add_request()` / `step()` / `abort_request()` 签名不变

## 三、核心数据结构

### 3.1 请求生命周期数据结构

stateDiagram-v2

```

[*] --> WAITING: add_request()
WAITING --> WAITING_FOR_STRUCTURED_OUTPUT_GRAMMAR: use_structured_output
WAITING_FOR_STRUCTURED_OUTPUT_GRAMMAR --> WAITING: grammar compiled
WAITING --> WAITING_FOR_REMOTE_KVS: need remote KV
WAITING_FOR_REMOTE_KVS --> RUNNING: KV received
WAITING --> RUNNING: scheduled
RUNNING --> PREEMPTED: preempted (low priority / OOM)
PREEMPTED --> RUNNING: rescheduled
RUNNING --> FINISHED_STOPPED: stop token / stop string
RUNNING --> FINISHED_LENGTH_CAPPED: max_tokens reached
RUNNING --> FINISHED_ABORTED: client abort
RUNNING --> FINISHED_ERROR: internal error
RUNNING --> FINISHED_REPETITION: repetition detected
FINISHED_STOPPED --> [*]
FINISHED_LENGTH_CAPPED --> [*]
FINISHED_ABORTED --> [*]
FINISHED_ERROR --> [*]
FINISHED_REPETITION --> [*]
FINISHED_IGNORED --> [*]: prompt too long

```

### 3.2 Request (内部请求表示)

来源: `v1/request.py`

# `request.py` L59-L100 (精简)

```

class Request:
    def __init__(
        self,
        request_id: str,
        prompt_token_ids: list[int] | None,
        sampling_params: SamplingParams | None,
        pooling_params: PoolingParams | None,
        client_index: int = 0,
        arrival_time: float | None = None,
    ):

```

```

prompt_embeds: torch.Tensor | None = None,
mm_features: list[MultiModalFeatureSpec] | None = None,
lora_request: LoRARequest | None = None,
block_hasher: Callable | None = None,
...
):
    self.request_id = request_id
    self.status = RequestStatus.WAITING           # 初始状态
    self.sampling_params = sampling_params
    self.prompt_token_ids = prompt_token_ids
    self._output_token_ids: list[int] = []        # 生成的 token
    self.num_computed_tokens = 0                  # 已计算 token 数
    self.block_hashes: list[BlockHash] = []       # prefix caching hash
    self.events: list[EngineCoreEvent] = []       # 事件追踪
    self.num_preemptions = 0                      # 被抢占次数

```

关键字段语义:

| 字段                 | 类型                | 说明                                     |
|--------------------|-------------------|----------------------------------------|
| status             | RequestStatus     | 请求状态机（见上图）                             |
| output_token_ids   | ConstantList[int] | 只读视图，防止外部直接 append                     |
| all_token_ids      | ConstantList[int] | 包含 prompt + 所有生成 token                 |
| num_output_tokens  | int               | 当前已生成 token 数                          |
| block_hashes       | list[BlockHash]   | 每个 full block 的 hash，用于 prefix caching |
| is_prefill_chunk   | bool              | 是否处于非最终 prefill chunk                  |
| num_nans_in_logits | int               | logits 中 NaN 数量（用于检测异常）                |
| streaming_queue    | deque             | streaming session 续传队列                 |

### 3.3 EngineInput → EngineCoreRequest → Request 转换链

flowchart TD

```

subgraph UserAPI[" 用户 API 层"]
    U1["prompt='Hello'"]
    U2["prompt_token_ids=[1,2,3]"]
    U3["prompt_embeds=tensor(...)"]
end

subgraph EngineInput["EngineInput (TypedDict)"]
    TI["TokensInput<br/>type='token'<br/>prompt_token_ids=[]"]
    EI["EmbedsInput<br/>type='embeds'<br/>prompt_embeds=tensor"]
end

subgraph ECR["EngineCoreRequest (msgspec Struct)"]
    ECR1["request_id, prompt_token_ids,<br/>sampling_params, mm_features,<br/>lora_request, priority"]
end

subgraph Req["Request (内部)"]
    Req1["+ status, _output_token_ids,<br/>+ block_hashes, events,<br/>+ num_computed_tokens,<br/>+"]
end

U1 -->| "Renderer.render_chat()" | TI
U2 --> TI
U3 --> EI

```

```

TI -->|"InputProcessor.process_inputs()"| ECR1
EI --> ECR1
ECR1 -->|"Request.from_engine_core_request()"| Req1

style UserAPI fill:#e0f2f1,stroke:#00695c,color:#000
style EngineInput fill:#fff8e1,stroke:#ff8f00,color:#000
style ECR fill:#fce4ec,stroke:#c62828,color:#000
style Req fill:#e8f5e9,stroke:#2e7d32,color:#000

```

**EngineCoreRequest** 定义 (engine/init.py):

```

# __init__.py L80-L131
class EngineCoreRequest(
    msgspec.Struct,
    array_like=True,
    omit_defaults=True,
    gc=False,          # 禁用 GC 追踪 (性能关键)
):
    request_id: str
    prompt_token_ids: list[int] | None
    mm_features: list[MultiModalFeatureSpec] | None
    sampling_params: SamplingParams | None
    pooling_params: PoolingParams | None
    arrival_time: float
    lora_request: LoRARequest | None
    cache_salt: str | None
    data_parallel_rank: int | None
    prompt_embeds: torch.Tensor | None = None
    prompt_is_token_ids: list[bool] | None = None
    client_index: int = 0
    current_wave: int = 0
    priority: int = 0
    trace_headers: Mapping[str, str] | None = None
    resumable: bool = False
    external_req_id: str | None = None
    reasoning_ended: bool | None = None
    reasoning_parser_kwargs: dict[str, Any] | None = None

```

转换函数 (request.py L186-L209):

```

@classmethod
def from_engine_core_request(cls, request: EngineCoreRequest, block_hasher) -> "Request":
    return cls(
        request_id=request.request_id,
        client_index=request.client_index,
        prompt_token_ids=request.prompt_token_ids,
        prompt_embeds=request.prompt_embeds,
        mm_features=request.mm_features,
        sampling_params=request.sampling_params,
        pooling_params=request.pooling_params,
        arrival_time=request.arrival_time,
        lora_request=request.lora_request,
        block_hasher=block_hasher,
        resumable=request.resumable,
        ...
    )

```



### 3.4 输出数据结构体系

来源: v1/outputs.py

```
classDiagram
    class SamplerOutput {
        +sampled_token_ids: Tensor
        +logprobs_tensors: LogprobsTensors
    }
    class ModelRunnerOutput {
        +req_ids: list~str~
        +req_id_to_index: dict
        +sampled_token_ids: list~list~int~~
        +logprobs: LogprobsLists
        +prompt_logprobs_dict: dict
        +pooler_output: list
        +kv_connector_output: KVConnectorOutput
        +num_nans_in_logits: dict
    }
    class EngineCoreOutput {
        +request_id: str
        +new_token_ids: list~int~
        +new_logprobs: LogprobsLists
        +finish_reason: FinishReason
        +events: list~EngineCoreEvent~
        +prefill_stats: PrefillStats
    }
    class EngineCoreOutputs {
        +outputs: list~EngineCoreOutput~
        +scheduler_stats: SchedulerStats
        +timestamp: float
        +finished_requests: set~str~
        +utility_output: UtilityOutput
    }

    ModelRunnerOutput --> EngineCoreOutputs : scheduler.update_from_output()
    EngineCoreOutput --> EngineCoreOutputs : 聚合
    SamplerOutput --> ModelRunnerOutput : sample_tokens()
```

关键数据结构详解:

**SamplerOutput** (outputs.py L118-L124):

```
@dataclass
class SamplerOutput:
    sampled_token_ids: torch.Tensor      # [num_reqs, max_num_generated_tokens]
    logprobs_tensors: LogprobsTensors | None # logprob 信息
```

**ModelRunnerOutput** (outputs.py L166-L206):

```
@dataclass
class ModelRunnerOutput:
    req_ids: list[str]                # 请求 ID 列表
    req_id_to_index: dict[str, int]   # ID → 索引映射
    sampled_token_ids: list[list[int]] # 每个请求生成的 token IDs
    logprobs: LogprobsLists | None    # log probability
    prompt_logprobs_dict: dict        # prompt 阶段 logprob
```

```

pooler_output: list[Tensor | None]      # pooling 模型输出
kv_connector_output: KVConnectorOutput | None # KV transfer 信息
num_nans_in_logits: dict | None        # NaN 检测

```

**EngineCoreOutput** (engine/init.py L161-L191):

```

class EngineCoreOutput(msgspec.Struct, array_like=True, omit_defaults=True, gc=False):
    request_id: str
    new_token_ids: list[int]
    new_logprobs: LogprobsLists | None
    finish_reason: FinishReason | None # STOP/LENGTH/ABORT/ERROR/REPETITION
    events: list[EngineCoreEvent] | None
    prefill_stats: PrefillStats | None

```

### 3.5 FinishReason 状态机

```

# engine/__init__.py L42-L64
class FinishReason(enum.IntEnum):
    STOP = 0      # stop string / stop token emitted
    LENGTH = 1    # max_tokens consumed or max_model_len reached
    ABORT = 2     # aborted by client
    ERROR = 3     # retryable internal error (+ 500)
    REPETITION = 4 # repetitive pattern (hallucination)

```

---

## 四、设计原则

### 4.1 配置驱动 (Configuration-Driven)

vLLM 通过 28+ 配置模块 驱动全部行为，核心配置聚合在 VllmConfig:

graph TB

```

VC["VllmConfig<br/> 聚合根"] --> MC["ModelConfig<br/> 模型配置"]
VC --> CC["CacheConfig<br/>KV Cache 配置"]
VC --> PC["ParallelConfig<br/> 并行配置"]
VC --> SC["SchedulerConfig<br/> 调度配置"]
VC --> DC["DeviceConfig<br/> 设备配置"]
VC --> LC["LoadConfig<br/> 加载配置"]
VC --> AC["AttentionConfig<br/> 注意力配置"]
VC --> KC["KernelConfig<br/>Kernel 配置"]
VC --> CoC["CompilationConfig<br/> 编译配置"]
VC --> SpC["SpeculativeConfig<br/> 推测解码配置"]
VC --> OC["ObservabilityConfig<br/> 可观测性"]
VC --> QCfg["QuantizationConfig<br/> 量化配置"]
VC --> RC["ReasoningConfig<br/> 推理配置"]
VC --> SOC["StructuredOutputsConfig<br/> 结构化输出"]
VC --> KvTC["KVTransferConfig<br/>KV Transfer"]
VC --> LCfg["LoRAConfig<br/>LoRA 配置"]
VC --> OffCfg["OffloadConfig<br/>Offload 配置"]
VC --> PrCfg["ProfilerConfig<br/>Profiler"]
VC --> MamCfg["MambaConfig<br/>Mamba 配置"]
VC --> MMCfg["MultimodalConfig<br/> 多模态配置"]

style VC fill:#e1f5fe,stroke:#01579b,color:#000,stroke-width:2px
style MC fill:#e8f5e9,stroke:#2e7d32,color:#000
style CC fill:#e8f5e9,stroke:#2e7d32,color:#000

```

```

style PC fill:#e8f5e9,stroke:#2e7d32,color:#000
style SC fill:#e8f5e9,stroke:#2e7d32,color:#000
style DC fill:#e8f5e9,stroke:#2e7d32,color:#000
style LC fill:#e8f5e9,stroke:#2e7d32,color:#000
style AC fill:#e8f5e9,stroke:#2e7d32,color:#000
style KC fill:#e8f5e9,stroke:#2e7d32,color:#000
style CoC fill:#e8f5e9,stroke:#2e7d32,color:#000
style SpC fill:#e8f5e9,stroke:#2e7d32,color:#000
style OC fill:#e8f5e9,stroke:#2e7d32,color:#000
style QCfg fill:#e8f5e9,stroke:#2e7d32,color:#000
style RC fill:#e8f5e9,stroke:#2e7d32,color:#000
style SOC fill:#e8f5e9,stroke:#2e7d32,color:#000
style KvTC fill:#e8f5e9,stroke:#2e7d32,color:#000
style LCfg fill:#e8f5e9,stroke:#2e7d32,color:#000
style OffCfg fill:#e8f5e9,stroke:#2e7d32,color:#000
style PrCfg fill:#e8f5e9,stroke:#2e7d32,color:#000
style MamCfg fill:#e8f5e9,stroke:#2e7d32,color:#000
style MMCfg fill:#e8f5e9,stroke:#2e7d32,color:#000

```

`**VllmConfig.__post_init__()` (vllm.py L758-L1401) 在初始化时执行大量交叉验证和默认值推导:

- 优化级别应用 (O0/O1/O2/O3)
- async scheduling 自动启用判断
- cudagraph capture sizes 计算
- SP (Sequence Parallelism) 阈值推导
- platform-specific defaults 应用
- KV transfer 兼容性检查

优化级别系统:

```

# vllm.py L68-L265
class OptimizationLevel(IntEnum):
    O0 = 0 # 无优化, 最快启动
    O1 = 1 # Dynamo+Inductor + Piecewise CUDAGraph
    O2 = 2 # Full + Piecewise CUDAGraph (默认)
    O3 = 3 # O2 + FlashInfer autotune

```

## 4.2 注册表模式 (Registry Pattern)

模型注册表 `model_executor/models/registry.py` 维护了 **150+** 模型架构的注册表:

```

# registry.py L70-L221 (精简)
_TEXT_GENERATION_MODELS = {
    "LlamaForCausalLM": ("llama", "LlamaForCausalLM"),
    "Qwen3ForCausalLM": ("qwen3", "Qwen3ForCausalLM"),
    "DeepseekV3ForCausalLM": ("deepseek_v2", "DeepseekV3ForCausalLM"),
    "Gemma4ForCausalLM": ("gemma4", "Gemma4ForCausalLM"),
    # ... 共 200+ 条目
}

_EMBEDDING_MODELS = { ... } # Embedding 模型
_MULTIMODAL_MODELS = { ... } # 多模态模型
_SPECULATIVE_DECODING_MODELS = { ... } # 推测解码模型

_VLLM_MODELS = {
    **_TEXT_GENERATION_MODELS,
    **_EMBEDDING_MODELS,
}

```

```

    **_MULTIMODAL_MODELS,
    **_SPECULATIVE_DECODING_MODELS,
    ...
}

ModelRegistry = _ModelRegistry({
    model_arch: _LazyRegisteredModel(    # 懒加载! 避免 import 时初始化 CUDA
        module_name=f"vllm.model_executor.models.{mod_relname}",
        class_name=cls_name,
    )
    for model_arch, (mod_relname, cls_name) in _VLLM_MODELS.items()
})

```

关键设计: 使用 `_LazyRegisteredModel` 实现延迟导入, 避免在非 GPU 进程中初始化 CUDA context。

多模态处理器注册表 `multimodal/registry.py` 使用装饰器模式注册处理器:

```

# registry.py L142-L174 (精简)
class MultiModalRegistry:
    def register_processor(
        self,
        processor: MultiModalFactory[_I],
        *,
        info: ProcessingInfoFactory[_I],
        dummy_inputs: DummyInputsBuilderFactory[_I],
    ):
        def wrapper(model_cls: N) -> N:
            model_cls._processor_factory = _ProcessorFactories(
                info=info, dummy_inputs=dummy_inputs, processor=processor,
            )
            return model_cls
        return wrapper

# 使用示例 (在模型文件中):
# @MULTIMODAL_REGISTRY.register_processor(MyProcessor.factory, ...)
# class MyModel(nn.Module): ...

```

### 4.3 策略模式 (Strategy Pattern)

注意力后端选择器 `v1/attention/selector.py` 根据运行时参数选择最优注意力后端:

flowchart TD

```

SEL["get_attn_backend()<br/>selector.py L53-L103"] --> CONFIG["构建 AttentionSelectorConfig"]
CONFIG --> CACHE{"<br/>@cache 装饰器 <br/>_cached_get_attn_backend()"}
CACHE --> PLATFORM["current_platform.<br/>get_attn_backend_cls()"]
PLATFORM --> BACKEND["AttentionBackend 子类"]

```

```

BACKEND --> FA["FlashAttention"]
BACKEND --> FI["FlashInfer"]
BACKEND --> TRITON["TritonAttention"]
BACKEND --> ROCM["ROCM AITER"]
BACKEND --> MLA["MLA Backends<br/>(DeepSeek)"]
BACKEND --> MAMBA["Mamba Attention"]

```

```

style SEL fill:#e1f5fe,stroke:#01579b,color:#000
style BACKEND fill:#fce4ec,stroke:#c62828,color:#000

```

选择参数空间 (selector.py L22-L50):

```
class AttentionSelectorConfig(NamedTuple):
    head_size: int
    dtype: torch.dtype
    kv_cache_dtype: CachedType | None
    block_size: int | None
    use_mla: bool = False           # DeepSeek MLA
    has_sink: bool = False          # Sink attention
    use_sparse: bool = False        # Sparse attention
    use_mm_prefix: bool = False     # Multimodal prefix
    use_per_head_quant_scales: bool = False
    attn_type: str = AttentionType.DECODER
    use_non_causal: bool = False
    use_batch_invariant: bool = False # Batch invariant mode
```

量化后端选择 量化配置通过 QuantizationConfig 及其子类驱动不同量化后端的选择: - **FP8**: Fp8Config → Fp8Linear - **GPTQ**: GptqConfig → GptqMarlinLinear - **AWQ**: AwqConfig → AwqLinear - **NVFP4**: Nvfp4Config → Nvfp4Linear

#### 4.4 工厂模式 (Factory Pattern)

**Executor** 工厂 如4.1节所述, Executor.get\_class() 是经典的工厂方法:

```
# abstract.py L48-L92
@staticmethod
def get_class(vllm_config: VllmConfig) -> type["Executor"]:
    backend = parallel_config.distributed_executor_backend
    match backend:
        case "ray": return RayExecutorV2 or RayDistributedExecutor
        case "mp": return MultiprocExecutor
        case "uni": return UniProcExecutor
        case "external_launcher": return ExecutorWithExternalLauncher
        case type(): return backend # 用户自定义子类
        case str(): return resolve_obj_by_qualname(backend) # 按限定名加载
```

**EngineCoreClient** 工厂 core\_client.py 中的 make\_client() 根据 multiprocess\_mode 和 asyncio\_mode 创建不同的客户端实现:

- **inproc** 模式: 直接持有 EngineCore 引用
- **multiproc** 模式: 通过 ZMQ socket 与 EngineCoreProc 通信
- **asyncio** 模式: 异步版本客户端

**KV Cache Offload** 工厂 kv\_offload/factory.py 根据 kv\_transfer\_config.kv\_connector 选择不同的 KV offload 后端: - "OffloadingConnector": CPU offloading - "LMCacheConnectorV1": LMCache 集成 - NIXL Connector: RDMA-based transfer

#### 4.5 其他重要设计模式

| 模式    | 应用场景                        | 位置                     |
|-------|-----------------------------|------------------------|
| 观察者模式 | 统计日志 / metrics 收集           | metrics/               |
| 适配器模式 | 不同平台 (CUDA/ROCm/XPU/TPU) 差异 | platforms/             |
| 装饰器模式 | 编译 pass 注入 / tracing        | compilation/           |
| 原型模式  | Dummy input 构建              | multimodal/processing/ |

| 模式   | 应用场景                | 位置                                      |
|------|---------------------|-----------------------------------------|
| 命令模式 | Utility method 远程调用 | EngineCoreProc._handle_client_request() |

## 五、关键交互时序

### 5.1 完整推理请求生命周期

```
sequenceDiagram
    participant Client as 客户端
    participant API as OpenAI API
    participant Eng as LLMEngine
    participant IP as InputProcessor
    participant Core as EngineCore
    participant Sched as Scheduler
    participant Exec as Executor
    participant MR as ModelRunner
    participant OP as OutputProcessor

    Client->>API: POST /v1/chat/completions
    API->>Eng: add_request(prompt, params)
    Eng->>IP: process_inputs(prompt, params)
    IP->>Eng: EngineCoreRequest
    Eng->>Core: add_request(request)
    Core->>Sched: add_request(Request)

    loop 每个 iteration
        Core->>Sched: schedule()
        Sched-->>Core: SchedulerOutput
        Core->>Exec: execute_model(SchedulerOutput)
        Exec->>MR: forward(batch)
        MR-->>Exec: hidden_states
        Exec->>MR: sample_tokens(grammar_output)
        MR-->>Exec: sampled_token_ids
        Exec-->>Core: ModelRunnerOutput
        Core->>Sched: update_from_output(output)
        Sched-->>Core: EngineCoreOutputs
        Core-->>Eng: EngineCoreOutputs
    end

    Eng->>OP: process_outputs(outputs)
    OP-->>Eng: RequestOutput[]
    Eng->>API: stream chunks / final response
    API-->>Client: JSON response
```

### 5.2 多进程架构下的 IPC 流程

```
sequenceDiagram
    participant FE as 前端进程 <br/>(API Server)
    participant ECP as EngineCoreClient
    participant IS as InputSocket Thread
    participant EQ as input_queue
    participant EC as EngineCore<br/>(Busy Loop)
```

```

participant OS as OutputSocket Thread
participant OQ as output_queue

FE->>ECP: add_request(req)
ECP->>IS: ZMQ SEND (msgpack)
IS->>EQ: put((ADD, Request))
EQ->>EC: get() ← busy loop 轮询
EC->>EC: step() → schedule → execute → update
EC->>OQ: put((client_idx, EngineCoreOutputs))
OQ->>OS: ZMQ SEND (msgpack)
OS->>ECP: recv() → get_output()
ECP-->>FE: EngineCoreOutputs

```

## 六、目录导航速查

| 目录              | 层次    | 核心文件                                   | 一句话说明             |
|-----------------|-------|----------------------------------------|-------------------|
| entrypoints/    | L1    | openai/api_server.py                   | HTTP API 入口       |
| engine/         | L2    | llm_engine.py                          | 别名重导出到 v1         |
| v1/engine/      | L2    | llm_engine.py, core.py                 | 引擎门面 + 核心循环       |
| v1/core/sched/  | L3    | scheduler.py,<br>interface.py          | 调度算法              |
| v1/executor/    | L4    | abstract.py,<br>uniproc_executor.py    | 分布式执行             |
| v1/worker/      | L4-L5 | gpu/model_runner.py,<br>worker_base.py | Worker 抽象与 GPU 执行 |
| model_executor/ | L5    | models/registry.py,<br>models/llama.py | 模型加载与执行           |
| v1/attention/   | L5-L6 | selector.py, backends/                 | 注意力后端选择           |
| kernels/        | L6    | vllm_c.py                              | Custom ops 入口     |
| config/         | 横切    | vllm.py                                | 全局配置聚合            |
| multimodal/     | 横切    | registry.py                            | 多模态处理器注册          |
| distributed/    | 横切    | parallel_state.py                      | 分布式通信原语           |

## 七、扩展阅读指引

阅读完本文档后，建议按以下顺序深入源码：

1. **EngineCore.step()** 一理解单步执行的完整流程
2. **Scheduler.schedule()** 一理解调度算法细节（chunked prefill / continuous batching / preempt）
3. **GPUModelRunner.execute\_model()** 一理解模型前向传播的完整链路
4. **InputProcessor.process\_inputs()** 一理解从 HTTP request 到 EngineCoreRequest 的完整转换
5. **AttentionSelectorConfig** 一理解注意力后端的策略选择机制

## vLLM 配置系统 (Config System) 深度分析

定位：vLLM 的配置系统是整个推理引擎的核心骨架，采用层级化聚合设计，以 `VllmConfig` 为顶层容器，统筹管理模型、缓存、并行、调度、设备、编译优化等 20+ 子配置域。本文档基于 `vllm/config/` 目录源码进行完整分析。

```

graph LR
    VC[VllmConfig<br/> 顶层聚合容器] --> MC[ModelConfig<br/> 模型架构]
    VC --> CC[CacheConfig<br/>KV 缓存]
    VC --> PC[ParallelConfig<br/> 并行策略]
    VC --> SC[SchedulerConfig<br/> 调度策略]
    VC --> DC[DeviceConfig<br/> 设备检测]
    VC --> LC[LoadConfig<br/> 权重加载]
    VC --> LoRA[LoRAConfig<br/> 适配器]
    VC --> QC[QuantizationConfig<br/> 量化方案]
    VC --> QC
    VC --> CmpC[CompilationConfig<br/>torch.compile]
    VC --> SpC[SpeculativeConfig<br/> 推测解码]
    VC --> OC[ObservabilityConfig<br/> 可观测性]
    VC --> OffC[OffloadConfig<br/> 权重卸载]
    VC --> RC[ReasoningConfig<br/> 推理链]
    VC --> SOC[StructuredOutputsConfig<br/> 结构化输出]

    style VC fill:#fff3e0,stroke:#e65100,color:#000
    style MC fill:#e3f2fd,stroke:#1565c0,color:#000
    style CC fill:#e8f5e9,stroke:#2e7d32,color:#000
    style PC fill:#fce4ec,stroke:#c62828,color:#000
    style SC fill:#f3e5f5,stroke:#7b1fa2,color:#000
    style DC fill:#ede7f6,stroke:#512da8,color:#000
    style LC fill:#fff8e1,stroke:#ff8f00,color:#000
    style CmpC fill:#e0f2f1,stroke:#00695c,color:#000

```

## 目录

- 一、VllmConfig 主配置类
  - 1.1 设计理念：聚合容器模式
  - 1.2 完整子配置字段清单
  - 1.3 Config Hash 与缓存机制
  - 1.4 **post\_init** 验证流程
- 二、各子配置详解
  - 2.1 ModelConfig —模型架构与 dtype
  - 2.2 CacheConfig —KV 缓存管理
  - 2.3 ParallelConfig —并行执行
  - 2.4 SchedulerConfig —调度策略
  - 2.5 DeviceConfig —设备检测
  - 2.6 LoadConfig —权重加载
  - 2.7 LoRAConfig —LoRA 适配器
  - 2.8 QuantizationConfig —量化方案
  - 2.9 CompilationConfig —torch.compile 与 CUDA Graphs
  - 2.10 SpeculativeConfig —推测解码
  - 2.11 ObservabilityConfig —可观测性
  - 2.12 OffloadConfig / ReasoningConfig / StructuredConfigs 等
- 三、配置验证与合并逻辑
  - 3.1 model\_validator 方法
  - 3.2 device 配置如何影响其他配置
- 四、环境变量覆盖机制
- 五、OptimizationLevel (O0-O3) 各级别详解



## 一、VllmConfig 主配置类

### 1.1 设计理念：聚合容器模式

VllmConfig 采用 “上帝对象” (God Object) 聚合模式，作为唯一顶层配置入口，将所有子配置域组合为一个扁平化的命名空间。这种设计的好处是：

- 单点传递：整个代码库只需传递一个 vllm\_config 对象
- 跨域验证：可以在 \_\_post\_init\_\_ 中实现跨配置域的约束检查
- Hash 一致性：通过统一的 compute\_hash() 确保编译缓存的正确性

源码位置：vllm.py:269-354

```
@config(config=ConfigDict(arbitrary_types_allowed=True))
class VllmConfig:
    """Dataclass which contains all vllm-related configuration.
    This simplifies passing around the distinct configurations in the codebase."""

    model_config: ModelConfig = None
    cache_config: CacheConfig = Field(default_factory=CacheConfig)
    parallel_config: ParallelConfig = Field(default_factory=ParallelConfig)
    scheduler_config: SchedulerConfig = Field(
        default_factory=SchedulerConfig.default_factory,
    )
    device_config: DeviceConfig = Field(default_factory=DeviceConfig)
    load_config: LoadConfig = Field(default_factory=LoadConfig)
    # ... 更多子配置字段
```

### 1.2 完整子配置字段清单

下表列出了 VllmConfig 中所有子配置引用字段：

| 字段名                       | 类型                         | 默认值     | 必需 | 用途                   |
|---------------------------|----------------------------|---------|----|----------------------|
| model_config              | ModelConfig                | None    | —  | 模型架构、dtype、tokenizer |
| cache_config              | CacheConfig                | Factory | —  | KV 缓存块大小、GPU 内存比例    |
| parallel_config           | ParallelConfig             | Factory | —  | TP/PP/DP 并行度         |
| scheduler_config          | SchedulerConfig            | Factory | —  | 调度策略、延迟阈值            |
| device_config             | DeviceConfig               | Factory | —  | 设备检测与选择              |
| load_config               | LoadConfig                 | Factory | —  | 权重加载方式               |
| offload_config            | OffloadConfig              | Factory | —  | 模型权重卸载到 CPU          |
| attention_config          | AttentionConfig            | Factory | —  | Attention 后端选择       |
| mamba_config              | MambaConfig                | Factory | —  | Mamba SSM 缓存配置       |
| kernel_config             | KernelConfig               | Factory | —  | 自定义算子配置              |
| lora_config               | LoRAConfig \\\nNone        | None    | —  | LoRA 适配器             |
| speculative_config        | SpeculativeConfig \\\nNone | None    | —  | 推测解码参数               |
| structured_outputs_config | StructuredOutputsConfig    | Factory | —  | 结构化输出                |
| observability_config      | ObservabilityConfig        | Factory | —  | 可观测性设置               |

| 字段名                    | 类型                              | 默认值     | 必需 | 用途                          |
|------------------------|---------------------------------|---------|----|-----------------------------|
| quant_config           | QuantizationConfig<br>\\ None   | None    | —  | 量化方案（运行时解析）                 |
| compilation_config     | CompilationConfig               | Factory | —  | torch.compile / CUDA Graphs |
| profiler_config        | ProfilerConfig                  | Factory | —  | 性能分析器                       |
| kv_transfer_config     | KVTransferConfig<br>\\ None     | None    | —  | 分布式 KV 缓存传输                 |
| kv_events_config       | KVEventsConfig<br>\\ None       | None    | —  | KV 事件发布                     |
| ec_transfer_config     | ECTransferConfig<br>\\ None     | None    | —  | 分布式 EC 缓存传输                 |
| reasoning_config       | ReasoningConfig<br>\\ None      | None    | —  | 推理模型（CoT）                   |
| weight_transfer_config | WeightTransferConfig<br>\\ None | None    | —  | RL 训练权重传输                   |

全局控制字段:

| 字段名                | 类型                | 默认值        | 用途                                |
|--------------------|-------------------|------------|-----------------------------------|
| instance_id        | str               | "" (自动生成)  | 实例唯一标识                            |
| optimization_level | OptimizationLevel | 02         | 优化级别 O0-O3                        |
| performance_mode   | str               | "balanced" | balanced/interactivity/throughput |
| shutdown_timeout   | int               | 0          | 关闭宽限期 (秒)                         |

### 1.3 Config Hash 与缓存机制

每个配置类都实现了 `compute_hash()` 方法，用于生成该配置的唯一哈希值。`VllmConfig.compute_hash()` 聚合所有子配置的 hash，最终生成一个 10 位十六进制字符串，作为编译缓存的 key。

核心原则（见 `vllm.py:362-468`）:

只纳入影响计算图结构的因子，排除运行时参数（如 `gpu_memory_utilization`、`num_gpu_blocks`）和纯调试参数。

```
def compute_hash(self) -> str:
    factors: list[Any] = []
    from vllm import __version__
    vllm_factors = []
    vllm_factors.append(__version__) # 版本号

    if self.model_config:
        vllm_factors.append(self.model_config.compute_hash())
    if self.cache_config:
        vllm_factors.append(self.cache_config.compute_hash())
    # ... 聚合所有子配置的 hash
    factors.append(vllm_factors)

    hash_str = safe_hash(str(factors).encode(), usedforsecurity=False).hexdigest()[:10]
    return hash_str
```

Hash 因子排除示例（以 `ModelConfig` 为例，见 `model.py:355-398`）:

```

ignored_factors = {
    "convert", "tokenizer", "tokenizer_mode", "seed",
    "hf_config_path", "allowed_local_media_path",
    "enforce_eager", "logprobs_mode", "disable_cascade_attn",
    "skip_tokenizer_init", "served_model_name",
    # ... 这些不影响计算图结构
}

```

#### 1.4 \_\_post\_init\_\_ 验证流程

VllmConfig.\_\_post\_init\_\_() 是整个配置系统的核心验证枢纽（约 **650** 行），位于 vllm.py:758-1401，按顺序执行以下关键步骤：

1. 生成 instance\_id (时间戳)
2. try\_verify\_and\_update\_config() — 模型特定校验
3. model\_config.verify\_with\_parallel\_config() — TP 头数整除校验
4. LoRA 配置校验
5. Mamba stochastic rounding 校验
6. 量化配置解析 (\_get\_quantization\_config)
7. DeepGemm 自动禁用检测
8. 异步调度 (async\_scheduling) 兼容性判断
9. NCCL DP 同步策略决定
10. Cascade attention 与 async spec 解码互斥
11. 编译模式决定 (enforce\_eager / TORCH\_COMPILE\_DISABLE)
12. quant\_fp8 自定义算子启用
13. 平台默认值应用 (current\_platform.apply\_config\_platform\_defaults)
14. OptimizationLevel 默认应用
15. Kernel IR op priority 设置
16. CUDA Graph sizes 计算 (\_set\_cudagraph\_sizes)
17. Compile ranges 计算 (\_set\_compile\_ranges)
18. Splitting ops 设置 for V1
19. Sequence Parallelism 配置
20. Hybrid KV cache manager 规则
21. KV transfer config 后处理
22. 最终兼容性检查

## 二、各子配置详解

### 2.1 ModelConfig — 模型架构与 dtype

源码位置: model.py:109-2200

ModelConfig 是最复杂的子配置类，负责：- 从 HuggingFace 加载模型配置 (hf\_config) - 推断数据类型 (dtype) - 确定 tokenizer 模式和最大序列长度 - 判断模型属性 (MoE、多模态、encoder-decoder、混合注意力等)

关键字段表

| 字段                | 类型              | 默认值               | 说明                                   |
|-------------------|-----------------|-------------------|--------------------------------------|
| model             | str             | "Qwen/Qwen3-0.6B" | 模型名称或路径                              |
| tokenizer         | str \  None     | None (= model)    | Tokenizer 路径                         |
| tokenizer_mode    | str             | "auto"            | auto/hf/slow/mistral/deepseek_v32/v4 |
| trust_remote_code | bool            | False             | 是否信任远程代码                             |
| dtype             | str/torch.dtype | "auto"            | auto/half/float16/bfloat16/float32   |
| seed              | int             | 0                 | 随机种子                                 |

| 字段                     | 类型              | 默认值    | 说明                           |
|------------------------|-----------------|--------|------------------------------|
| max_model_len          | int \  None     | None   | 最大上下文长度（自动推导）                |
| quantization           | str \  None     | None   | 量化方法                         |
| enforce_eager          | bool            | False  | 强制 eager 模式                  |
| revision               | str \  None     | None   | 模型版本<br>(branch/tag/commit)  |
| runner                 | str             | "auto" | generate/pooling/draft       |
| convert                | str             | "auto" | 模型转换类型                       |
| enable_sleep_mode      | bool            | False  | 启用 GPU sleep 模式              |
| max_logprobs           | int             | 20     | 最大 logprobs 返回数              |
| disable_sliding_window | bool            | False  | 禁用滑动窗口                       |
| disable_cascade_attn   | bool            | True   | 禁用级联注意力                      |
| skip_tokenizer_init    | bool            | False  | 跳过 tokenizer 初始化             |
| enable_prompt_embeds   | bool            | False  | 启用 prompt embeddings 输入      |
| served_model_name      | str\ list\ None | None   | API 服务名                      |
| model_impl             | str             | "auto" | vllm/transformers/terratorch |
| renderer_num_workers   | int             | 1      | 渲染线程池大小                      |

**dtype** 解析逻辑 源码: model.py:1992-2037

```

_STR_DTYPE_TO_TORCH_DTYPE = {
    "half": torch.float16,
    "float16": torch.float16,
    "float": torch.float32,
    "float32": torch.float32,
    "bfloat16": torch.bfloat16,
}

_FLOAT16_NOT_SUPPORTED_MODELS = {
    "gemma2": "Numerical instability.",
    "gemma3": "Numerical instability.",
    "plamo2": "Numerical instability.",
    "glm4": "Numerical instability.",
}

```

当 dtype="auto" 时: 1. FP32/BF16 模型 → 优先 BF16 (pooling 模型优先 FP16) 2. 根据平台支持的 dtype 列表降级 3. 特定模型 (Gemma2/3, PLaMo2, GLM4) 禁止使用 FP16

**max\_model\_len** 推导 源码: model.py:2064-2200

优先级链: 1. 用户显式指定 → 直接使用 2. HF config 中的 model\_max\_length / max\_position\_embeddings 3. RoPE scaling factor 修正 4. Tokenizer config 中的 model\_max\_length (取较小值) 5. 若推导值为 inf → 回退到 2048 并警告

重要属性 (**Properties**)

| 属性           | 类型   | 说明                                     |
|--------------|------|----------------------------------------|
| is_moe       | bool | 是否为 MoE 模型 (num_experts > 0)           |
| is_quantized | bool | 是否量化 (hf_config 有 quantization_config) |

| 属性                  | 类型          | 说明                                               |
|---------------------|-------------|--------------------------------------------------|
| is_encoder_decoder  | bool        | 是否为 encoder-decoder 架构                           |
| is_multimodal_model | bool        | 是否为多模态模型                                         |
| is_hybrid           | bool        | 是否为混合注意力模型（attention + mamba/linear attention）   |
| use_mla             | bool        | 是否使用 MLA（DeepSeek V3 系列）                         |
| is_nvfp4_quantized  | bool        | 是否为 NVFP4 量化                                     |
| attn_type           | str         | decoder/encoder/encoder_decoder/hybrid/attention |
| head_dtype          | torch.dtype | 头部层 dtype（pooling 模型默认 fp32）                     |

## 2.2 CacheConfig — KV 缓存管理

源码位置: cache.py:42-268

关键字段表

| 字段                       | 类型             | 默认值                        | 说明                  |
|--------------------------|----------------|----------------------------|---------------------|
| block_size               | int            | 16<br>(DEFAULT_BLOCK_SIZE) | KV 缓存块大小（token 数）   |
| gpu_memory_utilization   | float          | 0.92                       | GPU 内存利用率（0-1）      |
| cache_dtype              | str            | "auto"                     | KV 缓存数据类型           |
| enable_prefix_caching    | bool           | True                       | 启用前缀缓存              |
| prefix_caching_hash_algo | str            | "sha256"                   | 前缀缓存哈希算法            |
| num_gpu_blocks_override  | int \   None   | None                       | 覆盖 GPU 块数（测试用）      |
| sliding_window           | int \   None   | None                       | 滑动窗口大小              |
| kv_cache_memory_bytes    | int \   None   | None                       | KV 缓存内存字节数          |
| kv_offloading_size       | float \   None | None                       | KV 卸载到 CPU 的大小（GiB） |
| kv_offloading_backend    | str            | "native"                   | native/lmcache      |
| mamba_block_size         | int \   None   | None                       | Mamba 缓存块大小         |
| mamba_cache_dtype        | str            | "auto"                     | Mamba 缓存 dtype      |
| mamba_ssm_cache_dtype    | str            | "auto"                     | Mamba SSM 状态 dtype  |
| mamba_cache_mode         | str            | "none"                     | all/align/none      |
| kv_sharing_fast_prefix   | bool           | False                      | KV 共享快速 prefill     |

支持的 KV Cache 数据类型 源码: cache.py:18-34

```
CacheDType = Literal[
    "auto",           # 自动选择（通常跟随模型 dtype）
    "float16",        # FP16
    "bfloat16",       # BF16
    "fp8",            # FP8 E4M3 (CUDA 11.8+)
    "fp8_e4m3",       # FP8 E4M3 显式指定
    "fp8_e5m2",       # FP8 E5M2
    "fp8_inc",        # Intel GPU FP8
    "fp8_ds_mla",     # DeepSeek MLA 专用 FP8
    "turboquant_k8v4", # TurboQuant K8V4
```

```

        "turboquant_4bit_nc", # TurboQuant 4-bit
        "turboquant_3bit_nc", # TurboQuant 3-bit
        "int8_per_token_head", # Per-token-head INT8
        "fp8_per_token_head", # Per-token-head FP8
        "nvfp4",             # NVFP4 (Blackwell 专用)
    ]

```

**block\_size** 默认值与验证 源码: cache.py:223-236

```

@model_validator(mode="after")
def _apply_block_size_default(self) -> "CacheConfig":
    if self._block_size_resolved:
        return self
    object.__setattr__(self, "_block_size_resolved", True)
    if self.block_size is None:
        object.__setattr__(self, "block_size", self.DEFAULT_BLOCK_SIZE) # 16
    else:
        object.__setattr__(self, "user_specified_block_size", True)
    return self

```

## 2.3 ParallelConfig 一并行执行

源码位置: parallel.py:108-957

这是配置系统中第二复杂的类，管理多种并行维度及其交互。

关键字段表

| 字段                            | 类型   | 默认值                       | 说明                           |
|-------------------------------|------|---------------------------|------------------------------|
| pipeline_parallel_size        | int  | 1                         | 流水线并行度 (PP)                  |
| tensor_parallel_size          | int  | 1                         | 张量并行度 (TP)                   |
| prefill_context_parallel_size | int  | 1                         | Prefill 上下文并行度 (PCP)         |
| data_parallel_size            | int  | 1                         | 数据并行度 (DP)                   |
| data_parallel_size_local      | int  | 1                         | 本地 DP 度                      |
| data_parallel_rank            | int  | 0                         | DP rank                      |
| data_parallel_backend         | str  | "mp"                      | ray/mp                       |
| data_parallel_external_hbm    | bool | False                     | 外部负载均衡模式                     |
| data_parallel_hybrid_lb       | bool | False                     | 混合 LB 模式                     |
| enable_expert_parallel        | bool | False                     | 专家并行 (EP)                    |
| enable_ep_weight_filter       | bool | False                     | EP 权重过滤 (减少 I/O)             |
| enable_eplb                   | bool | False                     | EP 负载均衡                      |
| expert_placement_strategy     | str  | "linear"                  | linear/round_robin           |
| all2all_backend               | str  | "allgather_reducescatter" | All2All 后端                   |
| enable_dbo                    | bool | False                     | 双批重叠 (Dual Batch Overlap)    |
| ubatch_size                   | int  | 0                         | 微批次大小                        |
| decode_context_parallel_size  | int  | 1                         | Decode 上下文并行 (DCP)           |
| dcp_comm_backend              | str  | "ag_rs"                   | DCP 通信后端                     |
| distributed_executor_backend  | str  | None (自动选择)               | ray/mp/uni/external_launcher |
| worker_cls                    | str  | "auto"                    | Worker 类                     |
| master_addr                   | str  | "127.0.0.1"               | 多节点主地址                       |

| 字段                        | 类型   | 默认值   | 说明              |
|---------------------------|------|-------|-----------------|
| master_port               | int  | 29501 | 多节点主端口          |
| nnodes                    | int  | 1     | 节点数             |
| numa_bind                 | bool | False | NUMA 绑定         |
| disable_custom_all_reduce | bool | False | 禁用自定义 AllReduce |

**world\_size** 计算 源码: parallel.py:760-770

```
def __post_init__(self) -> None:
    self.world_size = (
        self.pipeline_parallel_size
        * self.tensor_parallel_size
        * self.prefill_context_parallel_size
    )
    if self.distributed_executor_backend == "external_launcher":
        self.world_size *= self.data_parallel_size
```

注意: DCP 不改变 world\_size, 它复用 TP 组内的 GPU。TP 必须能被 DPC 整除:

```
if self.tensor_parallel_size % self.decode_context_parallel_size != 0:
    raise ValueError(f"tp_size={...} must be divisible by dcp_size={...}")
```

**All2All** 后端选项 源码: parallel.py:40-51

```
All2AllBackend = Literal[
    "naive",           # 朴素实现
    "pplx",           # PPLX (已移除)
    "deepep_high_throughput",  # DeepEP 高吞吐
    "deepep_low_latency",     # DeepEP 低延迟
    "mori",            # MORI 内核
    "nixl_ep",        # NIXL EP
    "allgather_reducescatter", # AG + RS (默认)
    "flashinfer_all2allv",    # FlashInfer A2A (NVLink 双向)
    "flashinfer_nvlink_two_sided", # FlashInfer NVLink 双向
    "flashinfer_nvlink_one_sided", # FlashInfer NVLink 单向
]
```

**DCP (Decode Context Parallel)** 约束 源码: parallel.py:310-343

DCP 用于将 decode 阶段的 KV cache 计算分散到多个 GPU 上, 仅适用于非 MLA 的 GQA/MQA 模型:

```
# TP > total_kv_heads (MQA/GQA 条件)
# DCP <= TP // total_kv_heads
# q_per_kv % dcp_size == 0
```

## 2.4 SchedulerConfig —调度策略

源码位置: scheduler.py:26-307

关键字段表

| 字段          | 类型  | 默认值        | 说明                     |
|-------------|-----|------------|------------------------|
| runner_type | str | "generate" | generate/pooling/draft |

| 字段                           | 类型               | 默认值    | 说明                    |
|------------------------------|------------------|--------|-----------------------|
| max_num_batched_tokens       | int              | 2048   | 单次迭代最大 token 数        |
| max_num_scheduled_tokens     | int \  None      | None   | 调度器最大发出 token 数       |
| max_num_seqs                 | int              | 128    | 最大并发序列数               |
| max_num_partial_prefills     | int              | 1      | chunked prefill 并发数   |
| max_long_partial_prefills    | int              | 1      | 长 prompt 并发 prefill 数 |
| long_prefill_token_threshold | int              | 0      | 长 prompt 阈值           |
| enable_chunked_prefill       | bool             | True   | 启用分块 prefill          |
| policy                       | str              | "fcfs" | fcfs/priority 调度策略    |
| disable_chunked_mm_inputs    | bool             | False  | 禁用多模态分块输入             |
| scheduler_cls                | str/type \  None | None   | 自定义调度器类               |
| disable_hybrid_kv_cache      | bool \  None     | None   | 禁用混合 KV 管理            |
| scheduler_reserve_full       | bool             | True   | 保留完整输入序列长度            |
| async_scheduling             | bool \  None     | None   | 异步调度                  |
| stream_interval              | int              | 1      | 流式间隔 (token 数)        |

**InitVar 参数** SchedulerConfig 使用两个 InitVar 参数，它们不存储在实例中，仅在初始化时使用：

```
max_model_len: InitVar[int] # 来自 ModelConfig
is_encoder_decoder: InitVar[bool] # 来自 ModelConfig
```

验证规则 源码: scheduler.py:259-306

1. max\_num\_batched\_tokens < max\_model\_len 且未启用 chunked prefill → 报错
2. max\_num\_batched\_tokens < max\_num\_seqs → 报错
3. max\_num\_batched\_tokens > max\_num\_seqs \* max\_model\_len → 警告
4. max\_num\_partial\_prefills > 1 需要 enable\_chunked\_prefill=True
5. Encoder-decoder 模型自动禁用 chunked prefill 和 prefix caching

## 2.5 DeviceConfig — 设备检测

源码位置: device.py:17-73

关键字段表

| 字段          | 类型                             | 默认值    | 说明      |
|-------------|--------------------------------|--------|---------|
| device      | Device \  torch.device \  None | "auto" | 设备类型    |
| device_type | str                            | (自动推断) | 运行时设备类型 |

设备类型选项

```
Device = Literal["auto", "cuda", "cpu", "tpu", "xpu"]
```

当 device="auto" 时（默认），通过 current\_platform.device\_type 自动检测。对于 TPU 等特殊设备，会将 device 设为 None（因为 TPU 在 CPU 上处理输入）。

## 2.6 LoadConfig — 权重加载

源码位置: load.py:23-137



## 关键字段表

| 字段                        | 类型           | 默认值               | 说明                              |
|---------------------------|--------------|-------------------|---------------------------------|
| load_format               | str          | "auto"            | 权重格式                            |
| download_dir              | str \   None | None              | 下载目录                            |
| safetensors_load_strategy | str \   None | None              | safetensors 加载策略                |
| model_loader_extra_config | dict         | {}                | 模型加载器额外配置                       |
| device                    | str \   None | None              | 加载目标设备                          |
| ignore_patterns           | list[str]    | ["original/**/*"] | 忽略的文件模式                         |
| use_tqdm_on_load          | bool         | True              | 显示进度条                           |
| pt_load_map_location      | str/dict     | "cpu"             | PyTorch checkpoint map_location |

## 支持的加载格式

- "auto": 自动尝试 safetensors → pt
- "pt": PyTorch bin 格式
- "safetensors": Safetensors 格式
- "instanttensor": InstantTensor (CUDA Direct I/O + 分布式预取)
- "npcache": NumPy 缓存加速
- "dummy": 随机初始化 (性能分析用)
- "tensorizer": CoreWeave Tensorizer
- "runai\_streamer" / "runai\_streamer\_sharded": RunAI 流式加载
- "bitsandbytes": BitsAndBytes 量化加载
- "sharded\_state": 预分片 checkpoint
- "gguf": GGUF 格式
- "mistral": Mistral 整合 safetensors

## safetensors 加载策略

| 策略                        | 说明                     |
|---------------------------|------------------------|
| None (默认)                 | 内存映射懒加载; NFS 上自动预取     |
| "lazy"                    | 内存映射, 无 NFS 预取         |
| "eager"                   | 全部读入 CPU 内存 (适合网络文件系统) |
| "prefetch"                | 预取到 OS 页缓存             |
| "torchao": TorchAO 重建张量子类 |                        |

## 2.7 LoRAConfig — LoRA 适配器

源码位置: lora.py:31-124

## 关键字段表

| 字段                  | 类型   | 默认值   | 说明                                             |
|---------------------|------|-------|------------------------------------------------|
| max_lora_rank       | int  | 16    | 最大 LoRA rank<br>(1/8/16/32/64/128/256/320/512) |
| max_loras           | int  | 1     | 单批次最大 LoRA 数                                   |
| fully_sharded_loras | bool | False | 全分片 LoRA (高序列长度时更快)                            |

| 字段                      | 类型                | 默认值                | 说明                       |
|-------------------------|-------------------|--------------------|--------------------------|
| max_cpu_loras           | int \  None       | None (= max_loras) | CPU 内存中最大 LoRA 数         |
| lora_dtype              | torch.dtype/str   | "auto"             | LoRA 数据类型（默认跟随模型）        |
| target_modules          | list[str] \  None | None               | 限制 LoRA 目标模块             |
| default_mm_loras        | dict \  None      | None               | 多模态默认 LoRA 映射            |
| enable_tower_connector  | bool              | False              | 启用视觉编码器/连接器              |
| specialize_active_loras | bool              | False              | 按活跃 LoRA 数量特化 CUDA Graph |

验证规则

1. max\_cpu\_loras >= max\_loras
2. VLLM\_LORA\_ENABLE\_DUAL\_STREAM 仅支持 CUDA 平台
3. fully\_sharded\_loras 与双流不兼容

## 2.8 QuantizationConfig 一量化方案

源码位置: quantization.py:12-125

vLLM 的量化配置分为两部分: 1. 离线量化: 通过 `ModelConfig.quantization` 指定 (如 awq, gptq, fp8) 2. 在线量化: 通过 `OnlineQuantizationConfigArgs` 在运行时对权重进行量化

在线量化方案枚举

```
class OnlineQuantScheme(Enum):
    FP8_PER_TENSOR = "fp8_per_tensor" # FP8 按张量缩放
    FP8_PER_BLOCK = "fp8_per_block" # FP8 按块缩放 (DeepSeek 风格)
    INT8_PER_CHANNEL_WEIGHT_ONLY = "int8_per_channel_weight_only" # INT8 仅权重量化
    MXFP8 = "mxfp8" # MicroScaling FP8
```

### OnlineQuantizationConfigArgs

| 字段                     | 类型                        | 默认值  | 说明            |
|------------------------|---------------------------|------|---------------|
| global_scheme          | OnlineQuantScheme \  None | None | 全局量化方案        |
| linear_scheme_override | OnlineQuantScheme \  None | None | Linear 层覆盖    |
| moe_scheme_override    | OnlineQuantScheme \  None | None | MoE 层覆盖       |
| ignore                 | list[str]                 | []   | 跳过量化的层 (支持正则) |

量化配置解析流程 在 `VllmConfig.__post_init__` 中调用 `_get_quantization_config()`:

```
@staticmethod
def _get_quantization_config(model_config, load_config):
    if model_config.quantization is not None:
        quant_config = get_quant_config(model_config, load_config)
        # 1. 检查 GPU compute capability 最低要求
```

```

# 2. 检查 dtype 兼容性
# 3. 调用 maybe_update_config() 更新 hf_config
return quant_config
return None

```

## 2.9 CompilationConfig —torch.compile 与 CUDA Graphs

源码位置: compilation.py:372-1499

这是第三复杂的配置类，控制模型编译优化的方方面面。

### CompilationMode 枚举

```

class CompilationMode(IntEnum):
    NONE = 0 # 无编译, 纯 eager 模式
    STOCK_TORCH_COMPILE = 1 # 标准 torch.compile
    DYNAMO_TRACE_ONCE = 2 # 单次 Dynamo trace
    VLLM_COMPILE = 3 # vLLM 自定义 Inductor 后端 (推荐)

```

### CUDAGraphMode 枚举

```

class CUDAGraphMode(Enum):
    NONE = 0 # 无 CUDA Graph
    PIECEWISE = 1 # 仅分段 CUDA Graph
    FULL = 2 # 全量 CUDA Graph
    FULL_DECODE_ONLY = (FULL, NONE) # 仅 decode 阶段全量
    FULL_AND_PIECEWISE = (FULL, PIECEWISE) # decode 全量 + prefill 分段 (默认最优)

```

关键字段表

| 字段                              | 类型                      | 默认值             | 说明                                       |
|---------------------------------|-------------------------|-----------------|------------------------------------------|
| mode                            | CompilationMode         | None (→ O2 决定)  | 编译模式                                     |
| backend                         | str                     | "" (→ inductor) | 编译后端                                     |
| custom_ops                      | list[str]               | []              | 自定义算子开关                                  |
| splitting_ops                   | list[str] \ \ None      | None            | (+op/-op/all/none)<br>从 CUDA Graph 排除的算子 |
| cudagraph_mode                  | CUDAGraphMode           | None (→ O2 决定)  | CUDA Graph 模式                            |
| cudagraph_capture_size          | list[int] \ \ None      | None            | CUDA Graph 捕获尺寸                          |
| max_cudagraph_capture_size      | int                     | None            | 最大捕获尺寸                                   |
| cudagraph_num_of_warmups        | int                     | 0               | Warmup 次数                                |
| debug_dump_path                 | Path \ \ None           | None            | 调试信息转储路径                                 |
| cache_dir                       | str                     | ""              | 编译缓存目录                                   |
| compile_sizes                   | list[int\ str] \ \ None | None            | Inductor 编译尺寸                            |
| compile_ranges_endpoints        | list[int] \ \ None      | None            | 编译范围端点                                   |
| inductor_compile_config         | dict                    | {}              | Inductor 额外配置                            |
| inductor_passes                 | dict                    | {}              | 自定义 Inductor pass                        |
| ir_enable_torch_wrap            | bool \ \ None           | None            | IR torch wrap 开关                         |
| use_inductor_graph_partitioning | bool \ \ None           | None            | Inductor 图分区                             |
| pass_config                     | PassConfig              | Factory         | 自定义融合 pass 配置                            |
| dynamic_shapes_config           | DynamicShapesConfig     | Factory         | 动态形状配置                                   |
| fast_moe_cold_start             | bool \ \ None           | None            | MOE 快速冷启动                                |

| 字段                         | 类型   | 默认值   | 说明                 |
|----------------------------|------|-------|--------------------|
| compile_mm_encoder         | bool | False | 编码多模态 encoder      |
| cudagraph_specialize_graph | bool | True  | LoRA 特化 CUDA Graph |

**PassConfig**（自定义融合 **Pass**） 源码: compilation.py:107-305

| 字段                     | 类型           | 默认值  | 说明                          |
|------------------------|--------------|------|-----------------------------|
| fuse_norm_quant        | bool         | None | RMSNorm + 量化融合              |
| fuse_act_quant         | bool         | None | SiLU+Mul + 量化融合             |
| fuse_attn_quant        | bool         | None | Attention + 量化融合            |
| enable_sp              | bool         | None | 序列并行 (Sequence Parallelism) |
| fuse_gemm_comms        | bool         | None | 异步 TP (GEMM + 通信融合)         |
| fuse_allreduce_rms     | bool         | None | AllReduce + RMSNorm 融合      |
| eliminate_noops        | bool         | True | 消除空操作                       |
| fuse_act_padding       | bool         | None | RMSNorm + padding 融合 (ROCm) |
| fuse_mla_dual_rms_norm | bool         | None | MLA 双 RMSNorm 融合            |
| fuse_rope_kvcache      | bool         | None | RoPE + KV cache 融合 (ROCm)   |
| sp_min_token_num       | int \   None | None | SP 最小 token 数阈值             |

注意: 值为 **None** 表示”由 OptimizationLevel 决定”，而非禁用。

## DynamicShapesConfig

| 字段                     | 类型                | 默认值    | 说明                                    |
|------------------------|-------------------|--------|---------------------------------------|
| type                   | DynamicShapesType | BACKED | backed/unbacked/backed_size_oblivious |
| evaluate_guards        | bool              | False  | 调试动态形状 guard                          |
| assume_32_bit_indexing | bool              | False  | 假设 32 位索引                             |

## 2.10 SpeculativeConfig —推测解码

源码位置: speculative.py:73-1063

推测解码 (Speculative Decoding / Spec Decode) 通过小型草稿模型 (draft model) 提前预测多个 token，再由目标模型 (target model) 并行验证，从而提升吞吐。

推测方法类型

```
SpeculativeMethod = Literal[
    "ngram",           # N-gram 模式匹配
    "medusa",          # Medusa head
    "mlp_speculator",  # MLP Speculator
    "draft_model",     # 独立草稿模型
```

```

    "suffix",                # Suffix decoding (Arctic)
    "eagle", "eagle3",      # EAGLE/EAGLE3
    "dflash",               # DFlash
    MTPModelTypes,         # Multi-Token Prediction (20+ 变体)
    NgramGPUPTypes,        # Ngram GPU
]

```

关键字段表

| 字段                             | 类型                           | 默认值         | 说明                 |
|--------------------------------|------------------------------|-------------|--------------------|
| num_speculative_tokens         | int                          | None (必需)   | 推测 token 数         |
| model                          | str \  None                  | None        | 草稿模型路径             |
| method                         | SpeculativeMethod<br>\  None | None (自动检测) | 推测方法               |
| draft_tensor_parallel_size     | int \  None                  | None        | 草稿模型 TP            |
| quantization                   | str \  None                  | None        | 草稿模型量化             |
| max_model_len                  | int \  None                  | None        | 草稿模型最大长度           |
| disable_padded_draft_batch     | bool                         | False       | 禁用填充草稿批次           |
| use_local_argmax_reductions    | bool                         | False       | 局部 argmax 减少通信     |
| prompt_lookup_max/min          | int \  None                  | None        | N-gram 窗口大小        |
| parallel_drafting              | bool                         | False       | 并行 drafting        |
| rejection_sample_method        | str                          | "standard"  | standard/synthetic |
| draft_sample_method            | str                          | "greedy"    | greedy/gumbel      |
| suffix_decoding_max_tree_depth | int                          | 24          | Suffix 解码树深度       |

**MTP (Multi-Token Prediction)** 支持的模型 vLLM 支持 20+ 种 MTP 模型变体, 包括: - **DeepSeek** 系列: deepseek\_mtp, deepseek\_v32 (→ mtp), deepseek\_v4 (→ mtp) - **Qwen** 系列: qwen3\_next\_mtp, qwen3\_5\_mtp - **GLM** 系列: glm4\_moe\_mtp, glm4\_moe\_lite\_mtp, glm\_ocr\_mtp - 其他: ernie\_mtp, nemotron\_h\_mtp, gemma4\_mtp, hy\_v3\_mtp, step3p5\_mtp 等

完整列表见: speculative.py:34-53

## 2.11 ObservabilityConfig —可观测性

源码位置: observability.py:17-152

关键字段表

| 字段                               | 类型                | 默认值   | 说明                            |
|----------------------------------|-------------------|-------|-------------------------------|
| show_hidden_metrics_for_versions | str \  None       | None  | 显示某版本起隐藏的指标                   |
| otlp_traces_endpoint             | str \  None       | None  | OpenTelemetry traces 目标 URL   |
| collect_detailed_traces          | dict[str] \  None | None  | 详细追踪模块 (model/worker/all)     |
| kv_cache_metrics                 | bool              | False | KV 缓存驻留指标                     |
| kv_cache_metrics_sample_rate     | float             | 0.01  | KV 指标采样率 (0-1]                |
| cuda_graph_metrics               | bool              | False | CUDA Graph 指标                 |
| enable_layerwise_nvtx_profiling  | bool              | False | 逐层 NVTX 追踪                    |
| enable_mfu_metrics               | bool              | False | MFU (Model FLOPs Utilization) |
| enable_mm_processor_stats        | bool              | False | 多模态处理器统计                      |

| 字段                               | 类型   | 默认值   | 说明     |
|----------------------------------|------|-------|--------|
| enable_logging_iteration_details | bool | False | 迭代详情日志 |

2.12 OffloadConfig / ReasoningConfig / StructuredConfigs 等

OffloadConfig 一权重卸载 源码: offload.py:80-138

| 字段                             | 类型       | 默认值    | 说明                |
|--------------------------------|----------|--------|-------------------|
| offload_backend                | str      | "auto" | auto/uva/prefetch |
| uva.cpu_offload_gb             | float    | 0      | UVA 卸载量 (GiB/GPU) |
| uva.cpu_offload_params         | set[str] | set()  | UVA 目标参数名         |
| prefetch.offload_group_size    | int      | 0      | 每 N 层一组           |
| prefetch.offload_num_in_group  | int      | 1      | 每组卸载数             |
| prefetch.offload_prefetch_step | int      | 1      | 预取步进              |

ReasoningConfig 一推理链 (Chain-of-Thought) 源码: reasoning.py:13-107

| 字段                  | 类型  | 默认值 | 说明                                 |
|---------------------|-----|-----|------------------------------------|
| reasoning_parser    | str | ""  | ReasoningParser 名称                 |
| reasoning_start_str | str | ""  | 推理开始标记 (如 ")   """)   推理结束标记 (如" ) |

Token IDs 通过 initialize\_token\_ids() 从字符串自动推导。

StructuredOutputsConfig 一结构化输出 源码: structured\_outputs.py:18-74

| 字段                            | 类型   | 默认值    | 说明                                                 |
|-------------------------------|------|--------|----------------------------------------------------|
| backend                       | str  | "auto" | auto/xgrammar/guidance/outlines/lm-format-enforcer |
| disable_any_whitespace        | bool | False  | 紧凑 JSON 输出                                         |
| disable_additional_properties | bool | False  | 禁用 additionalProperties                            |
| reasoning_parser              | str  | ""     | 推理解析器                                              |
| enable_in_reasoning           | bool | False  | 在推理中使用结构化输入                                        |

其他配置类简表

| 配置类              | 文件             | 核心用途                                      |
|------------------|----------------|-------------------------------------------|
| AttentionConfig  | attention.py   | Attention 后端选择 (FLASH_ATTN/XFORMERS etc.) |
| MambaConfig      | mamba.py       | Mamba SSM 缓存与状态管理                         |
| KernelConfig     | kernel.py      | 自定义算子配置 (FlashInfer autotune 等)           |
| ProfilerConfig   | profiler.py    | 性能分析器配置                                   |
| KVTransferConfig | kv_transfer.py | 分布式 KV 缓存传输                               |
| KVEventsConfig   | kv_events.py   | KV 事件发布                                   |

| 配置类                  | 文件                 | 核心用途           |
|----------------------|--------------------|----------------|
| ECTransferConfig     | ec_transfer.py     | EC 缓存传输        |
| PoolerConfig         | pooler.py          | Pooling 模型输出配置 |
| MultiModalConfig     | multimodal.py      | 多模态输入配置        |
| WeightTransferConfig | weight_transfer.py | RL 训练权重传输      |

### 三、配置验证与合并逻辑

#### 3.1 model\_validator 方法

VllmConfig 定义了两个 @model\_validator(mode="after") 方法:

validate\_nvfp4\_kv\_cache\_with\_mla 源码: vllm.py:1995-2004

```
@model_validator(mode="after")
def validate_nvfp4_kv_cache_with_mla(self) -> "VllmConfig":
    if self.model_config is None:
        return self
    if self.cache_config.cache_dtype == "nvfp4" and self.model_config.use_mla:
        raise ValueError(
            "nvfp4 KV cache is not supported with MLA "
            "(Multi-head Latent Attention) backends."
        )
    return self
```

约束: NVFP4 KV 缓存与 MLA (DeepSeek V3 系列的多头潜在注意力) 不兼容。

validate\_mamba\_block\_size 源码: vllm.py:2007-2018

```
@model_validator(mode="after")
def validate_mamba_block_size(self) -> "VllmConfig":
    mamba_block_size_is_set = (
        self.cache_config.mamba_block_size is not None
        and self.cache_config.mamba_block_size != self.model_config.max_model_len
    )
    if mamba_block_size_is_set and not self.cache_config.enable_prefix_caching:
        raise ValueError(
            "--mamba-block-size can only be set with --enable-prefix-caching"
        )
    return self
```

约束: Mamba block\_size 仅在启用前缀缓存时有意义。

#### 3.2 device 配置如何影响其他配置

DeviceConfig 在 \_\_post\_init\_\_ 中通过 current\_platform 自动检测设备类型, 这个信息会传播影响到:

1. ParallelConfig:
  - 非 CUDA 设备自动禁用 custom all-reduce
  - TPU 且 VLLM\_XLA\_USE\_SPMD 时使用 uni 后端
  - 多节点时强制 mp 后端
2. CompilationConfig:
  - ROCm 平台启用 AITER 特定融合 pass
  - 非 CUDA-like 平台禁用 CUDA Graph

- Intel GPU 使用 `fp8_inc` 作为默认 KV cache dtype
3. **ModelConfig:**
    - `sleep_mode` 仅在 cuda/hip 平台可用
    - encoder-decoder + ROCm 自动 enforce eager
  4. **CacheConfig:**
    - 不同平台的默认 `block_size` 可能不同
    - KV cache dtype 的可用选项因平台而异

平台检测入口: `vllm.py:983`

```
current_platform.apply_config_platform_defaults(self)
```

## 四、环境变量覆盖机制

源码位置: `env_override.py` (全文件, 约 760 行)

`env_override.py` 在 `import vllm` 时立即执行, 主要功能包括:

### 4.1 CUDA 兼容性路径设置

源码: `env_override.py:39-85`

通过 `VLLM_ENABLE_CUDA_COMPATIBILITY` 环境变量设置 `LD_LIBRARY_PATH`, 使旧版 CUDA 驱动能运行新版 PyTorch 编译的程序。查找优先级:

1. `VLLM_CUDA_COMPATIBILITY_PATH` 显式指定
2. `$CONDA_PREFIX/cuda-compat`
3. `/usr/local/cuda-{version}/compat`

### 4.2 PyTorch 环境变量预设

源码: `env_override.py:101-113`

```
os.environ["PYTORCH_NVML_BASED_CUDA_CHECK"] = "1" # 避免 CUDA 初始化
os.environ["TORCHINDUCTOR_COMPILE_THREADS"] = "1" # 单线程编译
os.environ.setdefault("TRITON_CACHE_AUTOTUNING", "1") # Triton 自动调优缓存
os.environ.setdefault("TILELANG_CLEANUP_TEMP_FILES", "1") # TileLang 清理
```

### 4.3 PyTorch 版本特定 Monkey-Patch

`env_override.py` 包含大量针对特定 PyTorch 版本的 monkey-patch, 修复上游 bug:

| 版本          | Patch 内容                                                              | 上游 Issue/PR            |
|-------------|-----------------------------------------------------------------------|------------------------|
| 2.9.0       | <code>memory_plan_reuse</code> 修复                                     | pytorch/pytorch#165514 |
| 2.9.0       | <code>get_graph_partition_signature</code> 修复                         | pytorch/pytorch#165815 |
| 2.9.0       | <code>should_partition</code> 修复 (origin_node 访问)                     | vLLM #26678            |
| 2.9.0+      | <code>get_raw_stream</code> 缺失 workaround                             | vLLM #30905            |
| 2.10.0      | <code>FxGraphCachePickler.dumps</code> <code>ValueError</code> 修复     | pytorch/pytorch#176557 |
| 2.11-2.11.x | <code>constrain_to_fx_strides</code> <code>FakeScriptObject</code> 修复 | pytorch/pytorch#175973 |
| 2.11-2.11.x | <code>indirect_assert</code> <code>VecMask</code> 构造修复                | pytorch/pytorch#178148 |



这些 patch 通过 `is_torch_equal_or_newer()` 进行版本守卫，只在匹配的 PyTorch 版本上生效。

## 五、OptimizationLevel (O0-O3) 各级别详解

源码位置: `vllm.py:68-265`

OptimizationLevel 是 vLLM 的一键优化旋钮，在启动时间和推理性能之间提供 4 个级别的权衡。每个级别对应一组预定义的 `compilation_config` 和 `kernel_config` 默认值。

### 5.1 OptimizationLevel 枚举定义

```
class OptimizationLevel(IntEnum):
    O0 = 0 # 无优化, 最快启动
    O1 = 1 # 快速优化 (Dynamo + Piecewise CG)
    O2 = 2 # 完整优化 (O1 + Full + Piecewise CG) ← 默认
    O3 = 3 # 目前等同于 O2 + FlashInfer autotune
```

### 5.2 各级别详细配置

**O0** 一无优化 (极速启动) 适用场景: 开发调试、快速实验、首次验证

```
OPTIMIZATION_LEVEL_O0 = {
    "compilation_config": {
        "pass_config": {
            "fuse_norm_quant": False, # 不融合 Norm+Quant
            "fuse_act_quant": False, # 不融合 Act+Quant
            "fuse_allreduce_rms": False, # 不融合 AR+RMS
            "fuse_attn_quant": False, # 不融合 Attn+Quant
            "enable_sp": False, # 不启用序列并行
            "fuse_gemm_comms": False, # 不融合 GEMM+Comms
            "fuse_act_padding": False, # 不融合 Act+Padding
            "fuse_mla_dual_rms_norm": False, # 不融合 MLA Dual-RMS
            "fuse_rope_kvcache": False, # 不融合 RoPE+KVC
        },
        "cudagraph_mode": CUDAGraphMode.NONE, # 无 CUDA Graph
        "use_inductor_graph_partition": False, # 无图分区
    },
    "kernel_config": {
        "enable_flashinfer_autotune": False, # 无 FlashInfer autotune
    },
}
```

特征总结: - 最快启动速度 (无需编译预热) - 最小内存占用 (无 CUDA Graph 缓冲区) - 无任何 kernel fusion - 无 CUDA Graph 加速 - 吞吐量最低

**O1** 一快速优化 适用场景: 对延迟敏感但需要基本性能的场景

```
OPTIMIZATION_LEVEL_O1 = {
    "compilation_config": {
        "pass_config": {
            "fuse_norm_quant": enable_norm_fusion, # 条件启用
            "fuse_act_quant": enable_act_fusion, # 条件启用
            "fuse_allreduce_rms": False, #
            "fuse_attn_quant": False, #
            "enable_sp": False, #
        },
    },
}
```

```

        "fuse_gemm_comms": False, #
        "fuse_act_padding": enable_norm_pad_fusion, # 条件启用
        "fuse_mla_dual_rms_norm": enable_mla_dual_rms_norm_fusion, # 条件启用
        "fuse_rope_kvcache": False, #
    },
    "cudagraph_mode": CUDAGraphMode.PIECEWISE, # 分段 CUDA Graph
    "use_inductor_graph_partition": False, #
},
"kernel_config": {
    "enable_flashinfer_autotune": False, #
},
}

```

条件启用的 **fusion** 函数（根据实际配置动态决定）：

| Fusion 函数                       | 启用条件                                         |
|---------------------------------|----------------------------------------------|
| enable_norm_fusion              | RMSNorm 或 FP8 quant 自定义 op 启用时               |
| enable_act_fusion               | SiLU+Mul 或 FP8 quant 自定义 op 启用时；或 FP4 模型     |
| enable_norm_pad_fusion          | AITER RMSNorm + hidden_size==288 (gpt-oss) 时 |
| enable_mla_dual_rms_norm_fusion | AITER 有 fused_qk_rmsnorm 时                   |

特征总结: - 启用 `torch.compile` (VLLM\_COMPILE mode) - 分段 CUDA Graph (Piecewise) - prefill 灵活, decode 加速 - 基本 kernel fusion (Norm+Quant, Act+Quant) - 无 Full CUDA Graph - 无 Sequence Parallelism - 无 FlashInfer autotune

**O2** 一完整优化（默认级别） 适用场景: 生产部署, 平衡启动时间和吞吐量

```

OPTIMIZATION_LEVEL_02 = {
    "compilation_config": {
        "pass_config": {
            "fuse_norm_quant": enable_norm_fusion, #
            "fuse_act_quant": enable_act_fusion, #
            "fuse_allreduce_rms": enable_allreduce_rms_fusion, # 新增!
            "fuse_attn_quant": IS_QUANTIZED, # 量化模型
            "enable_sp": IS_DENSE, # 密集模型
            "fuse_gemm_comms": IS_DENSE, # 密集模型
            "fuse_act_padding": enable_norm_pad_fusion, #
            "fuse_mla_dual_rms_norm": ..., #
            "fuse_rope_kvcache": enable_rope_kvcache_fusion, # 新增!
        },
        "cudagraph_mode": CUDAGraphMode.FULL_AND_PIECEWISE, # 全量 + 分段
        "use_inductor_graph_partition": False,
    },
    "kernel_config": {
        "enable_flashinfer_autotune": False, # (留待 O3)
    },
}

```

相比 **O1** 新增的优化: - `fuse_allreduce_rms`: AllReduce + RMSNorm 融合 (Hopper+ + TP>1 + FlashInfer) - `fuse_attn_quant`: Attention + 量化融合 (量化模型) - `enable_sp`: 序列并行 (密集模型 + TP>1) - `fuse_gemm_comms`: 异步 TP (GEMM + NCCL 通信融合) - `fuse_rope_kvcache`: RoPE + KV cache 更新

融合 (ROCm/AITER) - **Full + Piecewise CUDA Graph**: decode 阶段全量 capture, prefill/mixed 阶段分段

特征总结: - 所有 O1 的优化 - Full CUDA Graph for decode (最大程度减少 decode 阶段开销) - Sequence Parallelism (减少 TP 通信瓶颈) - Async TP (隐藏 AllReduce 延迟) - 适用于大多数生产工作负载 - 启动时间较长 (需要 capture 更多尺寸的 CUDA Graph)

**O3** - 激进优化 适用场景: 对吞吐量极其敏感的离线推理、benchmark

```
OPTIMIZATION_LEVEL_O3 = {
    "compilation_config": {
        # 与 O2 完全相同的 pass_config ...
        "cudagraph_mode": CUDAGraphMode.FULL_AND_PIECEWISE,
        "use_inductor_graph_partition": False,
    },
    "kernel_config": {
        "enable_flashinfer_autotune": True,          # 唯一区别!
    },
}
```

相比 **O2** 唯一新增: - **FlashInfer Autotune**: 自动搜索最优的 FlashInfer kernel 配置 (如 tile 大小)

特征总结: - 所有 O2 的优化 - FlashInfer autotune (可能显著提升 attention 性能) - 最长启动时间 (autotune 需要额外 benchmark 时间) - autotune 过程中可能有正确性问题 (FlashInfer #3197)

### 5.3 OptimizationLevel 应用机制

源码: vllm.py:1012-1013, vllm.py:647-674

# 在 `__post_init__` 中应用

```
default_config = OPTIMIZATION_LEVEL_TO_CONFIG[self.optimization_level]
self._apply_optimization_level_defaults(default_config)
```

应用策略 (`_apply_optimization_level_defaults`):

1. 递归遍历配置字典, 匹配嵌套的 dataclass 字段
2. 仅覆盖仍为 `None` 的字段 (用户显式设置的值优先级更高)
3. 支持 `callable` 值: 若默认值是函数, 则以 `self` 为 root 调用以获取运行时决策

```
def _set_config_default(self, config_obj, key, value):
    if getattr(config_obj, key) is None:
        setattr(config_obj, key, value(self) if callable(value) else value)
```

这意味着像 `enable_sp: IS_DENSE` 这样的配置会在运行时根据模型是否为 dense 模型来决定是否启用。

### 5.4 Performance Mode 交互

除了 OptimizationLevel, vLLM 还提供 `performance_mode` 参数 (vllm.py:346-351):

```
performance_mode: PerformanceMode = "balanced"
# "balanced"          # 平衡模式 (默认)
# "interactivity"     # 交互优先 (细粒度 CG, 低延迟 kernel)
# "throughput"        # 吞吐优先 (大 CG, 激进 batching)
```

**Performance Mode** 影响 (vllm.py:1561-1587):

当 `performance_mode == "interactivity"` 时: - CUDA Graph capture 尺寸变为 `[1, 2, ..., interactivity_max]` (更细粒度, 减少 padding 开销) - `interactivity_max = min(max_cudagraph_capture_size, 32)`

当 `performance_mode == "throughput"` 时: - 使用标准尺寸表 [1, 2, 4, 8, 16, ..., 256, ...] (更大步长, 减少 capture 数量)

## 5.5 各级别对比总览

| 特性                   | O0      | O1        | O2 (默认)    | O3            |
|----------------------|---------|-----------|------------|---------------|
| torch.compile        |         | VLLM      | VLLM       | VLLM          |
| CUDA Graph           |         | Piecewise | Full+Piece | Full+Piece    |
| Norm+Quant Fusion    |         | 条件        |            |               |
| Act+Quant Fusion     |         | 条件        |            |               |
| AllReduce+RMS        |         |           | (Hopper+)  |               |
| Fusion               |         |           |            |               |
| Attn+Quant Fusion    |         |           | 量化时        | 量化时           |
| Sequence Parallelism |         |           | Dense      | Dense         |
| Async TP             |         |           | Dense      | Dense         |
| (GEMM+Comms)         |         |           |            |               |
| RoPE+KVCache         |         |           | (ROCm)     | (ROCm)        |
| Fusion               |         |           |            |               |
| FlashInfer Autotune  |         |           |            |               |
| 启动速度                 | fastest | fast      | slow       | slowest       |
| 吞吐潜力                 | lowest  | low       | high       | highest       |
| 延迟表现                 | good    | good      | best       | best          |
| 内存占用                 | least   | low       | medium     | medium        |
| 适用场景                 | 开发/调试   | 低延迟服务     | 生产部署 (推荐)  | 高吞吐 benchmark |

## 附录: 配置类汇总速查表

| 配置类                        | 文件               | 核心职责    | 关键决策                                            |
|----------------------------|------------------|---------|-------------------------------------------------|
| <b>VllmConfig</b>          | vllm.py          | 顶层聚合容器  | Hash 计算、跨域验证、优化级别应用                             |
| <b>ModelConfig</b>         | model.py         | 模型定义    | dtype、max_len、架构推断、tokenizer                    |
| <b>CacheConfig</b>         | cache.py         | KV 缓存   | block_size、dtype、prefix caching、offloading      |
| <b>ParallelConfig</b>      | parallel.py      | 并行拓扑    | TP/PP/DP/DCP/EP、world_size、All2All              |
| <b>SchedulerConfig</b>     | scheduler.py     | 请求调度    | batch tokens、chunked prefill、policy             |
| <b>DeviceConfig</b>        | device.py        | 设备抽象    | 自动检测 cuda/cpu/tpu/xpu                           |
| <b>LoadConfig</b>          | load.py          | 权重加载    | format、strategy、download_dir                    |
| <b>LoRAConfig</b>          | lora.py          | LoRA 适配 | rank、数量、dtype、目标模块                              |
| <b>QuantizationConfig</b>  | quantization.py  | 量化      | 在线/离线方案、per-tensor/per-block                    |
| <b>CompilationConfig</b>   | compilation.py   | 编译优化    | mode/backend/cudagraph/pass/fusion              |
| <b>SpeculativeConfig</b>   | speculative.py   | 推测解码    | method/draft                                    |
| <b>ObservabilityConfig</b> | observability.py | 可观测性    | model/MTP/EAGLE/Ngram                           |
| <b>OffloadConfig</b>       | offload.py       | 权重卸载    | traces/metrics/NVTX/MFU                         |
| <b>ReasoningConfig</b>     | reasoning.py     | 推理链     | UVA zero-copy / prefetch<br>CoT start/end token |

| 配置类                     | 文件                    | 核心职责  | 关键决策                        |
|-------------------------|-----------------------|-------|-----------------------------|
| StructuredOutputsConfig | structured_outputs.py | 结构化输出 | JSON schema / regex backend |

文档生成时间: 2026-05-10 / 基于 vLLM 最新源码

## vLLM 引擎核心 (Engine Core) 深度分析

定位: 本文档深入分析 vLLM V1 架构中引擎层 (Engine Layer) 的核心实现, 涵盖从请求输入到模型推理再到输出返回的完整数据流管线。引擎层是 vLLM 的中枢神经, 负责协调调度器 (Scheduler)、执行器 (Executor)、输入预处理器 (InputProcessor) 和输出后处理器 (OutputProcessor) 等关键组件。

```
graph TB
    subgraph EngineCore ["EngineCore - 执行循环核心"]
        direction TB
        IC["InputProcessor<br/> 请求预处理"]
        SCH["Scheduler<br/> 调度策略"]
        WK["Worker/Executor<br/> 模型执行"]
        OP["OutputProcessor<br/> 输出后处理"]
        DT["Detokenizer<br/> 反 Tokenization"]
    end

    subgraph Clients ["引擎客户端"]
        LE["LLMEngine<br/> 同步引擎"]
        AL["AsyncLLM<br/> 异步引擎"]
    end

    subgraph Aux ["辅助组件"]
        COORD["DPCoordinator<br/> 多节点协调"]
        EX["Exceptions<br/> 异常体系"]
    end

    LE -->| 同步调用 | EC_Client[EngineCoreClient]
    AL -->| 异步 ZMQ | EC_Client
    EC_Client --> EngineCore
    IC -->|"EngineCoreRequest"| SCH
    SCH -->|"SchedulerOutput"| WK
    WK -->|"ModelRunnerOutput"| SCH
    SCH -->|"EngineCoreOutputs"| OP
    OP --> DT
    OP -->|"RequestOutput"| LE
    OP -->|"RequestOutputCollector"| AL
    COORD -->|Wave 协调 | EngineCore
    EX -. ->| 错误传播 | LE
    EX -. ->| 错误传播 | AL

    style EngineCore fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
    style IC fill:#e3f2fd,stroke:#2196f3,stroke-width:1px
    style SCH fill:#fff3e0,stroke:#ff9800,stroke-width:1px
    style WK fill:#fce4ec,stroke:#e91e63,stroke-width:1px
    style OP fill:#f3e5f5,stroke:#9c27b0,stroke-width:1px
```

```
style DT fill:#e0f2f1,stroke:#009688,stroke-width:1px
style LE fill:#e8f5e9,stroke:#4caf50,stroke-width:1px
style AL fill:#e8f5e9,stroke:#4caf50,stroke-width:1px
```

---

## 目录

- 一、LLMEngine 类
    - 1.1 初始化流程
    - 1.2 核心方法分析
    - 1.3 与 EngineCoreClient 的交互
  - 二、AsyncLLM 异步引擎
    - 2.1 异步架构设计
    - 2.2 高级 API 封装
    - 2.3 Output Handler 后台循环
  - 三、EngineCore 解耦执行循环
    - 3.1 EngineCore 基类
    - 3.2 EngineCoreProc 进程封装
    - 3.3 核心调度-执行循环
    - 3.4 数据流详解
  - 四、InputProcessor 请求预处理
  - 五、OutputProcessor 输出后处理
  - 六、Detokenizer 异步反 Tokenization
  - 七、Coordinator 多节点协调
  - 八、异常处理机制
  - 附录：处理管线完整数据流图
- 

## 一、LLMEngine 类

源码文件: llm\_engine.py

LLMEngine 是 vLLM V1 向后兼容的同步引擎入口, 对外暴露与 V0 兼容的 API 接口, 内部通过 EngineCoreClient 与运行在独立进程中的 EngineCore 通信。

### 1.1 初始化流程

初始化遵循 配置解析 → Executor 选择 → Worker 创建 → 组件装配 的链路:

```
# llm_engine.py L50-132
```

```
class LLMEngine:
    """Legacy LLMEngine for backwards compatibility."""

    def __init__(self, vllm_config, executor_class, log_stats, ...):
        # 配置解析
        self.vllm_config = vllm_config
        self.model_config = vllm_config.model_config

        # 初始化 tracing (可选)
        tracing_endpoint = self.observability_config.otlp_traces_endpoint
        if tracing_endpoint is not None:
            init_tracer("vllm.llm_engine", tracing_endpoint)

        # Data Parallel 组初始化 (非 multiprocessing 模式)
        if (not multiprocessing_mode and parallel_config.data_parallel_size > 1
```

```

        and not self.external_launcher_dp):
            self.dp_group = parallel_config.stateless_init_dp_group()

# 创建 Renderer (负责 tokenization 和 multimodal 处理)
self.renderer = renderer_from_config(self.vllm_config)

# 创建 InputProcessor ——将 EngineInput 转换为 EngineCoreRequest
self.input_processor = InputProcessor(self.vllm_config, renderer) # L93

# 创建 OutputProcessor ——将 EngineCoreOutputs 转换为 RequestOutput
self.output_processor = OutputProcessor( # L96
    renderer.tokenizer,
    log_stats=self.log_stats,
    stream_interval=...,
    tracing_enabled=...,
)

# 创建 EngineCoreClient ——与后台 EngineCore 进程通信
self.engine_core = EngineCoreClient.make_client( # L104
    multiprocess_mode=multiprocess_mode,
    asyncio_mode=False,
    vllm_config=vllm_config,
    executor_class=executor_class,
    log_stats=self.log_stats,
)

# 统计日志管理器 (可选)
if self.log_stats:
    self.logger_manager = StatLoggerManager(...)

```

初始化组件关系图:

| 组件                | 职责                                | 对应源码位置 |
|-------------------|-----------------------------------|--------|
| Renderer          | Tokenization + Multimodal 输入渲染    | L90    |
| InputProcessor    | 原始输入 → EngineCoreRequest          | L93    |
| OutputProcessor   | EngineCoreOutputs → RequestOutput | L96    |
| EngineCoreClient  | 与后台 EngineCore 进程的 IPC 通信         | L104   |
| StatLoggerManager | 性能指标收集与日志                         | L114   |

## 1.2 核心方法分析

**1.2.1 add\_request()** —添加推理请求 这是用户提交推理请求的入口点。完整流程包含输入验证 → 预处理 → 请求分发:

```

# llm_engine.py L209-285
def add_request(self, request_id, prompt, params, arrival_time=None, ...):
    # 验证 request_id 类型
    if not isinstance(request_id, str):
        raise TypeError(f"request_id must be a string...")

```

```

# 如果传入的是已处理的 EngineCoreRequest (已废弃方式)
if isinstance(prompt, EngineCoreRequest):
    request = prompt # L233
else:
    # 通过 InputProcessor 将原始输入转换为 EngineCoreRequest
    request = self.input_processor.process_inputs( # L241
        request_id, prompt, params,
        supported_tasks=self.get_supported_tasks(),
        arrival_time=arrival_time,
        lora_request=lora_request,
        ...
    )

# 分配内部 request_id (追加随机后缀保证唯一性)
self.input_processor.assign_request_id(request) # L254

# 获取采样参数
params = request.params
n = params.n if isinstance(params, SamplingParams) else 1

if n == 1:
    # 单采样: 直接添加到 OutputProcessor 和 EngineCore
    self.output_processor.add_request(request, ...) # L265
    self.engine_core.add_request(request) # L267
    return req_id

# n>1 并行采样: Fan-out 为多个子请求
parent_req = ParentRequest(request) # L271
for idx in range(n):
    request_id, child_params = parent_req.get_child_info(idx)
    child_request = request if idx == n - 1 else copy(request)
    child_request.request_id = request_id
    child_request.sampling_params = child_params
    self.output_processor.add_request(child_request, ...)
    self.engine_core.add_request(child_request) # L283
return req_id

```

关键设计要点: -  $n>1$  并行采样通过 ParentRequest 实现 Fan-out 模式, 每个子请求拥有独立的 sampling seed - 内部 request\_id 通过 assign\_request\_id() 追加 8 位随机 UUID, 避免外部 ID 冲突

**1.2.2 step()** — 执行一步推理 step() 是引擎的核心调度循环方法, 每次调用完成一次完整的 schedule → execute → output 流水线:

```

# llm_engine.py L287-325
def step(self) -> list[RequestOutput | PoolingRequestOutput]:
    # Dummy batch 处理 (DP 场景下某些 rank 可能需要空转)
    if self.should_execute_dummy_batch:
        self.should_execute_dummy_batch = False
        self.engine_core.execute_dummy_batch()
        return []

    # 从 EngineCore 获取输出
    with record_function_or_nullcontext("llm_engine step: get_output"):
        outputs = self.engine_core.get_output() # L295

```



```

# 通过 OutputProcessor 处理输出 (detokenize + logprobs + 构建 RequestOutput)
with record_function_or_nullcontext("llm_engine step: process_outputs"):
    iteration_stats = IterationStats() if self.log_stats else None
    processed_outputs = self.output_processor.process_outputs( # L300
        outputs.outputs,
        engine_core_timestamp=outputs.timestamp,
        iteration_stats=iteration_stats,
    )
    self.output_processor.update_scheduler_stats(outputs.scheduler_stats)

# 中止因 stop string 匹配而完成的请求
with record_function_or_nullcontext("llm_engine step: abort_requests"):
    self.engine_core.abort_requests(processed_outputs.reqs_to_abort) # L309

# 记录统计信息
with record_function_or_nullcontext("llm_engine step: record_stats"):
    if self.logger_manager is not None and ...:
        self.logger_manager.record(...)

return processed_outputs.request_outputs # L325

```

### 1.2.3 abort\_request() —中止请求

```

# llm_engine.py L203-207
def abort_request(self, request_ids: list[str], internal: bool = False) -> None:
    """Remove request_ids from EngineCore and Detokenizer."""
    # 先在 OutputProcessor 中清理状态并生成 abort 输出
    request_ids = self.output_processor.abort_requests(request_ids, internal)
    # 再通知 EngineCore 中止对应请求
    self.engine_core.abort_requests(request_ids)

```

中止操作是双轨制的：同时在 OutputProcessor（前端状态机）和 EngineCore（后端调度器）中清理。

### 1.3 与 EngineCoreClient 的交互

LLMEngine 不直接操作 EngineCore，而是通过 EngineCoreClient 抽象层进行通信。根据运行模式不同，EngineCoreClient 有三种实现：

| Client 类型      | 使用场景           | 通信方式          |
|----------------|----------------|---------------|
| InprocClient   | 进程内调试          | 直接函数调用        |
| SyncMPCClient  | LLMEngine 同步模式 | ZMQ + 后台进程    |
| AsyncMPCClient | AsyncLLM 异步模式  | ZMQ + asyncio |

```

# core_client.py L69-78 (节选)
class EngineCoreClient(ABC):
    """
    Subclasses:
    * InprocClient: In process EngineCore (for VO-style LLMEngine use)
    * SyncMPCClient: ZMQ + background proc EngineCore (for LLM)
    * AsyncMPCClient: ZMQ + background proc EngineCore w/ asyncio (for AsyncLLM)
    """

```

## 二、AsyncLLM 异步引擎

源码文件: `async_llm.py`

AsyncLLM 是 vLLM V1 的异步引擎实现, 实现了 `EngineClient` 协议, 专为高并发 API 服务场景设计。

### 2.1 异步架构设计

```
graph LR
    subgraph Frontend["API Server (asyncio event loop)"]
        G["generate() / encode()"]
        AQ["RequestOutputCollector Queue"]
    end

    subgraph Backend["Background Tasks"]
        OH["output_handler()<br/>(asyncio.Task)"]
    end

    subgraph Process["EngineCore Process"]
        EC["EngineCore.busy_loop()"]
    end

    G -->|"add_request()"| IPC_IN[ZMQ Input Socket]
    IPC_IN --> EC
    EC -->|"EngineCoreOutputs"| IPC_OUT[ZMQ Output Socket]
    IPC_OUT --> OH
    OH -->|"put()"| AQ
    AQ -->|"get() / get_nowait()"| G

    style G fill:#e8f5e9
    style OH fill:#fff3e0
    style EC fill:#fce4ec
```

初始化核心差异 (对比 `LLMEngine`):

```
# async_llm.py L73-153 (节选)
class AsyncLLM(EngineClient):
    def __init__(self, vllm_config, executor_class, log_stats, ...):
        # ... (配置初始化与 LLMEngine 类似)

        # 关键差异: 使用 make_async_mp_client 创建异步客户端
        self.engine_core = EngineCoreClient.make_async_mp_client( # L146
            vllm_config=vllm_config,
            executor_class=executor_class,
            log_stats=self.log_stats,
            client_addresses=client_addresses,
            client_count=client_count,
            client_index=client_index,
        )

        # output_handler: 后台 asyncio task, 持续从 EngineCore 拉取输出
        self.output_handler: asyncio.Task | None = None
        try:
            asyncio.get_running_loop()
            self._run_output_handler() # L174: 立即启动
        except RuntimeError:
```

pass # event loop 未就绪时延迟启动

## 2.2 高级 API 封装

**2.2.1 generate() 一流式生成 API** generate() 是 AsyncLLM 最核心的方法，封装了请求提交 → 流式输出 → 异常处理的完整生命周期：

```
# async_llm.py L524-636
async def generate(self, prompt, sampling_params, request_id, *,
                   lora_request=None, ...) -> AsyncGenerator[RequestOutput, None]:
    q: RequestOutputCollector | None = None
    try:
        # 提交请求（内部调用 add_request）
        q = await self.add_request(request_id, prompt, sampling_params, ...) # L559

        # 从队列中流式拉取输出
        finished = False
        while not finished:
            # 优先非阻塞读取（避免不必要的 task switch）
            out = q.get_nowait() or await q.get() # L579

            assert isinstance(out, RequestOutput)
            finished = out.finished
            if out is not STREAM_FINISHED:
                yield out # L586: yield 给调用方

    except (asyncio.CancelledError, GeneratorExit): # L591
        # 客户端断开连接 → 自动 abort
        if q is not None:
            await self.abort(q.request_id, internal=True)
        raise

    except EngineDeadError: # L599
        # EngineCore 死亡 → 不可恢复
        raise

    except InputStreamError as e: # L611
        # 输入流异常 → 直接传播原始原因
        raise e.cause from e

    except Exception as e: # L619
        # 其他异常 → 包装为 EngineGenerateError（可恢复）
        raise EngineGenerateError() from e
    finally:
        if q is not None:
            q.close() # L635: 清理资源
```

异常处理层次：

| 异常类型                | 含义              | 可恢复性 | 处理方式     |
|---------------------|-----------------|------|----------|
| CancelledError      | 客户端断开           | -    | abort 请求 |
| EngineDeadError     | EngineCore 进程崩溃 | 不可恢复 | 直接抛出     |
| InputStreamError    | 输入流生成器异常        | -    | 传播原始异常   |
| ValueError          | 参数校验失败          | -    | 直接抛出     |
| EngineGenerateError | 推理过程异常          | 可恢复  | 包装后抛出    |

| 异常类型 | 含义 | 可恢复性 | 处理方式 |
|------|----|------|------|
|------|----|------|------|

**2.2.2 add\_request()** 一异步请求添加 支持普通请求和流式输入 (**Streaming Input**) 两种模式:

```
# async_llm.py L280-398
async def add_request(self, request_id, prompt, params, ...) -> RequestOutputCollector:
    if self.errorred:
        raise EngineDeadError()

    # 流式输入场景: prompt 是 AsyncGenerator[StreamingInput]
    if isinstance(prompt, AsyncGenerator): # L316
        return await self._add_streaming_input_request(...)

    # 普通请求: 通过 InputProcessor 预处理
    request = self.input_processor.process_inputs(...) # L349

    # 创建该请求专用的输出收集队列
    queue = RequestOutputCollector(params.output_kind, request.request_id) # L376

    # n>1 时 fan-out 子请求
    if is_pooling or params.n == 1:
        await self._add_request(request, ..., queue)
        return queue

    parent_request = ParentRequest(request)
    for idx in range(parent_params.n):
        ...
        await self._add_request(child_request, ..., queue)
    return queue
```

流式输入 (**Streaming Input**) 是一种高级特性, 允许在模型推理过程中逐步提供输入 (如逐 chunk 发送长文档)。其实现通过 `_add_streaming_input_request()` 方法创建一个后台 task 来消费输入流:

```
# async_llm.py L417-501
async def _add_streaming_input_request(self, input_stream, ...):
    async def handle_inputs():
        cancelled = False
        try:
            async for input_chunk in input_stream: # L461: 逐步消费
                req = self.input_processor.process_inputs(
                    request_id=internal_req_id,
                    prompt=input_chunk.prompt, # 每个 chunk 独立处理
                    ...
                )
                await self._add_request(req, ..., queue)
        except Exception as error:
            queue.put(InputStreamError(error)) # L489: 错误传播
        finally:
            # 发送最终空请求标记输入结束
            await self._add_request(final_req, None, None, 0, queue) # L495

    queue._input_stream_task = asyncio.create_task(handle_inputs()) # L500
    return queue
```

## 2.3 Output Handler 后台循环

`_run_output_handler()` 是 AsyncLLM 的心脏——一个在后台持续运行的 `asyncio Task`，负责从 EngineCore 拉取输出并分发给各请求的输出队列：

```
# async_llm.py L637-707
def _run_output_handler(self):
    """Background loop: pulls from EngineCore and pushes to AsyncStreams."""
    if self.output_handler is not None:
        return

    # 捕获引用避免循环引用（确保可被 GC 回收）
    engine_core = self.engine_core
    output_processor = self.output_processor
    logger_ref = [self.logger_manager]          # 可变引用，支持 elastic EP 更新
    chunk_size = envs.VLLM_V1_OUTPUT_PROC_CHUNK_SIZE

    async def output_handler():
        try:
            while True:
                # 异步从 EngineCore 获取一批输出
                outputs = await engine_core.get_output_async()    # L660

                # 分片处理（避免长时间阻塞事件循环）
                engine_core_outputs = outputs.outputs
                for start in range(0, num_outputs, chunk_size):
                    end = start + chunk_size
                    outputs_slice = engine_core_outputs[start:end]

                    # 处理输出: detokenize + logprobs + 分发到各请求队列
                    processed_outputs = output_processor.process_outputs(
                        outputs_slice, outputs.timestamp, iteration_stats
                    )
                    # 注意: processed_outputs.request_outputs 为空
                    # 因为输出已被 put 到各请求的 RequestOutputCollector 队列

                    # 让出控制权给其他 asyncio task
                    if end < num_outputs:
                        await asyncio.sleep(0)                    # L683

                    # 中止 stop string 匹配的请求
                    if processed_outputs.reqs_to_abort:
                        await engine_core.abort_requests_async(
                            processed_outputs.reqs_to_abort
                        )

                    # 记录统计日志
                    if logger_ref[0]:
                        logger_ref[0].record(...)
        except Exception as e:
            logger.exception("AsyncLLM output_handler failed.")
            output_processor.propagate_error(e)                    # L705: 广播错误

    self.output_handler = asyncio.create_task(output_handler())    # L707
```

关键设计: - 分片处理 (`chunk_size`): 防止单次 `process_outputs()` 处理过多输出导致事件循环阻塞 - 无循环引用: 通过局部变量捕获 + mutable list 引用实现 - 错误广播: `propagate_error()` 将错误放入所有活跃请求的队列

---

### 三、EngineCore 解耦执行循环

源码文件: `core.py`

`EngineCore` 是 `vLLM V1` 的解耦执行循环核心, 实现了 `schedule` → `execute` → `update` 的完整推理管线。它被设计为运行在独立进程中 (通过 `EngineCoreProc` 包装), 通过 `ZMQ` 与前端通信。

#### 3.1 EngineCore 基类

*# core.py L91-229 (节选)*

```
class EngineCore:
    """Inner loop of vLLM's Engine."""

    def __init__(self, vllm_config, executor_class, log_stats, ...):
        # 加载插件
        load_general_plugins() # L103

        # 创建 ModelExecutor (决定分布式执行策略)
        self.model_executor = executor_class(vllm_config) # L118

        # 初始化 KV Cache (含 memory profiling)
        kv_cache_config = self._initialize_kv_caches(vllm_config) # L128

        # 创建 Scheduler (调度策略)
        Scheduler = vllm_config.scheduler_config.get_scheduler_cls()
        self.scheduler: SchedulerInterface = Scheduler( # L145
            vllm_config=vllm_config,
            kv_cache_config=kv_cache_config,
            ...
        )

        # Pipeline Parallelism 批次队列
        self.batch_queue_size = self.model_executor.max_concurrent_batches
        if self.batch_queue_size > 1:
            self.batch_queue = deque(maxlen=self.batch_queue_size) # L194

        # 选择 step 函数 (普通 vs batch_queue 模式)
        self.step_fn = (
            self.step if self.batch_queue is None
            else self.step_with_batch_queue # L214
        )
```

`KV Cache` 初始化流程 `_initialize_kv_caches()` 是引擎启动中最关键的步骤之一:

*# core.py L232-310*

`@instrument(span_name="Prepare model")`

```
def _initialize_kv_caches(self, vllm_config) -> KVCacheConfig:
    # 获取模型所需的 KV cache 规格
    kv_cache_specs = self.model_executor.get_kv_cache_specs() # L236

    # Memory profiling: 确定可用 GPU 内存
```

```

available_gpu_memory = self.model_executor.determine_available_memory() # L250
self.available_gpu_memory_for_kv_cache = available_gpu_memory[0]

# 计算 KV cache 配置 (可能触发 auto-fit 降低 max_model_len)
kv_cache_configs = get_kv_cache_configs(
    vllm_config, kv_cache_specs, available_gpu_memory
)
# auto-fit 后需要同步 max_model_len 到 workers
if max_model_len_after != max_model_len_before:
    self.collective_rpc("update_max_model_len", args=(max_model_len_after,))

# 初始化 KV cache 并 warmup
self.model_executor.initialize_from_config(kv_cache_configs) # L283

```

### 3.2 EngineCoreProc 进程封装

EngineCoreProc 将 EngineCore 包装为可在后台进程中运行的实体，通过 **ZMQ socket** 实现进程间通信：

graph TB

```

subgraph FrontendProcess["Frontend Process"]
    ECC["EngineCoreClient"]
    ZIN["ZMQ DEALER (input)"]
    ZOUT["ZMQ ROUTER/PULL (output)"]
end

subgraph EngineProcess["EngineCore Process"]
    IT["Input Thread<br/>process_input_sockets()"]
    IQ["input_queue<br/>(Queue)"]
    ECP["EngineCoreProc<br/>run_busy_loop()"]
    OT["Output Thread<br/>process_output_sockets()"]
    OQ["output_queue<br/>(Queue)"]
    EC["EngineCore<br/>step() loop"]
end

ECC <-->|ZMQ| ZIN
ZIN --> IT
IT --> IQ
IQ --> ECP
ECP --> EC
EC --> OQ
OQ --> OT
OT --> ZOUT
ZOUT <-->|ZMQ| ECC

style IT fill:#e3f2fd
style OT fill:#e3f2fd
style EC fill:#e8f5e9,stroke:#4caf50,stroke-width:2px

```

核心组件：

| 组件           | 类型                                   | 职责              |
|--------------|--------------------------------------|-----------------|
| input_queue  | Queue[tuple[RequestType, Any]]       | 存储来自 ZMQ 的待处理请求 |
| output_queue | Queue[tuple[int, EngineCoreOutputs]] | 存储待发送给前端的输出     |

| 组件            | 类型                   | 职责                                     |
|---------------|----------------------|----------------------------------------|
| input_thread  | Thread (daemon=True) | 运行 process_input_sockets(), 处理 ZMQ 输入  |
| output_thread | Thread (daemon=True) | 运行 process_output_sockets(), 处理 ZMQ 输出 |
| aborts_queue  | Queue[list[str]]     | 高优先级 abort 请求队列                        |

### 3.3 核心调度-执行循环

**3.3.1 Busy Loop** 主循环 run\_busy\_loop() 是 EngineCore 的主事件循环:

```
# core.py L1164-1172
def run_busy_loop(self):
    """Core busy loop of the EngineCore."""
    while self._handle_shutdown(): # L1166: shutdown 检查
        # 处理输入队列中的请求
        self._process_input_queue() # L1168
        # 执行一步 engine core (schedule + execute + output)
        self._process_engine_step() # L1170
    raise SystemExit # L1172: 正常退出
```

**3.3.2 step()** 单步调度执行 step() 实现了最核心的 Schedule → Execute → Update 三阶段流水线:

```
# core.py L402-431
def step(self) -> tuple[dict[int, EngineCoreOutputs], bool]:
    """Schedule, execute, and make output."""

    # 空检查: 无请求则直接返回
    if not self.scheduler.has_requests():
        return {}, False

    # SCHEDULE: 调度器决定哪些请求被执行
    scheduler_output = self.scheduler.schedule() # L413

    # EXECUTE: 将调度结果发送给 model_executor 执行
    future = self.model_executor.execute_model(
        scheduler_output, non_block=True # L414
    )

    # 获取 grammar bitmask (structured output 场景)
    grammar_output = self.scheduler.get_grammar_bitmask(scheduler_output) # L415

    # 等待模型执行完成
    with (
        self.log_error_detail(scheduler_output), # 错误详情记录
        self.log_iteration_details(scheduler_output), # 迭代统计记录
    ):
        model_output = future.result() # L420
        if model_output is None:
            model_output = self.model_executor.sample_tokens(grammar_output) # L422

    # 处理执行期间收到的 abort 请求
    self._process_aborts_queue() # L426
```



```

# UPDATE: 用模型输出更新调度器状态, 生成 EngineCoreOutputs
engine_core_outputs = self.scheduler.update_from_output(
    scheduler_output, model_output
)
# L428

return engine_core_outputs, scheduler_output.total_num_scheduled_tokens > 0

```

**3.3.3 step\_with\_batch\_queue() — Pipeline Parallelism 模式** 当启用 Pipeline Parallelism (PP) 且 `max_concurrent_batches > 1` 时, 使用 `step_with_batch_queue()` 以消除 pipeline bubble:

# core.py L443-559 (简化逻辑)

```

def step_with_batch_queue(self):
    """
    执行流程:
    1. 若 batch_queue 未滿 → 尝试调度新 batch (高优先级)
    2. batch_queue 已滿或无可调度请求 → 阻塞等待最早 batch 完成
    3. 用完成的 batch 输出更新 scheduler
    """

    # 尝试填充 batch_queue
    if self.scheduler.has_requests():
        scheduler_output = self.scheduler.schedule()
        exec_future = self.model_executor.execute_model(scheduler_output, non_block=True)
        # ... sample_tokens 或 deferred sampling ...

        # 加入队列
        batch_queue.appendleft((future, scheduler_output, exec_future)) # L499

        # 队列未滿且有新请求 → 立即返回 (优先填满队列)
        if model_executed and len(batch_queue) < self.batch_queue_size and ...:
            return None, True

        # 阻塞等待最早的 batch 完成
        future, scheduler_output, exec_model_fut = batch_queue.pop() # L516
        model_output = future.result()
        # ... 处理输出 ...

```

### 3.4 数据流详解

**EngineCoreEvent / EngineCoreOutput 数据结构** 定义在 `__init__.py` 中:

# \_\_init\_\_.py L134-L158

```

class EngineCoreEventType(enum.IntEnum):
    QUEUED = 1 # 请求进入队列
    SCHEDULED = 2 # 请求被调度执行
    PREEMPTED = 3 # 请求被抢占

```

```

class EngineCoreEvent(msgspec.Struct):
    type: EngineCoreEventType
    timestamp: float # 单调时间戳

```

# \_\_init\_\_.py L161-191

```

class EngineCoreOutput(msgspec.Struct):
    request_id: str

```

```

new_token_ids: list[int]                # 新生成的 token IDs
new_logprobs: LogprobsLists | None      # token 级 logprobs
new_prompt_logprobs_tensors: ...        # prompt logprobs (仅 prefill)
pooling_output: torch.Tensor | None     # pooling 模型输出
finish_reason: FinishReason | None      # 完成原因
stop_reason: int | str | None           # 停止原因 (stop string 等)
events: list[EngineCoreEvent] | None    # 事件时间线
kv_transfer_params: dict | None         # KV transfer 参数
prefill_stats: PrefillStats | None      # Prefill 统计
routed_experts: np.ndarray | None       # MoE routing 信息

@property
def finished(self) -> bool:
    return self.finish_reason is not None

```

**EngineCoreRequestType** 请求类型枚举

```

# __init__.py L237-250
class EngineCoreRequestType(enum.Enum):
    ADD = b"\x00"    # 添加请求
    ABORT = b"\x01"  # 中止请求
    START_DP_WAVE = b"\x02" # DP wave 启动通知
    UTILITY = b"\x03" # 工具方法调用 (collective_rpc 等)
    EXECUTOR_FAILED = b"\x04" # Executor 失败标记
    WAKEUP = b"\x05" # 唤醒空闲引擎 (shutdown 时使用)

```

请求处理分发 `_handle_client_request()` 根据请求类型进行分发:

```

# core.py L1266-1299
def _handle_client_request(self, request_type, request):
    if request_type == EngineCoreRequestType.WAKEUP:
        return
    elif request_type == EngineCoreRequestType.ADD:
        req, request_wave = request
        if self._reject_add_in_shutdown(req):
            return
        self.add_request(req, request_wave) # L1277
    elif request_type == EngineCoreRequestType.ABORT:
        self.abort_requests(request) # L1279
    elif request_type == EngineCoreRequestType.UTILITY:
        # 工具方法调用 (profile, reset_cache, lora 操作等)
        client_idx, call_id, method_name, args = request
        output = UtilityOutput(call_id)
        get_result = lambda: (
            (method := getattr(self, method_name))
            and method(*self._convert_msgspeg_args(method, args))
        )
        self._invoke_utility_method(method_name, get_result, output, enqueue_output) # L1293

```

#### 四、InputProcessor 请求预处理

源码文件: `input_processor.py`

**InputProcessor** 负责将用户原始输入转换为 **EngineCore** 可以处理的标准化请求 (**EngineCoreRequest**)，是引擎管线的第一环。

核心职责

```
graph TD
    IN["用户输入 <br/>(PromptType / EngineInput)"] --> VAL["参数验证 <br/>_validate_params()"]
    VAL --> LRA["LoRA 验证 <br/>_validate_lora()"]
    LRA --> PP["输入预处理 <br/>InputPreprocessor.preprocess()"]
    PP --> SPLIT["Encoder/Decoder<br/> 拆分"]
    SPLIT --> MV["多模态?<br/>mm_features"]
    MV -->|Yes| MM["多模态特征提取 <br/>MultiModalFeatureSpec"]
    MV -->|No| TOK["Token IDs / Embeds"]
    MM --> ECR["EngineCoreRequest"]
    TOK --> ECR
    ECR --> RID["assign_request_id()<br/> 分配内部 ID"]

    style IN fill:#e3f2fd
    style ECR fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
```

**process\_inputs()** 主方法

**process\_inputs()** 是 **InputProcessor** 的核心方法:

```
# input_processor.py L234-377
def process_inputs(
    self, request_id, prompt, params, supported_tasks,
    arrival_time=None, lora_request=None, ...
) -> EngineCoreRequest:

    # 参数验证
    self._validate_params(params, supported_tasks)           # L248
    self._validate_lora(lora_request)                         # L249

    # Data Parallel rank 校验
    parallel_config = self.vllm_config.parallel_config
    dp_size = parallel_config.data_parallel_size
    ...
    if data_parallel_rank is not None and not (0 <= data_parallel_rank < num_ranks):
        raise ValueError(...)

    # 输入预处理 (tokenization + multimodal)
    if isinstance(prompt, dict) and "type" in prompt:
        processed_inputs = prompt                             # L272: 已预处理
    else:
        processed_inputs = self.input_preprocessor.preprocess( # L283
            prompt, tokenization_kwargs=tokenization_kwargs,
        )

    # 平台级验证
    current_platform.validate_request(processed_inputs, params) # L288

    # Encoder/Decoder 输入拆分
    encoder_inputs, decoder_inputs = split_enc_dec_input(processed_inputs) # L290
    self._validate_model_inputs(encoder_inputs, decoder_inputs) # L291
```

```

# 提取 prompt tokens / embeds
if decoder_inputs["type"] == "embeds":
    prompt_embeds = decoder_inputs["prompt_embeds"]
    prompt_token_ids = decoder_inputs.get("prompt_token_ids")
else:
    prompt_token_ids = decoder_inputs["prompt_token_ids"]
    prompt_embeds = None

# 处理 SamplingParams / PoolingParams
if isinstance(params, SamplingParams):
    sampling_params = params.clone()
    if sampling_params.max_tokens is None:
        # 自动计算 max_tokens = max_model_len - prompt_length
        seq_len = length_from_prompt_token_ids_or_embeds(
            prompt_token_ids, prompt_embeds
        )
        sampling_params.max_tokens = self.model_config.max_model_len - seq_len
    sampling_params.update_from_generation_config(...)
    sampling_params.update_from_tokenizer(self.tokenizer)
else:
    pooling_params = params.clone()

# 多模态特征处理
mm_features = None
if decoder_inputs["type"] == "multimodal":
    # 按 position 排序多模态输入
    sorted_mm_idxes = argsort_mm_positions(decoder_mm_positions)
    mm_features = []
    for modality, idx in sorted_mm_idxes:
        base_mm_hash = decoder_mm_hashes[modality][idx]
        mm_features.append(MultiModalFeatureSpec(
            data=decoder_mm_inputs[modality][idx],
            modality=modality,
            identifier=self._get_mm_identifier(base_mm_hash, lora_request),
            mm_position=decoder_mm_positions[modality][idx],
            mm_hash=base_mm_hash,
        ))

# 构造并返回 EngineCoreRequest
return EngineCoreRequest(
    request_id=request_id,
    prompt_token_ids=prompt_token_ids,
    prompt_embeds=prompt_embeds,
    mm_features=mm_features,
    sampling_params=sampling_params,
    pooling_params=pooling_params,
    arrival_time=arrival_time,
    lora_request=lora_request,
    priority=priority,
    data_parallel_rank=data_parallel_rank,
    trace_headers=trace_headers,
    resumable=resumable,
)

```

# L362

assign\_request\_id() —请求 ID 分配

```
# input_processor.py L215-L232
@staticmethod
def assign_request_id(request: EngineCoreRequest):
    """Replace the externally supplied request ID with an internal request ID
    that adds 8 random characters in order to ensure uniqueness."""
    if request.external_req_id is not None:
        raise ValueError(...)

    request.external_req_id = request.request_id # L224: 保存原始 ID
    if envs.VLLM_DISABLE_REQUEST_ID_RANDOMIZATION:
        # 仅用于调试/测试
        request.request_id = f"{request.external_req_id}"
    else:
        # 追加 8 位十六进制随机 UUID
        request.request_id = f"{request.external_req_id}-{random.uuid():.8}" # L232
```

设计意图：外部 request\_id 由用户指定，可能不唯一；内部 request\_id 保证全局唯一性。外部 ID 用于 abort 查询时的用户友好接口。

## 五、OutputProcessor 输出后处理

源码文件：output\_processor.py

OutputProcessor 是引擎管线的最后一环，负责将 EngineCoreOutput 转化为面向用户的 RequestOutput，包括 detokenization、logprobs 计算、stop string 检测和统计信息收集。

架构概览

```
graph TD
    ECO["EngineCoreOutputs<br/>(list[EngineCoreOutput])"] --> PO["process_outputs()"]
    PO --> LOOP{" 遍历每个 <br/>EngineCoreOutput"}
    LOOP --> STATS["_update_stats_from_output()<br/> 统计更新"]
    LOOP --> DET["detokenizer.update()<br/> 反 Tokenization"]
    DET --> STOP{"stop_string<br/> 检测"}
    STOP -->| 匹配 | FR["finish_reason=STOP"]
    STOP -->| 未匹配 | CONT[" 继续生成"]
    LOOP --> LP["logprobs_processor.update()<br/>Logprobs 计算"]
    LOOP --> RO["make_request_output()<br/> 构建 RequestOutput"]
    RO --> QUEUE{" 有 queue?"}
    QUEUE -->|Yes AsyncLLM| PUT["req_state.queue.put(output)<br/> 放入请求队列"]
    QUEUE -->|No LLMEngine| RET[" 追加到返回列表"]

    style ECO fill:#fce4ec
    style RO fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
```

核心数据结构

**RequestState** —请求状态跟踪 **RequestState** 维护每个活跃请求的完整状态：

```
# output_processor.py L132-209
class RequestState:
    def __init__(self, request_id, external_req_id, parent_req,
                 request_index, lora_request, output_kind, prompt,
```

```

        prompt_token_ids, prompt_embeds, logprobs_processor,
        detokenizer, max_tokens_param, arrival_time, queue, ...):

# 身份信息
self.request_id = request_id
self.external_req_id = external_req_id
self.parent_req = parent_req          # n>1 时的父请求

# Prompt 信息
self.prompt = prompt
self.prompt_token_ids = prompt_token_ids
self.prompt_embeds = prompt_embeds
self.prompt_len = length_from_prompt_token_ids_or_embeds(...)

# 处理器
self.logprobs_processor = logprobs_processor
self.detokenizer = detokenizer

# 状态标志
self.is_prefilling = True             # 是否仍在 prefill 阶段
self.streaming_input = False          # 是否为流式输入

# 输出控制
self.stream_interval = stream_interval # 流式输出间隔
self.sent_tokens_offset = 0            # 已发送 token 偏移量

# 输出目标
self.queue = queue                    # AsyncLLM: RequestOutputCollector
                                     # LLMEngine: None (直接返回列表)

```

**RequestOutputCollector** 一异步输出收集器 `RequestOutputCollector` 为 `AsyncLLM` 模式下的每个请求提供异步生产者-消费者队列:

```

# output_processor.py L48-109
class RequestOutputCollector:
    def __init__(self, output_kind, request_id):
        self.aggregate = output_kind == RequestOutputKind.DELTA # DELTA 模式需聚合
        self.request_id = request_id
        self.output = None                                       # 当前输出
        self.ready = asyncio.Event()                             # 就绪信号

    def put(self, output):                                       # 非阻塞写入
        if self.output is None or isinstance(output, Exception):
            self.output = output
            self.ready.set()
        elif isinstance(output, RequestOutput):
            self.output.add(output, aggregate=self.aggregate)   # DELTA 模式聚合

    async def get(self):                                         # 阻塞读取
        while (output := self.output) is None:
            await self.ready.wait()
        self.output = None
        self.ready.clear()
        if isinstance(output, Exception):

```

```

        raise output
    return output

def get_nowait(self):                # 非阻塞读取
    ...

```

process\_outputs() 一核心处理方法

process\_outputs() 是 OutputProcessor 的主处理循环, 也是 vLLM V1 中唯一定义了遍历整个 **batch** 循环的位置:

# output\_processor.py L597-712

```

def process_outputs(self, engine_core_outputs, engine_core_timestamp, iteration_stats):
    """

```

**NOTE FOR DEVELOPERS:**

*vLLM V1 minimizes the number of python loops over the full batch to ensure system overheads are minimized. This is the only function that should loop over EngineCoreOutputs.*

```

request_outputs = []
reqs_to_abort = []

```

```

for engine_core_output in engine_core_outputs:                # L627: 唯一的 batch 遍历
    req_id = engine_core_output.request_id
    req_state = self.request_states.get(req_id)
    if req_state is None:
        continue                # 已 abort 的请求

    # 更新统计信息
    self._update_stats_from_output(req_state, engine_core_output, ...) # L635

    # 处理 prefill+decode 过渡
    if req_state.is_prefilling:
        if engine_core_output.prefill_stats is not None:
            req_state.num_cached_tokens = ...                # L648: prefix cache 命中数
            req_state.is_prefilling = False                # L651

    # Detokenization + Stop String 检测
    if engine_core_output.pooling_output is None:
        stop_string = req_state.detokenizer.update(                # L657
            new_token_ids, finish_reason == FinishReason.STOP
        )
        if stop_string:
            finish_reason = FinishReason.STOP                # L661: stop string 匹配
            stop_reason = stop_string

    # Logprobs 计算
    req_state.logprobs_processor.update_from_output(engine_core_output) # L666

    # 构建输出对象
    if request_output := req_state.make_request_output(...): # L669
        if req_state.streaming_input:
            request_output.finished = False                # L678: 流式输入未结束

```

```

        if req_state.queue is not None:
            # AsyncLLM 模式: 放入请求专属队列
            req_state.queue.put(request_output)                # L682
        else:
            # LLMEngine 模式: 追加到返回列表
            request_outputs.append(request_output)              # L685

    # 清理已完成请求
    if finish_reason is not None:
        if req_state.streaming_input:
            # 流式输入: 应用下一个 chunk
            ...                                                # L689-694
        else:
            self._finish_request(req_state)                    # L696
            if not engine_core_output.finished:
                # EngineCore 未结束但 detokenizer 检测到 stop string
                reqs_to_abort.append(req_id)                    # L700: 需要 abort

    return OutputProcessorOutput(request_outputs, reqs_to_abort)

```

## Stream Interval 控制

`stream_interval` 参数控制流式输出的最小 `token` 间隔, 减少高频小包传输的开销:

```

# output_processor.py L272-301 (RequestState.make_request_output 中)
def make_request_output(self, new_token_ids, pooling_output, finish_reason, ...):
    if self.stream_interval > 1:
        # 仅在以下条件满足时发送输出:
        # 1. 请求已完成, 或
        # 2. 是第一个 token, 或
        # 3. 已达到 stream_interval 间隔
        if not (
            finished
            or self.sent_tokens_offset == 0
            or self.detokenizer.num_output_tokens() - self.sent_tokens_offset >= self.stream_interval
        ):
            return None                                         # 跳过本次输出

    if self.output_kind == RequestOutputKind.DELTA:
        # DELTA 模式: 只发送增量 token
        new_token_ids = self.detokenizer.output_token_ids[self.sent_tokens_offset:]
        self.sent_tokens_offset = self.detokenizer.num_output_tokens()

```

## 六、Detokenizer 异步反 Tokenization

源码文件: `detokenizer.py`

Detokenizer 负责将模型输出的 `token IDs` 序列增量转换为可读文本, 同时检测 `stop string` 匹配。

类层次结构

```

classDiagram
    class IncrementalDetokenizer {
        +token_ids: list[int]
    }

```



```

    +output_token_ids: list[int]
    +num_output_tokens() int
    +update(new_token_ids, stop_terminated) str | None
    +get_next_output_text(finished, delta) str
    +from_new_request(tokenizer, request) IncrementalDetokenizer
}

class BaseIncrementalDetokenizer {
    +stop: list[str]
    +min_tokens: int
    +include_stop_str_in_output: bool
    +stop_buffer_length: int
    +output_text: str
    +update(new_token_ids, stop_terminated) str | None
    +get_next_output_text(finished, delta) str
    +decode_next(token_id) str *abstract*
}

class FastIncrementalDetokenizer {
    -tokenizer: Tokenizer
    -stream: DecodeStream
    -skip_special_tokens: bool
    +decode_next(token_id) str
}

class SlowIncrementalDetokenizer {
    -tokenizer: TokenizerLike
    -tokens: list[str]
    -prefix_offset: int
    -read_offset: int
    -prompt_len: int
    +decode_next(token_id) str
    +output_token_ids: list[int]
    +num_output_tokens() int
}

IncrementalDetokenizer <|-- BaseIncrementalDetokenizer
BaseIncrementalDetokenizer <|-- FastIncrementalDetokenizer
BaseIncrementalDetokenizer <|-- SlowIncrementalDetokenizer

```

工厂方法：自动选择策略

from\_new\_request() 根据 tokenizer 类型自动选择最优实现:

```

# detokenizer.py L49-65
@classmethod
def from_new_request(cls, tokenizer, request):
    assert request.sampling_params is not None

    if tokenizer is None:
        # 无 tokenizer → 跳过 detokenization
        return IncrementalDetokenizer()

    # 条件: tokenizers >= 0.22.0 且使用 PreTrainedTokenizerFast
    USE_FAST_DETOKENIZER = version.parse(tokenizers.__version__) >= version.parse("0.22.0")

```

```

if USE_FAST_DETOKENIZER and isinstance(tokenizer, PreTrainedTokenizerFast):
    # 快速路径: 使用 tokenizers 库原生 DecodeStream (Rust 实现)
    return FastIncrementalDetokenizer(tokenizer, request) # L62

# 慢速回退: 纯 Python 增量 detokenization
return SlowIncrementalDetokenizer(tokenizer, request) # L65

```

### FastIncrementalDetokenizer —快速路径

利用 Huggingface tokenizers 库 (Rust 加速) 的 DecodeStream:

```

# detokenizer.py L167-247
class FastIncrementalDetokenizer(BaseIncrementalDetokenizer):
    def __init__(self, tokenizer, request):
        super().__init__(request)
        self.tokenizer: Tokenizer = tokenizer._tokenizer

        # 使用 native prefill 用 prompt tokens 初始化 decode stream
        self.stream = tokenizers.decoders.DecodeStream(
            ids=request.prompt_token_ids, # L184: 预填充 prompt
            skip_special_tokens=self.skip_special_tokens,
        )

    def decode_next(self, next_token_id: int) -> str:
        token = self._protected_step(next_token_id) # L211

        # 处理 special tokens 之间的空格控制
        if not self.spaces_between_special_tokens:
            special_token = self.added_token_ids.get(next_token_id)
            is_special = special_token is not None
            if is_special and self.last_special:
                token = special_token # L218: 不加前缀空格
            self.last_special = is_special

        return token or ""

    def _protected_step(self, next_token_id):
        try:
            token = self.stream.step(self.tokenizer, next_token_id)
        except (OverflowError, TypeError):
            # 无效 token ID 处理
            ...
        except Exception as e:
            if not str(e).startswith(INVALID_PREFIX_ERR_MSG):
                raise e
            # 恢复无效 UTF-8 前缀导致的 DecodeStream 状态损坏
            self.stream = tokenizers.decoders.DecodeStream(...)
            token = self.stream.step(self.tokenizer, next_token_id)
        return token

```

### Stop String 检测机制

check\_stop\_strings() 在每次 detokenize 后检查是否匹配停止字符串:

```

# detokenizer.py L309-344

```

```

def check_stop_strings(output_text, new_char_count, stop, include_in_output):
    """Check if any stop strings are matched and truncate sequence.

    Returns tuple (stop_string, offset) if matched or else None.
    offset: length to truncate to, or -1 for no truncation.
    """
    if not new_char_count or not stop:
        return None

    for stop_str in stop:
        stop_string_len = len(stop_str)
        # 只搜索新增文本区域 (避免重复搜索)
        stop_index = output_text.find(stop_str, 1 - new_char_count - stop_string_len)
        if stop_index == -1:
            continue

        if include_in_output:
            stop_index += stop_string_len
            if stop_index >= len(output_text):
                return stop_str, -1

        return stop_str, stop_index

    return None

**BaseIncrementalDetokenizer.update() 整合了增量 detokenize + stop check:
# detokenizer.py L95-142
def update(self, new_token_ids, stop_terminated) -> str | None:
    # 处理 stop token 排除
    if stop_terminated and not self.include_stop_str_in_output:
        skipped_stop_token_id = new_token_ids[-1]
        new_token_ids = new_token_ids[:-1]
    else:
        skipped_stop_token_id = None

    # 增量 detokenize
    stop_check_offset = len(self.output_text)
    for new_token_id in new_token_ids:
        self.token_ids.append(new_token_id)
        self.output_text += self.decode_next(new_token_id)
        if self.min_tokens and self.num_output_tokens() <= self.min_tokens:
            stop_check_offset = len(self.output_text)

    # Stop string 检测
    stop_string = None
    if self.stop and self.num_output_tokens() > self.min_tokens:
        stop = check_stop_strings(
            output_text=self.output_text,
            new_char_count=len(self.output_text) - stop_check_offset,
            stop=self.stop,
            include_in_output=self.include_stop_str_in_output,
        )
        if stop is not None:
            stop_string, truncate_to = stop

```

```

        if truncate_to != -1:
            self.output_text = self.output_text[:truncate_to] # L140: 截断

    return stop_string

```

---

## 七、Coordinator 多节点协调

源码文件: coordinator.py

DPCoordinator (Data Parallel Coordinator) 是  $DP > 1$  部署场景下的中央协调进程, 位于多个 Engine Core 进程和前端 API Server 之间。

架构角色

```

graph TB
    subgraph Frontends["Frontend Processes (N)"]
        F1["API Server 1"]
        F2["API Server 2"]
    end

    subgraph Coordinator["DPCoordinator Process"]
        XPUB_F["XPUB Socket<br/>(front_publish)"]
        PULL_B["PULL Socket<br/>(back_output)"]
        XPUB_B["XPUB Socket<br/>(back_publish)"]
    end

    subgraph Engines["EngineCore Processes (DP size)"]
        E0["EngineCore DP0"]
        E1["EngineCore DP1"]
        EN["EngineCore DPn"]
    end

    F1 <-->|SUB/PUB| XPUB_F
    F2 <-->|SUB/PUB| XPUB_F
    E0 <-->|PUB/SUB| XPUB_B
    E1 <-->|PUB/SUB| XPUB_B
    EN <-->|PUB/SUB| XPUB_B
    E0 -->|PUSH| PULL_B
    E1 -->|PUSH| PULL_B
    EN -->|PUSH| PULL_B

    XPUB_F -->| 发布 stats + wave state| F1
    XPUB_F -->| 发布 stats + wave state| F2
    XPUB_B -->| 广播 START_DP_WAVE| E0
    XPUB_B -->| 广播 START_DP_WAVE| E1
    PULL_B -->| 收集 stats| Coordinator

    style Coordinator fill:#fff3e0,stroke:#ff9800,stroke-width:2px

```

### DPCoordinator 类

DPCoordinator 是 coordinator 进程的包装类, 负责进程生命周期管理:

# coordinator.py L23-143

```

class DPCoordinator:
    """Coordinator process used for data-parallel deployments (DP>1).

    职责:
    * 收集各 DP engine 的负载统计 (waiting/running queue lengths),
      发布给所有 front-end 用于负载均衡决策。
    * 跟踪当前 DP "request wave" 编号和 engines 运行状态。
      通过 all-reduce 操作在 DPEngineCoreProc._has_global_unfinished_reqs
      中同步。
    * 广播 START_DP_WAVE 消息唤醒处于 paused 状态的 engines。
    """

    def __init__(self, parallel_config, enable_wave_coordination=True):
        dp_size = parallel_config.data_parallel_size
        host = parallel_config.data_parallel_master_ip

        # 绑定地址 (区分 local 和 remote)
        front_publish_address = bind_address(local_only)      # → Frontend PUB
        back_output_address = bind_address(local_only_eng)    # ← Engine PUSH
        back_publish_address = bind_address(local_only_eng)   # → Engine PUB

        # 启动 coordinator 子进程
        self.proc = context.Process(
            target=DPCoordinatorProc.run_coordinator,
            name="VLLM_DP_Coordinator",
            kwargs={},
            daemon=True,
        )
        self.proc.start()

```

### DPCoordinatorProc 一核心协调逻辑

DPCoordinatorProc 运行在 coordinator 进程内, 维护三个 ZMQ socket 的事件循环:

```

# coordinator.py L194-447 (简化)
def process_input_socket(self, front_publish_address, back_output_address,
                        back_publish_address, zmq_addr_pipe=None):
    decoder = MsgpackDecoder(EngineCoreOutputs)
    current_wave = 0
    engines_running = False

    with (
        make_zmq_socket(front_publish_address, zmq.XPUB, bind=True) as publish_front,  # → Frontend
        make_zmq_socket(back_output_address, zmq.PULL, bind=True) as output_back,      # ← Engine
        make_zmq_socket(back_publish_address, zmq.XPUB, bind=True) as publish_back,    # → Engine
    ):
        poller = zmq.Poller()
        poller.register(publish_front, zmq.POLLIN)
        poller.register(publish_back, zmq.POLLIN)
        poller.register(output_back, zmq.POLLIN)

        while True:
            events = poller.poll(timeout=max(min_timeout, wait_for - elapsed))

            if not events:

```

```

        # 超时: 发布当前统计到 front-end
        publish_front.send(msgpack.encode((engine_counts, current_wave, engines_running)))
        continue

    if publish_back in events:
        # Engine 订阅消息 (新 engine 加入等)
        ...

    if publish_front in events:
        # Frontend 新请求到达 (wave 协调): 广播 START_DP_WAVE
        engine_to_exclude, wave = decoded
        engines_running = True
        self._send_start_wave(publish_back, current_wave, engine_to_exclude)

    if output_back in events:
        # Engine 统计/wave 状态更新
        outputs = decoder.decode(buffer)
        eng_index = outputs.engine_index
        scheduler_stats = outputs.scheduler_stats
        if scheduler_stats:
            # 更新本地负载统计
            self.engines[eng_index].request_counts = [
                scheduler_stats.num_waiting_reqs,
                scheduler_stats.num_running_reqs,
            ]

        # Wave 协调
        if (wave := outputs.wave_complete) is not None:
            current_wave = wave + 1
            engines_running = False  # 所有 engine 进入 paused

```

## Wave 协议机制

Wave 协议是 DP 场景下同步各 **engine** 生命周期的核心机制:

1. **Running** 阶段: engines 处理当前 wave 的所有请求
2. **Pause** 阶段: 所有请求完成后, 通过 all-reduce 确认全局完成, engines 进入 paused
3. **Wave Complete** 通知: rank 0 engine 通过 coordinator 通知 front-end
4. **Wake Up**: 新请求到达时, coordinator 广播 *START\_DP\_WAVE*, engines 重新进入 running

stateDiagram-v2

```

[*] --> Running: 收到请求 / START_DP_WAVE
Running --> Paused: 所有请求完成 <br/>(all-reduce 确认)
Paused --> Running: START_DP_WAVE<br/>(新请求到达)
Paused --> [*]: Shutdown

```

```

state Running {
    [*] --> Processing
    Processing --> Processing: step()
    Processing --> Draining: 无新请求
}

```

## 八、异常处理机制

源码文件: exceptions.py

vLLM V1 定义了两层异常体系, 精确区分可恢复和不可恢复的错误:

```
# exceptions.py L1-18
class EngineGenerateError(Exception):
    """Raised when a AsyncLLM.generate() fails. Recoverable."""
    pass

class EngineDeadError(Exception):
    """Raised when the EngineCore dies. Unrecoverable."""

    def __init__(self, *args, suppress_context: bool = False, **kwargs):
        ENGINE_DEAD_MESSAGE = (
            "EngineCore encountered an issue. "
            "See stack trace (above) for the root cause."
        )
        super().__init__(ENGINE_DEAD_MESSAGE, *args, **kwargs)
        # 抑制无关的 ZMQError 堆栈 (使 LLMMODE 下的报错更清晰)
        self.__suppress_context__ = suppress_context
```

异常传播路径

```
graph TD
    EC_ERR["EngineCore 进程异常"] --> OT["output_thread 检测到 <br/>ENGINE_CORE_DEAD"]
    OT --> ECC_ERR["EngineCoreClient<br/> 检测到 dead"]
    ECC_ERR --> EDE["EngineDeadError"]

    GEN_ERR["generate() 内部异常"] --> EGE["EngineGenerateError"]
    IS_ERR["InputStreamError"] --> PROP["propagate_error()"]
    PROP --> ALL_Q["所有请求队列 <br/> 收到异常"]

    EDE --> | 不可恢复 | FRONTEND["API Server:<br/> 返回 503"]
    EGE --> | 可恢复 | FRONTEND
    ALL_Q --> FRONTEND

    style EDE fill:#ffcdd2,stroke:#c62828,stroke-width:2px
    style EGE fill:#fff9c4,stroke:#f9a825,stroke-width:1px
```

异常在 AsyncLLM.generate() 中的处理

参考 async\_llm.py L588-635 中的异常分层处理:

| 异常来源          | 异常类型             | 触发条件      | 处理方式                    |
|---------------|------------------|-----------|-------------------------|
| 客户端断开         | CancelledError   | 客户端关闭连接   | abort 请求, re-raise      |
| EngineCore 崩溃 | EngineDeadError  | 后台进程死亡    | 直接 re-raise (不可恢复)      |
| 输入流错误         | InputStreamError | 用户输入生成器异常 | 传播原始 cause              |
| 参数校验          | ValueError       | 无效参数      | 直接 re-raise             |
| 推理过程          | Exception        | 未知运行时错误   | 包装为 EngineGenerateError |

## Error Propagation —广播到所有请求

`OutputProcessor.propagate_error()` 在 `output_handler` 捕获到异常时调用，将错误广播到所有活跃请求：

```
# output_processor.py L464-469
def propagate_error(self, e: Exception):
    """Propagate error to all generate() tasks."""
    for _, state in self.request_states.items():
        assert state.queue is not None
        state.queue.put(e)  # 放入每个请求的输出队列
```

这使得所有正在 `await generate()` 的调用方都能收到错误通知并优雅退出。

## EngineCore 进程内的错误处理

在 `EngineCoreProc.run_engine_core()` 中，异常会被捕获并通过 ZMQ 通知前端：

```
# core.py L1064-1147
@staticmethod
def run_engine_core(*args, dp_rank=0, **kwargs):
    engine_core = None
    try:
        ...  # 初始化
        engine_core.run_busy_loop()  # L1129: 主循环
    except SystemExit:
        raise
    except Exception as e:
        if engine_core is None:
            logger.exception("EngineCore failed to start.")
        else:
            logger.exception("EngineCore encountered a fatal error.")
            engine_core._send_engine_dead()  # L1139: 发送 ENGINE_CORE_DEAD
            raise e
    finally:
        signal.signal(signal.SIGTERM, signal.SIG_DFL)
        if engine_core is not None:
            engine_core.shutdown()
```

`_send_engine_dead()` 确保 DEAD 消息在进程退出前发出：

```
# core.py L1358-1365
def _send_engine_dead(self):
    self.output_queue.put_nowait(EngineCoreProc.ENGINE_CORE_DEAD)  # L1362
    # 等待 output_thread 发送完毕
    self.output_thread.join(timeout=5.0)
    if self.output_thread.is_alive():
        logger.fatal("vLLM shutdown signal failed to send...")
```

---

## 附录：处理管线完整数据流图

以下是 vLLM V1 Engine Core 从请求接入到响应返回的端到端数据流：

```
sequenceDiagram
    participant User as 用户/API Server
    participant AL as AsyncLLM / LLMEngine
    participant IP as InputProcessor
```



```

participant ECC as EngineCoreClient
participant EP as EngineCoreProc
participant EC as EngineCore
participant SCH as Scheduler
participant ME as ModelExecutor
participant OP as OutputProcessor
participant DT as Detokenizer

User->>AL: generate(prompt, params)
AL->>IP: process_inputs(prompt, params)
IP->>IP: validate_params + validate_lora
IP->>IP: preprocess (tokenize + multimodal)
IP->>IP: assign_request_id()
IP-->>AL: EngineCoreRequest
AL->>OP: add_request(request)
AL->>ECC: add_request_async(request)
ECC->>EP: ZMQ SEND (ADD, request)
EP->>EP: input_thread → input_queue
EP->>EC: _handle_client_request(ADD)
EC->>SCH: add_request(request)

loop Engine Core Busy Loop
    EC->>SCH: schedule()
    SCH-->>EC: SchedulerOutput
    EC->>ME: execute_model(scheduler_output)
    ME-->>EC: ModelRunnerOutput
    EC->>SCH: update_from_output()
    SCH-->>EC: EngineCoreOutputs
    EC->>EP: output_queue.put(outputs)
end

EP->>ECC: ZMQ SEND (outputs)
ECC-->>AL: get_output_async()
AL->>OP: process_outputs(outputs)

loop For each EngineCoreOutput
    OP->>DT: update(new_token_ids)
    DT-->>OP: text + stop_string
    OP->>OP: logprobs processing
    OP->>OP: make_request_output()
    OP-->>AL: queue.put(RequestOutput)
end

AL-->>User: yield RequestOutput (streaming)

```

#### 各阶段数据格式转换总结

| 阶段                     | 输入格式                           | 输出格式              | 关键转换                                 |
|------------------------|--------------------------------|-------------------|--------------------------------------|
| <b>InputProcessing</b> | PromptType /<br>EngineInput    | EngineCoreRequest | Tokenization, Multimodal<br>encoding |
| <b>Schedule</b>        | EngineCoreRequest →<br>Request | SchedulerOutput   | Batching, Priority sorting           |
| <b>Execute</b>         | SchedulerOutput                | ModelRunnerOutput | GPU kernel launch                    |

| 阶段                         | 输入格式                                   | 输出格式                               | 关键转换                                                                  |
|----------------------------|----------------------------------------|------------------------------------|-----------------------------------------------------------------------|
| Update<br>OutputProcessing | ModelRunnerOutput<br>EngineCoreOutputs | EngineCoreOutputs<br>RequestOutput | Sampling, Finish detection<br>Detokenization, Logprobs,<br>Stop check |

文档版本: 基于 vLLM V1 engine core 源码分析  
涉及核心文件: - llm\_engine.py 一同步引擎入口 - async\_llm.py 一异步引擎实现 - core.py 一 EngineCore 执行循环 - input\_processor.py 一请求预处理 - output\_processor.py 一输出后处理 - detokenizer.py 一反 Tokenization - coordinator.py 一多节点协调 - exceptions.py 一异常定义 - \_\_init\_\_.py 一数据结构定义

## vLLM 调度系统深度分析

定位: 本文档深入剖析 vLLM v1 架构中的调度系统, 涵盖 Scheduler 核心逻辑、Continuous Batching 原理、调度策略、请求生命周期管理等关键机制。

### 调度系统架构总览

```
graph TB
    subgraph SchedulerCore["Scheduler 核心组件"]
        direction TB
        Waiting["Waiting Queue<br/> 等待队列"]
        Running["Running List<br/> 运行列表"]
        Skipped["Skipped Waiting<br/> 跳过队列"]
        Finished["Finished Set<br/> 完成集合"]
    end

    subgraph RequestFlow[" 请求状态流转"]
        NewReq[" 新请求"] --> Waiting
        Waiting --> |" 调度成功"| Running
        Running --> |" 完成"| Finished
        Running --> |" 抢占"| Waiting
        Waiting --> |" 异步依赖"| Skipped
        Skipped --> |" 依赖就绪"| Waiting
    end

    subgraph ScheduleLogic["schedule() 核心逻辑"]
        direction TB
        Step1["1. 调度 RUNNING 请求 <br/>(decode/prefill chunk)"]
        Step2["2. 调度 WAITING 请求 <br/>(新请求/恢复请求)"]
        Step3["3. KV 缓存块分配"]
        Step4["4. 构建 SchedulerOutput"]
        Step1 --> Step2 --> Step3 --> Step4
    end

    subgraph Output[" 输出数据结构"]
        NewReqData["NewRequestData<br/> 新请求数据"]
        CachedReqData["CachedRequestData<br/> 缓存请求数据"]
        SchedTokens["num_scheduled_tokens<br/> 调度令牌数"]
        BlockIds["block_ids<br/>KV 缓存块 ID"]
    end
```

```
end

SchedulerCore --> ScheduleLogic
ScheduleLogic --> Output

style SchedulerCore fill:#fce4ec
style RequestFlow fill:#e3f2fd
style ScheduleLogic fill:#f3e5f5
style Output fill:#e8f5e9
```

一、Scheduler 类核心逻辑

1.1 类定义与核心属性

Scheduler 类位于 scheduler.py:62，是 vLLM v1 调度系统的核心实现。

```
class Scheduler(SchedulerInterface):
    def __init__(
        self,
        vllm_config: VllmConfig,
        kv_cache_config: KVCacheConfig,
        structured_output_manager: StructuredOutputManager,
        block_size: int,
        hash_block_size: int | None = None,
        mm_registry: MultiModalRegistry = MULTIMODAL_REGISTRY,
        include_finished_set: bool = False,
        log_stats: bool = False,
    ) -> None:
```

核心数据结构:

| 属性                    | 类型                 | 说明                           |
|-----------------------|--------------------|------------------------------|
| self.requests         | dict[str, Request] | 所有请求的映射表 (req_id -> Request) |
| self.waiting          | RequestQueue       | 等待调度的请求队列                    |
| self.skipped_waiting  | RequestQueue       | 因异步依赖被跳过的请求队列                |
| self.running          | list[Request]      | 正在运行的请求列表                    |
| self.finished_req_ids | set[str]           | 已完成请求 ID 集合                  |
| self.policy           | SchedulingPolicy   | 调度策略 (FCFS/PRIORITY)         |

调度约束参数 (scheduler.py:100-107):

```
self.max_num_running_reqs = self.scheduler_config.max_num_seqs
self.max_num_scheduled_tokens = (
    self.scheduler_config.max_num_scheduled_tokens
    if self.scheduler_config.max_num_scheduled_tokens
    else self.scheduler_config.max_num_batched_tokens
)
self.max_model_len = vllm_config.model_config.max_model_len
```

1.2 schedule() 方法完整逻辑

schedule() 方法是调度器的核心入口，位于 scheduler.py:310-903。

### 1.2.1 方法签名与返回值

```
def schedule(self) -> SchedulerOutput:
```

返回 `SchedulerOutput` 对象，包含本次调度的所有信息。

### 1.2.2 调度算法核心思想 源码注释 (scheduler.py:311-320) 清晰阐述了设计理念:

**NOTE(woosuk) on the scheduling algorithm:** There's no “decoding phase” nor “prefill phase” in the scheduler. Each request just has the `num_computed_tokens` and `num_tokens_with_spec`. At each step, the scheduler tries to assign tokens to the requests so that each request's `num_computed_tokens` can catch up its `num_tokens_with_spec`. This is general enough to cover chunked prefills, prefix caching, speculative decoding, and the “jump decoding” optimization.

核心洞察: 调度器不区分“prefill 阶段”和“decode 阶段”，而是统一处理——每个请求都有 `num_computed_tokens` (已计算 token 数) 和 `num_tokens_with_spec` (目标 token 数)，调度器的目标就是让前者追赶后者。

### 1.2.3 调度流程详解 阶段一: 调度 **RUNNING** 请求 (scheduler.py:345-514)

```
while req_index < len(self.running) and token_budget > 0:
    request = self.running[req_index]

    # 计算需要调度的新 token 数
    num_new_tokens = (
        request.num_tokens_with_spec
        + request.num_output_placeholders
        - request.num_computed_tokens
    )

    # 长预填充阈值限制
    if 0 < self.scheduler_config.long_prefill_token_threshold < num_new_tokens:
        num_new_tokens = self.scheduler_config.long_prefill_token_threshold

    num_new_tokens = min(num_new_tokens, token_budget)

    # 分配 KV 缓存块
    new_blocks = self.kv_cache_manager.allocate_slots(
        request, num_new_tokens,
        num_lookahead_tokens=self.num_lookahead_tokens,
    )

    if new_blocks is None:
        # 需要抢占低优先级请求
        preempted_req = self.running.pop()
        self._preempt_request(preempted_req, scheduled_timestamp)
        preempted_reqs.append(preempted_req)
        continue

    # 成功调度
    scheduled_running_reqs.append(request)
    num_scheduled_tokens[request_id] = num_new_tokens
    token_budget -= num_new_tokens
```

阶段二: 调度 **WAITING** 请求 (scheduler.py:525-804)

```

while (self.waiting or self.skipped_waiting) and token_budget > 0:
    if len(self.running) == self.max_num_running_reqs:
        break

    request_queue = self._select_waiting_queue_for_scheduling()
    request = request_queue.peek_request()

    # 获取已缓存的 token (prefix caching)
    if request.num_computed_tokens == 0:
        new_computed_blocks, num_new_local_computed_tokens = (
            self.kv_cache_manager.get_computed_blocks(request)
        )
        # KVConnector: 获取远程缓存的 token
        if self.connector is not None:
            ext_tokens, load_kv_async = (
                self.connector.get_num_new_matched_tokens(
                    request, num_new_local_computed_tokens
                )
            )

    # 分配 KV 缓存块
    new_blocks = self.kv_cache_manager.allocate_slots(
        request, num_new_tokens,
        num_new_computed_tokens=num_new_local_computed_tokens,
        new_computed_blocks=new_computed_blocks,
    )

    if new_blocks is None:
        break # 无法分配, 停止调度

    # 请求从 WAITING 状态转为 RUNNING
    self.running.append(request)
    request.status = RequestStatus.RUNNING

```

1.2.4 请求队列选择策略 `_select_waiting_queue_for_scheduling()` 方法 (scheduler.py:1529-1539):

```

def _select_waiting_queue_for_scheduling(self) -> RequestQueue | None:
    if self.policy == SchedulingPolicy.FCFS:
        return self.skipped_waiting or self.waiting or None

    # PRIORITY mode: 比较两个队列头部请求的优先级
    if self.waiting and self.skipped_waiting:
        waiting_req = self.waiting.peek_request()
        skipped_req = self.skipped_waiting.peek_request()
        return self.waiting if waiting_req < skipped_req else self.skipped_waiting

    return self.waiting or self.skipped_waiting or None

```

### 1.3 请求状态管理

1.3.1 RequestStatus 枚举 定义于 request.py:310-319:

```

class RequestStatus(enum.IntEnum):
    WAITING = enum.auto()
    WAITING_FOR_STRUCTURED_OUTPUT_GRAMMAR = enum.auto()

```

```
WAITING_FOR_REMOTE_KVS = enum.auto()
WAITING_FOR_STREAMING_REQ = enum.auto()
RUNNING = enum.auto()
PREEMPTED = enum.auto()
# FINISHED_* 状态...
```

1.3.2 状态转换图

```
stateDiagram-v2
    [*] --> WAITING: add_request()
    WAITING --> RUNNING: schedule() 成功
    WAITING --> WAITING_FOR_REMOTE_KVS: KVConnector 异步加载
    WAITING_FOR_REMOTE_KVS --> WAITING: KV 加载完成
    RUNNING --> PREEMPTED: 抢占
    PREEMPTED --> WAITING: 重新入队
    RUNNING --> FINISHED_STOPPED: 生成完成
    RUNNING --> FINISHED_ABORTED: 请求中止
    WAITING --> FINISHED_ABORTED: 请求中止
```

1.4 请求抢占机制

```
_preempt_request() 方法 (scheduler.py:910-930):

def _preempt_request(self, request: Request, timestamp: float) -> None:
    """Preempt a request and put it back to the waiting queue."""
    assert request.status == RequestStatus.RUNNING

    # 释放 KV 缓存块
    self.kv_cache_manager.free(request)
    self.encoder_cache_manager.free(request)

    # 重置请求状态
    request.status = RequestStatus.PREEMPTED
    request.num_computed_tokens = 0
    request.num_preemptions += 1

    # 放回等待队列头部
    self.waiting.prepend_request(request)
```

抢占触发条件: 1. KV 缓存不足, 无法为新请求分配块 2. 优先级调度时, 高优先级请求需要资源

二、Continuous Batching 原理

2.1 静态批处理 vs Continuous Batching

| 特性      | 静态批处理      | Continuous Batching |
|---------|------------|---------------------|
| 批次组成    | 固定请求集合     | 动态增减请求              |
| 请求加入    | 批次开始时一次性加入 | 任意时刻可加入新请求          |
| 请求退出    | 批次结束时统一退出  | 完成即退出               |
| GPU 利用率 | 请求长度差异导致浪费 | 持续保持高利用率            |
| 延迟      | 等待最慢请求完成   | 完成即返回               |

## 2.2 Chunked Prefill 实现

配置参数 (scheduler.py:84):

```
enable_chunked_prefill: bool = True
```

核心机制: 将长 prefill 请求分块处理, 每次调度只处理部分 token。

```
# scheduler.py:636-638
```

```
num_new_tokens = request.num_tokens - num_computed_tokens
threshold = self.scheduler_config.long_prefill_token_threshold
if 0 < threshold < num_new_tokens:
    num_new_tokens = threshold
```

Chunked Prefill 的优势:

1. 降低首 token 延迟: 长请求不会阻塞短请求
2. 提高 GPU 利用率: prefill 和 decode 可以混合执行
3. 更好的公平性: 避免长请求独占资源

## 2.3 Prefill-Decode 混合调度

vLLM v1 不显式区分 prefill 和 decode 阶段, 而是通过 num\_computed\_tokens 统一处理:

```
# scheduler.py:946-948
```

```
request.is_prefill_chunk = request.num_computed_tokens < (
    request.num_tokens + request.num_output_placeholders
)
```

判断逻辑: - is\_prefill\_chunk = True: 请求仍处于 prefill 阶段 - is\_prefill\_chunk = False: 请求处于 decode 阶段

---

## 三、调度策略细节

### 3.1 SchedulingPolicy 枚举

定义于 request\_queue.py:13-17:

```
class SchedulingPolicy(Enum):
    """Enum for scheduling policies."""
    FCFS = "fcfs"
    PRIORITY = "priority"
```

### 3.2 FCFS (First-Come-First-Served) 策略

FCFSRequestQueue 实现 (request\_queue.py:75-128):

```
class FCFSRequestQueue(deque[Request], RequestQueue):
    """A first-come-first-served queue that supports deque operations."""

    def add_request(self, request: Request) -> None:
        self.append(request)

    def pop_request(self) -> Request:
        return self.popleft()

    def peek_request(self) -> Request:
        return self[0]
```

```
def prepend_request(self, request: Request) -> None:
    self.appendleft(request)
```

特点: - 基于 deque 实现, O(1) 的队首/队尾操作 - 按请求到达时间 (arrival\_time) 顺序处理 - 简单高效, 适合大多数场景

### 3.3 PRIORITY 策略

PriorityRequestQueue 实现 (request\_queue.py:131-198):

```
class PriorityRequestQueue(RequestQueue):
    """
    A priority queue that supports heap operations.

    Respects the ordering defined in the Request class, where
    requests with a smaller value of `priority` are processed first.
    If multiple requests have the same priority, the one with the earlier
    `arrival_time` is processed first.
    """

    def __init__(self) -> None:
        self._heap: list[Request] = []

    def add_request(self, request: Request) -> None:
        heapq.heappush(self._heap, request)

    def pop_request(self) -> Request:
        return heapq.heappop(self._heap)
```

Request 比较方法 (request.py:296-307):

```
def __lt__(self, other: "Request") -> bool:
    """
    Compare two requests based on priority, arrival time, and request ID.
    Used in priority scheduling.
    """
    if self.priority != other.priority:
        return self.priority < other.priority
    if self.arrival_time != other.arrival_time:
        return self.arrival_time < other.arrival_time
    if self.request_id != other.request_id:
        return self.request_id < other.request_id
    return id(self) < id(other)
```

优先级排序规则: 1. 首先按 priority 值升序 (值越小优先级越高) 2. 相同优先级按 arrival\_time 升序 3. 相同到达时间按 request\_id 字典序

### 3.4 双策略调度权衡

| 策略       | 优势              | 劣势           | 适用场景      |
|----------|-----------------|--------------|-----------|
| FCFS     | 实现简单、公平性好、延迟可预测 | 无法区分请求重要性    | 通用场景      |
| PRIORITY | 支持业务优先级、SLA 控制  | 可能导致低优先级请求饥饿 | 多租户、付费优先级 |



### 3.5 抢占与恢复机制

抢占场景 (scheduler.py:436-468):

# 当无法分配 KV 缓存块时触发抢占

```
while True:
    new_blocks = self.kv_cache_manager.allocate_slots(...)

    if new_blocks is not None:
        break

    # 抢占最低优先级请求
    if self.policy == SchedulingPolicy.PRIORITY:
        preempted_req = max(
            self.running,
            key=lambda r: (r.priority, r.arrival_time),
        )
    else:
        preempted_req = self.running.pop()

    self._preempt_request(preempted_req, scheduled_timestamp)
```

恢复机制: 被抢占的请求被放回 waiting 队列头部, 下次调度时优先处理。

---

## 四、SchedulerOutput 数据结构

### 4.1 类定义

位于 output.py:180-256:

```
@dataclass
class SchedulerOutput:
    # 新调度的请求列表
    scheduled_new_reqs: list[NewRequestData]

    # 已缓存请求的调度数据
    scheduled_cached_reqs: CachedRequestData

    # req_id -> num_scheduled_tokens
    num_scheduled_tokens: dict[str, int]

    # 总调度 token 数
    total_num_scheduled_tokens: int

    # req_id -> spec_token_ids (推测解码)
    scheduled_spec_decode_tokens: dict[str, list[int]]

    # req_id -> encoder input indices (多模态)
    scheduled_encoder_inputs: dict[str, list[int]]

    # 公共前缀块数 (用于 cascade attention)
    num_common_prefix_blocks: list[int]

    # 已完成请求 ID
    finished_req_ids: set[str]
```

```

# 需释放的 encoder 缓存
free_encoder_mm_hashes: list[str]

# 被抢占请求 ID
preempted_req_ids: set[str] | None = None

# 结构化输出相关
has_structured_output_requests: bool = False
pending_structured_output_tokens: bool = False

# KV Connector 元数据
kv_connector_metadata: KVConnectorMetadata | None = None

```

#### 4.2 NewRequestData 结构

```

@dataclass
class NewRequestData:
    req_id: str
    prompt_token_ids: list[int] | None
    mm_features: list[MultiModalFeatureSpec]
    sampling_params: SamplingParams | None
    pooling_params: PoolingParams | None
    block_ids: tuple[list[int], ...] # KV 缓存块 ID
    num_computed_tokens: int
    lora_request: LoRARequest | None
    prompt_embeds: torch.Tensor | None = None

```

#### 4.3 CachedRequestData 结构

```

@dataclass
class CachedRequestData:
    req_ids: list[str]
    resumed_req_ids: set[str] # 从抢占中恢复的请求
    new_token_ids: list[list[int]] # 仅 PP 使用
    all_token_ids: dict[str, list[int]]
    new_block_ids: list[tuple[list[int], ...] | None]
    num_computed_tokens: list[int]
    num_output_tokens: list[int]

```

#### 4.4 下游消费流程

```

sequenceDiagram
    participant Scheduler
    participant SchedulerOutput
    participant ModelRunner
    participant Worker

    Scheduler->>SchedulerOutput: schedule() 返回
    SchedulerOutput->>ModelRunner: 传递调度结果
    ModelRunner->>ModelRunner: 解析 NewRequestData
    ModelRunner->>ModelRunner: 更新 CachedRequestData
    ModelRunner->>Worker: 准备模型输入
    Worker->>Worker: 执行 forward

```

```
Worker-->>Scheduler: ModelRunnerOutput
Scheduler->>Scheduler: update_from_output()
```

---

## 五、AsyncScheduler 异步调度变体

### 5.1 类定义

位于 `async_scheduler.py:12-60`:

```
class AsyncScheduler(Scheduler):
    def __init__(self, *args, **kwargs) -> None:
        super().__init__(*args, **kwargs)
        # 可复用的推测解码占位符
        self._spec_token_placeholders: list[int] = [-1] * self.num_spec_tokens
```

### 5.2 核心差异

异步调度启用条件 (`scheduler.py:168-176`):

```
def get_scheduler_cls(self) -> type["SchedulerInterface"]:
    if self.scheduler_cls is None:
        if self.async_scheduling:
            from vllm.v1.core.sched.async_scheduler import AsyncScheduler
            return AsyncScheduler
        from vllm.v1.core.sched.scheduler import Scheduler
        return Scheduler
```

### 5.3 关键方法重写

`**_update_after_schedule()` (`async_scheduler.py:18-35`):

```
def _update_after_schedule(self, scheduler_output: SchedulerOutput) -> None:
    super()._update_after_schedule(scheduler_output)
    spec_decode_tokens = scheduler_output.scheduled_spec_decode_tokens

    for req_id in scheduler_output.num_scheduled_tokens:
        request = self.requests[req_id]
        if request.is_prefill_chunk:
            continue

        # 异步调度: 预先为下一个 token 分配占位符
        cur_num_spec_tokens = len(spec_decode_tokens.get(req_id, ()))
        request.num_output_placeholders += 1 + cur_num_spec_tokens
        request.spec_token_ids = self._spec_token_placeholders
```

`**_update_request_with_output()` (`async_scheduler.py:37-60`):

```
def _update_request_with_output(
    self, request: Request, new_token_ids: list[int]
) -> tuple[list[int], bool]:
    # 处理强制抢占时丢弃异步 token
    if request.discard_latest_async_tokens:
        request.discard_latest_async_tokens = False
        return [], False

    new_token_ids, stopped = super()._update_request_with_output(
```

```

        request, new_token_ids
    )

    # 更新占位符计数
    request.num_output_placeholders -= len(new_token_ids)

    # 缓存新 token 的 KV 块
    if status_before_update == RequestStatus.RUNNING:
        self.kv_cache_manager.cache_blocks(
            request, request.num_computed_tokens - request.num_output_placeholders
        )

    return new_token_ids, stopped

```

#### 5.4 异步调度优势

1. 减少 GPU 空闲: 在等待模型输出时预先调度下一步
2. 更好的流水线: 重叠调度计算与模型执行
3. 推测解码优化: 提前准备 draft token 占位符

## 六、调度决策流程图

flowchart TD

```

Start([schedule 开始]) --> Init[初始化调度变量]
Init --> CheckPause{检查暂停状态}

CheckPause -->|PAUSED_ALL| ReturnEmpty[返回空 SchedulerOutput]
CheckPause -->|UNPAUSED/PAUSED_NEW| ScheduleRunning[调度 RUNNING 请求]

ScheduleRunning --> LoopRunning{还有 RUNNING<br/> 且 token_budget > 0?}
LoopRunning -->|是| CalcTokens[计算 num_new_tokens]
CalcTokens --> AllocBlocks{分配 KV 块成功?}

AllocBlocks -->|是| AddScheduled[加入调度列表]
AddScheduled --> LoopRunning

AllocBlocks -->|否| Preempt[抢占低优先级请求]
Preempt --> AllocBlocks

LoopRunning -->|否| CheckPreempted{有被抢占请求?}
CheckPreempted -->|是| ScheduleWaiting
CheckPreempted -->|否 且 UNPAUSED| ScheduleWaiting

ScheduleWaiting --> LoopWaiting{还有 WAITING<br/> 且 token_budget > 0?}
LoopWaiting -->|是| SelectQueue[选择队列]
SelectQueue --> PeekReq[查看队首请求]
PeekReq --> CheckBlocked{请求被阻塞?}

CheckBlocked -->|是| TryPromote[尝试提升状态]
TryPromote --> CheckBlocked

CheckBlocked -->|否| GetCached[获取已缓存 token]
GetCached --> AllocNewBlocks{分配新 KV 块}

```

```

AllocNewBlocks -->| 成功 | MoveToRunning[移至 RUNNING]
MoveToRunning --> LoopWaiting

AllocNewBlocks -->| 失败 | StopWaiting[停止调度]

LoopWaiting -->| 否 | BuildOutput[构建 SchedulerOutput]
StopWaiting --> BuildOutput
ReturnEmpty --> End([返回])
BuildOutput --> End

style Start fill:#e1f5fe
style End fill:#e8f5e9
style AllocBlocks fill:#fff3e0
style AllocNewBlocks fill:#fff3e0
style Preempt fill:#ffebee

```

## 七、关键配置参数

### 7.1 SchedulerConfig 核心参数

| 参数                           | 类型   | 默认值    | 说明                 |
|------------------------------|------|--------|--------------------|
| max_num_batched_tokens       | int  | 2048   | 单次迭代最大处理 token 数   |
| max_num_scheduled_tokens     | int  | None   | 调度器单次最大发出 token 数  |
| max_num_seqs                 | int  | 128    | 单次迭代最大序列数          |
| enable_chunked_prefill       | bool | True   | 启用 chunked prefill |
| long_prefill_token_threshold | int  | 0      | 长预填充阈值             |
| policy                       | str  | "fcfs" | 调度策略               |
| async_scheduling             | bool | None   | 启用异步调度             |

### 7.2 参数验证逻辑

```

# scheduler.py:259-278
def verify_max_model_len(self, max_model_len: int):
    if (
        self.max_num_batched_tokens < max_model_len
        and not self.enable_chunked_prefill
    ):
        raise ValueError(
            f"max_num_batched_tokens ({self.max_num_batched_tokens}) is "
            f"smaller than max_model_len ({max_model_len})."
        )

```

## 八、总结

vLLM v1 的调度系统通过以下核心设计实现了高效的 LLM 推理服务:

1. 统一的调度抽象: 不区分 prefill/decode 阶段, 通过 `num_computed_tokens` 统一处理
2. **Continuous Batching**: 动态增减请求, 最大化 GPU 利用率

3. **Chunked Prefill**: 长请求分块处理, 降低首 token 延迟
4. 灵活的调度策略: 支持 FCFS 和 PRIORITY 两种策略
5. 抢占与恢复机制: 在资源不足时优雅降级
6. 异步调度优化: 减少 GPU 空闲时间

这些设计使得 vLLM 能够在高吞吐量和低延迟之间取得良好平衡, 适用于各种生产场景。

## PagedAttention 核心创新深度分析

### 定位

**PagedAttention** 是 **vLLM** 的核心创新, 它通过借鉴操作系统的虚拟内存管理思想, 将 KV cache 以固定大小的 Block (页) 为单位进行分配和管理, 从而解决了传统 LLM 推理系统中 KV cache 内存利用率低的根本问题。PagedAttention 不仅大幅提升了内存效率 (2-4x), 还天然支持内存共享 (Memory Sharing) 和前缀缓存 (Prefix Caching) 等高级优化。

### 核心价值总览

graph TB

subgraph 传统问题 [" 传统 KV Cache 管理问题"]

A1[" 固定分配策略"] --> B1[" 内部碎片 <br/> (预留空间未使用)"]

A2[" 外部碎片 <br/> (无法合并的空闲块)"]

A3[" 内存利用率低 <br/>&lt; 50%"]

B1 --> C1["GPU 资源浪费严重"]

B2 --> C1

A3 --> C1

end

subgraph PagedAttention 方案 [" PagedAttention 创新方案"]

D1[" 页式存储管理 <br/>Block = 16 tokens"] --> E1[" 按需动态分配"]

D2[" 逻辑 → 物理映射 <br/>BlockTable"] --> E2[" 消除内部碎片"]

D3[" 全局内存池 <br/>BlockPool"] --> E3[" 统一资源调度"]

E1 --> F1[" 内存利用率 &gt; 90%"]

E2 --> F1

E3 --> F1

end

subgraph 核心优势 [" 核心优势"]

G1["2-4x 内存节省"]

G2["Copy-on-Write 共享"]

G3[" 自动 Prefix Caching"]

G4[" 支持多种 Attention 类型"]

F1 --> G1

D1 --> G2

D2 --> G3

D3 --> G4

end

style 传统问题 fill:#ffebee,stroke:#f44336,stroke-width:2px,color:#000

style PagedAttention 方案 fill:#f3e5f5,stroke:#9c27b0,stroke-width:2px,color:#000

style 核心优势 fill:#e8f5e9,stroke:#4caf50,stroke-width:2px,color:#000

## 一、问题背景：传统 KV Cache 的内存困境

### 1.1 KV Cache 的本质

在自回归语言模型推理中，每个 token 的生成需要访问之前所有已生成 token 的 Key-Value (KV) 状态。这些 KV 状态必须被缓存以避免重复计算，形成 **KV Cache**。

内存占用公式：

$$\text{KV Cache 大小} = 2 \times \text{num\_layers} \times \text{num\_heads} \times \text{head\_size} \times \text{seq\_len} \times \text{dtype\_size}$$

对于典型配置 (Llama-7B: 32 层, 32 头, 128 维, fp16) : - 序列长度 2048: ~1GB/request - 序列长度 8192: ~4GB/request - 序列长度 32768: ~16GB/request

### 1.2 传统方案的三大缺陷

缺陷一：固定分配导致的内部碎片 传统系统为每个请求 预分配最大序列长度 的连续内存：

# 伪代码：传统固定分配方式

```
max_seq_len = 2048
```

```
actual_seq_len = 100 # 实际只用了 100 个 token
```

```
allocated_memory = max_seq_len * memory_per_token # 分配了 2048 个 slot
```

```
wasted_memory = (max_seq_len - actual_seq_len) * memory_per_token # 浪费 95%!
```

实际场景示例： | 场景 | 最大序列长 | 平均序列长 | 利用率 | |——|———|———|———| | 对话系统 | 4096 | 512 | 12.5% | | 代码补全 | 8192 | 256 | 3.1% | | 长文档 QA | 32768 | 4096 | 12.5% |

缺陷二：外部碎片问题 即使总空闲内存足够，但由于 地址不连续，无法满足新的分配请求：

graph LR

subgraph 内存布局 [" 传统方案的内存碎片化"]

A["Req A<br/> 已释放 <br/>(256 slots)"] --- B["Req B<br/> 运行中 <br/>(512 slots)"] --- C["Req C<br/> 运行中 <br/>(512 slots)"]

end

subgraph 新请求 [" 新请求需求"]

E[" 需要 512 连续 slots"]

end

A -.->|" 不连续"| E

C -.->|" 不连续"| E

style 内存布局 fill:#fff3e0,stroke:#ff9800,stroke-width:2px

style 新请求 fill:#ffcdd2,stroke:#f44336,stroke-width:2px

缺陷三：无法共享相同前缀 多个请求可能具有相同的 prompt 前缀（如系统提示词），但传统方案每个请求都维护独立的副本：

# 场景：10 个请求共享相同的 system prompt (1024 tokens)

```
num_requests = 10
```

```
shared_prefix_length = 1024
```

```
total_wasted = num_requests * shared_prefix_length * memory_per_token # 重复存储 10 份!
```

## 二、PagedAttention 设计思想

### 2.1 操作系统虚拟内存类比

PagedAttention 深度借鉴了操作系统中的 虚拟内存分页机制：

```
graph TB
    subgraph OS_虚拟内存 ["操作系统虚拟内存（类比）"]
        direction LR
        OS_VA["虚拟地址空间 <br/>(进程视角)"] --> OS_PT["页表 <br/>(Page Table)"] --> OS_PA["物理内存帧 <br/>(物理地址)"]
        OS_VA --> |"逻辑页面"| OS_PT
        OS_PT --> |"物理帧号"| OS_PA
    end

    subgraph PagedAttention["vLLM PagedAttention"]
        direction LR
        PA_LA["逻辑序列 <br/>(Request 视角)"] --> PA_BT["Block Table<br/>(映射表)"] --> PA_BP["物理 Block ID <br/>(GPU 内存地址)"]
        PA_LA --> |"逻辑 Block ID"| PA_BT
        PA_BT --> |"物理 Block ID"| PA_BP
    end

    OS_VA -. -> |"对应"| PA_LA
    OS_PT -. -> |"对应"| PA_BT
    OS_PA -. -> |"对应"| PA_BP

    style OS_虚拟内存 fill:#e3f2fd,stroke:#2196f3,stroke-width:2px
    style PagedAttention fill:#f3e5f5,stroke:#9c27b0,stroke-width:2px
```

核心概念映射：

| 操作系统概念          | PagedAttention 概念 | 说明              |
|-----------------|-------------------|-----------------|
| 页面 (Page)       | Block (块)         | 固定大小的内存单元       |
| 虚拟地址            | 逻辑 Token 位置       | Request 视角的连续地址 |
| 物理帧 (Frame)     | 物理 Block ID       | GPU 内存中的实际位置    |
| 页表 (Page Table) | Block Table       | 逻辑 → 物理映射       |
| 页面置换算法          | 缓存驱逐策略            | LRU/LFU 变体      |

2.2 Block（页）的概念与设计

Block Size 选择 vLLM 默认使用 16 tokens/block，这个值经过精心权衡：

```
# 来源：/workspace/vllm/v1/kv_cache_interface.py - KVCacheSpec 定义
@dataclass(frozen=True)
class KVCacheSpec:
    block_size: int # 每个 block 包含的 token 数量（默认 16）
```

选择依据：

```
graph TB
    subgraph 小BlockSize["小 Block Size (如 4)"]
        S1["内部碎片少"]
        S2["Block Table 过大"]
        S3["映射开销高"]
        S4["Prefix Cache 粒度过细"]
    end

    subgraph 大BlockSize["大 Block Size (如 64)"]
        L1["内部碎片多"]
        L2["Block Table 紧凑"]
        L3["映射开销低"]
        L4["Prefix Cache 粒度粗"]
    end
```



```

end

subgraph 最优选择 [" 最优选择: Block Size = 16"]
    01[" 平衡内部碎片 (< 6.25%)"]
    02[" 合理的 Block Table 大小"]
    03[" 适中的映射开销"]
    04[" 适合大多数场景"]
end

style 小 BlockSize fill:#ffebee,stroke:#f44336
style 大 BlockSize fill:#fff3e0,stroke:#ff9800
style 最优选择 fill:#e8f5e9,stroke:#4caf50,stroke-width:3px

```

内部碎片计算:

最坏情况内部碎片 =  $(\text{block\_size} - 1) / \text{block\_size} = 15/16 = 6.25\%$   
 平均内部碎片  $\text{block\_size} / 2 / \text{block\_size} = 50\%/16 = 3.125\%$

### 2.3 逻辑地址 → 物理地址映射 (BlockTable)

BlockTable 是连接逻辑序列和物理内存的核心数据结构:

*# 来源: /workspace/vllm/v1/worker/block\_table.py - BlockTable 类定义*

```

class BlockTable:
    def __init__(
        self,
        block_size: int,                # Block 大小 (tokens)
        max_num_reqs: int,              # 最大并发请求数
        max_num_blocks_per_req: int,    # 每请求最大 Block 数
        max_num_batched_tokens: int,    # 最大 batch token 数
        pin_memory: bool,               # 是否 pin memory
        device: torch.device,           # 设备类型
        kernel_block_size: int,         # kernel 使用的 block size
        cp_kv_cache_interleave_size: int, # CP 交错大小
    ):
        # 核心: block_table 存储逻辑 → 物理映射
        # 维度: [max_num_reqs, max_num_blocks_per_req]
        self.block_table = self._make_buffer(
            self.max_num_reqs,
            self.max_num_blocks_per_req,
            dtype=torch.int32
        )
        # 记录每行实际使用的 block 数量
        self.num_blocks_per_row = np.zeros(max_num_reqs, dtype=np.int32)

        # slot_mapping: 将每个 token 映射到具体的物理 slot
        # 用于 attention kernel 直接寻址
        self.slot_mapping = self._make_buffer(
            self.max_num_batched_tokens,
            dtype=torch.int64
        )
    )

```

映射过程可视化:

```

graph TB
    subgraph 请求视图 [" 请求的逻辑视图 (连续) "]
        LA0["Token 0-15<br/>Logical Block 0"]
    end

```

```

    LA1["Token 16-31<br/>Logical Block 1"]
    LA2["Token 32-47<br/>Logical Block 2"]
    LA3["Token 48-63<br/>Logical Block 3"]
end

subgraph BlockTable 映射 ["Block Table 映射表"]
    BT0["Entry 0 → Physical Block 5"]
    BT1["Entry 1 → Physical Block 12"]
    BT2["Entry 2 → Physical Block 3"]
    BT3["Entry 3 → Physical Block 8"]
end

subgraph 物理内存 [" 物理 GPU 内存 (分散) "]
    PB5["Physical Block 5<br/>[Token 0-15]"]
    PB12["Physical Block 12<br/>[Token 16-31]"]
    PB3["Physical Block 3<br/>[Token 32-47]"]
    PB8["Physical Block 8<br/>[Token 48-63]"]
end

LA0 -->|" 查询"| BT0
LA1 -->|" 查询"| BT1
LA2 -->|" 查询"| BT2
LA3 -->|" 查询"| BT3

BT0 --> PB5
BT1 --> PB12
BT2 --> PB3
BT3 --> PB8

style 请求视图 fill:#e3f2fd,stroke:#2196f3,stroke-width:2px
style BlockTable 映射 fill:#fff3e0,stroke:#ff9800,stroke-width:2px
style 物理内存 fill:#e8f5e9,stroke:#4caf50,stroke-width:2px

```

### Slot Mapping 计算核心逻辑 (Triton Kernel):

```

# 来源: /workspace/vllm/v1/worker/block_table.py - _compute_slot_mapping_kernel
@triton.jit
def _compute_slot_mapping_kernel(
    num_tokens,
    max_num_tokens,
    query_start_loc_ptr,      # [num_reqs + 1], int32
    positions_ptr,            # [num_tokens], int64
    block_table_ptr,          # [max_num_reqs, max_num_blocks_per_req], int32
    block_table_stride,       # max_num_blocks_per_req
    block_size,
    slot_mapping_ptr,          # [max_num_tokens], int64
    TOTAL_CP_WORLD_SIZE: tl.constexpr,
    TOTAL_CP_RANK: tl.constexpr,
    CP_KV_CACHE_INTERLEAVE_SIZE: tl.constexpr,
    PAD_ID: tl.constexpr,
    BLOCK_SIZE: tl.constexpr,
):
    req_idx = tl.program_id(0)
    start_idx = tl.load(query_start_loc_ptr + req_idx).to(tl.int64)
    end_idx = tl.load(query_start_loc_ptr + req_idx + 1).to(tl.int64)

```

```

virtual_block_size = block_size * TOTAL_CP_WORLD_SIZE
row_offset = req_idx * block_table_stride

for i in range(start_idx, end_idx, BLOCK_SIZE):
    offsets = i + tl.arange(0, BLOCK_SIZE)
    mask = offsets < end_idx
    pos = tl.load(positions_ptr + offsets, mask=mask, other=0)

    # 1. 计算逻辑 block 编号
    block_indices = pos // virtual_block_size

    # 2. 从 Block Table 查找物理 block 编号
    block_numbers = tl.load(
        block_table_ptr + row_offset + block_indices
    ).to(tl.int64)

    # 3. 计算 block 内偏移
    virtual_block_offsets = pos - block_indices * virtual_block_size

    # 4. 处理 Context Parallelism 的数据分布
    is_local = (
        virtual_block_offsets // CP_KV_CACHE_INTERLEAVE_SIZE
    ) % TOTAL_CP_WORLD_SIZE == TOTAL_CP_RANK
    local_block_offsets = (
        virtual_block_offsets // (TOTAL_CP_WORLD_SIZE * CP_KV_CACHE_INTERLEAVE_SIZE)
    ) * CP_KV_CACHE_INTERLEAVE_SIZE + (
        virtual_block_offsets % CP_KV_CACHE_INTERLEAVE_SIZE
    )

    # 5. 组合最终的物理 slot ID
    slot_ids = block_numbers * block_size + local_block_offsets
    slot_ids = tl.where(is_local, slot_ids, PAD_ID)
    tl.store(slot_mapping_ptr + offsets, slot_ids, mask=mask)

```

## 2.4 Block Pool (物理内存池)

BlockPool 是所有物理 Block 的集中管理者:

# 来源: `/workspace/vllm/v1/core/block_pool.py` - *BlockPool* 类定义

```

class BlockPool:
    """BlockPool 管理 KVCacheBlocks。
    提供 allocate、free 和 cache 方法。free_block_queue 存储空闲 blocks
    (包括缓存启用时的驱逐候选)。cached_block_hash_to_block 实现
    block hash 到 cached block 的映射, 支持前缀缓存查找。
    """
    def __init__(
        self,
        num_gpu_blocks: int,           # GPU 上总的 block 数量
        enable_caching: bool,          # 是否启用前缀缓存
        hash_block_size: int,          # hash 计算的粒度
        enable_kv_cache_events: bool=False, # 是否启用事件追踪
        metrics_collector=None,        # 性能指标收集器
    ):
        assert isinstance(num_gpu_blocks, int) and num_gpu_blocks > 0

```

```

self.num_gpu_blocks = num_gpu_blocks
self.enable_caching = enable_caching
self.hash_block_size = hash_block_size

# 所有 KV-cache blocks (预分配)
self.blocks: list[KVCacheBlock] = [
    KVCacheBlock(idx) for idx in range(num_gpu_blocks)
]

# 空闲 block 队列 (双向链表, 支持高效插入/删除)
self.free_block_queue = FreeKVCacheBlockQueue(self.blocks)

# 前缀缓存查找表: {block_hash -> KVCacheBlock}
self.cached_block_hash_to_block: BlockHashToBlockMap = BlockHashToBlockMap()

# 特殊的 null block (占位符, block_id=0)
self.null_block = self.free_block_queue.popleft()
self.null_block.is_null = True

```

Block 数据结构:

```

# 来源: /workspace/vllm/v1/core/kv_cache_utils.py - KVCacheBlock 定义
@dataclass
class KVCacheBlock:
    block_id: int          # 物理块编号
    ref_cnt: int = 0       # 引用计数 (用于共享和回收)
    block_hash: BlockHashWithGroupId | None = None # 前缀缓存的 hash 值
    is_null: bool = False  # 是否是 null block (占位符)

```

---

### 三、关键实现深度解析

#### 3.1 KVCacheManager: 统一入口与协调者

KVCacheManager 是对外暴露的核心接口, 负责协调多个 SingleTypeKVCacheManager:

```

# 来源: /workspace/vllm/v1/core/kv_cache_manager.py - KVCacheManager 类
class KVCacheManager:
    def __init__(
        self,
        kv_cache_config: KVCacheConfig,
        max_model_len: int,
        hash_block_size: int,
        max_num_batched_tokens: int | None = None,
        enable_caching: bool = True,
        use_eagle: bool = False,
        log_stats: bool = False,
        ...
    ) -> None:
        self.max_model_len = max_model_len
        self.enable_caching = enable_caching
        self.use_eagle = use_eagle

# 创建 coordinator 来管理多个 KV cache group
self.coordinator = get_kv_cache_coordinator(
    kv_cache_config=kv_cache_config,

```

```

        max_model_len=self.max_model_len,
        max_num_batched_tokens=max_num_batched_tokens,
        use_eagle=self.use_eagle,
        enable_caching=self.enable_caching,
        hash_block_size=hash_block_size,
        ...
    )

    # 核心组件
    self.num_kv_cache_groups = len(kv_cache_config.kv_cache_groups)
    self.block_pool = self.coordinator.block_pool # 全局内存池
    self.kv_cache_config = kv_cache_config

    # 预创建空对象（避免 GC 开销）
    self.empty_kv_cache_blocks = KVCacheBlocks(
        tuple() for _ in range(self.num_kv_cache_groups))
    )

```

**3.1.1 allocate\_slots():** 核心分配方法 这是最复杂也最重要的方法，实现了完整的分配流程:

# 来源: `/workspace/vllm/v1/core/kv_cache_manager.py - allocate_slots` 方法

```

def allocate_slots(
    self,
    request: Request,
    num_new_tokens: int,
    num_new_computed_tokens: int = 0,
    new_computed_blocks: KVCacheBlocks | None = None,
    num_lookahead_tokens: int = 0,
    num_external_computed_tokens: int = 0,
    delay_cache_blocks: bool = False,
    num_encoder_tokens: int = 0,
    full_sequence_must_fit: bool = False,
) -> KVCacheBlocks | None:
    """ 为请求分配 slots 以追加新 token。

    分配包含三个阶段:
    1. 释放 comp 中不必要的 blocks 并检查是否有足够空闲 blocks
    2. 处理 prefix tokens (comp + new_comp + ext_comp):
       - 释放不必要的 blocks (如在滑动窗口外)
       - 为 sliding window 内的 ext_comp tokens 分配新 blocks
    3. 分配待计算的 tokens 的新 blocks (new + lookahead)

    Blocks 布局示意:
    -----
    | < comp > | < new_comp > | < ext_comp > | < new > | < lookahead > |
    -----
                                         | < to be computed > |
    """

    # 参数验证
    if num_new_tokens == 0 and num_external_computed_tokens == 0:
        raise ValueError("num_new_tokens must be greater than 0...")

    # 计算各种 token 数量

```

```

num_local_computed_tokens = (
    request.num_computed_tokens + num_new_computed_tokens
)
total_computed_tokens = min(
    num_local_computed_tokens + num_external_computed_tokens,
    self.max_model_len,
)

# ===== 阶段 0: 可选的全序列适配检查 =====
if full_sequence_must_fit:
    num_blocks_to_allocate = self.coordinator.get_num_blocks_to_allocate(...)
    if num_blocks_to_allocate > self.block_pool.get_num_free_blocks():
        return None # 无法容纳完整序列

# ===== 阶段 1: 释放滑动窗口外的 blocks =====
self.coordinator.remove_skipped_blocks(
    request.request_id, total_computed_tokens
)

# ===== 阶段 2: 计算需要的 block 数量 =====
num_blocks_to_allocate = self.coordinator.get_num_blocks_to_allocate(...)

# 检查是否有足够的空闲 blocks
if num_blocks_to_allocate > self.block_pool.get_num_free_blocks():
    return None # 分配失败

# ===== 阶段 3: 处理前缀缓存命中的 blocks =====
if (new_computed_block_list is not self.empty_kv_cache_blocks.blocks
    or num_external_computed_tokens > 0):
    self.coordinator.allocate_new_computed_blocks(
        request_id=request.request_id,
        new_computed_blocks=new_computed_block_list,
        num_local_computed_tokens=num_local_computed_tokens,
        num_external_computed_tokens=num_external_computed_tokens,
    )

# ===== 阶段 4: 分配新 blocks =====
new_blocks = self.coordinator.allocate_new_blocks(
    request.request_id,
    num_tokens_need_slot,
    num_tokens_main_model,
    num_encoder_tokens,
)

# ===== 阶段 5: 缓存已完成的 blocks =====
if not self.enable_caching or delay_cache_blocks:
    return self.create_kv_cache_blocks(new_blocks)

# 只缓存 "finalized" 的 tokens (排除 draft tokens)
num_tokens_to_cache = min(
    total_computed_tokens + num_new_tokens,
    request.num_tokens,
)
self.coordinator.cache_blocks(request, num_tokens_to_cache)

```

```
return self.create_kv_cache_blocks(new_blocks)
```

分配流程图:

flowchart TD

```
A["allocate_slots() 被调用"] --> B{"full_sequence_must_fit?"}
B -->|Yes| C["检查完整序列是否适配"]
C -->|不够空间| X["返回 None"]
C -->|足够| D["继续"]
B -->|No| D

D --> E["remove_skipped_blocks()<br/> 释放滑动窗口外的 blocks"]

E --> F["get_num_blocks_to_allocate()<br/> 计算所需 block 数"]

F --> G{"空闲 blocks 足够?"}
G -->|No| X
G -->|Yes| H{"有前缀缓存命中?"}

H -->|Yes| I["allocate_new_computed_blocks()<br/> 复用缓存 blocks"]
H -->|No| J["跳过"]

I --> J

J --> K["allocate_new_blocks()<br/> 从 BlockPool 分配新 blocks"]

K --> L{"enable_caching?"}
L -->|No| M["返回新分配的 blocks"]
L -->|Yes| N{"delay_cache?"}
N -->|Yes| M
N -->|No| O["cache_blocks()<br/> 缓存完成的 blocks"]
O --> M
```

```
style A fill:#e3f2fd,stroke:#2196f3
```

```
style X fill:#ffcdd2,stroke:#f44336
```

```
style M fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
```

### 3.1.2 free(): 释放方法

# 来源: `/workspace/vllm/v1/core/kv_cache_manager.py` - `free` 方法

```
def free(self, request: Request) -> None:
```

```
    """ 释放请求分配的所有 blocks。
```

```
    按 reverse order 释放, 这样尾部 blocks 先被释放,
    这在启用 caching 时有利于缓存驱逐策略。
    """
```

```
    self.coordinator.free(request.request_id)
```

释放流程详解:

sequenceDiagram

```
participant Scheduler as 调度器
participant KVCM as KVCacheManager
participant Coord as Coordinator
participant STMgr as SingleTypeMgr
```

```

participant BP as BlockPool

Scheduler->>KVCM: free(request)
KVCM->>Coord: free(request_id)

loop 每个 KV Cache Group
    Coord->>STMgr: free(request_id)
    STMgr->>STMgr: 获取 req_to_blocks[request_id]
    STMgr->>STMgr: reverse(blocks) [逆序]
    STMgr->>BP: free_blocks(ordered_blocks)

    loop 每个 block
        BP->>BP: block.ref_cnt -= 1
        Note right of BP: 如果 ref_cnt == 0 且不是 null_block,<br/> 则加入 free_block_queue
    end

    STMgr->>STMgr: 清理 num_cached_block
end

Note over Scheduler,BP: 释放完成, blocks 可被重新分配

```

**3.1.3 can\_allocate():** 容量检查 虽然源码中没有显式的 can\_allocate() 方法, 但容量检查逻辑内嵌在 allocate\_slots() 中:

```

# 来源: /workspace/vllm/v1/core/kv_cache_manager.py - allocate_slots 中的检查
if num_blocks_to_allocate > self.block_pool.get_num_free_blocks():
    return None # 返回 None 表示无法分配

```

容量检查的关键点:

1. 实时检查: 每次分配前都查询当前可用 blocks
2. 全局视角: BlockPool.get\_num\_free\_blocks() 提供全局可用数量
3. 保守估计: 考虑了可能的缓存驱逐开销

## 3.2 BlockPool: 内存池的精细管理

### 3.2.1 get\_new\_blocks(): 分配新 blocks

```

# 来源: /workspace/vllm/v1/core/block_pool.py - get_new_blocks 方法
def get_new_blocks(self, num_blocks: int) -> list[KVCacheBlock]:
    """ 从空闲 block 池中获取新 blocks。

    注意: 此函数不检查 block cache。
    """
    if num_blocks > self.get_num_free_blocks():
        raise ValueError(f"Cannot get {num_blocks} free blocks from the pool")

    # 从队列头部取出 blocks (O(1) 操作)
    ret: list[KVCacheBlock] = self.free_block_queue.popleft_n(num_blocks)

    # 初始化新分配的 blocks
    if self.enable_caching:
        for block in ret:
            # 如果该 block 在缓存中, 先驱逐
            self._maybe_evict_cached_block(block)
            assert block.ref_cnt == 0

```



```

        block.ref_cnt += 1 # 引用计数 +1
    if self.metrics_collector:
        self.metrics_collector.on_block_allocated(block)
else:
    for block in ret:
        assert block.ref_cnt == 0
        block.ref_cnt += 1
        if self.metrics_collector:
            self.metrics_collector.on_block_allocated(block)

    return ret

```

关键设计点:

1. **O(1)** 分配: 使用双向链表实现 `free_block_queue`, 头部弹出为常数时间
2. 自动驱逐: 如果取出的 `block` 正好是被缓存的 `block`, 自动执行驱逐
3. 引用计数: 维护准确的引用计数以支持共享

### 3.2.2 `_maybe_evict_cached_block()`: 智能驱逐

# 来源: `/workspace/vllm/v1/core/block_pool.py` - `_maybe_evict_cached_block` 方法

```

def _maybe_evict_cached_block(self, block: KVCacheBlock) -> bool:
    """ 如果 block 在 cached_block_hash_to_block 中,
    则重置其 hash 元数据并从缓存中驱逐。
    """
    # 清理指标跟踪 (防止泄漏)
    if self.metrics_collector:
        self.metrics_collector.on_block_evicted(block)

    block_hash = block.block_hash
    if block_hash is None:
        return False # 没有 hash, 无需驱逐

    # 尝试从缓存中移除
    if self.cached_block_hash_to_block.pop(block_hash, block.block_id) is None:
        return False # 不在缓存中

    # 重置 hash 元数据
    block.reset_hash()

    # 记录事件 (如果启用)
    if self.enable_kv_cache_events:
        self.kv_event_queue.append(
            BlockRemoved(
                block_hashes=[maybe_convert_block_hash(get_block_hash(block_hash))],
                medium=MEDIUM_GPU,
                group_idx=get_group_id(block_hash),
            )
        )
    return True

```

### 3.2.3 `touch()`: 增加引用计数 (用于共享)

# 来源: `/workspace/vllm/v1/core/block_pool.py` - `touch` 方法

```

def touch(self, blocks: Sequence[KVCacheBlock]) -> None:
    """touch 一个 block 会将其引用计数增加 1,

```

并可能将其从 *free queue* 中移除。

当一个 *block* 被另一个具有相同前缀的请求命中时使用。

```
"""
for block in blocks:
    # ref_cnt==0 表示 block 在 free list 中 (即驱逐候选),
    # 所以将其移除
    if block.ref_cnt == 0 and not block.is_null:
        self.free_block_queue.remove(block)
    block.ref_cnt += 1
    if self.metrics_collector:
        self.metrics_collector.on_block_accessed(block)
```

### 3.2.4 free\_blocks(): 释放 blocks 回池

# 来源: /workspace/vllm/v1/core/block\_pool.py - free\_blocks 方法

```
def free_blocks(self, ordered_blocks: Iterable[KVCacheBlock]) -> None:
    """ 释放一系列 blocks。blocks 应按驱逐优先级排序,
    第一个 block 会被首先驱逐。
    """
    blocks_list = list(ordered_blocks)

    # 减少所有 blocks 的引用计数
    for block in blocks_list:
        block.ref_cnt -= 1

    # 将 ref_cnt==0 且不是 null_block 的 blocks 加入空闲队列
    self.free_block_queue.append_n([
        block for block in blocks_list
        if block.ref_cnt == 0 and not block.is_null
    ])
```

BlockPool 状态转换图:

stateDiagram-v2

```
[*] --> Free: 初始化
Free --> Allocated: get_new_blocks()
Allocated --> InUse: ref_cnt >= 1
InUse --> Cached: cache_full_blocks()
Cached --> InUse: touch() [被其他请求共享]

InUse --> Free: free_blocks() [ref_cnt == 0]
Cached --> Evicted: _maybe_evict_cached_block()
Evicted --> Free: ref_cnt == 0
Cached --> Free: free_blocks() [ref_cnt == 0]

note right of Free: 在 free_block_queue 中
note right of Allocated: 刚从队列取出
note right of InUse: 被 1+ 个请求使用
note right of Cached: 已完成且在 hash 表中
note right of Evicted: hash 已清除, 等待回收到 Free
```

### 3.3 SingleTypeKVCacheManager: 特定注意力类型的精细管理

SingleTypeKVCacheManager 是抽象基类, 针对不同注意力类型有不同实现:

```

# 来源: /workspace/vllm/v1/core/single_type_kv_cache_manager.py - 类层次结构
class SingleTypeKVCacheManager(ABC):
    """ 处理特定类型注意力层的 KV cache 管理逻辑的抽象基类。 """

    def __init__(self, kv_cache_spec, block_pool, ...):
        self.block_size = kv_cache_spec.block_size
        self.block_pool = block_pool
        self.enable_caching = enable_caching

        # 核心数据结构: request_id -> 分配的 blocks 列表
        self.req_to_blocks: defaultdict[str, list[KVCacheBlock]] = defaultdict(list)

        # 记录每个 request 已缓存的 block 数量
        self.num_cached_block: dict[str, int] = {}

    @abstractmethod
    def find_longest_cache_hit(cls, block_hashes, max_length, ...):
        """ 查找最长缓存命中前缀 """
        raise NotImplementedError

    @abstractmethod
    def get_num_common_prefix_blocks(self, running_request_id):
        """ 获取公共前缀 block 数量 """
        raise NotImplementedError

```

具体实现类:

| Manager 类                    | 适用场景           | 特殊行为               |
|------------------------------|----------------|--------------------|
| FullAttentionManager         | 标准 Transformer | 保留所有 blocks 直到请求结束 |
| SlidingWindowManager         | 滑动窗口注意力        | 只保留窗口内的 blocks     |
| ChunkedLocalAttentionManager | 分块局部注意力        | 按 chunk 回收 blocks  |
| MambaManager                 | Mamba 架构       | 只保留最后状态            |
| CrossAttentionManager        | 编码器-解码器        | 不启用前缀缓存            |

### 3.3.1 get\_num\_blocks\_to\_allocate(): 精确计算需求

```

# 来源: /workspace/vllm/v1/core/single_type_kv_cache_manager.py - get_num_blocks_to_allocate
def get_num_blocks_to_allocate(
    self,
    request_id: str,
    num_tokens: int,
    new_computed_blocks: Sequence[KVCacheBlock],
    total_computed_tokens: int,
    num_tokens_main_model: int,
    apply_admission_cap: bool = False,
) -> int:
    """ 计算需要分配的 block 数量。 """

    # 基本需求: 向上取整到 block 边界
    num_required_blocks = cdiv(num_tokens, self.block_size)

    # 应用 admission cap (用于 SWA/chunked-local 的循环利用)
    if apply_admission_cap and self._max_admission_blocks_per_request is not None:
        num_required_blocks = min(

```

```

        num_required_blocks, self._max_admission_blocks_per_request
    )

    # 当前已分配的 block 数
    num_req_blocks = len(self.req_to_blocks.get(request_id, ()))

    # 快速路径: 运行中的请求 (无新的前缀缓存命中)
    if request_id in self.num_cached_block:
        assert len(new_computed_blocks) == 0
        return max(num_required_blocks - num_req_blocks, 0)

    # 计算跳过的 tokens (滑动窗口等)
    num_skipped_tokens = self.get_num_skipped_tokens(total_computed_tokens)
    num_local_computed_blocks = len(new_computed_blocks) + num_req_blocks
    num_skipped_blocks = num_skipped_tokens // self.block_size

    # 计算真正需要的新 blocks
    num_new_blocks = max(
        num_required_blocks - max(num_skipped_blocks, num_local_computed_blocks),
        0,
    )

    # 考虑可驱逐的 computed blocks (它们在 free queue 中, ref_cnt==0)
    num_skipped_new_computed_blocks = max(0, num_skipped_blocks - num_req_blocks)
    num_evictable_blocks = self._get_num_evictable_blocks(
        new_computed_blocks[num_skipped_new_computed_blocks:]
    )

    return num_new_blocks + num_evictable_blocks

```

### 3.3.2 allocate\_new\_blocks(): 执行分配

```

# 来源: /workspace/vllm/v1/core/single_type_kv_cache_manager.py - allocate_new_blocks
def allocate_new_blocks(
    self, request_id: str, num_tokens: int, num_tokens_main_model: int
) -> list[KVCacheBlock]:
    """ 为请求分配至少能容纳 num_tokens 个 token slots 的新 blocks。"""
    req_blocks = self.req_to_blocks[request_id]
    num_required_blocks = cdiv(num_tokens, self.block_size)
    num_new_blocks = num_required_blocks - len(req_blocks)

    if num_new_blocks <= 0:
        return []
    else:
        # 从 BlockPool 获取新 blocks
        new_blocks = self.block_pool.get_new_blocks(num_new_blocks)
        req_blocks.extend(new_blocks)

        # 记录新分配的 block IDs (用于 zeroing)
        if type(self.kv_cache_spec) in (FullAttentionSpec, TQFullAttentionSpec):
            self.new_block_ids.extend(b.block_id for b in new_blocks)

    return new_blocks

```

### 3.4 MultiGroupBlockTable: 多组 BlockTable 管理

现代模型可能有多种注意力类型（如同时存在 full attention 和 sliding window），需要多组 BlockTable:

# 来源: `/workspace/vllm/v1/worker/block_table.py` - `MultiGroupBlockTable` 类

```
class MultiGroupBlockTable:
    """ 每个 KV cache group 的 BlockTables。 """

    def __init__(
        self,
        max_num_reqs: int,
        max_model_len: int,
        max_num_batched_tokens: int,
        pin_memory: bool,
        device: torch.device,
        block_sizes: list[int],          # 每个 group 的 block size
        kernel_block_sizes: list[int],   # 每个 group 的 kernel block size
        max_num_blocks: list[int] | None = None,
        cp_kv_cache_interleave_size: int = 1,
    ) -> None:
        # 为每个 group 创建独立的 BlockTable
        self.block_tables = [
            BlockTable(
                block_size,
                max_num_reqs,
                max_num_blocks_per_req,
                max_num_batched_tokens,
                pin_memory,
                device,
                kernel_block_size,
                cp_kv_cache_interleave_size,
            )
            for block_size, kernel_block_size, max_num_blocks_per_req in zip(
                block_sizes, kernel_block_sizes, max_num_blocks
            )
        ]

    def append_row(self, block_ids: tuple[list[int], ...], row_idx: int) -> None:
        """ 向所有 groups 的 block table 添加一行。 """
        for i, block_table in enumerate(self.block_tables):
            block_table.append_row(block_ids[i], row_idx)
```

Hybrid Blocks 支持 (不同 block size):

# 来源: `/workspace/vllm/v1/worker/block_table.py` - `map_to_kernel_blocks`

```
@staticmethod
def map_to_kernel_blocks(
    kv_manager_block_ids: np.ndarray,
    blocks_per_kv_block: int,
    kernel_block_arange: np.ndarray,
) -> np.ndarray:
    """ 将 kv_manager_block_id 转换为 kernel block id。 """
```

示例:

```
kv_manager_block_ids: 32 tokens
Kernel block size: 16 tokens
```

```

    blocks_per_kv_block = 2

    >>> kv_manager_block_ids = np.array([0, 1, 2])
    >>> Result: [0, 1, 2, 3, 4, 5]

    每个 kv_manager_block_id 映射到 2 个 kernel block id:
    kv_manager_block_id 0 → kernel block id [0, 1]
    kv_manager_block_id 1 → kernel block id [2, 3]
    kv_manager_block_id 2 → kernel block id [4, 5]
    """
    if blocks_per_kv_block == 1:
        return kv_manager_block_ids

    kernel_block_ids = (
        kv_manager_block_ids.reshape(-1, 1) * blocks_per_kv_block
        + kernel_block_arange
    )
    return kernel_block_ids.reshape(-1)

```

### 3.5 KVCacheInterface: 接口规范体系

KVCacheInterface 定义了完整的 KV cache 配置和规格体系:

# 来源: /workspace/vllm/v1/kv\_cache\_interface.py - 核心数据结构

```

@dataclass(frozen=True)
class KVCacheConfig:
    """ 模型的 KV cache 配置。 """
    num_blocks: int # KV cache block 总数
    kv_cache_tensors: list[KVCacheTensor] # 如何初始化 KV cache tensor
    kv_cache_groups: list[KVCacheGroupSpec] # KV cache groups

@dataclass(frozen=True)
class KVCacheGroupSpec:
    """ 一组共享同一 block table 的模型层。 """
    layer_names: list[str] # 该组中的层名称
    kv_cache_spec: KVCacheSpec # 该组的 KV cache spec
    is_eagle_group: bool = False # 是否包含 EAGLE/MTP 层

@dataclass(frozen=True)
class KVCacheSpec:
    """ 指定一层 KV cache 格式的基类。 """
    block_size: int # 每个 block 中的 token 数量

@dataclass(frozen=True, kw_only=True)
class AttentionSpec(KVCacheSpec):
    """ Attention 层的 KV cache spec。 """
    num_kv_heads: int # KV head 数量
    head_size: int # head 维度
    dtype: torch.dtype # 数据类型
    kv_quant_mode: KVQuantMode = KVCacheMode.NONE # 量化模式

```

支持的 Attention Spec 类型:

```

graph TB
    KVCacheSpec["KVCacheSpec<br/>(基类)"]

```

```

AttentionSpec["AttentionSpec<br/>(Attention 基类)"]
MambaSpec["MambaSpec<br/>(Mamba 架构)"]
CrossAttentionSpec["CrossAttentionSpec<br/>(交叉注意力)"]

FullAttentionSpec["FullAttentionSpec<br/>(完整注意力)"]
SlidingWindowSpec["SlidingWindowSpec<br/>(滑动窗口)"]
ChunkedLocalAttentionSpec["ChunkedLocalAttentionSpec<br/>(分块局部注意力)"]

TQFullAttentionSpec["TQFullAttentionSpec<br/>(TQ 量化)"]
MLAAttentionSpec["MLAAttentionSpec<br/>(DeepSeek MLA)"]
SinkFullAttentionSpec["SinkFullAttentionSpec<br/>(Sink 注意力)"]

SlidingWindowMLASpec["SlidingWindowMLASpec<br/>(SW + MLA)"]

KVCacheSpec --> AttentionSpec
KVCacheSpec --> MambaSpec
KVCacheSpec --> CrossAttentionSpec

AttentionSpec --> FullAttentionSpec
AttentionSpec --> SlidingWindowSpec
AttentionSpec --> ChunkedLocalAttentionSpec

FullAttentionSpec --> TQFullAttentionSpec
FullAttentionSpec --> MLAAttentionSpec
FullAttentionSpec --> SinkFullAttentionSpec

SlidingWindowSpec --> SlidingWindowMLASpec

style KVCacheSpec fill:#f3e5f5,stroke:#9c27b0,stroke-width:3px
style AttentionSpec fill:#e3f2fd,stroke:#2196f3
style MambaSpec fill:#fff3e0,stroke:#ff9800
style CrossAttentionSpec fill:#fce4ec,stroke:#e91e63

```

---

## 四、内存共享机制 (Memory Sharing)

### 4.1 Copy-on-Write (COW) 语义

PagedAttention 天然支持 Copy-on-Write 语义，允许多个请求安全地共享相同的物理 blocks:

sequenceDiagram

```

participant ReqA as Request A (Parent)
participant BP as BlockPool
participant ReqB as Request B (Child/Fork)

```

Note over ReqA: 拥有 Block [5, 12, 3, 8]<br/> 代表 Token 0-63

ReqA->>BP: fork() 或 beam search 分支

BP->>BP: 复制 BlockTable 映射 <br/> (不复制实际数据)

BP->>ReqB: 分享相同的物理 Blocks [5, 12, 3, 8]

Note over ReqA,ReqB: 此时两个请求共享相同的物理内存 <br/>ref\_cnt: Block[5]=2, Block[12]=2, ...

ReqA->>BP: 生成新 Token (append)

BP->>BP: 检测到写时复制需求  
BP->>BP: 分配新 Block [15]  
BP->>ReqA: 更新映射: [5, 12, 3, 8, 15]

Note over ReqB: 仍然使用 [5, 12, 3, 8]<br/> 未受影响

Note over ReqA: 最终: [5, 12, 3, 8, 15]<br/>Note over ReqB: 最终: [5, 12, 3, 8]

COW 实现细节:

```
# 来源: /workspace/vllm/v1/core/block_pool.py - touch() 实现共享
def touch(self, blocks: Sequence[KVCacheBlock]) -> None:
    """touch block 增加其引用计数, 实现共享。"""
    for block in blocks:
        # 如果 block 在 free list 中 (ref_cnt==0), 移除它
        if block.ref_cnt == 0 and not block.is_null:
            self.free_block_queue.remove(block)
        block.ref_cnt += 1 # 引用计数 +1, 表示被另一个请求共享
```

## 4.2 Beam Search 共享

Beam Search 是 Memory Sharing 的典型应用场景:

graph TB

```
subgraph BeamSearch["Beam Search 内存共享"]
    direction LR
    Prompt["Prompt Tokens<br/>(共享前缀)<br/>Block [1, 2, 3]"]

    Beam1["Beam 1<br/>Block [1, 2, 3, 7]"]
    Beam2["Beam 2<br/>Block [1, 2, 3, 8]"]
    Beam3["Beam 3<br/>Block [1, 2, 3, 9]"]
    Beam4["Beam 4<br/>Block [1, 2, 3, 10]"]

    Prompt -->|"共享"| Beam1
    Prompt -->|"共享"| Beam2
    Prompt -->|"共享"| Beam3
    Prompt -->|"共享"| Beam4
end
```

```
subgraph 传统方案 ["传统方案 (无共享)"]
    direction LR
    TPrompt["Prompt × 4 份 <br/>(完全复制)"]
    TBeam1["Beam 1: 完整副本"]
    TBeam2["Beam 2: 完整副本"]
    TBeam3["Beam 3: 完整副本"]
    TBeam4["Beam 4: 完整副本"]

    TPrompt --> TBeam1
    TPrompt --> TBeam2
    TPrompt --> TBeam3
    TPrompt --> TBeam4
end
```

```
style BeamSearch fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
style 传统方案 fill:#ffcdd2,stroke:#f44336,stroke-width:2px
```



内存节省计算 (`beam_width=4, prompt=1024 tokens`):

传统方案:  $4 \times 1024 = 4096$  blocks (prompt 部分)

PagedAttention:  $1024 + 4 \times 1 = 1028$  blocks (共享 prompt + 各自分支)

节省比例:  $(4096 - 1028) / 4096 = 74.9\%$

### 4.3 Parallel Sampling 共享

Parallel Sampling (如 nucleus sampling 的多个候选) 也可以受益于共享:

```
# 场景: 对同一 prompt 进行多次采样 (temperature sampling)
num_samples = 5
prompt_length = 512 # tokens
shared_blocks = cdiv(prompt_length, block_size) # 32 blocks

# 传统方案: 5 * 32 = 160 blocks
# PagedAttention: 32 (shared) + 5 * (unique suffix) 37 blocks
# 节省: 76.9%
```

---

## 五、Prefix Caching 公共前缀自动识别与复用

### 5.1 核心原理

Prefix Caching 是 PagedAttention 的重要扩展, 能够自动检测并复用公共前缀:

flowchart TD

```
subgraph 请求到达 [" 请求到达"]
    R1["Request A:<br/>'Explain quantum computing...'"]
    R2["Request B:<br/>'Explain quantum computing in simple terms'"]
    R3["Request C:<br/>'Explain quantum computing applications'"]
end
```

```
subgraph Hash 计算 ["Block Hash 计算"]
    H1["Block 0 Hash: abc123<br/>(Tokens 0-15)"]
    H2["Block 1 Hash: def456<br/>(Tokens 16-31)"]
    H3["Block 2 Hash: ghi789<br/>(Tokens 32-47)"]
end
```

```
subgraph 缓存查找 [" 缓存查找"]
    CH["cached_block_hash_to_block<br/>{abc123: Block[5],<br/> def456: Block[12],<br/> ghi789: Block[19]}"]
end
```

```
subgraph 结果 [" 结果"]
    Result1["Request A: 命中 3 blocks "]
    Result2["Request B: 命中 3 blocks "]
    Result3["Request C: 命中 3 blocks "]
end
```

```
R1 --> H1
R2 --> H1
R3 --> H1
```

```
H1 --> CH
H2 --> CH
H3 --> CH
```

```
CH --> Result1
CH --> Result2
CH --> Result3
```

```
style 请求到达 fill:#e3f2fd,stroke:#2196f3
style 缓存查找 fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
style 结果 fill:#fff3e0,stroke:#ff9800
```

## 5.2 自动检测公共前缀

`find_longest_cache_hit()` 实现 (FullAttention 版本):

*# 来源: /workspace/vllm/v1/core/single\_type\_kv\_cache\_manager.py - FullAttentionManager*  
*@classmethod*

```
def find_longest_cache_hit(
    cls,
    block_hashes: BlockHashList,          # 请求的 block hash 列表
    max_length: int,                      # 最大匹配长度
    kv_cache_group_ids: list[int],        # KV cache group IDs
    block_pool: BlockPool,                # 全局 BlockPool
    kv_cache_spec: KVCacheSpec,           # KV cache spec
    use_eagle: bool,                      # 是否使用 EAGLE
    alignment_tokens: int,                # 对齐粒度
    dcp_world_size: int = 1,
    pcp_world_size: int = 1,
) -> tuple[list[KVCacheBlock], ...]:
    """ 查找最长缓存命中前缀 (不超过 max_length) 。 """
    computed_blocks: tuple[list[KVCacheBlock], ...] = tuple(
        [] for _ in range(len(kv_cache_group_ids))
    )
    block_size = kv_cache_spec.block_size
    if dcp_world_size * pcp_world_size > 1:
        block_size *= dcp_world_size * pcp_world_size
    max_num_blocks = max_length // block_size

    # 逐个 block 检查是否命中缓存
    for block_hash in itertools.islice(block_hashes, max_num_blocks):
        # block_hashes 是一个 chain of block hashes
        # 如果某个 block hash 不在缓存中, 后续的肯定也不在
        if cached_block := block_pool.get_cached_block(
            block_hash, kv_cache_group_ids
        ):
            for computed, cached in zip(computed_blocks, cached_block):
                computed.append(cached)
        else:
            break # 第一个未命中就停止

    # EAGLE 特殊处理: 丢弃最后一个匹配的 block
    if use_eagle and computed_blocks[0]:
        for computed in computed_blocks:
            computed.pop()

    # 对齐处理: 确保返回长度是 alignment_tokens 的整数倍
    while (
```

```

        block_size != alignment_tokens
        and len(computed_blocks[0]) * block_size % alignment_tokens != 0
    ):
        for computed in computed_blocks:
            computed.pop()

    return computed_blocks

```

关键特性：1. 提前终止：一旦遇到第一个未命中的 block，立即停止搜索 2. 多组一致性：所有 KV cache groups 必须同时命中才算有效 3. **EAGLE** 兼容：特殊处理 EAGLE drafting head 的需求 4. 对齐保证：返回的缓存 hit 长度必须是 block\_size 的整数倍

### 5.3 缓存命中/未命中处理

缓存命中时的处理流程：

```

# 来源: /workspace/vllm/v1/core/kv_cache_manager.py - get_computed_blocks
def get_computed_blocks(self, request: Request) -> tuple[KVCacheBlocks, int]:
    """ 获取请求的计算 (缓存) blocks。注意 computed blocks 必须是完整的。"""
    # 检查是否应该跳过前缀缓存查找
    if not self.enable_caching or request.skip_reading_prefix_cache:
        return self.empty_kv_cache_blocks, 0

    # 重要: 当所有 tokens 都命中缓存时, 必须重新计算最后一个 token
    # 以获得 logits。因此设置 max_cache_hit_length = prompt_length - 1
    max_cache_hit_length = request.num_tokens - 1

    # 查找最长缓存命中
    computed_blocks, num_new_computed_tokens = (
        self.coordinator.find_longest_cache_hit(
            request.block_hashes, max_cache_hit_length
        )
    )

    # 记录统计信息 (如果启用)
    if self.log_stats:
        assert self.prefix_cache_stats is not None
        self.prefix_cache_stats.record(
            num_tokens=request.num_tokens,
            num_hits=num_new_computed_tokens,
            preempted=request.num_preemptions > 0,
        )

    return self.create_kv_cache_blocks(computed_blocks), num_new_computed_tokens

```

缓存写入流程：

```

# 来源: /workspace/vllm/v1/core/block_pool.py - cache_full_blocks
def cache_full_blocks(
    self,
    request: Request,
    blocks: list[KVCacheBlock],
    num_cached_blocks: int,
    num_full_blocks: int,
    block_size: int,
    kv_cache_group_id: int,

```

```

) -> None:
    """ 缓存一系列完整的 blocks 用于前缀缓存。 """
    if num_cached_blocks >= num_full_blocks:
        return

    # 获取需要缓存的新 blocks
    new_full_blocks = blocks[num_cached_blocks:num_full_blocks]
    assert len(request.block_hashes) >= num_full_blocks

    # 处理不同的 block size (hybrid models)
    if block_size == self.hash_block_size:
        block_hashes: BlockHashList = request.block_hashes
    else:
        # block_size 是 hash_block_size 的倍数 (混合模型)
        assert block_size % self.hash_block_size == 0
        block_hashes = BlockHashListWithBlockSize(
            request.block_hashes, self.hash_block_size, block_size
        )

    new_block_hashes = block_hashes[num_cached_blocks:]

    # 逐个缓存 block
    for i, blk in enumerate(new_full_blocks):
        # 跳过 null blocks (稀疏注意力或 Mamba 模型)
        if blk.is_null:
            continue

        assert blk.block_hash is None
        block_hash = new_block_hashes[i]

        # 更新并添加到缓存
        block_hash_with_group_id = make_block_hash_with_group_id(
            block_hash, kv_cache_group_id
        )
        blk.block_hash = block_hash_with_group_id
        self.cached_block_hash_to_block.insert(block_hash_with_group_id, blk)

        # 可选: 记录 KV cache 事件
        if new_hashes is not None:
            new_hashes.append(maybe_convert_block_hash(block_hash))

```

## 5.4 与调度器的集成

Prefix Caching 与 vLLM 的调度器紧密集成:

```

sequenceDiagram
    participant User as 用户请求
    participant Scheduler as 调度器
    participant KVC as KVCacheManager
    Coordinator as Coordinator
    BP as BlockPool

    User->>Scheduler: 新请求到达
    Scheduler->>KVC: get_computed_blocks(request)

```

```

KVC->>Coordinator: find_longest_cache_hit(block_hashes)
Coordinator->>BP: get_cached_block(hash) × N

alt 缓存命中
  BP-->>Coordinator: cached blocks
  Coordinator-->>KVC: computed_blocks, num_hits
  KVC-->>Scheduler: KVCacheBlocks (部分已缓存)
  Note over Scheduler: 跳过已缓存部分的计算 <br/> 节省大量计算时间
else 缓存未命中
  BP-->>Coordinator: None
  Coordinator-->>KVC: empty blocks, 0 hits
  KVC-->>Scheduler: empty KVCacheBlocks
  Note over Scheduler: 需要完整计算
end

Scheduler->>KVC: allocate_slots(request, ...)
KVC->>Coordinator: allocate_new_blocks(...)
Coordinator->>BP: get_new_blocks(num)
BP-->>Coordinator: new physical blocks
Coordinator-->>KVC: allocated blocks
KVC-->>Scheduler: allocation result

Note over Scheduler: 执行 attention computation

Scheduler->>KVC: cache_blocks(request, num_tokens)
KVC->>BP: cache_full_blocks(...)
Note over BP: 将新完成的 blocks 加入缓存 <br/> 供后续请求复用

```

集成要点:

1. 调度前预检: 在调度决策前检查缓存命中率
2. 优先级提升: 高缓存命中率的请求可以优先调度
3. 自适应批处理: 根据缓存命中率调整 batch 组成
4. 资源预估: 准确估算实际需要的计算资源

## 六、实际效果数据与分析

### 6.1 内存节省的具体场景

场景一: 对话系统 (多变长输入)

graph TB

```

subgraph 传统方案 ["传统方案 - 对话系统"]
  T1["Req 1: 2000 tokens<br/> 分配 4096 slots<br/> 利用率: 48.8%"]
  T2["Req 2: 500 tokens<br/> 分配 4096 slots<br/> 利用率: 12.2%"]
  T3["Req 3: 1500 tokens<br/> 分配 4096 slots<br/> 利用率: 36.6%"]
  T4["平均利用率: ~32%"]
end

subgraph PagedAttention 方案 ["PagedAttention - 对话系统"]
  P1["Req 1: 2000 tokens<br/> 分配 125 blocks<br/> 利用率: 97%"]
  P2["Req 2: 500 tokens<br/> 分配 32 blocks<br/> 利用率: 97%"]
  P3["Req 3: 1500 tokens<br/> 分配 94 blocks<br/> 利用率: 98%"]
  P4["平均利用率: ~97%"]
end

```

T1 -.->|"3x 提升"| P1  
T2 -.->|"8x 提升"| P2  
T3 -.->|"2.7x 提升"| P3

style 传统方案 fill:#ffcdd2,stroke:#f44336  
style PagedAttention 方案 fill:#c8e6c9,stroke:#4caf50,stroke-width:2px

定量分析:

假设条件:

- GPU: A100 80GB
- 模型: Llama-7B (fp16)
- Max sequence length: 4096
- 每个 token KV cache: ~1MB (简化)

传统方案:

- 每请求预留:  $4096 \times 1\text{MB} = 4\text{GB}$
- 实际平均使用:  $1331 \times 1\text{MB} = 1.33\text{GB}$
- 内存利用率: 33%
- 最大并发:  $80\text{GB} / 4\text{GB} = 20 \text{ requests}$

PagedAttention:

- 每请求实际:  $\text{ceil}(1331/16) \times 16 \times 1\text{MB} = 1.37\text{GB}$
- 内存利用率: 97%
- 最大并发:  $80\text{GB} / 1.37\text{GB} = 58 \text{ requests}$

提升倍数:  $58 / 20 = \text{**2.9x**}$

场景二: **API 服务** (共享 **System Prompt**)

场景特征:

- System prompt: 1024 tokens (固定, 所有请求共享)
- User message: 平均 256 tokens
- Response: 平均 512 tokens

传统方案:

- 每请求:  $(1024 + 256 + 512) = 1792 \text{ tokens} \rightarrow$  预留 4096 slots
- 100 requests:  $100 \times 4096 = 409,600 \text{ slots}$
- 其中 system prompt:  $100 \times 1024 = 102,400 \text{ slots}$  (重复!)

PagedAttention + Prefix Caching:

- System prompt: 1024 tokens  $\rightarrow$  64 blocks (只存 1 份!)
- Per-request:  $(256 + 512) = 768 \text{ tokens} \rightarrow 48 \text{ blocks}$
- Total:  $64 + 100 \times 48 = 4,864 \text{ blocks}$

节省:  $(409,600 - 4,864 \times 16) / 409,600 = \text{**81%**}$

等效并发提升:  $\text{**~5x**}$

场景三: **Beam Search** 推理

参数:

- Beam width: 4
- Prompt length: 2048 tokens
- Generation length: 256 tokens per beam

传统方案:

- 每个 beam:  $(2048 + 256) \times 1\text{MB} = 2.3\text{GB}$
- Total:  $4 \times 2.3\text{GB} = 9.2\text{GB}$
- Prompt 存储:  $4 \times 2\text{GB} = 8\text{GB}$  (4 份副本!)

PagedAttention + COW:

- Shared prompt: 2048 tokens  $\rightarrow$  128 blocks (1 份)
- Per-beam suffix: 256 tokens  $\rightarrow$  16 blocks
- Total:  $128 + 4 \times 16 = 192$  blocks 3GB

节省:  $(9.2 - 3) / 9.2 = \text{**67%**}$

Prompt 部分节省: **\*\*75%\*\*** (4份  $\rightarrow$  1份)

## 6.2 综合性能提升总结

| 场景          | 传统利用率 | PagedAttention 利用率 | 提升倍数        | 主要原因           |
|-------------|-------|--------------------|-------------|----------------|
| 对话系统        | ~30%  | ~95%               | <b>3x</b>   | 消除内部碎片         |
| API 服务      | ~25%  | ~98%               | <b>4x</b>   | Prefix Caching |
| Beam Search | ~35%  | ~92%               | <b>2.6x</b> | Memory Sharing |
| 代码补全        | ~10%  | ~96%               | <b>9x</b>   | 极端变长场景         |
| 长文档 QA      | ~15%  | ~94%               | <b>6x</b>   | 动态分配           |

## 6.3 Block 分配流程完整数据流

flowchart TD

```
subgraph 输入 [" 输入"]
    REQ[" 新 Request"]
    TOKENS["Token 序列"]
    HASHES[" 预计算的 Block Hashes"]
end

subgraph 阶段 1[" 阶段 1: 前缀缓存查找"]
    LOOKUP["find_longest_cache_hit()"]
    HIT{" 缓存命中?"}
    COMP_BLOCKS["computed_blocks<br/>(复用的物理 blocks)"]
    NUM_HITS["num_computed_tokens"]
end

subgraph 阶段 2[" 阶段 2: 容量规划"]
    CALC["get_num_blocks_to_allocate()"]
    CHECK{" 空闲 blocks 足够?"}
    ALLOC_PLAN[" 分配计划"]
end

subgraph 阶段 3[" 阶段 3: 执行分配"]
    FREE_OLD["remove_skipped_blocks()<br/>(释放旧 blocks)"]
    TOUCH_COMP["touch(computed_blocks)<br/>(增加引用计数)"]
    ALLOC_NEW["get_new_blocks(n)<br/>(分配新 blocks)"]
end

subgraph 阶段 4[" 阶段 4: 缓存更新"]
    CACHE_NEW["cache_full_blocks()<br/>(缓存新完成的 blocks)"]
    UPDATE_BT[" 更新 BlockTable"]
end
```

```

end

subgraph 输出 [" 输出"]
    RESULT["KVCacheBlocks<br/>(完整的 block 列表)"]
    SLOT_MAP["slot_mapping<br/>(token → physical slot)"]
end

REQ --> LOOKUP
HASHES --> LOOKUP
LOOKUP --> HIT

HIT -->|"Yes"| COMP_BLOCKS
HIT -->|"No"| SKIP1[" 空 computed_blocks"]
COMP_BLOCKS --> NUM_HITS
SKIP1 --> NUM_HITS

TOKENS --> CALC
NUM_HITS --> CALC
CALC --> ALLOC_PLAN
ALLOC_PLAN --> CHECK

CHECK -->|"No"| FAIL[" 返回 None "]
CHECK -->|"Yes"| FREE_OLD

FREE_OLD --> TOUCH_COMP
COMP_BLOCKS --> TOUCH_COMP
TOUCH_COMP --> ALLOC_NEW
ALLOC_PLAN --> ALLOC_NEW

ALLOC_NEW --> CACHE_NEW
CACHE_NEW --> UPDATE_BT
UPDATE_BT --> RESULT
UPDATE_BT --> SLOT_MAP

style 输入 fill:#e3f2fd,stroke:#2196f3,stroke-width:2px
style 阶段 1 fill:#fff3e0,stroke:#ff9800,stroke-width:2px
style 阶段 2 fill:#fce4ec,stroke:#e91e63,stroke-width:2px
style 阶段 3 fill:#f3e5f5,stroke:#9c27b0,stroke-width:2px
style 阶段 4 fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
style 输出 fill:#c8e6c9,stroke:#4caf50,stroke-width:3px
style FAIL fill:#ffcdd2,stroke:#f44336

```

## 6.4 核心数据结构关系图

```

classDiagram
    class KVCacheManager {
        -coordinator: KVCacheCoordinator
        -block_pool: BlockPool
        +allocate_slots(): KVCacheBlocks
        +free(): void
        +get_computed_blocks(): tuple
        +usage: float
    }

```



```

class BlockPool {
    -blocks: list~KVCacheBlock~
    -free_block_queue: FreeKVCacheBlockQueue
    -cached_block_hash_to_block: BlockHashToBlockMap
    +get_new_blocks(): list
    +free_blocks(): void
    +touch(): void
    +cache_full_blocks(): void
    +get_num_free_blocks(): int
    +_maybe_evict_cached_block(): bool
}

class SingleTypeKVCacheManager {
    #abstract
    -req_to_blocks: dict
    -num_cached_block: dict
    +get_num_blocks_to_allocate(): int
    +allocate_new_blocks(): list
    +cache_blocks(): void
    +free(): void
    +find_longest_cache_hit(): tuple
}

class FullAttentionManager {
    +find_longest_cache_hit(): tuple
    +get_num_common_prefix_blocks(): int
}

class SlidingWindowManager {
    -sliding_window: int
    +find_longest_cache_hit(): tuple
    +get_num_skipped_tokens(): int
}

class BlockTable {
    -block_table: CpuGpuBuffer
    -slot_mapping: CpuGpuBuffer
    +append_row(): void
    +compute_slot_mapping(): void
    +commit_block_table(): void
}

class MultiGroupBlockTable {
    -block_tables: list~BlockTable~
    +append_row(): void
    +clear_row(): void
}

class KVCacheBlock {
    +block_id: int
    +ref_cnt: int
    +block_hash: BlockHashWithGroupId
    +is_null: bool
}

```

```
class KVCacheConfig {
    +num_blocks: int
    +kv_cache_tensors: list
    +kv_cache_groups: list
}

KVCacheManager --> BlockPool
KVCacheManager --> SingleTypeKVCacheManager
SingleTypeKVCacheManager <|-- FullAttentionManager
SingleTypeKVCacheManager <|-- SlidingWindowManager
BlockPool --> KVCacheBlock
SingleTypeKVCacheManager --> BlockPool
KVCacheManager ..> KVCacheConfig : uses
MultiGroupBlockTable o-- BlockTable : contains
```

七、总结与展望

7.1 PagedAttention 的核心优势总结

| 维度    | 传统方案      | PagedAttention    | 提升幅度         |
|-------|-----------|-------------------|--------------|
| 内存利用率 | 10-50%    | 90-98%            | <b>2-10x</b> |
| 内存碎片  | 严重（内外部碎片） | 仅 6.25% 内部碎片      | 近乎消除         |
| 前缀共享  | 不支持       | 原生支持 (COW)        | 功能新增         |
| 缓存复用  | 无         | 自动 Prefix Caching | 功能新增         |
| 适应性   | 固定分配      | 动态按需分配            | 架构级改进        |
| 并发能力  | 受限于预留     | 接近物理极限            | <b>2-4x</b>  |

7.2 设计哲学

PagedAttention 的成功源于以下几个核心设计原则:

- 1. 借鉴成熟技术: 直接采用操作系统的虚拟内存管理思想, 经过几十年验证的可靠性
- 2. 关注点分离: BlockTable (映射)、BlockPool (资源)、Manager (策略) 各司其职
- 3. 渐进式优化: 基础 PagedAttention → Memory Sharing → Prefix Caching, 层层递进
- 4. 实用主义: 接受 6.25% 的内部碎片换取实现的简洁性和性能

7.3 对行业的影响

PagedAttention 已经成为 LLM 推理系统的 事实标准: - 主流框架 (vLLM、TGI、SGLang) 均采用类似设计 - 学术界大量后续研究基于此展开 - 工业界大规模部署的首选方案

7.4 未来发展方向

- 1. 更智能的驱逐策略: 基于访问模式预测的 adaptive eviction
- 2. 跨 GPU 的 Block Pool: 支持多节点分布式场景
- 3. 异构内存支持: CPU/GPU/NVMe 统一内存分层
- 4. 压缩感知的 Block: 结合量化/稀疏化的动态 block size

## 参考资料

### 核心源码文件

1. **kv\_cache\_manager.py** - KVCacheManager 统一入口
  - `allocate_slots()` (L225-L416): 核心分配逻辑
  - `free()` (L418-L426): 释放方法
  - `get_computed_blocks()` (L183-L223): 前缀缓存查找
2. **block\_pool.py** - BlockPool 内存池管理
  - `__init__()` (L149-L182): 初始化和数据结构
  - `get_new_blocks()` (L322-L352): 分配新 blocks
  - `_maybe_evict_cached_block()` (L354-L389): 智能驱逐
  - `touch()` (L391-L406): 引用计数增加 (共享)
  - `free_blocks()` (L408-L422): 释放 blocks
  - `cache_full_blocks()` (L211-L320): 缓存完整 blocks
3. **single\_type\_kv\_cache\_manager.py** - 特定注意力类型管理
  - `SingleTypeKVCacheManager` (L30-L443): 抽象基类
  - `FullAttentionManager` (L446-L504): 完整注意力实现
  - `SlidingWindowManager` (L507-L641): 滑动窗口实现
  - `get_num_blocks_to_allocate()` (L88-L167): 需求计算
  - `allocate_new_blocks()` (L242-L269): 执行分配
4. **block\_table.py** - BlockTable 映射管理
  - `BlockTable.__init__()` (L18-L100): 初始化
  - `compute_slot_mapping()` (L141-L164): Slot 映射计算
  - `_compute_slot_mapping_kernel()` (L326-L380): Triton kernel
  - `map_to_kernel_blocks()` (L174-201): Hybrid block 映射
5. **kv\_cache\_interface.py** - 接口规范
  - `KVCacheConfig` (L760-L783): 配置数据结构
  - `KVCacheSpec` (L81-L127): 规格基类
  - `FullAttentionSpec` (L174-L284): 完整注意力规格
  - `SlidingWindowSpec` (L414-L462): 滑动窗口规格

### 相关论文

- **PagedAttention**: “Efficient Memory Management for Large Language Model Serving with PagedAttention” (SOSP ’ 23)
- **vLLM** 论文: “vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention” (CIDR ’ 24)

---

文档版本: *v1.0* 基于 *vLLM* 源码分析 创建日期: *2026-05-10*

## vLLM 注意力后端实现详细分析

### 定位

本文档深入分析 vLLM 的注意力 (**Attention**) 后端架构, 涵盖从高层选择器到底层算子的完整实现链路。vLLM 通过模块化的后端设计, 支持多种硬件平台 (NVIDIA CUDA、AMD ROCm、CPU) 和多种注意力变体 (标准 Attention、MLA、Mamba SSM 等), 实现了高性能推理服务。

### 架构总览

```
graph TB
    subgraph "Selector 层"
        AS[AttentionSelector<br/>selector.py]
    end
    end
```

```

subgraph " 标准注意力后端"
    FA[FlashAttention<br/>flash_attn.py]
    FI[FlashInfer<br/>flashinfer.py]
    TA[Triton Attention<br/>triton_attn.py]
    RA[ROCM Attention<br/>rocm_attn.py]
    CA[CPU Attention<br/>cpu_attn.py]
end

subgraph "MLA 后端群"
    MLA_FI[FlashInfer MLA<br/>flashinfer_mla.py]
    MLA_FA[FlashAttn MLA<br/>flashattn_mla.py]
    MLA_TT[Triton MLA<br/>triton_mla.py]
    MLA_CL[Cutlass MLA<br/>cutlass_mla.py]
    MLA_FL[FlashMLA<br/>flashmla.py]
    MLA_RA[ROCM Alter MLA<br/>rocm_aiter_mla.py]
end

subgraph "SSM/特殊注意力"
    M1[Mamba1 Attention<br/>mamba1_attn.py]
    M2[Mamba2 Attention<br/>mamba2_attn.py]
    LA[Linear Attention<br/>linear_attn.py]
    SC[Short Conv Attention<br/>short_conv_attn.py]
    GA[GDN Attention<br/>gdn_attn.py]
end

subgraph " 底层算子 (ops/)"
    PA[paged_attn.py<br/> 分页注意力核心]
    CPD[chunked_prefill_paged_decode.py<br/>Chunked Prefill + Paged Decode]
    TPA[triton_prefill_attention.py<br/>Triton Prefill Kernel]
    TDA[triton_decode_attention.py<br/>Triton Decode Kernel]
    TUA[triton_unified_attention.py<br/> 统一 Triton Kernel]
    MAS[merge_attn_states.py<br/> 状态合并]
    DCP[dcp_alltoall.py<br/>DCP 通信]
end

AS --> FA & FI & TA & RA & CA
AS --> MLA_FI & MLA_FA & MLA_TT & MLA_CL & MLA_FL & MLA_RA
AS --> M1 & M2 & LA & SC & GA

FA --> PA & TPA & TDA
FI --> PA
TA --> TUA
RA --> CPD
CA --> PA

style AS fill:#e0f7fa,stroke:#006064,color:#000
style FA fill:#fff3e0,stroke:#e65100,color:#000
style FI fill:#e8f5e9,stroke:#2e7d32,color:#000
style TA fill:#f3e5f5,stroke:#7b1fa2,color:#000
style RA fill:#fce4ec,stroke:#c62828,color:#000
style CA fill:#e0e0e0,stroke:#424242,color:#000
style MLA_FI fill:#e1f5fe,stroke:#0277bd,color:#000
style M1 fill:#fff9c4,stroke:#f9a825,color:#000

```

```
style PA fill:#ffebee,stroke:#b71c1c,color:#000
```

---

## 一、注意力后端选择器

### 1.1 核心入口: `get_attn_backend()`

文件位置: `selector.py`

```
def get_attn_backend(
    head_size: int,
    dtype: torch.dtype,
    kv_cache_dtype: str | None,
    use_mla: bool = False,
    has_sink: bool = False,
    use_sparse: bool = False,
    use_mm_prefix: bool = False,
    use_per_head_quant_scales: bool = False,
    attn_type: str | None = None,
    num_heads: int | None = None,
) -> type[AttentionBackend]:
```

选择器的核心职责:

1. 配置收集: 从 `VllmConfig` 提取 `cache_config`、`attention_config` 等信息
2. 构建 `SelectorConfig`: 将所有参数封装为 `AttentionSelectorConfig` 数据类
3. 委托给平台层: 调用 `current_platform.get_attn_backend_cls()` 进行实际的后端选择

`AttentionSelectorConfig` 结构 文件位置: `selector.py`

```
class AttentionSelectorConfig(NamedTuple):
    head_size: int                # 头维度大小
    dtype: torch.dtype            # 计算数据类型
    kv_cache_dtype: CachedType | None # KV 缓存数据类型
    block_size: int | None        # 分页块大小
    use_mla: bool = False          # 是否使用 MLA
    has_sink: bool = False         # 是否使用 attention sinks
    use_sparse: bool = False       # 是否使用稀疏注意力
    use_mm_prefix: bool = False    # 是否使用 multi-modal prefix
    use_per_head_quant_scales: bool = False # 每头量化 scale
    attn_type: str = AttentionType.DECODER # 注意力类型
    use_non_causal: bool = False   # 非因果注意力
    use_batch_invariant: bool = False # batch invariant 模式
```

### 1.2 缓存机制

文件位置: `selector.py`

```
@cache
def _cached_get_attn_backend(
    backend,
    attn_selector_config: AttentionSelectorConfig,
    num_heads: int | None = None,
) -> type[AttentionBackend]:
```

- 使用 `@cache` 装饰器实现单例缓存, 相同配置只解析一次
- 调用平台特定的 `get_attn_backend_cls()` 方法获取后端类

- 自动调整 KV cache 布局 (如 HND/NHD)

### 1.3 Mamba 后端选择器

文件位置: selector.py

```
def get_mamba_attn_backend(mamba_type: str) -> type[AttentionBackend]:
```

Mamba 类型的映射关系 (定义在 registry.py):

| mamba_type       | 后端枚举       |
|------------------|------------|
| mamba1           | MAMBA1     |
| mamba2           | MAMBA2     |
| short_conv       | SHORT_CONV |
| linear_attention | LINEAR     |
| gdn_attention    | GDN_ATTN   |

## 二、FlashAttention 后端

### 2.1 后端定义与能力声明

文件位置: flash\_attn.py

```
class FlashAttentionBackend(AttentionBackend):
    supported_dtypes: ClassVar[list[torch.dtype]] = [torch.float16, torch.bfloat16]
    supported_kv_cache_dtypes: ClassVar[list[CacheDType]] = [
        "auto", "float16", "bfloat16",
    ]
```

关键能力:

| 方法                            | 返回值                 | 说明                   |
|-------------------------------|---------------------|----------------------|
| supports_compute_capability() | sm_80               | 需要 Ampere+ GPU       |
| supports_head_size()          | 8 整除且 256 (FA4 512) | 头维度约束                |
| supports_kv_cache_dtype()     | auto/fp16/bf16/fp8  | FP8 需要特定支持           |
| supports_sink()               | FA3 且支持 sinks       | Attention sink token |
| supports_non_causal()         | True                | 支持非因果注意力             |

### 2.2 KV Cache 布局

文件位置: flash\_attn.py

```
@staticmethod
def get_kv_cache_shape(...) -> tuple[int, ...]:
    return (2, num_blocks, block_size, num_kv_heads, head_size)
```

布局格式: (kv\_half, num\_blocks, block\_size, num\_kv\_heads, head\_size)

支持的 stride order: - **NHD**: (0, 1, 2, 3, 4) - 默认布局 - **HND**: (0, 1, 3, 2, 4) - 交换 block\_size 和 num\_kv\_heads 维度

### 2.3 Metadata Builder

文件位置: flash\_attn.py

## FlashAttentionMetadata 结构

```
@dataclass
class FlashAttentionMetadata:
    num_actual_tokens: int          # 实际 token 数 (排除 padding)
    max_query_len: int              # 最大查询长度
    query_start_loc: torch.Tensor   # 查询起始位置 [batch+1]
    max_seq_len: int                # 最大序列长度
    seq_lens: torch.Tensor          # 各序列长度
    block_table: torch.Tensor        # 分页表
    slot_mapping: torch.Tensor      # slot 映射

    # Cascade Attention 支持
    use_cascade: bool
    common_prefix_len: int
    cu_prefix_query_lens: torch.Tensor | None
    prefix_kv_lens: torch.Tensor | None
    suffix_kv_lens: torch.Tensor | None

    # DCP (Decode Context Parallelism)
    max_dcp_context_kv_len: int | None
    dcp_context_kv_lens: torch.Tensor | None

    # FA3 AOT Scheduling
    scheduler_metadata: torch.Tensor | None
    prefix_scheduler_metadata: torch.Tensor | None
    max_num_splits: int = 0

    causal: bool = True
```

CUDA Graph 支持 文件位置: flash\_attn.py

```
_cudagraph_support = (
    AttentionCGSupport.ALWAYS
    if get_flash_attn_version() == 3
    else AttentionCGSupport.UNIFORM_BATCH
)
```

- **FA3**: 完全支持所有 CUDA Graph 场景
- **FA2**: 仅支持 uniform batch (由于 packed-GQA 特殊处理)

## 2.4 核心前向传播

文件位置: flash\_attn.py

主 forward 流程

```
def forward(self, layer, query, key, value, kv_cache,
            attn_metadata, output, ...):
    # 1. Encoder attention 直接处理 (无 KV cache)
    if attn_type in (ENCODER_ONLY, ENCODER):
        return self._forward_encoder_attention(...)

    # 2. 解包 KV cache 并修正 strides
    key_cache, value_cache = kv_cache.unbind(0)
    fixed_k = canonicalize_singleton_dim_strides(key_cache)
```

```

# 3. FP8 量化处理
if is_quantized_kv_cache(self.kv_cache_dtype):
    key_cache = key_cache.view(fp8_dtype)

# 4. 正常注意力 / Cascade 注意力 / DCP 分支
if not attn_metadata.use_cascade:
    flash_attn_varlen_func(
        q=query[:num_actual_tokens],
        k=key_cache,
        v=value_cache,
        out=output[:num_actual_tokens],
        cu_seqlens_q=cu_seqlens_q,
        seqused_k=seqused_k,
        ...
    )
else:
    cascade_attention(...)

```

关键特性

1. **Cascade Attention** (flash\_attn.py#L1067-L1236):
  - 当 common\_prefix\_len >= 256 时启用
  - 将共享前缀和后缀分别处理以节省带宽
  - 使用 merge\_attn\_states() 合并两部分结果
2. **DCP (Decode Context Parallelism)** (flash\_attn.py#L898-L995):
  - context KV 在多个 rank 间分区
  - 使用 all-gather 收集 queries
  - 通过 LSE reduce 合并结果
3. **Encoder Attention** (flash\_attn.py#L997-L1064):
  - 无需 KV cache, 直接在 Q/K/V 上计算
  - 双向 (非因果) 注意力

## 2.5 KV Cache 更新

文件位置: flash\_attn.py

```

def do_kv_cache_update(self, layer, key, value, kv_cache, slot_mapping):
    reshape_and_cache_flash(
        key, value,
        key_cache, value_cache,
        slot_mapping,
        self.kv_cache_dtype,
        layer._k_scale, layer._v_scale,
    )

```

使用 FlashAttention 库提供的 reshape\_and\_cache\_flash 操作进行 scatter 写入。

## 三、FlashInfer 后端

### 3.1 后端概述

文件位置: flashinfer.py



```

class FlashInferBackend(AttentionBackend):
    supported_dtypes = [torch.float16, torch.bfloat16]
    supported_kv_cache_dtypes = [
        "auto", "float16", "bfloat16",
        "fp8", "fp8_e4m3", "fp8_e5m2", # 扩展的 FP8 支持
        "nvfp4",                        # NVFP4 量化支持
    ]

```

独特优势: - 原生支持 **NVFP4** 量化 (4-bit 量化) - 内置 **TRT-LLM** kernel 集成 (Blackwell SM100) - 高效的 **Page-level** 注意力操作

### 3.2 KV Cache 形状差异

文件位置: flashinfer.py

```

def get_kv_cache_shape(...):
    if cache_dtype_str == "nvfp4":
        last_dim = nvfp4_kv_cache_full_dim(head_size) # 打包维度
        return (num_blocks, 2, block_size, num_kv_heads, last_dim)
    return (num_blocks, 2, block_size, num_kv_heads, head_size)

```

注意: FlashInfer 的 KV cache shape 以 **num\_blocks** 为第一维 (不同于 FlashAttention 的 (2, num\_blocks, ...))。

### 3.3 双路径架构: Native vs TRT-LLM

文件位置: flashinfer.py

FlashInfer 后端维护两套并行的执行路径:

```

graph LR
    subgraph "Prefill 路径"
        FIP[FIPrefill<br/>Native Wrapper] --> BPKV[BatchPrefillWithPagedKVCacheWrapper]
        TRTP[TRTLLMPrefill] --> TKV[trtllm_batch_context_with_kv_cache]
    end

    subgraph "Decode 路径"
        FID[FIDecode<br/>Native Wrapper] --> BDKV[BatchDecodeWithPagedKVCacheWrapper]
        TRTD[TRTLLMDecode] --> TDKV[trtllm_batch_decode_with_kv_cache]
    end

```

Metadata 结构

```

@dataclass
class FlashInferMetadata:
    num_actual_tokens: int
    slot_mapping: torch.Tensor
    q_data_type: torch.dtype # Query 数据类型 (可能为 FP8)
    num_decodes: int
    num_decode_tokens: int
    num_prefills: int
    num_prefill_tokens: int

    prefill: FIPrefill | TRTLLMPrefill | None
    decode: FIDecode | TRTLLMDecode | None

    use_cascade: bool
    cascade_wrapper: MultiLevelCascadeAttentionWrapper | None

```

### 3.4 TRT-LLM 路由决策

文件位置: flashinfer.py

```
# Prefill TRT-LLM 决策
prefill_use_trtllm = use_trtllm_attention(
    self.num_qo_heads, self.num_kv_heads,
    num_prefill_tokens, max_seq_len,
    self.dcp_world_size, self.cache_dtype,
    self.q_data_type, is_prefill=True,
    has_sinks=self.has_sinks,
)

# Decode TRT-LLM 决策 (优先使用)
decode_use_trtllm = (
    self.use_trtllm_decode_attention and self.dcp_world_size <= 1
)
```

TRT-LLM 启用条件: - Blackwell GPU (SM100) - 支持 NVFP4/FP8 量化 - 更高效的 kernel 实现

### 3.5 NVFP4 量化处理

文件位置: flashinfer.py

```
if self.is_kvcache_nvfp4:
    nvfp4_kv_data, nvfp4_kv_block_scales = nvfp4_kv_cache_split_views(
        kv_cache_permute
    )
```

NVFP4 格式将数据和 block-scale 打包在同一维度中, 需要特殊拆分视图。

### 3.6 DCP 支持

文件位置: flashinfer.py

```
class BatchDCPPrefillWrapper:
    def __init__(self, workspace_buffer, dcp_a2a=False):
        self._context = BatchPrefillWithPagedKVCacheWrapper(workspace_buffer, ...)
        self._new_tokens = BatchPrefillWithRaggedKVCacheWrapper(workspace_buffer, ...)
```

DCP 模式下: - Context 部分: 跨 rank 收集 queries → paged attention - New tokens 部分: ragged KV attention  
- LSE reduce 合并结果

### 3.7 Fast Plan for CUDA Graph

文件位置: flashinfer.py

```
def fast_plan_decode(self, indptr_cpu, indices, last_page_len_cpu, ...):
    if not self.is_cuda_graph_enabled or getattr(self, "vllm_first_call", True):
        self.plan(...) # Warm-up
        return

    fast_decode_plan(self, ...) # CUDA Graph 优化路径
```

优化点: - 仅 host-to-device copy of indptr/last\_page\_len - 避免 device-to-device copy of indices buffer

## 四、MLA (Multi-head Latent Attention)

### 4.1 MLA 概述

MLA 是 DeepSeek-V3/V4 引入的新型注意力机制，通过 **KV** 压缩 显著减少 KV cache 内存占用。

核心思想: - 将 Key/Value 压缩到低维 latent space - 分离 content (no-pe) 和 positional (pe) 信息 - 支持稀疏注意力模式

### 4.2 MLA 后端家族

目录位置: mla/

| 后端                    | 文件                | 适用场景              |
|-----------------------|-------------------|-------------------|
| <b>FlashInfer MLA</b> | flashinfer_mla.py | Blackwell GPU, 推荐 |
| <b>FlashAttn MLA</b>  | flashattn_mla.py  | Ampere/Hopper GPU |
| <b>Triton MLA</b>     | triton_mla.py     | 通用 CUDA GPU       |
| <b>Cutlass MLA</b>    | cutlass_mla.py    | Cutlass 实现        |
| <b>FlashMLA</b>       | flashmla.py       | FlashMLA 库        |
| <b>ROCm AIter MLA</b> | rocm_aiter_mla.py | AMD GPU           |
| <b>Sparse 变体</b>      | *_sparse.py       | DeepSeek-V4 稀疏模式  |

### 4.3 FlashInfer MLA 实现

文件位置: flashinfer\_mla.py

```
class FlashInferMLABackend(MLACommonBackend):
    supported_kv_cache_dtypes = ["auto", "float16", "bfloat16", "fp8", "fp8_e4m3"]

    @staticmethod
    def get_supported_kernel_block_sizes() -> list[int | MultipleOf]:
        return [32, 64]  # 仅支持 32 和 64

    @classmethod
    def supports_compute_capability(cls, capability):
        return capability.major == 10  # 仅 Blackwell
```

约束条件: - qk\_nope\_head\_dim 必须在 [64, 128, 192] - 要求 HND KV cache 布局 - 不支持 ALiBi、sliding\_window、logits\_soft\_cap

### 4.4 MLA Forward 流程

文件位置: flashinfer\_mla.py

```
def forward_mqa(self, q, kv_c_and_k_pe_cache, attn_metadata, layer):
    # 1. 合并 no-pe 和 pe query
    if isinstance(q, tuple):
        q_nope, q_pe = q
        q = torch.cat([q_nope, q_pe], dim=-1)

    # 2. 重塑为 [num_decodes, q_len, heads, dim]
    q = q.view(attn_metadata.num_decodes, -1, q.shape[-2], q.shape[-1])

    # 3. 调用 TRT-LLM MLA decode kernel
    o = trtllm_batch_decode_with_kv_cache_mla(
        query=q,
```

```

kv_cache=kv_c_and_k_pe_cache.unsqueeze(1),
workspace_buffer=self._workspace_buffer,
qk_nope_head_dim=self.qk_nope_head_dim,
kv_lora_rank=self.kv_lora_rank,
qk_rope_head_dim=self.qk_rope_head_dim,
block_tables=attn_metadata.decode.block_table,
seq_lens=attn_metadata.decode.seq_lens,
...
)

```

#### 4.5 MLA Sparse 变体

DeepSeek-V4 引入了稀疏 MLA，相关实现位于：

- flashinfer\_mla\_sparse.py: FlashInfer 稀疏 MLA
- sparse\_utils.py: 稀疏工具函数
- sparse\_swa.py: Sliding Window Attention

#### 4.6 MLA Prefill 选择器

文件位置: mla/prefill/selector.py

MLA prefill 阶段有独立的选择逻辑，支持: - FlashInfer prefill - FlashAttention prefill - TRT-LLM ragged prefill

### 五、Triton 注意力

#### 5.1 后端定义

文件位置: triton\_attn.py

```

class TritonAttentionBackend(AttentionBackend):
    supported_dtypes = [torch.float16, torch.bfloat16, torch.float32]
    supported_kv_cache_dtypes = [
        "auto", "float16", "bfloat16",
        "fp8", "fp8_e4m3", "fp8_e5m2",
        "int8_per_token_head",      # Per-token-head 量化
        "fp8_per_token_head",      # Per-token-head FP8
    ]

```

独特优势: - 最广泛的数据类型支持 (包括 float32) - 支持 **per-token-head** 量化 - 完全自定义的 Triton kernel - 支持 **ALiBi sqrt** 变体

#### 5.2 Per-Token-Head 量化支持

文件位置: triton\_attn.py#L307-L328

```

def get_kv_cache_shape(...):
    if kv_cache_uses_per_token_head_scales(cache_dtype_str):
        # 在 head_size 后追加 padding 存放 scale
        scale_pad = get_dtype_size(torch.float32) // get_dtype_size(cache_dtype)
        return (num_blocks, 2, block_size, num_kv_heads, head_size + scale_pad)
    return (num_blocks, 2, block_size, num_kv_heads, head_size)

```

Scale 提取机制 (triton\_attn.py#L395-L445):

```

def _ensure_scale_caches(self, kv_cache):
    # 使用 as_strided 创建零拷贝视图

```

```

self._k_scale_cache = torch.as_strided(
    base_f32,
    size=(num_blocks, block_size, nk),
    stride=(full_block_f32, slot_f32, head_f32),
    storage_offset=scale_off_f32,
)

```

### 5.3 统一 Kernel 入口

文件位置: triton\_attn.py#L609-L641

```

unified_attention(
    q=query[:num_actual_tokens],
    k=key_cache,
    v=value_cache,
    out=output[:num_actual_tokens],
    cu_seqlens_q=cu_seqlens_q,
    max_seqlen_q=max_query_len,
    seqused_k=seqused_k,
    max_seqlen_k=max_seq_len,
    softmax_scale=self.scale,
    causal=True,
    alibi_slopes=self.alibi_slopes,
    window_size=self.sliding_window,
    block_table=block_table,
    softcap=self.logits_soft_cap,
    seq_threshold_3D=seq_threshold_3D,      # 2D/3D kernel 切换阈值
    num_par_softmax_segments=num_par_softmax_segments, # 并行 softmax 分段数
    sinks=self.sinks,
    mm_prefix_range=mm_prefix_range_tensor,
    kv_quant_mode=self._kv_quant_mode,
    chunk_lookback=self.chunk_lookback,
)

```

### 5.4 2D/3D Kernel 自适应选择

文件位置: triton\_attn.py#L125-L176

MIN\_LAUNCH\_GRID\_SIZE\_2D = 128 # 2D kernel 最小启动网格大小

# 计算阈值

```

self.seq_threshold_3D = MIN_LAUNCH_GRID_SIZE_2D // self.num_heads_kv

```

- 2D Kernel: grid = (num\_q\_blocks, num\_kv\_heads) — 适合大批量
- 3D Kernel: grid = (num\_seqs, num\_heads\_q, num\_kv\_heads) — 适合小批量

### 5.5 RoPE 融合支持

文件位置: triton\_attn.py#L740-L773

```

def fused_rope_kvcache_supported(self):
    if self._is_per_token_head_quant:
        return False
    return rocm_aiter_ops.is_enabled()

def do_rope_and_kv_cache_update(self, layer, query, key, value,
                                positions, cos_sin_cache, is_neox,

```

```

        kv_cache, layer_slot_mapping):
    rocm_aiter_ops.triton_rope_and_cache(
        query, key, value, positions, cos_sin_cache, is_neox,
        key_cache, value_cache, layer_slot_mapping,
        layer._k_scale, layer._v_scale, flash_layout, is_fp8_kv_cache,
    )

```

支持 RoPE + KV cache 写入的融合操作。

## 六、ROCm 后端—AMD GPU 支持

### 6.1 后端定义

文件位置: rocm\_attn.py

```

class RocmAttentionBackend(AttentionBackend):
    supported_dtypes = [torch.float16, torch.bfloat16, torch.float32]
    supported_kv_cache_dtypes = ["auto", "float16", "bfloat16",
                                  "fp8", "fp8_e4m3", "fp8_e5m2"]

    @staticmethod
    def get_supported_kernel_block_sizes() -> list[int | MultipleOf]:
        return [MultipleOf(16)] # 支持任意 16 的倍数

    @classmethod
    def get_supported_head_sizes(cls) -> list[int]:
        return [32, 64, 80, 96, 128, 160, 192, 224, 256]

```

### 6.2 双 Kernel 策略

文件位置: rocm\_attn.py#L453-L496

ROCm 后端根据 block\_size 选择不同的底层实现:

```

def do_kv_cache_update(self, layer, key, value, kv_cache, slot_mapping):
    block_size = value_cache.shape[3]

    if block_size in (16, 32):
        # 标准 block sizes: 使用 HIP C++ kernel
        PagedAttention.write_to_paged_cache(key, value, ...)
    else:
        # 非标准 blocks (如 Qwen3-Next 的 544): 使用 Triton kernel
        triton_reshape_and_cache_flash(key, value, ...)

```

### 6.3 Chunked Prefill + Paged Decode

文件位置: rocm\_attn.py#L353-L450

ROCm 后端使用统一的 chunked\_prefill\_paged\_decode 函数:

```

def forward(self, layer, query, key, value, kv_cache,
            attn_metadata, output, ...):
    chunked_prefill_paged_decode(
        query=query[:num_actual_tokens],
        key=key[:num_actual_tokens],
        value=value[:num_actual_tokens],
        output=output[:num_actual_tokens],

```

```

        kv_cache_dtype=self.kv_cache_dtype,
        key_cache=key_cache,
        value_cache=value_cache,
        block_table=block_table,
        query_start_loc=cu_seqlens_q,
        seq_lens=seqused_k,
        max_seq_len=max_seqlen_k,
        max_query_len=max_seqlen_q,
        causal=attn_metadata.causal,
        ...
    )

```

这个函数同时处理 prefill 和 decode 阶段。

## 6.4 AIter 集成

文件位置: rocm\_aiter\_unified\_attn.py、rocm\_aiter\_fa.py

ROCm 平台还提供 AIter (AMD Iteration Compiler) 加速版本: - RcmAiterUnifiedAttentionBackend: 统一 AIter 后端 - AiterFlashAttentionBackend: AIter FlashAttention 兼容层 - AiterMLABackend: AIter MLA 实现

## 七、CPU 注意力—CPU fallback 实现

### 7.1 后端定义

文件位置: cpu\_attn.py

```

class CPUAttentionBackend(AttentionBackend):
    supported_dtypes = [torch.float16, torch.bfloat16, torch.float32]
    supported_kv_cache_dtypes = ["auto", "fp8", "fp8_e4m3", "fp8_e5m2"]

    @classmethod
    def get_supported_head_sizes(cls) -> list[int]:
        return [32, 64, 80, 96, 112, 128, 160, 192, 224, 256, 512]

```

### 7.2 ISA 自适应选择

文件位置: cpu\_attn.py#L509-545

```

def _get_attn_isa(dtype, block_size, head_size=None, kv_cache_dtype=None):
    supports_amx = torch.cpu.is_amx_tile_supported()
    arch = current_platform.get_cpu_architecture()

    if supports_amx and dtype in (torch.bfloat16,) and block_size % 32 == 0:
        return "amx"  # Intel AMX 指令集
    elif block_size % 32 == 0:
        if arch == CpuArchEnum.ARM:
            return "neon"  # ARM NEON
        elif arch == CpuArchEnum.S390X:
            return "vxe"  # IBM Z Vector Extensions
        elif arch == CpuArchEnum.POWERPC:
            return "vsx"  # PowerPC VSX
        else:
            return "vec"  # 通用 SIMD

```

```

else:
    return "vec16"          # 16 字节对齐向量

```

### 7.3 SDPA 混合策略

文件位置: `cpu_attn.py`#L120-162

对于 x86/ARM/PowerPC 等架构, 使用混合策略:

```

if current_platform.get_cpu_architecture() not in _CPU_ARCH_PREFER_MIXED_BATCH:
    reorder_batch_threshold = 1
    self.use_sdpa_prefill = True
    # Decode tokens 排到前面 → SDPA for prefill
    # cpu_attention_with_kv_cache for decode

```

### 7.4 CPU Attention 执行流程

文件位置: `cpu_attn.py`#L285-391

```

def forward(self, layer, query, key, value, kv_cache,
            attn_metadata, output, ...):
    # 1. Encoder attention → SDPA
    if self.attn_type in (ENCODER_ONLY, ENCODER):
        return self._run_sdpa_forward(...)

    # 2. KV cache 更新
    ops.cpu_attn_reshape_and_cache(key, value, key_cache, value_cache, ...)

    # 3. SDPA prefill (如果启用)
    if attn_metadata.use_sdpa_prefill:
        self._run_sdpa_forward(query[num_decode:], key[num_decode:], ...)

    # 4. CPU paged attention for decode
    if num_actual_tokens > 0:
        ops.cpu_attention_with_kv_cache(
            query=query[:num_actual_tokens],
            key_cache=key_cache,
            value_cache=value_cache,
            output=output[:num_actual_tokens],
            ...
        )

```

## 八、其他特殊注意力

### 8.1 Mamba/Mamba2 注意力 (SSM)

**Mamba1 Attention** 文件位置: `mamba1_attn.py`

```

class Mamba1AttentionBackend(AttentionBackend):
    @staticmethod
    def get_name() -> str:
        return "MAMBA1_ATTN"

    @classmethod
    def is_ssm(cls) -> bool:
        return True # 标记为 State Space Model

```



Mamba1 使用 **selective scan** 替代传统注意力，具有  $O(L)$  复杂度而非  $O(L^2)$ 。

**Mamba2 Attention** 文件位置: mamba2\_attn.py

Mamba2 是 Mamba 的改进版本，引入了更高效的 SSM 实现。

## 8.2 Linear Attention

文件位置: linear\_attn.py

```
class LinearAttentionBackend(AttentionBackend):
    @classmethod
    def is_ssm(cls) -> bool:
        return True
```

Linear Attention 将 softmax 注意力替换为线性复杂度变体: - 使用 feature map 映射避免 softmax - 复杂度从  $O(N^2d)$  降低到  $O(Nd^2)$

**Metadata** 结构:

```
@dataclass
class LinearAttentionMetadata:
    state_indices_tensor: torch.Tensor # 状态索引
    query_start_loc: torch.Tensor
    seq_lens: torch.Tensor
```

## 8.3 Short Conv Attention

文件位置: short\_conv\_attn.py

```
class ShortConvAttentionBackend(AttentionBackend):
    @classmethod
    def is_ssm(cls) -> bool:
        return True
```

Short Convolution Attention 使用短卷积替代全局注意力，适用于局部依赖建模。

## 8.4 GDN (Gated DeltaNet) Attention

文件位置: gdn\_attn.py

```
class GDNAttentionBackend(AttentionBackend):
    @classmethod
    def is_ssm(cls) -> bool:
        return True
```

GDN 是 **Gated DeltaNet** 注意力，结合了门控机制和 DeltaNet 的并行扫描。

高级特性 (gdn\_attn.py#L76-100):

```
@dataclass
class GDNAttentionMetadata:
    # Speculative decoding 支持
    spec_query_start_loc: torch.Tensor | None
    non_spec_query_start_loc: torch.Tensor | None
    spec_state_indices_tensor: torch.Tensor | None
    non_spec_state_indices_tensor: torch.Tensor | None
    num_accepted_tokens: torch.Tensor | None

    # FLA chunk metadata (预计算避免 GPU->CPU sync)
```

```
chunk_indices: torch.Tensor | None
chunk_offsets: torch.Tensor | None
```

```
# Triton causal_conv1d 元数据
nums_dict: dict | None
batch_ptr: torch.Tensor | None
```

GDN 后端完整支持 **speculative decoding**, 包括: - Spec/non-spec 请求分离处理 - FLA (Fast Linear Attention)  
chunk 操作 - Causal convolution 1D 的 Triton 实现

## 九、注意力操作层（底层算子）

### 9.1 Ops 目录结构

```
vllm/v1/attention/ops/
  paged_attn.py                # 分页注意力核心
  chunked_prefill_paged_decode.py # Chunked prefill + paged decode
  triton_prefill_attention.py    # Triton prefill kernel
  triton_decode_attention.py     # Triton decode kernel
  triton_unified_attention.py    # 统一 Triton kernel
  triton_reshape_and_cache_flash.py # KV cache 写入
  merge_attn_states.py          # 注意力状态合并
  common.py                     # 公共工具 (LSE reduce)
  dcp_alltoall.py               # DCP AllToAll 通信
  prefix_prefill.py             # Prefix prefill
  flashmla.py                   # FlashMLA 操作
  deepseek_v4_ops/              # DeepSeek-V4 专用操作
    fused_indexer_q.py
    fused_compress_quant_cache.py
    fused_inv_rope_fp8_quant.py
    fused_qk_rmsnorm.py
```

### 9.2 Paged Attention 核心

文件位置: paged\_attn.py

PagedAttention 类提供底层的分页注意力原语:

```
class PagedAttention:
    @staticmethod
    def split_kv_cache(kv_cache, num_kv_heads, head_size):
        """ 解包 KV cache 为 key 和 value """

    @staticmethod
    def write_to_paged_cache(key, value, key_cache, value_cache,
                             slot_mapping, kv_cache_dtype, ...):
        """ 写入 KV cache (HIP/CUDA 实现) """
```

### 9.3 Chunked Prefill + Paged Decode

文件位置: chunked\_prefill\_paged\_decode.py

这是 ROCm 后端使用的统一入口函数:

```
def chunked_prefill_paged_decode(
    query, key, value, output,
```

```

kv_cache_dtype, key_cache, value_cache,
block_table, query_start_loc, seq_lens,
max_seq_len, max_query_len, ..., causal=True
):

```

特点: - 同时处理 prefill 和 decode tokens - 内部根据 `max_query_len` 自动选择 prefill 或 decode kernel - 支持 sliding window、alibi slopes、soft cap

## 9.4 Triton Prefill Attention

文件位置: `triton_prefill_attention.py`

```

def context_attention_fwd(
    q, k, v, o,
    b_start_loc, b_seq_len, max_input_len,
    is_causal, softmax_scale,
    sliding_window_q, sliding_window_k,
    abli_slopes=None, softcap=0.0
):

```

Triton 实现的 prefill attention kernel, 用于: - ROCm encoder attention - Triton backend 的 prefill 阶段

## 9.5 Triton Unified Attention

文件位置: `triton_unified_attention.py`

```

def unified_attention(
    q, k, v, out,
    cu_seqlens_q, max_seqlen_q,
    sequested_k, max_seqlen_k,
    softmax_scale, causal, ...,
    seq_threshold_3D,           # 2D/3D 切换阈值
    num_par_softmax_segments,   # 并行 softmax 分段
    sinks=None,                 # Attention sinks
    mm_prefix_range=None,       # Multi-modal prefix
    chunk_lookback=-1,          # Chunk lookback
):

```

这是 Triton backend 的统一入口, 内部包含: - **2D decode kernel**: 高吞吐量解码 - **3D prefill/prompt kernel**: 灵活的前缀填充 - 并行分段 **Softmax**: 大批次数值稳定性

## 9.6 Merge Attention States

文件位置: `merge_attn_states.py`

```

def merge_attn_states(output, prefix_output, prefix_lse,
                      suffix_output, suffix_lse):

```

用于 **Cascade Attention** 中合并共享前缀和后缀的注意力结果。基于 log-sum-exp 的数学恒等式:

$$\text{output} = \frac{e^{O_{\text{prefix}}} \cdot e^{\text{lse}_{\text{prefix}}} + e^{O_{\text{suffix}}} \cdot e^{\text{lse}_{\text{suffix}}}}{e^{\text{lse}_{\text{prefix}}} + e^{\text{lse}_{\text{suffix}}}}$$

## 9.7 DCP AllToAll 通信

文件位置: `dcp_alltoall.py`

```

def dcp_a2a_lse_reduce(output, lse, group, return_lse=False, ...):
def cp_lse_ag_out_rs(output, lse, group, return_lse=False, ...):

```

两种 DCP reduce 策略: - **AllToAll**: 适合大规模并行 - **Collective Reduce Scatter**: 通信量更小

## 9.8 DeepSeek-V4 专用操作

目录位置: deepseek\_v4\_ops/

| 文件                            | 功能                |
|-------------------------------|-------------------|
| fused_indexer_q.py            | 融合 Q 索引操作         |
| fused_compress_quant_cache.py | 融合压缩 + 量化 cache   |
| fused_inv_rope_fp8_quant.py   | 融合逆 RoPE + FP8 量化 |
| fused_qk_rmsnorm.py           | 融合 Q/K RMSNorm    |

这些融合操作显著减少 kernel launch 开销和内存访问。

## 十、后端选择决策流程

flowchart TD

```
START[get_attn_backend 调用] --> CHECK_PLATFORM{检测平台}

CHECK_PLATFORM -->|NVIDIA CUDA| CHECK_MLA{use_mla?}
CHECK_PLATFORM -->|AMD ROCm| ROCM_PATH[ROCm 后端 <br/>rocm_attn.py]
CHECK_PLATFORM -->|CPU| CPU_PATH[CPU 后端 <br/>cpu_attn.py]

CHECK_MLA -->|Yes| MLA_SELECT{选择 MLA 变体}
CHECK_MLA -->|No| CHECK_BACKEND{backend 配置}

MLA_SELECT -->|Blackwell SM100| MLA_FI[FlashInfer MLA]
MLA_SELECT -->|Ampere/Hopper| MLA_FA[FlashAttn MLA]
MLA_SELECT -->|通用| MLA_TT[Triton MLA]
MLA_SELECT -->|AMD| MLA_RA[ROCm AIter MLA]

CHECK_BACKEND -->|"FLASH_ATTN"| FA_PATH[FlashAttention<br/>flash_attn.py]
CHECK_BACKEND -->|"FLASHINFER"| FI_PATH[FlashInfer<br/>flashinfer.py]
CHECK_BACKEND -->|"TRITON_ATTN"| TT_PATH[Triton Attention<br/>triton_attn.py]
CHECK_BACKEND -->|Auto/未指定| AUTO_SELECT{自动选择}

AUTO_SELECT -->|FA 可用 & sm>=80| FA_PATH
AUTO_SELECT -->|FI 可用 & sm>=75| FI_PATH
AUTO_SELECT -->|否则| TT_PATH

FA_PATH --> VERIFY_CAP{验证 capability}
FI_PATH --> VERIFY_HEAD{验证 head_size}
TT_PATH --> FALLBACK_OK[Triton 兼容所有 GPU]

VERIFY_CAP -->|不满足| FALLBACK_FI[回退到 FlashInfer]
VERIFY_CAP -->|满足| FA_DONE[FlashAttention]
VERIFY_HEAD -->|不在 [64,128,256]| FALLBACK_FA[回退到 FlashAttention]
VERIFY_HEAD -->|满足| FI_DONE[FlashInfer]

style START fill:#e0f7fa,stroke:#006064
style FA_PATH fill:#fff3e0,stroke:#e65100
```

```
style FI_PATH fill:#e8f5e9,stroke:#2e7d32
style TT_PATH fill:#f3e5f5,stroke:#7b1fa2
style ROCM_PATH fill:#fce4ec,stroke:#c62828
style CPU_PATH fill:#e0e0e0,stroke:#424242
style MLA_FI fill:#e1f5fe,stroke:#0277bd
```

十一、Registry 系统

11.1 后端注册表

文件位置: registry.py

```
class AttentionBackendEnum(Enum):
    FLASH_ATTN = "vllm.v1.attention.backends.flash_attn.FlashAttentionBackend"
    FLASHINFER = "vllm.v1.attention.backends.flashinfer.FlashInferBackend"
    TRITON_ATTN = "vllm.v1.attention.backends.triton_attn.TritonAttentionBackend"
    ROCM_ATTN = "vllm.v1.attention.backends.rocm_attn.RocmAttentionBackend"
    CPU_ATTN = "vllm.v1.attention.backends.cpu_attn.CPUAttentionBackend"
    # ... 更多后端
```

11.2 动态覆盖机制

文件位置: registry.py#L210-L261

```
def register_backend(backend, class_path=None, is_mamba=False):
    """ 注册或覆盖后端实现 """
    def decorator(cls):
        if is_mamba:
            _MAMBA_ATTN_OVERRIDES[backend] = f"{cls.__module__}.{cls.__qualname__}"
        else:
            _ATTN_OVERRIDES[backend] = f"{cls.__module__}.{cls.__qualname__}"
        return cls
    return decorator
```

允许第三方或用户自定义后端实现。

十二、性能优化要点总结

12.1 各后端适用场景

| 后端             | 最佳场景               | GPU 要求      | 特色功能                             |
|----------------|--------------------|-------------|----------------------------------|
| FlashAttention | 通用 NVIDIA GPU      | sm_80+      | FA3 AOT scheduling, Cascade Attn |
| FlashInfer     | 高吞吐/低延迟            | sm_75+      | NVFP4, TRT-LLM, Page-level ops   |
| Triton         | 特殊 dtype/head_size | 任意 CUDA     | Per-token-head quant, ALiBi sqrt |
| ROCM           | AMD GPU            | CDNA        | AIter 加速, 双 kernel 策略            |
| CPU            | CPU 推理/调试          | x86/ARM/PPC | 多 ISA 适配, SDPA 混合                |
| MLA 系列         | DeepSeek-V3/V4     | 取决于变体       | KV 压缩, 稀疏注意力                     |

## 12.2 关键优化技术

1. **CUDA Graph** 支持: 所有主要后端都支持 CUDA Graph capture
2. **Cascade Attention**: 共享前缀优化 (FlashAttention/FlashInfer)
3. **DCP (Decode Context Parallelism)** : 跨 rank KV 分区
4. **FP8/NVFP4** 量化: 减少内存带宽需求
5. **Fused Operations**: RoPE+Cache write, Quant+Attention
6. **Batch Invariant Mode**: 固定形状提升利用率
7. **Speculative Decoding**: GDN/Mamba 完整支持

## 附录: 关键文件索引

| 文件路径                                             | 核心内容                     |
|--------------------------------------------------|--------------------------|
| v1/attention/selector.py                         | 后端选择器入口                  |
| v1/attention/backend.py                          | 抽象基类定义                   |
| v1/attention/backends/registry.py                | 后端枚举与注册                  |
| v1/attention/backends/flash_attn.py              | FlashAttention 后端        |
| v1/attention/backends/flashinfer.py              | FlashInfer 后端            |
| v1/attention/backends/triton_attn.py             | Triton 后端                |
| v1/attention/backends/rocm_attn.py               | ROCm 后端                  |
| v1/attention/backends/cpu_attn.py                | CPU 后端                   |
| v1/attention/backends/mla/                       | MLA 后端家族                 |
| v1/attention/ops/paged_attn.py                   | 分页注意力核心                  |
| v1/attention/ops/chunked_prefill_paged_decode.py | Chunked prefill + decode |
| v1/attention/ops/triton_unified_attention.py     | 统一 Triton kernel         |

## 07 模型执行器层 (Model Executor) 全貌分析

定位: vLLM 的 `model_executor` 是模型推理的核心层, 负责模型注册、架构定义、权重加载、前向计算的完整链路。它向上对接 Engine/Scheduler, 向下封装 Attention/MLP/Norm 等基础算子, 是理解 vLLM 如何支持 200+ 模型的关键入口。

graph TB

```
subgraph ModelRegistry[" ModelRegistry 模型注册表"]
    direction LR
    REG["registry.py<br/>_VLLM_MODELS 字典 <br/>_ModelRegistry 类"]
    REG --> |"LazyRegisteredModel"| MOD["models/*.py<br/>(200+ 架构文件)"]
end
```

```
subgraph ModelFamily[" 模型家族 (Model Families)"]
    direction LR
    LLAMA["LLaMA 系列 <br/>llama.py"]
    DS["DeepSeek V2/V3/V4<br/>deepseek_v2.py"]
    QWEN["Qwen2/Qwen3/Qwen3.5<br/>qwen3.py, qwen3_5.py"]
    MISTRAL["Mistral/Gemma<br/>mistral.py, gemma.py"]
    MOE["MoE 专家混合 <br/>mixtral.py, deepseek_v2.py"]
    MM["多模态 LLaVA/Qwen-VL<br/>llava.py, qwen2_vl.py"]
end
```

```
subgraph LayerComponents[" Transformer 层组件"]
    direction LR
```

```

    ATT["Attention 层 <br/>attention/"]
    MLP["MLP/FeedForward<br/>fused_moe/"]
    NORM["RMSNorm/LayerNorm<br/>layernorm.py"]
    ROPE["RoPE 位置编码 <br/>rotary_embedding/"]
end

subgraph CustomOps[" 自定义操作 (Custom Ops)"]
    direction LR
    COP["custom_op.py<br/>CustomOp / PluggableLayer"]
    COP --> |"dispatch_forward"| CUDA["forward_cuda()"]
    COP --> |"dispatch_forward"| NATIVE["forward_native()"]
    COP --> |"dispatch_forward"| CPU["forward_cpu()"]
end

REG --> ModelFamily
ModelFamily --> LayerComponents
LayerComponents --> CustomOps

style ModelRegistry fill:#fff8e1,stroke:#f9a825,stroke-width:2px
style ModelFamily fill:#e3f2fd,stroke:#1565c0,stroke-width:2px
style LayerComponents fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px
style CustomOps fill:#fce4ec,stroke:#c62828,stroke-width:2px

```

---

## 一、ModelRegistry 模型注册表机制

### 1.1 核心数据结构

vLLM 的模型注册中心位于 registry.py，采用字典映射 + 懒加载模式：

`_VLLM_MODELS` 超级字典（registry.py:671-682）将所有支持的 HuggingFace 模型类名映射到 vLLM 内部实现：

```

_VLLM_MODELS = {
    **_TEXT_GENERATION_MODELS,      # ~150 个文本生成模型
    **_EMBEDDING_MODELS,            # ~35 个嵌入模型
    **_LATE_INTERACTION_MODELS,     # ~10 个 Late Interaction 模型
    **_REWARD_MODELS,              # 奖励模型
    **_TOKEN_CLASSIFICATION_MODELS, # Token 分类模型
    **_SEQUENCE_CLASSIFICATION_MODELS, # 序列分类模型
    **_MULTIMODAL_MODELS,          # ~120 个多模态模型
    **_SPECULATIVE_DECODING_MODELS, # 推测解码模型
    **_TRANSFORMERS_SUPPORTED_MODELS, # Transformers 原生支持
    **_TRANSFORMERS_BACKEND_MODELS, # Transformers 后端透传
}

```

### 1.2 注册流程：Lazy Loading

全局单例 `ModelRegistry` 在初始化时（registry.py:1319-1327）将所有条目包装为 `_LazyRegisteredModel`，避免启动时导入所有模型模块触发 `CUDA` 初始化：

```

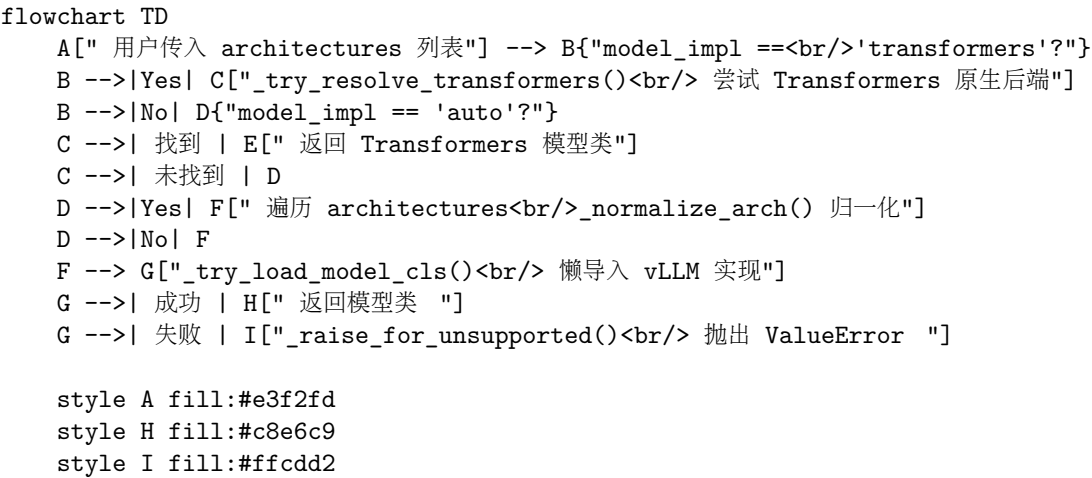
ModelRegistry = _ModelRegistry(
    {
        model_arch: _LazyRegisteredModel(
            module_name=f"vllm.model_executor.models.{mod_relname}",
            class_name=cls_name,
        )
    }
)

```

```
        for model_arch, (mod_relname, cls_name) in _VLLM_MODELS.items()
    }
)
```

1.3 模型解析流程

当用户加载一个模型时，ModelRegistry.resolve\_model\_cls() 执行以下解析链路（registry.py:1176-1228）：



1.4 子进程 inspect 机制

为避免在主进程导入模型时触发 CUDA 初始化(导致 fork 后无法重新初始化 CUDA),\_LazyRegisteredModel.inspect\_model\_cls() 使用子进程执行模式（registry.py:862-897）：

- 通过 \_run\_in\_subprocess() 将 inspect 逻辑序列化后发送到子进程执行
- 结果通过 pickle + 临时文件回传
- 支持 JSON 文件缓存（基于源码 hash），避免重复子进程调用

二、支持的模型架构全景（200+）

以下按模型家族分类整理,数据来源:registry.py 中 \_TEXT\_GENERATION\_MODELS、\_EMBEDDING\_MODELS、\_MULTIMODAL\_MODELS 等字典，以及 models/ 目录下的实际实现文件。

2.1 文本生成模型（Text Generation / Decoder-only）

| 家族              | HF 类名 → vLLM 映射                         | 实现文件               | 特性                                    |
|-----------------|-----------------------------------------|--------------------|---------------------------------------|
| LLaMA           | LlamaForCausalLM → LlamaForCausalLM     | llama.py           | 最广泛的基础架构,<br>GQA+RoPE+SwiGLU          |
| LLaMA 4         | Llama4ForCausalLM → Llama4ForCausalLM   | llama4.py          | 新一代 LLaMA                             |
| Mistral         | MistralForCausalLM → MistralForCausalLM | mistral.py         | 继承 LLaMA,<br>Sliding Window Attention |
| Mistral Large 3 | MistralLarge3ForCausalLM                | mistral_large_3.py | 大规模 Mistral                           |



| 家族                                      | HF 类名 → vLLM 映射                         | 实现文件                              | 特性                                |
|-----------------------------------------|-----------------------------------------|-----------------------------------|-----------------------------------|
| <b>Mixtral (MoE)</b>                    | MixtralForCausalLM → MixtralForCausalLM | mixtral.py                        | 8×7B/8×22B<br>稀疏 MoE              |
| <b>DeepSeek V1/V2</b>                   | DeepseekForCausalLM/V2ForCausalLM       | deepseek_v2.py                    | MHA +<br>Dense                    |
| <b>DeepSeek V3</b>                      | DeepseekV3ForCausalLM                   | deepseek_v2.py                    | MLA + MoE                         |
| <b>DeepSeek V4</b>                      | DeepseekV4ForCausalLM                   | deepseek_v4.py                    | MLA + MTP                         |
| <b>Qwen2</b>                            | Qwen2ForCausalLM → Qwen2ForCausalLM     | qwen2.py                          | 阿里通义千问<br>二代                      |
| <b>Qwen2-MoE</b>                        | Qwen2MoeForCausalLM                     | qwen2_moe.py                      | Qwen2 MoE<br>变体                   |
| <b>Qwen3</b>                            | Qwen3ForCausalLM                        | qwen3.py                          | 阿里通义千问<br>三代                      |
| <b>Qwen3-MoE</b>                        | Qwen3MoeForCausalLM                     | qwen3_moe.py                      | Qwen3 MoE<br>变体                   |
| <b>Qwen3.5</b>                          | Qwen3_5ForCausalLM                      | qwen3_5.py                        | 阿里通义千问<br>3.5 代                   |
| <b>Gemma</b>                            | GemmaForCausalLM                        | gemma.py                          | Google<br>Gemma<br>(GeLU+RMSNorm) |
| <b>Gemma 2</b>                          | Gemma2ForCausalLM                       | gemma2.py                         | Gemma 2 代                         |
| <b>Gemma 3/4</b>                        | Gemma3/4ForCausalLM                     | gemma3.py,<br>gemma4.py           | 多模态<br>Gemma                      |
| <b>Phi / Phi3</b>                       | PhiForCausalLM / Phi3ForCausalLM        | phi.py, phi3.py                   | Microsoft Phi<br>系列               |
| <b>Phi-MoE</b>                          | PhiMoEForCausalLM                       | phimoe.py                         | Microsoft Phi<br>MoE              |
| <b>GPT-2 /<br/>GPT-J /<br/>GPT-NeoX</b> | 各自 ForCausalLM                          | gpt2.py, gpt_j.py,<br>gpt_neox.py | 经典架构                              |
| <b>Bloom</b>                            | BloomForCausalLM                        | bloom.py                          | BigScience<br>Bloom               |
| <b>OPT</b>                              | OPTForCausalLM                          | opt.py                            | Meta OPT                          |
| <b>Falcon</b>                           | FalconForCausalLM                       | falcon.py                         | TII Falcon                        |
| <b>Baichuan</b>                         | BaiChuan/BaichuanForCausalLM            | baichuan.py                       | 百川系列                              |
| <b>ChatGLM / GLM</b>                    | ChatGLM/GLM/Glm4ForCausalLM             | chatglm.py, glm.py,<br>glm4.py    | 智谱 GLM 系<br>列                     |
| <b>InternLM</b>                         | InternLM/2/3ForCausalLM                 | internlm2.py                      | 书生<br>InternLM                    |
| <b>OLMo /<br/>OLMo2</b>                 | Olmo/Olmo2ForCausalLM                   | olmo.py, olmo2.py                 | AllenAI<br>OLMo                   |
| <b>Mamba /<br/>Mamba2</b>               | Mamba/Mamba2ForCausalLM                 | mamba.py,<br>mamba2.py            | SSM 架构 (无<br>Attention)           |
| <b>Jamba</b>                            | JambaForCausalLM                        | jamba.py                          | Hybrid<br>(Mamba+Attention)       |
| <b>StableLM</b>                         | StableLMEpoch/StableLmForCausalLM       | stablelm.py                       | Stability AI                      |
| <b>StarCoder2</b>                       | Starcoder2ForCausalLM                   | starcoder2.py                     | 代码生成                              |
| <b>Granite /<br/>Granite-MoE</b>        | Granite/GraniteMoeForCausalLM           | granite.py,<br>granitemoe.py      | IBM Granite                       |

| 家族               | HF 类名 → vLLM 映射                                       | 实现文件                           | 特性                  |
|------------------|-------------------------------------------------------|--------------------------------|---------------------|
| <b>Cohere</b>    | Cohere/Cohere2ForCausalLM                             | commandr.py,<br>cohere2_moe.py | Cohere 系列           |
| <b>Command-R</b> |                                                       |                                |                     |
| <b>DBRX</b>      | DbrxForCausalLM                                       | dbrx.py                        | Databricks<br>MoE   |
| <b>Arctic</b>    | ArcticForCausalLM                                     | arctic.py                      | Snowflake<br>Arctic |
| <b>Nemotron</b>  | Nemotron/HForCausalLM                                 | nemotron.py,<br>nemotron_h.py  | NVIDIA<br>Nemotron  |
| <b>Grok</b>      | Grok1ForCausalLM                                      | grok1.py                       | xAI Grok-1          |
| <b>Exaone</b>    | Exaone/4/MoEForCausalLM                               | exaone.py,<br>exaone_moe.py    | LG AI<br>Research   |
| <b>MiniCPM</b>   | MiniCPM/3ForCausalLM                                  | minicpm.py,<br>minicpm3.py     | 面壁<br>MiniCPM       |
| <b>HunYuan</b>   | HunYuanMoEV1/DenseV1ForCausalLM                       | hunyuan_v1.py                  | 腾讯混元                |
| <b>TeleChat</b>  | TeleChat/2/3ForCausalLM                               | telechat2.py                   | 中国电信                |
| <b>Step</b>      | Step1/3Text/3p5ForCausalLM                            | step1.py,<br>step3_text.py     | 阶跃星辰                |
| <b>Solar</b>     | SolarForCausalLM                                      | solar.py                       | Upstage<br>Solar    |
| <b>Plamo</b>     | Plamo2/3ForCausalLM                                   | plamo2.py, plamo3.py           | NAVER<br>Plamo      |
| 其他               | RW/FlexOlmo/Persimmon/Orion/Laguna/Zamba2各自...<br>... | 各自.py 文件                       | 30+ 其他架构            |

## 2.2 嵌入模型 (Embedding Models)

| 家族                   | HF 类名                   | 实现文件               | 用途               |
|----------------------|-------------------------|--------------------|------------------|
| <b>BERT</b>          | BertModel               | bert.py            | 文本嵌入             |
| <b>RoBERTa</b>       | RobertaModel            | roberta.py         | 文本嵌入             |
| <b>BGE-M3</b>        | BgeM3EmbeddingModel     | roberta.py         | 多功能嵌入            |
| <b>GTE</b>           | GteModel/NewModel       | bert_with_rope.py  | 阿里 GTE           |
| <b>Jina</b>          | JinaEmbeddingsV5Model   | jina.py            | Jina Embeddings  |
| <b>ModernBERT</b>    | ModernBertModel         | modernbert.py      | 现代化 BERT         |
| <b>NomicBERT</b>     | NomicBertModel          | bert_with_rope.py  | Nomic            |
| <b>CLIP / Siglip</b> | CLIPModel / SiglipModel | clip.py, siglip.py | 视觉嵌入             |
| <b>ColPali</b>       | ColPaliForRetrieval     | colpali.py         | Late Interaction |
| <b>ColBERT</b>       | HF_ColBERT              | colbert.py         | Late Interaction |

## 2.3 多模态模型 (Multimodal Models)

| 家族                     | HF 类名                                  | 实现文件               | 模态         |
|------------------------|----------------------------------------|--------------------|------------|
| <b>LLaVA</b>           | LlavaForConditionalGeneration          | llava.py           | 图像         |
| <b>LLaVA-NeXT</b>      | LlavaNextForConditionalGeneration      | llava_next.py      | 图像 (动态分辨率) |
| <b>LLaVA-OneVision</b> | LlavaOnevisionForConditionalGeneration | llava_onevision.py | 图像 + 视频    |
| <b>Qwen-VL</b>         | QwenVLForConditionalGeneration         | qwen_vl.py         | 图像         |
| <b>Qwen2-VL</b>        | Qwen2VLForConditionalGeneration        | qwen2_vl.py        | 图像 (动态分辨率) |
| <b>Qwen2.5-VL</b>      | Qwen2_5_VLForConditionalGeneration     | qwen2_5_vl.py      | 图像         |

| 家族                  | HF 类名                                               | 实现文件                     | 模态         |
|---------------------|-----------------------------------------------------|--------------------------|------------|
| <b>Qwen3-VL</b>     | Qwen3VLForConditionalGeneration                     | qwen3_vl.py              | 图像         |
| <b>Qwen2-Audio</b>  | Qwen2AudioForConditionalGeneration                  | qwen2_audio.py           | 音频         |
| <b>Qwen2.5-Omni</b> | Qwen2_5OmniForConditionalGeneration                 | qwen2_5_omni_thinker.py  | 全模态        |
| <b>InternVL</b>     | InternVLChatModel                                   | internvl.py              | 图像         |
| <b>DeepSeek-VL</b>  | DeepseekVLV2ForCausalLM                             | deepseek_vl2.py          | 图像         |
| <b>DeepSeek-OCR</b> | DeepseekOCR/OCR2ForCausalLM                         | deepseek_ocr.py          | OCR        |
| <b>Phi3-V /</b>     | Phi3VForCausalLM /                                  | phi3v.py, phi4mm.py      | 视觉语言       |
| <b>Phi4-MM</b>      | Phi4MMForCausalLM                                   |                          |            |
| <b>Gemma 3 MM</b>   | Gemma3ForConditionalGeneration                      | gemma3_mm.py             | 图像         |
| <b>Pixtral</b>      | PixtralForConditionalGeneration                     | pixtral.py               | 图像 (原生多图像) |
| <b>Molmo /</b>      | MolmoForCausalLM /                                  | molmo.py, molmo2.py      | 图像         |
| <b>Molmo2</b>       | Molmo2ForConditionalGeneration                      |                          |            |
| <b>MiniCPM-V /</b>  | MiniCPMV / MiniCPMO                                 | minicpmv.py, minicpmo.py | 图像         |
| <b>-O</b>           |                                                     |                          |            |
| <b>GLM-4V</b>       | GLM4VForCausalLM                                    | glm4v.py                 | 视觉         |
| <b>Whisper</b>      | WhisperForConditionalGeneration                     | whisper.py               | 语音识别 (ASR) |
| <b>Ultravox</b>     | UltravoxModel                                       | ultravox.py              | 语音 + 文本    |
| <b>NVLM-D</b>       | NVLM_D_Model                                        | nvlm_d.py                | 多模态        |
| <b>H2OVL</b>        | H2OVLChatModel                                      | h2ovl.py                 | 视觉         |
| <b>Ovis</b>         | Ovis / Ovis2_5                                      | ovis.py, ovis2_5.py      | 视觉         |
| <b>MiMo /</b>       | MiMo/MoM/V2ForCausalLM                              | mimo.py, mimo_v2.py      | 全模态        |
| <b>MiMo-V2</b>      |                                                     |                          |            |
| 其他                  | Idefics3/Fuyu/Chameleon/Aria/SnapLM/Tarsier/<br>... |                          | 50+ 其他多模态  |

## 2.4 推测解码 / Speculative Decoding 模型

| 类型                           | HF 类名                                | 实现文件                     | 说明           |
|------------------------------|--------------------------------------|--------------------------|--------------|
| <b>EAGLE-1/2</b>             | EagleLlama/Mistral/CohereForCausalLM | for_eagle.py 等           | EAGLE 推测解码   |
| <b>EAGLE-3</b>               | Eagle3Llama/Qwen/DeepSeekForCausalLM | for_eagle3.py 等          | EAGLE-3 推测解码 |
| <b>Medusa</b>                | MedusaModel                          | medusa.py                | Medusa 头部    |
| <b>MTP</b>                   | DeepSeekMTP/V4MTP/Gemma4MTP          | deepseek_mtp.py 等        | 多 Token      |
| <b>(Multi-Token)</b>         |                                      |                          | Prediction   |
| <b>DFlash</b>                | DFlashDraftModel                     | qwen3_dflash.py          | Qwen3 DFlash |
|                              |                                      |                          | Draft        |
| <b>Extract Hidden States</b> | ExtractHiddenStatesModel             | extract_hidden_states.py | 提取隐状态        |

## 三、模型接口定义体系

### 3.1 核心接口: VllmModel

位于 interfaces\_base.py:47-57, 这是所有 **vLLM** 模型必须实现的最低契约:

```
@runtime_checkable
class VllmModel(Protocol[T_co]):
    """The interface required for all models in vLLM."""

    def __init__(self, vllm_config: VllmConfig, prefix: str = "") -> None: ...
```

```
def embed_input_ids(self, input_ids: torch.Tensor) -> torch.Tensor:
    """Apply token embeddings to `input_ids`."""
    ...
```

```
def forward(self, input_ids: torch.Tensor,
            positions: torch.Tensor) -> T_co: ...
```

三个核心方法签名约定: - `__init__(vllm_config, prefix)` —统一构造入口 - `embed_input_ids(input_ids)`  
 —Token → Embedding 映射 - `forward(input_ids, positions)` —前向推理

### 3.2 文本生成接口: `VllmModelForTextGeneration`

位于 `interfaces_base.py`:114-124:

```
@runtime_checkable
class VllmModelForTextGeneration(VllmModel[T], Protocol[T]):
    def compute_logits(self, hidden_states: T) -> T | None:
        """Return `None` if TP rank > 0."""
        ...
```

额外要求 `compute_logits()` 方法用于 logits 计算 (TP rank > 0 时返回 None)。

### 3.3 Pooling 接口: `VllmModelForPooling`

位于 `interfaces_base.py`:148-213:

```
@runtime_checkable
class VllmModelForPooling(VllmModel[T_co], Protocol[T_co]):
    is_pooling_model: ClassVar[Literal[True]] = True
    default_seq_pooling_type: ClassVar[SequencePoolingType] = "LAST"
    default_tok_pooling_type: ClassVar[TokenPoolingType] = "ALL"
    attn_type: ClassVar[AttnTypeStr] = "decoder"
    score_type: ClassVar[ScoreType] = "bi-encoder"
    pooler: Pooler # The pooler is only called on TP rank 0.
```

### 3.4 扩展能力接口一览 (`interfaces.py`)

`interfaces.py` 定义了丰富的可选能力接口:

| 接口名                                | 标志位                                         | 功能说明                    |
|------------------------------------|---------------------------------------------|-------------------------|
| <code>SupportsMultiModal</code>    | <code>supports_multimodal = True</code>     | 多模态输入支持 (图像/音频/视频)      |
| <code>SupportsLoRA</code>          | <code>supports_lora = True</code>           | LoRA 适配器支持              |
| <code>SupportsPP</code>            | <code>supports_pp = True</code>             | Pipeline Parallelism 支持 |
| <code>MixtureOfExperts</code>      | (属性检查)                                      | MoE 专家混合架构              |
| <code>HasInnerState</code>         | <code>has_inner_state = True</code>         | 内部状态 (如 Mamba SSM)      |
| <code>IsAttentionFree</code>       | <code>is_attention_free = True</code>       | 无 Attention (纯 SSM)     |
| <code>IsHybrid</code>              | <code>is_hybrid = True</code>               | 混合架构 (Attention + SSM)  |
| <code>SupportsTranscription</code> | <code>supports_transcription = True</code>  | 语音转文字 (ASR)             |
| <code>SupportsRealtime</code>      | <code>supports_realtime = True</code>       | 实时流式 ASR                |
| <code>SupportsEagle</code>         | <code>supports_eagle = True</code>          | EAGLE-1/2 推测解码          |
| <code>SupportsEagle3</code>        | <code>supports_eagle3 = True</code>         | EAGLE-3 推测解码            |
| <code>SupportsMRoPE</code>         | <code>supports_mrope = True</code>          | 多维度 RoPE (3D 位置)        |
| <code>SupportsXDRoPE</code>        | <code>supports_xdrope = True</code>         | 扩展维 RoPE (4D 位置)        |
| <code>SupportsScoreTemplate</code> | <code>supports_score_template = True</code> | Score Template 支持       |

| 接口名                        | 标志位                                  | 功能说明                       |
|----------------------------|--------------------------------------|----------------------------|
| SupportsEncoderCudaGraph   | supports_encoder_cudagraph = True    | 编码器 CUDA Graph 加速          |
| SupportsQuant              | (基类继承)                               | 量化配置自动绑定                   |
| SupportsMambaPrefixCaching | supports_mamba_prefix_caching = True | Mamba 前缀缓存                 |
| HasNoOps                   | has_noops = True                     | 含 No-op 占位层                |
| SupportsCrossEncoding      | score_type = "cross-encoder"         | Cross-Encoder 打分           |
| SupportsLateInteraction    | score_type = "late-interaction"      | Late Interaction (ColBERT) |

## 四、主要模型家族深入分析

### 4.1 LLaMA 系列—最广泛的基础架构

文件: llama.py

LLaMA 是 vLLM 中被最多模型复用的基础架构。从 registry 可以看到, Aquila、InternLM、CWM、Xverse、TeleChat3、IQuestCoder 等大量模型都直接复用 ("llama", "LlamaForCausalLM")。

核心组件结构

```
classDiagram
    class LlamaForCausalLM {
        +lm_head: ParallelLMHead
        +model: LlamaModel
        +forward(input_ids, positions, ...) → logits
    }
    class LlamaModel {
        +embed_tokens: VocabParallelEmbedding
        +layers: list~LlamaDecoderLayer~
        +norm: RMSNorm
        +embed_input_ids(input_ids) → Tensor
        +forward(input_ids, positions, intermediate_tensors) → hidden_states
    }
    class LlamaDecoderLayer {
        +self_attn: LlamaAttention
        +mlp: LlamaMLP
        +input_layernorm: RMSNorm
        +post_attention_layernorm: RMSNorm
        +forward(positions, hidden_states, residual)
    }
    class LlamaAttention {
        +qkv_proj: QKVParallelLinear
        +o_proj: RowParallelLinear
        +rotary_emb: RotaryEmbedding
        +attn: Attention
        +forward(positions, hidden_states) → output
    }
    class LlamaMLP {
        +gate_up_proj: MergedColumnParallelLinear
```

```

+down_proj: RowParallelLinear
+act_fn: SiluAndMul
+forward(x) → output
}

LlamaForCausalLM --> LlamaModel
LlamaModel --> LlamaDecoderLayer : *layers
LlamaDecoderLayer --> LlamaAttention : self_attn
LlamaDecoderLayer --> LlamaMLP : mlp
LlamaDecoderLayer --> RMSNorm : input/post LN

```

关键代码: **LlamaMLP** —SwiGLU 激活函数 (llama.py:81-121)

```

class LlamaMLP(nn.Module):
    def __init__(self, hidden_size, intermediate_size, hidden_act, ...):
        super().__init__()
        self.gate_up_proj = MergedColumnParallelLinear(
            input_size=hidden_size,
            output_sizes=[intermediate_size] * 2, # gate 和 up 合并
            ...
        )
        self.down_proj = RowParallelLinear(
            input_size=intermediate_size,
            output_size=hidden_size,
            ...
        )
        self.act_fn = SiluAndMul() # SwiGLU:  $x * SiLU(gate(x)) * up(x)$ 

    def forward(self, x):
        x, _ = self.gate_up_proj(x)
        x = self.act_fn(x)
        x, _ = self.down_proj(x)
        return x

```

关键代码: **LlamaAttention** —GQA + RoPE (llama.py:124-233)

```

class LlamaAttention(nn.Module):
    def __init__(self, config, hidden_size, num_heads, num_kv_heads, ...):
        # ... TP 分片逻辑 ...
        self.qkv_proj = QKVParallelLinear(...) # Q/K/V 合并投影
        self.o_proj = RowParallelLinear(...)
        self.rotary_emb = get_rope(self.head_dim, max_position=...)
        self.attn = Attention(...)

    def forward(self, positions, hidden_states):
        qkv, _ = self.qkv_proj(hidden_states)
        q, k, v = qkv.split([self.q_size, self.kv_size, self.kv_size], dim=-1)
        q, k = self.rotary_emb(positions, q, k) # RoPE 旋转位置编码
        attn_output = self.attn(q, k, v) # 注意力计算
        output, _ = self.o_proj(attn_output)
        return output

```

关键代码: **LlamaModel.forward** —PP 支持与 EAGLE 输出 (llama.py:395-434)

```

def forward(self, input_ids, positions, intermediate_tensors,
            inputs_embeds=None, **extra_layer_kwargs):
    # PP first rank: embed tokens
    if get_pp_group().is_first_rank:
        if inputs_embeds is not None:
            hidden_states = inputs_embeds
        else:
            hidden_states = self.embed_input_ids(input_ids)
        residual = None
    else:
        # PP non-first rank: receive from previous stage
        hidden_states = intermediate_tensors["hidden_states"]
        residual = intermediate_tensors["residual"]

    aux_hidden_states = self._maybe_add_hidden_state([], 0, hidden_states, residual)

    for idx, layer in enumerate(islice(self.layers, self.start_layer, self.end_layer)):
        hidden_states, residual = layer(positions, hidden_states, residual, ...)
        self._maybe_add_hidden_state(aux_hidden_states, idx + 1, hidden_states, residual)

    if not get_pp_group().is_last_rank:
        return IntermediateTensors({"hidden_states": hidden_states, "residual": residual})

    hidden_states, _ = self.norm(hidden_states, residual)

    if len(aux_hidden_states) > 0:
        return hidden_states, aux_hidden_states # EAGLE 需要 auxiliary hidden states
    return hidden_states

```

## 4.2 DeepSeek V2/V3 – MoE + MLA（多头潜在注意力）

文件: deepseek\_v2.py

DeepSeek V3 引入了两大创新: MLA (Multi-head Latent Attention) 和 MoE (Mixture of Experts)。

DeepSeekV2MoE – 共享专家 + 路由专家 (deepseek\_v2.py:245-318+)

```

class DeepseekV2MoE(nn.Module):
    def __init__(self, config, parallel_config, quant_config, prefix=""):
        super().__init__()
        self.n_routed_experts = config.n_routed_experts # 被路由的专家数
        self.n_shared_experts = config.n_shared_experts # 共享专家数

        # Router: 决定 token 发送给哪些专家
        self.gate = GateLinear(config.hidden_size, config.n_routed_experts, ...)

        # EPLB (Expert Parallel Load Balancing) 配置
        self.enable_eplb = parallel_config.enable_eplb
        self.n_redundant_experts = eplb_config.num_redundant_experts
        self.n_logical_experts = self.n_routed_experts
        self.n_physical_experts = self.n_logical_experts + self.n_redundant_experts

        # Routed experts: 使用 FusedMoE 高性能 kernel
        self.experts = FusedMoE(...)
        # Shared experts: 可选的共享全连接层

```

```

if config.n_shared_experts is not None:
    self.shared_experts = DeepseekV2MLP(...)

```

**DeepSeek V4 —MTP (Multi-Token Prediction)** 文件: deepseek\_v4.py, deepseek\_v4\_mtp.py

DeepSeek V4 进一步引入了 Multi-Token Prediction 能力, 支持一次前向推理预测多个后续 token。

#### 4.3 Qwen2/Qwen3/Qwen3.5 —国产大模型代表

文件: qwen2.py, qwen3.py, qwen3\_5.py

**Qwen3 —复用 Qwen2 架构 (qwen3.py:52-53)**

```

from .qwen2 import Qwen2MLP as Qwen3MLP      # 直接复用 Qwen2 的 MLP
from .qwen2 import Qwen2Model                # 直接复用 Qwen2 的 Model

```

Qwen3 的 Attention 层做了独立实现以支持 **Dual Chunk Attention** (双块注意力) 等新特性 (qwen3.py:59-100):

```

class Qwen3Attention(nn.Module):
    def __init__(self, hidden_size, num_heads, num_kv_heads,
                  rope_parameters, max_position=4096*32,
                  dual_chunk_attention_config=None, ...):
        # ... 标准 GQA 设置 ...
        self.dual_chunk_attention_config = dual_chunk_attention_config
        self.qkv_proj = QKVParallelLinear(hidden_size, self.head_dim, ...)

```

**Qwen3.5 —混合架构 (Hybrid) (qwen3\_5.py:39-44)**

```

from vllm.model_executor.layers.mamba.gdn_linear_attn import GatedDeltaNetAttention
from vllm.model_executor.layers.mamba.mamba_utils import (
    MambaStateCopyFunc, MambaStateShapeCalculator, ...
)

```

```

from .interfaces import HasInnerState, IsHybrid, MixtureOfExperts, ...

```

Qwen3.5 是 **IsHybrid** 模型——同时包含 Attention 层和 Mamba (SSM) 层, 实现了两者的优势互补。

#### 4.4 Mistral / Gemma —开源主流架构

文件: mistral.py, gemma.py

**Mistral —LLaMA 的精简变体 (mistral.py:77-79)**

```

class MistralAttention(LlamaAttention): # 直接继承 LLaMA 的 Attention!
    ...

class MistralMLP(nn.Module):          # 与 LLaMA MLP 结构相同
    ...

```

Mistral 几乎完全复用 LLaMA 的实现, 主要差异在于 **Sliding Window Attention** (滑动窗口注意力) 和 **RoPE** 缩放策略。

**Gemma —GeLU 替代 SwiGLU (gemma.py:33-33, gemma.py:60-80)**

```

from vllm.model_executor.layers.activation import GeluAndMul # GeLU 而非 SiLU!
from vllm.model_executor.layers.layernorm import GemmaRMSNorm # Gemma 专用 Norm

```

```
@cache
```



```
def _get_gemma_act_fn(hidden_act, hidden_activation):
    if hidden_activation is None:
        return GeluAndMul(approximate="tanh") # 近似 GeLU
    elif hidden_activation == "gelu_pytorch_tanh":
        return GeluAndMul(approximate="tanh")
    ...
```

Gemma 的核心区别: - 激活函数: GeLU (近似 tanh) 替代 LLaMA 的 SwiGLU (SiLU) - 归一化: GemmaRMSNorm (归一化前先乘以 weight) - 支持 **GGUF** 量化

#### 4.5 MoE 模型—专家混合架构

Mixtral (mixtral.py:77-100+)

```
class MixtralMoE(nn.Module):
    """A tensor-parallel MoE implementation for Mixtral that shards each expert
    across all ranks."""
    def __init__(self, num_experts, top_k, hidden_size, intermediate_size, ...):
        super().__init__()
        # 使用 FusedMoE 进行高性能 MoE 计算
        self.experts = FusedMoE(
            num_experts=num_experts,
            top_k=top_k,
            hidden_size=hidden_size,
            intermediate_size=intermediate_size,
            ...
        )
```

MoE 模型通用接口 (interfaces.py:844-926)

```
@runtime_checkable
class MixtureOfExperts(Protocol):
    expert_weights: MutableSequence[Sequence[Tensor]] # 各层专家权重
    num_moe_layers: int # MoE 层数
    num_expert_groups: int # 专家组数
    num_logical_experts: int # 逻辑专家数
    num_physical_experts: int # 物理专家数
    num_local_physical_experts: int # 本地物理专家数
    num_routed_experts: int # 被路由专家数
    num_shared_experts: int # 共享专家数
    num_redundant_experts: int # 冗余专家数 (EPLB)
    moe_layers: Iterable[nn.Module] # MoE 层列表

    def set_eplb_state(self, expert_load_view, logical_to_physical_map,
                       logical_replica_count) -> None: ...
    def update_physical_experts_metadata(self, num_physical_experts,
                                         num_local_physical_experts) -> None: ...
```

#### 4.6 多模态模型—视觉语言模型

LLaVA 架构 (llava.py:75-80)

```
class LlavaImagePixelInputs(TensorSchema):
    """Dimensions: bn(batch*images), c(channels=3), h(height), w(width)"""
    ...
```

```
class LlavaForConditionalGeneration(nn.Module, SupportsMultiModal, ...):
    supports_multimodal: ClassVar[Literal[True]] = True

    def __init__(self, *, vllm_config: VllmConfig, prefix: str = ""):
        # Vision Tower: CLIP ViT encoder
        self.vision_tower = CLIPVisionModel(...)
        # Multi-modal Projector: 将视觉特征投影到 LLM 空间
        self.multi_modal_projector = nn.Sequential(
            ColumnParallelLinear(...),
            nn.GELU(),
            RowParallelLinear(...),
        )
        # Language Model: 基于 LLaMA
        self.language_model = LlamaForCausalLM(vllm_config=vllm_config, ...)
```

多模态模型的核心流程:

```
flowchart LR
    IMG[" 图像输入"] --> VT["Vision Tower<br/>(CLIP/Siglip/Pixtral)"]
    VT --> VF[" 视觉特征 <br/>(patch embeddings)"]
    VF --> PROJ["Multi-modal Projector<br/>(线性投影)"]
    PROJ --> MM_EMB[" 多模态 Embeddings"]
    TOK["Token IDs"] --> EMB["Text Embeddings"]
    MM_EMB --> MERGE[" 合并 Embeddings"]
    EMB --> MERGE
    MERGE --> LLM["Language Model<br/>(LLaMA/Qwen/etc.)"]
    LLM --> OUT[" 生成输出"]

    style IMG fill:#fff3e0
    style LLM fill:#e8f5e9
```

五、Transformer 层组件详解

5.1 Attention 层

位于 `model_executor/layers/attention/` 目录，提供多种注意力实现:

| 文件                                      | 类名                                 | 用途                       |
|-----------------------------------------|------------------------------------|--------------------------|
| <code>attention.py</code>               | <code>Attention</code>             | 标准 Decoder-only 注意力 (核心) |
| <code>encoder_only_attention.py</code>  | <code>EncoderOnlyAttention</code>  | 双向注意力 (嵌入模型用)            |
| <code>mha_attention.py</code>           | <code>MLAAttention</code>          | DeepSeek MLA 多头潜在注意力     |
| <code>chunked_local_attention.py</code> | <code>ChunkedLocalAttention</code> | 分块局部注意力                  |
| <code>cross_attention.py</code>         | <code>CrossAttention</code>        | 交叉注意力 (多模态用)             |
| <code>mm_encoder_attention.py</code>    | <code>MMEncoderAttention</code>    | 多模态编码器注意力                |
| <code>static_sink_attention.py</code>   | <code>StaticSinkAttention</code>   | 静态 Sink 注意力              |

导出列表 (`init.py`):

```
__all__ = [
    "Attention", "ChunkedLocalAttention", "CrossAttention",
    "EncoderOnlyAttention", "MLAAttention",
    "MMEncoderAttention", "StaticSinkAttention",
]
```

## 5.2 MLP / FeedForward 层

标准 **MLP** (**SwiGLU**) 如 llama.py:81-121 所示, 使用 `MergedColumnParallelLinear` 合并 gate/up 投影 + `SiluAndMul` 激活 + `RowParallelLinear` 下投影。

**MoE 层** (**FusedMoE**) 位于 `model_executor/layers/fused_moe/`, 是 vLLM MoE 推理的核心:

- 主入口: `fused_moe/init.py` — `FusedMoE` 类
- **Expert** 实现: `experts/` 目录下有 15+ 种专家 kernel 实现
  - `cutlass_moe.py` — NVIDIA CUTLASS MoE
  - `deep_gemm_moe.py` — DeepGEMM MoE
  - `flashinfer_cutlass_moe.py` — FlashInfer + CUTLASS
  - `trtllm_fp8_moe.py`, `trtllm_bf16_moe.py` — TensorRT-LLM MoE
  - `aiter_mxfp4_w4a8_moe.py` — AMD AITER MXFP4 MoE
  - `cpu_moe.py`, `xpu_moe.py` — CPU/XPU 回退
- 配置文件: `configs/` 目录含数百个针对不同 GPU 型号/E/dtype 的预调优配置

## 5.3 RMSNorm / LayerNorm

文件: `layernorm.py`

RMSNorm 作为 CustomOp 注册 (`layernorm.py:37-38`):

```
@CustomOp.register("rms_norm")
class RMSNorm(CustomOp):
    """Root mean square normalization.
    Computes  $x \rightarrow w * x / \sqrt{E[x^2] + \epsilon}$ """
    def __init__(self, hidden_size, eps=1e-6, var_hidden_size=None,
                 has_weight=True, dtype=None):
        super().__init__()
        self.hidden_size = hidden_size
        self.variance_epsilon = eps
        self.weight = nn.Parameter(torch.ones(hidden_size, dtype=weight_dtype))
```

RMSNorm 提供多后端分发: - `forward_cuda()` — CUDA kernel (高性能) - `forward_native()` — PyTorch 原生实现 (fallback / torch.compile 兼容) - `forward_cpu()` / `forward_xpu()` / `forward_tpu()` — 其他平台

此外还有 **GemmaRMSNorm** (Gemma 专用, 先乘 weight 再 normalize)、**LayerNorm** (标准 LayerNorm)、**PolyNorm** (多项式归一化) 等变体。

## 5.4 RoPE 位置编码

目录: `model_executor/layers/rotary_embedding/` (19 个文件)

vLLM 实现了极其丰富的 **RoPE** 变体, 覆盖主流模型的所有位置编码方案:

| 文件                                       | RoPE 变体          | 使用模型           |
|------------------------------------------|------------------|----------------|
| <code>base.py</code>                     | 基础 RoPE          | LLaMA, 多数模型    |
| <code>linear_scaling_rope.py</code>      | 线性缩放             | LLaMA 2 长版本    |
| <code>ntk_scaling_rope.py</code>         | NTK-aware 缩放     | Code LLaMA     |
| <code>dynamic_ntk_scaling_rope.py</code> | 动态 NTK           | Grok-1         |
| <code>dynamic_ntk_alpha_rope.py</code>   | 动态 NTK- $\alpha$ | DeepSeek       |
| <code>yarn_scaling_rope.py</code>        | YaRN 缩放          | Mistral, Suno  |
| <code>llama3_rope.py</code>              | LLaMA 3 RoPE     | LLaMA 3        |
| <code>mrope.py</code>                    | M-RoPE (3D)      | Qwen2-VL       |
| <code>xdrope.py</code>                   | XD-RoPE (4D)     | Qwen2.5-VL     |
| <code>deepseek_scaling_rope.py</code>    | DeepSeek 缩放      | DeepSeek V2/V3 |

| 文件                                         | RoPE 变体            | 使用模型       |
|--------------------------------------------|--------------------|------------|
| <code>gemma4_rope.py</code>                | Gemma 4 RoPE       | Gemma 4    |
| <code>fope.py</code>                       | FOPE               | 某些新模型      |
| <code>telechat3_scaling_rope.py</code>     | TeleChat3 缩放       | TeleChat 3 |
| <code>dual_chunk_rope.py</code>            | Dual Chunk         | Qwen3      |
| <code>llama4_vision_rope.py</code>         | LLaMA 4 Vision     | LLaMA 4    |
| <code>ernie45_vl_rope.py</code>            | Ernie 4.5 VL       | Ernie 4.5  |
| <code>phi3_long_rope_scaled_rope.py</code> | Phi-3 Long RoPE    | Phi-3      |
| <code>mrope_interleaved.py</code>          | M-RoPE Interleaved | 某些 VL 模型   |

统一入口 `get_rope()` 根据配置自动选择对应的 RoPE 实现。

## 六、自定义操作注册 (Custom Op)

文件: `custom_op.py`

### 6.1 CustomOp —基础设施

`CustomOp` 是 vLLM 所有自定义操作的基类 (`custom_op.py:103-107`):

```
class CustomOp(nn.Module):
    """
    Base class for custom ops.
    Dispatches the forward method to the appropriate backend.
    """
```

核心设计理念: 通过装饰器注册 + 运行时分发, 实现一套代码适配多硬件后端。

### 6.2 注册机制

```
op_registry: dict[str, type["CustomOp"] | type["PluggableLayer"]] = {}
op_registry_out: dict[str, type["CustomOp"] | type["PluggableLayer"]] = {}
```

两种注册方式:

1. `@CustomOp.register(name)` (`custom_op.py:307-320`) —注册到 `op_registry`:

```
@classmethod
def register(cls, name: str, dynamic_arg_dims=None):
    def decorator(op_cls):
        assert name not in op_registry, f"Duplicate op name: {name}"
        op_cls.name = name
        op_cls._dynamic_arg_dims = dynamic_arg_dims
        op_registry[name] = op_cls
        return op_cls
    return decorator
```

2. `@CustomOp.register_out(name)` (`custom_op.py:332-350`) —注册到 `op_registry_out` (Out-of-Tree 替换):

```
@classmethod
def register_out(cls, _decorated_op_cls=None, name=None):
    def decorator(op_cls):
        reg_name = name or cls.__name__
        op_registry_out[reg_name] = op_cls
    return decorator
```

```

        return op_cls
    ...

```

### 6.3 后端分发机制

`dispatch_forward()` 方法 (custom\_op.py:174-207) 根据当前运行平台选择对应的前向方法:

flowchart TD

```

    INIT["CustomOp.__init__()"] --> DISPATCH["dispatch_forward()"]
    DISPATCH --> ENABLED{"enabled()?"}
    ENABLED -->|Yes| PLATFORM{"current_platform?"}
    ENABLED -->|No| COMPILE["torch.compile(forward_native)"]
    PLATFORM -->|ROCm (AMD)| HIP["forward_hip()"]
    PLATFORM -->|CPU| CPU_F["forward_cpu()"]
    PLATFORM -->|TPU| TPU["forward_tpu()"]
    PLATFORM -->|XPU| XPU["forward_xpu()"]
    PLATFORM -->|OOT| OOT["forward_oot()"]
    PLATFORM -->|CUDA (默认)| CUDA["forward_cuda()"]

```

```

style INIT fill:#e3f2fd
style CUDA fill:#c8e6c9
style HIP fill:#fff3e0
style COMPILE fill:#fce4ec

```

### 6.4 启用/禁用控制

`enabled()` 类方法 (custom\_op.py:272-289) 通过 `CompilationConfig.custom_ops` 控制:

```

@classmethod
def enabled(cls) -> bool:
    compilation_config = get_cached_compilation_config()
    custom_ops = compilation_config.custom_ops
    enabled = f"{cls.name}" in custom_ops
    disabled = f"-{cls.name}" in custom_ops
    return (CustomOp.default_on() or enabled) and not disabled

```

用户可通过 `--enforce-eager` 或自定义 `op` 配置来精细控制每个 `custom op` 的启用状态。

### 6.5 PluggableLayer —可插拔层

`PluggableLayer` (custom\_op.py:32-100) 与 `CustomOp` 不同:

- **CustomOp**: 同一个类, 不同 **forward** 方法分派到不同后端 (适合 kernel 级别的替换)
- **PluggableLayer**: 整个类可以被 **OOT** 替换 (适合架构级别的替换, 如替换整个 attention 层)

两者均支持 `register_OOT` 用于第三方插件替换内置实现。

## 七、模型注册与加载完整流程

sequenceDiagram

```

participant User as 用户
participant Engine as LLMEngine
participant MR as ModelRegistry
participant Lazy as LazyRegisteredModel
participant SubProc as 子进程 (inspect)
participant ModelCls as 模型类 (如 LlamaForCausalLM)

```

```

participant Loader as WeightLoader

User->>Engine: LLM(model="meta-llama/Llama-3.1-8B")
Engine->>Engine: 解析 config.json → architectures=["LlamaForCausalLM"]
Engine->>MR: resolve_model_cls(["LlamaForCausalLM"], config)
MR->>MR: _normalize_arch("LlamaForCausalLM")
MR->>Lazy: _try_load_model_cls()
Lazy->>Lazy: importlib.import_module("vllm.model_executor.models.llama")
Lazy-->>MR: 返回 LlamaForCausalLM 类
MR-->>Engine: (LlamaForCausalLM, "LlamaForCausalLM")

Engine->>MR: inspect_model_cls(architectures, config)
MR->>SubProcess: _run_in_subprocess(_ModelInfo.from_model_cls)
Note over SubProcess: 子进程中安全导入模型 <br/> 避免 CUDA 重复初始化
SubProcess-->>MR: _ModelInfo(各能力标志位)
MR-->>Engine: (model_info, arch)

Engine->>Engine: 根据 model_info 创建 Runner
Engine->>ModelCls: __init__(vllm_config=vllm_config)
ModelCls->>ModelCls: 创建 embed_tokens, layers, norm, lm_head
Engine->>Loader: load_weights(model, weights_iter)
Loader->>ModelCls: model.load_weights(weights)
Note over ModelCls: stacked_params_mapping<br/> 合并 QKV/GateUp 权重

Engine-->>User: 模型就绪

```

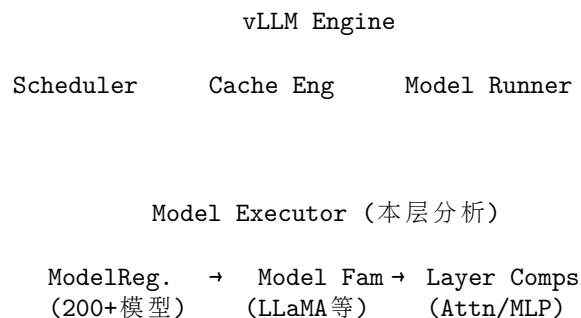
---

## 八、架构总结

### 8.1 设计亮点

1. **Lazy Loading** + 子进程 **Inspect**: 避免启动时导入 200+ 模型文件，同时绕过 CUDA fork 限制
2. 架构复用: LLaMA 作为“元架构”，被 Aquila/InternLM/TeleChat/Qwen 等 20+ 模型直接复用
3. 接口协议驱动: 通过 Python Protocol (structural typing) 定义能力接口，无需强制继承
4. **CustomOp** 多后端分发: 一套代码自动适配 CUDA/ROCm/CPU/TPU/XPU/OOT
5. **PluggableLayer** OOT 扩展: 支持第三方插件零侵入式替换任意层组件
6. **MoE** 丰富生态: 15+ 种 Expert Kernel 实现 + 数百个预调优配置 + EPLB 负载均衡

### 8.2 模型执行器的角色定位



```
Custom Op (多后端分发)
CUDA / ROCm / CPU / TPU / XPU / OOT
-
```

```
Kernels (CUDA/Triton/Custom Ops)
vllm._custom_ops / attention kernels / etc.
```

## vLLM Worker 与执行框架分析

### 定位

本文档深入分析 vLLM v1 架构中的 执行框架 (**Executor**) 和 **Worker** 层次结构。执行框架负责模型在不同硬件设备上的调度与执行，是 vLLM 推理引擎的核心基础设施。该框架采用分层设计，通过 **Executor** → **Worker** → **ModelRunner** 三层抽象，实现了对单 GPU、多 GPU、多节点以及异构硬件的统一管理。

```
graph TB
    subgraph "vLLM 执行框架层次"
        A[Engine 调度层] --> B[Executor 执行层]
        B --> C[Worker 工作进程]
        C --> D[ModelRunner 推理执行]
    end

    end

    subgraph "Executor 实现"
        B1[UniProcExecutor<br/> 单进程] -->|fill:#ede7f6| B
        B2[MultiprocExecutor<br/> 多进程] -->|fill:#ede7f6| B
        B3[RayExecutorV2<br/> 分布式] -->|fill:#ede7f6| B
    end

    end

    subgraph "Worker 实现"
        C1[GPUWorker] -->|fill:#ede7f6| C
        C2[CPUWorker] -->|fill:#ede7f6| C
        C3[XPUWorker] -->|fill:#ede7f6| C
    end

    end

    subgraph "ModelRunner"
        D1[GPUModelRunner] -->|fill:#ede7f6| D
        D2[CPUModelRunner] -->|fill:#ede7f6| D
        D3[XPUModelRunner] -->|fill:#ede7f6| D
    end

    end

    style B fill:#ede7f6,stroke:#673ab7,stroke-width:3px,color:#000
    style C fill:#ede7f6,stroke:#673ab7,stroke-width:3px,color:#000
    style D fill:#ede7f6,stroke:#673ab7,stroke-width:3px,color:#000
```

---

## 一、Executor 抽象基类

### 1.1 核心职责

**Executor** 是所有执行器的抽象基类，定义了统一的执行接口。它负责：- 生命周期管理：初始化、健康检查、关闭 - **RPC** 通信：向所有 Worker 广播控制消息 - 模型执行：协调模型前向传播和采样 - 资源管理：KV Cache、LoRA、内存休眠/唤醒

源码位置：abstract.py

### 1.2 类定义与关键属性

```
class Executor(ABC):
    """Abstract base class for vLLM executors.

    An executor is responsible for executing the model on one device,
    or it can be a distributed executor that can execute the model on multiple devices.
    """

    uses_ray: bool = False # whether the executor uses Ray for orchestration.
    supports_pp: bool = False # whether the executor supports PP
```

- 第 37-46 行：定义类别属性标识是否使用 Ray 以及是否支持 Pipeline Parallelism。

### 1.3 Executor 选择策略

get\_class() 方法根据配置动态选择 Executor 实现：

```
@staticmethod
def get_class(vllm_config: VllmConfig) -> type["Executor"]:
    executor_class: type[Executor]
    parallel_config = vllm_config.parallel_config
    distributed_executor_backend = parallel_config.distributed_executor_backend
    if isinstance(distributed_executor_backend, type):
        if not issubclass(distributed_executor_backend, Executor):
            raise TypeError(...)
        executor_class = distributed_executor_backend
    elif distributed_executor_backend == "ray":
        if envs.VLLM_USE_RAY_V2_EXECUTOR_BACKEND:
            from vllm.v1.executor.ray_executor_v2 import RayExecutorV2
            executor_class = RayExecutorV2
        else:
            from vllm.v1.executor.ray_executor import RayDistributedExecutor
            executor_class = RayDistributedExecutor
    elif distributed_executor_backend == "mp":
        from vllm.v1.executor.multiproc_executor import MultiprocExecutor
        executor_class = MultiprocExecutor
    elif distributed_executor_backend == "uni":
        from vllm.v1.executor.uniproc_executor import UniProcExecutor
        executor_class = UniProcExecutor
    # ... 其他分支
```

- 第 47-92 行：支持以下后端选择：
  - "uni" → UniProcExecutor（单进程）
  - "mp" → MultiprocExecutor（多进程）
  - "ray" → RayExecutorV2 / RayDistributedExecutor（Ray 分布式）
  - 自定义类或字符串限定名



## 1.4 统一接口定义

### 初始化与配置

```
def __init__(self, vllm_config: VllmConfig) -> None:
    self.vllm_config = vllm_config
    self.model_config = vllm_config.model_config
    self.cache_config = vllm_config.cache_config
    self.lora_config = vllm_config.lora_config
    self.load_config = vllm_config.load_config
    self.parallel_config = vllm_config.parallel_config
    self.scheduler_config = vllm_config.scheduler_config
    self.device_config = vllm_config.device_config
    self.speculative_config = vllm_config.speculative_config
    self.observability_config = vllm_config.observability_config
    self._init_executor()
    self.is_sleeping = False
    self.sleeping_tags: set[str] = set()
    self.kv_output_aggregator: KVOutputAggregator | None = None
```

- 第 94-112 行: 存储完整配置并调用子类实现的 `_init_executor()`。

### 核心 RPC 接口

```
@abstractmethod
def collective_rpc(
    self,
    method: str | Callable[[WorkerBase], _R],
    timeout: float | None = None,
    args: tuple = (),
    kwargs: dict | None = None,
    non_block: bool = False,
):
    raise NotImplementedError
```

- 第 198-202 行: 这是 `Executor` 的核心抽象方法, 用于向所有 `Worker` 发送 RPC 调用。
- 支持同步和异步 (返回 `Future`) 两种模式。
- `method` 可以是方法名字符串或可序列化的 `Callable`。

### 模型执行接口

```
def execute_model(
    self, scheduler_output: SchedulerOutput, non_block: bool = False
) -> ModelRunnerOutput | None | Future[ModelRunnerOutput | None]:
    output = self.collective_rpc(
        "execute_model", args=(scheduler_output,), non_block=non_block
    )
    return output[0]

def sample_tokens(
    self, grammar_output: GrammarOutput | None, non_block: bool = False
) -> ModelRunnerOutput | Future[ModelRunnerOutput]:
    output = self.collective_rpc(
        "sample_tokens", args=(grammar_output,), non_block=non_block
    )
    return output[0]
```

- 第 221-247 行: 将 Scheduler 的输出分发给 Worker 执行, 并收集结果。

其他管理接口

| 方法名                                                  | 功能                 | 行号      |
|------------------------------------------------------|--------------------|---------|
| <code>initialize_from_config()</code>                | 初始化 KV Cache 并编译模型 | 118-137 |
| <code>determine_available_memory()</code>            | 确定可用 GPU 内存        | 146-147 |
| <code>check_health()</code>                          | 健康检查               | 274-278 |
| <code>shutdown()</code>                              | 关闭 Executor        | 280-282 |
| <code>sleep()</code> / <code>wake_up()</code>        | 内存休眠/唤醒            | 322-360 |
| <code>add_lora()</code> / <code>remove_lora()</code> | LoRA 管理            | 296-306 |

## 二、UniProcExecutor 单进程执行器

### 2.1 适用场景

适用场景: 单 GPU 推理、开发调试、简单部署场景。

特点: - 在主进程中直接创建 Worker, 无额外进程开销 - 支持异步调度 (`async_scheduling`) - 适用于 `tensor_parallel_size=1` 且无 Pipeline Parallelism 的场景

源码位置: `uniproc_executor.py`

### 2.2 实现细节

初始化流程

```
class UniProcExecutor(Executor):
    def _init_executor(self) -> None:
        """Initialize the worker and load the model."""
        self.driver_worker = WorkerWrapperBase(rpc_rank=0)
        distributed_init_method, rank, local_rank = self._distributed_args()
        kwargs = dict(
            vllm_config=self.vllm_config,
            local_rank=local_rank,
            rank=rank,
            distributed_init_method=distributed_init_method,
            is_driver_worker=True,
            shared_worker_lock=Lock(),
        )

        self.driver_worker.init_worker(all_kwargs=[kwargs])
        self.driver_worker.init_device()

        if envs.VLLM_ELASTIC_EP_SCALE_UP_LAUNCH:
            self.driver_worker.elastic_ep_execute("load_model")
        else:
            self.driver_worker.load_model()
            current_platform.update_block_size_for_backend(self.vllm_config)
```

- 第 46-66 行: 创建单个 WorkerWrapperBase 实例并在当前进程初始化。

分布式参数获取

```
def _distributed_args(self) -> tuple[str, int, int]:
    """Return (distributed_init_method, rank, local_rank)."""
    distributed_init_method = get_distributed_init_method(get_ip(), get_open_port())
    device_info = self.vllm_config.device_config.device.__str__().split(":")
    local_rank = int(device_info[1]) if len(device_info) > 1 else 0
    return distributed_init_method, 0, local_rank
```

- 第 68-74 行: 单进程模式下 rank 始终为 0。

RPC 实现 (直接函数调用)

```
def collective_rpc(
    self,
    method: str | Callable,
    timeout: float | None = None,
    args: tuple = (),
    kwargs: dict | None = None,
    non_block: bool = False,
    single_value: bool = False,
) -> Any:
    if kwargs is None:
        kwargs = {}

    if not non_block:
        result = run_method(self.driver_worker, method, args, kwargs)
        return result if single_value else [result]

    try:
        result = run_method(self.driver_worker, method, args, kwargs)
        if isinstance(result, AsyncModelRunnerOutput):
            return AsyncOutputFuture(result, single_value)
        future = Future[Any]()
        future.set_result(result if single_value else [result])
    except Exception as e:
        future = Future[Any]()
        future.set_exception(e)
    return future
```

- 第 80-105 行: 关键点: UniProcExecutor 的 RPC 是直接函数调用, 无需序列化或进程间通信。

异步输出封装

```
class AsyncOutputFuture(Future):
    def __init__(self, async_output: AsyncModelRunnerOutput, single_value: bool):
        self.async_output = async_output
        self.single_value = single_value
        super().__init__()

    def result(self, timeout=None):
        if timeout is not None:
            raise RuntimeError("timeout not implemented")

        if not super().done():
            try:
```

```

        output = self.async_output.get_output()
        self.set_result(output if self.single_value else [output])
    except Exception as e:
        self.set_exception(e)
    return super().result()

```

- 第 26-42 行：封装异步输出为标准 Future 接口。

### 2.3 ExecutorWithExternalLauncher

```

class ExecutorWithExternalLauncher(UniProcExecutor):
    """An executor that uses external launchers to launch engines,
    specially designed for torchrun-compatible launchers...
    """

```

- 第 149-195 行：支持 torchrun 等外部启动器，使用环境变量 RANK、LOCAL\_RANK、MASTER\_ADDR 进行分布式初始化。

## 三、MultiProcExecutor 多进程执行器

### 3.1 适用场景

适用场景：多 GPU 单机推理、Tensor Parallelism、Pipeline Parallelism。

特点：- 使用 Python multiprocessing 创建多个 Worker 进程 - 基于 ZMQ MessageQueue 进行进程间通信 - 支持跨节点扩展（通过 NCCL 数据平面）

源码位置：multiproc\_executor.py

### 3.2 进程架构

```

graph TB
    subgraph "主进程 (Executor)"
        E[MultiProcExecutor]
        MQ_Broad[rpc_broadcast_mq<br/> 广播消息队列]
        MQ_Resp[response_mqs<br/> 响应消息队列]
        Monitor[WorkerMonitor<br/> 健康监控线程]
    end

    subgraph "Worker 进程"
        W0[WorkerProcess-0<br/>rank=0]
        W1[WorkerProcess-1<br/>rank=1]
        WN[WorkerProcess-N<br/>rank=N]
    end

    E --> MQ_Broad
    E --> MQ_Resp
    E --> Monitor

    MQ_Broad -->|SchedulerOutput| W0
    MQ_Broad -->|SchedulerOutput| W1
    MQ_Broad -->|SchedulerOutput| WN

    W0 -->|ModelRunnerOutput| MQ_Resp
    W1 -->|ModelRunnerOutput| MQ_Resp
    WN -->|ModelRunnerOutput| MQ_Resp

```

```

Monitor -->| 监控进程存活 | W0
Monitor -->| 监控进程存活 | W1
Monitor -->| 监控进程存活 | WN

```

### 3.3 初始化流程详解

```

def _init_executor(self) -> None:
    self._finalizer = weakref.finalize(self, self.shutdown)
    self.is_failed = False
    self.failure_callback: FailureCallback | None = None

    tp_size, pp_size, pcg_size = self._get_parallel_sizes()
    assert self.world_size == tp_size * pp_size * pcg_size, (...)

    set_multiprocessing_worker_envs()

    distributed_init_method = get_distributed_init_method(
        get_loopback_ip(), get_open_port()
    )
    self.rpc_broadcast_mq: MessageQueue | None = None
    scheduler_output_handle: Handle | None = None

```

- 第 109-131 行：初始化并行参数和通信基础设施。

并行尺寸计算

```

def _get_parallel_sizes(self) -> tuple[int, int, int]:
    self.world_size = self.parallel_config.world_size
    assert self.world_size % self.parallel_config.nnodes_within_dp == 0, (...)
    self.local_world_size = self.parallel_config.local_world_size
    tp_size = self.parallel_config.tensor_parallel_size
    pp_size = self.parallel_config.pipeline_parallel_size
    pcg_size = self.parallel_config.prefill_context_parallel_size
    return tp_size, pp_size, pcg_size

```

- 第 248-259 行：计算 TP、PP、PCP 尺寸，world\_size 必须等于三者的乘积。

### Worker 进程创建

```

for local_rank in range(self.local_world_size):
    global_rank = global_start_rank + local_rank
    is_driver_worker = self._is_driver_worker(global_rank)
    with cpu_omp_manager.configure_omp_envs(
        rank=global_rank, local_rank=local_rank
    ):
        unready_worker_handle = WorkerProc.make_worker_process(
            vllm_config=self.vllm_config,
            local_rank=local_rank,
            rank=global_rank,
            distributed_init_method=distributed_init_method,
            input_shm_handle=scheduler_output_handle,
            shared_worker_lock=shared_worker_lock,
            is_driver_worker=is_driver_worker,
            inherited_fds=inherited_fds,

```

```

    )
    unready_workers.append(unready_worker_handle)

```

- 第 175-191 行: 为每个 local\_rank 创建 Worker 进程。

### 3.4 进程间通信机制 (MessageQueue)

消息队列架构

```

# Leader node 初始化广播 MQ
if self.parallel_config.node_rank_within_dp == 0:
    max_chunk_bytes = envs.VLLM_MQ_MAX_CHUNK_BYTES_MB * 1024 * 1024
    mq_connect_ip = get_ip()
    self.rpc_broadcast_mq = MessageQueue(
        self.world_size,
        self.local_world_size,
        max_chunk_bytes=max_chunk_bytes,
        connect_ip=mq_connect_ip,
    )
    scheduler_output_handle = self.rpc_broadcast_mq.export_handle()

```

- 第 134-156 行: Leader 节点创建共享内存广播队列。

RPC 调用流程

```

def collective_rpc(
    self,
    method: str | Callable,
    timeout: float | None = None,
    args: tuple = (),
    kwargs: dict | None = None,
    non_block: bool = False,
    unique_reply_rank: int | None = None,
    kv_output_aggregator: KVOutputAggregator | None = None,
) -> Any:
    assert self.rpc_broadcast_mq is not None, (...)

    deadline = None if timeout is None else time.monotonic() + timeout
    kwargs = kwargs or {}

    # 序列化方法
    if isinstance(method, str):
        send_method = method
    else:
        send_method = cloudpickle.dumps(method, protocol=pickle.HIGHEST_PROTOCOL)

    # 入队到广播队列
    self.rpc_broadcast_mq.enqueue((send_method, args, kwargs, output_rank))

    # 从响应队列获取结果
    def get_response():
        responses = []
        for mq in response_mqs:
            dequeue_timeout = (
                None if deadline is None else (deadline - time.monotonic())
            )

```

```

        status, result = mq.dequeue(timeout=dequeue_timeout)
        if status != WorkerProc.ResponseStatus.SUCCESS:
            raise RuntimeError(...)
        responses.append(result)
    return responses[0] if output_rank is not None else responses

future = FutureWrapper(
    self.futures_queue,
    get_response=get_response,
    aggregate=aggregate,
)
return future if non_block else future.result()

```

- 第 339-403 行: 完整的 RPC 调用链路:
  1. 将方法调用信息入队到 `rpc_broadcast_mq`
  2. 所有 Worker 从该队列接收消息
  3. Worker 执行后将结果写入各自的 `worker_response_mq`
  4. Executor 从响应队列收集结果

## 5. WorkerProc 内部实现

Worker 主循环

```

@staticmethod
def worker_main(*args, **kwargs):
    shutdown_requested = threading.Event()

    def signal_handler(signum, frame):
        nonlocal shutdown_requested
        if not shutdown_requested.is_set():
            shutdown_requested.set()
            raise SystemExit()

    signal.signal(signal.SIGTERM, signal_handler)
    signal.signal(signal.SIGINT, signal_handler)

    worker = None
    ready_writer = kwargs.pop("ready_pipe")
    death_pipe = kwargs.pop("death_pipe", None)

    try:
        worker = WorkerProc(*args, **kwargs)
        worker.monitor_death_pipe(death_pipe, shutdown_requested)

        ready_writer.send({
            "status": WorkerProc.READY_STR,
            "handle": worker.worker_response_mq.export_handle(),
            "peer_response_handles": worker.peer_response_handles,
        })

        worker.rpc_broadcast_mq.wait_until_ready()
        worker.worker_response_mq.wait_until_ready()
        ready_writer.close()

        worker.worker_busy_loop()
    
```

```

except Exception:
    logger.exception("WorkerProc failed.")
finally:
    if worker is not None:
        worker.shutdown()

```

- 第 792-896 行: Worker 进程的主入口函数。

### Busy Loop 消息处理

```

def worker_busy_loop(self):
    """Main busy loop for Multiprocessing Workers"""
    assert self.rpc_broadcast_mq is not None
    while True:
        method, args, kwargs, output_rank = self.rpc_broadcast_mq.dequeue(
            indefinite=True
        )
        try:
            if isinstance(method, str):
                func = getattr(self.worker, method)
            elif isinstance(method, bytes):
                func = partial(cloudpickle.loads(method), self.worker)

            output = func(*args, **kwargs)
        except Exception as e:
            if hasattr(e, "add_note"):
                e.add_note(traceback.format_exc())
            logger.exception("WorkerProc hit an exception.")
            if output_rank is None or self.rank == output_rank:
                self.handle_output(e)
            continue

        if output_rank is None or self.rank == output_rank:
            self.handle_output(output)

```

- 第 944-970 行: Worker 的核心事件循环, 持续从广播队列接收 RPC 调用并执行。

### 异步输出处理

```

def async_output_busy_loop(self):
    """Entrypoint for the thread which handles outputs asynchronously."""
    from vllm.platforms import current_platform

    if hasattr(self.worker, "device"):
        current_platform.set_device(self.worker.device)

    while True:
        output = self.async_output_queue.get()
        self.enqueue_output(output)

```

- 第 926-942 行: 当启用 `async_scheduling` 时, 使用独立线程处理异步输出以避免阻塞。

## 3.6 健康监控机制

```

def start_worker_monitor(self, inline=False) -> None:
    workers = self.workers

```



```

self_ref = weakref.ref(self)

def monitor_workers():
    sentinels = [h.proc.sentinel for h in workers]
    died = multiprocessing.connection.wait(sentinels)
    _self = self_ref()
    if not _self or getattr(_self, "shutting_down", False):
        return
    _self.is_failed = True
    proc_name = next(h.proc.name for h in workers if h.proc.sentinel == died[0])
    logger.error("Worker proc %s died unexpectedly...", proc_name)
    _self.shutdown()
    callback = _self.failure_callback
    if callback is not None:
        _self.failure_callback = None
        callback()

Thread(target=monitor_workers, daemon=True, name="MultiprocWorkerMonitor").start()

```

- 第 267-298 行：使用后台线程监控 Worker 进程存活状态，任何 Worker 异常退出都会触发 Executor 关闭。

## 四、RayExecutorV2 Ray 分布式执行器

### 4.1 适用场景

适用场景：多节点集群推理、弹性扩缩容、资源隔离。

特点：- 基于 **Ray Actor** 实现真正的分布式执行 - 复用 MultiprocExecutor 的 MessageQueue 通信机制 - 支持 Placement Group 进行资源调度 - 自动发现 GPU 分配

源码位置：ray\_executor\_v2.py

### 4.2 类继承关系

```

class RayExecutorV2(MultiprocExecutor):
    """Ray-based distributed executor using MessageQueue communication.

    Inherits from MultiprocExecutor to reuse the MQ-based control plane
    and NCCL data plane. Workers are Ray actors.

    Async scheduling is enabled, inherited from MultiprocExecutor.
    This is critical for RayExecutorV2 to be performant.
    """

    uses_ray: bool = True
    supports_pp: bool = True

```

- 第 205-217 行：继承自 MultiprocExecutor，复用其通信和控制逻辑。

### 4.3 初始化流程（10 步）

```

sequenceDiagram
    participant Driver as Driver Process
    participant Ray as Ray Cluster
    participant Actors as Ray Workers (Actors)

```

```

Driver->>Ray: 1. initialize_ray_cluster()
Driver->>Driver: 2. Build bundle assignments
Driver->>Driver: 3. Resolve TCPStore IP
Driver->>Driver: 4. Create broadcast MessageQueue
Driver->>Ray: 5. Spawn RayWorkerProc actors (deferred init)
Driver->>Actors: 6. Discover GPU IDs via runtime context
Actors-->>Driver: Return node_id & gpu_ids
Driver->>Actors: 7. initialize_worker() with correct CUDA_VISIBLE_DEVICES
Actors-->>Driver: Return response MQ handles
Driver->>Actors: 8. Collect response MQ handles
Driver->>Actors: 9. Start run() busy loop
Driver->>Driver: 10. wait_until_ready() barrier

```

步骤 1-4: 集群与通信初始化

```

def _init_executor(self) -> None:
    self._finalizer = weakref.finalize(self, self.shutdown)
    self.is_failed = False
    self.failure_callback = None
    self.shutting_down = False
    self.shutdown_lock = threading.Lock()

    # Step 1: Initialize Ray cluster and retrieve placement group
    initialize_ray_cluster(self.parallel_config, require_gpu_on_driver=False)
    placement_group = self.parallel_config.placement_group

    # Step 2: Build bundle assignments
    if envs.VLLM_RAY_BUNDLE_INDICES:
        bundle_to_node_id = get_bundles_for_indices(...)
    else:
        bundle_to_node_id = get_bundles_sorted_by_node(placement_group)

    # Step 3: Resolve IP for torch.distributed TCPStore
    dist_ip = bundle_assignments[0]["node_ip"]
    distributed_init_method = get_distributed_init_method(dist_ip, get_open_port())

    # Step 4: Create broadcast MessageQueue
    max_chunk_bytes = envs.VLLM_MQ_MAX_CHUNK_BYTES_MB * 1024 * 1024
    n_local = sum(1 for a in bundle_assignments if a["node_id"] == driver_node)
    self.rpc_broadcast_mq = MessageQueue(
        self.world_size, n_local,
        max_chunk_bytes=max_chunk_bytes,
        connect_ip=ray.util.get_node_ip_address(),
    )

```

- 第 249-310 行: 初始化 Ray 集群和消息队列基础设施。

步骤 5: 创建 Ray Actor (延迟初始化)

```

for bundle_idx in range(self.world_size):
    bundle = bundle_assignments[bundle_idx]
    is_driver_worker = self._is_driver_worker(bundle["rank"])
    is_driver_node = bundle["node_id"] == driver_node

    scheduling_strategy = PlacementGroupSchedulingStrategy(

```

```

        placement_group=placement_group,
        placement_group_bundle_index=bundle["bundle_id_idx"],
    )

    actor_name = build_actor_name(
        instance_id, bundle["rank"], tp_size, pp_size, pcp_size
    )

    actor = (
        ray.remote(RayWorkerProc)
        .options(
            name=actor_name,
            num_cpus=0,
            **resource_kwargs,
            scheduling_strategy=scheduling_strategy,
            runtime_env=runtime_env,
        )
        .remote(
            vllm_config=self.vllm_config,
            rank=bundle["rank"],
            distributed_init_method=distributed_init_method,
            input_shm_handle=scheduler_output_handle,
            is_driver_worker=is_driver_worker,
            is_driver_node=is_driver_node,
        )
    )

```

- 第 326-357 行: 使用 Placement Group 调度策略创建 Actor, 但此时不完成完整初始化。

#### 步骤 6-7: GPU 发现与 Worker 初始化

```

# Step 6: Discover GPU IDs assigned to each worker via Ray runtime context.
worker_node_and_gpu_ids = ray.get(
    [h.actor.get_node_and_gpu_ids.remote() for h in self.ray_worker_handles]
)

# Step 7: Initialize workers with correct local_rank and CUDA_VISIBLE_DEVICES
init_worker_refs = []
for i, (node_id, _) in enumerate(worker_node_and_gpu_ids):
    local_rank = node_workers[node_id].index(i)
    worker_env_vars = {
        current_platform.device_control_env_var: ",".join(
            map(str, node_gpus[node_id])
        ),
    }
    self.ray_worker_handles[i].local_rank = local_rank
    init_worker_refs.append(
        self.ray_worker_handles[i].actor.initialize_worker.remote(
            local_rank, worker_env_vars, self.driver_env_vars
        )
    )
ray.get(init_worker_refs)

```

- 第 369-398 行: 关键设计: 先让 Actor 运行以获取实际分配的 GPU ID, 再设置正确的 CUDA\_VISIBLE\_DEVICES 并完成初始化。这允许多个 vLLM 实例在同一节点共存。

#### 4.4 RayWorkerProc 实现

```
class RayWorkerProc(WorkerProc):
    """Worker process that runs inside a Ray actor.

    Initialization is split into two phases:
    1. __init__: lightweight setup, stores init args (no device/model init)
    2. initialize_worker: called after GPU IDs are discovered, completes
       the full WorkerProc initialization with the correct local_rank and
       CUDA_VISIBLE_DEVICES.
    """
```

- 第 75-101 行: 两阶段初始化设计。

#### GPU 发现

```
def get_node_and_gpu_ids(self) -> tuple[str, list[int]]:
    """Return (node_id, gpu_ids) assigned to this actor by Ray."""
    node_id = ray.get_runtime_context().get_node_id()
    device_key = current_platform.ray_device_key
    gpu_ids = ray.get_runtime_context().get_accelerator_ids()[device_key]
    return node_id, [int(x) for x in gpu_ids]
```

- 第 123-132 行: 利用 Ray runtime context 获取分配的物理 GPU ID。

#### 完整初始化

```
def initialize_worker(
    self,
    local_rank: int,
    env_vars: dict[str, str],
    driver_env_vars: dict[str, str] | None = None,
) -> None:
    if driver_env_vars:
        for key, value in driver_env_vars.items():
            os.environ.setdefault(key, value)
    for key, value in env_vars.items():
        os.environ[key] = value

    self.local_rank = local_rank
    super().__init__(
        local_rank=local_rank,
        **self._init_kwargs,
    )
```

- 第 134-156 行: 设置环境变量后调用父类的完整初始化。

#### 4.5 健康监控 (基于 ObjectRef)

```
def start_worker_monitor(self, inline=False) -> None:
    run_refs = [h.run_ref for h in self.ray_worker_handles if h.run_ref is not None]

    def monitor_workers():
        while not _should_stop() and ray.is_initialized():
            try:
                done, _ = ray.wait(run_refs, num_returns=1, timeout=5.0)
            except Exception:
```

```

        return
    if not done or _should_stop():
        continue

    dead_ranks = [ref_to_rank[r] for r in done]
    executor.is_failed = True
    executor.shutdown()
    if executor.failure_callback is not None:
        callback = executor.failure_callback
        executor.failure_callback = None
        callback()
    return

t = threading.Thread(target=monitor_workers, daemon=True, name="RayWorkerMonitor")
t.start()

```

- 第 429-479 行: 通过 `ray.wait()` 监控 Actor 的 `run()` `ObjectRef` 来检测异常退出。

---

## 五、WorkerBase Worker 抽象基类

### 5.1 设计目标

`WorkerBase` 定义了所有硬件类型 Worker 的统一接口，实现了硬件无关的抽象层。具体 Worker 实现 (GPU/CPU/XPU) 只需关注硬件特定的初始化和执行逻辑。

源码位置: `worker_base.py`

### 5.2 核心接口定义

```

class WorkerBase:
    """Worker interface that allows vLLM to cleanly separate implementations for
    different hardware. Also abstracts control plane communication, e.g., to
    communicate request metadata to other workers.
    """

    def __init__(
        self,
        vllm_config: VllmConfig,
        local_rank: int,
        rank: int,
        distributed_init_method: str,
        is_driver_worker: bool = False,
    ) -> None:
        self.vllm_config = vllm_config
        self.model_config = vllm_config.model_config
        # ... 存储所有配置子项
        self.device: torch.device | None = None
        self.model_runner: nn.Module | None = None

```

- 第 38-88 行: 基础构造函数存储配置并声明设备与模型运行器引用。

### 5.3 关键抽象方法

| 方法                                      | 功能描述                  | 行号      |
|-----------------------------------------|-----------------------|---------|
| <code>init_device()</code>              | 初始化设备状态 (GPU/CPU/XPU) | 106-110 |
| <code>load_model()</code>               | 加载模型到目标设备             | 130-132 |
| <code>execute_model()</code>            | 执行模型前向传播              | 134-143 |
| <code>sample_tokens()</code>            | Token 采样              | 145-149 |
| <code>get_kv_cache_spec()</code>        | 获取 KV Cache 规范        | 90-92   |
| <code>compile_or_warm_up_model()</code> | 编译/预热模型               | 94-100  |

#### 5.4 `execute_model()` 接口详解

```
def execute_model(
    self, scheduler_output: SchedulerOutput
) -> ModelRunnerOutput | AsyncModelRunnerOutput | None:
    """If this method returns None, sample_tokens should be called immediately after
    to obtain the ModelRunnerOutput.

    Note that this design may be changed in future if/when structured outputs
    parallelism is re-architected.
    """
```

- 第 134-143 行: 两阶段执行模式:
  - 返回 `ModelRunnerOutput` 或 `AsyncModelRunnerOutput`: 直接得到结果
  - 返回 `None`: 需要立即调用 `sample_tokens()` 获取结果 (用于分离采样逻辑的场景)

#### 5.5 `WorkerWrapperBase` 代理类

```
class WorkerWrapperBase:
    """
    This class represents one process in an executor/engine. It is responsible
    for lazily initializing the worker and handling the worker's lifecycle.
    """

    def __init__(self, rpc_rank: int = 0, global_rank: int | None = None) -> None:
        self.rpc_rank: int = rpc_rank
        self.global_rank: int = self.rpc_rank if global_rank is None else global_rank
        self.worker: WorkerBase
        self.vllm_config: VllmConfig
```

- 第 179-209 行: `WorkerWrapperBase` 是 `Worker` 的包装器, 负责延迟初始化和生命周期管理。

#### 动态 `Worker` 类加载

```
def init_worker(self, all_kwargs: list[dict[str, Any]]) -> None:
    kwargs = all_kwargs[self.rpc_rank]
    vllm_config: VllmConfig | None = kwargs.get("vllm_config")

    parallel_config = vllm_config.parallel_config
    if isinstance(parallel_config.worker_cls, str):
        worker_class: type[WorkerBase] = resolve_obj_by_qualname(
            parallel_config.worker_cls
        )

    if parallel_config.worker_extension_cls:
        worker_extension_cls = resolve_obj_by_qualname(
```

```

        parallel_config.worker_extension_cls
    )
    # 动态注入扩展类
    worker_class.__bases__ = worker_class.__bases__ + (worker_extension_cls,)

    with set_current_vllm_config(self.vllm_config):
        self.worker = worker_class(**kwargs)

```

- 第 222-305 行：关键特性：
  - 通过字符串限定名动态加载 Worker 类
  - 支持通过 `worker_extension_cls` 动态注入扩展功能（Mixin 模式）
  - 使用 `set_current_vllm_config` 确保配置在 Worker 初始化期间可用

属性代理

```

def __getattr__(self, attr: str):
    return getattr(self.worker, attr)

```

- 第 319-320 行：将未定义的属性访问代理到底层的 `worker` 对象，使 `WorkerWrapperBase` 透明地表现为 `Worker`。

多模态缓存处理

```

def _apply_mm_cache(self, scheduler_output: SchedulerOutput) -> None:
    mm_cache = self.mm_receiver_cache
    if mm_cache is None:
        return

    for req_data in scheduler_output.scheduled_new_reqs:
        req_data.mm_features = mm_cache.get_and_update_features(
            req_data.mm_features
        )

```

```

def execute_model(
    self, scheduler_output: SchedulerOutput
) -> ModelRunnerOutput | AsyncModelRunnerOutput | None:
    self._apply_mm_cache(scheduler_output)
    return self.worker.execute_model(scheduler_output)

```

- 第 322-337 行：在执行模型前自动应用多模态缓存。

## 六、GPUWorker GPU 工作进程

### 6.1 概述

`GPUWorker` 是 vLLM 最核心的 Worker 实现，负责 GPU 上的模型加载、执行和资源管理。它是生产环境中最常用的 Worker 类型。

源码位置：gpu\_worker.py

### 6.2 初始化流程

```

class Worker(WorkerBase):
    def __init__(
        self,
        vllm_config: VllmConfig,

```

```

    local_rank: int,
    rank: int,
    distributed_init_method: str,
    is_driver_worker: bool = False,
):
    super().__init__(...)

    precision = envs.VLLM_FLOAT32_MATMUL_PRECISION
    torch.set_float32_matmul_precision(precision)

    from vllm.distributed.elastic_ep.elastic_execute import ElasticEPScalingExecutor
    self.elastic_ep_executor = ElasticEPScalingExecutor(self)

    self._sleep_saved_buffers: dict[str, torch.Tensor] = {}
    self.weight_transfer_engine = WeightTransferEngineFactory.create_engine(...) if ... else None
    self.profiler: Any | None = None
    self.use_v2_model_runner = envs.VLLM_USE_V2_MODEL_RUNNER
    self._pp_send_work: list[Handle] = []

```

- 第 106-158 行：初始化精度设置、弹性 EP 执行器、权重传输引擎等组件。

### 6.3 设备初始化

```

@instrument(span_name="Init device")
def init_device(self):
    if self.device_config.device_type == "cuda":
        os.environ.pop("NCCL_ASYNC_ERROR_HANDLING", None)

        # DP 本地秩调整
        dp_local_rank = self.parallel_config.data_parallel_rank_local
        if dp_local_rank is None:
            dp_local_rank = self.parallel_config.data_parallel_index
        tp_pp_world_size = (
            self.parallel_config.pipeline_parallel_size
            * self.parallel_config.tensor_parallel_size
        )
        self.local_rank += dp_local_rank * tp_pp_world_size

        self.device = torch.device(f"cuda:{self.local_rank}")
        torch.accelerator.set_device_index(self.device)

        current_platform.check_if_supports_dtype(self.model_config.dtype)

        # 初始化分布式环境
        init_worker_distributed_environment(
            self.vllm_config, self.rank,
            self.distributed_init_method, self.local_rank,
            current_platform.dist_backend,
        )

        set_random_seed(self.model_config.seed)

        gc.collect()
        torch.accelerator.empty_cache()

```



```

self.init_snapshot = MemorySnapshot(device=self.device)
self.requested_memory = request_memory(init_snapshot, self.cache_config)

```

- 第 239-309 行: 关键步骤:
  1. 根据 DP 配置调整 local\_rank
  2. 设置 CUDA 设备
  3. 初始化 NCCL 分布式环境
  4. 获取内存快照用于后续 profiling

## ModelRunner 选择

```

if self.use_v2_model_runner:
    from vllm.v1.worker.gpu.model_runner import GPUModelRunner as GPUModelRunnerV2
    self.model_runner: GPUModelRunner = GPUModelRunnerV2(
        self.vllm_config, self.device
    )
else:
    from vllm.v1.worker.gpu_model_runner import GPUModelRunner as GPUModelRunnerV1
    self.model_runner = GPUModelRunnerV1(self.vllm_config, self.device)

```

- 第 316-330 行: 根据环境变量选择 V1 或 V2 ModelRunner。

## 6.4 模型加载

```

def load_model(self, *, load_dummy_weights: bool = False) -> None:
    with (
        self._maybe_get_memory_pool_context(tag="weights"),
        set_current_vllm_config(self.vllm_config),
        self._scoped_allocator_max_split(max_split_size_mb=20),
    ):
        self.model_runner.load_model(load_dummy_weights=load_dummy_weights)

```

- 第 338-345 行: 使用内存池上下文和优化的分配器设置来加载模型。

## 6.5 内存 Profiling

```

@torch.inference_mode()
def determine_available_memory(self) -> int:
    """Profiles the peak memory usage of the model to determine how much
    memory can be used for KV cache without OOMs.
    """
    with memory_profiling(
        self.init_snapshot,
        weights_memory=int(self.model_runner.model_memory_usage),
    ) as profile_result:
        self.model_runner.profile_run()
        profile_torch_peak = torch.accelerator.memory_stats(self.device).get(
            "allocated_bytes.all.peak", 0
        )

        cudagraph_memory_estimate = 0
        if current_platform.is_cuda() and ...:
            cudagraph_memory_estimate = self.model_runner.profile_cudagraph_memory()

    # 计算可用 KV cache 内存
    free_gpu_memory = profile_result.after_profile.free_memory

```

```

self.available_kv_cache_memory_bytes = (
    self.requested_memory
    - profile_result.non_kv_cache_memory
    - cudagraph_memory_estimate_applied
)
return int(self.available_kv_cache_memory_bytes)

```

- 第 354-506 行: 核心功能: 通过 dummy forward pass 测量峰值内存, 计算可用于 KV Cache 的内存量。

## 6.6 模型执行

```

@torch.inference_mode()
def execute_model(
    self, scheduler_output: "SchedulerOutput"
) -> ModelRunnerOutput | AsyncModelRunnerOutput | None:
    # 确保 PP 非阻塞发送已完成
    if self._pp_send_work:
        for handle in self._pp_send_work:
            handle.wait()
        self._pp_send_work = []

    intermediate_tensors = None
    forward_pass = scheduler_output.total_num_scheduled_tokens > 0

    # PP 中间张量接收 (非首 rank)
    if forward_pass and not get_pp_group().is_first_rank:
        tensor_dict, comm_handles, comm_postprocess = (
            get_pp_group().irecv_tensor_dict(...)
        )
        intermediate_tensors = AsyncIntermediateTensors(
            tensor_dict, comm_handles=comm_handles,
            comm_postprocess=comm_postprocess,
        )

    with self.annotate_profile(scheduler_output):
        output = self.model_runner.execute_model(
            scheduler_output, intermediate_tensors
        )

    # Pooling 模型特殊处理
    if self.use_v2_model_runner and self.model_runner.is_pooling_model and output is None:
        output = self.model_runner.pool()

    if isinstance(output, ModelRunnerOutput | AsyncModelRunnerOutput | NoneType):
        return output

    # PP 中间张量发送 (非末尾 rank)
    assert isinstance(output, IntermediateTensors)
    self._pp_send_work = get_pp_group().isend_tensor_dict(
        output.tensors, ...
    )
    return None

```

- 第 783-871 行: 执行流程:
  1. 等待上一轮 PP 发送完成

2. 如果非 PP 首 rank, 接收中间张量 (异步)
3. 调用 ModelRunner 执行前向传播
4. 如果非 PP 末尾 rank, 异步发送中间张量给下一阶段

## 6.7 AsyncIntermediateTensors 异步通信

```
class AsyncIntermediateTensors(IntermediateTensors):
    """IntermediateTensors with lazy comm synchronization"""

    def __init__(
        self,
        tensors: dict[str, torch.Tensor],
        comm_handles: list[Handle] | None = None,
        comm_postprocess: list[Callable[[], None]] | None = None,
    ) -> None:
        super().__init__(tensors)
        self._comm_handles = comm_handles
        self._comm_postprocess = comm_postprocess
        self._comm_waited = False

    def wait_for_comm(self) -> None:
        if self._comm_waited:
            return
        if self._comm_handles:
            for handle in self._comm_handles:
                handle.wait()
        if self._comm_postprocess:
            for fn in self._comm_postprocess:
                fn()
        self._comm_waited = True

    def __getattr__(self, name: str):
        if name == "tensors" and not object.__getattr__(self, "_comm_waited"):
            object.__getattr__(self, "wait_for_comm")()
        return object.__getattr__(self, name)
```

- 第 74-103 行: 惰性同步: 只有在实际访问 `.tensors` 时才等待通信完成, 实现计算与通信的重叠。

## 6.8 休眠/唤醒机制

```
def sleep(self, level: int = 1) -> None:
    from vllm.device_allocator.cumem import CuMemAllocator

    free_bytes_before_sleep = torch.cuda.mem_get_info()[0]

    if level == 2:
        model = self.model_runner.model
        self._sleep_saved_buffers = {
            name: buffer.cpu().clone() for name, buffer in model.named_buffers()
        }

    allocator = CuMemAllocator.get_instance()
    allocator.sleep(offload_tags=("weights",) if level == 1 else tuple())
    ...
```

```

def wake_up(self, tags: list[str] | None = None) -> None:
    from vllm.device_allocator.cumem import CuMemAllocator

    allocator = CuMemAllocator.get_instance()
    allocator.wake_up(tags)

    if len(self._sleep_saved_buffers):
        model = self.model_runner.model
        for name, buffer in model.named_buffers():
            if name in self._sleep_saved_buffers:
                buffer.data.copy_(self._sleep_saved_buffers[name].data)
        self._sleep_saved_buffers = {}

```

- 第 160-199 行: **CuMem** 集成:
  - Level 1: 仅卸载权重
  - Level 2: 同时保存 buffer 到 CPU 并卸载权重和 KV Cache
  - 唤醒时恢复 buffer 并重建 KV Cache 结构

## 6.9 编译与预热

```

@instrument(span_name="Warmup (GPU)")
def compile_or_warm_up_model(self) -> CompilationTimes:
    warmup_sizes: list[int] = []

    if self.vllm_config.compilation_config.mode == CompilationMode.VLLM_COMPILE:
        compile_sizes = self.vllm_config.compilation_config.compile_sizes
        warmup_sizes = compile_sizes.copy() if compile_sizes is not None else []
        # ... 计算 warmup sizes

    for size in sorted(warmup_sizes, reverse=True):
        logger.info("Compile and warming up model for size %d", size)
        self.model_runner._dummy_run(size, skip_eplb=True, remove_lora=False)

    kernel_warmup(self)

    cuda_graph_memory_bytes = 0
    if not self.model_config.enforce_eager:
        cuda_graph_memory_bytes = self.model_runner.capture_model()

    # V2 ModelRunner 特殊处理
    if self.use_v2_model_runner:
        warmup_kernels(self.model_runner, self.execute_model, self.sample_tokens)
    elif get_pp_group().is_last_rank:
        # V1: Warm up sampler
        hidden_states, last_hidden_states = self.model_runner._dummy_run(num_tokens=max_num_reqs, ...)
        if self.model_runner.is_pooling_model:
            self.model_runner._dummy_pooler_run(hidden_states)
        else:
            self.model_runner._dummy_sampler_run(hidden_states=last_hidden_states)

    return CompilationTimes(
        language_model=self.compilation_config.compilation_time,
        encoder=self.compilation_config.encoder_compilation_time,
    )

```

- 第 574-727 行: 预热流程:
  1. 按指定 batch size 执行 dummy run 进行编译
  2. 内核预热 (kernel\_warmup)
  3. CUDA Graph 捕获
  4. Sampler 预热 (分配最大形状缓冲区避免碎片化)

## 七、GPUModelRunner 模型运行器

### 7.1 概述

GPUModelRunner 是 GPU 上实际的模型执行引擎, 负责: - 模型前向传播编排 - Attention 计算 - Sampling (Token 采样) - KV Cache 管理 - CUDA Graph 捕获与执行

源码位置: gpu\_model\_runner.py

### 7.2 核心依赖与导入

```
from vllm.v1.attention.backend import (
    AttentionBackend, AttentionCGSupport,
    AttentionMetadata, AttentionMetadataBuilder, AttentionType,
    CommonAttentionMetadata,
)
from vllm.v1.sample.sampler import Sampler
from vllm.v1.sample.rejection_sampler import RejectionSampler
from vllm.v1.spec_decode.metadata import SpecDecodeMetadata
from vllm.v1.worker.gpu_input_batch import CachedRequestState, InputBatch
from vllm.v1.worker.gpu_ubatch_wrapper import UBatchWrapper
from vllm.v1.kv_cache_interface import (
    KVCacheConfig, KVCacheSpec, FullAttentionSpec, SlidingWindowSpec, ...
)
```

- 第 1-200 行: 导入注意力后端、采样器、推测解码、输入批处理等核心组件。

### 7.3 输入批处理 (InputBatch)

CachedRequestState 缓存请求状态

```
@dataclass
class CachedRequestState:
    req_id: str
    prompt_token_ids: list[int] | None
    mm_features: list[MultiModalFeatureSpec]
    sampling_params: SamplingParams | None
    generator: torch.Generator | None

    block_ids: tuple[list[int], ...]
    num_computed_tokens: int
    output_token_ids: list[int]

    mrope_positions: torch.Tensor | None = None
    lora_request: LoRARequest | None = None
    prompt_embeds: torch.Tensor | None = None
    pooling_params: PoolingParams | None = None

    @property
```

```
def num_tokens(self) -> int:
    return self.num_prompt_tokens + len(self.output_token_ids)
```

- gpu\_input\_batch.py 第 33-76 行：维护每个请求的完整状态，包括 token IDs、block 表、采样参数等。

### InputBatch 批处理容器

```
class InputBatch:
    def __init__(
        self,
        max_num_reqs: int,
        max_model_len: int,
        max_num_batched_tokens: int,
        device: torch.device,
        pin_memory: bool,
        vocab_size: int,
        block_sizes: list[int],
        kernel_block_sizes: list[int],
        # ... 更多参数
    ):
        # Token ID 缓冲区
        self.token_ids_cpu_tensor = torch.zeros(
            (max_num_reqs, max_model_len),
            device="cpu", dtype=torch.int32, pin_memory=False,
        )

        # Block table
        self.block_table = MultiGroupBlockTable(...)

        # 采样相关缓冲区
        self.temperature = torch.empty((max_num_reqs,), dtype=torch.float32, device=device)
        self.top_p = torch.empty((max_num_reqs,), dtype=torch.float32, device=device)
```

- gpu\_input\_batch.py 第 91-199 行：预分配固定大小的缓冲区，避免动态分配带来的性能开销。

## 7.4 Micro-batching 策略 (UBatching)

### UBatchContext 微批次上下文

```
class UBatchContext:
    """
    Context manager for micro-batching synchronization using threading events.
    """

    def __init__(
        self,
        id: int,
        comm_stream: torch.cuda.Stream,
        compute_stream: torch.cuda.Stream,
        forward_context: ForwardContext,
        ready_barrier: threading.Barrier,
        cpu_wait_event: threading.Event,
        cpu_signal_event: threading.Event,
        gpu_comm_done_event: torch.Event,
        gpu_compute_done_event: torch.Event,
        schedule: str = "default",
    ):
```

```

):
    self.id = id
    self.comm_stream = comm_stream
    self.compute_stream = compute_stream
    # ... 同步原语

```

- ubatching.py 第 20-49 行: 使用双流 (communication stream + compute stream) 和线程事件实现微批次间的同步。

流切换操作

```

def switch_to_comm_sync(self):
    """Synchronize and switch from compute to communication stream."""
    self._signal_compute_done()
    self.update_stream(self.comm_stream)
    self._wait_compute_done()

def switch_to_compute_sync(self):
    """Synchronize and switch from communication to compute stream."""
    self._signal_comm_done()
    self.update_stream(self.compute_stream)
    self._wait_comm_done()

def yield_(self):
    """Yield control to another microbatch thread."""
    self.current_stream = current_stream()
    self._cpu_yield()
    self.update_stream(self.current_stream)

```

- ubatching.py 第 113-131 行: 流重叠优化: 通过显式的流切换和同步, 实现通信与计算的流水线化。

## 7.5 模型前向传播编排

GPUModelRunner 的 `execute_model()` 方法是整个推理的核心入口, 其典型流程如下:

flowchart TD

```

A[receive_scheduler_output] --> B[build_input_batch]
B --> C{batch_type?}
C -->|Prefill| D[prefill_forward]
C -->|Decode| E[decode_forward]
C -->|Mixed| F[split_and_execute]
D --> G[attention_forward]
E --> G
F --> G
G --> H[sampling]
H --> I[update_kv_cache]
I --> J[return ModelRunnerOutput]

```

关键执行路径

1. 输入构建: 将 SchedulerOutput 转换为 InputBatch
2. **Attention Metadata** 构建: 生成 attention backend 所需的 metadata
3. 前向传播: 调用模型的 forward 方法
4. **Sampling**: 对最后一个位置的 logits 进行采样
5. **KV Cache** 更新: 将新的 KV states 写入 cache blocks

## 7.6 Sampling 执行

```
# Sampler 组件
from vllm.v1.sample.sampler import Sampler
from vllm.v1.sample.rejection_sampler import RejectionSampler
from vllm.v1.sample.logits_processor import LogitsProcessors, build_logitsprocs
```

- 第 167-171 行: 采样子系统包括:
  - **LogitsProcessor**: logits 后处理 (温度、top-p、重复惩罚等)
  - **Sampler**: 实际采样 (greedy、beam\_search、random 等)
  - **RejectionSampler**: 推测解码时的验证采样

## 7.7 KV Cache 写入

KV Cache 的写入由 Attention Backend 负责, 不同类型的 attention 有不同的实现:

```
from vllm.v1.kv_cache_interface import (
    FullAttentionSpec,      # 标准 full attention
    SlidingWindowSpec,     # 滑动窗口 attention
    ChunkedLocalAttentionSpec, # 分块局部 attention
    CrossAttentionSpec,     # 交叉 attention (encoder-decoder)
    EncoderOnlyAttentionSpec, # 仅 encoder
)
```

- 第 140-152 行: KV Cache 规范定义了不同的 attention 模式及其对应的 cache 结构。

---

## 八、CPUWorker / CPUModelRunner CPU 执行路径

### 8.1 CPUWorker 实现

源码位置: cpu\_worker.py

```
class CPUWorker(Worker):
    def __init__(
        self,
        vllm_config: VllmConfig,
        local_rank: int,
        rank: int,
        distributed_init_method: str,
        is_driver_worker: bool = False,
    ):
        # NUMA 绑定
        allowed_memory_nodes = get_visible_memory_node()
        allowed_cpu_list = get_allowed_cpu_list()
        cpu_core = allowed_cpu_list[0]

        torch.ops._C.init_cpu_memory_env([cpu_core.numa_node])

        memory_status = get_memory_node_info(cpu_core.numa_node)
        memory_fraction = vllm_config.cache_config.gpu_memory_utilization
        self.requested_cpu_memory = math.ceil(
            memory_status.total_memory * memory_fraction
        )

        super().__init__(vllm_config, local_rank, rank,
```



```
distributed_init_method, is_driver_worker)
self.parallel_config.disable_custom_all_reduce = True
```

- 第 29-86 行: CPU 特有配置:
  - NUMA 节点绑定
  - 基于 memory fraction 的 CPU 内存限制
  - 禁用自定义 all-reduce (CPU 使用 Gloo/oneCCL)

## 8.2 设备初始化差异

```
def init_device(self):
    # 检查预加载库
    def check_preloaded_libs(name: str):
        ld_preload_list = os.environ.get("LD_PRELOAD", "")
        if name not in ld_preload_list:
            logger.warning("%s is not found in LD_PRELOAD...")

    if sys.platform.startswith("linux"):
        check_preloaded_libs("libtcmalloc")    # 内存分配器
        if current_platform.get_cpu_architecture() == CpuArchEnum.X86:
            check_preloaded_libs("libiomp")     # OpenMP 库

    # 禁用 torch.set_num_threads (已绑核)
    torch.set_num_threads = skip_set_num_threads

    # 分布式初始化
    os.environ["VLLM_DIST_IDENT"] = self.distributed_init_method.split(":")[-1]
    init_worker_distributed_environment(...)

    # 使用 CPUModelRunner
    self.model_runner: CPUModelRunner = CPUModelRunner(
        self.vllm_config, torch.device("cpu")
    )
```

- 第 100-143 行: CPU 初始化的关键差异:
  - 检查 tcmalloc/iomp 等性能库
  - 使用 CPUModelRunner 替代 GPU 版本
  - 不进行 GPU 内存 profiling

## 8.3 限制

```
def sleep(self, level: int = 1) -> None:
    logger.warning("sleep mode is not supported on CPU, ignore it.")

def wake_up(self, tags: list[str] | None = None) -> None:
    logger.warning("sleep mode is not supported on CPU, ignore it.")
```

- 第 145-150 行: CPU 不支持休眠/唤醒机制。

# 九、XPUWorker 其他硬件支持

## 9.1 XPUWorker (Intel GPU)

源码位置: xpu\_worker.py

```

class XPUWorker(Worker):
    """A XPU worker class."""

    def __init__(self, vllm_config, local_rank, rank,
                 distributed_init_method, is_driver_worker=False):
        super().__init__(...)
        device_config = self.device_config
        assert device_config.device_type == "xpu"
        assert current_platform.is_xpu()

    def init_device(self):
        device = self.device_config.device
        if isinstance(device, torch.device) and device.type == "xpu":
            self.device = torch.device(f"xpu:{self.local_rank}")
            torch.accelerator.set_device_index(self.device)
            current_platform.check_if_supports_dtype(self.model_config.dtype)
            torch.accelerator.empty_cache()
            self.init_gpu_memory = torch.xpu.get_device_properties(
                self.local_rank
            ).total_memory

        # oneCCL 配置
        ENV_CCL_ATL_TRANSPORT = os.getenv("CCL_ATL_TRANSPORT", "ofi")
        os.environ["CCL_ATL_TRANSPORT"] = ENV_CCL_ATL_TRANSPORT
        os.environ["LOCAL_WORLD_SIZE"] = str(self.parallel_config.world_size)
        os.environ["LOCAL_RANK"] = str(self.local_rank)

        init_worker_distributed_environment(...)

        # oneCCL 全局 all_reduce 预热
        if torch.distributed.is_xccl_available():
            torch.distributed.all_reduce(torch.zeros(1).xpu())

        # 使用 XPUModelRunner
        self.model_runner: XPUModelRunner | XPUModelRunnerV2 = ...

```

- 第 25-100+ 行: **XPU** 特性:
  - 使用 Intel XPU 设备
  - 通过 oneCCL 进行分布式通信
  - 使用专用的 XPUModelRunner / XPUModelRunnerV2

## 9.2 硬件支持矩阵

| 硬件平台            | Worker 类  | ModelRunner               | 通信后端        | 休眠支持 |
|-----------------|-----------|---------------------------|-------------|------|
| NVIDIA GPU      | GPUWorker | GPUModelRunner<br>(V1/V2) | NCCL        |      |
| CPU             | CPUWorker | CPUModelRunner            | Gloo/oneCCL |      |
| Intel GPU (XPU) | XPUWorker | XPUModelRunner<br>(V1/V2) | oneCCL      |      |
| Neuron (TPU)    | 外部支持      | -                         | -           | -    |

## 十、输入批处理与优化

### 10.1 InputBatch 架构

源码位置: `gpu_input_batch.py`

InputBatch 是 GPU Worker 的核心数据结构, 负责:

1. 请求状态管理: 维护所有活跃请求的状态
2. 张量缓冲区管理: 预分配 GPU/CPU 张量
3. **Block Table** 维护: 管理 KV Cache 的物理块映射
4. 采样参数聚合: 合并各请求的采样配置

缓冲区布局

graph LR

```
subgraph "InputBatch 内存布局"
    A[token_ids_cpu<br/>max_num_reqs × max_model_len<br/>int32, CPU]
    B[num_tokens_no_spec<br/>max_num_reqs, int32]
    C[num_prompt_tokens<br/>max_num_reqs, int32]
    D[num_computed_tokens<br/>max_num_reqs, int32]
    E[block_table<br/>MultiGroupBlockTable]
    F[temperature<br/>max_num_reqs, float32, GPU]
    G[top_p<br/>max_num_reqs, float32, GPU]
    H[top_k<br/>max_num_reqs, int, GPU]
    I[min_p<br/>max_num_reqs, float32, GPU]
    J[presence_penalty<br/>max_num_reqs, float32, GPU]
    K[frequency_penalty<br/>max_num_reqs, float32, GPU]
    L[repetition_penalty<br/>max_num_reqs, float32, GPU]
end
```

### 10.2 Padding 优化策略

InputBatch 采用固定大小缓冲区 + 动态有效区域的设计:

```
# 固定分配 (避免碎片化)
self.token_ids_cpu_tensor = torch.zeros(
    (max_num_reqs, max_model_len),
    device="cpu", dtype=torch.int32, pin_memory=False,
)
```

```
# 运行时只使用有效部分
# 有效请求数: 0 ~ actual_num_reqs
# 每个 request 的有效 token 数: 0 ~ req.num_tokens
```

- `gpu_input_batch.py` 第 133-138 行: 优化要点:
  - 预分配最大可能大小的缓冲区
  - 使用 `numpy` 视图进行高效的 CPU 端操作
  - 可选的 `pin_memory` 加速 CPU→GPU 传输

### 10.3 MultiGroupBlockTable

```
self.block_table = MultiGroupBlockTable(
    max_num_reqs=max_num_reqs,
    max_model_len=max_model_len,
    max_num_batched_tokens=max_num_batched_tokens,
    pin_memory=pin_memory,
    device=device,
```

```

    block_sizes=block_sizes,
    kernel_block_sizes=kernel_block_sizes,
    max_num_blocks=max_num_blocks_per_req,
    cp_kv_cache_interleave_size=cp_kv_cache_interleave_size,
)

```

- `gpu_input_batch.py` 第 171-181 行: 支持多种 KV Cache 组 (例如: full attention + sliding window + cross attention), 每种组可能有不同的 block size。

## 10.4 Micro-batching 流水线

UBatching (Micro-batching) 是 vLLM 的关键性能优化技术, 将一个大的 batch 拆分为多个 micro-batch 以实现:

1. 通信-计算重叠: 在一个 micro-batch 计算的同时, 另一个进行通信
2. 更好的缓存利用率: 较小的 batch 更适合 L1/L2 缓存
3. 内存节省: 减少激活值的峰值内存

sequenceDiagram

```

    participant MB1 as MicroBatch-1
    participant MB2 as MicroBatch-2
    participant Comm as Communication Stream
    participant Compute as Compute Stream

```

Note over MB1,MB2: Default: 2 microbatches

```

MB1->>Comm: AllReduce (layer N)
MB1->>Compute: Forward (layer N+1)
MB1->>MB1: switch_to_comm_sync()

```

```

MB2->>Compute: Forward (layer N)
MB2->>Comm: AllReduce (layer N)
MB2->>MB2: switch_to_compute_sync()

```

Note over Comm,Compute: Streams overlap execution

## 十一、Executor 选择与请求处理流程

### 11.1 Executor 选择决策树

flowchart TD

```

Start[用户启动 vLLM] --> CheckBackend{distributed_executor_backend?}

```

```

CheckBackend -->|"uni"| UniProc[UniProcExecutor<br/> 单进程, 单 GPU]

```

```

CheckBackend -->|"mp"| MultiProc[MultiprocExecutor<br/> 多进程, 多 GPU]

```

```

CheckBackend -->|"ray"| Ray[RayExecutorV2<br/>Ray 分布式]

```

```

CheckBackend -->|"custom"| Custom[自定义 Executor]

```

```

UniProc --> InitWorker[WorkerWrapperBase<br/>+ GPUWorker]

```

```

MultiProc --> ForkWorkers[Fork N 个 Worker 进程]

```

```

Ray --> SpawnActors[Spawn N 个 Ray Actors]

```

```

ForkWorkers --> InitWorker

```

```

SpawnActors --> InitWorker

```

```

InitWorker --> LoadModel[load_model<br/> 加载权重]

```

```

LoadModel --> Profile[determine_available_memory<br/> 内存 profiling]
Profile --> InitKVCache[initialize_from_config<br/> 分配 KV Cache]
InitKVCache --> Warmup[compile_or_warm_up_model<br/> 编译/预热]

Warmup --> Ready[就绪, 等待请求]

Ready --> RecvReq[接收 SchedulerOutput]
RecvReq --> ExecModel[execute_model<br/> 前向传播]
ExecModel --> Sample[sample_tokens<br/>Token 采样]
Sample --> ReturnOutput[返回 ModelRunnerOutput]
ReturnOutput --> RecvReq

```

## 11.2 典型请求处理流程

以下是单次 `execute_model` 调用的完整流程:

```

sequenceDiagram
    participant Engine as LLMEngine
    participant Executor as Executor
    participant Worker as Worker
    participant Runner as ModelRunner
    participant Attn as AttentionBackend
    participant Sampler as Sampler

    Engine->>Executor: execute_model(scheduler_output)
    Executor->>Executor: collective_rpc("execute_model", ...)

    alt UniProcExecutor
        Executor->>Worker: run_method(worker, "execute_model", ...)
    else MultiProcExecutor
        Executor->>Worker: rpc_broadcast_mq.enqueue(method, args)
        Worker->>Worker: worker_busy_loop() dequeues
    end

    Worker->>Runner: execute_model(scheduler_output, intermediate_tensors)

    Runner->>Runner: build_input_batch(scheduler_output)
    Runner->>Attn: build_metadata(batch)
    Runner->>Runner: model.forward(input_ids, positions, kv_cache, attn_metadata)

    loop For each transformer layer
        Runner->>Attn: forward(q, k, v, metadata)
        Attn->>Attn: compute_attention / write_kv_cache
    end

    Runner->>Sampler: sample(hidden_states, sampling_metadata)
    Sampler->>Sampler: apply_logits_processors
    Sampler->>Sampler: sample_token (greedy/random/beam)
    Sampler-->>Runner: SamplerOutput

    Runner-->>Worker: ModelRunnerOutput

    alt Non-blocking Mode
        Worker->>Worker: handle_output(async_output)
        Worker-->>Executor: Future[ModelRunnerOutput]
    end

```

```
else Blocking Mode
    Worker-->>Executor: ModelRunnerOutput
end

Executor-->>Engine: ModelRunnerOutput
```

11.3 性能关键路径

| 阶段           | 耗时占比   | 优化手段                                     |
|--------------|--------|------------------------------------------|
| 输入构建         | 5-10%  | 预分配缓冲区、numpy 操作                          |
| Attention 计算 | 30-50% | FlashAttention、PagedAttention、CUDA Graph |
| FFN 计算       | 20-30% | MoE 路由优化、kernel fusion                   |
| Sampling     | 5-15%  | GPU sampling、speculative decoding        |
| KV Cache 写入  | 10-20% | 非阻塞写、cache optimization                  |

总结

本文档全面分析了 vLLM v1 的执行框架架构，涵盖了从 Executor 抽象到具体硬件实现的完整链条：

架构亮点

- 1. 三层抽象：Executor（调度）→ Worker（设备）→ ModelRunner（推理），职责清晰
- 2. 多后端支持：UniProc / MultiProc / Ray 三种执行模式平滑切换
- 3. 高效 IPC：基于 ZMQ MessageQueue 的零拷贝进程间通信
- 4. 异步执行：支持 async\_scheduling 实现更高的吞吐量
- 5. 硬件无关：统一的 Worker 接口支持 GPU/CPU/XPU 等多种硬件
- 6. 性能优化：CUDA Graph、Micro-batching、通信-计算重叠等

关键代码路径

- **Executor** 选择：abstract.py:47-92
- **Worker** 创建：worker\_base.py:222-305
- 模型执行：gpu\_worker.py:783-871
- **RPC** 通信：multiproc\_executor.py:339-403
- **Ray** 初始化：ray\_executor\_v2.py:249-427

文档版本：v1.0 基于 vLLM 最新源码分析

vLLM KV Cache 系统完整分析

定位：本文档系统性地分析 vLLM V1 架构下的 KV Cache 全栈实现，涵盖缓存管理层次、多 GPU 协调、接口定义、CPU 卸载、跨节点传输、Block Table 管理及前缀缓存机制。

```
graph TB
    subgraph " 缓存管理器层次"
        KM["KVCacheManager<br/> 顶层抽象管理器"]
        COORD["KVCacheCoordinator<br/> 多 Group 协调器"]
        ST["SingleTypeKVCacheManager<br/> 单类型基类"]
        FA["FullAttentionManager"]
        SW["SlidingWindowManager"]
        CLA["ChunkedLocalAttentionManager"]
```

```

    MB["MambaManager"]
    CA["CrossAttentionManager"]
    SFA["SinkFullAttentionManager"]
end

subgraph " 协调与接口"
    UNITARY["UnitaryKVCacheCoordinator<br/>(单 Group)"]
    HYBRID["HybridKVCacheCoordinator<br/>(混合模型)"]
    NOCACHE["NoPrefixCacheCoordinator<br/>(无前缀缓存)"]
    KVI["KVCacheInterface<br/> 统一 API 定义"]
    KVC["KVCacheConfig / KVCacheSpec<br/> 数据结构"]
end

subgraph " 扩展能力"
    OFFLOAD["KV Offload CPU 卸载 <br/>OffloadingManager"]
    SIMPLE["Simple KV Offload<br/> 轻量级 CPU 卸载"]
    TRANSFER["KV Transfer 跨节点传输 <br/>KVConnectorBase_V1"]
    BT["Block Table 管理 <br/>MultiGroupBlockTable"]
    METRICS["KV Cache Metrics<br/> 统计与监控"]
    ENC["EncoderCacheManager<br/> 编码器输出缓存"]
end

KM --> COORD
COORD --> UNITARY
COORD --> HYBRID
COORD --> NOCACHE
COORD --> ST
ST --> FA
ST --> SW
ST --> CLA
ST --> MB
ST --> CA
SFA -. -> FA

KVI --> KVC
OFFLOAD --> COORD
SIMPLE --> KVI
TRANSFER --> KVI
BT --> KM
METRICS --> KM
ENC --> KM

style KM fill:#e0f2f1,stroke:#00695c,color:#000
style COORD fill:#e0f2f1,stroke:#00695c,color:#000
style KVI fill:#e0f2f1,stroke:#00695c,color:#000
style OFFLOAD fill:#fff3e0,stroke:#e65100,color:#000
style TRANSFER fill:#e8eaf6,stroke:#283593,color:#000
style BT fill:#fce4ec,stroke:#c62828,color:#000

```

## 目录

- 一、KV Cache Manager 层次
  - 1.1 KVCacheManager — 顶层抽象

- 1.2 SingleTypeKVCacheManager —单类型基类
- 1.3 具体管理器子类
- 1.4 EncoderCacheManager —编码器缓存
- 1.5 层次关系总览
- 二、KV Cache Coordinator 多 GPU 缓存协调
- 三、KV Cache 接口定义
- 四、KV Cache Metrics 缓存使用统计
- 五、KV Offload CPU 卸载
- 六、Simple KV Offload
- 七、KV Transfer 跨节点传输
- 八、Block Table 管理
- 九、前缀缓存 Prefix Caching
- 十、数据流全景图

一、KV Cache Manager 层次

1.1 KVCacheManager —顶层抽象

KVCacheManager 是调度器 (Scheduler) 与底层缓存系统的唯一交互入口。它隐藏了内部数据结构细节，通过 KVCacheBlocks 数据类与 Scheduler 通信。

核心职责：

| 方法                    | 功能            | 关键逻辑                                        |
|-----------------------|---------------|---------------------------------------------|
| get_computed_blocks() | 查询前缀缓存命中块     | 委托给<br>coordinator.find_longest_cache_hit() |
| allocate_slots()      | 为请求分配新 slot   | 三阶段：释放跳过块 → 分配已计算块 → 分配新块                   |
| free()                | 释放请求的所有块      | 反序释放，尾部优先                                   |
| cache_blocks()        | 将计算完成的块标记为可复用 | 仅缓存 num_tokens 以内的”最终确定” token              |

allocate\_slots() 的 Blocks 布局（源码注释原文）：

```

-----
| < comp > | < new_comp > | < ext_comp > | < new > | < lookahead > |
-----
                                     | < to be computed > |
-----
                                     | < to be allocated > |
-----
                                     | < to be cached (roughly, |
                                     | details below)> |
-----

```

初始化流程 (kv\_cache\_manager.py:107-160):

```

class KVCacheManager:
    def __init__(self, kv_cache_config, max_model_len, hash_block_size, ...):
        self.coordinator = get_kv_cache_coordinator(
            kv_cache_config=kv_cache_config,
            max_model_len=self.max_model_len,
            ...
        )
        self.block_pool = self.coordinator.block_pool
        self.num_kv_cache_groups = len(kv_cache_config.kv_cache_groups)

```



```

# 预构建空 KVCacheBlocks, 避免 GC 开销
self.empty_kv_cache_blocks = KVCacheBlocks(
    tuple(()) for _ in range(self.num_kv_cache_groups))
)

```

KVCacheBlocks 数据结构 (kv\_cache\_manager.py:22-103):

```

@dataclass
class KVCacheBlocks:
    """ 分配结果, 作为 Scheduler 和 KVCacheManager 的接口。"""
    blocks: tuple[Sequence[KVCacheBlock], ...]
    # blocks[i][j] 表示第 i 个 kv_cache_group 的第 j 个 block

    def get_block_ids(self, allow_none=False):
        """ 转换为 block_id 列表 """

```

## 1.2 SingleTypeKVCacheManager — 单类型基类

SingleTypeKVCacheManager 是一个抽象基类, 处理单一注意力类型的 KV cache 管理逻辑。

核心状态:

```

class SingleTypeKVCacheManager(ABC):
    def __init__(self, kv_cache_spec, block_pool, enable_caching, ...):
        self.block_size = kv_cache_spec.block_size
        self.kv_cache_spec = kv_cache_spec
        self.block_pool = block_pool
        # request_id -> 已分配的 blocks 列表
        self.req_to_blocks: defaultdict[str, list[KVCacheBlock]] = defaultdict(list)
        # request_id -> 已缓存的 block 数量
        self.num_cached_block: dict[str, int] = {}

```

关键方法签名:

| 方法                             | 说明                                  |
|--------------------------------|-------------------------------------|
| get_num_blocks_to_allocate()   | 计算需要分配的 block 数量 (考虑 admission cap) |
| allocate_new_computed_blocks() | 将前缀缓存命中的块追加到请求                      |
| allocate_new_blocks()          | 为待计算 token 分配新 block                |
| cache_blocks()                 | 将满 block 标记为可复用                     |
| free()                         | 释放请求的 blocks (反序)                   |
| find_longest_cache_hit() (抽象)  | 查找最长公共前缀缓存命中                        |
| remove_skipped_blocks()        | 移除注意力窗口外的块                          |
| get_num_skipped_tokens() (虚方法) | 计算需跳过的 token 数                      |

**admission cap** 机制 (single\_type\_kv\_cache\_manager.py:88-167):

对于 SlidingWindow 和 ChunkedLocalAttention 这类回收型 **spec**, 运行时会对每个请求的 block 预留数量施加上限 (`_max_admission_blocks_per_request`), 防止 chunked prefill 期间过度分配导致死锁:

```

def get_num_blocks_to_allocate(self, ..., apply_admission_cap=False):
    num_required_blocks = cdiv(num_tokens, self.block_size)
    if apply_admission_cap and self._max_admission_blocks_per_request is not None:
        num_required_blocks = min(
            num_required_blocks, self._max_admission_blocks_per_request
        )

```

### 1.3 具体管理器子类

**FullAttentionManager** FullAttentionManager 用于全注意力层（含 ChunkedLocalAttention）。

**find\_longest\_cache\_hit** 实现一从左到右顺序扫描 block hash 链:

```
@classmethod
def find_longest_cache_hit(cls, block_hashes, max_length, ...):
    computed_blocks = tuple([] for _ in range(len(kv_cache_group_ids)))
    for block_hash in itertools.islice(block_hashes, max_num_blocks):
        if cached_block := block_pool.get_cached_block(block_hash, kv_cache_group_ids):
            for computed, cached in zip(computed_blocks, cached_block):
                computed.append(cached)
        else:
            break # hash 不匹配则终止
    if use_eagle and computed_blocks[0]:
        for computed in computed_blocks:
            computed.pop() # EAGLE 需要丢弃最后一块以重算
```

**get\_num\_common\_prefix\_blocks** 一通过比较 ref\_cnt 与请求数判断公共前缀:

```
def get_num_common_prefix_blocks(self, running_request_id):
    blocks = self.req_to_blocks[running_request_id]
    num_common_blocks = 0
    for block in blocks:
        if block.ref_cnt == len(self.req_to_blocks):
            num_common_blocks += 1
        else:
            break
    return num_common_blocks
```

**SlidingWindowManager** SlidingWindowManager 处理滑动窗口注意力。

核心特点一从右向左反向搜索，找到满足滑动窗口连续性要求的最大匹配:

```
@classmethod
def find_longest_cache_hit(cls, block_hashes, max_length, ...):
    sliding_window_contiguous_blocks = cdiv(kv_cache_spec.sliding_window - 1, ...)
    # 从右向左搜索，找到连续匹配后提前退出
    for i in range(max_num_blocks - 1, -1, -1):
        if cached_block := block_pool.get_cached_block(block_hashes[i], ...):
            computed[i] = cached_block
            num_contiguous_blocks += 1
            if num_contiguous_blocks >= sliding_window_contiguous_blocks:
                match_found = True
                break
        else:
            num_contiguous_blocks = 0 # 重置连续计数
```

跳过 token 计算 (single\_type\_kv\_cache\_manager.py:606-632):

```
sliding_window=4, num_computed_tokens=7
Tokens: [ 0  1  2  3  4  5  6  7 ]
        | ---- computed ----|
                        ^ next token
                        |-----| sliding window
        |--skipped---|
→ get_num_skipped_tokens(7) == 4
```

```
def get_num_skipped_tokens(self, num_computed_tokens):
    return max(0, num_computed_tokens - self.sliding_window + 1)
```

**ChunkedLocalAttentionManager** ChunkedLocalAttentionManager 处理分块局部注意力。

策略：窗口外的 block 直接标记为 null（视为已计算），窗口内按前缀缓存查找：

```
@classmethod
def find_longest_cache_hit(cls, block_hashes, max_length, ...):
    local_attention_start_idx = (
        max_length // kv_cache_spec.attention_chunk_size * kv_cache_spec.attention_chunk_size
    )
    # 窗口外标记为 null
    computed_blocks = tuple(
        [block_pool.null_block] * local_attention_start_block_idx
        for _ in range(len(kv_cache_group_ids))
    )
    # 窗口内按 hash 查找
    for i in range(local_attention_start_block_idx, max_num_blocks):
        if cached_block := block_pool.get_cached_block(block_hashes[i], ...):
            computed.append(cached_block)
        else:
            break
```

**MambaManager** MambaManager 处理 Mamba/SSM 线性注意力层。

特殊之处：- **Align** 模式下只保留最近 2 个 block 的状态（last\_state\_block\_idx 追踪）- 不支持同步步骤内复用其他请求产生的 block（cached\_blocks\_this\_step 过滤）- 投机解码时分配额外的 speculative blocks

**CrossAttentionManager** CrossAttentionManager 处理 encoder-decoder 模型的交叉注意力。

不参与前缀缓存：编码器状态是每个请求唯一的，不存在可共享的前缀。

**SinkFullAttentionManager** SinkFullAttentionManager 继承自 FullAttentionManager，额外预分配 sink token 的常驻 block。

**Spec → Manager** 映射表 (single\_type\_kv\_cache\_manager.py:1142-1152):

```
spec_manager_map: dict[type[KVCacheSpec], type[SingleTypeKVCacheManager]] = {
    FullAttentionSpec: FullAttentionManager,
    TQFullAttentionSpec: FullAttentionManager,
    MLAAttentionSpec: FullAttentionManager,
    SlidingWindowSpec: SlidingWindowManager,
    SlidingWindowMLASpec: SlidingWindowManager,
    ChunkedLocalAttentionSpec: ChunkedLocalAttentionManager,
    MambaSpec: MambaManager,
    CrossAttentionSpec: CrossAttentionManager,
    SinkFullAttentionSpec: SinkFullAttentionManager,
}
```

## 1.4 EncoderCacheManager — 编码器缓存

EncoderCacheManager 管理多模态模型中编码器输出的缓存。

设计目标：避免对相同的多模态输入（如图像）重复执行昂贵的编码器计算。

核心数据结构：

| 字段                 | 类型                    | 用途                                  |
|--------------------|-----------------------|-------------------------------------|
| cached             | dict[str, set[str]]   | mm_hash → 引用该 entry 的 request ID 集合 |
| freeable           | OrderedDict[str, int] | 无引用的 mm_hash → 编码器 embedding 数量     |
| freed              | list[str]             | 自上次调用后被实际驱逐的 mm_hash 列表             |
| num_free_slots     | int                   | 当前可用容量                              |
| num_freeable_slots | int                   | 可回收容量（含无引用条目）                       |

关键操作流程:

1. **check\_and\_update\_cache()** — 检查是否已缓存, 更新引用计数
2. **can\_allocate()** — 容量检查, 不足时从 **freeable** 中 LRU 驱逐
3. **allocate()** — 预留空间, 更新 bookkeeping
4. **free\_encoder\_input()** — 释放引用, 无引用时移入 **freeable**

还有 **EncoderDecoderCacheManager** (encoder\_cache\_manager.py:323-381) — encoder-decoder 模型的临时简化版本, 仅用于调度目的。

## 1.5 层次关系总览

classDiagram

```
class KVCacheManager {
    +coordinator: KVCacheCoordinator
    +block_pool: BlockPool
    +get_computed_blocks()
    +allocate_slots()
    +free()
    +cache_blocks()
}
```

```
class KVCacheCoordinator {
    +abstract
    +find_longest_cache_hit()
    +single_type_managers: tuple
}
```

```
class UnitaryKVCacheCoordinator
class HybridKVCacheCoordinator
class NoPrefixCacheCoordinator
```

```
class SingleTypeKVCacheManager {
    +abstract
    +req_to_blocks: dict
    +find_longest_cache_hit()*
    +get_num_skipped_tokens()
}
```

```
class FullAttentionManager
class SlidingWindowManager
class ChunkedLocalAttentionManager
class MambaManager
class CrossAttentionManager
```

```

class EncoderCacheManager {
    +cached: dict
    +freeable: OrderedDict
    +check_and_update_cache()
    +can_allocate()
}

KVCacheManager --> KVCacheCoordinator
KVCacheCoordinator <|-- UnitaryKVCacheCoordinator
KVCacheCoordinator <|-- HybridKVCacheCoordinator
KVCacheCoordinator <|-- NoPrefixCacheCoordinator
KVCacheCoordinator --> SingleTypeKVCacheManager
SingleTypeKVCacheManager <|-- FullAttentionManager
SingleTypeKVCacheManager <|-- SlidingWindowManager
SingleTypeKVCacheManager <|-- ChunkedLocalAttentionManager
SingleTypeKVCacheManager <|-- MambaManager
SingleTypeKVCacheManager <|-- CrossAttentionManager

```

## 二、KV Cache Coordinator 多 GPU 缓存协调

KVCacheCoordinator 是连接 KVCacheManager 与各 SingleTypeKVCacheManager 的中间协调层，负责在多个 KV cache group 之间统一调度。

### 2.1 三种 Coordinator 变体

| 类名            | 适用场景              | 特点                                                              |
|---------------|-------------------|-----------------------------------------------------------------|
| NoPrefixCache | 前缀缓存禁用或不支持        | <code>find_longest_cache_hit</code> 始终返回空；支持任意数量 group          |
| Unitary       | 单一 KV cache group | 直接委托给唯一的 manager；<br><code>hash_block_size == block_size</code> |
| Hybrid        | 多种注意力类型的混合模型      | 迭代固定点算法协调多组 cache hit 长度                                        |

### 2.2 HybridKVCacheCoordinator 一固点算法

混合模型（如 Attention + Mamba、Attention + SlidingWindow）需要协调不同 attention type 对 cache hit 长度的约束。

算法核心 (kv\_cache\_coordinator.py:487-591):

```

def find_longest_cache_hit(self, block_hashes, max_cache_hit_length):
    hit_length = max_cache_hit_length
    while True:
        curr_hit_length = hit_length
        for idx, (spec, group_ids, manager_cls) in enumerate(self.attention_groups):
            if isinstance(spec, FullAttentionSpec) and cached_blocks is not None:
                # FullAttn 是向下封闭的: 只需查找一次, 后续仅 trim
                curr_hit_length = curr_hit_length // spec.block_size * spec.block_size
                continue

        hit_blocks = manager_cls.find_longest_cache_hit(

```

```

        ..., max_length=_max_length, alignment_tokens=self.lcm_block_size
    )
    _new_hit_length = len(hit_blocks[0]) * spec.block_size
    curr_hit_length = _new_hit_length

    if curr_hit_length >= hit_length:
        break # 收敛: 长度不再缩减
    hit_length = curr_hit_length
    if is_simple_hybrid:
        break # 简单混合 (1 FullAttn + 1 其他) 一次迭代即够

```

关键设计决策:

1. **Full Attention** 优先: 将 FullAttentionSpec 组排在 attention\_groups 首位, 因其高效的左到右扫描可提供更紧的初始上界
2. **LCM** 对齐: `lcm_block_size = lcm(*all_block_sizes)` 确保 hit length 是所有 group block size 的公倍数
3. **EAGLE** 处理: 每个 eagle group 最多执行一次 last-block-drop, 通过 `eagle_verified` 集合去重

**Group** 分组与验证 (`kv_cache_coordinator.py:436-470`):

```

def verify_and_split_kv_cache_groups(self):
    attention_groups = []
    for i, g in enumerate(self.kv_cache_config.kv_cache_groups):
        manager_cls = self.single_type_managers[i].__class__
        spec = g.kv_cache_spec
        # 按 (spec_type, manager_cls) 合并相同类型的 group
        for existing_spec, group_ids, existing_cls in attention_groups:
            if existing_spec == spec:
                group_ids.append(i)
                break
        else:
            attention_groups.append((spec, [i], manager_cls))
    # FullAttn 排第一
    self.attention_groups = sorted(attention_groups, key=lambda x: not isinstance(x[0], FullAttentionSpec))

```

## 2.3 Coordinator 工厂函数

`get_kv_cache_coordinator()` 根据配置自动选择 Coordinator 类型:

```

def get_kv_cache_coordinator(...):
    if not enable_caching:
        return KVCacheCoordinatorNoPrefixCache(...)
    if len(kv_cache_config.kv_cache_groups) == 1:
        return UnitaryKVCacheCoordinator(...)
    return HybridKVCacheCoordinator(...)

```

## 三、KV Cache 接口定义

### 3.1 KVCacheConfig 一全局配置

KVCacheConfig 描述模型的整体 KV cache 配置:

```

@dataclass
class KVCacheConfig:
    num_blocks: int # KV cache block 总数

```

```

kv_cache_tensors: list[KVCacheTensor] # 每个 tensor 的初始化信息
kv_cache_groups: list[KVCacheGroupSpec] # KV cache 分组

```

```

@property
def has_mamba_layers(self) -> bool: ...
@property
def needs_kv_cache_zeroing(self) -> bool: ...

```

### 3.2 KVCacheSpec 层次体系

KVCacheSpec 是所有 cache spec 的基类:

```

classDiagram
    class KVCacheSpec {
        +block_size: int
        +page_size_bytes*
        +max_memory_usage_bytes()*
    }

    class AttentionSpec {
        +num_kv_heads: int
        +head_size: int
        +dtype: torch.dtype
        +kv_quant_mode: KVQuantMode
    }

    class FullAttentionSpec {
        +sliding_window: int|None
        +attention_chunk_size: int|None
    }

    class TQFullAttentionSpec {
        +tq_slot_size: int
    }

    class MLAAAttentionSpec {
        +cache_dtype_str: str|None
        +compress_ratio: int
        +alignment: int|None
    }

    class SlidingWindowSpec {
        +sliding_window: int
        +max_admission_blocks_per_request()
    }

    class ChunkedLocalAttentionSpec {
        +attention_chunk_size: int
    }

    class MambaSpec {
        +shapes: tuple
        +mamba_cache_mode: str
        +num_speculative_blocks: int
    }

```

```

class CrossAttentionSpec
class SinkFullAttentionSpec {
    +sink_len: int|None
}
class EncoderOnlyAttentionSpec

KVCacheSpec <|-- AttentionSpec
AttentionSpec <|-- FullAttentionSpec
FullAttentionSpec <|-- TQFullAttentionSpec
FullAttentionSpec <|-- MLAAAttentionSpec
FullAttentionSpec <|-- SinkFullAttentionSpec
AttentionSpec <|-- SlidingWindowSpec
AttentionSpec <|-- ChunkedLocalAttentionSpec
KVCacheSpec <|-- MambaSpec
AttentionSpec <|-- CrossAttentionSpec
AttentionSpec <|-- EncoderOnlyAttentionSpec

```

量化模式 (kv\_cache\_interface.py:31-72):

```

class KVQuantMode(IntEnum):
    NONE = 0                # 无量化
    FP8_PER_TENSOR = 1      # per-tensor FP8
    INT8_PER_TOKEN_HEAD = 2  # per-token-head INT8
    FP8_PER_TOKEN_HEAD = 3   # per-token-head FP8
    NVFP4 = 4               # NVFP4 打包量化

```

KVCacheGroupSpec (kv\_cache\_interface.py#L744-756):

```

@dataclass
class KVCacheGroupSpec:
    layer_names: list[str]      # 该组包含的层名列表
    kv_cache_spec: KVCacheSpec  # 该组的 cache spec
    is_eagle_group: bool = False # 是否包含 EAGLE/MTP 草稿层

```

### 3.3 UniformTypeKVCacheSpecs

UniformTypeKVCacheSpecs 用于处理同一类型但参数不同的多层合并（如 DeepSeekV4 的不同 head\_size 层）。

## 四、KV Cache Metrics 缓存使用统计

KVCacheMetricsCollector 提供采样式的 Block 级别生命周期指标收集。

### 4.1 架构设计

```

graph LR
    subgraph "Block 生命周期事件"
        A[on_block_allocated] --> B[BlockMetricsState]
        C[on_block_accessed] --> B
        D[on_block_evicted] --> E[KVCacheEvictionEvent]
        B --> E
    end

    subgraph "BlockMetricsState"
        B1[birth_time_ns]
    end

```



```

        B2[last_access_ns]
        B3[access_history deque~4]
    end

    subgraph " 输出指标"
        E1[lifetime_seconds]
        E2[idle_seconds]
        E3[reuse_gaps_seconds]
    end

    B --> E1
    B --> E2
    B3 --> E3

```

## 4.2 核心实现

```

class KVCacheMetricsCollector:
    def __init__(self, sample_rate=0.01):
        self.sample_rate = sample_rate          # 默认 1% 采样率
        self.block_metrics: dict[int, BlockMetricsState] = {}
        self._eviction_events: list[KVCacheEvictionEvent] = []

    def should_sample_block(self) -> bool:
        return random.random() < self.sample_rate

    def on_block_allocated(self, block): ...    # 记录创建时间
    def on_block_accessed(self, block): ...     # 更新访问时间和历史
    def on_block_evicted(self, block): ...      # 生成驱逐事件并移除追踪
    def drain_events(self) -> list: ...         # 取出并清空事件队列

```

BlockMetricsState (kv\_cache\_metrics.py:16-43):

- access\_history 使用 deque(maxlen=4) 有界记录, 防止高频访问导致无限增长
- get\_reuse\_gaps\_seconds() 返回相邻两次访问的时间间隔列表

## 五、KV Offload CPU 卸载

KV Offload 子系统提供将 GPU KV cache 块卸载到 CPU 内存的能力, 支持可插拔的缓存策略。

### 5.1 整体架构

```

graph TB
    subgraph "Scheduler 侧"
        OS["OffloadingSpec<br/>(CPUOffloadingSpec)"]
        OM["OffloadingManager<br/>(CPUOffloadingManager)"]
        RM["FilterReusedOffloadingManager<br/>(复用频率门控装饰器)"]
    end

    subgraph "Worker 侧"
        OW["OffloadingWorker"]
        H_GPU_CPU["SingleDirectionHandler<br/>(GPU→CPU)"]
        H_CPU_GPU["SingleDirectionHandler<br/>(CPU→GPU)"]
        SOR["SharedOffloadRegion<br/>(mmap 共享内存)"]
    end

```

```

subgraph " 策略层"
    POLICY["CachePolicy (ABC)"]
    LRU["LRUCachePolicy"]
    ARC["ARCCachePolicy"]
end

OS --> OM
RM --> OM
OM --> POLICY
POLICY <|-- LRU
POLICY <|-- ARC
OW --> H_GPU_CPU
OW --> H_CPU_GPU
H_GPU_CPU --> SOR
H_CPU_GPU --> SOR

```

## 5.2 OffloadingManager 抽象接口

OffloadingManager 定义了 Scheduler 侧的核心原语:

| 方法                   | 功能                | 说明                              |
|----------------------|-------------------|---------------------------------|
| lookup(key)          | 检查 block 是否已卸载且就绪 | 返回 True/False/None (None 表示需重试) |
| prepare_load(keys)   | 准备读取              | 保护不被驱逐; 返回 LoadStoreSpec        |
| touch(keys)          | 标记最近使用            | 更新 LRU 位置                       |
| complete_load(keys)  | 标记加载完成            | 解除保护                            |
| prepare_store(keys)  | 准备卸载写入            | 可能触发驱逐; 返回 PrepareStoreOutput   |
| complete_store(keys) | 标记存储完成            | block 变为可加载                     |

关键数据结构:

- OffloadKey = block\_hash(字节) + group\_idx(4 字节大端) —唯一标识一个卸载块
- GPULoadStoreSpec —GPU 端 load/store 规格, 含 group\_sizes 和 block\_indices
- CPULoadStoreSpec —CPU 端 load/store 规格

## 5.3 CPUOffloadingManager 实现

CPUOffloadingManager 是 CPU 卸载的具体管理器。

Block Pool 管理:

```

class CPUOffloadingManager(OffloadingManager):
    def __init__(self, num_blocks, cache_policy="lru"):
        self._num_blocks = num_blocks
        self._num_allocated_blocks = 0      # 已分配总数
        self._free_list: list[int] = []     # 可重用 block ID 列表
        self._policy: CachePolicy = policy_cls(cache_capacity=num_blocks)

```

prepare\_store() 流程 (cpu/manager.py:119-172):

1. 过滤已存储的 blocks
2. 计算需要驱逐的数量 (len(keys\_to\_store) - free\_blocks)
3. 调用 policy.evict() 执行驱逐 (排除 protected set)
4. 从 block pool 分配新 block (优先重用 free\_list)

5. 将 (key, block) 插入 policy

## 5.4 缓存策略

**CachePolicy** 抽象 CachePolicy 定义策略接口:

```
class CachePolicy(ABC):
    @abstractmethod
    def get(self, key) -> BlockStatus | None: ...
    @abstractmethod
    def insert(self, key, block) -> None: ...
    @abstractmethod
    def remove(self, key) -> None: ...
    @abstractmethod
    def touch(self, keys) -> None: ...
    @abstractmethod
    def evict(self, n, protected) -> list | None: ...
```

**BlockStatus** 使用 ctypes Structure, 含 **ref\_cnt** (引用计数, -1 表示未就绪) 和 **block\_id**。

**LRU 策略** LRUCachePolicy 基于 OrderedDict:

- touch(): move\_to\_end(key) 移至末尾
- evict(): 从头部扫描 ref\_cnt == 0 且非 protected 的候选者

**ARC 策略** ARCCachePolicy 实现 Adaptive Replacement Cache, 维护 T1 (最近访问)、T2 (频繁访问) 和 ghost 列表。

## 5.5 FilterReusedOffloadingManager — 复用门控

FilterReusedOffloadingManager 是一个装饰器, 包装任意 OffloadingManager, 在 prepare\_store() 前过滤低频 block。

核心逻辑:

```
class FilterReusedOffloadingManager(OffloadingManager):
    def lookup(self, key, req_context):
        # 在 LRU tracker 中记录访问次数
        if key in self.counts:
            self.counts.move_to_end(key)
            self.counts[key] += 1
        else:
            self.counts[key] = 1
        return self._backing.lookup(key, req_context)

    def prepare_store(self, keys, req_context):
        # 只传递达到阈值的 keys
        eligible = [k for k in keys if self.counts.get(k, 0) >= self.store_threshold]
        return self._backing.prepare_store(eligible, req_context)
```

参数说明: - store\_threshold: 必须 2 (1 等于不过滤) - max\_tracker\_size: LRU tracker 最大条目数 (默认 64,000)

## 5.6 Worker 侧传输引擎

**SingleDirectionOffloadingHandler** SingleDirectionOffloadingHandler 处理单向异步传输。

核心机制: - 每个 transfer 使用独立 CUDA stream - Transfer 严格按提交顺序串行执行 (前一个完成后再开始下一个) - 支持 **sub-block** 映射: 当 offloaded block size > GPU block size 时自动拆分

```
compute_sub_block_ptrs() (gpu_worker.py:39-86):  
  
# 例: gpu_block_size=16, cpu_block_size=32 (block_size_factor=2)  
# kv_manager_block_id 0 → kernel block [0, 1]  
# kv_manager_block_id 1 → kernel block [2, 3]  
kernel_block_ids = (  
    kv_manager_block_ids.reshape(-1, 1) * blocks_per_kv_block  
    + kernel_block_arange # [0, 1, ..., factor-1]  
) .reshape(-1)
```

**SharedOffloadRegion** —mmap 共享内存 SharedOffloadRegion 实现多 worker 间的共享 CPU 缓存区域。

布局设计:

文件路径: /dev/shm/vllm\_offload\_{instance\_id}.mmap

物理布局 (interleaved):

```
worker0_block0 | worker1_block0 | ... | worker{M-1}_block0  
worker0_block1 | worker1_block1 | ... | worker{M-1}_block1  
...
```

row\_stride = cpu\_page\_size \* num\_workers  
每个 worker 通过 stride 视图访问自己的区域

关键特性: - 第一个打开文件的 worker 成为 creator 并调用 ftruncate - 其他 worker 等待文件大小达标后 mmap - 使用 MADV\_POPULATE\_WRITE 预分配物理页 - 支持通过 cudaHostRegister 注册为 pinned memory 以加速 DMA

**CPUOffloadingSpec** —规格工厂 CPUOffloadingSpec 将所有组件组装在一起:

```
class CPUOffloadingSpec(OffloadingSpec):  
    def get_manager(self) -> OffloadingManager:  
        # 创建 CPUOffloadingManager (+ 可选 FilterReused 包装)  
        ...  
  
    def get_handlers(self, kv_caches):  
        # 创建 CpuGpuOffloadingHandlers (GPU CPU 双向)  
        yield GPULoadStoreSpec, CPULoadStoreSpec, handlers.gpu_to_cpu_handler  
        yield CPULoadStoreSpec, GPULoadStoreSpec, handlers.cpu_to_gpu_handler
```

## 六、Simple KV Offload

Simple KV Offload 是一个轻量级的 CPU 卸载方案, 不依赖完整的 OffloadingManager 抽象框架, 直接基于 KVCacheCoordinator 的 CPU 副本工作。

### 6.1 架构对比

| 特性       | KV Offload (完整版) | Simple KV Offload      |
|----------|------------------|------------------------|
| 复杂度      | 高 (多抽象层)         | 低 (直接操作 BlockPool)     |
| 缓存策略     | LRU / ARC        | 继承 GPU prefix cache 行为 |
| Store 模式 | eager / lazy     | eager + lazy 双模式       |
| 多节点支持    | (via connector)  | 单机                     |

|      |                  |                   |
|------|------------------|-------------------|
| 特性   | KV Offload (完整版) | Simple KV Offload |
| 适用场景 | 生产级部署            | 快速验证、原型           |

## 6.2 SimpleCPUOffloadScheduler

SimpleCPUOffloadScheduler 是调度器侧的管理器。

初始化—从 GPU 配置派生 CPU 配置 (simple\_kv\_offload/manager.py:160-187):

```
@staticmethod
def _derive_cpu_config(gpu_config, cpu_capacity_bytes):
    gpu_total_bytes = sum(t.size for t in gpu_config.kv_cache_tensors)
    num_gpu_blocks = gpu_config.num_blocks
    num_cpu_blocks = max(1, num_gpu_blocks * cpu_capacity_bytes // gpu_total_bytes)
    # 保持相同的 kv_cache_groups, 仅缩放 num_blocks
    return KVCacheConfig(num_blocks=num_cpu_blocks, ...)
```

两种 **Store** 模式:

**Lazy Mode** —游标扫描 \_\_prepare\_lazy\_store\_specs(): 使用单遍游标扫描 GPU free queue, 选择接近 eviction 边缘的 cached block 进行卸载。

```
# 游标沿 free_queue 前进, 覆盖 target_free 个位置
while covered < self._target_free and len(gpu_ids) < num_cpu_free:
    bhash = node.block_hash
    if bhash is not None and not node.is_null and cpu_pool 未缓存该 hash:
        gpu_ids.append(node.block_id)
        block_hashes.append(bhash)
    covered += 1
    node = node.next_free_block
```

**Eager Mode** —请求驱动 \_\_prepare\_eager\_store\_specs(): 跟踪每个请求的新计算 block, 确认 KV 数据已写回后批量卸载。

两阶段扫描: 1. **Phase 1:** 分类 block 为 cached vs to-store (跳过 null 和已缓存的) 2. **Phase 2:** 批量分配 CPU block 并 stamp hash

## 6.3 SimpleCPUOffloadWorker

SimpleCPUOffloadWorker 是 Worker 侧处理器。

**KV Cache** 注册 (worker.py:66-182): - 自动去重共享同一底层 storage 的多个层 - 推导物理 layout (FlashAttn 的 (2, num\_blocks, ...) vs FlashInfer 的 (num\_blocks, ...)) - 分配 pinned CPU tensor (使用 cudaHostRegister 避免 PyTorch 的 2 幂向上取整)

**DmaCopyBackend** (copy\_backend.py): - 后台线程执行 cuMemcpyBatchAsync - 低优先级 CUDA stream (不干扰模型计算) - Event 驱动的完成通知

## 6.4 Metadata 定义

SimpleCPUOffloadMetadata (Scheduler → Worker):

```
@dataclass
class SimpleCPUOffloadMetadata(KVConnectorMetadata):
    load_event: int # 加载任务 ID (-1 = 无)
    load_gpu_blocks: list[int]
    load_cpu_blocks: list[int]
```

```

store_event: int          # 存储任务 ID
store_gpu_blocks: list[int]
store_cpu_blocks: list[int]
need_flush: bool          # 是否有请求被抢占

```

SimpleCPUOffloadWorkerMetadata (Worker → Scheduler):

```

@dataclass
class SimpleCPUOffloadWorkerMetadata(KVConnectorWorkerMetadata):
    completed_store_events: dict[int, int] # event_idx → 完成计数
    # aggregate() 跨 worker 汇总

```

## 七、KV Transfer 跨节点传输

KV Transfer 子系统支持在 \*\* disaggregated 架构 \*\* 中跨节点传输 KV cache 数据（如 Prefill 节点 → Decode 节点）。

### 7.1 KVConnectorBase\_V1 核心接口

KVConnectorBase\_V1 定义了双向通信协议：

```

sequenceDiagram
    participant S as Scheduler<br/>(KVConnector)
    participant W as Worker<br/>(KVConnector)
    participant R as Remote<br/>(KV Cache)

    Note over S: get_num_new_matched_tokens()
    S->>R: 查询远程 cache 命中数
    R-->>S: (hit_count, is_async)

    Note over S: build_connector_meta()
    S->>W: KVConnectorMetadata<br/>(load/store specs)

    Note over W: start_load_kv()
    W->>R: 异步加载 KV 数据
    W->>W: 模型前向计算 <br/>(与 load 重叠)

    Note over W: save_kv_layer()
    W->>R: 异步保存 KV 数据

    Note over W: get_finished()
    W-->>S: finished_recving/sending req_ids

    Note over S: update_connector_output()
    S->>S: 处理完成事件

```

**Scheduler** 侧方法:

| 方法                           | 说明                       |
|------------------------------|--------------------------|
| get_num_new_matched_tokens() | 查询远程 cache 可匹配的新 token 数 |
| update_state_after_alloc()   | 分配后更新状态                  |
| build_connector_meta()       | 构建 metadata 发送给 worker   |
| update_connector_output()    | 处理 worker 回传的完成事件        |
| request_finished()           | 请求结束时的清理，返回是否延迟释放        |

| 方法                         | 说明             |
|----------------------------|----------------|
| <code>take_events()</code> | 收集 KV cache 事件 |

**Worker** 侧方法:

| 方法                                     | 说明                      |
|----------------------------------------|-------------------------|
| <code>register_kv_caches()</code>      | 注册本地 KV cache tensors   |
| <code>bind_connector_metadata()</code> | 接收 scheduler 的 metadata |
| <code>start_load_kv()</code>           | 启动异步加载                  |
| <code>wait_for_layer_load()</code>     | 等待指定层加载完成               |
| <code>save_kv_layer()</code>           | 启动异步保存某层                |
| <code>wait_for_save()</code>           | 等待所有保存完成                |
| <code>get_finished()</code>            | 上报完成的异步传输               |
| <code>handle_preemptions()</code>      | 抢占前的同步处理                |

## 7.2 连接器注册与发现

KVConnectorFactory 通过名称注册制管理连接器类型:

# 内置注册

```
OffloadingSpecFactory.register_spec(
    "CPUOffloadingSpec",
    "vllm.v1.kv_offload.cpu.spec",
    "CPUOffloadingSpec"
)
```

`kv_transfer_state.py` 管理全局连接器实例:

```
_KV_CONNECTOR_AGENT: KVConnectorBaseType | None = None

def ensure_kv_transfer_initialized(vllm_config, kv_cache_config):
    global _KV_CONNECTOR_AGENT
    _KV_CONNECTOR_AGENT = KVConnectorFactory.create_connector(
        config=vllm_config, role=KVConnectorRole.WORKER, ...
    )
```

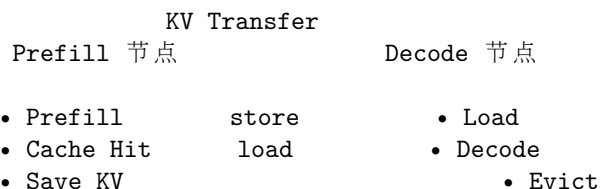
## 7.3 支持的连接器类型

`kv_connector/v1/` 目录下实现了多种连接器后端:

| 连接器        | 文件                                           | 用途                |
|------------|----------------------------------------------|-------------------|
| LMCache    | <code>lmcache_connector.py</code>            | LMCache 集成        |
| NIXL       | <code>nixl/connector.py</code>               | NVIDIA NIXL 传输    |
| P2P NCCL   | <code>p2p/p2p_nccl_connector.py</code>       | 点对点 NCCL 传输       |
| Mooncake   | <code>mooncake/mooncake_connector.py</code>  | Mooncake 分离架构     |
| HF3FS      | <code>hf3fs/hf3fs_connector.py</code>        | HDFS/Federated FS |
| Moriio     | <code>moriio/moriio_connector.py</code>      | Moriio 分布式缓存      |
| Offloading | <code>offloading_connector.py</code>         | 通用卸载连接器           |
| Simple CPU | <code>simple_cpu_offload_connector.py</code> | 简单 CPU 卸载         |

## 7.4 Disaggregated Prefill workflow

在分离式架构中，Prefill 节点和 Decode 节点分别运行，KV cache 通过 Connector 传输：



**HMA (Hybrid Memory Allocator)** 支持：通过 SupportsHMA 接口标记支持混合内存分配的连接器，额外实现 request\_finished\_all\_groups() 方法处理跨 group 的异步释放。

## 八、Block Table 管理

### 8.1 BlockTable

BlockTable 管理单个 KV cache group 的 block table，是 attention kernel 直接消费的数据结构。

核心字段：

```
class BlockTable:
    def __init__(self, block_size, max_num_reqs, max_num_blocks_per_req,
                  max_num_batched_tokens, pin_memory, device,
                  kernel_block_size, cp_kv_cache_interleave_size):
        # hybrid block 支持: allocation block_size kernel block_size
        if kernel_block_size == block_size:
            self.blocks_per_kv_block = 1
            self.use_hybrid_blocks = False
        else:
            self.blocks_per_kv_block = block_size // kernel_block_size
            self.use_hybrid_blocks = True

        self.block_table = self._make_buffer(max_num_reqs, max_num_blocks_per_req, int32)
        self.slot_mapping = self._make_buffer(max_num_batched_tokens, int64)
        self.num_blocks_per_row = np.zeros(max_num_reqs, dtype=np.int32)
```

**Hybrid Block 映射** (block\_table.py:173-201):

当 allocation block size 与 kernel block size 不同时（如混合模型），需要映射：

```
@staticmethod
def map_to_kernel_blocks(kv_manager_block_ids, blocks_per_kv_block, kernel_block_arange):
    """
    例: kv_manager block size=32, kernel block size=16
    input:  [0, 1, 2]
    output: [0, 1, 2, 3, 4, 5]
    # 每个 manager block → blocks_per_kv_block 个 kernel blocks
    """
    kernel_block_ids = (
        kv_manager_block_ids.reshape(-1, 1) * blocks_per_kv_block
        + kernel_block_arange
    ).reshape(-1)
```

主要操作：



| 方法                                          | 功能                            |
|---------------------------------------------|-------------------------------|
| <code>append_row(block_ids, row_idx)</code> | 向请求行追加 block IDs              |
| <code>add_row(block_ids, row_idx)</code>    | 设置请求行的 block IDs              |
| <code>clear_row(row_idx)</code>             | 清空请求行                         |
| <code>move_row(src, tgt)</code>             | 移动行 (用于 swap)                 |
| <code>swap_row(src, tgt)</code>             | 交换两行                          |
| <code>compute_slot_mapping()</code>         | Triton kernel 计算 slot mapping |
| <code>commit_block_table(num_reqs)</code>   | 将 CPU 数据复制到 GPU               |

## 8.2 MultiGroupBlockTable

MultiGroupBlockTable 为每个 KV cache group 维护一个独立的 BlockTable:

```
class MultiGroupBlockTable:
    def __init__(self, max_num_reqs, max_model_len, max_num_batched_tokens,
                 pin_memory, device, block_sizes, kernel_block_sizes, ...):
        self.block_tables = [
            BlockTable(block_size, ..., kernel_block_size, ...)
            for block_size, kernel_block_size in zip(block_sizes, kernel_block_sizes)
        ]

    def append_row(self, block_ids, row_idx):
        for i, block_table in enumerate(self.block_tables):
            block_table.append_row(block_ids[i], row_idx)
```

TRTLLM 对齐: `max_num_blocks` 会向上对齐到 `128 / block_size` 的倍数以满足某些 attention 后端的要求。

## 8.3 Slot Mapping Kernel

`_compute_slot_mapping_kernel` 是一个 Triton kernel, 将 token position 映射到物理 slot ID:

```
@triton.jit
def _compute_slot_mapping_kernel(
    num_tokens, query_start_loc_ptr, positions_ptr,
    block_table_ptr, block_table_stride, block_size,
    slot_mapping_ptr,
    TOTAL_CP_WORLD_SIZE, TOTAL_CP_RANK,
    CP_KV_CACHE_INTERLEAVE_SIZE, PAD_ID, BLOCK_SIZE):
    # 对每个 request:
    #   virtual_block_size = block_size * TOTAL_CP_WORLD_SIZE
    #   block_index = pos // virtual_block_size
    #   判断是否属于当前 rank 的 local slot
    #   local_slot = block_number * block_size + local_offset
    #   不属于当前 rank 则填 PAD_SLOT_ID
```

**Context Parallel 支持:** 通过 `TOTAL_CP_WORLD_SIZE` 和 `TOTAL_CP_RANK` 参数支持 Decode Context Parallel 和 Prefill Context Parallel。`CP_KV_CACHE_INTERLEAVE_SIZE` 控制 KV cache 在 rank 间的交错粒度。

## 九、前缀缓存 Prefix Caching

Prefix Caching 是 vLLM 的核心性能优化特性, 通过检测并复用公共 prompt 前缀来避免重复计算。

### 9.1 查找最长缓存命中

入口: KVCacheManager.get\_computed\_blocks()

```
def get_computed_blocks(self, request: Request) -> tuple[KVCacheBlocks, int]:
    if not self.enable_caching or request.skip_reading_prefix_cache:
        return self.empty_kv_cache_blocks, 0

    # 必须至少重算最后一个 token 以获取 logits
    max_cache_hit_length = request.num_tokens - 1
    computed_blocks, num_new_computed_tokens = (
        self.coordinator.find_longest_cache_hit(
            request.block_hashes, max_cache_hit_length
        )
    )

    if self.log_stats:
        self.prefix_cache_stats.record(
            num_tokens=request.num_tokens,
            num_hits=num_new_computed_tokens,
            preempted=request.num_preemptions > 0,
        )

    return self.create_kv_cache_blocks(computed_blocks), num_new_computed_tokens
```

### 9.2 各 Manager 的查找策略对比

| Manager       | 搜索方向    | 匹配策略                       | 特殊处理                |
|---------------|---------|----------------------------|---------------------|
| FullAttention | 左 → 右   | 顺序匹配直到 miss                | EAGLE pop 最后一块      |
| SlidingWindow | 右 → 左   | 反向找连续 window 内匹配           | null padding; EAGLE |
| ChunkedLocal  | 分区 + 顺序 | 窗口外 =null, 窗口内<br>=hash 查找 | 不支持 EAGLE           |
| Mamba         | 右 → 左   | 只取最后一个匹配块                  | align mode 特殊处理     |

### 9.3 FullAttentionManager 详细流程

find\_longest\_cache\_hit:

输入: block\_hashes=[H0, H1, H2, H3, H4, ...], max\_length=20, block\_size=16

Step 1: 检查 H0 → cache hit? → 是 → computed=[B0]

Step 2: 检查 H1 → cache hit? → 是 → computed=[B0, B1]

Step 3: 检查 H2 → cache miss! → break

返回: ([B0, B1], 32) # 2 blocks × 16 tokens = 32 tokens

### 9.4 SlidingWindowManager 详细流程

find\_longest\_cache\_hit:

输入: sliding\_window=8, block\_size=4, max\_length=20

需要连续匹配:  $\text{ceil}((8-1)/4) = 1$  个 block (实际上需要更多以覆盖窗口)

从右向左搜索:

```
i=4: H4 → hit? → [_, _, _, B4], contiguous=1
i=3: H3 → hit? → [_, _, B3, B4], contiguous=2 达标!
→ trim trailing → [_, _, B3, B4]
```

返回: ([NULL, NULL, NULL, B3, B4], 16) # 4 blocks, 但前3个是null

### 9.5 缓存命中后的处理路径

`allocate_slots()` 中的三阶段分配 (kv\_cache\_manager.py:225-416):

阶段 1: `remove_skipped_blocks()`

→ 释放滑动窗口外的旧 block (替换为 `null_block`)

阶段 2: `allocate_new_computed_blocks()`

→ `touch` 前缀缓存命中的 block (防止被驱逐)

→ 追加到请求的 block 列表

→ 为 `external computed token` 分配新 block

阶段 3: `allocate_new_blocks()`

→ 为待计算的新 token 分配 block

→ 返回新分配的 block 列表

最后: `cache_blocks()`

→ 将满 block 标记为可复用 (写入 `block_hash → block` 映射)

### 9.6 BlockPool 中的缓存数据结构

`BlockHashToBlockMap` 维护 `hash→block` 的映射:

```
class BlockHashToBlockMap:
    """block_hash → KVCacheBlock | dict[int, KVCacheBlock]"""
    def get_one_block(self, key) -> KVCacheBlock | None: ...
    def insert(self, key, block) -> None: ...
```

配合 `FreeKVCacheBlockQueue` (free list) 实现: - 新分配的 block 来自 free list - 被 cache 的 block 进入 `cached_block_hash_to_block` 映射 - 当 `ref_cnt` 降为 0 时, `cached block` 可被重新利用或驱逐

### 9.7 与调度器的集成

Prefix Cache 与调度器的交互点:

1. 调度前查询: `get_computed_blocks()` 告知调度器有多少 token 可以跳过
2. 全序列准入检查: `full_sequence_must_fit=True` 时确保整个序列能放入
3. **Cascade Attention** 优化: `get_num_common_prefix_blocks()` 让调度器知道公共前缀长度, 用于 cascade attention 的 KV fuse 优化
4. 统计报告: `make_prefix_cache_stats()` 每 interval 输出命中率等指标

## 十、数据流全景图

flowchart TB

```
subgraph Request ["Request 生命周期"]
    REQ["新 Request 到达"]
end
```

```
subgraph Scheduler ["Scheduler 侧"]
```

```

    GCM["KVCacheManager.get_computed_blocks()<br/>→ find_longest_cache_hit()"]
    ALLOC["KVCacheManager.allocate_slots()<br/>→ 三阶段分配"]
    FREE["KVCacheManager.free()"]
    CACHE["KVCacheManager.cache_blocks()"]
end

subgraph Coordinator ["Coordinator 层"]
    COORD_FIND["Coordinator.find_longest_cache_hit()<br/> 固定点算法 (Hybrid)"]
    COORD_ALLOC["Coordinator.allocate_new_blocks()<br/> 分发到各 SingleTypeMgr"]
end

subgraph Managers ["SingleType Managers"]
    FA_MGR["FullAttentionManager<br/> 左 → 右顺序匹配"]
    SW_MGR["SlidingWindowManager<br/> 右 → 左反向匹配"]
    CLA_MGR["ChunkedLocalAttnMgr<br/> 分区 + 窗口内匹配"]
    MB_MGR["MambaManager<br/> 末尾单块匹配"]
end

subgraph BlockPool ["BlockPool"]
    BP_ALLOC["get_new_blocks()"]
    BP_FREE["free_blocks()"]
    BP_CACHE["cache_full_blocks()"]
    BP_LOOKUP["get_cached_block()"]
    BP_EVICT["evict_blocks()"]
end

subgraph BlockTable ["BlockTable"]
    BT_APPEND["append_row()"]
    BT_COMPUTE["compute_slot_mapping()<br/>Triton Kernel"]
    BT_COMMIT["commit_block_table()"]
end

subgraph Offload [" 可选: Offload Path"]
    PREP_STORE["prepare_store()"]
    PREP_LOAD["prepare_load()"]
    TRANSFER["GPU CPU Async Transfer"]
end

REQ --> GCM
GCM --> COORD_FIND
COORD_FIND --> FA_MGR & SW_MGR & CLA_MGR & MB_MGR
FA_MGR & SW_MGR & CLA_MGR & MB_MGR --> BP_LOOKUP
BP_LOOKUP --> GCM

GCM --> ALLOC
ALLOC --> COORD_ALLOC
COORD_ALLOC --> FA_MGR & SW_MGR & CLA_MGR & MB_MGR
FA_MGR & SW_MGR & CLA_MGR & MB_MGR --> BP_ALLOC
BP_ALLOC --> ALLOC

ALLOC --> BT_APPEND
BT_APPEND --> BT_COMPUTE
BT_COMPUTE --> BT_COMMIT

```

```

ALLOC --> CACHE
CACHE --> BP_CACHE

FREE --> BP_FREE
BP_FREE --> BP_EVICT

BP_CACHE -.->" 可选"| PREP_STORE
PREP_STORE --> TRANSFER
TRANSFER --> PREP_LOAD
PREP_LOAD -.-> ALLOC

style REQ fill:#e3f2fd,stroke:#1565c0,color:#000
style GCM fill:#e0f2f1,stroke:#00695c,color:#000
style ALLOC fill:#e0f2f1,stroke:#00695c,color:#000
style COORD_FIND fill:#f3e5f5,stroke:#7b1fa2,color:#000
style BP_CACHE fill:#fff8e1,stroke:#f9a825,color:#000
style TRANSFER fill:#fce4ec,stroke:#c62828,color:#000

```

## 附录：关键文件索引

| 文件路径                                    | 核心内容                                                    |
|-----------------------------------------|---------------------------------------------------------|
| kv_cache_manager.py                     | KVCacheManager 顶层抽象、KVCacheBlocks 数据类                   |
| single_type_kv_cache_manager.py         | 单类型基类及 6 种具体 Manager 实现                                 |
| encoder_cache_manager.py                | 编码器输出缓存管理                                               |
| kv_cache_coordinator.py                 | 三种 Coordinator (NoPrefix/Unitary/Hybrid)                |
| kv_cache_interface.py                   | KVCacheSpec 层次体系、KVCacheConfig、量化模式                     |
| kv_cache_metrics.py                     | 采样式 Block 生命周期指标收集                                      |
| kv_offload/base.py                      | OffloadingManager 抽象接口、LoadStoreSpec                    |
| kv_offload/reuse_manager.py             | FilterReusedOffloadingManager 复用门控                      |
| kv_offload/cpu/manager.py               | CPUOffloadingManager 具体实现                               |
| kv_offload/cpu/policies/                | LRU / ARC 缓存替换策略                                        |
| kv_offload/cpu/gpu_worker.py            | GPU CPU 异步传输 Handler                                    |
| kv_offload/cpu/shared_offload_region.py | mmap 共享内存区域                                             |
| kv_offload/cpu/spec.py                  | CPUOffloadingSpec 规格工厂                                  |
| simple_kv_offload/manager.py            | SimpleCPUOffloadScheduler (lazy/eager)                  |
| simple_kv_offload/worker.py             | SimpleCPUOffloadWorker                                  |
| simple_kv_offload/copy_backend.py       | DMA Copy 后台线程                                           |
| kv_connector/v1/base.py                 | KVConnectorBase_V1 完整接口定义                               |
| kv_transfer_state.py                    | 全局连接器实例管理                                               |
| block_table.py                          | BlockTable / MultiGroupBlockTable + Triton slot mapping |
| block_pool.py                           | BlockPool、BlockHashToBlockMap、FreeKVCacheBlockQueue     |

## 10. 采样与生成策略分析

定位：本文档深入分析 vLLM 的采样 (Sampling) 与生成策略核心机制，涵盖从用户配置参数到最终 token 生成的完整数据流。

## 总体架构概览

```
graph LR
    subgraph SamplingParams ["SamplingParams<br/> 采样参数"]
        A1[temperature]
        A2[top_p / top_k]
        A3[penalties]
        A4[stop sequences]
        A5[max_tokens]
    end

    subgraph Core ["Sampler 采样核心"]
        B1[Logits Processors]
        B2[Temperature Scaling]
        B3[TopK/TopP Filtering]
        B4[Token Sampling]
    end

    subgraph Advanced ["高级采样策略"]
        C1[RejectionSampler<br/> 拒绝采样]
        C2[ParallelSampling<br/> 并行采样]
        C3[Logprobs Tracking<br/> 概率追踪]
        C4[ThinkingBudgetState<br/> 推理预算控制]
    end

    SamplingParams --> Core
    Core --> Advanced

    style SamplingParams fill:#fbe9e7,stroke:#e64a19,color:#000
    style Core fill:#fff3e0,stroke:#f57c00,color:#000
    style Advanced fill:#fce4ec,stroke:#c62828,color:#000
```

---

## 一、SamplingParams 采样参数详解

源码文件: `sampling_params.py`

`SamplingParams` 是 vLLM 中定义文本生成采样参数的核心数据类，遵循 OpenAI 文本补全 API 规范，并额外支持 beam search。

### 1.1 核心参数定义与默认值

```
# 源码位置: sampling_params.py L168-174
class SamplingParams(
    PydanticMsgspecMixin,
    msgspec.Struct,
    omit_defaults=True,
    dict=True,
):
    """Sampling parameters for text generation."""
```

基础采样参数

| 参数          | 类型    | 默认值 | 说明                                                      |
|-------------|-------|-----|---------------------------------------------------------|
| n           | int   | 1   | 每个 prompt 返回的输出数量（最大受 VLLM_MAX_N_SEQUENCES 控制，默认 16384） |
| temperature | float | 1.0 | 控制随机性。0 表示贪婪解码；低值更确定性，高值更随机                             |
| top_p       | float | 1.0 | 核累积概率阈值 (0, 1]。1.0 表示考虑所有 token                         |
| top_k       | int   | 0   | 考虑的最高概率 token 数量。0 或 -1 表示禁用                            |
| min_p       | float | 0.0 | 最小概率阈值 [0, 1]，相对于最高概率 token                             |

源码定义 (sampling\_params.py L205-218):

```
temperature: float = 1.0
"""Controls the randomness of the sampling. Lower values make the model
more deterministic, while higher values make it more random. Zero
means greedy sampling."""
top_p: float = 1.0
"""Controls the cumulative probability of the top tokens to consider.
Must be in (0, 1]. Set to 1 to consider all tokens."""
top_k: int = 0
"""Controls the number of top tokens to consider. Set to 0 (or -1) to
consider all tokens."""
min_p: float = 0.0
"""Represents the minimum probability for a token to be considered,
relative to the probability of the most likely token. Must be in [0, 1].
Set to 0 to disable this."""
```

## 1.2 Penalty 惩罚机制参数

**Presence Penalty**（存在惩罚） 源码: sampling\_params.py L193-196

```
presence_penalty: float = 0.0
"""Penalizes new tokens based on whether they appear in the generated text
so far. Values > 0 encourage the model to use new tokens, while values < 0
encourage the model to repeat tokens."""
```

- 取值范围: [-2.0, 2.0]（在 `_verify_args` 中验证）
- 作用原理: 如果 token 已出现在生成文本中，对其 logit 加上/减去该惩罚值
- 特点: 只关注 token 是否出现，不关心出现次数

**Frequency Penalty**（频率惩罚） 源码: sampling\_params.py L197-200

```
frequency_penalty: float = 0.0
"""Penalizes new tokens based on their frequency in the generated text so
far. Values > 0 encourage the model to use new tokens, while values < 0
encourage the model to repeat tokens."""
```

- 取值范围: [-2.0, 2.0]
- 作用原理: 根据 token 在已生成文本中的出现频率进行线性惩罚
- 特点: 频率越高，惩罚越大

**Repetition Penalty** (重复惩罚) 源码: `sampling_params.py` L201-204

```
repetition_penalty: float = 1.0
"""Penalizes new tokens based on whether they appear in the prompt and the
generated text so far. Values > 1 encourage the model to use new tokens,
while values < 1 encourage the model to repeat tokens."""
```

- 取值范围: ( $> 0$ )
- 作用原理: 对已在 prompt 和生成文本中出现的 token 的 logit 进行乘法惩罚
- 特点: 与前两个不同, 这是乘法操作而非加法

### 1.3 Stop Sequences 与 Token 控制

源码: `sampling_params.py` L221-234

```
stop: str | list[str] | None = None
"""String(s) that stop the generation when they are generated."""
```

```
stop_token_ids: list[int] | None = None
"""Token IDs that stop the generation when they are generated."""
```

```
ignore_eos: bool = False
"""Whether to ignore the EOS token and continue generating."""
```

```
max_tokens: int | None = 16
"""Maximum number of tokens to generate per output sequence."""
```

```
min_tokens: int = 0
"""Minimum number of tokens to generate before EOS can be generated."""
```

### 1.4 Logprobs 相关参数

源码: `sampling_params.py` L236-258

```
logprobs: int | None = None
"""Number of log probabilities to return per output token.
When set to -1, return all vocab_size log probabilities."""
```

```
prompt_logprobs: int | None = None
"""Number of log probabilities to return per prompt token.
When set to -1, return all vocab_size log probabilities."""
```

```
logprob_token_ids: list[int] | None = None
"""Specific token IDs to return logprobs for. More efficient than
logprobs=-1 when you only need logprobs for a small set of tokens."""
```

```
flat_logprobs: bool = False
"""Whether to return logprobs in flatten format (FlatLogprob)."""
```

### 1.5 其他重要参数

| 参数                                         | 默认值               | 说明                              |
|--------------------------------------------|-------------------|---------------------------------|
| <code>seed</code>                          | <code>None</code> | 随机种子, 用于可复现生成                   |
| <code>detokenize</code>                    | <code>True</code> | 是否对输出进行 <code>detokenize</code> |
| <code>skip_special_tokens</code>           | <code>True</code> | 是否跳过特殊 token                    |
| <code>spaces_between_special_tokens</code> | <code>True</code> | 特殊 token 之间是否添加空格               |



| 参数                         | 默认值                          | 说明                                |
|----------------------------|------------------------------|-----------------------------------|
| include_stop_str_in_output | False                        | 输出中是否包含停止字符串                      |
| output_kind                | RequestOutputKind.CUMULATIVE | 输出模式: CUMULATIVE/DELTA/FINAL_ONLY |
| thinking_token_budget      | None                         | 推理阶段的最大 token 预算                  |
| repetition_detection       | None                         | N-gram 重复检测参数                     |

## 1.6 参数验证逻辑

源码: sampling\_params.py L391-437

```
def __post_init__(self) -> None:
    # 温度过低警告
    if 0 < self.temperature < _MAX_TEMP: # _MAX_TEMP = 1e-2
        logger.warning(f"temperature {self.temperature} is less than {_MAX_TEMP}...")
        self.temperature = max(self.temperature, _MAX_TEMP)

    # 贪婪采样时强制重置相关参数
    if self.temperature < _SAMPLING_EPS: # _SAMPLING_EPS = 1e-5
        self.top_p = 1.0
        self.top_k = 0
        self.min_p = 0.0
        self._verify_greedy_sampling()
```

## 1.7 采样类型判断

源码: sampling\_params.py L614-620

```
@cached_property
def sampling_type(self) -> SamplingType:
    if self.temperature < _SAMPLING_EPS:
        return SamplingType.GREEDY # 贪婪解码
    if self.seed is not None:
        return SamplingType.RANDOM_SEED # 带种子的随机采样
    return SamplingType.RANDOM # 纯随机采样
```

## 二、Sampler 采样核心

源码文件: sampler.py

Sampler 是 vLLM v1 架构中的核心采样层，负责从模型输出的 logits 中采样下一个 token。

### 2.1 整体处理流程

源码: sampler.py L22-60

flowchart TD

```
A[Input Logits] --> B{需要 Logprobs?}
B -->|Yes| C[计算/克隆 Raw Logprobs]
B -->|No| D[转换为 Float32]
C --> D
D --> E[apply_logits_processors]
E --> F[sample 方法]
F --> G{All Greedy?}
```

```

G -->|Yes| H[argmax 采样]
G -->|No| I{有 Greedy 请求?}
I -->|Yes| J[预计算 Greedy 结果]
I -->|No| K[仅 Random 采样]
J --> L[Apply Temperature]
K --> L
L --> M[Argmax-Invariant Processors]
M --> N[TopK/TopP 过滤]
N --> O[Random Sample]
O --> P{混合模式?}
P -->|Yes| Q[torch.where 合并]
P -->|No| R[返回结果]
Q --> R
R --> S[Gather Top-K Logprobs]
S --> T[返回 SamplerOutput]

style A fill:#e3f2fd,stroke:#1976d2,color:#000
style T fill:#c8e6c9,stroke:#388e3c,color:#000

```

## 2.2 forward() 主入口方法

源码: sampler.py L68-143

```

def forward(
    self,
    logits: torch.Tensor,                # [batch_size, vocab_size]
    sampling_metadata: SamplingMetadata,
    predict_bonus_token: bool = False,
    logprobs_mode_override: LogprobsMode | None = None,
) -> SamplerOutput:
    # 1. 保存原始 logits 用于 top-k logprobs 计算
    num_logprobs = sampling_metadata.max_num_logprobs
    if num_logprobs is not None:
        if logprobs_mode == "raw_logprobs":
            raw_logprobs = self.compute_logprobs(logits)
        elif logprobs_mode == "raw_logits":
            raw_logprobs = logits.clone() if logits.dtype == torch.float32 \
                else logits.to(torch.float32)

    # 2. 转换为 float32
    logits = logits.to(torch.float32)

    # 3. 应用 logits processors
    logits = self.apply_logits_processors(logits, sampling_metadata, predict_bonus_token)

    # 4. 执行采样
    sampled, processed_logprobs = self.sample(logits, sampling_metadata)

    # 5. 处理特定 token IDs 的 logprobs (高效模式)
    if sampling_metadata.logprob_token_ids:
        logprob_token_ids_tensors = self.gather_specific_token_logprobs(...)

    # 6. 收集 top-k logprobs
    if num_logprobs is not None and num_logprobs != -1:
        logprobs_tensors = self.gather_logprobs(raw_logprobs, num_logprobs, sampled)

```

```

# 7. 构建输出
return SamplerOutput(
    sampled_token_ids=sampled.unsqueeze(-1), # [batch_size, 1]
    logprobs_tensors=logprobs_tensors,
)

```

### 2.3 apply\_logits\_processors() - Logits Processor 链

源码: sampler.py L357-406

此方法按顺序应用所有 logits 处理器:

```

def apply_logits_processors(self, logits, sampling_metadata, predict_bonus_token):
    # 1. 准备 output_token_ids (结合 spec tokens)
    output_token_ids = sampling_metadata.output_token_ids
    if predict_bonus_token and (any_penalties_or_bad_words or needs_thinking_combine):
        output_token_ids = self._combine_outputs_with_spec_tokens(
            output_token_ids, sampling_metadata.spec_token_ids
        )

    # 2. 应用 allowed token ids 白名单
    if sampling_metadata.allowed_token_ids_mask is not None:
        logits.masked_fill_(sampling_metadata.allowed_token_ids_mask, float("-inf"))

    # 3. 应用 bad words 排除
    if bad_words_token_ids:
        apply_bad_words(logits, bad_words_token_ids, output_token_ids)

    # 4. 应用非 argmax-invariant processors (影响贪婪采样)
    for processor in sampling_metadata.logitsprocs.non_argmax_invariant:
        logits = processor.apply(logits)

    # 5. 应用 penalties (repetition/frequency/presence)
    logits = self.apply_penalties(logits, sampling_metadata, output_token_ids)

    # 6. 应用 thinking budget state
    if holder is not None and holder.has_tracked_requests():
        holder.update_state(output_token_ids, spec_token_ids)
        logits = holder.apply_to_logits(logits, predict_bonus_token, spec_token_ids)

    return logits

```

**Processor** 执行顺序: 1. Allowed Token IDs 白名单过滤 2. Bad Words 排除 3. Non-argmax-invariant processors (如 MinTokens、LogitBias) 4. Penalties (Repetition/Frequency/Presence) 5. Thinking Budget State 强制 token

### 2.4 sample() 核心采样方法

源码: sampler.py L232-288

```

def sample(self, logits, sampling_metadata, logprobs_mode_override=None):
    """Sample logits based on sampling metadata."""

    assert not (sampling_metadata.all_greedy and sampling_metadata.all_random)

    # ===== 阶段 1: 处理贪婪采样请求 =====

```

```

if sampling_metadata.all_random:
    greedy_sampled = None
else:
    greedy_sampled = self.greedy_sample(logits) # argmax
    if sampling_metadata.all_greedy:
        # 全部是贪婪采样, 直接返回
        processed_logprobs = ...
        return greedy_sampled, processed_logprobs

# ===== 阶段 2: Temperature 缩放 =====
logits = self.apply_temperature(
    logits, sampling_metadata.temperature, sampling_metadata.all_random
)

# ===== 阶段 3: Argmax-invariant processors (仅影响随机采样) =====
for processor in sampling_metadata.logitsprocs.argmax_invariant:
    logits = processor.apply(logits) # 如 min_p processor

# ===== 阶段 4: TopK/TopP 过滤 + 随机采样 =====
random_sampled, processed_logprobs = self.topk_topp_sampler(
    logits,
    sampling_metadata.generators,
    sampling_metadata.top_k,
    sampling_metadata.top_p,
)

# ===== 阶段 5: 混合模式合并 =====
if greedy_sampled is None:
    return random_sampled, processed_logprobs

# 使用 torch.where 按 temperature 阈值选择贪婪或随机结果
sampled = torch.where(
    sampling_metadata.temperature < _SAMPLING_EPS, # 1e-5
    greedy_sampled,
    random_sampled,
    out=greedy_sampled, # 复用 tensor 以节省内存
)
return sampled, processed_logprobs

```

## 2.5 Temperature 应用逻辑

源码: sampler.py L216-226

```

@staticmethod
def apply_temperature(logits, temp, all_random):
    """
    使用 in-place 除法避免创建新 tensor。
    对贪婪请求避免除以零。
    """
    if not all_random:
        temp = torch.where(temp < _SAMPLING_EPS, 1.0, temp)
    return logits.div_(temp.unsqueeze(dim=1))

```

关键点: - 贪婪采样的请求 temperature 设为 1.0 (相当于不缩放) - 使用 div\_() in-place 操作节省内存 - 实际公式:  
 $\text{logits\_after} = \text{logits} / \text{temperature}$

## 2.6 Logprobs 计算与收集

源码: sampler.py L290-342

`compute_logprobs()`

```
@staticmethod
def compute_logprobs(logits):
    """ 计算 log_softmax """
    return logits.log_softmax(dim=-1, dtype=torch.float32)
```

`gather_logprobs()`

```
@staticmethod
def gather_logprobs(logprobs, num_logprobs, token_ids):
    """
    收集 top-k 和采样 token 的 logprobs。

    Args:
        logprobs: [num_tokens, vocab_size]
        num_logprobs: 保留的最大 logprobs 数量
        token_ids: 采样/prompt token IDs [num_tokens]

    Returns:
        indices: [num_tokens, num_logprobs+1] (采样 token + top-k)
        logprobs: [num_tokens, num_logprobs+1]
        ranks: [num_tokens]
    """
    # 1. 找到 top-k 值
    topk_logprobs, topk_indices = torch.topk(logprobs, num_logprobs, dim=-1)

    # 2. 获取采样 token 的 logprob
    token_ids = token_ids.unsqueeze(-1)
    token_logprobs = logprobs.gather(-1, token_ids)

    # 3. 计算采样 token 的 rank
    token_ranks = batched_count_greater_than(logprobs, token_logprobs)

    # 4. 拼接: [采样 token, top-k tokens]
    indices = torch.cat((token_ids, topk_indices), dim=1)
    logprobs = torch.cat((token_logprobs, topk_logprobs), dim=1)

    return LogprobsTensors(indices, logprobs, token_ranks)
```

## 2.7 特定 Token ID 的 Logprobs 收集 (高效模式)

源码: sampler.py L145-214

当只需要少量特定 token 的 logprobs 时 (如 scoring 任务), 使用此高效方法:

```
def gather_specific_token_logprobs(self, logits, logprob_token_ids, sampled):
    """
    使用 Triton kernel 高效计算特定 token IDs 的 logprobs。

    性能优势:
    - 比 sparse gather 快约 1.4x (batch size > 1)
```

```

- 使用 fused kernel (log_softmax + gather) 减少内存带宽需求
"""
batch_size = logits.shape[0]
max_num_tokens = max(len(tids) for tids in logprob_token_ids.values())

# 创建 padded tensor: [batch_size, max_num_tokens + 1]
# 第一列是采样的 token
token_ids_tensor = torch.zeros(batch_size, max_num_tokens + 1, ...)
token_ids_tensor[:, 0] = sampled

# 使用 fused Triton kernel 计算
logprobs = compute_token_logprobs(logits, token_ids_tensor)

# 计算 rank
sampled_logits = logits.gather(-1, sampled.unsqueeze(-1))
token_ranks = (logits > sampled_logits).sum(dim=-1)

return LogprobsTensors(token_ids_tensor, logprobs, token_ranks)

```

---

### 三、RejectionSampler 拒绝采样

源码文件: rejection\_sampler.py

RejectionSampler 实现 speculative decoding (推测解码) 的核心算法, 严格遵循论文 Fast Inference from Transformers via Speculative Decoding。

#### 3.1 核心概念

源码: rejection\_sampler.py L37-58

```

class RejectionSampler(nn.Module):
    """
    Terminology:
    - accepted tokens: 基于 draft/target 概率关系接受的 token
    - recovered tokens: 基于调整后概率分布采样的 token (被拒绝时)
    - bonus tokens: 所有 draft token 都接受时追加的 bonus token
    - output tokens: 最终输出 = accepted + recovered + bonus tokens
    """

```

#### 3.2 forward() 主流程

源码: rejection\_sampler.py L87-195

```

sequenceDiagram
    participant D as Draft Model
    participant RS as RejectionSampler
    participant T as Target Model
    participant S as Sampler

    D->>RS: draft_token_ids + draft_probs
    T->>RS: target_logits
    RS->>S: bonus_logits → sample bonus_token
    S-->>RS: bonus_token_ids
    RS->>RS: apply_logits_processors to target
    RS->>RS: rejection_sample()

```

```

RS->>RS: collect logprobs
RS-->>RS: SamplerOutput

def forward(self, metadata, draft_probs, logits, sampling_metadata):
    # 1. 采样 Bonus Token
    bonus_logits = logits[metadata.bonus_logits_indices]
    bonus_sampler_output = self.sampler(
        logits=bonus_logits,
        sampling_metadata=replace(sampling_metadata, max_num_logprobs=-1),
        predict_bonus_token=True,
        logprobs_mode_override="processed_logits" if self.is_processed_logprobs_mode \
            else "raw_logits",
    )
    bonus_token_ids = bonus_sampler_output.sampled_token_ids

    # 2. 处理 Target Logits
    raw_target_logits = logits[metadata.target_logits_indices]
    target_logits = raw_target_logits.to(torch.float32)
    target_logits = self.apply_logits_processors(target_logits, sampling_metadata, metadata)
    target_logits = apply_sampling_constraints(target_logits, ...)

    # 3. 执行拒绝采样
    output_token_ids = rejection_sample(
        metadata.draft_token_ids,
        metadata.num_draft_tokens,
        metadata.max_spec_len,
        metadata.cu_num_draft_tokens,
        draft_probs,
        target_logits,
        bonus_token_ids,
        sampling_metadata,
        synthetic_mode=self.synthetic_mode,
        synthetic_conditional_rates=self.synthetic_conditional_rates,
    )

    # 4. 收集 logprobs (如果需要)
    if sampling_metadata.max_num_logprobs is not None:
        logprobs_tensors = self._get_logprobs_tensors(...)

    return SamplerOutput(sampled_token_ids=output_token_ids, logprobs_tensors=logprobs_tensors)

```

### 3.3 rejection\_sample() 核心算法

源码: rejection\_sampler.py L392-503

```

def rejection_sample(
    draft_token_ids,          # [num_tokens]
    num_draft_tokens,        # [batch_size]
    max_spec_len,
    cu_num_draft_tokens,     # [batch_size] cumulative
    draft_probs,             # [num_tokens, vocab_size] or None
    target_logits,           # [num_tokens, vocab_size]
    bonus_token_ids,         # [batch_size, 1]
    sampling_metadata,
    synthetic_mode=False,

```

```

    synthetic_conditional_rates=None,
) -> torch.Tensor:
    """
    Rejection Sampling 核心算法实现。

    Output shape: [batch_size, max_spec_len + 1]
    PLACEHOLDER_TOKEN_ID (-1) 表示被拒绝的位置
    """

    # 创建输出 buffer
    output_token_ids = torch.full(
        (batch_size, max_spec_len + 1),
        PLACEHOLDER_TOKEN_ID, # -1
        dtype=torch.int32,
        device=device,
    )

    # ===== Phase 1: Greedy 采样请求的拒绝采样 =====
    if not sampling_metadata.all_random:
        target_argmax = target_logits.argmax(dim=-1)
        rejection_greedy_sample_kernel[(batch_size,)](
            output_token_ids, cu_num_draft_tokens, draft_token_ids,
            target_argmax, bonus_token_ids, is_greedy, max_spec_len,
            uniform_probs, synthetic_conditional_rates,
            SYNTHETIC_MODE=synthetic_mode,
        )
        if sampling_metadata.all_greedy:
            return output_token_ids

    # ===== Phase 2: 计算 Target 概率分布 =====
    target_probs = target_logits.softmax(dim=-1, dtype=torch.float32)

    # ===== Phase 3: 采样 Recovered Tokens =====
    recovered_token_ids = sample_recovered_tokens(...)

    # ===== Phase 4: Random 采样请求的拒绝采样 =====
    rejection_random_sample_kernel[(batch_size,)](
        output_token_ids, ..., draft_probs, target_probs,
        bonus_token_ids, recovered_token_ids, uniform_probs, ...
    )

    return output_token_ids

```

### 3.4 Triton Kernel 实现细节

`rejection_greedy_sample_kernel` (贪婪模式) 源码: `rejection_sampler.py` L708-757

```

@triton.jit(do_not_specialize=["max_spec_len"])
def rejection_greedy_sample_kernel(
    output_token_ids_ptr,      # [batch_size, max_spec_len + 1]
    cu_num_draft_tokens_ptr,   # [batch_size]
    draft_token_ids_ptr,       # [num_tokens]
    target_argmax_ptr,         # [num_tokens]
    bonus_token_ids_ptr,       # [batch_size]
    is_greedy_ptr,             # [batch_size] or None

```



```

max_spec_len,
...):
req_idx = tl.program_id(0)
is_greedy = True if is_greedy_ptr is None else tl.load(is_greedy_ptr + req_idx)

start_idx = 0 if req_idx == 0 else tl.load(cu_num_draft_tokens_ptr + req_idx - 1)
end_idx = tl.load(cu_num_draft_tokens_ptr + req_idx)
num_draft_tokens = end_idx - start_idx

rejected = False
for pos in range(num_draft_tokens):
    if not rejected:
        draft_token_id = tl.load(draft_token_ids_ptr + start_idx + pos)
        target_argmax_id = tl.load(target_argmax_ptr + start_idx + pos)

        if SYNTHETIC_MODE:
            # Synthetic mode: 基于预设接受率
            uniform_prob = tl.load(uniform_probs_ptr + start_idx + pos)
            rate = tl.load(synthetic_conditional_rates_ptr + pos)
            accepted = uniform_prob < rate
            token_id = draft_token_id if accepted else target_argmax_id
            rejected = not accepted
        else:
            # Standard mode: 比较 draft 和 target.argmax
            token_id = target_argmax_id
            rejected = draft_token_id != target_argmax_id

        tl.store(output_token_ids_ptr + req_idx * (max_spec_len + 1) + pos, token_id)

# 所有 token 都接受时, 追加 bonus token
if not rejected:
    bonus_token_id = tl.load(bonus_token_ids_ptr + req_idx)
    tl.store(output_token_ids_ptr + req_idx * (max_spec_len + 1) + num_draft_tokens,
            bonus_token_id)

```

rejection\_random\_sample\_kernel (随机模式) 源码: rejection\_sampler.py L762-827

```

@triton.jit(do_not_specialize=["max_spec_len"])
def rejection_random_sample_kernel(...):
    """
    Standard Rejection Sampling:
    - 接受条件:  $u \sim \text{Uniform}(0,1)$ ,  $p(x)/q(x) \geq u$ 
      其中  $p=\text{target prob}$ ,  $q=\text{draft prob}$ 
    - 拒绝时: 使用 pre-sampled recovered token
    """
    for pos in range(num_draft_tokens):
        if not rejected:
            draft_token_id = tl.load(draft_token_ids_ptr + start_idx + pos)
            uniform_prob = tl.load(uniform_probs_ptr + start_idx + pos)

            if SYNTHETIC_MODE:
                rate = tl.load(synthetic_conditional_rates_ptr + pos)
                accepted = uniform_prob < rate
            else:

```

```

        if NO_DRAFT_PROBS:
            draft_prob = 1 # ngram mode
        else:
            draft_prob = tl.load(draft_probs_ptr + ...)
            target_prob = tl.load(target_probs_ptr + ...)

        # 核心: 接受条件检查
        accepted = draft_prob > 0 and target_prob / draft_prob >= uniform_prob

    if accepted:
        token_id = draft_token_id
    else:
        rejected = True
        token_id = tl.load(recovered_token_ids_ptr + start_idx + pos)

    tl.store(...)

```

### 3.5 Recovered Token 采样

源码: rejection\_sampler.py L659-703

```

def sample_recovered_tokens(max_spec_len, num_draft_tokens, ...):
    """
    从修正后的概率分布  $q' = \max(0, p - q)$  中采样 recovered tokens。

    使用 Gumbel-Max trick 的变体进行高效采样:
    - 生成指数分布随机变量  $q \sim \text{Exp}(1)$ 
    - 选择  $\text{argmax}(\text{prob} * \text{inv}_q)$  作为 sampled token
    """
    # 为每个 request 生成指数分布变量
    q = torch.empty((batch_size, vocab_size), dtype=torch.float32, device=device)
    q.exponential_()

    # 使用指定 generator 保证可复现性
    for i, generator in sampling_metadata.generators.items():
        if num_draft_tokens[i] > 0:
            q[i].exponential_(generator=generator)

    inv_q = q.reciprocal()

    # Triton kernel: 对每个位置找到最大 score 的 token
    sample_recovered_tokens_kernel[(batch_size, max_spec_len)](
        recovered_token_ids, cu_num_draft_tokens, draft_token_ids,
        draft_probs, target_probs, inv_q, vocab_size, BLOCK_SIZE,
        NO_DRAFT_PROBS=draft_probs is None,
    )
    return recovered_token_ids

```

### 3.6 apply\_sampling\_constraints()

源码: rejection\_sampler.py L506-561

对 target logits 应用 temperature、top\_k、top\_p 约束:

```

def apply_sampling_constraints(logits, cu_num_draft_tokens, sampling_metadata):
    """Process logits with temperature scaling and top_k/top_p filtering."""

```

```

if sampling_metadata.all_greedy:
    return logits

# 将 batch 级参数扩展到 token 级
temperature = expand_batch_to_tokens(sampling_metadata.temperature, ...)
logits.div_(temperature.unsqueeze(-1))

top_k = expand_batch_to_tokens(sampling_metadata.top_k, ...) if needed
top_p = expand_batch_to_tokens(sampling_metadata.top_p, ...) if needed

return apply_top_k_top_p(logits, top_k, top_p)

```

---

## 四、ParallelSampling 并行采样

源码文件: parallel\_sampling.py

ParentRequest 类管理并行采样场景下的父子请求关系，主要用于 beam search 和  $n > 1$  场景。

### 4.1 ParentRequest 数据结构

源码: parallel\_sampling.py L13-51

```

class ParentRequest:
    """Info, state & processing for parallel sampling request.

    Store parent request ID and sampling params.
    Facilitate generating child request sampling params.
    """
    request_id: str # 内部请求 ID
    external_req_id: str # 外部请求 ID (面向客户端)
    sampling_params: SamplingParams # 采样参数

    child_requests: set[str] # 子请求集合 (用于跟踪完成状态)
    output_aggregator: list[CompletionOutput] # 聚合子输出 (FINAL_ONLY 模式)
    max_num_generation_tokens: int # 跨子请求的最大生成长度
    cached_child_sampling_params: SamplingParams | None # 缓存的子参数

```

### 4.2 初始化逻辑

源码: parallel\_sampling.py L36-50

```

def __init__(self, request: EngineCoreRequest) -> None:
    assert request.external_req_id is not None
    sampling_params = request.params

    self.request_id = request.request_id
    self.external_req_id = request.external_req_id
    self.sampling_params = sampling_params

    self.child_requests = set()

    # 根据输出模式初始化聚合器
    self.output_aggregator = (
        [cast(CompletionOutput, None)] * sampling_params.n
        if (sampling_params.output_kind == RequestOutputKind.FINAL_ONLY)
    )

```

```

        else []
    )

    self.max_num_generation_tokens = 0
    self.cached_child_sampling_params = None

```

#### 4.3 子请求参数生成

源码: parallel\_sampling.py L52-81

```

def _get_child_sampling_params(self, index: int) -> SamplingParams:
    """Efficiently obtain child sampling_params.

    Optimization:
    - If seed is None: cache and reuse child params (all children identical)
    - If seed is set: each child gets unique seed (seed + index)
    """
    seed = self.sampling_params.seed
    if self.cached_child_sampling_params:
        return self.cached_child_sampling_params # 复用缓存

    child_sampling_params = copy(self.sampling_params)
    child_sampling_params.n = 1 # 每个子请求只生成 1 个序列

    if seed is None:
        self.cached_child_sampling_params = child_sampling_params # 缓存
    else:
        child_sampling_params.seed = seed + index # 唯一种子

    return child_sampling_params

```

#### 4.4 子请求信息获取

源码: parallel\_sampling.py L83-94

```

def get_child_info(self, index: int) -> tuple[str, SamplingParams]:
    """Get child request ID and sampling params.

    Child request ID format: "{index}_{parent_request_id}"
    Example: "2_req_abc123"
    """
    child_req_id = f"{index}_{self.request_id}"
    self.child_requests.add(child_req_id)
    return child_req_id, self._get_child_sampling_params(index)

```

#### 4.5 输出聚合与完成检测

源码: parallel\_sampling.py L100-126

```

def get_outputs(self, child_request_id, completion_output):
    """
    处理子请求完成的输出。

    Returns:
    (outputs, finished): outputs 列表和是否全部完成
    """

```

```

already_finished_and_returned: bool = False

if completion_output.finished():
    if child_request_id in self.child_requests:
        self.child_requests.remove(child_request_id)
    else:
        already_finished_and_returned = True

if self.sampling_params.output_kind != RequestOutputKind.FINAL_ONLY:
    # Streaming mode: 直接返回当前输出
    outputs = [] if already_finished_and_returned else [completion_output]
else:
    # FINAL_ONLY mode: 聚合所有子输出
    self.output_aggregator[completion_output.index] = completion_output
    outputs = [] if self.child_requests else self.output_aggregator

finished = not self.child_requests # 所有子请求都完成
return outputs, finished

```

#### 4.6 统计信息记录

源码: parallel\_sampling.py L128-150

```

def observe_num_generation_tokens(self, num_generation_tokens: int):
    """Track maximum generation tokens across children."""
    self.max_num_generation_tokens = max(
        num_generation_tokens, self.max_num_generation_tokens
    )
    return self.max_num_generation_tokens

@staticmethod
def observe_finished_request(parent_req, iteration_stats, num_generation_tokens):
    """Record stats when all children finished."""
    n_param = parent_req.n if parent_req is not None else 1

    if parent_req is not None:
        num_generation_tokens = parent_req.observe_num_generation_tokens(num_generation_tokens)

    if parent_req is None or not parent_req.child_requests:
        iteration_stats.max_num_generation_tokens_iter.append(num_generation_tokens)
        iteration_stats.n_params_iter.append(n_param)

```

## 五、Logprobs 计算与追踪

### 5.1 数据结构定义 (v0 层)

源码文件: logprobs.py

**Logprob** 数据类 源码: logprobs.py L12-25

```

@dataclass
class Logprob:
    """Infos for supporting OpenAI compatible logprobs and token ranks.

```

```

Attributes:
    logprob: The logprob of chosen token
    rank: The vocab rank of chosen token (>=1)
    decoded_token: The decoded chosen token string
"""
logprob: float
rank: int | None = None
decoded_token: str | None = None

```

**FlatLogprobs** 高性能数据结构 源码: logprobs.py L31-153

```

@dataclass
class FlatLogprobs(MutableSequence[LogprobsOnePosition | None]):
    """
    Flat logprobs of a request into multiple primitive type lists.

    Performance Advantage:
    - Reduces GC overhead significantly vs list[dict[int, Logprob]]
    - Flattens logprob info into primitive type lists regardless of sequence length
    - Introduces constant amount of objects (O(1) vs O(n*topk))

    Structure:
    - start_indices/end_indices: ranges for each position
    - token_ids: flattened token IDs across all positions
    - logprobs: flattened logprob values
    - ranks: flattened rank information
    - decoded_tokens: flattened decoded strings
    """
    start_indices: list[int] = field(default_factory=list)
    end_indices: list[int] = field(default_factory=list)
    token_ids: list[int] = field(default_factory=list)
    logprobs: list[float] = field(default_factory=list)
    ranks: list[int | None] = field(default_factory=list)
    decoded_tokens: list[str | None] = field(default_factory=list)

```

内存布局示例:

```

Position 0: [sampled_tok, top1, top2] → indices[0:3]
Position 1: [sampled_tok, top1]       → indices[3:5]

```

```

Flattened:
token_ids:    [tok_a, tok_b, tok_c, tok_d, tok_e]
logprobs:     [-0.1, -1.2, -2.3, -0.5, -1.8]
ranks:        [5, 1, 2, 3, 1]
decoded_tokens: ["The", " a", "An", " cat", " dog"]

```

## 5.2 LogprobsProcessor (v1 Engine 层)

源码文件: logprobs.py (v1)

源码: v1/engine/logprobs.py L29-67

```

@dataclass
class LogprobsProcessor:
    tokenizer: TokenizerLike | None
    logprobs: SampleLogprobs | None          # 采样 logprobs
    prompt_logprobs: PromptLogprobs | None   # prompt logprobs

```

```

cumulative_logprob: float | None      # 累积 logprob
num_logprobs: int | None              # top-logprobs 数量
num_prompt_logprobs: int | None      # prompt logprobs 数量

```

初始化 源码: v1/engine/logprobs.py L43-67

```

@classmethod
def from_new_request(cls, tokenizer, request):
    """Create LogprobsProcessor from engine core request."""
    sampling_params = request.sampling_params
    num_logprobs = sampling_params.num_logprobs
    num_prompt_logprobs = sampling_params.prompt_logprobs

    return cls(
        tokenizer=tokenizer,
        cumulative_logprob=(None if num_logprobs is None else 0.0),
        logprobs=(
            None if num_logprobs is None
            else create_sample_logprobs(sampling_params.flat_logprobs)
        ),
        prompt_logprobs=(
            None if num_prompt_logprobs is None
            else create_prompt_logprobs(sampling_params.flat_logprobs)
        ),
        num_prompt_logprobs=num_prompt_logprobs,
        num_logprobs=num_logprobs,
    )

```

采样 **Logprobs** 更新 源码: v1/engine/logprobs.py L69-119

```

def _update_sample_logprobs(self, logprobs_lists):
    """
    Update with sample logprobs from EngineCore.

    Processing steps:
    1. Extract token_ids, logprobs, ranks from GPU tensors
    2. Detokenize token IDs using tokenizer
    3. Verify/correct UTF-8 replacement characters
    4. Update cumulative logprob (sum of sampled token logprobs)
    5. Append to container using append_logprobs_for_next_position()
    """
    token_ids_lst, logprobs_lst, ranks_lst, _ = logprobs_lists

    for rank_np, logprobs_np, token_ids_np in zip(ranks_lst, logprobs_lst, token_ids_lst):
        rank = rank_np.tolist()
        logprobs = logprobs_np.tolist()
        token_ids = token_ids_np.tolist()

        # Detokenize
        if self.tokenizer is not None:
            decoded_tokens_list = convert_ids_list_to_tokens(self.tokenizer, token_ids)
            context_token_ids = self._get_sampled_context_ids(self.logprobs)
            decoded_tokens = self._verify_tokens(decoded_tokens_list, token_ids, context_token_ids)
        else:
            decoded_tokens = NONES

```

```

    # Update cumulative logprob (first entry is always sampled token)
    sampled_token_logprob = logprobs[0]
    self.cumulative_logprob += sampled_token_logprob

    # Append to container
    append_logprobs_for_next_position(
        self.logprobs, token_ids, logprobs, decoded_tokens, rank, self.num_logprobs
    )

```

**Prompt Logprobs 更新** 源码: v1/engine/logprobs.py L121-187

```

def _update_prompt_logprobs(self, prompt_logprobs_tensors):
    """
    Update with prompt logprobs from EngineCore.

    Handles multi-chunk prefills by aggregating over chunks.
    Final output returned via pop_prompt_logprobs().
    """
    token_ids, logprobs, ranks, _ = prompt_logprobs_tensors
    num_prompt_tokens, num_logprobs = logprobs.shape

    # Detokenize all at once for efficiency
    all_decoded_tokens = (
        None if self.tokenizer is None
        else convert_ids_list_to_tokens(self.tokenizer, token_ids.flatten().tolist())
    )

    for pos in range(num_prompt_tokens):
        offset = pos * num_logprobs
        offset_end = offset + num_logprobs

        # Apply UTF-8 correction per position
        decoded_tokens_for_pos = self._verify_tokens(
            decoded_tokens_list=all_decoded_tokens[offset:offset_end],
            tokens=token_ids_list[pos],
            context_token_ids=self._get_sampled_context_ids(self.prompt_logprobs),
        )

        append_logprobs_for_next_position(
            self.prompt_logprobs, token_ids_list[pos],
            prompt_logprobs[pos], decoded_tokens_for_pos,
            prompt_token_ranks[pos], self.num_prompt_logprobs
        )

```

### 5.3 UTF-8 替换字符修正

源码: v1/engine/logprobs.py L249-310

当 byte-fallback tokenization 将多字节 UTF-8 字符拆分到不同 token 时, 单独解码会产生替换字符 U+FFFD ( )。此机制使用上下文 token 重建正确文本:

```

def _correct_decoded_token(self, token_id, context_token_ids):
    """Correct a decoded token containing replacement character.

    Algorithm:

```



```

1. Try decoding with increasing context window (1-4 preceding tokens)
2. Find boundary between clean context and byte-fallback tokens
3. Extract suffix after clean prefix
4. Handle normalization prefix mismatch via longest common prefix
"""
max_ctx = min(len(context_token_ids), 4)

for num_ctx in range(1, max_ctx + 1):
    context = context_token_ids[-num_ctx:]
    full_decoded = self.tokenizer.decode(context + [token_id])

    if full_decoded.endswith(" "):
        continue

    # Find clean/non-byte-fallback boundary
    clean_end = len(context)
    for j in range(len(context) - 1, -1, -1):
        if self.tokenizer.decode([context[j]]).endswith(" "):
            clean_end = j
    else:
        break

    # Extract correct suffix
    clean_prefix = self.tokenizer.decode(context[:clean_end]) if clean_end > 0 else ""
    if full_decoded.startswith(clean_prefix):
        return full_decoded[len(clean_prefix):]

    # Fallback: longest common prefix
    common_len = sum(1 for a, b in zip(clean_prefix, full_decoded) if a == b)
    return full_decoded[common_len:]

return ""

```

## 5.4 Top-Logprobs 实现机制

数据流图:

flowchart LR

```

subgraph GPU["GPU (Sampler)"]
    A[Model Logits] --> B[log_softmax]
    B --> C[topk k=logprobs]
    C --> D[gather sampled token]
    D --> E[LogprobsTensors<br/>indices + logprobs + ranks]
end

subgraph CPU["CPU (LogprobsProcessor)"]
    E --> F[Transfer to CPU]
    F --> G[Detokenize token_ids]
    G --> H[UTF-8 Correction]
    H --> I[Build Logprob objects]
    I --> J[Append to FlatLogprobs/list]
end

subgraph Output["API Response"]
    J --> K[top_logprobs array]
end

```

```

        K --> L[token_logprob<br/>+ top-k alternatives]
    end

    style GPU fill:#e3f2fd,stroke:#1976d2,color:#000
    style CPU fill:#fff3e0,stroke:#f57c00,color:#000
    style Output fill:#c8e6c9,stroke:#388e3c,color:#000

```

OpenAI 兼容格式:

```

{
  "token": "The",
  "logprob": -0.1234,
  "top_logprobs": [
    {"token": " The", "logprob": -0.1234},
    {"token": " An", "logprob": -1.5678},
    {"token": " a", "logprob": -2.3456}
  ]
}

```

## 六、ThinkingBudgetState 推理预算控制

源码文件: thinking\_budget\_state.py

ThinkingBudgetStateHolder 管理 reasoning/thinking 模式下的 token 预算控制，用于限制模型“思考”阶段的 token 数量。

### 6.1 核心功能

源码: thinking\_budget\_state.py L32-33

```

class ThinkingBudgetStateHolder:
    """Tracks thinking sections and forces end tokens when budget is exceeded."""

```

主要职责: 1. 检测 think start/end token 序列 2. 跟踪 thinking token 计数 3. 当预算耗尽时强制生成 end-of-thinking token 4. 处理 speculative decoding 下的预算计算

### 6.2 初始化

源码: thinking\_budget\_state.py L38-83

```

def __init__(self, reasoning_config, max_num_seqs, num_spec_tokens, device, is_pin_memory):
    self.is_enabled = reasoning_config is not None

    if reasoning_config is not None:
        rs = reasoning_config.reasoning_start_token_ids
        re = reasoning_config.reasoning_end_token_ids
        self.think_start_token_ids = rs if rs else []
        self.think_end_token_ids = re if re else []

    self.device = device
    self._state: dict[int, dict[str, Any]] = {} # per-request state
    self.in_spec_mode = num_spec_tokens > 0

    # Pre-allocate GPU tensors for masking and forcing
    if self.num_spec_tokens > 0:
        self.mask = torch.zeros(max_num_reqs * (num_spec_tokens + 1), dtype=torch.bool, device=device)

```

```

        self.force_token_ids = torch.full((max_num_reqs * (num_spec_tokens + 1),), -1, dtype=torch.long)
    else:
        self.mask = torch.zeros(max_num_reqs, dtype=torch.bool, device=device)
        self.force_token_ids = torch.full((max_num_reqs,), -1, dtype=torch.long, device=device)

```

### 6.3 Per-Request 状态初始化

源码: thinking\_budget\_state.py L188-244

```

def _init_state_entry(self, prompt_tok_ids, thinking_token_budget):
    """Initialize per-request thinking budget state.

    State fields:
    - in_think: currently in thinking section
    - in_end: budget exhausted, forcing end tokens
    - check_count_down: remaining budget countdown
    - think_count: total thinking tokens generated
    - force_index: positions where forced tokens should be applied
    - continue_thinking: started in think mode from prompt
    """
    last_start = self._find_last_sequence_index(prompt_tok_ids, self.think_start_token_ids)
    last_end = self._find_last_sequence_index(prompt_tok_ids, self.think_end_token_ids)

    in_think = last_start > last_end

    if in_think:
        think_count = len(prompt_tok_ids) - (last_start + len(self.think_start_token_ids))
        countdown = thinking_token_budget - think_count
        in_end = countdown <= 0 # Already exhausted in prompt?
    else:
        think_count = 0
        countdown = thinking_token_budget

    return {
        "in_think": in_think,
        "in_end": in_end,
        "check_count_down": countdown,
        "think_count": think_count,
        "end_count": 0,
        "force_index": [],
        # ... other fields
    }

```

### 6.4 状态更新逻辑

源码: thinking\_budget\_state.py L125-165

```

def update_state(self, output_token_ids, spec_token_ids, repeat_indices=None):
    """Refresh output/spec from sampling rows and recompute think state."""
    for seq_idx, state in list(self._state.items()):
        # Update output token IDs (handle repeat_indices for spec decode)
        if last_row_for_req is not None:
            state["output_tok_ids"] = output_token_ids[output_row]
        else:
            state["output_tok_ids"] = output_token_ids[seq_idx]

```

```

# Strip spec draft suffix for accurate counting
if len(state["spec_token_ids"]) > 0 and len(state["output_tok_ids"]) >= spec_len:
    state["output_tok_ids"] = state["output_tok_ids"][:-spec_len]

# Recompute think state
self._update_think_state(state)

```

## 6.5 Think State 更新核心逻辑

源码: thinking\_budget\_state.py L246-449

```

def _update_think_state(self, state):
    """
    Complex state machine for tracking thinking sections:

    States:
        found <start>
        Normal                Thinking
        found <end>

        budget exceeded

        Forcing End
        (in_end)

    """
    # Detect start/end transitions
    if state["start_thinking"] == -1:
        start_thinking = self._find_last_sequence_index(state["output_tok_ids"], self.think_start_token)
    if state["end_thinking"] == -1:
        end_thinking = self._find_last_sequence_index(state["output_tok_ids"], self.think_end_token)

    # Handle various transition cases
    if absolute_start_pos >= 0 and absolute_end_pos >= 0:
        if absolute_start_pos > absolute_end_pos:
            # Case: ...<end>...<start>... entering think mode
            new_think_count = current_length - (absolute_start_pos + start_len)
            state["in_think"] = True
            state["think_count"] = new_think_count
        else:
            # Case: ...<start>...<end>... exiting think mode
            state["in_think"] = False
            state["think_count"] = 0

    # Check budget exhaustion
    if state["in_think"]:
        remaining_budget = max(0, state["thinking_token_budget"] - state["think_count"])
        total_thinking_tokens = state["think_count"] + len(state["spec_token_ids"]) + 1

        if total_thinking_tokens > state["thinking_token_budget"]:
            # Transition to forcing mode
            state["in_think"] = False

```

```

state["in_end"] = True
# Calculate force_index based on remaining budget vs spec tokens
if 0 < remaining_budget < spec_len:
    state["force_index"] = [remaining_budget]
elif remaining_budget <= 0:
    state["force_index"] = [0]
else:
    state["force_index"] = [len(state["spec_token_ids"])]

```

## 6.6 Logits 强制应用

源码: thinking\_budget\_state.py L451-527

```

def _apply_forcing_to_logits(self, logits, predict_bonus_token, spec_token_ids_for_layout):
    """Mask and bump logits for forced end-of-thinking tokens.

    When in_end=True and force_index is set:
    1. Mask out all tokens except the required end token
    2. Set end token logit to very high value (1e9) to force selection
    """
    self.mask[:] = False
    has_active_thinking = any(state.get("in_end", False) for state in self._state.values())

    if has_active_thinking:
        active_indices = self.mask.nonzero(as_tuple=False).view(-1)

        if len(active_indices) > 0:
            force_tokens = self.force_token_ids[active_indices]
            logits[active_indices, force_tokens] = 1e9 # Force selection

    return logits

```

状态机可视化:

stateDiagram-v2

```

[*] --> Normal: Request Start
Normal --> Thinking: Detected \<start\> tokens
Thinking --> Normal: Detected \<end\> tokens
Thinking --> ForcingEnd: Budget Exhausted
ForcingEnd --> Normal: All end tokens emitted
ForcingEnd --> Thinking: Rejection sampler rejected end token

note right of Thinking: Counting down budget...
note right of ForcingEnd: Forcing end tokens via logit manipulation

```

## 七、Speculative Decode Metadata 推测解码元数据

源码文件: metadata.py

SpecDecodeMetadata 封装推测解码所需的所有元数据信息, 用于协调 draft model 和 target model 之间的数据流转。

### 7.1 数据结构定义

源码: metadata.py L10-24

```

@dataclass
class SpecDecodeMetadata:
    # Draft tokens (flattened across batch)
    draft_token_ids: torch.Tensor          # [num_tokens] - 所有 draft token IDs
    num_draft_tokens: list[int]            # [batch_size] - 每个 request 的 draft token 数
    cu_num_draft_tokens: torch.Tensor      # [batch_size] - 累积 draft token 数 (用于索引)
    cu_num_sampled_tokens: torch.Tensor    # [batch_size] - 累积采样 token 数

    # Index into combined logits tensor
    target_logits_indices: torch.Tensor    # [num_tokens] - target model logits 索引
    bonus_logits_indices: torch.Tensor     # [batch_size] - bonus token logits 索引
    logits_indices: torch.Tensor          # [num_tokens + batch_size] - 完整 logits 索引

    def __post_init__(self):
        self.max_spec_len = max(self.num_draft_tokens) # 最大推测长度

```

## 7.2 字段说明与用途

| 字段                    | Shape                     | 说明                                    |
|-----------------------|---------------------------|---------------------------------------|
| draft_token_ids       | [num_total_draft_tokens]  | Draft model 生成的所有 token IDs (展平)      |
| num_draft_tokens      | [batch_size]              | 每个 request 的 draft token 数量           |
| cu_num_draft_tokens   | [batch_size]              | 累积数量, 用于将 batch 维度映射到展平维度             |
| cu_num_sampled_tokens | [batch_size]              | 累积采样数量 (含 bonus token)                |
| target_logits_indices | [num_tokens]              | 在组合 logits tensor 中的 target logits 位置 |
| bonus_logits_indices  | [batch_size]              | Bonus token 的 logits 位置               |
| logits_indices        | [num_tokens + batch_size] | 完整索引映射                                |
| max_spec_len          | scalar                    | batch 中最大的推测长度                        |

## 7.3 Dummy Metadata 工厂方法

源码: metadata.py L30-66

```

@classmethod
def make_dummy(cls, draft_token_ids: list[list[int]], device) -> "SpecDecodeMetadata":
    """
    Create dummy metadata for testing/debugging.

    Example:
        metadata = SpecDecodeMetadata.make_dummy(
            draft_token_ids=[[1, 2, 3], [4, 5]],
            device="cuda"
        )
        # batch_size=2, num_draft_tokens=[3, 2], max_spec_len=3
    """
    batch_size = len(draft_token_ids)
    num_draft_tokens = [len(ids) for ids in draft_token_ids]
    num_sampled_tokens = [len(ids) + 1 for ids in draft_token_ids] # +1 for bonus
    flattened_draft_token_ids = sum(draft_token_ids, [])
    num_tokens = len(flattened_draft_token_ids)

```

```

# Build cumulative sums for indexing
cu_num_draft_tokens = np.cumsum(num_draft_tokens, dtype=np.int32)
cu_num_sampled_tokens = np.cumsum(num_sampled_tokens, dtype=np.int32)

# Initialize index tensors (zeros for dummy)
target_logits_indices = torch.zeros(num_tokens, dtype=torch.int32, device=device)
bonus_logits_indices = torch.zeros(batch_size, dtype=torch.int32, device=device)
logits_indices = torch.zeros(num_tokens + batch_size, dtype=torch.int32, device=device)

return cls(...)

```

## 7.4 在 RejectionSampler 中的使用

数据流向:

flowchart TD

```

subgraph Input[" 输入"]
    A[draft_token_ids]
    B[num_draft_tokens]
    C[target_logits]
    D[draft_probs]
end

subgraph Metadata["SpecDecodeMetadata"]
    E[cu_num_draft_tokens]
    F[target_logits_indices]
    G[bonus_logits_indices]
end

subgraph Processing["RejectionSampler"]
    H[Bonus Token Sampling]
    I[Target Logits Processing]
    J[Rejection Sampling Kernel]
end

A --> E
B --> E
C --> F
F --> I
G --> H
E --> J
D --> J
I --> J
H --> J

```

```

style Input fill:#e3f2fd,stroke:#1976d2,color:#000
style Metadata fill:#fff3e0,stroke:#f57c00,color:#000
style Processing fill:#c8e6c9,stroke:#388e3c,color:#000

```

---

## 八、完整采样数据流图

### 8.1 从 Logits 到 Token 的端到端流程

flowchart TB

```
subgraph Model[" 模型输出"]
    L[Raw Logits<br/>shape: batch_size × vocab_size]
end

subgraph Preprocess[" 预处理阶段"]
    P1[Float32 转换]
    P2[Allowed Token IDs 白名单]
    P3[Bad Words 过滤]
    P4[Non-Argmax Processors<br/>MinTokens / LogitBias]
    P5[Penalties 应用 <br/>Repetition / Frequency / Presence]
    P6[Thinking Budget State]
end

subgraph Sampling[" 采样决策"]
    S1{All Greedy?}
    S2[Argmax 采样]
    S3[Temperature 缩放]
    S4[Argmax-Invariant Processors<br/>min_p]
    S5[Top-K 过滤]
    S6[Top-P 过滤]
    S7[Random Sampling<br/>Multinomial/Categorical]
    S8[Mixed Selection<br/>torch.where]
end

subgraph Postprocess[" 后处理"]
    R1[Compute Logprobs<br/>log_softmax]
    R2[Gather Top-K Logprobs]
    R3[Rank Calculation]
    R4[Specific Token Logprobs<br/>if logprob_token_ids]
end

subgraph Output[" 最终输出"]
    O1[Sampled Token IDs<br/>batch_size × 1]
    O2[LogprobsTensors<br/>indices + logprobs + ranks]
end

L --> P1 --> P2 --> P3 --> P4 --> P5 --> P6
P6 --> S1
S1 -->|Yes| S2 --> O1
S1 -->|No| S3 --> S4 --> S5 --> S6 --> S7
S7 --> S8
S2 --> R1
S8 --> R1
R1 --> R2 --> R3 --> R4 --> O2

style Model fill:#ffebee,stroke:#c62828,color:#000
style Preprocess fill:#e3f2fd,stroke:#1976d2,color:#000
style Sampling fill:#fff3e0,stroke:#f57c00,color:#000
style Postprocess fill:#f3e5f5,stroke:#7b1fa2,color:#000
style Output fill:#e8f5e9,stroke:#388e3c,color:#000
```



## 8.2 Speculative Decoding 采样流程

sequenceDiagram

```
participant DM as Draft Model
participant SM as SpecDecodeMetadata
participant RS as RejectionSampler
participant SP as Sampler
participant TM as Target Model

DM->>SM: Generate draft tokens
SM->>SM: Build metadata<br/>(indices, counts)
SM->>RS: forward(metadata, draft_probs, target_logits)

RS->>SP: Sample bonus token<br/>(supports top_k/top_p!)
SP-->>RS: bonus_token_ids

RS->>RS: Process target logits<br/>(processors, constraints)
RS->>RS: rejection_sample()<br/>Triton kernels

alt Standard Mode
    RS->>RS: For each position:<br/>accept if p/q >= u<br/>else use recovered token
else Synthetic Mode
    RS->>RS: Accept based on<br/>pre-computed rates
end

RS->>RS: Gather logprobs for accepted tokens
RS-->>RS: SamplerOutput<br/>(accepted + recovered + bonus)
```

## 8.3 关键数值常量

| 常量                                 | 值                 | 定义位置                                  | 用途                                  |
|------------------------------------|-------------------|---------------------------------------|-------------------------------------|
| <code>_SAMPLING_EPS</code>         | <code>1e-5</code> | <code>sampling_params.py</code> L25   | 温度阈值，低于此值为贪婪采样                      |
| <code>_MAX_TEMP</code>             | <code>1e-2</code> | <code>sampling_params.py</code> L26   | 最低有效温度警告阈值                          |
| <code>MAX_LOGPROB_TOKEN_IDS</code> | <code>128</code>  | <code>sampling_params.py</code> L28   | <code>logprob_token_ids</code> 最大长度 |
| <code>PLACEHOLDER_TOKEN_ID</code>  | <code>-1</code>   | <code>rejection_sampler.py</code> L30 | 被拒绝 token 的占位符                      |
| <code>GREEDY_TEMPERATURE</code>    | <code>0</code>    | <code>rejection_sampler.py</code> L31 | 贪婪采样温度值                             |
| <code>MAX_SPEC_LEN</code>          | <code>128</code>  | <code>rejection_sampler.py</code> L34 | 单步最大推测 token 数                      |

## 九、性能优化要点

### 9.1 内存优化

1. **In-place Operations:** Temperature 使用 `div_()` 避免 tensor 复制
2. **Tensor Reuse:** `torch.where(out=greedy_sampled)` 复用输出 buffer
3. **FlatLogprobs:** 扁平化数据结构减少 GC 开销 ( $O(1)$  vs  $O(n \times k)$  对象数)
4. **Int32 存储:** 采样 token IDs 使用 `int32` 减少 GPU→CPU 传输量

### 9.2 计算优化

1. **Fused Triton Kernels:** 拒绝采样使用自定义 Triton kernel 避免 Python 循环开销

- 2. **Gumbel-Max Trick**: Recovered token 采样使用指数分布变量优化
- 3. **Early Exit**: 贪婪采样请求跳过随机采样路径
- 4. **Batched Operations**: Penalties 和 processors 支持批量计算

9.3 缓存优化

- 1. **Child Params Cache**: ParallelSampling 缓存无 seed 场景的子参数
- 2. **Skip Clone**: skip\_clone=True 时使用浅拷贝
- 3. **Prefix Cache**: skip\_reading\_prefix\_cache 优化 prompt logprobs 的 cache 读取

十、总结

vLLM 的采样系统是一个高度优化的多层架构:

| 层级   | 组件                  | 职责                        |
|------|---------------------|---------------------------|
| 配置层  | SamplingParams      | 用户参数定义与验证                 |
| 处理层  | LogitsProcessors    | Logits 变换链（白名单、惩罚、约束等）    |
| 采样层  | Sampler             | 核心采样逻辑（贪婪/随机/混合）          |
| 加速层  | RejectionSampler    | Speculative decoding 拒绝采样 |
| 并行层  | ParentRequest       | 多序列并行采样管理                 |
| 追踪层  | LogprobsProcessor   | Token 概率追踪与 API 格式化       |
| 控制层  | ThinkingBudgetState | Reasoning 模式预算控制          |
| 元数据层 | SpecDecodeMetadata  | 推测解码协调信息                  |

设计亮点: - 模块化 **Processor** 链: 清晰的 argmax-invariant / non-argmax-invariant 分类 - 高性能 **Triton Kernels**: GPU 原生实现的拒绝采样算法 - 灵活的混合模式: 同一 batch 内支持贪婪 + 随机混合采样 - 内存感知优化: In-place 操作、tensor 复用、扁平化数据结构 - **OpenAI** 兼容: 完整支持 logprobs、token ranks、UTF-8 修正等特性

vLLM 多模态处理管线分析

定位: 本文档深入剖析 vLLM 的多模态 (Multimodal) 输入处理全流程, 涵盖从原始媒体数据加载、解析、编码到最终注入 LLM 的完整管线。

```
graph TB
    subgraph Input[" 多模态输入"]
        A[Image URL / Base64 / File]
        B[Audio URL / Base64 / File]
        C[Video URL / Base64 / File]
    end

    subgraph Connector[" Media Connectors"]
        D[MediaConnector<br/> 统一加载接口]
        E[ImageMediaIO]
        F[AudioMediaIO]
        G[VideoMediaIO]
    end

    subgraph Parse[" 输入解析"]
        H[MultiModalDataParser<br/> 模态识别与归一化]
```

```

end

subgraph Processor[" Processor 管线"]
    I[BaseMultiModalProcessor.apply]
    J[HF Processor 调用]
    K[Prompt Update<br/> 占位符注入]
    L[Placeholder 定位]
end

subgraph Encoder[" 特征提取"]
    M[CLIP / SigLIP 视觉编码器]
    N[Whisper 音频编码器]
    O[Encoder Budget 管理]
end

subgraph Output[" 输出"]
    P[prompt_token_ids + mm_kwargs<br/>+ mm_placeholders]
end

A --> D
B --> D
C --> D
D --> E & F & G
E & F & G --> H
H --> I
I --> J
J --> K
K --> L
L --> M & N
M & N --> O
O --> P

style Input fill:#f1f8e9,stroke:#7cb342
style Connector fill:#e3f2fd,stroke:#1976d2
style Parse fill:#fff3e0,stroke:#f57c00
style Processor fill:#f3e5f5,stroke:#7b1fa2
style Encoder fill:#fce4ec,stroke:#c62828
style Output fill:#e0f2f1,stroke:#00796b

```

---

## 目录

- 一、MultiModalRegistry 多模态处理器注册表
  - 二、媒体连接器 Media Connectors
  - 三、处理器 Processors
  - 四、输入解析
  - 五、特征提取管线
  - 六、支持的多模态模型
  - 七、Assets 系统
  - 八、端到端数据流
-

## 一、MultiModalRegistry 多模态处理器注册表

源码位置: `registry.py`

`MultiModalRegistry` 是整个多模态子系统的中央调度枢纽, 负责:

1. 将模型类与对应的多模态处理器工厂绑定
2. 根据模型配置动态创建处理器实例
3. 管理多模态缓存策略
4. 提供 dummy inputs 生成能力 (用于 profiling)

### 1.1 注册机制

采用装饰器模式, 通过 `register_processor()` 方法将处理器工厂绑定到模型类:

```
# registry.py:142-174
def register_processor(
    self,
    processor: MultiModalProcessorFactory[_I],
    *,
    info: ProcessingInfoFactory[_I],
    dummy_inputs: DummyInputsBuilderFactory[_I],
):
    """Register a multi-modal processor to a model class."""

    def wrapper(model_cls: N) -> N:
        if "_processor_factory" in model_cls.__dict__:
            logger.warning(
                "Model class %s already has a multi-modal processor "
                "registered to %s. It is overwritten by the new one.",
                model_cls,
                self,
            )

        model_cls._processor_factory = _ProcessorFactories(
            info=info,
            dummy_inputs=dummy_inputs,
            processor=processor,
        )

        return model_cls

    return wrapper
```

核心数据结构 `_ProcessorFactories` (第 82-95 行) 封装了三个工厂函数:

| 工厂                        | 类型                                      | 职责                                                    |
|---------------------------|-----------------------------------------|-------------------------------------------------------|
| <code>info</code>         | <code>ProcessingInfoFactory</code>      | 创建 <code>BaseProcessingInfo</code> , 提供模型配置信息         |
| <code>dummy_inputs</code> | <code>DummyInputsBuilderFactory</code>  | 创建 <code>BaseDummyInputsBuilder</code> , 用于 profiling |
| <code>processor</code>    | <code>MultiModalProcessorFactory</code> | 创建 <code>BaseMultiModalProcessor</code> , 核心处理逻辑      |

## 1.2 处理器创建流程

```
# registry.py:211-230
def create_processor(
    self,
    model_config: "ModelConfig",
    *,
    tokenizer: TokenizerLike | None = None,
    cache: BaseMultiModalProcessorCache | None = None,
) -> BaseMultiModalProcessor[BaseProcessingInfo]:
    """Create a multi-modal processor for a specific model and tokenizer."""
    if not model_config.is_multimodal_model:
        model_name = model_config.served_model_name or model_config.model
        raise ValueError(f"{model_name} is not a multimodal model")

    model_cls = self._get_model_cls(model_config)          # 获取模型架构类
    factories = model_cls._processor_factory                # 获取绑定的工厂

    ctx = self._create_processing_ctx(model_config, tokenizer) # 构建上下文

    return factories.build_processor(ctx, cache=cache)      # 构建处理器
```

调用链: `create_processor()` → `_get_model_cls()` → 获取 `_processor_factory` → `build_processor()`

## 1.3 模态支持检查

```
# registry.py:103-140
def supports_multimodal_inputs(self, model_config: "ModelConfig") -> bool:
    """
    Checks if the model supports multimodal inputs.
    Returns True if the model is multimodal with any non-zero supported modalities.
    """
    if not model_config.is_multimodal_model:
        return False

    mm_config = model_config.get_multimodal_config()
    try:
        info = self._create_processing_info(model_config, tokenizer=None)
    except ValueError:
        # 无已注册的 processor, 回退到纯文本模式
        logger.warning_once(...)
        return False

    # 检查所有支持的模态 limit 是否都为 0
    if all(
        mm_config.get_limit_per_prompt(modality) == 0
        for modality in info.supported_mm_limits
    ):
        if mm_config.enable_mm_embeds:
            return True # 预计算嵌入模式仍需 MM 基础设施
        return False

    return True
```

## 1.4 缓存管理

Registry 提供四种缓存策略（第 268-347 行）：

| 缓存类型           | 场景               | 说明            |
|----------------|------------------|---------------|
| None           | 禁用缓存             | 不使用任何缓存       |
| processor_only | 单进程 / DP + 外部 LB | 仅在 API 进程内缓存  |
| lru            | IPC 支持           | 基于 LRU 的跨进程缓存 |
| shm            | IPC 支持           | 基于共享内存的高性能缓存  |

## 1.5 Dummy Inputs 生成

用于启动时的内存 profiling：

```
# registry.py:232-266
def get_dummy_mm_inputs(
    self,
    model_config: "ModelConfig",
    mm_counts: Mapping[str, int],
    *,
    cache: BaseMultiModalProcessorCache | None = None,
    processor: BaseMultiModalProcessor | None = None,
) -> MultiModalInput:
    """Create dummy data for profiling the memory usage of a model."""
    seq_len = model_config.max_model_len

    if processor is None:
        processor = self.create_processor(model_config, cache=cache)

    # 生成 dummy processor inputs 并应用处理
    processor_inputs = processor.dummy_inputs.get_dummy_processor_inputs(...)
    mm_inputs = processor.apply(processor_inputs, timing_ctx=TimingContext(enabled=False))

    # 补齐到 max_model_len
    prompt_token_ids = mm_inputs["prompt_token_ids"]
    total_len = len(prompt_token_ids)
    if total_len < seq_len:
        prompt_token_ids.extend([0] * (seq_len - total_len))

    return mm_inputs
```

## 二、媒体连接器 Media Connectors

源码目录: vllm/multimodal/media/

Media Connectors 负责从多种来源加载原始媒体数据（HTTP URL、data URL、本地文件），并将其转换为统一的内部表示。

### 2.1 基类 MediaIO

源码位置: base.py

```
# base.py:46-83
class MediaIO(ABC, Generic[_T]):
```

```
"""Configuration values can be user-provided either by --media-io-kwarg
or by the runtime API field "media_io_kwarg"."""
```

```
@classmethod
def merge_kwarg(cls, default_kwarg, runtime_kwarg):
    """Merge config-level kwarg and request-level kwarg."""
    merged = dict(default_kwarg or {})
    if runtime_kwarg:
        merged.update(runtime_kwarg)
    return merged

@abstractmethod
def load_bytes(self, data: bytes) -> _T: ...

@abstractmethod
def load_base64(self, media_type: str, data: str) -> _T: ...

@abstractmethod
def load_file(self, filepath: Path) -> _T: ...
```

关键辅助类 **MediaWithBytes** (第 14-43 行): 将媒体对象与其原始字节绑定, 确保缓存一致性。

## 2.2 Image Connector — 图像加载与预处理

源码位置: `image.py`

```
# image.py:20-98
class ImageMediaIO(MediaIO[Image.Image]):
    def __init__(self, image_mode: str = "RGB", **kwarg) -> None:
        self.image_mode = image_mode
        self.kwarg = kwarg
        # RGBA -> RGB 转换的背景色配置
        rgba_bg = kwarg.get("rgba_background_color", (255, 255, 255))
        self.rgba_background_color = rgba_bg

    def load_bytes(self, data: bytes) -> MediaWithBytes[Image.Image]:
        image = Image.open(BytesIO(data))
        image.load()
        image = self._convert_image_mode(image) # 自动转换色彩空间
        return MediaWithBytes(image, data)

    def load_base64(self, media_type: str, data: str) -> MediaWithBytes[Image.Image]:
        return self.load_bytes(pybase64.b64decode(data, validate=True))
```

特性: - 自动色彩空间转换 (RGBA→RGB) - 支持自定义背景色 - 返回 **MediaWithBytes** 包装以保留原始字节

**ImageEmbeddingMediaIO** (第 101-139 行): 专门处理预计算的图像嵌入, 支持 `numpy (.npy)` 和 `PyTorch` 序列化格式。

## 2.3 Audio Connector — 音频处理

源码位置: `audio.py`

音频加载采用双后端降级策略:

```
# audio.py:132-160
def load_audio(path, *, sr=None, mono=True):
    try:
```

```

        return load_audio_soundfile(path, sr=sr, mono=mono)    # 优先: soundfile
    except ImportError as exc:
        logger.error("Failed to load audio via soundfile: %r", exc)
    except soundfile.LibsndfileError as exc:
        if exc.code not in _BAD_SF_CODES:
            raise # 已知格式检测失败才降级
# 降级到 PyAV (FFmpeg)
    try:
        return load_audio_pyav(path, sr=sr, mono=mono)
    except ImportError:
        raise # 让 PlaceholderModule 的消息传播

```

两种加载实现对比:

| 后端        | 函数                     | 特点                |
|-----------|------------------------|-------------------|
| soundfile | load_audio_soundfile() | 高精度, 原生采样率        |
| PyAV      | load_audio_pyav()      | FFmpeg 后端, 广泛格式支持 |

PyAV 实现细节 (audio.py:45-109): - 自动重采样 (av.AudioResampler) - 多声道 → 单声道平均 - 流式解码避免内存爆炸

2.4 Video Connector — 视频帧提取

源码位置: video.py

```

# video.py:19-147
class VideoMediaIO(MediaIO[tuple[npt.NDArray, dict[str, Any]]]):
    def __init__(self, image_io, num_frames=32, **kwargs):
        self.image_io = image_io
        self.num_frames = num_frames
        # 支持运行时覆盖视频后端
        video_loader_backend = (
            kwargs.pop("video_backend", None) or envs.VLLM_VIDEO_LOADER_BACKEND
        )
        self.video_loader = VIDEO_LOADER_REGISTRY.load(video_loader_backend)

```

特殊功能—JPEG 序列支持 (video.py:74-138):

当 media\_type 为 video/jpeg 时, 将 base64 编码的 JPEG 帧序列拼接为视频:

```

if media_type.lower() == "video/jpeg":
    frame_parts = data.split(",", self.num_frames)[:self.num_frames]
    frames = np.stack([
        np.asarray(load_frame(frame_data)) for frame_data in frame_parts
    ])
    metadata = {
        "total_num_frames": total_num_frames,
        "fps": fps,
        "duration": duration,
        "video_backend": "jpeg_sequence",
        "frames_indices": frames_indices,
        "do_sample_frames": ...,
    }
    return frames, metadata

```



## 2.5 统一接口 MediaConnector

源码位置: `connector.py`

`MediaConnector` 是面向用户的统一入口, 协调各模式 IO 并提供:

URL 加载路由

```
graph LR
    A[load_from_url] --> B{URL Scheme?}
    B -->|http/https| C[HTTP Download + Cache]
    B -->|data| D[Data URL 解析]
    B -->|file| E[Local File Load]

    C --> F{Cached?}
    F -->|Yes| G[Return Cached Bytes]
    F -->|No| H[Download + Cache]

    style A fill:#e3f2fd,stroke:#1976d2
    style B fill:#fff3e0,stroke:#f57c00
```

*# connector.py:286-319*

```
def load_from_url(self, url, media_io, *, fetch_timeout=None):
    url_spec = parse_url(url)

    if url_spec.scheme and url_spec.scheme.startswith("http"):
        self._assert_url_in_allowed_media_domains(url_spec)
        cached = self._get_cached_bytes(url)
        if cached is not None:
            return media_io.load_bytes(cached)
        data = connection.get_bytes(url_spec.url, timeout=fetch_timeout, ...)
        self._put_cached_bytes(url, data)
        return media_io.load_bytes(data)

    if url_spec.scheme == "data":
        return self._load_data_url(url_spec, media_io)
    if url_spec.scheme == "file":
        return self._load_file_url(url_spec, media_io)
```

媒体下载缓存 `connector.py:127-229` 实现了带 TTL 和大小限制的 LRU 缓存:

- 环境变量控制: `VLLM_MEDIA_CACHE`, `VLLM_MEDIA_CACHE_MAX_SIZE_MB`, `VLLM_MEDIA_CACHE_TTL_HOURS`
- 淘汰策略: 先淘汰过期条目, 再按 LRU 淘汰直到低于大小限制
- 原子写入: 通过临时文件 + `rename` 保证安全性

异步支持

```
# connector.py:321-368
async def load_from_url_async(self, url, media_io, *, fetch_timeout=None):
    """Asynchronously load media from a URL."""
    loop = asyncio.get_running_loop()
    # 使用 ThreadPoolExecutor 执行阻塞 IO
    cached = await loop.run_in_executor(global_thread_pool, self._get_cached_bytes, url)
    if cached is not None:
        future = loop.run_in_executor(global_thread_pool, media_io.load_bytes, cached)
        return await future
    # ... HTTP async download
```

模态映射表

```
# connector.py:43-47
MODALITY_IO_MAP: dict[str, type[MediaIO]] = {
    "audio": AudioMediaIO,
    "image": ImageMediaIO,
    "video": VideoMediaIO,
}
```

---

### 三、处理器 Processors

源码目录: vllm/multimodal/processing/

Processor 是多模态处理的核心引擎，负责将原始媒体数据转换为模型可消费的张量表示。

#### 3.1 处理上下文 InputProcessingContext

源码位置: context.py

```
# context.py:89-109
@dataclass(frozen=True)
class InputProcessingContext:
    """Contains information about the model which may be used to modify the inputs."""
    model_config: ModelConfig
    tokenizer: TokenizerLike | None

    def get_hf_config(self, typ=None) -> PretrainedConfig: ...
    def get_hf_processor(self, typ=None, **kwargs) -> ProcessorMixin: ...
    def call_hf_processor(self, hf_processor, data, kwargs={}) -> BatchFeature: ...
```

关键职责: - 封装模型配置和 tokenizer - 提供类型安全的 HF config / processor 访问 - 统一调用 HF processor 并处理后处理 (dtype 转换等)

#### 3.2 BaseProcessingInfo 一模型特定信息

源码位置: context.py:295-488

```
class BaseProcessingInfo:
    def __init__(self, ctx: InputProcessingContext):
        self.ctx = ctx

    @property
    def model_id(self) -> str:
        return self.ctx.model_config.model

    @abstractmethod
    def get_supported_mm_limits(self) -> Mapping[str, int | None]:
        """ 返回每个模态支持的最大数量, None 表示无限制 """
        raise NotImplementedError

    @cached_property
    def allowed_mm_limits(self) -> Mapping[str, int]:
        """ 用户配置限制与模型固有限制的较小值 """
        ...

    def validate_num_items(self, modality: str, num_items: int) -> None:
```

```

    """ 验证输入项数是否超限 """
    ...

def parse_mm_data(self, mm_data, *, validate=True) -> MultiModalDataItems:
    """ 解析并验证多模态数据字典 """
    ...

```

### 3.3 BaseMultiModalProcessor 一核心处理基类

源码位置: processor.py:972-1708

这是最关键的类，定义了标准的多模态处理流水线。

整体处理流程

```

sequenceDiagram
    participant User
    participant Processor as BaseMultiModalProcessor
    participant HF as HF Processor
    participant Cache as Processor Cache

    User->>Processor: apply(ProcessorInputs)
    Processor->>Processor: _cached_apply_hf_processor()
    alt Cache Hit
        Cache-->>Processor: cached mm_kwargs + updates
    else Cache Miss
        Processor->>HF: _call_hf_processor(prompt + mm_data)
        HF-->>Processor: input_ids + processed tensors
        Processor->>Processor: _get_mm_fields_config()
        Processor->>Processor: _get_mm_hashes()
        Processor->>Processor: _get_mm_prompt_updates()
    end
    Processor->>Processor: _maybe_apply_prompt_updates()
    Processor->>Processor: _find_mm_placeholders()
    Processor-->>User: MultiModalInput

```

#### apply() 主方法详解

```

# processor.py:1663-1707
def apply(self, inputs: ProcessorInputs, timing_ctx: TimingContext) -> MultiModalInput:
    """
    Process multi-modal inputs to be used in vLLM.

    Main steps:
    1. Apply HF Processor on prompt text and multi-modal data together
    2. Find and update sequences with placeholder tokens
    3. Extract placeholder token information from processed token IDs
    """
    # Step 1: 应用 HF Processor (带缓存)
    prompt_ids, mm_info, is_update_applied = \
        self._cached_apply_hf_processor(inputs, timing_ctx)

    # Step 2: 应用 Prompt Updates (占位符替换/插入)
    prompt_ids, mm_placeholders = self._maybe_apply_prompt_updates(
        mm_items=inputs.mm_data_items,

```

```

        prompt_ids=prompt_ids,
        mm_kwargs=mm_info.kwargs,
        mm_prompt_updates=mm_info.prompt_updates,
        is_update_applied=is_update_applied,
    )

    # Step 3: 构建 PlaceholderRange
    mm_placeholder_ranges = {
        modality: [item.to_range() for item in placeholders]
        for modality, placeholders in mm_placeholders.items()
    }

    return mm_input(
        prompt_token_ids=prompt_ids,
        mm_kwargs=mm_info.kwargs,
        mm_hashes=mm_info.hashes,
        mm_placeholders=mm_placeholder_ranges,
    )

```

### 3.4 Prompt Update 机制

Prompt Update 是 vLLM 多模态系统的核心创新之一，用于在 token 序列中标记多模态特征的位置。

两种更新模式

```

# processor.py:292-295
class UpdateMode(str, Enum):
    INSERT = "insert" # 在目标位置之后插入占位符
    REPLACE = "replace" # 替换目标内容为占位符

```

#### PromptInsertion 一插入模式

```

# processor.py:353-419
@dataclass
class PromptInsertion(PromptUpdate):
    """Defines how to insert placeholder tokens into a prompt."""

    insertion: PromptUpdateContent # 要插入的内容

    # 示例: 在 <s> token 后插入 N 个 <image> 占位符
    # PromptInsertion(
    #     modality="image",
    #     target="<s>",
    #     insertion="<image>" * image_feature_size,
    # )

```

#### PromptReplacement 一替换模式

```

# processor.py:422-496
@dataclass
class PromptReplacement(PromptUpdate):
    """Defines how to replace portions of an input prompt with placeholder tokens."""

    replacement: PromptUpdateContent # 替换后的内容

```

```

# 示例: 将每个 <image> 替换为 feature_size 个 <image> token
# PromptReplacement(
#     modality="image",
#     target="<image>",
#     replacement="<image>" * image_feature_size,
# )

```

### PromptIndex 一位置定位

```

# processor.py:141-186
class PromptIndexTargets:
    @staticmethod
    def start() -> PromptIndex:
        """ 解析为 prompt 开头 (第一个 token 之前) """

    @staticmethod
    def prefix(seq: PromptSeq) -> PromptIndex:
        """ 解析为给定前缀之后的位置 """

    @staticmethod
    def end() -> PromptIndex:
        """ 解析为 prompt 末尾 (最后一个 token 之后) """

```

### PromptUpdateDetails 一细粒度控制

```

# processor.py:205-269
@dataclass
class PromptUpdateDetails(Generic[_S]):
    full: _S # 完整内容
    is_embed: Callable | None # 可选: 选择哪些位置需要 embedding

    @staticmethod
    def select_text(seq, embed_text: str):
        """ 仅对匹配 embed_text 的位置分配 embedding """

    @staticmethod
    def select_token_id(seq, embed_token_id: int):
        """ 仅对指定 token ID 的位置分配 embedding """

```

## 3.5 HF Processor 集成

vLLM 的 Processor 层封装了 HuggingFace ProcessorMixin, 提供多种调用模式:

| 方法                                             | 场景           | 说明                   |
|------------------------------------------------|--------------|----------------------|
| <code>_apply_hf_processor_text_mm()</code>     | 文本 + 多模态联合处理 | 标准 HF processor 调用   |
| <code>_apply_hf_processor_text_only()</code>   | 仅文本          | 生成 dummy 多模态数据配合     |
| <code>_apply_hf_processor_tokens_only()</code> | token IDs    | 大多数 HF processor 不支持 |
| <code>_apply_hf_processor_mm_only()</code>     | 仅多模态数据       | 生成 dummy 文本配合        |

## 3.6 缓存感知处理

processor.py:1441-1510 实现了缓存感知的 HF Processor 调用:

```

def _cached_apply_hf_processor(self, inputs, timing_ctx):
    cache = self.cache

```

```

if cache is None or passthrough_data:
    return self._apply_hf_processor(inputs, timing_ctx) # 无缓存时回退

# 计算哈希
mm_hashes = inputs.get_mm_hashes(self.info.model_id)

# 筛选未缓存的项
mm_is_cached, mm_missing_data_items = self._get_cache_missing_items(
    cache, inputs.mm_data_items, mm_hashes
)

# 仅对缺失项调用 HF Processor (不应用 prompt update)
prompt_ids, mm_missing_processed_data, _ = self._apply_hf_processor_main(
    prompt=inputs.prompt,
    mm_items=mm_missing_data_items,
    enable_hf_prompt_update=False, # 关键: 延迟到合并后
)

# 合并缓存结果与新鲜计算结果
mm_kwargs, mm_prompt_updates = self._merge_mm_kwargs(
    cache, mm_hashes, mm_is_cached,
    mm_missing_kwargs, mm_missing_prompt_updates
)

```

### 3.7 Dummy Inputs Builder

源码位置: `dummy_inputs.py`

用于生成 profiling 所需的虚拟输入数据:

```

# dummy_inputs.py:28-92
class BaseDummyInputsBuilder(ABC, Generic[_I]):
    @abstractmethod
    def get_dummy_text(self, mm_counts) -> str:
        """ 构建对应 mm_counts 的文本输入 """
        raise NotImplementedError

    @abstractmethod
    def get_dummy_mm_data(self, seq_len, mm_counts, mm_options):
        """ 构建最大可能占位符数量的多模态输入 """
        raise NotImplementedError

    def get_dummy_processor_inputs(self, seq_len, mm_counts, mm_options):
        """ 组合文本和多模态数据为完整 ProcessorInputs """
        ...

```

内置的辅助方法 (`dummy_inputs.py:94-187`):

- `_get_dummy_audios()`: 生成零填充的音频波形
- `_get_dummy_images()`: 生成白色 PIL Image
- `_get_dummy_videos()`: 生成白色帧序列

### 3.8 ProcessorInputs — 输入数据结构

源码位置: `inputs.py`

```

# inputs.py:12-71
@dataclass
class ProcessorInputs:
    """Represents the keyword arguments to BaseMultiModalProcessor.apply()"""

    prompt: str | list[int]                # 输入 prompt
    mm_data_items: MultiModalDataItems      # 解析后的多模态数据
    mm_uuid_items: MultiModalUUIDItems | None # 用户提供的 UUID (可选)
    hf_processor_mm_kwargs: Mapping         # HF processor 额外参数
    tokenization_kwargs: Mapping           # 分词参数

    def get_mm_hashes(self, model_id: str) -> MultiModalHashes:
        """ 计算每个多模态项的内容哈希, 用于缓存键 """
        ...

```

### 3.9 EncDecMultiModalProcessor

源码位置: processor.py:1710-1785

针对 **Encoder-Decoder** 架构 (如 Whisper-based 模型) 的特殊处理器:

```

class EncDecMultiModalProcessor(BaseMultiModalProcessor[_I]):
    @abstractmethod
    def create_encoder_prompt(self, prompt, mm_items) -> str | list[int]:
        """ 创建编码器输入 prompt """
        raise NotImplementedError

    def create_decoder_prompt(self, prompt, mm_items) -> str | list[int]:
        """ 创建解码器输入 prompt (默认直接使用原始 prompt) """
        return prompt

    def apply(self, inputs, timing_ctx) -> MultiModalEncDecInput:
        encoder_prompt = self.create_encoder_prompt(inputs.prompt, inputs.mm_data_items)
        encoder_processor_inputs = ProcessorInputs(encoder_prompt, ...)
        encoder_inputs = super().apply(encoder_processor_inputs, timing_ctx)
        return self._get_enc_dec_inputs(prompt, mm_items, encoder_inputs)

```

## 四、输入解析

源码位置: parse.py

### 4.1 MultiModalDataParser 一数据归一化引擎

**MultiModalDataParser** 是将用户传入的 **MultiModalDataDict** 归一化为结构化 **MultiModalDataItems** 的核心组件。

支持的模态子解析器

```

# parse.py:671-677
def _get_subparsers(self) -> Mapping[str, ModalityDataParser]:
    return {
        "audio": self._parse_audio_data,
        "image": self._parse_image_data,
        "video": self._parse_video_data,
    }

```

```

        "vision_chunk": self._parse_vision_chunk_data, # 统一图像/视频块
    }

```

数据类型识别 parse.py:512-521 的 is\_embeddings() 方法自动检测预计算嵌入:

```

@classmethod
def is_embeddings(cls, data: object) -> TypeGuard[torch.Tensor | list[torch.Tensor]]:
    if isinstance(data, torch.Tensor):
        return data.ndim == 3 # (batch, seq_len, hidden_size)
    if is_list_of(data, torch.Tensor) and len(data) > 0:
        return data[0].ndim == 2 # [(seq_len, hidden_size), ...]
    return False

```

## 4.2 各模态解析逻辑

图像解析

```

# parse.py:592-611
def _parse_image_data(self, data):
    if self.is_embeddings(data):
        return ImageEmbeddingItems(data, self.expected_hidden_size)

    # 单张图: PIL.Image / ndarray(3D) / tensor(3D)
    if isinstance(data, (PILImage.Image, MediaWithBytes)) or \
        (isinstance(data, (np.ndarray, torch.Tensor)) and data.ndim == 3):
        data_items = [data]
    # 多张图: list / ndarray(4D) / tensor(4D)
    elif isinstance(data, (np.ndarray, torch.Tensor)):
        data_items = [elem for elem in data]
    else:
        data_items = data

    return ImageProcessorItems(data_items)

```

音频解析

```

# parse.py:553-590
def _parse_audio_data(self, data):
    if self.is_embeddings(data):
        return AudioEmbeddingItems(data, self.expected_hidden_size)

    # 支持格式: tuple(waveform, sr) / list / np.ndarray(1D) / tensor(1D)
    # 自动重采样和声道归一化
    for data_item in data_items:
        audio, orig_sr = self._get_audio_with_sr(data_item)
        if orig_sr is not None:
            new_audio = self.audio_resampler.resample(audio, orig_sr=orig_sr)
        if self.target_channels is not None:
            new_audio = normalize_audio(new_audio, spec)
        new_audios.append(new_audio)

    return AudioProcessorItems(new_audios)

```

视频解析



```

# parse.py:613-653
def _parse_video_data(self, data):
    if self.is_embeddings(data):
        return VideoEmbeddingItems(data, self.expected_hidden_size)

    # 支持格式: list[PIL.Image] / ndarray(4D) / tensor(4D) / tuple(ndarray, metadata)
    # 可选 metadata 提取 (fps, duration, frames_indices 等)
    ...
    return VideoProcessorItems(new_videos, metadata=metadata_lst)

```

#### 4.3 数据项类层次

```

classDiagram
    class ModalityDataItems {
        +get_count() int
        +get(index) item
        +get_all() list
        +get_processor_data() Mapping
        +get_passthrough_data() Mapping
    }

    class ProcessorBatchItems {
        +_unwrap(item) raw_item
    }

    class EmbeddingItems {
        -_validate_ndim()
        -_validate_hidden_size()
        +get_feature_size() int
    }

    class DictEmbeddingItems {
        +fields_config
        +required_fields
    }

    class AudioProcessorItems
    class AudioEmbeddingItems
    class ImageProcessorItems {
        +get_image_size() ImageSize
    }
    class ImageEmbeddingItems
    class VideoProcessorItems {
        +get_num_frames() int
        +get_frame_size() ImageSize
    }
    class VideoEmbeddingItems

    ModalityDataItems <|-- ProcessorBatchItems
    ModalityDataItems <|-- EmbeddingItems
    ModalityDataItems <|-- DictEmbeddingItems

    ProcessorBatchItems <|-- AudioProcessorItems
    ProcessorBatchItems <|-- ImageProcessorItems

```

```

ProcessorBatchItems <|-- VideoProcessorItems

EmbeddingItems <|-- AudioEmbeddingItems
EmbeddingItems <|-- ImageEmbeddingItems
EmbeddingItems <|-- VideoEmbeddingItems

```

#### 4.4 嵌入验证

parse.py:128-215 实现了严格的多模态嵌入验证:

```

def validate_embedding_ndim(tensor, modality, index=None):
    """Single embeddings should be 2D, batched should be 3D."""
    if tensor.ndim < 2 or tensor.ndim > 3:
        raise ValueError(
            f"{modality.capitalize()} embedding must be 2D or 3D, "
            f"got {tensor.ndim}D with shape {tuple(tensor.shape)}"
        )

```

EmbeddingItems 类还会验证 hidden dimension 是否匹配模型预期, 防止维度不一致导致的推理时崩溃。

## 五、特征提取管线

### 5.1 视觉编码器

vLLM 内置了优化的视觉编码器实现, 用于将图像/视频像素转换为特征向量。

**CLIP Vision Model** 源码位置: model\_executor/models/clip.py

CLIP (Contrastive Language-Image Pre-training) 视觉编码器是 LLaVA 系列模型的核心组件。vLLM 的实现支持: - 并行注意力优化 - 动态 patch 数量处理 - 特征选择策略 ("full", "patch", etc.)

**SigLIP Vision Model** 源码位置: model\_executor/models/siglip.py

SigLIP (Sigmoid Loss for Language-Image Pre-training) 是 CLIP 的改进版本, 被 LLaVA-NeXT 等新模型广泛采用。

在 LLaVA-NeXT 中 (llava\_next.py:33):

```

from .siglip import SiglipVisionModel
from .clip import CLIPVisionModel
# 根据模型配置选择视觉编码器

```

### 5.2 音频编码器

Whisper 系列音频编码器用于处理音频输入, 将波形转换为语义特征向量。

### 5.3 Encoder Budget Management

源码位置: encoder\_budget.py

Encoder Budget 是 vLLM 多模态系统的关键资源管理机制, 用于控制编码器的计算和内存开销。

**MultiModalBudget** 类

```

# encoder_budget.py:44-192
class MultiModalBudget:
    """Helper class to calculate budget information for multi-modal models."""

```

```

def __init__(self, vllm_config, mm_registry):
    self.model_config = model_config = vllm_config.model_config
    self.scheduler_config = scheduler_config = vllm_config.scheduler_config

    # 创建 processor 以获取 token 数信息
    cache = mm_registry.processor_only_cache_from_config(vllm_config)
    processor = mm_registry.create_processor(model_config, cache=cache)

    # 区分 tower modalities 和 embedding-only modalities
    tower_modalities = {modality for modality in supported_mm_limits
                        if mm_limits.get(modality, 0) > 0}
    embed_only_modalities = {modality for modality in supported_mm_limits
                             if enable_mm_embeds and mm_limits.get(modality, 0) == 0}

    # 计算每个模态每项的最大 token 数
    all_mm_max_toks_per_item = get_mm_max_toks_per_item(...)

    # 计算编码器预算
    encoder_compute_budget, encoder_cache_size = compute_mm_encoder_budget(
        scheduler_config, active_mm_max_toks_per_item
    )

```

预算计算策略

```

# encoder_budget.py:140-180
def _get_max_items(self, modality, max_tokens_per_item):
    # 编码器预算限制
    max_encoder_items_per_batch = encoder_budget // max_tokens_per_item

    # 解码器预算限制 (基于 sequence length)
    max_items_per_prompt = max(
        1, min(mm_limit, self.max_model_len // max_tokens_per_item)
    )

    # batch 级别限制
    max_decoder_items_per_batch = max_num_reqs * max_items_per_prompt

    # 取较小值
    max_items_per_batch = max(
        1, min(max_encoder_items_per_batch, max_decoder_items_per_batch)
    )

    return max_items_per_prompt, max_items_per_batch

```

Budget 组成

| 组件                      | 来源                          | 作用                 |
|-------------------------|-----------------------------|--------------------|
| encoder_compute_budget  | compute_mm_encoder_budget() | 基于调度器配置的计算预算       |
| encoder_cache_size      | compute_mm_encoder_budget() | 缓存容量限制             |
| mm_max_toks_per_item    | Dummy input profiling       | 每个媒体项产生的最大 token 数 |
| mm_max_items_per_prompt | min(limit, seq_len/tokens)  | 每个 prompt 最大媒体数    |
| mm_max_items_per_batch  | min(encoder, decoder)       | 每个 batch 最大媒体总数    |

## 六、支持的多模态模型

### 6.1 LLaVA-NeXT

源码位置: `model_executor/models/llava_next.py`

LLaVA-NeXT 是 LLaVA 的增强版本, 支持任意分辨率图像处理。

核心特性: - 使用 `LlavaNextProcessor` 处理输入 - 支持 CLIP 或 SigLIP 作为视觉编码器 - dynamic resolution: 根据图像尺寸动态调整 token 数量 - anyres grid: 将大图像分割为多个 patch 处理

```
# llava_next.py:88-101
class LlavaNextProcessingInfo(BaseLlavaProcessingInfo):
    def get_hf_config(self) -> LlavaNextLikeConfig:
        return self.ctx.get_hf_config(LlavaNextConfig)

    def get_hf_processor(self, **kwargs):
        hf_processor = self.ctx.get_hf_processor(LlavaNextProcessor, **kwargs)
        # 补充 patch_size (某些模型配置中缺失)
        if hf_processor.patch_size is None:
            patch_size = self.get_vision_encoder_info().get_patch_size()
            hf_processor.patch_size = patch_size
        return hf_processor
```

Token 数计算 (`llava_next.py:104-139`):

```
def get_num_image_tokens(self, *, image_width, image_height):
    """Get the number of image tokens for the given image dimensions."""
    # 基础 feature size (基于 vision_feature_select_strategy)
    base_feature_size = get_num_selected_vision_tokens(
        vision_encoder_info.get_num_image_tokens(image_width, image_height),
        hf_config.vision_feature_select_strategy,
    )

    # AnyRes grid shape
    num_patch_height, num_patch_width = get_anyres_image_grid_shape(
        image_size=(image_height, image_width),
        grid_pinpoints=hf_config.image_grid_pinpoints,
        patch_size=vision_encoder_info.get_image_size(),
    )

    # Unpadded features + newline features
    unpadded_feature_size, newline_feature_size = self._get_num_unpadded_features(...)
    return unpadded_feature_size + newline_feature_size + base_feature_size
```

相关模型文件: - `llava.py` —LLaVA 1.5 基线 - `llava_onevision.py` —LLaVA-OneVision (含视频理解) - `llava_next_video.py` —LLaVA-NeXT-Video

### 6.2 Qwen2-VL / Qwen-VL 系列

源码位置: - `qwen2_vl.py` —Qwen2-VL - `qwen_vl.py` —Qwen-VL (第一代) - `qwen2_5_vl.py` —Qwen2.5-VL - `qwen3_vl.py` —Qwen3-VL - `qwen3_vl_moe.py` —Qwen3-VL-MoE

Qwen2-VL 是阿里通义千问团队开发的多模态大模型, 具有以下特点:

独特的输入表示 (`qwen2_vl.py:120-145`):

```

class Qwen2VLImagePixelInputs(TensorSchema):
    """
    Dimensions:
        - np: Total number of patches over each image
        - ni: Number of images
        - cps: Number of channels * patch_size * patch_size
    """
    type: Literal["pixel_values"]
    pixel_values: Annotated[torch.Tensor, TensorShape("np", "cps")]
    image_grid_thw: Annotated[torch.Tensor, TensorShape("ni", 3)]
    # image_grid_thw: (num_images, 3) in (grid_t, grid_h, grid_w) format

```

核心特性: - **M-RoPE (Multimodal Rotary Positional Embeddings)**: 统一处理 2D 空间和 1D 时间位置  
 - 动态分辨率: naive/dynamic/smart 三种 resize 策略 - 原生视频支持: 通过 temporal 维度扩展 - 视觉-语言统一架构: 共享 transformer backbone

### 6.3 Pixtral

源码位置: model\_executor/models/pixtral.py

Pixtral 是 Mistral AI 发布的开源多模态模型, 基于 Mistral 架构扩展。

核心特性 (pixtral.py:1-99):

```

# 使用 MistralCommon 协议进行图像分块处理
from mistral_common.protocol.instruct.chunk import ImageChunk, TextChunk
from transformers.models.pixtral import (
    PixtralRotaryEmbedding,
    apply_rotary_pos_emb,
    position_ids_in_meshgrid,
)

```

- 基于 mistral\_common 的图像分块协议
- 2D Rotary Embedding (meshgrid 式位置编码)
- 支持 xformers FlashAttention 加速
- 原生 Eagle3 推测解码支持

## 七、Assets 系统

源码目录: vllm/assets/

Assets 系统提供测试和示例用的多模态资源管理, 包括自动下载和本地缓存。

### 7.1 基础设施

源码位置: base.py

```

# base.py:10-40
VLLM_S3_BUCKET_URL = "https://vllm-public-assets.s3.us-west-2.amazonaws.com"

def get_cache_dir() -> Path:
    """ 获取资源缓存目录路径 """
    path = Path(envs.VLLM_ASSETS_CACHE)
    path.mkdir(parents=True, exist_ok=True)
    return path

@lru_cache

```

```
def get_vllm_public_assets(filename: str, s3_prefix: str | None = None) -> Path:
    """ 从 S3 下载资源文件并返回本地路径 """
    asset_path = get_cache_dir() / "vllm_public_assets" / filename
    if not asset_path.exists():
        global_http_connection.download_file(
            f"{VLLM_S3_BUCKET_URL}/{filename}", asset_path, ...
        )
    return asset_path
```

## 7.2 图像资源

源码位置: image.py

# image.py:15-62

```
ImageAssetName = Literal[
    "stop_sign", "cherry_blossom", "hato",
    "2560px-Gfp-wisconsin-madison-the-nature-boardwalk",
    "Grayscale_8bits_palette_sample_image",
    "1280px-Venn_diagram_rgb", "RGBA_comp",
    # ... 更多测试图像
]
```

```
@dataclass(frozen=True)
```

```
class ImageAsset:
```

```
    name: ImageAssetName
```

```
    @property
```

```
    def pil_image(self) -> Image.Image:
```

```
        return self.pil_image_ext(ext="jpg")
```

```
    @property
```

```
    def image_embeds(self) -> torch.Tensor:
```

```
        """ 预计算的图像嵌入 (仅用于 LLaVA 1.5 测试) """
```

```
        image_path = self.get_path("pt")
```

```
        return torch.load(image_path, map_location="cpu", weights_only=True)
```

## 7.3 音频资源

源码位置: audio.py

# audio.py:17-37

```
AudioAssetName = Literal["winning_call", "mary_had_lamb"]
```

```
@dataclass(frozen=True)
```

```
class AudioAsset:
```

```
    name: AudioAssetName
```

```
    @property
```

```
    def audio_and_sample_rate(self) -> tuple[npt.NDArray, float]:
```

```
        audio_path = get_vllm_public_assets(filename=self.filename, s3_prefix=ASSET_DIR)
```

```
        return load_audio(audio_path, sr=None)
```

```
    @property
```

```
    def url(self) -> str:
```

```
        return urljoin(VLLM_S3_BUCKET_URL, f"{ASSET_DIR}/{self.name}.ogg")
```

## 7.4 视频资源

源码位置: video.py

*# video.py:103-137*

```
VideoAssetName = Literal["baby_reading"]
```

```
@dataclass(frozen=True)
```

```
class VideoAsset:
```

```
    name: VideoAssetName
```

```
    num_frames: int = -1 # -1 表示全部帧
```

```
    @property
```

```
    def pil_images(self) -> list[Image.Image]:
```

```
        return video_to_pil_images_list(self.video_path, self.num_frames)
```

```
    @property
```

```
    def np_ndarrays(self) -> npt.NDArray:
```

```
        return video_to_ndarrays(self.video_path, self.num_frames)
```

```
    @property
```

```
    def metadata(self) -> dict[str, Any]:
```

```
        return video_get_metadata(self.video_path, self.num_frames)
```

视频资源的特殊之处在于它从 **HuggingFace Dataset** 下载（而非 S3），并使用 OpenCV 进行帧提取。

---

## 八、端到端数据流

下面展示一个完整的图像理解请求在 vLLM 多模态管线中的流转过程:

sequenceDiagram

participant Client

participant API as API Server

participant Conn as MediaConnector

participant Parser as DataParser

participant Proc as MultiModalProcessor

participant HF as HF Processor

participant Enc as Vision/Audio Encoder

participant LLM as LLM Backbone

Client->>API: POST /generate {prompt: " 描述这张图", image: "http://..."}

API->>Conn: fetch\_image(url)

Conn->>Conn: load\_from\_url() → HTTP download / cache lookup

Conn->>Conn: ImageMediaIO.load\_bytes() → PIL.Image

Conn-->>API: MediaWithBytes(Image, original\_bytes)

API->>Parser: parse\_mm\_data({"image": [pil\_image]})

Parser->>Parser: \_parse\_image\_data()

Parser->>Parser: 归一化为 ImageProcessorItems

Parser-->>API: MultiModalDataItems

API->>Proc: apply(ProcessorInputs)

Proc->>Proc: \_cached\_apply\_hf\_processor()

Proc->>HF: call\_hf\_processor(text=prompt, images=[image])

HF->>HF: 图像预处理 (resize, normalize)  
HF->>HF: 文本 tokenize  
HF-->>Proc: BatchFeature{input\_ids, pixel\_values, ...}  
  
Proc->>Proc: \_get\_mm\_fields\_config() → 字段元数据  
Proc->>Proc: \_get\_mm\_hashes() → 内容哈希  
Proc->>Proc: \_get\_mm\_prompt\_updates() → PromptReplacement rules  
  
Proc->>Proc: \_maybe\_apply\_prompt\_updates()  
Note over Proc: 将 <image> 替换为 N 个 <image> placeholder tokens  
  
Proc->>Proc: \_find\_mm\_placeholders()  
Note over Proc: 定位所有 placeholder 的位置范围  
  
Proc-->>API: MultiModalInput{<br/>prompt\_token\_ids,<br/>mm\_kwargs={pixel\_values},<br/>mm\_placeholders}<br/>  
  
API->>Enc: encode(pixel\_values) [首次或 cache miss]  
Enc-->>API: image\_features [batch, num\_patches, hidden\_size]  
  
API->>LLM: forward(<br/>input\_ids=prompt\_token\_ids,<br/>mm\_placeholders→inject image\_features<br/>)  
LLM-->>Client: generated text response

关键设计原则总结

| 原则   | 实现方式                            |
|------|---------------------------------|
| 懒加载  | Processor 工厂模式，按需创建             |
| 缓存优先 | 多级缓存（process / LRU / shm）减少重复计算 |
| 可扩展性 | 注册表模式 + Protocol 抽象，易于添加新模态     |
| 类型安全 | 泛型 + TypeGuard 确保编译期检查          |
| 容错降级 | soundfile → PyAV 双后端，优雅降级       |
| 安全验证 | 域名白名单、路径沙箱、embedding 维度校验       |

vLLM 量化方案全面分析

定位: vLLM 量化（Quantization）子系统的全面架构分析，涵盖从配置层到 CUDA 内核的完整技术栈。

```
graph TB
    subgraph QM[" 量化方案矩阵"]
        direction LR
        FP8["FP8 E4M3"]
        INT8["INT8 W8A8"]
        INT4["INT4 W4A16"]
        GPTQ["GPTQ 2-8bit"]
        AWQ["AWQ 4bit"]
        GGUF["GGUF 多精度"]
        NVFP4["NVFP4 Blackwell"]
        MXFP4["MXFP4 Microscaling"]
        MXFP8["MXFP8 Microscaling"]
        MARLIN["Marlin 4-bit"]
        MACHETE["Machete Hopper+"]
    end

    subgraph IL[" 实现层次"]
```



```

    QC["QuantizationConfig<br/> 配置层"]
    QM["QuantizeMethodBase<br/> 方法层"]
    LK["LinearKernel<br/> 内核选择层"]
    CK["CUDA Kernel<br/> 算子实现层"]
end

subgraph KT[" 内核类型"]
    ScaledMM["ScaledMM<br/> 缩放矩阵乘法"]
    MPKernel["MPLinearKernel<br/> 混合精度内核"]
    Mxfp8K["Mxfp8LinearKernel<br/>MXFP8 专用内核"]
    NvFp4K["NvFp4LinearKernel<br/>NVFP4 专用内核"]
end

QM --> QC
LK --> QM
CK --> LK
ScaledMM --> CK
MPKernel --> CK
Mxfp8K --> CK
NvFp4K --> CK

FP8 --> QC
INT8 --> QC
INT4 --> QC
GPTQ --> QC
AWQ --> QC
GGUF --> QC
NVFP4 --> QC
MXFP4 --> QC
MXFP8 --> QC
MARLIN --> QC
MACHETE --> QC

style QM fill:#fff9c4,stroke:#f9a825
style KT fill:#e1bee7,stroke:#8e24aa
style IL fill:#b3e5fc,stroke:#0277bd
style QM fill:#fff9c4,stroke:#f9a825

```

---

## 目录

- 一、量化方案矩阵
- 二、FP8 量化
- 三、INT8 / INT4 量化（GPTQ / AWQ）
- 四、GGUF 格式
- 五、NVFP4 Blackwell 4-bit 浮点
- 六、MXFP4 / MXFP8 块缩放浮点
- 七、Marlin 高性能 4-bit 内核
- 八、Machete 新一代量化内核
- 九、量化线性层与 GEMM 操作
- 十、量化配置体系
- 十一、量化方案选择决策树
- 十二、量化数据流全景

一、量化方案矩阵

vLLM 支持的量化方案覆盖了从浮点到整数的全谱系，下表为各方案的横向对比：

| 方案                   | 权重精度                         | 激活精度           | 用途/特点                      | 最小 GPU 能力      | 配置类                 | 关键文件                          |
|----------------------|------------------------------|----------------|----------------------------|----------------|---------------------|-------------------------------|
| <b>FP8</b>           | float8_e4m3fn                | BF16/FP16 (动态) | H100/H200 原生支持，在线/离线量化     | SM 75 (Turing) | Fp8Config           | fp8.py                        |
| <b>FP8 Per-Block</b> | float8_e4m3fn                | 动态 128-block   | DeepSeek 风格块级量化            | SM 75          | Fp8Config           | fp8.py:292-311                |
| <b>MXFP8</b>         | float8 + uint8 scale         | BF16           | MicroScaling FP8, 32 元素块缩放 | SM 80          | ModelOptMxFP8Config | mxfp8config.py                |
| <b>GPTQ</b>          | INT4/INT8 (packed int32)     | FP16           | 训练后量化，支持 act-order         | SM 60          | GPTQConfig          | gptq.py                       |
| <b>GPTQ-Marlin</b>   | UINT4B8/UINT8B8              | FP16/BF16      | Marlin 加速的 GPTQ            | SM 75          | GPTQMarlinConfig    | gptq_marlin.py                |
| <b>AWQ</b>           | UINT4 (packed int32)         | FP16           | 激活感知量化                     | SM 75          | AWQConfig           | awq.py                        |
| <b>AWQ-Marlin</b>    | UINT4                        | FP16/BF16      | Marlin 加速的 AWQ             | SM 75          | AWQMarlinConfig     | awq_marlin.py                 |
| <b>GGUF</b>          | Q2~Q8 (多精度)                  | FP16/BF16      | llama.cpp 兼容格式             | SM 60          | GGUFConfig          | gguf.py                       |
| <b>NVFP4</b>         | float4_e2m1fn (uint8 packed) | FP8-E4M3       | Blackwell 架构原生 4-bit 浮点    | SM 100+        | ModelOptNvFP4Config | nvfp4config.py, nvfp4/base.py |
| <b>MXFP4</b>         | uint8 (2×FP4/byte)           | BF16           | MicroScaling FP4, MoE 专用   | SM 80          | Mxfp4Config         | mxfp4.py                      |
| <b>Marlin</b>        | UINT4/UINT8/FP8              | FP8/INT8/FP8   | 高性能 4/8-bit GEMM 内核        | SM 75+         | —                   | marlin_utils.py               |
| <b>Machete</b>       | UINT4/UINT8                  | FP16/BF16      | Hopper (SM90) CUTLASS 内核   | SM 90          | —                   | machete.py, machete_utils.py  |

在线量化 (Online Quantization)

除上述离线量化方案外，vLLM 还支持在线量化——加载全精度 checkpoint 后在推理时动态量化：

```
# config/quantization.py 中定义
class OnlineQuantScheme(Enum):
    FP8_PER_TENSOR = "fp8_per_tensor" # FP8 per-tensor 缩放
    FP8_PER_BLOCK = "fp8_per_block" # FP8 128×128 块缩放 (DeepSeek 风格)
    INT8_PER_CHANNEL_WEIGHT_ONLY = "int8_per_channel_weight_only" # MoE 专家权重 INT8
    MXFP8 = "mxfp8" # MicroScaling FP8
```

源码位置：config/quantization.py:12-27

所有已注册量化方法

在 init.py:12-47 中通过 QuantizationMethods Literal 类型统一注册：

```
QuantizationMethods = Literal[
    "awq", "fp8", "fbgemm_fp8", "fp_quant", "modelopt",
    "modelopt_fp4", "modelopt_mxfp8", "modelopt_mixed",
    "gguf", "gptq_marlin", "awq_marlin", "gptq",
    "humming", "compressed-tensors", "bitsandbytes",
    "experts_int8", "quark", "moe_wna16", "torchao",
    "inc", "mxfp4", "gpt_oss_mxfp4", "deepseek_v4_fp8",
    "cpu_awq", "online",
    "fp8_per_tensor", "fp8_per_block",
    "int8_per_channel_weight_only", "mxfp8",
]
```

---

## 二、FP8 量化

### 2.1 架构概述

FP8 是 vLLM 中最重要的量化方案之一，充分利用 NVIDIA H100/H200 的原生 FP8 Tensor Core 支持。其核心设计围绕两个维度展开：

1. 离线 vs 在线：是否以 FP8 格式存储 checkpoint
2. 激活策略：static（静态）vs dynamic（动态）缩放

graph TD

```
FC["Fp8Config"] --> OL["Offline 离线 <br/>is_checkpoint_fp8_serialized=True"]
FC --> ON["Online 在线 <br/>is_checkpoint_fp8_serialized=False"]
```

```
OL --> FLM["Fp8LinearMethod"]
ON --> FOLM["Fp8OnlineLinearMethod"]
```

```
OL --> FMM["Fp8MoEMethod"]
ON --> FOMM["Fp8OnlineMoEMethod"]
```

```
FC --> KV["Fp8KVCacheMethod"]
```

```
FLM --> KSEL["init_fp8_linear_kernel()<br/> 内核选择"]
FOLM --> KSEL
```

```
KSEL --> SM["torch._scaled_mm"]
KSEL --> MK["MarlinFP8ScaledMM"]
KSEL --> DG["DeepGEMM"]
KSEL --> BS["BlockScaledMM"]
KSEL --> FI["FlashInfer"]
```

```
style FC fill:#fff9c4,stroke:#f9a825
style KSEL fill:#b3e5fc,stroke:#0277bd
```

### 2.2 Fp8Config 核心配置

定义于 fp8.py:97-225:

```
class Fp8Config(QuantizationConfig):
    def __init__(
        self,
        is_checkpoint_fp8_serialized: bool = False, # 是否为 FP8 序列化 checkpoint
        activation_scheme: str = "dynamic",         # "static" 或 "dynamic"
```

```

        ignored_layers: list[str] | None = None,          # 跳过量化的层
        weight_block_size: list[int] | None = None,      # 块级量化尺寸 [N, K]
    ) -> None:

```

关键属性说明:- `is_checkpoint_fp8_serialized`: 决定走 `Fp8LinearMethod`(离线) 还是 `Fp8OnlineLinearMethod`(在线) - `activation_scheme`: `dynamic` 模式下激活值每 token 动态计算缩放因子; `static` 模式使用预计算的固定缩放因子 - `weight_block_size`: 当设置为 `[128, 128]` 时启用 DeepSeek V3/R1 风格的  $128 \times 128$  块级量化

## 2.3 量化 Key 体系

FP8 的量化行为由 `QuantKey` 精确控制, 定义于 `quant_utils.py`:100-182:

| QuantKey                           | 含义                                     | 使用场景                |
|------------------------------------|----------------------------------------|---------------------|
| <code>kFp8StaticTensorSym</code>   | FP8 + 静态, per-tensor scale             | 静态激活量化              |
| <code>kFp8DynamicTensorSym</code>  | FP8 + 动态, per-tensor scale             | 标准 dynamic 模式       |
| <code>kFp8DynamicTokenSym</code>   | FP8 + 动态, per-token scale              | Cutlass 支持时的高效模式    |
| <code>kFp8Dynamic128Sym</code>     | FP8 + 动态, 128-block scale              | DeepSeek 块级量化 (激活侧) |
| <code>kFp8Static128BlockSym</code> | FP8 + 静态, $128 \times 128$ block scale | DeepSeek 块级量化 (权重侧) |

## 2.4 离线 FP8 Linear 方法

`Fp8LinearMethod` 的核心流程:

```

class Fp8LinearMethod(LinearMethodBase):
    def create_weights(self, layer, ...):
        # 1. 创建 FP8 权重参数
        weight = create_fp8_weight_parameter(...)
        # 2. 创建权重量级缩放因子
        scale = create_fp8_scale_parameter(...)
        # 3. 初始化线性内核 (自动选择最优后端)
        self.fp8_linear = init_fp8_linear_kernel(
            activation_quant_key=self.activation_quant_key,
            weight_quant_key=self.weight_quant_key,
            ...
        )

    def process_weights_after_loading(self, layer):
        # 处理融合模块 (如 QKV) 的多 shard 权重
        weight, weight_scale, input_scale = process_fp8_weight_tensor_strategy(
            weight, weight_scale, layer.logical_widths, ...
        )
        # 转置为 [K, N] 布局 (scaled_mm 要求)
        weight = weight.t()
        # 交给选定的内核做后处理 (如 Marlin 重排)
        self.fp8_linear.process_weights_after_loading(layer)

    def apply(self, layer, x, bias=None):
        if envs.VLLM_BATCH_INVARIANT:
            # batch invariant 模式: 反量化到 BF16 再计算

```

```

        return self._bf16_fallback(layer, x, bias)
    if self.use_marlin:
        return self.fp8_linear.apply_weights(layer, x, bias)
    return self.fp8_linear.apply_weights(layer, x, bias)

```

## 2.5 在线 FP8 量化

Fp8OnlineLinearMethod 的关键区别在于使用 **meta device** 延迟分配:

```

class Fp8OnlineLinearMethod(Fp8LinearMethod):
    uses_meta_device: bool = True # 标记: 在 meta device 上创建权重

    def create_weights(self, layer, ...):
        # 权重在 meta device 上创建 (不占实际显存)
        weight = ModelWeightParameter(
            data=torch.empty(..., device="meta", dtype=params_dtype),
            ...
        )
        initialize_online_processing(layer)

    def process_weights_after_loading(self, layer):
        # 加载完成后在线量化: BF16 → FP8
        qweight, weight_scale = ops.scaled_fp8_quant(layer.weight, scale=None)
        replace_parameter(layer, "weight", qweight.data)
        replace_parameter(layer, "weight_scale", weight_scale.data)

```

核心量化操作 `ops.scaled_fp8_quant` 将全精度权重动态转换为 FP8。

## 2.6 FP8 MoE 方法

Fp8MoEMethod 和 Fp8OnlineMoEMethod 为 MoE 层提供 FP8 量化支持:

- 权重格式: `w13_weight` (gate\_up 融合) 和 `w2_weight` (down\_proj), dtype 为 `float8_e4m3fn`
- 缩放因子: per-tensor (`w13_scale`, `w2_scale`) 或 per-block (`w13_weight_scale_inv`)
- 后端选择: 通过 `select_fp8_moe_backend()` 自动选择 FlashInfer/AITER/CUTLASS 等后端
- 权重重排: `convert_to_fp8_moe_kernel_format()` 将权重转换为各后端的运行时格式

## 2.7 MXFP8 (MicroScaling FP8)

MXFP8 是 NVIDIA 推出的微缩放浮点格式, 每 32 个元素共享一个 uint8 缩放因子。相关实现在 `modelopt.py` 和 `mxfp8/` 目录中。

支持的内核后端:- `MarlinMxfp8LinearKernel` —Marlin 加速的 MXFP8 - `FlashInferCutlassMxfp8LinearKernel`  
 —FlashInfer + CUTLASS - `EmulationMxfp8LinearKernel` —仿真模式 (非 Blackwell 设备)

## 三、INT8 / INT4 量化 (GPTQ / AWQ)

### 3.1 GPTQ 量化

GPTQ 是基于近似二阶信息的训练后量化方法。

**GPTQConfig** 定义于 `gptq.py:44-223`:

```

class GPTQConfig(QuantizationConfig):
    def __init__(
        self,

```

```

weight_bits: int,          # 支持 2/3/4/8 bit
group_size: int,          # 分组大小, -1 表示 per-channel
desc_act: bool,           # 是否启用 activation ordering
lm_head_quantized: bool,  # 是否量化 lm_head
dynamic: dict,            # GPTQModel 的逐模块动态配置
autoround_version: str="", # AutoRound 版本标识
modules_in_block_to_quantize: list[str] | None = None,
checkpoint_format: str="", # "gptq_v2" 或 ""
):
    self.pack_factor = Fraction(32, self.weight_bits) # 打包因子

```

**GPTQLinearMethod** gptq.py:231-399:

```

class GPTQLinearMethod(LinearMethodBase):
    def create_weights(self, layer, ...):
        # 量化权重: 按行打包进 int32
        qweight = PackedvLLMParameter(
            data=torch.empty(
                input_size_per_partition // self.quant_config.pack_factor,
                output_size_per_partition,
                dtype=torch.int32,
            ),
            input_dim=0, output_dim=1,
            packed_dim=0, # 沿输入维度打包
            packed_factor=self.quant_config.pack_factor,
        )
        # Activation order 索引
        g_idx = RowvLLMParameter(...)
        # 零点 (打包)
        qzeros = PackedColumnParameter(...)
        # 缩放因子
        scales = ChannelQuantScaleParameter(...) # or GroupQuantScaleParameter

    def process_weights_after_loading(self, layer):
        # Exllama shuffle: 按 g_idx 重排权重
        if layer.exllama_state == ExllamaState.UNINITIALIZED:
            if self.quant_config.desc_act:
                layer.g_idx.data = torch.argsort(layer.g_idx).to(torch.int)
                ops.gptq_shuffle(layer.qweight, layer.g_idx, self.quant_config.weight_bits)

    def apply(self, layer, x, bias=None):
        output = ops.gptq_gemm(
            reshaped_x, layer.qweight, layer.qzeros,
            layer.scales, layer.g_idx,
            layer.exllama_state == ExllamaState.READY,
            self.use_v2_format, # GPTQ v1/v2 格式差异
            self.quant_config.weight_bits,
        )

```

**GPTQ v1 vs v2** 差异: v2 格式对零点的处理方式不同, 需要不同的 GEMM 内核。

**GPTQ 动态配置** GPTQModel 引入的动态配置允许对模型的不同层使用不同量化参数:

```

# gptq.py:61-83 中的示例
dynamic = {

```

```

r"+:.*\.(?:1[0-5])\..*": {"bits": 8},          # 第 10-15 层用 8bit
r"+:.*\.(?:1[6-9]|20|21)\..*": {"bits": 8, "group_size": 64},
r"-:.*\.moe\..*": {},                          # 跳过所有 MoE 层
}

```

### 3.2 AWQ 量化

AWQ (Activation-aware Weight Quantization) 通过分析激活分布来保护重要权重。

AWQConfig awq.py:34-95:

```

class AWQConfig(QuantizationConfiguration):
    def __init__(self, weight_bits=4, group_size=128, zero_point=True,
                 modules_to_not_convert=None):
        # AWQ 固定为 4-bit
        assert self.weight_bits == 4
        self.pack_factor = 32 // self.weight_bits # = 8

```

AWQLinearMethod awq.py:172-286:

```

class AWQLinearMethod(LinearMethodBase):
    def create_weights(self, layer, ...):
        # AWQ 沿输出维度打包 (与 GPTQ 不同!)
        qweight = PackedvLLMParameter(
            data=torch.empty(
                input_size_per_partition,
                output_size_per_partition // self.quant_config.pack_factor,
                dtype=torch.int32,
            ),
            input_dim=0, output_dim=1,
            packed_dim=1, # 沿输出维度打包 (AWQ 特有)
            packed_factor=self.quant_config.pack_factor,
        )
        scales = GroupQuantScaleParameter(...)
        qzeros = PackedvLLMParameter(...)

    def apply(self, layer, x, bias=None):
        # 小 batch 启发式: 直接反量化 + matmul
        if x.shape[:-1].numel() >= 256 or envs.VLLM_BATCH_INVARIANT:
            out = ops.awq_dequantize(qweight, scales, qzeros, 0, 0, 0)
            out = torch.matmul(reshaped_x, out)
        else:
            # 大 batch: 使用优化的 AWQ GEMM kernel
            out = ops.awq_gemm(reshaped_x, qweight, scales, qzeros, pack_factor)

```

AWQ vs GPTQ 打包方向对比: - GPTQ: qweight 形状 [K//pack, N], 沿输入维 (dim 0) 打包 - AWQ: qweight 形状 [K, N//pack], 沿输出维 (dim 1) 打包

### 3.3 AWQ → Marlin 格式转换

由于 AWQ 使用非标准的 bit 打包顺序, 转换为 Marlin 格式需要特殊处理。

awq\_marlin.py:67-149 定义了转换逻辑:

```

_REVERSE_AWQ_PACK_ORDER = [0, 4, 1, 5, 2, 6, 3, 7] # AWQ 特有的位排列

def _convert_awq_to_standard_format(layer, w_q_name, w_zp_name, size_bits):

```

```

""" 将 AWQ 的非标准格式转换为 GPTQ-like 标准格式 """
pack_factor = 32 // size_bits
# 1. 解包 AWQ qweight, 修复位顺序
unpacked = (qw.unsqueeze(-1) >> shifts) & mask
unpacked = unpacked[:, :, reverse_order] # 修正 AWQ 位序
# 2. 从沿输出维打包转为沿输入维打包
new_qw = repack_along_input_dim(unpacked)
# 3. 同样处理 qzeros
new_qz = convert_and_repack_zeros(qz)

```

---

## 四、GGUF 格式

### 4.1 概述

GGUF 是 llama.cpp 定义的单文件模型格式，支持多种量化精度。vLLM 通过 gguf.py 提供完整的 GGUF 推理支持。

### 4.2 支持的量化类型

gguf.py:166-198:

```

UNQUANTIZED_TYPES = {WeightType.F32, WeightType.F16, WeightType.BF16}
STANDARD_QUANT_TYPES = {
    WeightType.Q4_0, WeightType.Q4_1,      # 标准 4-bit
    WeightType.Q5_0, WeightType.Q5_1,      # 标准 5-bit
    WeightType.Q8_0, WeightType.Q8_1,      # 标准 8-bit
}
KQUANT_TYPES = {                               # K-quantization (改进版)
    WeightType.Q2_K, WeightType.Q3_K,
    WeightType.Q4_K, WeightType.Q5_K, WeightType.Q6_K,
}
IMATRIX_QUANT_TYPES = {                       # I-Matrix 量化 (重要性矩阵)
    WeightType.IQ1_M, WeightType.IQ1_S,
    WeightType.IQ2_XXS, WeightType.IQ2_XS, ...
}

```

### 4.3 GGUF GEMM 操作

gguf.py:201-233 中的 `_fused_mul_mat_gguf` 实现了三级内核选择:

```

def _fused_mul_mat_gguf(x, qweight, qweight_type):
    # 1. MMVQ (向量-矩阵量化乘法): 适合小 batch (batch_size <= 2~16)
    if x.shape[0] <= mmvq_safe and qweight_type in MMVQ_QUANT_TYPES:
        y = ops.ggml_mul_mat_vec_a8(qweight, x, qweight_type, ...)
    # 2. MMQ (矩阵-矩阵量化乘法): 标准批量推理
    elif qweight_type in MMQ_QUANT_TYPES:
        y = ops.ggml_mul_mat_a8(qweight, x, qweight_type, ...)
    # 3. 反量化回退: 无专用 kernel 时先反量化再 matmul
    elif qweight_type in DEQUANT_TYPES:
        weight = ops.ggml_dequantize(qweight, qweight_type, ...)
        y = x @ weight.T

```

### 4.4 GGUF MoE 支持

gguf.py:564-669 的 `GGUFMoEMethod`:



- 当两个权重都支持 MMQ 且 `x.shape[0] > 64` 时使用 fused MoE kernel (`ops.ggml_moe_a8`)
- 否则使用逐 expert 的向量 kernel (`ops.ggml_moe_a8_vec`)
- 最终回退到逐 token 逐 expert 的慢速路径

## 4.5 GGUF 工具函数

gguf\_utils.py 提供: - `check_gguf_file()` - 检测文件是否为 GGUF 格式 - `is_remote_gguf()` - 检测远程 GGUF 模型 (如 `repo_id:Q4_K_M` 格式) - `is_valid_gguf_quant_type()` - 校验量化类型名称合法性

## 五、NVFP4 Blackwell 4-bit 浮点

### 5.1 概述

NVFP4 是 NVIDIA Blackwell 架构 (SM100+, 如 B200 GPU) 引入的原生 4-bit 浮点格式 (`float4_e2m1fn`), 具有以下特征:

- 数据类型: `float4_e2m1fn`, 2 个 FP4 值打包在一个 `uint8` 中
- 缩放方式: FP8-E4M3FN 格式的 block scale, 默认 group size = 16
- 全局缩放: 额外的标量全局缩放因子用于权重和激活

### 5.2 数据结构

`nvfp4/base.py:10-19`:

```
@dataclass
class NvFp4LinearLayerConfig:
    """ 所有 NVFP4 层共享相同结构:
    - packed uint8 权重 (每字节 2 个 FP4 值)
    - FP8-E4M3 per-block 权重缩放 (group size 16)
    - 权重和激活的全局标量缩放
    """
    pass
```

### 5.3 量化 Key 定义

`quant_utils.py:138-146`:

```
# 动态 NVFP4: 激活侧动态计算 FP8 block scale
kNvfp4DynamicGroupScale = ScaleDesc(FP8_DTYPE, False, GroupShape(1, 16))
kNvfp4Dynamic = QuantKey(FP4_DTYPE, scale=kNvfp4DynamicGroupScale,
                          scale2=kStaticTensorScale)

# 静态 NVFP4: 预计算的 FP8 block scale
kNvfp4StaticGroupScale = ScaleDesc(FP8_DTYPE, True, GroupShape(1, 16))
kNvfp4Static = QuantKey(FP4_DTYPE, scale=kNvfp4StaticGroupScale,
                        scale2=kStaticTensorScale)
```

### 5.4 NVFP4 内核后端

从 `linear/init.py:77-80` 可见注册的后端:

| 内核类                                   | 文件                            | 说明                        |
|---------------------------------------|-------------------------------|---------------------------|
| <code>CutlassNvFp4LinearKernel</code> | <code>nvfp4/cutlass.py</code> | CUTLASS 实现 (Blackwell 原生) |
| <code>MarlinNvFp4LinearKernel</code>  | <code>nvfp4/marlin.py</code>  | Marlin 内核适配               |

| 内核类                         | 文件                  | 说明                 |
|-----------------------------|---------------------|--------------------|
| FlashInferNvFp4LinearKernel | nvfp4/flashinfer.py | FlashInfer 后端      |
| FBGemmNvFp4LinearKernel     | nvfp4/fbgemm.py     | FBGEMM (ROCm)      |
| EmulationNvFp4LinearKernel  | nvfp4/emulation.py  | 仿真模式 (非 Blackwell) |

## 5.5 NVFP4 工具函数

nvfp4\_utils.py 提供关键工具:

```
def swizzle_blockscale(scale: torch.Tensor) -> torch.Tensor:
    """Pad 并 block-interleave FP4 block-scale 以匹配 CUTLASS/FlashInfer 布局"""
    # reshape -> permute -> contiguous 匹配内核期望的数据排布

def pad_nvfp4_weight_for_cutlass(weight, alignment=32):
    """ 填充 NVFP4 权重以满足 CUTLASS 的 32 对齐要求 """

def cutlass_fp4_supported() -> bool:
    """ 检测当前设备是否支持 CUTLASS FP4 """
```

## 六、MXFP4 / MXFP8 块缩放浮点

### 6.1 MXFP4 (MicroScaling FP4)

MXFP4 是面向 MoE 模型的 4-bit 微缩放浮点格式, 主要服务于 GPT-OSS 和 DeepSeek-V4 等模型。

mxfp4.py:42-101:

```
class Mxvp4Config(QuantizationConfig):
    @classmethod
    def get_min_capability(cls) -> int:
        return 80 # 需要 Ampere (SM80) 及以上

    @classmethod
    def get_supported_act_dtypes(cls) -> list[torch.dtype]:
        return [torch.bfloat16]

    def get_quant_method(self, layer, prefix):
        if isinstance(layer, LinearBase):
            # Linear 层暂未实现 MXFP4, 回退到 Unquantized
            return UnquantizedLinearMethod()
        elif isinstance(layer, FusedMoE):
            return GptOssMxvp4MoEMethod(layer.moe_config)
```

**MXFP4 权重布局:** - w13\_weight: 形状 [num\_experts, 2\*intermediate, hidden//2], dtype uint8 - 每 2 个 FP4 值打包在 1 个字节中 - w13\_weight\_scale: 形状 [num\_experts, 2\*intermediate, hidden//32], block size = 32

**MXFP4 后端选择** (通过 select\_mxvp4\_moe\_backend()) : - AITER\_MXFP4\_BF16 —AITER 后端 - FLASHINFER\_TRITLLM\_MXFP4\_MXFP8 —FlashInfer TRITLLM (支持 padding skip) - TRITON\_MXFP4\_BF16 — Triton kernel

### 6.2 MXFP8 (MicroScaling FP8)

MXFP8 每 32 个元素共享一个 uint8 缩放因子, 是 DeepSeek-V3 等模型的关键量化技术。

量化 Key 定义于 quant\_utils.py:154-158:

```
kMxfp8StaticScale = ScaleDesc(torch.uint8, True, GroupShape(1, 32))
kMxfp8Static = QuantKey(FP8_DTYPE, kMxfp8StaticScale, symmetric=True)
kMxfp8DynamicScale = ScaleDesc(torch.uint8, False, GroupShape(1, 32))
kMxfp8Dynamic = QuantKey(FP8_DTYPE, kMxfp8DynamicScale, symmetric=True)
```

MXFP8 专用内核位于 kernels/linear/mxfp8/ 目录: - MarlinMxfp8LinearKernel —Marlin 加速 - FlashInferCutlassMxfp8LinearKernel —FlashInfer + CUTLASS - EmulationMxfp8LinearKernel —仿真回退

## 七、Marlin 高性能 4-bit 内核

### 7.1 概述

Marlin 是专为 4-bit 权重量化设计的超高性能 GEMM 内核, vLLM 将其作为 GPTQ/AWQ/FP8 的首选加速后端。

### 7.2 支持的量化类型

marlin\_utils.py:41-79:

```
def query_marlin_supported_quant_types(has_zp=None, include_fp_type=True, ...):
    """
    has_zp=True (AWQ 风格): [scalar_types.uint4]
    has_zp=False (GPTQ 风格): [uint4b8, uint8b128] (+ FP8 types if include_fp_type)
    """
```

完整支持列表: - 整数型: uint4 (AWQ)、uint4b8 (GPTQ 4-bit)、uint8b128 (GPTQ 8-bit) - 浮点型: float8\_e4m3fn (FP8 权重)、float4\_e2m1f (FP4 权重)

### 7.3 Marlin 约束条件

marlin\_utils.py:25-35:

```
GPTQ_MARLIN_TILE = 16          # Marlin tile 尺寸
GPTQ_MARLIN_MIN_THREAD_N = 64  # 输出维度最小线程数
GPTQ_MARLIN_MIN_THREAD_K = 128 # 输入维度最小线程数
MARLIN_SUPPORTED_GROUP_SIZES = [-1, 32, 64, 128] # 支持的分组大小
```

形状约束验证函数 verify\_marlin\_supports\_shape() 检查: - output\_size % 64 == 0 - input\_size % 128 == 0 - 如果 group\_size < input\_size: input\_size % group\_size == 0

### 7.4 Marlin 权重预处理

Marlin 内核要求权重经过特殊的重排 (reordering) 和置换 (permutation):

```
# marlin_utils.py:292-312 — scale permutation
def marlin_permute_scales(s, size_k, size_n, group_size, is_a_8bit=False):
    scale_perm, scale_perm_single = get_scale_perms()
    # group quant: 使用 8x8 full permutation
    # channel quant: 使用 4x8 single permutation
    s = s.reshape((-1, len(scale_perm)))[:, scale_perm]
    return s.reshape((-1, size_n)).contiguous()

# marlin_utils.py:344-365 — zero-point permutation
def marlin_zero_points(zp, size_k, size_n, num_bits, is_a_8bit=False):
    # 1. 应用 scale permutation
    # 2. interleave 列维度
    # 3. pack 到 int32
```

## 7.5 Marlin Workspace

marlin\_utils.py:257-265:

```
def marlin_make_workspace_new(device, max_blocks_per_sm=1):
    """ 创建 Marlin workspace tensor
    大小 = SM 数量 × max_blocks_per_sm
    用于原子操作的同步
    """
    sms = num_compute_units(device.index)
    return torch.zeros(sms * max_blocks_per_sm, dtype=torch.int, device=device)
```

## 7.6 Marlin GEMM 调用

marlin\_utils.py:506-570:

```
def apply_gptq_marlin_linear(input, weight, weight_scale, weight_zp,
                             g_idx, g_idx_sort_indices, workspace,
                             wtype, ..., input_dtype=None):
    # 可选: W4A8 输入量化
    if input_dtype == torch.int8:
        reshaped_x, a_scales = marlin_quant_input(reshaped_x, torch.int8)
    elif input_dtype == torch.float8_e4m3fn:
        reshaped_x, a_scales = marlin_quant_input(reshaped_x, torch.float8_e4m3fn)

    output = ops.marlin_gemm(
        reshaped_x, None, weight, bias,
        weight_scale, a_scales, None,
        weight_zp, g_idx, g_idx_sort_indices, workspace,
        wtype, size_m=..., size_n=..., size_k=...,
        is_k_full=..., use_atomic_add=..., use_fp32_reduce=...,
    )
```

## 7.7 W4A8 扩展: INT8 / FP8 激活量化

Marlin 不仅支持 W4A16, 还扩展支持低精度激活:

```
# marlin_utils.py:474-493
def get_marlin_input_dtype(prefix=None):
    """ 通过环境变量 VLLM_MARLIN_INPUT_DTYPE 控制:
    - None: 标准 W4A16
    - "int8": W4A8-INT8 (需 SM75+)
    - "fp8": W4A8-FP8 (仅 SM89 H20 或 SM120 Blackwell)
    """
```

---

## 八、Machete 新一代量化内核

### 8.1 概述

Machete 是基于 CUTLASS 的新一代量化 GEMM 内核, 专门针对 NVIDIA Hopper (SM90) 架构优化, 提供比 Marlin 更高的吞吐量。

### 8.2 Machete 约束

machete\_utils.py:9-56:

```
MACHETE_PREPACKED_BLOCK_SHAPE = [64, 128] # 预打包块形状

def query_machete_supported_quant_types(zero_points):
    if zero_points:
        return [scalar_types.uint4, scalar_types.uint8]
    else:
        return [scalar_types.uint4b8, scalar_types.uint8b128]

def query_machete_supported_group_sizes(act_type):
    if act_type in [torch.float16, torch.bfloat16]:
        return [-1, 64, 128] # 支持 channel-wise / 64 / 128
    else:
        return [-1, 128]

def check_machete_supports_shape(in_features, out_features):
    # in_features % 64 == 0
    # out_features % 128 == 0
```

8.3 MacheteLinearKernel

mixed\_precision/machete.py:24-80:

```
class MacheteLinearKernel(MPLinearKernel):
    @classmethod
    def get_min_capability(cls) -> int:
        return 90 # 仅支持 Hopper (SM90)

    @classmethod
    def can_implement(cls, c: MPLinearLayerConfig):
        # 1. 必须是 CUDA 平台
        # 2. 必须是 SM90
        # 3. 不支持 act_order + TP 分区
        # 4. 量化类型必须在支持列表中
        # 5. group_size 必须在支持列表中
        # 6. 形状必须满足 64/128 对齐
```

8.4 Marlin vs Machete 对比

| 特性        | Marlin         | Machete        |
|-----------|----------------|----------------|
| 最低架构      | SM 75 (Turing) | SM 90 (Hopper) |
| 底层实现      | 自定义 CUDA       | CUTLASS        |
| 对齐要求      | N%64, K%128    | N%128, K%64    |
| 支持 ZP     | (uint4)        | (uint4, uint8) |
| Act Order |                | (TP 分区时)       |
| FP8 权重    |                |                |
| W4A8      | (INT8/FP8)     |                |
| 典型场景      | 通用 4-bit 推理    | Hopper 高吞吐     |

九、量化线性层与 GEMM 操作

9.1 内核选择体系

vLLM 的量化 GEMM 内核采用分层选择架构，定义于 kernels/linear/ 目录:

```

graph TB
    INIT["init_fp8_linear_kernel()"] --> SS["ScaledMM 子系统"]
    INIT --> INIT_NVFP4["init_nvfp4_linear_kernel()"]
    INIT --> INIT_MXFP8["init_mxfp8_linear_kernel()"]

    SS --> PY["PyTorch _scaled_mm"]
    SS --> CU["CutlassFP8ScaledMM"]
    SS --> FI["FlashInferFP8"]
    SS --> MR["MarlinFP8ScaledMM"]
    SS --> BS["Fp8BlockScaledMM"]
    SS --> DG["DeepGEMM"]
    SS --> AI["AiterInt8"]
    SS --> TR["TritonInt8"]
    SS --> ROCm["ROCmFP8"]
    SS --> CPU["CPUFp8BlockScaled"]

    MP["choose_mp_linear_kernel()"] --> ML["MarlinLinearKernel"]
    MP --> MC["MacheteLinearKernel"]
    MP --> EX["ExllamaLinearKernel"]
    MP --> CO["ConchLinearKernel"]
    MP --> AS["AllSparkLinearKernel"]
    MP --> CU_W4["CutlassW4A8LinearKernel"]
    MP --> DY4["Dynamic4bitLinearKernel"]
    MP --> TR4["TritonW4A16LinearKernel"]
    MP --> CPU_WNA["CPUWNA16LinearKernel"]

    style INIT fill:#fff9c4,stroke:#f9a825
    style SS fill:#b3e5fc,stroke:#0277bd
    style MP fill:#e1bee7,stroke:#8e24aa

```

## 9.2 ScaledMM 内核层次

scaled\_mm/init.py 注册了以下 FP8/INT8 缩放矩阵乘法内核:

| 内核类                                     | 精度         | 后端             | 条件                  |
|-----------------------------------------|------------|----------------|---------------------|
| PerTensorTorchFP8ScaledMMLinearKernel   | W8A8       | PyTorch native | 通用 fallback         |
| ChannelWiseTorchFP8ScaledMMLinearKernel | W8A8       | PyTorch native | per-channel         |
| RowWiseTorchFP8ScaledMMLinearKernel     | W8A8       | PyTorch native | per-row             |
| CutlassFP8ScaledMMLinearKernel          | W8A8       | CUTLASS        | Ampere+ CUDA        |
| CutlassInt8ScaledMMLinearKernel         | W8A8       | CUTLASS        | Ampere+ CUDA        |
| FlashInferFP8ScaledMMLinearKernel       | W8A8       | FlashInfer     | Hopper+             |
| MarlinFP8ScaledMMLinearKernel           | W8A8       | Marlin         | Turing+, 非 FP8 原生设备 |
| Fp8BlockScaledMMLinearKernel            | W8A8 Block | CUTLASS        | 块级缩放                |
| ROCmFP8ScaledMMLinearKernel             | W8A8       | ROCm           | AMD GPU             |
| AiterInt8ScaledMMLinearKernel           | W8A8       | AITER          | 华为昇腾                |
| TritonInt8ScaledMMLinearKernel          | W8A8       | Triton         | 通用                  |
| CPUFp8BlockScaledKernel                 | W8A8       | CPU            | CPU 推理              |

## 9.3 Mixed Precision 内核层次

mixed\_precision/ 目录包含整数量化内核:

| 内核类                     | 精度          | 后端          | 说明            |
|-------------------------|-------------|-------------|---------------|
| MarlinLinearKernel      | W4A16/W8A16 | Marlin CUDA | 主要 4/8-bit 内核 |
| MacheteLinearKernel     | W4A16/W8A16 | CUTLASS     | SM90 优化       |
| ExllamaLinearKernel     | W4A16       | Exllama     | GPTQ 原生格式     |
| ConchLinearKernel       | W4A16/W8A16 | Conch       | DeepSeek 内核   |
| AllSparkLinearKernel    | W4A16       | AllSpark    | 通义千问          |
| CutlassW4A8LinearKernel | W4A8        | CUTLASS     | W4A8 精度       |
| Dynamic4bitLinearKernel | W4A16       | Dynamic     | 动态 4-bit      |
| TritonW4A16LinearKernel | W4A16       | Triton      | 可移植           |
| CPUWNA16LinearKernel    | W4A16       | CPU         | CPU 推理        |

## 十、量化配置体系

### 10.1 基类层次

```
QuantizationConfig (ABC)          # base_config.py:70
    get_name() → QuantizationMethods
    get_supported_act_dtypes() → list[dtype]
    get_min_capability() → int      # 最低 GPU compute capability
    get_config_filenames() → list[str]
    from_config(dict) → Self
    get_quant_method(layer, prefix) → QuantizeMethodBase | None
```

```
QuantizeMethodBase (ABC)          # base_config.py:19
    create_weights(layer, ...)      # 创建量化参数
    apply(layer, x, bias) → Tensor  # 前向计算
    embedding(layer, x) → Tensor    # Embedding 查找 (可选)
    process_weights_after_loading(layer) # 权重后处理 (可选)
```

源码位置: base\_config.py

### 10.2 配置类清单

| 配置类                     | 方法名                | 权重位数                         | 最低能力   | 文件                  |
|-------------------------|--------------------|------------------------------|--------|---------------------|
| Fp8Config               | fp8                | 8-bit float                  | SM 75  | fp8.py              |
| GPTQConfig              | gptq               | 2/3/4/8-bit int              | SM 60  | gptq.py             |
| GPTQMarlinConfig        | gptq_marlin        | 4/8-bit int                  | SM 75  | gptq_marlin.py      |
| AWQConfig               | awq                | 4-bit int                    | SM 75  | awq.py              |
| AWQMarlinConfig         | awq_marlin         | 4-bit int                    | SM 75  | awq_marlin.py       |
| GGUFConfig              | gguf               | 多精度                          | SM 60  | gguf.py             |
| ModelOptFp8Config       | modelopt           | 8-bit float                  | SM 75  | modelopt.py         |
| ModelOptNvFp4Config     | modelopt_fp4       | 4-bit float                  | SM 100 | modelopt.py         |
| ModelOptMxPp8Config     | modelopt_mxfp8     | 8-bit float + uint8<br>scale | SM 80  | modelopt.py         |
| Mxfp4Config             | mxfp4              | 4-bit float (uint8)          | SM 80  | mxfp4.py            |
| BitsAndBytesConfig      | bitsandbytes       | 多精度                          | —      | bitsandbytes.py     |
| CompressedTensorsConfig | compressed-tensors | 多精度                          | —      | compressed_tensors/ |
| INCCConfig              | inc / auto-round   | 多精度                          | —      | inc.py              |
| TorchAOConfig           | torchao            | 多精度                          | —      | torchao.py          |
| HummingConfig           | humming            | 多精度                          | —      | humming.py          |
| ExpertsInt8Config       | experts_int8       | 8-bit int                    | —      | experts_int8.py     |

| 配置类                      | 方法名 | 权重位数 | 最低能力 | 文件      |
|--------------------------|-----|------|------|---------|
| OnlineQuantizationConfig |     | 动态   | —    | online/ |

10.3 参数类型体系

vLLM 定义了丰富的参数类型来描述不同量化格式的权重布局:

| 参数类型                       | 用途               | 来源              |
|----------------------------|------------------|-----------------|
| ModelWeightParameter       | 全精度权重            | parameter.py    |
| PerTensorScaleParameter    | Per-tensor 缩放因子  | parameter.py    |
| BlockQuantScaleParameter   | Block 量化缩放因子     | parameter.py    |
| ChannelQuantScaleParameter | Per-channel 缩放因子 | parameter.py    |
| GroupQuantScaleParameter   | Per-group 缩放因子   | parameter.py    |
| PackedvLLMParameter        | 打包的量化权重 (int32)  | parameter.py    |
| PackedColumnParameter      | 沿列打包的零点          | parameter.py    |
| RowvLLMParameter           | 行索引参数 (g_idx)    | parameter.py    |
| GGUFUninitializedParameter | GGUF 延迟初始化参数     | gguf.py:689-691 |

10.4 自动格式检测与覆盖

部分配置类实现了 `override_quantization_method()` 来自动检测 checkpoint 格式并推荐最优内核:

```
# gptq_marlin.py:220-245  —自动升级 GPTQ → GPTQ-Marlin
@classmethod
def override_quantization_method(cls, hf_quant_cfg, user_quant, hf_config=None):
    can_convert = cls.is_gptq_marlin_compatible(hf_quant_cfg)
    if can_convert and user_quant in (None, "marlin", "gptq_marlin"):
        return "gptq_marlin" # 自动切换到更快的 Marlin 内核

# awq_marlin.py:233-262  —自动升级 AWQ → AWQ-Marlin
@classmethod
def override_quantization_method(cls, hf_quant_cfg, user_quant, hf_config=None):
    can_convert = cls.is_awq_marlin_compatible(hf_quant_cfg)
    if can_convert and user_quant in (None, "marlin", "awq_marlin"):
        return "awq_marlin"

# gguf.py:86-93  —强制 GGUF 格式
@classmethod
def override_quantization_method(cls, hf_quant_cfg, user_quant, hf_config=None):
    if user_quant == "gguf":
        return "gguf" # 覆盖 HF config 中的其他量化设置
```

十一、量化方案选择决策树

```
graph TD
    START[" 开始选择量化方案"] --> Q1{"Checkpoint 格式?"}

    Q1 -->| "FP8 序列化" | FP8_OFFLINE["Fp8Config<br/>Fp8LinearMethod / Fp8MoEMethod"]
    Q1 -->| "FP16/BF16 全精度" | Q2{" 目标 GPU?"}
    Q1 -->| "GPTQ 格式" | Q3{" 可用 Marlin?"}
    Q1 -->| "AWQ 格式" | Q4{" 可用 Marlin?"}
```



```

Q1 -->|"GGUF 格式"| GGUF_R["GGUFConfig<br/>GGUFLinearMethod / GGUFMoEMethod"]
Q1 -->|"MXFP4/MXFP8"| MXFP["Mxfp4Config / ModelOptMxPp8Config"]
Q1 -->|"NVFP4"| NVFP4_R["ModelOptNvFp4Config<br/>NvFp4LinearKernel"]

Q2 -->|"H100/H200 (SM90)"| FP8_ONLINE["Fp8Config (online)<br/>Fp8OnlineLinearMethod"]
Q2 -->|" 其他 GPU"| Q2B{" 需要量化?"}
Q2B -->|" 否"| UNQUANT["UnquantizedLinearMethod"]
Q2B -->|" 是, 用户指定"| USER_Q[" 用户指定的量化方案"]

Q3 -->|" 是 (SM75+)"| GPTQ_MARLIN["GPTQMarlinConfig<br/>GPTQMarlinLinearMethod"]
Q3 -->|" 否"| GPTQ_BASE["GPTQConfig<br/>GPTQLinearMethod (gptq_gemm)"]

Q4 -->|" 是 (SM75+)"| AWQ_MARLIN["AWQMarlinConfig<br/>AWQMarlinLinearMethod"]
Q4 -->|" 否"| AWQ_BASE["AWQConfig<br/>AWQLinearMethod (awq_gemm)"]

FP8_OFFLINE --> Q5{" 选择 FP8 内核后端"}
FP8_ONLINE --> Q5
GPTQ_MARLIN --> Q6{" 选择 Marlin 变体"}
AWQ_MARLIN --> Q6

Q5 -->|"H100 原生 FP8"| NATIVE["_scaled_mm / CutlassFP8"]
Q5 -->|" 非原生 FP8"| MARLIN_FP8["MarlinFP8ScaledMM"]
Q5 -->|"DeepGEMM 可用"| DEEPGEMM["DeepGEMM kernel"]
Q5 -->|" 块级量化"| BLOCK_BS["Fp8BlockScaledMM"]

Q6 -->|"SM90 Hopper"| MACHETE_SEL["MacheteLinearKernel (优先)"]
Q6 -->|"SM75-SM89"| MARLIN_SEL["MarlinLinearKernel"]

style START fill:#fff9c4,stroke:#f9a825
style FP8_OFFLINE fill:#b3e5fc,stroke:#0277bd
style FP8_ONLINE fill:#b3e5fc,stroke:#0277bd
style GPTQ_MARLIN fill:#e1bee7,stroke:#8e24aa
style AWQ_MARLIN fill:#e1bee7,stroke:#8e24aa
style GGUF_R fill:#c8e6c9,stroke:#2e7d32
style NVFP4_R fill:#ffccbc,stroke:#e65100

```

---

## 十二、量化数据流全景

graph LR

```

    subgraph Load[" 权重加载"]
        CKPT["Checkpoint 文件 <br/>(SafeTensors/GGUF/HF)"]
        WL["WeightLoader<br/> 分片 & 加载"]
        CKPT --> WL
    end

    subgraph Create[" 创建参数"]
        CW["create_weights()<br/> 分配量化参数"]
        WL --> CW
    end

    subgraph Process[" 权重后处理"]
        PW["process_weights_after_loading()<br/>• 格式转换 <br/>• 权重重排 <br/>• 缩放因子计算"]
        CW --> PW
    end

```

```

end

subgraph Infer[" 推理执行"]
  AQ[" 激活量化 (动态方案)<br/>scaled_fp8_quant()"]
  GEMM[" 量化 GEMM<br/>• ops.marlin_gemm<br/>• ops.gptq_gemm<br/>• ops.awq_gemm<br/>• torch._scale
  DQ[" 反量化 / 输出"]
  PW --> AQ
  AQ --> GEMM
  GEMM --> DQ
end

subgraph Kernels[" CUDA 内核"]
  MARLIN_K["marlin_gemm.cu"]
  GPTQ_K["gptq_gemm.cu"]
  AWQ_K["awq_gemm.cu / awq_dequantize"]
  GGUF_K["ggml_dequantize / ggml_mul_mat_a8"]
  NVFP4_K["cutlass_fp4 GEMM"]
  GEMM --> MARLIN_K
  GEMM --> GPTQ_K
  GEMM --> AWQ_K
  GEMM --> GGUF_K
  GEMM --> NVFP4_K
end

style Load fill:#e3f2fd,stroke:#1565c0
style Create fill:#fce4ec,stroke:#c62828
style Process fill:#f3e5f5,stroke:#7b1fa2
style Infer fill:#e8f5e9,stroke:#2e7d32
style Kernels fill:#fff3e0,stroke:#ef6c00

```

#### 典型数据流示例

##### 示例 1: FP8 在线量化 (DeepSeek 风格)

###### BF16 Checkpoint

```

↓ [WeightLoader: 加载到 meta device]
↓ [Fp8OnlineLinearMethod.create_weights(): 创建 FP8 参数]
↓ [Fp8OnlineLinearMethod.process_weights_after_loading():
  ops.scaled_fp8_quant(weight) → (qweight: FP8, scale: FP32)
  weight.t() → [K, N] 布局
]
↓ [推理时每个 token:]
↓ [动态计算激活 scale]
↓ [torch._scaled_mm(x_fp8, w_fp8, x_scale, w_scale) → output]

```

##### 示例 2: GPTQ-Marlin 4-bit 量化

###### GPTQ Checkpoint (int32 packed weights)

```

↓ [GPTQMarlinLinearMethod.create_weights():
  qweight: [K//8, N] int32 (packed uint4)
  scales: [K//gs, N] fp16
  qzeros: [K//gs, N//8] int32 (packed)
  g_idx: [K] int32
]
↓ [process_weights_after_loading():

```

```

    g_idx sorting & argsort
    ops.gptq_shuffle(qweight, g_idx, 4)
    marlin_permute_scales(scales)
    marlin_zero_points(qzeros)
    marlin_make_workspace_new(device)
]
↓ [推理时:]
↓ [ops.marlin_gemm(x, qweight, scales, zeros, g_idx, ...)]

```

### 示例 3: GGUF Q4\_K\_M 量化

```

GGUF File (.gguf)
↓ [gguf_utils.py: 加载元数据 + 权重]
↓ [GGUFLinearMethod.create_weights():
    qweight: GGUFUninitializedParameter (延迟初始化)
    qweight_type: WeightType.Q4_K (uint8 标识)
]
↓ [process_weights_after_loading():
    检查量化类型有效性
    为融合层 (QKV/gate_up) 创建 padded weight
]
↓ [推理时 (_fused_mul_mat_gguf):]
↓ [batch_size threshold?]
↓ Yes → ops.ggml_mul_mat_vec_a8() [MMVQ 向量 kernel]
↓ No → ops.ggml_mul_mat_a8() [MMQ 矩阵 kernel]
↓ No kernel → ops.ggml_dequantize() + matmul [反量化回退]

```

### 附录: 关键源码路径索引

| 类别          | 路径                                 | 说明                                             |
|-------------|------------------------------------|------------------------------------------------|
| 量化配置入口      | config/quantization.py             | OnlineQuantScheme, resolve_online_quant_config |
| 量化方法注册      | layers/quantization/init.py        | QuantizationMethods, get_quantization_config   |
| 基类定义        | layers/quantization/base_config.py | QuantizationConfig, QuantizeMethodBase         |
| Schema 定义   | layers/quantization/schema.py      | KVCacheQuantSchema                             |
| FP8 量化      | layers/quantization/fp8.py         | Fp8Config, Fp8LinearMethod, Fp8MoEMethod       |
| GPTQ 量化     | layers/quantization/gptq.py        | GPTQConfig, GPTQLinearMethod                   |
| GPTQ-Marlin | layers/quantization/gptq_marlin.py | GPTQMarlinConfig, GPTQMarlinLinearMethod       |
| AWQ 量化      | layers/quantization/awq.py         | AWQConfig, AWQLinearMethod                     |
| AWQ-Marlin  | layers/quantization/awq_marlin.py  | AWQMarlinConfig, AWQMarlinLinearMethod         |
| GGUF 量化     | layers/quantization/gguf.py        | GGUFConfig, GGUFLinearMethod, GGUFMoEMethod    |
| GGUF 工具     | transformers_utils/gguf_utils.py   | GGUF 文件检测与解析                                   |
| MXFP4 量化    | layers/quantization/mx4p4.py       | Mx4p4Config, GptOssMx4p4MoEMethod              |

| 类别                 | 路径                                         | 说明                                                 |
|--------------------|--------------------------------------------|----------------------------------------------------|
| ModelOpt 系列        | layers/quantization/modelopt.py            | ModelOptFp8/NvFp4/MxFp8/Mixed Configs              |
| Marlin 工具          | layers/quantization/utils/marlin_utils.py  | Marlin 格式转换、shape 验证、workspace                     |
| Machete 工具         | layers/quantization/utils/machete_utils.py | Machete 约束查询                                       |
| NVFP4 工具           | layers/quantization/utils/nvfp4_utils.py   | NVFP4 swizzle/pad 工具                               |
| 量化工具集              | layers/quantization/utils/quant_utils.py   | QuantKey, GroupShape, scaled_quantize, pack/unpack |
| FP8 工具             | layers/quantization/utils/fp8_utils.py     | FP8 参数创建与处理                                        |
| Linear 内核总入口       | kernels/linear/init.py                     | 所有 linear kernel 的统一导出                             |
| ScaledMM 内核        | kernels/linear/scaled_mm/                  | FP8/INT8 缩放矩阵乘法内核族                                 |
| Mixed Precision 内核 | kernels/linear/mixed_precision/            | 整数量化内核族 (Marlin/Machete/Exllama...)                |
| MXFP8 内核           | kernels/linear/mx_fp8/                     | MXFP8 专用内核                                         |
| NVFP4 内核           | kernels/linear/nvfp4/                      | NVFP4 专用内核                                         |
| 在线量化               | layers/quantization/online/                | Online Quantization 实现                             |

## vLLM 分布式计算分析

定位：本文档深入分析 vLLM 分布式计算架构，涵盖并行策略、状态管理、通信原语、数据传输机制及高级弹性特性。

### graph TB

```

subgraph PS[" 并行策略 Parallel Strategies"]
    TP["Tensor Parallelism<br/> 张量并行"]
    PP["Pipeline Parallelism<br/> 流水线并行"]
    DP["Data Parallelism<br/> 数据并行"]
    EP["Expert Parallelism<br/> 专家并行 MoE"]
end

subgraph SM[" 并行状态管理 State Management"]
    GC["GroupCoordinator<br/> 通信组协调器"]
    PG["ProcessGroup<br/>NCCL / Gloo 通信域"]
    SG["StatelessGroupCoordinator<br/> 无状态协调器"]
end

subgraph DC[" 设备通信器 Device Communicators"]
    BASE["DeviceCommunicatorBase<br/> 基类"]
    CUDA["CudaCommunicator<br/>CUDA 通信"]
    PYNCCL["PyNcclCommunicator<br/>PyNCL NCCL 包装"]
    CAR["CustomAllreduce<br/> 自定义 AllReduce"]
    QAR["QuickAllReduce<br/> 快速 AllReduce ROCm"]
    SHM["MessageQueue<br/> 共享内存广播"]
    RAY["RayPPCommunicator<br/>Ray 通信"]
    CPU_COMM["CpuCommunicator<br/>CPU 通信"]
    XPU["XpuCommunicator<br/>XPU 通信"]
end

subgraph DT[" 数据传输 Data Transfer"]
    KV["KV Transfer<br/>KV 缓存跨节点传输"]
    WT["Weight Transfer<br/> 权重分发 NCCL/IPC"]
    EC_T["EC Transfer<br/> 纠删码传输"]

```

```

end

PS --> SM
SM --> DC
DC --> DT

style PS fill:#e8eaf6,stroke:#5e35b1,color:#1a237e
style SM fill:#fff3e0,stroke:#e65100,color:#bf360c
style DC fill:#e0f2f1,stroke:#00695c,color:#004d40
style DT fill:#fce4ec,stroke:#c62828,color:#880e4f

```

---

## 目录

- 一、并行策略
    - 1.1 Tensor Parallelism (TP) —张量并行
    - 1.2 Pipeline Parallelism (PP) —流水线并行
    - 1.3 Data Parallelism (DP) —数据并行
    - 1.4 Expert Parallelism (EP) —专家并行
  - 二、并行状态管理
    - 2.1 GroupCoordinator 通信组协调器
    - 2.2 并行组初始化流程
    - 2.3 NCCL 通信域建立
    - 2.4 StatelessGroupCoordinator 无状态协调器
  - 三、设备通信器 Device Communicators
    - 3.1 基类 DeviceCommunicatorBase
    - 3.2 CudaCommunicator —CUDA 通信器
    - 3.3 PyNcclCommunicator —PyNCL NCCL 包装
    - 3.4 CustomAllreduce —自定义 AllReduce
    - 3.5 QuickAllReduce —快速 AllReduce (ROCm)
    - 3.6 MessageQueue —共享内存广播
    - 3.7 Ray/CPU/XPU 通信器
    - 3.8 All2AllManager —MoE 全对全通信
  - 四、数据传输机制
    - 4.1 KV Transfer —跨节点 KV 缓存传输
    - 4.2 Weight Transfer —权重分发
    - 4.3 EC Transfer —纠删码传输
  - 五、高级特性
    - 5.1 Elastic EP —弹性专家并行
    - 5.2 EPLB —弹性负载均衡
    - 5.3 Stateless Coordinator —无状态协调器
    - 5.4 NIXL Utils —NIXL 工具集
- 

## 一、并行策略

vLLM 采用多维混合并行策略，将模型计算分布到多个 GPU/节点上。其核心布局顺序为：

**ExternalDP × DP × PP × PCP × TP**

其中 ExternalDP 为外部数据并行（如 verl 集成），DP 为耦合数据并行，PP 为流水线并行，PCP 为 Prefill Context Parallel，TP 为张量并行。

### 1.1 Tensor Parallelism (TP) 一张量并行

原理：将模型参数沿行或列切分到多个 GPU 上，每个 GPU 只持有模型的一部分参数。前向传播时，各 GPU 通过 **AllReduce** 同步梯度/激活值。

典型场景：Attention 中的 Q/K/V 投影矩阵、MLP 中的线性层。

关键特征：- 每个 GPU 持有模型的不同切片 - 每层前向传播后需要 AllReduce 同步 - 通信密集，要求 GPU 间高带宽互联（NVLink / XGMI）

源码中的组构建 (parallel\_state.py:L1578-L1592):

```
# Build the tensor model-parallel groups.
global _TP
assert _TP is None, "tensor model parallel group is already initialized"
group_ranks = all_ranks.view(-1, tensor_model_parallel_size).unbind(0)
group_ranks = [x.tolist() for x in group_ranks]
_TP = init_model_parallel_group(
    group_ranks,
    get_world_group().local_rank,
    backend,
    use_message_queue_broadcaster=True,
    group_name="tp",
)
```

### 1.2 Pipeline Parallelism (PP) 一流水线并行

原理：将模型的各层（layer）分配到不同 GPU 上，每个 GPU 负责连续的若干层。数据按 stage 顺序流经各 GPU。

关键特征：- 各 GPU 持有不同层的完整参数（非切分） - 相邻 GPU 间通过 P2P Send/Recv 传递隐藏状态 - 支持 1F1B、interleaved 等调度策略 - 显存占用随 GPU 数线性降低

源码中的组构建 (parallel\_state.py:L1636-L1651):

```
# Build the pipeline model-parallel groups.
global _PP
assert _PP is None, "pipeline model parallel group is already initialized"
group_ranks = (
    all_ranks.transpose(2, 4).reshape(-1, pipeline_model_parallel_size).unbind(0)
)
group_ranks = [x.tolist() for x in group_ranks]
_PP = init_model_parallel_group(
    group_ranks, get_world_group().local_rank, backend, group_name="pp"
)
```

### 1.3 Data Parallelism (DP) 一数据并行

原理：每个 GPU 持有完整的模型副本，不同 GPU 处理不同的输入数据（batch chunk）。通常用于多副本推理以提升吞吐。

关键特征：- 各 GPU 独立执行前向传播 - 不需要层间通信同步（与 TP/PP 不同） - 在 vLLM 中 DP 通常与 TP/PP 组合使用（即 TP × PP × DP 的 3D/4D 并行）

源码中的组构建 (parallel\_state.py:L1653-L1668):

```
global _DP
assert _DP is None, "data parallel group is already initialized"
group_ranks = all_ranks.transpose(1, 4).reshape(-1, data_parallel_size).unbind(0)
group_ranks = [x.tolist() for x in group_ranks]
if enable_elastic_ep:
```

```

_DP = _init_stateless_group(group_ranks, "dp", ...)
else:
    _DP = init_model_parallel_group(
        group_ranks, get_world_group().local_rank, backend, group_name="dp"
    )

```

#### 1.4 Expert Parallelism (EP) —专家并行 (MoE 场景)

原理: 针对 Mixture-of-Experts (MoE) 模型, 将不同的专家 (Expert) 分配到不同 GPU 上。Router 决定 token 路由到哪些专家, 通过 All2All 通信将 token 发送到对应专家所在的 GPU。

关键特征: - 仅适用于 MoE 模型 (如 Mixtral、DeepSeek、Qwen-MoE 等) - EP 组由 DP 和 TP 组合而来:  $EP = DP \times TP$  - 通信模式为 All2All (dispatch + combine) - 支持冗余专家 (redundant experts) 实现负载均衡

源码中的组构建 (parallel\_state.py:L1670-L1696):

```

global _EP
assert _EP is None, "expert parallel group is already initialized"
if config.model_config is None or config.model_config.is_moe:
    group_ranks = (
        all_ranks.transpose(1, 2)
        .reshape(
            -1,
            data_parallel_size * prefill_context_model_parallel_size * tensor_model_parallel_size,
        )
        .unbind(0)
    )
    if enable_elastic_ep:
        _EP = _init_stateless_group(group_ranks, "ep", ...)
    else:
        _EP = init_model_parallel_group(
            group_ranks, get_world_group().local_rank, backend, group_name="ep"
        )

```

并行策略选择流程

flowchart TD

```

A[模型部署] --> B{GPU 数量?}
B -->| 单卡 | C[无需并行]
B -->| 多卡 | D{模型类型?}

D -->| Dense | E{通信拓扑?}
D -->| MoE | F{是否需要 EP?}

E -->| 同节点 NVLink | G[TP 优先]
E -->| 跨节点 | H[TP + PP]

F -->| 是 | I[TP + PP + EP + DP]
F -->| 否 | G

I --> J{是否需要弹性扩缩容?}
J -->| 是 | K[Elastic EP + Stateless Coordinator]
J -->| 否 | L[固定并行配置]

G --> M[最终并行方案]
H --> M

```

K --> M  
L --> M  
C --> M

## 二、并行状态管理

### 2.1 GroupCoordinator 通信组协调器

GroupCoordinator 是 vLLM 分布式系统的核心抽象，封装了 PyTorch 的 ProcessGroup，提供统一的通信接口。

位置: parallel\_state.py:L290-L1130

核心属性:

| 属性                  | 类型                              | 说明                |
|---------------------|---------------------------------|-------------------|
| rank                | int                             | 全局 rank           |
| ranks               | list[int]                       | 组内所有全局 rank       |
| world_size          | int                             | 组大小               |
| local_rank          | int                             | 本地 rank (用于绑定设备)  |
| rank_in_group       | int                             | 组内 rank           |
| cpu_group           | ProcessGroup                    | CPU 通信组 (Gloo 后端) |
| device_group        | ProcessGroup                    | 设备通信组 (NCCL 后端)   |
| device_communicator | DeviceCommunicatorBase \ \nNone | 设备通信器             |
| mq_broadcaster      | MessageQueue \ \nNone           | 共享内存广播器           |

初始化逻辑 (parallel\_state.py:L319-L396):

```
def __init__(
    self,
    group_ranks: list[list[int]],
    local_rank: int,
    torch_distributed_backend: str | Backend,
    use_device_communicator: bool,
    use_message_queue_broadcaster: bool = False,
    group_name: str | None = None,
):
    # 为每个 rank 列表创建 device_group (NCCL) 和 cpu_group (Gloo)
    for ranks in group_ranks:
        device_group = torch.distributed.new_group(ranks, backend=torch_distributed_backend)
        cpu_group = torch.distributed.new_group(ranks, backend="gloo")
        if self.rank in ranks:
            self.ranks = ranks
            self.world_size = len(ranks)
            self.rank_in_group = ranks.index(self.rank)
            self_device_group = device_group
            self_cpu_group = cpu_group

    # 根据平台选择设备
    if current_platform.is_cuda_alike():
        self.device = torch.device(f"cuda:{local_rank}")
    elif current_platform.is_xpu():
        self.device = torch.device(f"xpu:{local_rank}")
```



```

# 初始化设备通信器
if use_device_communicator and self.world_size > 1:
    device_comm_cls = resolve_obj_by_qualname(current_platform.get_device_communicator_cls())
    self.device_communicator = device_comm_cls(...)

```

提供的通信操作:

| 方法                                                      | 说明                    |
|---------------------------------------------------------|-----------------------|
| <code>all_reduce(input_)</code>                         | 全归约 (支持 custom op 分发) |
| <code>all_gather(input_, dim)</code>                    | 全收集                   |
| <code>reduce_scatter(input_, dim)</code>                | 归约散射                  |
| <code>gather(input_, dst, dim)</code>                   | 收集到目标 rank            |
| <code>broadcast(input_, src)</code>                     | 广播                    |
| <code>send(tensor, dst) / recv(size, dtype, src)</code> | 点对点发送/接收              |
| <code>send_tensor_dict() / recv_tensor_dict()</code>    | 字典形式的张量传输             |
| <code>broadcast_object(obj, src)</code>                 | 对象广播                  |
| <code>barrier()</code>                                  | 屏障同步                  |

**Custom Op** 机制: vLLM 通过 PyTorch Custom Op 注册 `all_reduce/reduce_scatter/all_gather`, 使得 Dynamo/tracing 能正确处理分布式调用 (parallel\_state.py:L262-L287):

```

direct_register_custom_op(
    op_name="all_reduce",
    op_func=all_reduce,
    fake_impl=all_reduce_fake,
)

```

## 2.2 并行组初始化流程

入口函数: `initialize_model_parallel()` (parallel\_state.py:L1494-L1735)

该函数按照 `ExternalDP × DP × PP × PCP × TP` 的顺序创建所有并行组。核心逻辑是将全局 rank 张量 reshape 后按维度转置和 unbind 来划分各组:

```

def initialize_model_parallel(
    tensor_model_parallel_size: int = 1,
    pipeline_model_parallel_size: int = 1,
    prefill_context_model_parallel_size: int = 1,
    decode_context_model_parallel_size: int | None = 1,
    backend: str | None = None,
) -> None:
    # 布局: ExternalDP x DP x PP x PCP x TP
    all_ranks = torch.arange(world_size).reshape(
        -1,
        data_parallel_size,
        pipeline_model_parallel_size,
        prefill_context_model_parallel_size,
        tensor_model_parallel_size,
    )

    # TP 组: 取最后一个维度
    group_ranks = all_ranks.view(-1, tensor_model_parallel_size).unbind(0)
    _TP = init_model_parallel_group(..., group_name="tp")

```

```

# PP 组: 将 PP 维度转到最后并 unbind
group_ranks = all_ranks.transpose(2, 4).reshape(-1, pp_size).unbind(0)
_PP = init_model_parallel_group(..., group_name="pp")

# DP 组: 将 DP 维度转到最后并 unbind
group_ranks = all_ranks.transpose(1, 4).reshape(-1, dp_size).unbind(0)
_DP = init_model_parallel_group(..., group_name="dp")

# EP 组: 合并 DP×PCP×TP 维度 (仅 MoE 模型)
group_ranks = all_ranks.transpose(1, 2).reshape(
    -1, dp_size * pcp_size * tp_size
).unbind(0)
_EP = init_model_parallel_group(..., group_name="ep")

```

全局单例访问: 各并行组通过全局变量和 getter 函数访问:

```

_TP: GroupCoordinator | None = None
def get_tp_group() -> GroupCoordinator:
    assert _TP is not None, "tensor model parallel group is not initialized"
    return _TP

# 同理: _PP, _DP, _EP, _EPLB, _DCP, _PCP, _WORLD

```

## 2.3 NCCL 通信域建立

环境初始化: `init_distributed_environment()` (`parallel_state.py:L1358-L1474`)

```

def init_distributed_environment(
    world_size: int = -1,
    rank: int = -1,
    distributed_init_method: str = "env://",
    local_rank: int = -1,
    backend: str = "nccl",
    timeout: timedelta | None = None,
):
    # 多节点或多 DP 时调整 rank/world_size
    if parallel_config.nnodes > 1 or parallel_config.data_parallel_size > 1:
        rank = parallel_config.data_parallel_rank * world_size + rank
        world_size = parallel_config.world_size_across_dp

    # 初始化 PyTorch distributed
    torch.distributed.init_process_group(
        backend=backend,
        init_method=distributed_init_method,
        world_size=world_size,
        rank=rank,
        timeout=timeout,
    )

    # 创建 WORLD 组
    _WORLD = init_world_group(ranks, local_rank, backend)

```

节点检测: `_node_count()` 函数使用共享内存测试进程是否在同一节点 (`parallel_state.py:L2096-L2131`):

```

def _node_count(pg: ProcessGroup | StatelessProcessGroup) -> int:
    """Returns the total number of nodes in the process group."""
    node_assignment = [0] * world_size

```

```

next_node_id = 0
for current_rank in range(world_size):
    if node_assignment[current_rank] != 0:
        continue
    next_node_id += 1
    node_assignment[current_rank] = next_node_id
    same_node_flags = in_the_same_node_as(pg, current_rank)
    for other_rank, is_same_node in enumerate(same_node_flags):
        if is_same_node and node_assignment[other_rank] == 0:
            node_assignment[other_rank] = next_node_id
return next_node_id

```

## 2.4 StatelessGroupCoordinator 无状态协调器

位置: stateless\_coordinator.py:L61-L365

**StatelessGroupCoordinator** 是 **GroupCoordinator** 的扩展, 用于创建独立于 **PyTorch WORLD** 组的通信域。这是 Elastic EP 等动态扩缩容场景的关键基础设施。

核心特点: - 不依赖 PyTorch 默认的 WORLD ProcessGroup - 使用 TCP Store 进行元数据交换 - 动态分配端口创建独立的 device/cpu/tcp\_store 三组通信域 - 支持 Elastic EP 场景下运行时添加/移除 worker

初始化过程 (stateless\_coordinator.py:L94-L148):

```

class StatelessGroupCoordinator(GroupCoordinator):
    def __init__(self, group_ranks, local_rank, torch_distributed_backend,
                 use_device_communicator, coord_store, ...):
        for idx, ranks in enumerate(group_ranks):
            if self.rank in ranks:
                key = f"{group_name}_{idx}"
                if self.rank_in_group == 0:
                    ports, socks = _allocate_group_ports(key, host, coord_store)
                else:
                    ports = _fetch_group_ports(key, coord_store)

                device_port, cpu_port, tcp_store_port = ports

                # 创建三个独立通信域
                device_group = stateless_init_torch_distributed_process_group(
                    host, port=device_port, rank=self.rank_in_group,
                    world_size=self.world_size, backend=backend, ...)
                cpu_group = stateless_init_torch_distributed_process_group(
                    host, port=cpu_port, ..., backend="gloo", ...)
                tcp_store_group = StatelessProcessGroup.create(
                    host, port=tcp_store_port, ...)

```

## 三、设备通信器 Device Communicators

### 3.1 基类 DeviceCommunicatorBase

位置: base\_device\_communicator.py:L118-L373

所有设备通信器的抽象基类, 定义了统一的通信接口规范:

```

class DeviceCommunicatorBase:
    def __init__(self, cpu_group, device=None, device_group=None,

```

```

        unique_name="", global_ranks=None, global_world_size=None):
self.device = device or torch.device("cpu")
self.cpu_group = cpu_group
self.device_group = device_group
self.unique_name = unique_name

# 区分 stateless group 与普通 group
is_stateless = _world.pg_map.get(cpu_group, None) is None
if is_stateless:
    self.rank = cpu_group.rank()
    self.world_size = cpu_group.size()
else:
    self.rank = dist.get_rank(cpu_group)
    self.world_size = dist.get_world_size(cpu_group)

# EP 通信相关
self.is_ep_communicator = unique_name.split(":")[0] == "ep"
self.use_all2all = self.is_ep_communicator and use_ep

```

核心通信方法:

| 方法                                       | 默认实现                                                 | 说明           |
|------------------------------------------|------------------------------------------------------|--------------|
| <code>all_reduce(input_)</code>          | <code>dist.all_reduce</code>                         | 全归约          |
| <code>all_gather(input_, dim)</code>     | <code>dist.all_gather_into_tensor</code> + reshape   | 全收集          |
| <code>reduce_scatter(input_, dim)</code> | <code>torch.distributed.reduce_scatter_tensor</code> | 归约散射         |
| <code>gather(input_, dst, dim)</code>    | <code>dist.gather</code> + cat                       | 收集           |
| <code>send(tensor, dst)</code>           | <code>dist.send</code>                               | 发送           |
| <code>recv(size, dtype, src)</code>      | <code>dist.recv</code>                               | 接收           |
| <code>broadcast(tensor, src)</code>      | <code>dist.broadcast</code>                          | 广播           |
| <code>dispatch(...)</code>               | no-op                                                | MoE token 分发 |
| <code>combine(hidden_states)</code>      | no-op                                                | MoE 结果合并     |

### 3.2 CudaCommunicator —CUDA 通信器

位置: `cuda_communicator.py`:L25-L458

CUDA 平台的主要通信器实现, 集成了多种 AllReduce 优化路径:

初始化—多级 AllReduce 引擎 (`cuda_communicator.py`:L36-L172):

```

class CudaCommunicator(DeviceCommunicatorBase):
    def __init__(self, cpu_group, device, device_group, unique_name, ...):
        super().__init__(...)

    # 仅 TP 组启用自定义优化
    if "tp" not in unique_name:
        use_custom_allreduce = False
        use_torch_symm_mem = False
        use_flashinfer_allreduce = False
    else:
        use_custom_allreduce = _ENABLE_CUSTOM_ALL_REDUCE
        use_torch_symm_mem = envs.VLLM_ALLREDUCE_USE_SYMM_MEM
        use_flashinfer_allreduce = envs.VLLM_ALLREDUCE_USE_FLASHINFER

```

```

# PyNCL: 直接 NCCL 调用
self.pynccl_comm = PyNcclCommunicator(group, device)
if is_symmetric_memory_enabled():
    register_nccl_symmetric_ops(self.pynccl_comm)

# SymmMem: PyTorch 对称内存 AllReduce
if use_torch_symm_mem and current_platform.is_cuda():
    self.symm_mem_comm = SymmMemCommunicator(group, device)

# FlashInfer AllReduce
if use_flashinfer_allreduce and self.world_size > 1:
    self.fi_ar_comm = FlashInferAllReduce(group, device)

# Custom AllReduce: 基于 P2P 的自定义内核
if use_custom_allreduce and self.world_size > 1:
    self.ca_comm = CustomAllreduce(group, device, symm_mem_enabled=...)
    # ROCm MI300 额外启用 QuickAllReduce
    if current_platform.is_rocm():
        self.qr_comm = QuickAllReduce(group, device)

# MoE All2All 管理器
if self.use_all2all:
    if self.all2all_backend == "deepep_high_throughput":
        self.all2all_manager = DeepEPHTAll2AllManager(...)
    elif self.all2all_backend == "flashinfer_nvlink_two_sided":
        self.all2all_manager = FlashInferNVLinkTwoSidedManager(...)
    # ... 其他 backend

```

**AllReduce** 分发决策树 (cuda\_communicator.py:L174-L231):

```

def all_reduce(self, input_):
    # 优先级 1: NCCL Symmetric Memory AllReduce
    if should_nccl_symm_mem_allreduce(self.pynccl_comm.world_size, input_):
        out = torch.ops.vllm.all_reduce_symmetric_with_copy(input_)
        if out is not None:
            return out

    # 优先级 2: QuickAllReduce (ROCm MI300 专用)
    if qr_comm and not qr_comm.disabled and qr_comm.should_quick_allreduce(input_):
        return qr_comm.quick_all_reduce(input_)

    # 优先级 3: FlashInfer AllReduce
    if fi_ar_comm and not fi_ar_comm.disabled and fi_ar_comm.should_use-fi_ar(input_):
        return fi_ar_comm.all_reduce(input_)

    # 优先级 4: Custom AllReduce (基于 P2P IPC)
    if ca_comm and not ca_comm.disabled and ca_comm.should_custom_ar(input_):
        return ca_comm.custom_all_reduce(input_)

    # 优先级 5: Torch SymmMem
    if symm_mem_comm and symm_mem_comm.should_use_symm_mem(input_):
        return symm_mem_comm.all_reduce(input_)

    # 优先级 6: PyNCL (标准 NCCL)

```

```

if pyncccl_comm and not pyncccl_comm.disabled:
    return pyncccl_comm.all_reduce(input_)

# Fallback: 标准 PyTorch all_reduce
out = input_.clone()
torch.distributed.all_reduce(out, group=self.device_group)
return out

```

### 3.3 PyNcclCommunicator —PyNCL NCCL 包装

位置: pyncccl.py:L58-L421

直接封装 NCCL C API 的 Python 通信器, 绕过 PyTorch 的 NCCL 封装以获得更精细的控制能力:

```

class PyNcclCommunicator:
    def __init__(self, group, device, library_path=None):
        # 加载 NCCL 库
        self.nccl = NCCLLibrary(library_path)
        # 获取/broadcast NCCL UniqueId
        if self.rank == 0:
            self.unique_id = self.nccl.ncclGetUniqueId()
        else:
            self.unique_id = ncclUniqueId()
        # 广播 unique_id 到所有 rank
        tensor = torch.ByteTensor(list(self.unique_id.internal))
        dist.broadcast(tensor, src=ranks[0], group=group)
        # 初始化 NCCL communicator
        with torch.accelerator.device_index(device.index):
            self.comm = self.nccl.ncclCommInitRank(world_size, unique_id, rank)

```

支持的通信原语:

| 方法                                  | NCCL API           | 说明       |
|-------------------------------------|--------------------|----------|
| all_reduce(in, out, op, stream)     | ncclAllReduce      | 全归约      |
| all_gather(out, in, stream)         | ncclAllGather      | 全收集      |
| all_gatherv(out, in, sizes, stream) | ncclBroadcast 循环   | 变长全收集    |
| reduce_scatter(out, in, op, stream) | ncclReduceScatter  | 归约散射     |
| send(tensor, dst, stream)           | ncclSend           | 发送       |
| recv(tensor, src, stream)           | ncclRecv           | 接收       |
| broadcast(tensor, src, stream)      | ncclBroadcast      | 广播       |
| batch_isend_irecv(p2p_ops)          | ncclGroupStart/End | 批量异步 P2P |

### 3.4 CustomAllreduce —自定义 AllReduce

位置: custom\_all\_reduce.py:L50-L325

基于 GPU P2P 互访的自定义 AllReduce 实现, 利用 NVIDIA/NVLink 或 AMD XGMI 的高带宽直接内存访问来避免 NCCL 开销:

适用条件检查链 (custom\_all\_reduce.py:L71-L171):

```
class CustomAllreduce:
    _SUPPORTED_WORLD_SIZES = [2, 4, 6, 8]

    def __init__(self, group, device, max_size=8192*1024, symm_mem_enabled=False):
        # 1. 必须有自定义 AR 库
        if not custom_ar:
            return # disabled

        # 2. 不能挂在 NCCL 组上
        assert dist.get_backend(group) != dist.Backend.NCCL

        # 3. 所有 rank 必须在同一节点
        if not all(in_the_same_node_as(group, source_rank=0)):
            return # disabled

        # 4. world_size 必须在支持列表中
        if world_size not in CustomAllreduce._SUPPORTED_WORLD_SIZES:
            return

        # 5. 检查硬件全连接 (NVLink / XGMI)
        fully_connected = current_platform.is_fully_connected(physical_device_ids)
        if world_size > 2 and not fully_connected:
            return

        # 6. 测试实际 P2P 可达性
        if not current_platform.is_rocm() and not _can_p2p(rank, world_size):
            return

        # 分配 IPC 共享缓冲区
        self.meta_ptrs = self.create_shared_buffer(ops.meta_size() + max_size, group)
        self.buffer_ptrs = self.create_shared_buffer(max_size, group)
        self._ptr = ops.init_custom_ar(self.meta_ptrs, self.rank_data, rank, fully_connected)
        ops.register_buffer(self._ptr, self.buffer_ptrs)
        self.disabled = False
```

大小限制 (按 GPU 架构和 world\_size 分类) (all\_reduce\_utils.py:L31-L71):

```
CUSTOM_ALL_REDUCE_MAX_SIZES = {
    "9.0": { # Volta (V100)
        2: 64 * MiB,    4: 32 * MiB,
        6: MiB // 2,    8: MiB // 4,
    },
    "10.0": { # Ampere (A100)
        2: 2 * MiB,      4: 2 * MiB,
        6: 1 * MiB,      8: 1 * MiB,
    },
    "10.3": { # Ada/Hopper (H100, H20)
        2: 4 * MiB,      4: 4 * MiB,
        6: 8 * MiB,      8: 4 * MiB,
    },
}
```

P2P 可达性验证: 使用子进程进行真实的 IPC 内存读写测试 (all\_reduce\_utils.py:L212-L294):

```
def can_actually_p2p(batch_src, batch_tgt):
    """ 通过真实 IPC 内存读写验证 P2P 可达性 """
    smp = mp.get_context("spawn")
    producer_queue, consumer_queue, result_queue = ...
    p_src = smp.Process(target=producer, args=...)
    p_tgt = smp.Process(target=consumer, args=...)
    p_src.start(); p_tgt.start(); p_src.join(); p_tgt.join()
```

### 3.5 QuickAllReduce —快速 AllReduce (ROCm)

位置: quick\_all\_reduce.py:L45-L289

专为 AMD ROCm MI300 系列 (gfx94/gfx95) 设计的量化 AllReduce 实现:

量化等级 (quick\_all\_reduce.py:L34-L39):

```
class QuickReduceRegime(Enum):
    FP = 0          # float16/bfloat16 精度
    INT8 = 1        # INT8 量化
    INT6 = 2        # INT6 量化
    INT4 = 3        # INT4 量化
    NONE = 4        # 禁用
```

触发条件判断 (quick\_all\_reduce.py:L246-L268):

```
def should_quick_allreduce(self, inp):
    if self.disabled: return False
    if inp.dtype not in self._SUPPORTED_DTYPES: return False
    inp_size = inp.numel() * inp.element_size()
    if inp_size % 16 != 0: return False          # 字节对齐
    if not is_weak_contiguous(inp): return False
    return (
        inp_size <= self.qr_max_size
        and inp_size >= self._QR_MIN_SIZE[(dtype, self.world_size)][self.qr_quant_level.value]
    )
```

### 3.6 MessageQueue —共享内存广播

位置: shm\_broadcast.py:L353-L917

基于共享内存环形缓冲区 + ZMQ PUB/SUB 的高性能广播系统, 用于 TP 组内的对象广播 (如 grammar bitmask 等):

架构组成:

```
graph LR
    W[Writer Rank 0] -->|SHM RingBuffer| LR1[Local Reader 1]
    W -->|SHM RingBuffer| LR2[Local Reader 2]
    W -->|ZMQ PUB/SUB| RR1[Remote Reader 1]
    W -->|ZMQ PUB/SUB| RR2[Remote Reader 2]

    W --> SC[SpinCondition<br/>ZMQ Notify]
    SC --> LR1
    SC --> LR2
```

ShmRingBuffer 内存布局 (shm\_broadcast.py:L204-L303):

```
+-----+-----+
|          Data Region          |          Metadata Region          |
| chunk0 | chunk1 | ... | chunk | meta0 | meta1 | ... | metadata    |
```



```
| max_chunks x max_chunk_bytes | max_chunks x (1 + n_reader) bytes |
+-----+-----+-----+-----+-----+
```

Metadata per block:

```
+-----+-----+-----+-----+-----+
| written_flag | reader0_flag | reader1_flag | ... | readerN_flag |
+-----+-----+-----+-----+-----+
```

**SpinCondition** 自旋条件变量: 结合忙等待 (busy-spin) 和 ZMQ 通知, 高频读取时自旋, 低频时阻塞等待通知 (shm\_broadcast.py:L92-L201)。

**Memory Fence** 内存屏障: 确保跨进程共享内存的可见性 (shm\_broadcast.py:L55-L73):

```
def memory_fence():
    """Full memory barrier for shared memory synchronization."""
    with _memory_fence_lock:
        pass # Lock acquire/release provides full memory barrier semantics
```

### 3.7 Ray/CPU/XPU 通信器

**RayPPCommunicator** —Ray 流水线通信 位置: ray\_communicator.py:L22-L259

包装 vLLM\_PP GroupCoordinator 以适配 Ray Compiled Graph 的 Communicator 接口:

```
class RayPPCommunicator(Communicator):
    def __init__(self, world_size, comm_id, rank, actor_handles, ...):
        # 复用 vLLM 的 PP 设备通信器
        self._comm = get_pp_group().device_communicator
        self._rank = self._comm.rank_in_group
        self._build_actor_rank_mapping() # actor_id -> rank 映射
```

**CpuCommunicator** —CPU 通信器 位置: cpu\_communicator.py:L20-L227

CPU 平台的通信器, 可选启用基于共享内存 (\_CPUSHMDistributed) 的高效路径:

```
class CpuCommunicator(DeviceCommunicatorBase):
    def __init__(self, cpu_group, device, device_group, unique_name):
        super().__init__(...)
        # 当 X86/ARM 且 ranks 共享 SHM 组名时启用 SHM 通信
        if ((current_platform.get_cpu_architecture() in (CpuArchEnum.X86, CpuArchEnum.ARM))
            and hasattr(torch.ops._C, "init_shm_manager")
            and self._all_group_ranks_share_shm_group_name()):
            self.dist_module = _CPUSHMDistributed(self)
            self.supports_tensor_dict = isinstance(self.dist_module, _CPUSHMDistributed)
```

**XpuCommunicator** —XPU 通信器 位置: xpu\_communicator.py:L16-L60

Intel XPU 设备的通信器, AllReduce 回退到标准 dist.all\_reduce, All2All 使用 AgRs 后端:

```
class XpuCommunicator(DeviceCommunicatorBase):
    def all_reduce(self, input_):
        output = input_.clone()
        dist.all_reduce(output, group=self.device_group)
        return output
```

### 3.8 All2AllManager —MoE 全对全通信

位置: all2all.py:L40-L80

MoE 专家并行的核心通信原语，负责 token → expert 的 dispatch 和 expert output → token 的 combine:

**AgRsAll2AllManager** (默认后端): 基于 AllGather + ReduceScatter 实现:

```
class AgRsAll2AllManager(All2AllManagerBase):
    """All2All based on all-gather (dispatch) and reduce-scatter (combine)."""

    def dispatch(self, hidden_states, topk_weights, topk_ids, ...):
        # Gather from all DP ranks
        dp_metadata = get_forward_context().dp_metadata
        sizes = dp_metadata.get_chunk_sizes_across_dp_rank()
        gathered = get_ep_group().all_gatherv(
            [hidden_states, topk_weights, topk_ids], dim=0, sizes=sizes)
        # ... 本地 compute + scatter
```

支持的后端列表:

| 后端                              | 特点                   | 适用场景      |
|---------------------------------|----------------------|-----------|
| naive / allgather_reducescatter | 基于 AllGather+RS      | 通用/默认     |
| deepep_high_throughput          | DeepEP 高吞吐           | 大规模 MoE   |
| deepep_low_latency              | DeepEP 低延迟           | 延迟敏感      |
| flashinfer_nvlink_two_sided     | FlashInfer NVLink 双向 | NVLink 集群 |
| flashinfer_nvlink_one_sided     | FlashInfer NVLink 单向 | NVLink 集群 |
| mori                            | MoriIO 定制            | MoriIO 硬件 |
| nixl_ep                         | NIXL 传输              | RDMA 网络   |

四、数据传输机制

4.1 KV Transfer —跨节点 KV 缓存传输

位置: kv\_transfer/kv\_transfer\_state.py, 连接器位于 kv\_transfer/kv\_connector/v1/

架构设计: 分离 Scheduler 侧和 Worker 侧, 支持多种传输后端:

```
graph TB
    subgraph Scheduler["Scheduler 侧"]
        S_META[Metadata 绑定]
        S_ALLOC[update_state_after_alloc]
        S_REQ[request_finished]
        S_EVT[take_events]
    end

    subgraph Worker["Worker 侧"]
        W_PREEMPT[handle_preemptions]
        W_LOAD[start_load_kv]
        W_WAIT_L[wait_for_layer_load]
        W_SAVE[save_kv_layer]
        W_WAIT_S[wait_for_save]
        W_FIN[get_finished]
    end

    S_META --> W_LOAD
    S_ALLOC --> W_LOAD
    S_REQ --> W_SAVE
```

S\_EVT --> S\_FIN

```
style Scheduler fill:#e3f2fd,stroke:#1565c0,color:#0d47a1
style Worker fill:#fff8e1,stroke:#f9a825,color:#f57f17
```

连接器基类 **KVConnectorBase\_V1** (kv\_connector/v1/base.py) 定义了完整的生命周期接口。

支持的后端:

| 连接器                  | 文件                      | 说明           |
|----------------------|-------------------------|--------------|
| lmcache_connector    | lmcache_connector.py    | LMCache 集成   |
| p2p_nccl_connector   | p2p_nccl_connector.py   | P2P NCCL 直传  |
| nixl_connector       | connector.py            | NIXL/RDMA 传输 |
| mooncake_connector   | mooncake_connector.py   | Mooncake 分布式 |
| hf3fs_connector      | hf3fs_connector.py      | HDFS 分布式文件系统 |
| offloading_connector | offloading_connector.py | CPU 卸载       |
| moriio_connector     | moriio_connector.py     | MoriIO 传输    |

工厂模式创建 (kv\_transfer\_state.py:L51-L71):

```
def ensure_kv_transfer_initialized(vllm_config, kv_cache_config):
    global _KV_CONNECTOR_AGENT
    if vllm_config.kv_transfer_config is None:
        return
    if vllm_config.kv_transfer_config.is_kv_transfer_instance and _KV_CONNECTOR_AGENT is None:
        _KV_CONNECTOR_AGENT = KVConnectorFactory.create_connector(
            config=vllm_config, role=KVConnectorRole.WORKER, kv_cache_config=kv_cache_config)
```

## 4.2 Weight Transfer 一权重分发

位置: weight\_transfer/ 目录

支持从 Trainer 到 Inference Worker 的增量权重传输, 两种后端实现:

**NCCL Weight Transfer Engine** 位置: nccl\_engine.py

基于 NCCL Broadcast 的权重分发, 支持 **packed broadcast** 优化 (多张量打包批量传输):

```
class NCCLWeightTransferEngine(WeightTransferEngine[...]):
    def receive_weights(self, update_info, load_weights):
        if update_info.packed:
            # 打包批量接收
            packed_broadcast_consumer(
                iterator=state_dict_info_iterator(),
                group=self.model_update_group,
                src=0,
                post_unpack_func=load_weights,
                buffer_size_bytes=update_info.packed_buffer_size_bytes,
                num_buffers=update_info.packed_num_buffers,
            )
        else:
            # 逐个 broadcast
            for name, dtype_name, shape in zip(names, dtypes, shapes):
                weight = torch.empty(shape, dtype=dtype, device="cuda")
                self.model_update_group.broadcast(weight, src=0)
                load_weights([(name, weight)])
```

**StatelessProcessGroup** 集成: Trainer 和 Worker 通过 **StatelessProcessGroup** 建立独立于 vLLM WORLD 的 NCCL 通道 (nccl\_engine.py:L322-L340):

```
@staticmethod
def _stateless_init_process_group(master_address, master_port, rank, world_size, device):
    pg = StatelessProcessGroup.create(host=master_address, port=master_port, ...)
    pynccl = PyNcclCommunicator(pg, device=device)
    return pynccl
```

**IPC Weight Transfer Engine** 位置: ipc\_engine.py

基于 CUDA IPC 的零拷贝权重传输, 适用于同节点场景:

```
class IPCWeightTransferEngine(WeightTransferEngine[...]):
    def receive_weights(self, update_info, load_weights):
        for name, shape, ipc_handle in zip(names, shapes, ipc_handles):
            device_index = torch.accelerator.current_device_index()
            props = torch.cuda.get_device_properties(device_index)
            physical_gpu_id = str(props.uuid)
            handle = ipc_handle[physical_gpu_id]
            func, args = handle
            list_args[6] = device_index # 更新设备索引
            weight = func(*list_args) # 通过 IPC handle 重建张量
            weights.append((name, weight))
        load_weights(weights)
```

支持 Ray RPC 和 HTTP POST 两种传输模式。

### 4.3 EC Transfer — 纠删码传输

位置: ec\_transfer/ec\_transfer\_state.py, 连接器位于 ec\_transfer/ec\_connector/

用于 Encoder-only 模型的 encoder cache 分布式传输, 采用纠删码 (Erasure Coding) 提高传输可靠性:

**ECConnectorBase** 接口 (ec\_connector/base.py):

```
class ECConnectorBase(ABC):
    def __init__(self, vllm_config, role: ECConnectorRole):
        self._role = role # SCHEDULER 或 WORKER
        self._is_producer = vllm_config.ec_transfer_config.is_ec_producer
        self._is_consumer = vllm_config.ec_transfer_config.is_ec_consumer
```

**Scheduler** 侧操作: check\_caches\_exist(), update\_state\_after\_alloc(), request\_finished() **Worker** 侧操作: start\_load\_ec(), wait\_for\_save(), get\_finished()

---

## 五、高级特性

### 5.1 Elastic EP — 弹性专家并行

位置: elastic\_ep/ 目录

允许运行时动态增减 EP (专家并行) worker 数量, 实现 MoE 推理服务的弹性伸缩。

核心状态机 (elastic\_state.py):

**Scale-Up** (扩容) 状态机

```
stateDiagram-v2
    [*] --> WAIT_NEW_CORE_ENGINES_INIT: Existing Engine
    WAIT_NEW_CORE_ENGINES_INIT --> CREATE_STANDBY_GROUPS: 收到 INIT_READY
    CREATE_STANDBY_GROUPS --> TRANSFER_EXPERT_MAPPING: Barrier 完成
    TRANSFER_EXPERT_MAPPING --> WAIT_NEW_CORE_ENGINES_WEIGHTS_INIT
    WAIT_NEW_CORE_ENGINES_WEIGHTS_INIT --> TRANSFER_WEIGHTS: 收到 WEIGHTS_READY
    TRANSFER_WEIGHTS --> SYNC_KV_CACHE_MEMORY_SIZE
    SYNC_KV_CACHE_MEMORY_SIZE --> SWITCH_AND_PREPARE
    SWITCH_AND_PREPARE --> EPLB_RESHUFFLE
    EPLB_RESHUFFLE --> COMPLETE: 扩容完成

    [*] --> PRE_KV_INIT: New Engine
    PRE_KV_INIT --> PREPARE
    PREPARE --> EPLB_RESHUFFLE
    EPLB_RESHUFFLE --> COMPLETE: 新引擎就绪
```

两阶段 **Barrier** (elastic\_state.py:L152-L225):

```
def _staged_barrier(self, use_new_group, barrier_name):
    """ 两阶段 barrier 解决 EngineCore 收到通知时序不一致问题 """
    dp_group = self.new_dp_group if use_new_group else self.old_dp_group
    # 第一阶段: 带 timeout 的 TCPStore barrier
    timeout = None if dp_store.check([sync_key]) else timedelta(seconds=5)
    try:
        self._execute_tcp_store_barrier(dp_group, group_rank, group_size, barrier_id, timeout)
        torch.distributed.barrier(dp_group)
        return True
    except _BarrierTimeoutError:
        # timeout 表示有 EngineCore 还没收到通知, 跳过本次让它继续执行一步
        dp_store.compare_set(sync_key, "", b"1")
        return False # 下次再试
```

关键阶段说明:

| 阶段                        | 操作                | 说明                                                        |
|---------------------------|-------------------|-----------------------------------------------------------|
| CREATE_STANDBY_GROUPS     | 创建新的 DP 通信组       | 使用 <code>stateless_init_dp_group</code>                   |
| TRANSFER_EXPERT_MAPPING   | 广播 expert mapping | <code>collective_rpc</code><br>"broadcast_expert_mapping" |
| TRANSFER_WEIGHTS          | 传输权重到新 worker     | <code>collective_rpc</code><br>"transfer_weights"         |
| SYNC_KV_CACHE_MEMORY_SIZE | 同步 KV cache 内存预算  | <code>ParallelConfig.sync_kv_cache_memory_size</code>     |
| SWITCH_AND_PREPARE        | 切换到新通信组           | 销毁旧组, 替换为新组                                               |
| EPLB_RESHUFFLE            | EPLB 专家重排         | 重新平衡专家负载                                                  |

**Scale-Down** (缩容) 状态机

```
stateDiagram-v2
    [*] --> PREPARE: Remaining Engine
    PREPARE --> EPLB_RESHUFFLE
    EPLB_RESHUFFLE --> SWITCH_AND_PREPARE: 重排 + 切换 + 创建 standby
    SWITCH_AND_PREPARE --> COMPLETE: 缩容完成

    [*] --> PREPARE: Removing Engine
    PREPARE --> EPLB_RESHUFFLE
```

EPLB\_RESHUFFLE --> SWITCH\_AND\_REMOVE: 关闭引擎  
SWITCH\_AND\_REMOVE --> COMPLETE: 发送 SHUTDOWN\_COMPLETE

## 5.2 EPLB —弹性负载均衡

位置: `ep1b/` 目录, 核心状态 `ep1b_state.py`

Expert Parallelism Load Balancer, 通过动态重排物理专家 (Physical Experts) 到 GPU 来均衡 MoE 负载。

核心概念:

| 术语                    | 说明                                |
|-----------------------|-----------------------------------|
| Logical Expert        | 模型定义的逻辑专家 (如 DeepSeek-R1 的 256 个) |
| Physical Expert       | 实例化在具体设备上的专家副本                    |
| Redundant Expert      | 为热门逻辑专家创建的额外副本                    |
| Local Physical Expert | 当前设备上的物理专家                        |

示例: DeepSeek-R1 有 256 个 logical experts + 32 个 redundant experts = 288 个 physical experts, 32 个 EP rank → 每个 GPU 9 个 physical experts。

**EplbModelState** 核心数据结构 (`ep1b_state.py:L90-L208`):

```
@dataclass
class EplbModelState:
    physical_to_logical_map: torch.Tensor      # [num_layers, num_physical_experts]
    logical_to_physical_map: torch.Tensor      # [num_layers, num_logical, max_replicas]
    logical_replica_count: torch.Tensor        # [num_layers, num_logical_experts]
    expert_load_pass: torch.Tensor              # [num_layers, num_physical_experts]
    expert_load_window: torch.Tensor            # [window, num_layers, num_physical_experts]
    expert_buffer: list[torch.Tensor]           # 权重传输中间缓冲
    communicator: EplbCommunicator             # 专家权重通信器
    rebalanced: bool                           # 异步模式下是否有待提交的结果
```

**EPLB** 主循环 (`ep1b_state.py:L473-L605`):

```
def step(self, is_dummy=False, is_profile=False, log_stats=False):
    # 1. 记录/更新 expert load sliding window
    if not is_dummy and should_record:
        eplb_model_state.expert_load_window[self.expert_load_window_step].copy_(expert_load_pass)
        self.expert_load_window_step += 1

    # 2. 异步模式: 检查 async worker 是否完成
    if self.is_async and eplb_model_state.rebalanced and self._all_ranks_result_ready(...):
        _move_to_workspace(model_state, ep_rank)

    # 3. 达到间隔 -> 触发重排
    self.expert_rearrangement_step += 1
    if self.expert_rearrangement_step >= self.expert_rearrangement_step_interval:
        self.expert_rearrangement_step = 0
        self.rearrange()

def rearrange(self, is_profile=False, rank_mapping=None):
    # 1. 收集全局 expert load (all-reduce)
    global_expert_load_windows = self._allreduce_list(global_expert_load_windows)

    # 2. 调用 policy 计算新的映射
```

```

new_physical_to_logical_map = self.policy.rebalance_experts(
    global_expert_load_window.cpu(), num_replicas, num_groups, num_nodes, num_gpus, ...)

# 3. 就地重排专家权重
rearrange_expert_weights_inplace(old_map, new_map, weights, ep_group, communicator, ...)

# 4. 提交新映射
_commit_eplb_maps(model_state, new_physical_to_logical_map)

```

同步 vs 异步模式: - 同步: 主线程完成重排 (权重传输 + 映射更新) - 异步: 主线程只计算新映射, 后台线程 (async\_worker) 执行权重传输; 通过 rebalanced flag + pending\_result 协调

### 5.3 Stateless Coordinator 一无状态协调器

已在2.4节详细描述。此处补充其在 Elastic EP 中的应用:

Elastic EP 初始化 (parallel\_state.py:L1323-L1355):

```

def _init_elastic_ep_world(config, local_rank, backend, rank, world_size):
    global _WORLD, _NODE_COUNT
    parallel_config = config.parallel_config
    global_rank = parallel_config.data_parallel_rank * world_size + rank
    global_world_size = parallel_config.world_size_across_dp
    coord_store = get_cached_tcp_store_client(parallel_config.data_parallel_master_ip, ...)
    world = StatelessGroupCoordinator(
        group_ranks=group_ranks,
        local_rank=local_rank,
        torch_distributed_backend=backend,
        use_device_communicator=False,
        group_name="world",
        host=parallel_config.data_parallel_master_ip,
        coord_store=coord_store,
        global_rank=global_rank,
        global_world_size=global_world_size,
    )
    _WORLD = world

```

### 5.4 NIXL Utils —NIXL 工具集

位置: nixl\_utils.py

NVIDIA/AMD NIXL (Network Intercept eXchange Library) 的 Python 封装工具集, 用于高性能 RDMA 数据传输:

```

def _load_nixl_attr(name: str) -> Any:
    """ 懒加载 NIXL 模块, 自动区分 CUDA (nixl) 和 ROCm (rixl) """
    package_name = "rixl" if current_platform.is_rocm() else "nixl"
    module = importlib.import_module(f"{package_name}._api")
    value = getattr(module, attr_name, None)
    globals()[name] = value
    return value

```

导出的符号:

| 符号                | 来源                      | 用途            |
|-------------------|-------------------------|---------------|
| NixlWrapper       | nixl_agent / rixl_agent | NIXL agent 封装 |
| nixl_agent_config | nixl_agent_config       | Agent 配置      |

| 符号                | 来源        | 用途   |
|-------------------|-----------|------|
| nixlXferTelemetry | _bindings | 传输遥测 |

**UCX RCache** 修复: 自动设置 UCX\_RCACHE\_MAX\_UNRELEASED=1024 避免 UCX 内存泄漏。

## 六、通信流程总览

典型请求处理中的通信模式

sequenceDiagram

```

participant R0 as Rank 0 (TP0,PP0)
participant R1 as Rank 1 (TP1,PP0)
participant R2 as Rank 2 (TP0,PP1)
participant R3 as Rank 3 (TP1,PP1)

Note over R0,R3: Prefill Stage
R0->>R1: TP AllReduce (QKV projection)
R1->>R0: TP AllReduce (attention output)
R0->>R2: PP Send (hidden states)
R2->>R0: PP Recv (next layer input)

```

```

Note over R0,R3: Decode Stage (per step)
R0->>R1: TP AllReduce (QKV projection)
R1->>R0: TP AllReduce (attention output)
R0->>R2: PP Send (hidden states)
R2->>R0: PP Recv

```

```

Note over R0,R3: MoE Dispatch (if applicable)
R0->>R1: EP All2All Dispatch (tokens→experts)
R1->>R0: EP All2All Combine (outputs→tokens)

```

## AllReduce 路径选择决策

flowchart TD

```

A[AllReduce Request] --> B{TP Group?}
B -->|No| Z[Standard PyTorch all_reduce]
B -->|Yes| C{Symm Mem Enabled?}
C -->|Yes & Size Match| D[NCCL Symmetric Memory AR]
C -->|No| E{ROCM MI300?}
E -->|Yes| F{Quick AR 条件满足?}
F -->|Yes| G[QuickAllReduce 量化 AR]
F -->|No| H{FlashInfer AR?}
H -->|Yes| I[FlashInfer AllReduce]
H -->|No| J{Custom AR 条件满足?}
J -->|Yes| K[CustomAllreduce P2P IPC]
J -->|No| L{Torch SymmMem?}
L -->|Yes| M[SymmMemCommunicator]
L -->|No| N[PyNcclCommunicator]
N --> O{PyNCL Available?}
O -->|Yes| P[NCCL AllReduce via PyNCL]
O -->|No| Z
D --> Q[Return Result]

```



G --> Q  
I --> Q  
K --> Q  
M --> Q  
P --> Q  
Z --> Q

七、文件索引

| 文件路径                        | 核心内容                                             |
|-----------------------------|--------------------------------------------------|
| parallel_state.py           | GroupCoordinator、并行组初始化、通信操作注册                   |
| stateless_coordinator.py    | StatelessGroupCoordinator、独立通信域                  |
| base_device_communicator.py | DeviceCommunicatorBase 基类、<br>All2AllManagerBase |
| cuda_communicator.py        | CudaCommunicator、多级 AllReduce 调度                 |
| pyncl.py                    | PyNcclCommunicator、NCCL C API 封装                 |
| custom_all_reduce.py        | CustomAllreduce、P2P IPC AllReduce                |
| quick_all_reduce.py         | QuickAllReduce、ROCm 量化 AR                        |
| shm_broadcast.py            | MessageQueue、ShmRingBuffer、SpinCondition         |
| ray_communicator.py         | RayPPCommunicator                                |
| cpu_communicator.py         | CpuCommunicator、_CPUSHMDistributed               |
| xpu_communicator.py         | XpuCommunicator                                  |
| all2all.py                  | AgRsAll2AllManager 及变体                           |
| all_reduce_utils.py         | P2P 检测、size 配置表                                  |
| kv_transfer_state.py        | KV Transfer 初始化与生命周期                             |
| nccl_engine.py              | NCCL Weight Transfer                             |
| ipc_engine.py               | IPC Weight Transfer                              |
| ec_transfer_state.py        | EC Transfer 初始化                                  |
| elastic_state.py            | Elastic EP 状态机                                   |
| eplb_state.py               | EPLB 负载均衡状态与算法                                   |
| nixl_utils.py               | NIXL 工具函数                                        |

vLLM LoRA 适配器系统深度解析

定位

LoRA (Low-Rank Adaptation) 是 vLLM 中实现高效多租户适配器服务 (Multi-Tenant LoRA Serving) 的核心子系统。它基于 **Punica** 论文架构，支持在单次推理 batch 中同时运行多个不同的 LoRA 适配器，通过低秩矩阵分解和批量 **GEMM** 融合计算实现近乎零开销的 adapter 切换。本模块覆盖从配置、权重管理、请求绑定、层级替换到 GPU kernel 计算的完整链路。

```
graph TB
    subgraph LoRA_核心 [" LoRA 核心层"]
        LC[LoRAConfig<br/> 配置参数]
        LR[LoRARequest<br/> 请求绑定]
        LW[LoRALayerWeights<br/> 权重存储]
        LM[LoRAModel<br/> 模型封装]
    end

    subgraph Layer_实现 [" Layer 层实现"]
        BL[BaseLayerWithLoRA<br/> 基类抽象]
    end
```

```

    CPL[ColumnParallelLinear+LoRA]
    RPL[RowParallelLinear+LoRA]
    FMOE[FusedMoE+LoRA]
    LP[LogitsProcessor+LoRA]
    RL[ReplicatedLinear+LoRA]
    VPE[VocabParallelEmbedding+LoRA]
end

subgraph 运行时 [" 运行时引擎"]
    WM[WorkerLoRAManager<br/>Worker 级管理]
    MM[LoRAModelManager<br/>模型管理器]
    RES[LoRAResolver<br/>适配器解析]
    PW[PunicaWrapper<br/>GPU 计算库]
end

LC --> LM
LW --> LM
LR --> WM
LM --> MM
WM --> MM
RES --> WM
BL --> CPL
BL --> RPL
BL --> FMOE
BL --> LP
BL --> RL
BL --> VPE
CPL & RPL & FMOE & LP & RL & VPE --> PW
PW --> |"Triton/CUDA Kernel"| GPU["GPU 批量计算"]

style LoRA 核心 fill:#ffe0b2,stroke:#e65100,stroke-width:2px
style Layer 实现 fill:#fff3e0,stroke:#ef6c00,stroke-width:2px
style 运行时 fill:#fce4ec,stroke:#c62828,stroke-width:2px

```

---

## 一、LoRA 概述

### 1.1 Low-Rank Adaptation 原理

LoRA (Low-Rank Adaptation) 是一种参数高效微调方法，其核心思想是将全量微调的大矩阵更新  $\Delta W$  分解为两个低秩矩阵的乘积：

$$\Delta W = B \times A \quad \text{其中 } A \in \mathbb{R}^{r \times k}, B \in \mathbb{R}^{d \times r}$$

- $A$  (**lora\_a**): 降维投影矩阵，将输入从  $k$  维投影到  $r$  维 ( $r \ll \min(k, d)$ )
- $B$  (**lora\_b**): 升维恢复矩阵，将  $r$  维映射回  $d$  维
- **rank** ( $r$ ): 低秩维度，典型值为 8, 16, 32, 64
- **scaling factor**:  $\text{scaling} = \frac{\alpha}{r}$ ，其中  $\alpha$  是 lora\_alpha

前向计算公式：

$$h = W_0 x + \Delta W x = W_0 x + B A x \cdot \frac{\alpha}{r}$$

vLLM 的 LoRA 实现源自 Punica: Multi-Tenant LoRA Serving 论文，其关键创新在于：- **Batched GEMM**: 将

多个 LoRA adapter 的  $A$  和  $B$  矩阵堆叠为 4D 张量 (`max_loras, 1, rank, dim`)，利用 Triton/CUDA kernel 实现一次前向传播处理多个 adapter - **Dynamic Dispatch**: 通过 `token_lora_indices` 映射每个 token 到对应的 LoRA slot index，实现 per-token 级别的 adapter 选择

## 二、LoRAConfig 配置参数

源码位置: `lora.py`

LoRAConfig 是 vLLM 中控制 LoRA 行为的核心配置类，使用 Pydantic 进行参数校验。

### 2.1 关键配置项

| 参数名                                      | 类型                                   | 默认值                 | 说明                                                           |
|------------------------------------------|--------------------------------------|---------------------|--------------------------------------------------------------|
| <code>max_lora_rank</code>               | <code>MaxLoRARanks</code>            | 16                  | 支持的最大 LoRA rank, 可选值: 1/8/16/32/64/128/256/320/512           |
| <code>max_loras</code>                   | <code>int</code>                     | 1                   | 单个 batch 中同时存在的最大 LoRA adapter 数量                            |
| <code>fully_sharded_loras</code>         | <code>bool</code>                    | <code>False</code>  | 是否对 LoRA 权重做完整 tensor parallel 分片 (S-LoRA 策略)                |
| <code>max_cpu_loras</code>               | <code>int \   None</code>            | <code>None</code>   | CPU 内存中缓存的 LoRA 数量上限 (默认等于 <code>max_loras</code> )          |
| <code>lora_dtype</code>                  | <code>torch.dtype \   str</code>     | <code>"auto"</code> | LoRA 权重的数据类型, <code>auto</code> 时继承基础模型 dtype                |
| <code>target_modules</code>              | <code>list[str] \   None</code>      | <code>None</code>   | 限制 LoRA 应用的目标模块后缀列表 (如 <code>["o_proj", "qkv_proj"]</code> ) |
| <code>default_mm_loras</code>            | <code>dict[str, str] \   None</code> | <code>None</code>   | 多模态模型中模态到 LoRA 路径的默认映射                                       |
| <code>enable_tower_connector_lora</code> | <code>bool</code>                    | <code>False</code>  | 是否启用视觉编码器和 connector 的 LoRA (实验性功能)                          |
| <code>specialize_active_lora</code>      | <code>bool</code>                    | <code>False</code>  | 是否按活跃 LoRA 数量专门化 CUDA graph                                  |

### 2.2 配置校验逻辑

```
# /workspace/vllm/config/lora.py L101-L118
@model_validator(mode="after")
def _validate_lora_config(self) -> Self:
    if self.max_cpu_loras is None:
        self.max_cpu_loras = self.max_loras
    elif self.max_cpu_loras < self.max_loras:
        raise ValueError(
            f"max_cpu_loras ({self.max_cpu_loras}) must be >= "
            f"max_loras ({self.max_loras})."
```

```

    )
    if envs.VLLM_LORA_ENABLE_DUAL_STREAM and not current_platform.is_cuda_alike():
        raise ValueError("Dual CUDA streams are only supported on CUDA platforms.")
    ...
    return self

```

关键约束: - max\_cpu\_loras >= max\_loras: CPU 缓存容量必须不小于 GPU 并发容量 - Dual stream 仅在 CUDA 平台可用 - fully\_sharded\_loras 与 dual stream 不兼容

### 2.3 Hash 计算

compute\_hash() 方法用于唯一标识影响计算图的配置组合, 服务于 CUDA graph 缓存键:

```

# /workspace/vllm/config/lora.py L75-L99
def compute_hash(self) -> str:
    factors: list[Any] = []
    factors.append(self.max_lora_rank)
    factors.append(self.max_loras)
    factors.append(self.fully_sharded_loras)
    factors.append(self.lora_dtype)
    factors.append(self.enable_tower_connector_lora)
    factors.append(
        tuple(sorted(self.target_modules)) if self.target_modules else None
    )
    hash_str = safe_hash(str(factors).encode(), usedforsecurity=False).hexdigest()
    return hash_str

```

## 三、LoRAModel 模型封装

源码位置: lora\_model.py

LoRAModel 是一个已加载 LoRA adapter 的完整表示, 包含该 adapter 所有层的权重数据。

### 3.1 数据结构

```

# /workspace/vllm/lora/lora_model.py L24-L46
class LoRAModel:
    """A LoRA fine-tuned model."""

    def __init__(
        self,
        lora_model_id: int,          # 全局唯一的 LoRA ID (>0)
        rank: int,                  # 该 adapter 的实际 rank
        loras: dict[str, LoRALayerWeights], # module_name -> weights
    ) -> None:
        self.id = lora_model_id
        assert lora_model_id > 0, (
            f"a valid lora id should be greater than 0, got {self.id}"
        )
        self.rank = rank
        self.loras: dict[str, LoRALayerWeights] = loras

```

核心属性: - id: 全局唯一整数标识符, 用于索引 stacked 权重张量的第一维 - rank: 该 adapter 的实际 rank (必须 <= max\_lora\_rank) - loras: 字典结构 {module\_name: LoRALayerWeights}, key 为模型层名称 (如 model.layers.0.self\_attn.qkv\_proj)

### 3.2 从 Tensor 构建

```
# /workspace/vllm/lora/lora_model.py L73-L121
@classmethod
def from_lora_tensors(
    cls,
    lora_model_id: int,
    tensors: dict[str, torch.Tensor],    # 原始权重字典
    peft_helper: PEFTHelper,             # PEFT 配置信息
    device: str = "cuda",
    dtype: torch.dtype | None = None,
    model_vocab_size: int | None = None,
    weights_mapper: WeightsMapper | None = None,
    skip_prefixes: list[str] | None = None,
) -> "LoRAModel":
```

构建流程: 1. 遍历 `tensors` 中每个张量名称 2. 跳过 base embedding 权重和指定前缀的模块 3. 使用 `parse_fine_tuned_lora_name()` 解析出 `module_name` 和是否为 `lora_a` 4. 为每个 module 创建 `LoRALayerWeights` 对象并填入对应的 `lora_a` / `lora_b` 张量 5. 支持 `pin_memory` (CPU 端优化)

### 3.3 从本地 Checkpoint 加载

```
# /workspace/vllm/lora/lora_model.py L123-L244
@classmethod
def from_local_checkpoint(
    cls,
    lora_dir: str,                                # LoRA 目录路径
    expected_lora_modules: set[str],              # 期望的目标模块集合
    peft_helper: PEFTHelper,
    *,
    lora_model_id: int | None = None,
    device: str = "cuda",
    ...
) -> "LoRAModel":
```

支持的文件格式优先级: 1. **Tensorizer** 格式: `adapter_model.tensors` (通过 `TensorDeserializer`) 2. **SafeTensors** 格式: `adapter_model.safetensors` (推荐) 3. **PyTorch** 格式: `adapter_model.bin` 或 `adapter_model.pt`

模块校验机制: 加载时会检查 checkpoint 中的模块是否都在 `expected_lora_modules` 中, 防止加载错误的 adapter。

### 3.4 Clone 与查询接口

```
# /workspace/vllm/lora/lora_model.py L48-L63
def clone(self, lora_model_id: int) -> "LoRAModel":
    """ 返回共享底层张量的副本 (不同 ID) """
    return self.__class__(lora_model_id, rank=self.rank, loras=self.loras.copy())

def get_lora(self, module_name: str) -> LoRALayerWeights | None:
    """ 按模块名获取 LoRA 权重 """
    return self.loras.get(module_name, None)

def check_lora_name(self, lora_name: str) -> bool:
    return lora_name in self.loras
```

## 四、LoRAWeights 权重管理

源码位置: lora\_weights.py

定义了 LoRA 权重的数据结构和打包逻辑。

### 4.1 LoRALayerWeights 单层权重

```
# /workspace/vllm/lora/lora_weights.py L13-L96
class LoRALayerWeights:
    """LoRA weights for a layer composed of two low rank matrixes."""

    def __init__(
        self,
        module_name: str,          # 模块名
        rank: int,                 # rank
        lora_alpha: int,           # alpha 值
        lora_a: torch.Tensor,      # A 矩阵 shape: (rank, input_dim)
        lora_b: torch.Tensor,      # B 矩阵 shape: (output_dim, rank)
        scaling: float | None = None, # 缩放因子
    ) -> None:
        ...
        if scaling is None:
            self.scaling = self.lora_alpha / self.rank # 默认 scaling = alpha/rank
```

关键属性与方法:

| 属性/方法      | 返回值  | 说明                             |
|------------|------|--------------------------------|
| input_dim  | int  | lora_a.shape[1], 输入维度          |
| output_dim | int  | lora_b.shape[0], 输出维度          |
| is_packed  | bool | 是否为 packed 结构 (基类返回 False)     |
| optimize() | self | 将 scaling 合并入 lora_b (避免运行时乘法) |

**optimize** 操作: 将 scaling 因子预乘到 lora\_b 中, 减少推理时的计算量:

```
# /workspace/vllm/lora/lora_weights.py L36-L42
def optimize(self) -> "LoRALayerWeights":
    """Optimize the LoRA by merging the scaling into lora_b."""
    if self.scaling == 1:
        return self
    self.lora_b *= self.scaling
    self.scaling = 1
    return self
```

### 4.2 PackedLoRALayerWeights 一打包权重

用于 **packed layers** (如 qkv\_proj 将 Q/K/V 合并为一个线性层), 将多个子层的 LoRA 打包为一个对象:

```
# /workspace/vllm/lora/lora_weights.py L99-L249
class PackedLoRALayerWeights(LoRALayerWeights):
    def __init__(
        self,
        module_name: str,
        rank: int,
        lora_alphas: list[int | None], # 每个 sub-layer 的 alpha
        lora_a: list[torch.Tensor | None], # 每个 sub-layer 的 A
```

```

        lora_b: list[torch.Tensor | None], # 每个 sub-layer 的 B
        scaling: list[float] | None = None,
    ) -> None:

```

**pack** 方法: 将多个 LoRALayerWeights 打包为一个 PackedLoRALayerWeights:

```

@classmethod
def pack(cls, loras: GenericSequence["LoRALayerWeights | None"]) -> "PackedLoRALayerWeights":
    """Pack a list of LoRAs into a single LoRA.
    If LoRA is None, it signifies that the submodule does not have a LoRA.
    """

```

**pack\_moe** 方法: 专用于 MoE 层的打包, 按 (w1, w2, w3) 三元组组织 expert 权重:

# /workspace/vllm/lora/lora\_weights.py L154-L228

```

@classmethod
def pack_moe(
    cls,
    loras: GenericSequence["LoRALayerWeights | None"],
    module_name: str,
    is_non_gated_moe: bool = False,
) -> "PackedLoRALayerWeights":

```

MoE 打包的关键细节: - 按 num\_experts 遍历, 每个 expert 提取 (w1\_lora, w2\_lora, w3\_lora) - 使用 torch.stack 将同类型权重沿 expert 维度堆叠 - 对于 non-gated MoE (无 gate\_proj), 复用 w1 的权重作为 w3 并设置 last\_scaling=1.0 避免双重缩放

### 4.3 Dummy 权重创建

用于 warmup 和 slot 占位:

# /workspace/vllm/lora/lora\_weights.py L72-L96

```

@classmethod
def create_dummy_lora_weights(
    cls,
    module_name: str,
    input_dim: int,
    output_dim: int,
    rank: int,
    dtype: torch.dtype,
    device: torch.types.Device,
) -> "LoRALayerWeights":
    pin_memory = str(device) == "cpu" and is_pin_memory_available()
    lora_a = torch.zeros([rank, input_dim], dtype=dtype, device=device, pin_memory=pin_memory)
    lora_b = torch.zeros([output_dim, rank], dtype=dtype, device=device, pin_memory=pin_memory)
    return cls(module_name, rank=rank, lora_alpha=1, lora_a=lora_a, lora_b=lora_b)

```

## 五、LoRA Request 请求级绑定

源码位置: request.py

LoRARequest 是用户请求与 LoRA adapter 之间的绑定契约。

### 5.1 数据结构

# /workspace/vllm/lora/request.py L8-L66

```

class LoRARequest(

```

```

msgspec.Struct,
omit_defaults=True,
array_like=True,
):
    """
    Request for a LoRA adapter.

    lora_int_id must be globally unique for a given adapter.
    """

    lora_name: str # LoRA 名称 (人类可读)
    lora_int_id: int # 全局唯一整数 ID (> 0)
    lora_path: str = "" # Adapter 文件路径或 HuggingFace repo ID
    base_model_name: str | None = None # 基础模型名称
    tensorizer_config_dict: dict | None = None # Tensorizer 反序列化配置
    load_inplace: bool = False # 是否强制重新加载 (覆盖已有)

```

## 5.2 关键设计决策

**msgspec.Struct**: 使用 **msgspec** 而非 **dataclass**, 原因包括: - 高性能序列化/反序列化 - **array\_like=True** 支持 NumPy-like 的数组语义 - **omit\_defaults=True** 减少传输数据量

唯一性保证:

```

# /workspace/vllm/lora/request.py L32-L37
def __post_init__(self):
    if self.lora_int_id < 1:
        raise ValueError(f"id must be > 0, got {self.lora_int_id}")
    assert self.lora_path, "lora_path cannot be empty"

```

相等性与哈希: 基于 **lora\_name** 实现, 使得同名 adapter 在跨 engine 场景下可以正确去重:

```

# /workspace/vllm/lora/request.py L51-L66
def __eq__(self, value: object) -> bool:
    return isinstance(value, self.__class__) and self.lora_name == value.lora_name

def __hash__(self) -> int:
    return hash(self.lora_name)

```

便捷属性:

```

@property
def adapter_id(self):
    return self.lora_int_id

@property
def name(self):
    return self.lora_name

@property
def path(self):
    return self.lora_path

```

## 六、LoRA Layer 层实现

源码目录: layers/



vLLM 将原始模型中的 Linear/Embedding/MoE 层替换为带 LoRA 能力的版本，形成完整的层级继承体系。

## 6.1 类继承体系总览

```
graph TD
    NN["nn.Module"] --> BL["BaseLayerWithLoRA<br/> 抽象基类"]
    BL --> BLL["BaseLinearLayerWithLoRA<br/>Linear 公共逻辑"]

    BLL --> CPL["ColumnParallelLinearWithLoRA"]
    BLL --> MCPL["MergedColumnParallelLinearWithLoRA<br/>(gate_up_proj 等 2 slices)"]
    MCPL --> MCPLVS["MergedColumnParallelLinearVariableSliceWithLoRA<br/>(3+ slices)"]
    CPL --> QKVL["QKVParallelLinearWithLoRA<br/>(qkv_proj 1 slice)"]
    MCPL --> MQKVL["MergedQKVParallelLinearWithLoRA<br/>(qkv_proj 3 slices)"]

    CPL --> CPLS["ColumnParallelLinearWithShardedLoRA<br/>(S-LoRA)"]
    MCPL --> MCPLS["MergedColumnParallelLinearWithShardedLoRA"]
    QKVL --> QKVLS["QKVParallelLinearWithShardedLoRA"]
    MQKVL --> MQKVLS["MergedQKVParallelLinearWithShardedLoRA"]

    BLL --> RPL["RowParallelLinearWithLoRA"]
    RPL --> RPLS["RowParallelLinearWithShardedLoRA"]

    BLL --> RL["ReplicatedLinearWithLoRA"]

    BL --> FMoE["FusedMoEWithLoRA"]
    FMoE --> FMoE3D["FusedMoE3DWithLoRA"]

    BL --> LP["LogitsProcessorWithLoRA"]
    BL --> VPE["VocabParallelEmbeddingWithLoRA"]

    style BL fill:#e3f2fd,stroke:#1565c0
    style BLL fill:#bbdefb,stroke:#1976d2
    style CPL fill:#c8e6c9,stroke:#388e3c
    style RPL fill:#c8e6c9,stroke:#388e3c
    style FMoE fill:#fff9c4,stroke:#f9a825
    style LP fill:#f3e5f5,stroke:#7b1fa2
    style VPE fill:#ffccbc,stroke:#e64a19
```

## 6.2 BaseLayerWithLoRA — 抽象基类

源码: base.py

定义所有 LoRA layer 必须实现的接口契约:

*# /workspace/vllm/lora/layers/base.py L16-L78*

```
class BaseLayerWithLoRA(nn.Module):
    def slice_lora_a(self, lora_a): ...      # TP 分片 lora_a
    def slice_lora_b(self, lora_b): ...      # TP 分片 lora_b
    def create_lora_weights(...): ...        # 初始化 stacked 权重缓冲区
    def reset_lora(self, index: int): ...    # 重置某 slot 为零
    def set_lora(self, index, lora_a, lora_b): ... # 写入权重到指定 slot
    def set_mapping(self, punica_wrapper): ... # 绑定 PunicaWrapper
    @classmethod
    def can_replace_layer(cls, source_layer, lora_config, packed_modules_list, model_config): ...
```

### 6.3 BaseLinearLayerWithLoRA —Linear 公共基类

源码: base\_linear.py

封装所有 Linear 类型 LoRA layer 的公共逻辑, 包括权重缓冲区初始化、同步/异步 apply、dual-stream 支持等。

#### 6.3.1 权重缓冲区初始化

```
# /workspace/vllm/lora/layers/base_linear.py L106-L157
def create_lora_weights(
    self,
    max_loras: int,
    lora_config: LoRAConfig,
    model_config: PretrainedConfig | None = None,
) -> None:
    self.lora_config = lora_config
    # 根据 base_layer 类型决定输出维度
    if isinstance(self.base_layer, ReplicatedLinear):
        lora_a_out_size = lora_config.max_lora_rank
        lora_b_out_size = self.output_size
    elif isinstance(self.base_layer, ColumnParallelLinear):
        lora_a_out_size = (
            lora_config.max_lora_rank
            if not lora_config.fully_sharded_loras
            else divide(lora_config.max_lora_rank, self.tp_size)
        )
        lora_b_out_size = self.output_size
    elif isinstance(self.base_layer, RowParallelLinear):
        lora_a_out_size = lora_config.max_lora_rank
        lora_b_out_size = (
            self.output_size
            if not lora_config.fully_sharded_loras
            else divide(self.output_size, self.tp_size)
        )

    # 创建 stacked 权重缓冲区
    # shape: (max_loras, 1, lora_out_size, input/output_size)
    self.lora_a_stacked = tuple(
        torch.zeros(max_loras, 1, lora_a_out_size, self.input_size,
                    dtype=lora_config.lora_dtype, device=self.device)
        for _ in range(self.n_slices)
    )
    self.lora_b_stacked = tuple(
        torch.zeros(max_loras, 1, lora_b_out_size, lora_config.max_lora_rank,
                    dtype=lora_config.lora_dtype, device=self.device)
        for _ in range(self.n_slices)
    )
```

**stacked** 权重张量的形状设计: - 第 0 维 (max\_loras): 不同 LoRA adapter 的 slot - 第 1 维 (1): 为广播保留 (Punica kernel 要求) - 第 2-3 维: 实际权重矩阵维度

#### 6.3.2 同步与双流执行模式

```
# /workspace/vllm/lora/layers/base_linear.py L192-236
def apply(self, x: torch.Tensor, bias: torch.Tensor | None = None) -> torch.Tensor:
    if self._enable_aux_cuda_stream and is_forward_context_available():
```

```

        # 双 CUDA 流模式: base layer 和 LoRA 在不同流上并行执行
        output_size = sum(self.output_slices)
        return torch.ops.vllm.lora_linear_async(
            self.layer_name, output_size, x, bias
        )
    else:
        # 同步模式: 顺序执行
        return self._apply_sync(x, bias)

def _apply_sync(self, x, bias):
    output = self.base_layer.quant_method.apply(self.base_layer, x, bias)
    return self._apply_lora_to_output(x, output)

```

**Dual Stream 优化** (VLLM\_LORA\_ENABLE\_DUAL\_STREAM=True): - Base linear 在 default CUDA stream 上执行 - LoRA computation 在 auxiliary stream 上执行 - 两者通过 `torch.cuda.Event` 同步, 实现流水线重叠

### 6.3.3 LoRA 应用核心逻辑

```

# /workspace/vllm/lora/layers/base_linear.py L213-236
def _apply_lora_to_output(self, x: torch.Tensor, output: torch.Tensor) -> torch.Tensor:
    original_shape = output.shape if output.ndim == 3 else None

    # transformers backend 需要 flatten batch 维度
    if x.ndim == 3 and output.ndim == 3:
        output = output.flatten(0, 1)
        x = x.flatten(0, 1)

    # 调用 PunicaWrapper 执行批量 LoRA GEMM
    lora_output: torch.Tensor | None = self.punica_wrapper.add_lora_linear(
        output, x, self.lora_a_stacked, self.lora_b_stacked, 1.0, self.output_slices
    )
    if not current_platform.can_update_inplace():
        output = lora_output

    if original_shape is not None:
        output = output.reshape(original_shape)
    return output

```

## 6.4 ColumnParallelLinearWithLoRA

源码: `column_parallel_linear.py`

适用于 `ColumnParallelLinear` 及其变体的 LoRA 包装。核心特征是分片 `lora_b` (沿输出维度分片以匹配 TP)。

```

# /workspace/vllm/lora/layers/column_parallel_linear.py L102-L128
def slice_lora_b(self, lora_b: torch.Tensor) -> torch.Tensor:
    if self.is_merged_col_linear:
        # MergedColumnParallelLinear: 如 gate_up_proj, 需要拆分为两半再分别分片
        shard_size = self.output_size // 2
        offset = lora_b.shape[0] // 2
        left_weight = lora_b[self.tp_rank * shard_size : (self.tp_rank + 1) * shard_size, :]
        right_weight = lora_b[offset + self.tp_rank * shard_size : offset + (self.tp_rank + 1) * shard_size, :]
        lora_b = torch.cat([left_weight, right_weight], dim=0)
    else:
        # 标准 ColumnParallelLinear
        shard_size = self.output_size

```

```

        start_idx = self.tp_rank * shard_size
        end_idx = (self.tp_rank + 1) * shard_size
        lora_b = lora_b[start_idx:end_idx, :]
    return lora_b

```

子类族谱:

| 类名                                                  | 适用场景                                     | n_slices | 特殊逻辑                         |
|-----------------------------------------------------|------------------------------------------|----------|------------------------------|
| ColumnParallelLinearWithLoRA                        | 普通 ColumnParallelLinear                  | 1        | 分片 lora_b                    |
| MergedColumnParallelLinearWithLoRA                  | 第 2 种 slice 合并层                          | 2        | 分别处理两个 sub-lora              |
| QKVParallelLinearWithLoRA                           | 单一 LoRA (Q/K/V 分别按 head 数量分片)            | 1        | Q/K/V 分别按 head 数量分片          |
| MergedQKVParallelLinearWithLoRA                     | 三个独立 LoRA (Q/K/V 各自独立的 slice 和 sharding) | 3        | Q/K/V 各自独立的 slice 和 sharding |
| MergedColumnParallelLinearWithVariableSliceWithLoRA | 动态切片数                                    | 同左       | 动态切片数                        |
| *WithShardedLoRAS-LoRA fully-sharded 模式变体           | LoRA fully-sharded 模式                    | 同左       | 额外分片 lora_a                  |

**Fully-Sharded** 变体 (S-LoRA 策略): 基于论文 S-LoRA: Serving Thousands of Concurrent LoRA Adapters, 额外对 lora\_a 沿 rank 维度进行分片, 并在 `apply()` 中引入 `all_gather` + `add_expand` 两阶段计算:

```

# /workspace/vllm/lora/layers/column_parallel_linear.py L24-L80 (辅助函数 _mcp_apply)
def _mcp_apply(x, bias, layer):
    output = layer.base_layer.quant_method.apply(layer.base_layer, x, bias)
    x = x.view(-1, x.shape[-1])
    output, out_orig_shape = output.view(-1, output.shape[-1]), output.shape

    # Stage 1: Shrink (local GEMM with sharded lora_a)
    buffers = torch.empty_strided(buffer_shape, ...)
    buffers.zero_()
    shrunk_buffers = layer.punica_wrapper.add_shrink(buffers, x, layer.lora_a_stacked, 1.0)

    # All-gather intermediate results
    buffers = tensor_model_parallel_all_gather(buffers)

    # Stage 2: Expand (GEMM with lora_b, add to output)
    lora_output = layer.punica_wrapper.add_expand(
        output, buffers, layer.lora_b_stacked, layer.output_slices,
        offset_start=0, add_input=True,
    )
    ...

```

## 6.5 RowParallelLinearWithLoRA

源码: `row_parallel_linear.py`

适用于 `RowParallelLinear` 的 LoRA 包装。核心特征是分片 **lora\_a** (沿输入维度分片), 且 **lora\_b** 不分片 (因为 `RowParallel` 的输出需要 `all_reduce`)。

```

# /workspace/vllm/lora/layers/row_parallel_linear.py L32-40
def slice_lora_a(self, lora_a: torch.Tensor) -> torch.Tensor:
    shard_size = self.input_size
    start_idx = self.tp_rank * shard_size
    end_idx = (self.tp_rank + 1) * shard_size

```

```
lora_a = lora_a[:, start_idx:end_idx]
return lora_b
```

**Sharded** 变体 (RowParallelLinearWithShardedLoRA) 额外分片 `lora_b`, 采用类似 Column sharded 的 shrink + all\_reduce + expand 模式。

## 6.6 FusedMoEWithLoRA

源码: fused\_moe.py

最复杂的 LoRA layer, 为 MoE (Mixture of Experts) 层添加 LoRA 支持。涉及 `w1(gate_proj)`、`w2(down_proj)`、`w3(up_proj)` 三组权重的 LoRA 管理。

### 6.6.1 权重缓冲区结构

```
# /workspace/vllm/lora/layers/fused_moe.py L84-144
def _create_lora_a_weights(self, max_loras, lora_config):
    # w13 (w1/w3) lora_a: (max_loras, num_experts, rank, hidden_size) × _w13_slices
    self.w13_lora_a_stacked: tuple[torch.Tensor, ...] = tuple(
        torch.zeros((max_loras, self.base_layer.local_num_experts,
                     lora_config.max_lora_rank if not self.fully_sharded
                     else divide(lora_config.max_lora_rank, self.tp_size),
                     self.base_layer.hidden_size),
                     dtype=lora_config.lora_dtype, device=self.device)
        for _ in range(self._w13_slices) # gated MoE: 2 slices; non-gated: 1 slice
    )

    # w2 lora_a: (max_loras, num_experts, rank, intermediate_size_per_partition)
    self.w2_lora_a_stacked: tuple[torch.Tensor, ...] = (...)

def _create_lora_b_weights(self, max_loras, lora_config):
    # w13 lora_b: (max_loras, num_experts, intermediate_size_per_partition, rank)
    # w2 lora_b: (max_loras, num_experts, hidden_size[, //tp_size], rank)
```

特殊设计点: - 引入 `adapter_enabled` 张量跟踪哪些 slot 有有效 adapter - `_w13_slices`: gated MoE (如 Mixtral) 为 2 (`gate_proj` + `up_proj`), non-gated 为 1 - EP (Expert Parallelism) 兼容性检查

### 6.6.2 set\_lora 中的 TP 分片逻辑

```
# /workspace/vllm/lora/layers/fused_moe.py L282-344
def set_lora(self, index, lora_a, lora_b):
    assert isinstance(lora_a, list) # [w1_lora_a, w2_lora_a, w3_lora_a]
    assert isinstance(lora_b, list) # [w1_lora_b, w2_lora_b, w3_lora_b]

    self.reset_lora(index)
    self.adapter_enabled[index] = 1

    w1_lora_a, w2_lora_a, w3_lora_a = lora_a
    w1_lora_b, w2_lora_b, w3_lora_b = lora_b

    # 对 w1/w3 做 rank 维度分片 (if fully_sharded)
    slliced_w1_lora_a = self._slice_w13_a(w1_lora_a)
    slliced_w1_lora_b = self._slice_w13_b(w1_lora_b)

    # 对 w2 做相应分片
    sliced_w2_lora_a = self._slice_w2_a(w2_lora_a)
```

```

sliced_w2_lora_b = self._slice_w2_b(w2_lora_b)

# Copy to stacked buffers
self.w13_lora_a_stacked[0][index, :, :, :].copy_(sliced_w1_lora_a, non_blocking=True)
...

```

**6.6.3 FusedMoE3DWithLoRA** —3D MoE 变体 适用于 DeepSeek 等 3D MoE 架构，将 w1 和 w3 的 lora\_b 沿中间维度拼接：

```

# /workspace/vllm/lora/layers/fused_moe.py L388-401
def _create_lora_b_weights(self, max_loras, lora_config):
    # w13_lora_b: intermediate_size * 2 (w1+w3 concatenated)
    self.w13_lora_b_stacked: tuple[torch.Tensor] = tuple(
        torch.zeros((max_loras, self.base_layer.local_num_experts,
                      self.base_layer.intermediate_size_per_partition * 2,
                      lora_config.max_lora_rank), ...))
)

```

## 6.7 LogitsProcessorWithLoRA

源码: logits\_processor.py

为 LogitsProcessor (LM Head) 添加 LoRA 支持，允许 LoRA adapter 修改词汇表分布。

特殊之处：- vocab\_size 上限限制为 258048 - 处理 sharded-to-full vocabulary 映射 (TP 场景下的词表重排) - 使用专门的 add\_lora\_logits Punica 接口

```

# /workspace/vllm/lora/layers/logits_processor.py L141-190
def _get_logits(self, hidden_states, lm_head, embedding_bias=None):
    logits = actual_lm_head.quant_method.apply(actual_lm_head, hidden_states)
    if embedding_bias is not None:
        logits += embedding_bias

    logits = self.base_layer._gather_logits(logits)

    # Reindex for sharded vocab
    if self.sharded_to_full_mapping_gpu is not None:
        logits = logits[:, self.sharded_to_full_mapping_gpu]

    # Apply LoRA to logits
    lora_output = self.punica_wrapper.add_lora_logits(
        logits, hidden_states, self.lora_a_stacked, self.lora_b_stacked, 1.0
    )
    ...

```

## 6.8 ReplicatedLinearWithLoRA

源码: replicated\_linear.py

适用于 ReplicatedLinear (如 GateLinear)。无需 TP 分片 (因为权重本身是复制的)。

## 6.9 VocabParallelEmbeddingWithLoRA

源码: vocab\_parallel\_embedding.py

为 Embedding 层添加 LoRA，支持新增 token 的嵌入扩展。

关键实现：

```

# /workspace/vllm/lora/layers/vocal_parallel_embedding.py L96-126
def forward(self, x: torch.Tensor) -> torch.Tensor:
    num_tokens = x.shape[0]
    indices_1 = self.punica_wrapper._embeddings_indices[1][:num_tokens]

    # Lookup LoRA A embeddings (like an embedding table lookup)
    full_lora_a_embeddings = F.embedding(x + indices_1, self.lora_a_stacked_2d)

    full_output = self.base_layer.forward(x)

    # Apply LoRA: output += lora_A_embedding @ lora_B
    lora_output = self.punica_wrapper.add_lora_embedding(
        full_output, full_lora_a_embeddings, self.lora_b_stacked, add_input=True
    )
    return full_output.view_as(full_output_org)

```

注意: lora\_a 在 Embedding 场景下的角色类似于一个额外的嵌入查找表, 而非传统的矩阵乘法。

## 七、Punica Wrapper GPU LoRA 计算库集成

源码目录: punica\_wrapper/

Punica Wrapper 是 vLLM LoRA 系统的计算引擎抽象层, 封装了底层的 Triton/CUDA/XPU kernel 调用。

### 7.1 架构层次

graph TD

```

ABC["PunicaWrapperABC<br/> 抽象基类 (ABC)"] --> PB["PunicaWrapperBase<br/> 元数据管理 + 公共逻辑"]
PB --> PG["PunicaWrapperGPU<br/>Triton kernel 实现"]
PB --> PC["PunicaWrapperCPU<br/>PyTorch op 实现"]
PB --> PX["PunicaWrapperXPU<br/>Intel XPU IPEX kernel"]

```

```

PG --> |"lora_shrink"| TK1["Triton Shrink Kernel"]
PG --> |"lora_expand"| TK2["Triton Expand Kernel"]
PG --> |"fused_moe_lora"| TK3["Triton MoE LoRA Kernel"]
PC --> |"bgmv/sgmv"| T0["PyTorch Native Ops"]
PX --> |"bgmv"| X0["IPEX Custom Ops"]

```

```

style ABC fill:#ffebee,stroke:#c62828
style PB fill:#fff3e0,stroke:#e65100
style PG fill:#e8f5e9,stroke:#2e7d32
style PC fill:#e3f2fd,stroke:#1565c0
style PX fill:#f3e5f5,stroke:#7b1fa2

```

### 7.2 PunicaWrapperBase 一元数据中心

源码: punica\_base.py

维护 Multi-LoRA 推理所需的全部状态信息和映射数据。

#### 7.2.1 核心状态张量

```

# /workspace/vllm/lora/punica_wrapper/punica_base.py L131-166
class PunicaWrapperBase(PunicaWrapperABC):
    def __init__(self, max_num_batched_tokens, max_batches, device, **kwargs):

```

```

# Token 级别的 LoRA 索引映射 (decode 用)
self._token_lora_indices = torch.empty(
    max_num_batched_tokens, dtype=torch.long, device=device)
# Sampler 索引 (logits processor 用)
self._sampler_indices = torch.empty(...)
self._sampler_indices_padded = torch.empty(...)
# Embedding 索引 (embedding layer 用)
self._embeddings_indices = torch.empty(2, max_num_batched_tokens, ...)

# Prefill 阶段专用元数据
self._seq_start_locs = torch.empty(max_batches, ...) # 序列起始位置
self._seq_lengths = torch.empty(max_batches, ...) # 序列长度
self._lora_indices_per_batch = torch.empty(...) # 每 batch 的 LoRA ID

```

## 7.2.2 元数据更新流程

```

# /workspace/vllm/lora/punica_wrapper/punica_base.py L284-299
def update_metadata(self, mapping, lora_index_to_id, max_loras, vocab_size, **kwargs):
    # 1. 更新基础映射 (token → LoRA index)
    self._update_base_metadata(mapping, lora_index_to_id, max_loras, vocab_size)

    # 2. 如果是 prefill 阶段, 额外计算 batch 级元数据
    if mapping.is_prefill:
        self._update_prefill_metadata(self.token_lora_indices)
        self.is_prefill = True
    else:
        self.is_prefill = False

```

## 7.2.3 核心 Kernel 接口语义

| 方法                                                                | 语义公式                                                                                            | 用途                 |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------|
| <code>add_shrink(y, x, lora_a, scale)</code>                      | $y[i] += (x @ \text{lora\_a}[i]) * \text{scale}$                                                | LoRA A 的 GEMM (降维) |
| <code>add_expand(y, x, lora_b, slices, offset)</code>             | $y[:, \text{offset}:\text{offset}+\text{slice}] += x[i] @ \text{lora\_b}[i]$                    | LoRA B 的 GEMM (升维) |
| <code>add_lora_linear(y, x, lora_a, lora_b, scale, slices)</code> | shrink + expand 组合                                                                              | 完整 LoRA linear     |
| <code>add_lora_embedding(y, x, lora_b)</code>                     | $y += x @ \text{lora\_b}$                                                                       | Embedding LoRA     |
| <code>add_lora_logits(y, x, lora_a, lora_b, scale)</code>         | $\text{buffer} = (x @ \text{lora\_a}) * \text{scale};$<br>$y += \text{buffer} @ \text{lora\_b}$ | Logits LoRA        |
| <code>add_lora_fused_moe(...)</code>                              | fused MoE LoRA forward                                                                          | MoE LoRA           |
| <code>add_lora_w13(...)</code>                                    | w1/w3 LoRA + routing alignment                                                                  | MoE gate/up proj   |
| <code>add_lora_w2(...)</code>                                     | w2 LoRA (reuses w13 routing)                                                                    | MoE down proj      |

## 7.3 PunicaWrapperGPU — Triton 实现

源码: `punica_gpu.py`

使用 Triton 编写的自定义 kernel 实现, 是生产环境的主要 backend。

### 7.3.1 初始化与 CUDA Graph 专项化



```

# /workspace/vllm/lora/punica_wrapper/punica_gpu.py L40-73
def __init__(self, max_num_batched_tokens, max_batches, device, **kwargs):
    PunicaWrapperBase.__init__(self, ...)
    self.lora_config = kwargs["lora_config"]
    self.max_loras = self.lora_config.max_loras

    # 计算 captured LoRA counts 用于 CUDA Graph 专项化
    captured_lora_counts = get_captured_lora_counts(
        self.max_loras, self.lora_config.specialize_active_lora)

    # Token 级映射 meta (用于 decode 阶段的 sgmv)
    self.token_mapping_meta = LoRAKernelMeta.make(
        self.max_loras, max_num_batched_tokens, device=device,
        captured_lora_counts=captured_lora_counts)

    # Prompt 级映射 meta (用于 prefill/logits 的 sgmv)
    self.prompt_mapping_meta = LoRAKernelMeta.make(
        self.max_loras, max_num_batched_tokens, device=device,
        captured_lora_counts=captured_lora_counts)

```

**CUDA Graph 专项化 (specialize\_active\_lora):** 当启用时, 会为不同数量的活跃 LoRA (2 的幂次, 直到 max\_loras) 分别捕获 CUDA graph, 以优化不同负载模式的性能。

### 7.3.2 add\_lora\_linear 实现细节

```

# /workspace/vllm/lora/punica_wrapper/punica_gpu.py L203-264
def add_lora_linear(self, y, x, lora_a_stacked, lora_b_stacked, scale, output_slices, *, buffer=None, *):
    assert len(lora_a_stacked) == len(lora_b_stacked) == len(output_slices)
    assert buffer is None, "buffer should be created internally"

    r = lora_b_stacked[0].size(-1)
    # 创建 float32 buffer (Triton kernel 内部 zero)
    buffer = torch.empty(
        (len(output_slices), x.size(0), r),
        dtype=torch.float32, device=x.device)

    add_inputs = kwargs.pop("add_inputs", True)
    # Stage 1: Shrink - x @ lora_a -> buffer
    self.add_shrink(buffer, x, lora_a_stacked, scale, **kwargs)
    # Stage 2: Expand - buffer @ lora_b -> y
    self.add_expand(y, buffer, lora_b_stacked, output_slices, add_inputs=add_inputs, **kwargs)

```

**7.3.3 MoE LoRA 计算** MoE LoRA 是最复杂的计算路径, 涉及 token-expert routing 对齐和 fused GEMM:

```

# /workspace/vllm/lora/punica_wrapper/punica_gpu.py L489-620
def add_lora_w13(self, y, x, lora_a_stacked, lora_b_stacked,
    topk_ids, topk_weights, expert_map, w1, w2,
    num_tokens, top_k_num, max_loras, adapter_enabled,
    local_num_experts, top_k, num_slices, fully_sharded,
    use_tuned_config, token_lora_mapping=None):

    # 1. 获取最优 kernel config (tuned 或 heuristic)
    if use_tuned_config:
        shrink_config = get_lora_op_configs(op_type="fused_moe_lora_w13_shrink", ...)
        expand_config = get_lora_op_configs(op_type="fused_moe_lora_w13_expand", ...)

```

```

else:
    shrink_config = try_get_optimal_moe_lora_config(op_type="fused_moe_lora_w13_shrink", ...)
    ...

# 2. 对齐 block size (将 tokens 和 experts 组织为 block-sized chunks)
SPARSITY_FACTOR = 8
naive_block_assignment = (
    expert_map is None
    and num_tokens * top_k * SPARSITY_FACTOR <= local_num_experts * max_loras
)
(token_lora_mapping, sorted_token_ids_lora,
 expert_ids_lora, num_tokens_post_padded_lora) = self.moe_lora_align_block_size(...)

# 3. 执行 fused MoE LoRA forward
self.add_lora_fused_moe(y.view(-1, top_k_num, y.shape[-1]), x,
                        lora_a_stacked, lora_b_stacked, topk_weights,
                        _sorted, _eids, num_tokens_post_padded_lora, ...)

# 4. 返回 routing tensors 供 w2 复用
return (sorted_token_ids_lora, expert_ids_lora, num_tokens_post_padded_lora, token_lora_mapping)

```

## 7.4 PunicaWrapperCPU —PyTorch Op 实现

源码: `punica_cpu.py`

使用 PyTorch 原生操作实现, 适用于无 GPU/Triton 环境。区分 **prefill** 和 **decode** 两种模式:

```

# /workspace/vllm/lora/punica_wrapper/punica_cpu.py L38-163
class PunicaWrapperCPU(PunicaWrapperBase):
    # Decode: 使用 bgmv (batched GEMV) - per-token 粒度
    def _shrink_decode(self, y, x, w_t_all, scale):
        bgmv_shrink(x, w_t_all, y, self.token_lora_indices, scale)

    # Prefill: 使用 sgmv (segmented GEMV) - per-sequence 粒度
    def _shrink_prefill(self, y, x, w_t_all, scale):
        if self.no_lora:
            return
        sgmv_shrink(x, w_t_all, y, *self.prefill_metadata, scale)

```

**Prefill vs Decode** 的区别: - **Decode**: 每个 token 可能有不同的 LoRA, 使用 `token_lora_indices` 逐 token 索引 - **Prefill**: 同一 sequence 的所有 token 共享同一 LoRA, 使用 `prefill_metadata` (`seq_start_locs`, `seq_lengths`) 批量处理

## 7.5 PunicaWrapperXPU —Intel XPU 实现

源码: `punica_xpu.py`

针对 Intel XPU(GPU)平台的实现, 使用 IPEX 自定义 kernel(`bgmv_shrink`, `bgmv_expand`, `bgmv_expand_slice`), 并标记动态 shape 以支持 `torch.compile`:

```

# /workspace/vllm/lora/punica_wrapper/punica_xpu.py L46-48
def __init__(self, ...):
    ...
    torch._dynamo.mark_dynamic(self._token_lora_indices, 0)
    torch._dynamo.mark_dynamic(self._embeddings_indices, 1)
    torch._dynamo.mark_dynamic(self._sampler_indices_padded, 0)

```

## 八、LoRA Resolver 适配器解析

源码: resolver.py

定义 LoRA adapter 的发现与解析抽象接口，支持从不同来源（本地文件系统、HuggingFace Hub、云存储等）获取 adapter。

### 8.1 抽象接口

```
# /workspace/vllm/lora/resolver.py L14-41
class LoRAResolver(ABC):
    @abstractmethod
    async def resolve_lora(
        self, base_model_name: str, lora_name: str
    ) -> LoRARequest | None:
        """Abstract method to resolve and fetch a LoRA model adapter.

        Args:
            base_model_name: The name/identifier of the base model.
            lora_name: The name/identifier of the LoRA model to resolve.

        Returns:
            Optional[LoRARequest]: The resolved LoRA request, or None if not found.
        """
    pass
```

### 8.2 注册中心

```
# /workspace/vllm/lora/resolver.py L43-88
@dataclass
class _LoRAResolverRegistry:
    resolvers: dict[str, LoRAResolver] = field(default_factory=dict)

    def register_resolver(self, resolver_name: str, resolver: LoRAResolver) -> None: ...
    def get_resolver(self, resolver_name: str) -> LoRAResolver: ...
    def get_supported_resolvers(self) -> Set[str]: ...
```

```
LoRAResolverRegistry = _LoRAResolverRegistry() # 全局单例
```

注册中心设计：采用插件式架构，允许第三方注册自定义 resolver（如 S3 resolver、数据库 resolver 等），通过名称字符串进行查找。

---

## 九、WorkerManager Worker 级管理

源码: worker\_manager.py

WorkerLoRAManager 是 Worker 进程中 LoRA 生命周期的管理者，负责 adapter 的加载、卸载、激活和缓存。

### 9.1 架构角色

```
graph LR
    API["API Server<br/> 接收请求"] --> Engine["LLMEngine<br/> 调度中心"]
    Engine --> |"set_active_adapters()"| WM["WorkerLoRAManager<br/>Worker 端管理"]
    WM --> AM["LoRAModelManager<br/> 模型管理器"]
    AM --> |"slot 管理"| Slots["GPU Stacked Weights<br/>(max_loras slots)"]
```

```

WM --> |"_load_adapter()"| Disk[" 磁盘/HF Hub<br/>Adapter 文件"]
WM --> Resolver["LoRAResolver<br/> 路径解析"]

style WM fill:#ffe0b2,stroke:#e65100,stroke-width:2px
style AM fill:#fff3e0,stroke:#ef6c00,stroke-width:2px

```

## 9.2 WorkerLoRAManager —基础管理器

```

# /workspace/vllm/lora/worker_manager.py L25-223
class WorkerLoRAManager:
    """WorkerLoRAManager that manages LoRA models on the worker side.

    Every request, the requested LoRAs will be loaded (unless they are already
    loaded), and every other LoRA will be unloaded."""

```

核心生命周期方法:

| 方法                                                   | 功能                       |
|------------------------------------------------------|--------------------------|
| <code>create_lora_manager(model, vllm_config)</code> | 初始化 LoRAManager, 替换模型中的层 |
| <code>set_active_adapters(requests, mapping)</code>  | 主入口: 根据当前请求设置活跃 adapters |
| <code>add_adapter(adapter_request)</code>            | 加载单个 adapter 到 GPU       |
| <code>remove_adapter(adapter_id)</code>              | 卸载单个 adapter             |
| <code>list_adapters()</code>                         | 列出当前已加载的 adapter IDs     |
| <code>add_dummy_lora(lora_request, rank)</code>      | 添加 dummy LoRA (warmup 用) |
| <code>pin_adapter(adapter_id)</code>                 | 固定 adapter 不被驱逐          |

### 9.2.1 Adapter 加载流程

```

# /workspace/vllm/lora/worker_manager.py L99-157
def _load_adapter(self, lora_request: LoRARequest) -> LoRAModel:
    # 1. 确定期望的目标模块列表
    supported_lora_modules = self._adapter_manager.supported_lora_modules
    packed_modules_mapping = self._adapter_manager.packed_modules_mapping
    expected_lora_lst = []
    for module in supported_lora_modules:
        if module in packed_modules_mapping:
            expected_lora_lst.extend(packed_modules_mapping[module])
        else:
            expected_lora_lst.append(module)

    # 2. 解析 adapter 绝对路径
    lora_path = get_adapter_absolute_path(lora_request.lora_path)

    # 3. 加载 PEFT 配置
    peft_helper = PEFTHelper.from_local_dir(lora_path, ...)

    # 4. 校验 LoRA 配置合法性
    peft_helper.validate_legal(self.lora_config)

    # 5. 获取模型特定的权重映射和跳过前缀
    hf_to_vllm_mapper = getattr(model, "hf_to_vllm_mapper", None)
    lora_skip_prefixes = getattr(model, "lora_skip_prefixes", None)

    # 6. 从 checkpoint 加载 LoRAModel (到 CPU)

```

```

lora = self._lora_model_cls.from_local_checkpoint(
    lora_path, expected_lora_modules, peft_helper=peft_helper,
    lora_model_id=lora_request.lora_int_id, device="cpu", ...)
return lora

```

### 9.2.2 活跃 Adapter 应用策略

```

# /workspace/vllm/lora/worker_manager.py L178-206
def _apply_adapters(self, adapter_requests: set[Any]) -> None:
    existing_adapters = self.list_adapters()
    models_map = {
        adapter_request.adapter_id: adapter_request
        for adapter_request in adapter_requests if adapter_request
    }
    # 容量检查
    if len(models_map) > self._adapter_manager.adapter_slots:
        raise RuntimeError(f"Requested models ({len(models_map)}) > slots ({self._adapter_manager.adapter_slots})")

    requested_ids = set(models_map)
    # 卸载不再需要的 adapters
    for adapter_id in existing_adapters - requested_ids:
        self.remove_adapter(adapter_id)
    # 加载新需要的 adapters
    for adapter_id in requested_ids - existing_adapters:
        self.add_adapter(models_map[adapter_id])

```

策略特点：精确匹配策略——只保留当前批次需要的 adapters，其余全部卸载。

### 9.3 LRUWorkerLoRAManager —LRU 缓存管理器

继承 WorkerLoRAManager，增加 LRU 缓存淘汰 策略，适合频繁切换 adapter 的场景：

```

# /workspace/vllm/lora/worker_manager.py L226-302
class LRUWorkerLoRAManager(WorkerLoRAManager):
    """Uses an LRU Cache. Least recently used LoRAs will be unloaded
    if the cache is above capacity."""

    def add_adapter(self, lora_request: LoRARequest) -> bool:
        if (lora_request.lora_int_id not in self.list_adapters()
            or lora_request.load_inplace):
            # 先加载新 adapter (确保有效性)
            lora = self._load_adapter(lora_request)

            # 移除已有版本 (支持 inplace 更新)
            self._adapter_manager.remove_adapter(lora.id)

            # 容量超限时驱逐最久未使用的 adapter
            if len(self._adapter_manager) + 1 > self._adapter_manager.capacity:
                self._adapter_manager.remove_oldest_adapter()

            # 添加新 adapter
            loaded = self._adapter_manager.add_adapter(lora)
        else:
            # 已缓存则 touch 更新 LRU 位置
            loaded = (self._adapter_manager.get_adapter(lora_request.lora_int_id) is not None)

```

```

self._adapter_manager.activate_adapter(lora_request.lora_int_id)
return loaded

```

LRU 策略 vs 基础策略对比:

| 特性         | WorkerLoRAManager    | LRUCacheWorkerLoRAManager        |
|------------|----------------------|----------------------------------|
| 卸载策略       | 精确匹配（不需要的全卸载）        | LRU 淘汰（超容量时驱逐最旧的）                |
| 已缓存命中      | 直接跳过                 | touch 更新访问时间                     |
| Inplace 更新 | 不支持                  | 支持 ( <code>load_inplace</code> ) |
| 适用场景       | 固定 few-shot adapters | 大量 adapters 轮转                   |

## 十、动态加载/卸载机制

### 10.1 完整请求处理流程

sequenceDiagram

```

participant User as 用户请求
participant Engine as LLMEngine
participant Scheduler as Scheduler
participant Worker as WorkerLoRAManager
participant Manager as LoRAModelManager
participant Model as LoRA Layers
participant Punica as PunicaWrapper

```

```

User->>Engine: add_request(lora_request)
Engine->>Scheduler: schedule()

```

alt 新 LoRA 请求

```

Scheduler->>Worker: set_active_adapters({req1, req2})
Worker->>Worker: _apply_adapters()
Worker->>Worker: add_adapter(new_req)
Worker->>Worker: _load_adapter(new_req)
Worker-->>Disk: from_local_checkpoint()
Disk-->>Worker: LoRAModel (CPU)
Worker->>Manager: add_adapter(lora_model)
Manager->>Model: set_lora(slot_id, lora_a, lora_b)
Manager->>Punica: update_metadata(mapping)

```

end

```

Scheduler->>Worker: execute_model()
Worker->>Model: forward(hidden_states)
Model->>Punica: add_lora_linear(output, x, ...)
Punica-->>Model: LoRA applied output
Model-->>Worker: output_with_lora

```

alt LoRA 不再需要

```

Scheduler->>Worker: set_active_adapters({remaining})
Worker->>Manager: remove_adapter(old_id)
Manager->>Model: reset_lora(old_slot)

```

end

10.2 Slot-Based 权重管理

vLLM 采用 **pre-allocated slot** 策略管理 GPU 上的 LoRA 权重:

- 1. 初始化阶段: `create_lora_weights(max_loras)` 预分配形状为 `(max_loras, 1, rank, dim)` 的零张量
- 2. 加载阶段: `set_lora(slot_index, lora_a, lora_b)` 将权重 `copy_` 到对应 slot
- 3. 激活阶段: 通过 `LoRAMapping` 建立 `token → slot_index` 的映射关系
- 4. 计算阶段: Punica kernel 根据 `token_lora_indices` 从对应 slot 取权重计算
- 5. 卸载阶段: `reset_lora(slot_index)` 将对应 slot 清零

10.3 LoRAMapping —Token 到 Adapter 的映射

LoRAMapping 是连接请求级 LoRA binding 和 kernel 级计算的桥梁数据结构:

- **Decode** 阶段: 每个 token 对应一个 `lora_index` (-1 表示无 LoRA), 组成 `token_lora_indices` 向量
- **Prefill** 阶段: 每个 sequence 共享一个 `lora_index`, 配合 `seq_start_locs` 和 `seq_lengths` 使用

10.4 性能优化要点

| 优化技术                      | 说明                                                                               | 代码位置                                |
|---------------------------|----------------------------------------------------------------------------------|-------------------------------------|
| Batched GEMM              | 多个 LoRA 的 A/B 矩阵堆叠为 4D 张量, 单次 kernel 调用处理                                        | <code>lora_a/b_stacked</code>       |
| Scaling Pre-fusion        | <code>optimize()</code> 将 <code>alpha/r</code> 合入 <code>lora_b</code> , 消除运行时乘法  | <code>lora_weights.py</code> L36    |
| Dual CUDA Stream          | Base linear 和 LoRA 在不同流上并行执行                                                     | <code>base_linear.py</code> L238    |
| CUDA Graph Specialization | 按活跃 LoRA 数量 (2 的幂次) 专门化 graph                                                    | <code>specialize_active_lora</code> |
| Float32 Buffer            | Shrink 中间结果使用 float32, 参考 Triton issue #1387                                     | <code>punica_gpu.py</code> L246     |
| Dummy LoRA Reuse          | Warmup 阶段复用同一个 dummy LoRA 对象 (clone 不同 ID)                                       | <code>worker_manager.py</code> L159 |
| Non-blocking Copy         | <code>copy_(non_blocking=True)</code> 异步拷贝权重                                     | <code>set_lora</code> 方法            |
| S-LoRA Sharding           | Fully-sharded 模式下额外分片 <code>lora_a</code> , 减少通信量                                | <code>*WithShardedLoRA</code> 子类    |
| MoE Routing Reuse         | w13 的 routing 结果 ( <code>sorted_ids</code> , <code>expert_ids</code> ) 直接供 w2 复用 | <code>punica_gpu.py</code> L615     |

附录: 关键文件索引

| 文件                                | 核心职责                                           |
|-----------------------------------|------------------------------------------------|
| <code>config/lora.py</code>       | LoRAConfig 配置定义与校验                             |
| <code>lora/lora_model.py</code>   | LoRAModel 封装与 checkpoint 加载                    |
| <code>lora/lora_weights.py</code> | LoRALayerWeights / PackedLoRALayerWeights 数据结构 |

| 文件                                      | 核心职责                                                    |
|-----------------------------------------|---------------------------------------------------------|
| lora/request.py                         | LoRARequest 请求绑定数据结构                                    |
| lora/layers/base.py                     | BaseLayerWithLoRA 抽象基类                                  |
| lora/layers/base_linear.py              | BaseLinearLayerWithLoRA 公共逻辑                            |
| lora/layers/column_parallel_linear.py   | Column Parallel Linear + LoRA (含 QKV/Merged/Sharded 变体) |
| lora/layers/row_parallel_linear.py      | Row Parallel Linear + LoRA (含 Sharded 变体)               |
| lora/layers/fused_moe.py                | FusedMoE + LoRA (含 3D MoE 变体)                           |
| lora/layers/logits_processor.py         | LogitsProcessor + LoRA                                  |
| lora/layers/replicated_linear.py        | ReplicatedLinear + LoRA                                 |
| lora/layers/vocab_parallel_embedding.py | VocabParallelEmbedding + LoRA                           |
| lora/punica_wrapper/punica_base.py      | PunicaWrapper 抽象基类与元数据管理                                |
| lora/punica_wrapper/punica_gpu.py       | GPU/Triton kernel 实现                                    |
| lora/punica_wrapper/punica_cpu.py       | CPU PyTorch op 实现                                       |
| lora/punica_wrapper/punica_xpu.py       | Intel XPU/IPEX 实现                                       |
| lora/resolver.py                        | LoRA 适配器解析抽象接口与注册中心                                     |
| lora/worker_manager.py                  | Worker 级 LoRA 生命周期管理                                    |

## vLLM 服务接口层 (API Layer) 源码分析

定位：本文档分析 vLLM 的服务接口层 (Service/API Layer)，涵盖从 HTTP/gRPC 服务器入口、多协议 API 路由、CLI 命令到高级编程接口的完整链路。该层位于 Engine 层之上，负责将外部请求转化为引擎可处理的内部调用。

graph TB

```

subgraph API[" 服务接口层 Service Interface Layer"]
    direction TB
    HTTP["HTTP/FastAPI 服务器 <br/>api_server.py"]
    gRPC["gRPC Server<br/>grpc_server.py"]
    CLI["CLI 命令行 <br/>cli/"]
    LLM["LLM 高级 API<br/>llm.py"]
end

```

```

subgraph Routes[" 核心路由 Core Routes"]
    direction LR
    R1["/v1/chat/completions"]
    R2["/v1/completions"]
    R3["/v1/embeddings"]
    R4["/v1/messages<br/>Anthropic"]
    R5["/ping, /invocations<br/>SageMaker"]
    R6["/score, /rerank<br/>Pooling"]
end

```

```

subgraph Aux[" 辅助功能 Auxiliary Functions"]
    direction LR
    A1["Orca Metrics"]
    A2["SSL/TLS 热更新"]
    A3[" 访问日志过滤"]
    A4["Batch Serving"]
    A5["MCP Tool Server"]
end

```

```

HTTP --> Routes
gRPC --> Routes

```



```

CLI --> HTTP
CLI --> gRPC
CLI --> LLM
LLM --> HTTP
Aux --> HTTP

style API fill:#fce4ec,stroke:#c62828,stroke-width:2px
style Routes fill:#e3f2fd,stroke:#1565c0,stroke-width:2px
style Aux fill:#fff3e0,stroke:#ef6c00,stroke-width:2px

```

---

## 目录

- 一、OpenAI 兼容 API
    - 1.1 FastAPI 服务器入口
    - 1.2 路由注册体系
    - 1.3 核心端点定义
    - 1.4 流式响应 SSE 实现
    - 1.5 Orca Metrics
  - 二、Anthropic API
    - 2.1 协议模型 protocol.py
    - 2.2 路由定义 api\_router.py
    - 2.3 消息格式转换 serving.py
  - 三、gRPC Server
  - 四、CLI 入口
  - 五、Batch Serving 离线批处理
  - 六、SageMaker 集成
  - 七、MCP Tool Server 模型上下文协议工具服务
  - 八、Pooling 端点 嵌入池化接口
  - 九、LLM 高级 API
  - 十、SSL/TLS 支持
  - 十一、请求日志与访问过滤
  - 十二、API 请求路由与处理流程总览
- 

## 一、OpenAI 兼容 API

### 1.1 FastAPI 服务器入口

vLLM 的 OpenAI 兼容服务器入口位于 `api_server.py`，基于 FastAPI 框架构建。整个启动流程分为以下几个关键阶段：

#### 1.1.1 引擎客户端构建

```

# file: vllm/entrypoints/openai/api_server.py
# line: 77-105

@asynccontextmanager
async def build_async_engine_client(
    args: Namespace,
    *,
    usage_context: UsageContext = UsageContext.OPENAI_API_SERVER,
    client_config: dict[str, Any] | None = None,
) -> AsyncIterator[EngineClient]:

```

```

# 支持 forserver 多进程预加载模式
if os.getenv("VLLM_WORKER_MULTIPROC_METHOD") == "forkserver":
    multiprocessing.set_start_method("forkserver")
    multiprocessing.set_forkserver_preload(["vllm.v1.engine.async_llm"])
    forkserver.ensure_running()

engine_args = AsyncEngineArgs.from_cli_args(args)
# ... client_config 处理 ...

async with build_async_engine_client_from_engine_args(
    engine_args,
    usage_context=usage_context,
    client_config=client_config,
) as engine:
    yield engine

```

核心引擎创建逻辑在 `build_async_engine_client_from_engine_args` 中（第 108-154 行），它通过 `AsyncLLM.from_vllm_config()` 创建 V1 版本的 AsyncLLM 引擎实例，支持进程内直接使用和 **multiprocess RPC** 两种模式。

**1.1.2 FastAPI 应用构建—`build_app()`** `build_app()` 是整个 API 服务的核心装配函数,根据 `supported_tasks` 动态注册路由:

```

# file: vllm/entrypoints/openai/api_server.py
# line: 157-314 (核心部分)

def build_app(args, supported_tasks=None, model_config=None) -> FastAPI:
    app = FastAPI(lifespan=lifespan)

    # 1. 注册 vLLM Serve 扩展路由 (render/tokenize 等)
    register_vllm_serve_api_routers(app)

    # 2. 注册模型管理路由 (/v1/models)
    register_models_api_router(app)

    # 3. 注册 SageMaker 兼容路由
    register_sagemaker_api_router(app, supported_tasks, model_config)

    # 4. 根据 supported_tasks 条件注册各类生成路由
    if "generate" in supported_tasks:
        register_generate_api_routers(app)      # chat/completions/responses/anthropic
        attach_disagg_router(app)               # disaggregated serving
        attach_rlhf_router(app)                 # RLHF
        elastic_ep_attach_router(app)           # Elastic EP
        register_generative_scoring_api_router(app)

    if any(task in POOLING_TASKS for task in supported_tasks):
        register_pooling_api_routers(app, supported_tasks, model_config)

    # 5. 中间件配置
    app.add_middleware(CORSMiddleware, ...)
    app.exception_handler(HTTPException)(http_exception_handler)
    # ... 认证中间件、X-Request-ID 中间件、ScalingMiddleware ...

```

关键设计要点: - 条件化路由注册: 仅当 `supported_tasks` 包含对应能力时才注册路由, 避免无功能端点暴露 -

多层中间件栈: CORS → 认证 → RequestID → Scaling → WebSocket Metrics → 自定义中间件 - 异常处理  
分级: HTTPException / ValidationError / EngineGenerateError / EngineDeadError / GenerationError / Exception 六级异常处理器

## 1.2 路由注册体系

vLLM 采用模块化路由注册策略, 各子模块独立管理自己的路由:

```
graph TD
    App["build_app()"] --> SR["register_vllm_serve_api_routers()"]
    App --> MR["register_models_api_router()"]
    App --> SMR["register_sagemaker_api_router()"]
    App --> GR{"'generate' in tasks?"}
    GR -->|Yes| GAR["register_generate_api_routers()"]
    GAR --> CR["chat_completion router<br>/v1/chat/completions"]
    GAR --> COR["completion router<br>/v1/completions"]
    GAR --> RR["responses router<br>/v1/responses"]
    GAR --> AR["anthropic router<br>/v1/messages"]
    App --> PR{"pooling task?"}
    PR -->|Yes| PoolR["register_pooling_api_routers()<br>/v1/embeddings, /score, /rerank"]

    style App fill:#fce4ec,stroke:#c62828
    style GAR fill:#e8f5e9,stroke:#2e7d32
    style PoolR fill:#e3f2fd,stroke:#1565c0
```

所有路由注册器位于 generate/api\_router.py:

```
def register_generate_api_routers(app: FastAPI):
    register_chat_api_router(app)           # Chat Completions
    register_responses_api_router(app)       # Responses API
    register_completion_api_router(app)      # Text Completions
    register_anthropic_api_router(app)       # Anthropic Messages
```

## 1.3 核心端点定义

**1.3.1 Chat Completions** — /v1/chat/completions 定义于 chat\_completion/api\_router.py:

```
@router.post(
    "/v1/chat/completions",
    dependencies=[Depends(validate_json_request)],
    responses={
        HTTPStatus.OK.value: {"content": {"text/event-stream": {}}},
        ...
    },
)
@with_cancellation
@load_aware_call
async def create_chat_completion(request: ChatCompletionRequest, raw_request: Request):
    metrics_header_format = raw_request.headers.get(
        ENDPOINT_LOAD_METRICS_FORMAT_HEADER_LABEL, ""
    )
    handler = chat(raw_request)
    generator = await handler.create_chat_completion(request, raw_request)

    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(), status_code=generator.error.code)
    elif isinstance(generator, ChatCompletionResponse):
```

```

        return JSONResponse(content=generator.model_dump(),
                             headers=metrics_header(metrics_header_format))
    return StreamingResponse(content=generator, media_type="text/event-stream")

```

此外还支持 **Batch Chat Completion** 端点 `/v1/chat/completions/batch` (同文件第 77-102 行)。

**1.3.2 Text Completions** `—/v1/completions` 通过 `register_completion_api_router` 注册, 处理传统文本补全请求。

**1.3.3 Embeddings** `—/v1/embeddings & /v2/embed` 定义于 `pooling/embed/api_router.py`:

```

@router.post("/v1/embeddings", ...)
@with_cancellation
@load_aware_call
async def create_embedding(request: EmbeddingRequest, raw_request: Request):
    handler = embedding(raw_request)
    if handler is None:
        raise NotImplementedError("The model does not support Embeddings API")
    return await handler(request, raw_request)

```

同时提供 Cohere 兼容的 `/v2/embed` 端点。

## 1.4 流式响应 SSE 实现

vLLM 的流式响应遵循 **Server-Sent Events (SSE)** 协议。以 Chat Completion 为例:

```

sequenceDiagram
    participant Client
    participant FastAPI as FastAPI Router
    participant Serving as OpenAIServingChat
    participant Engine as AsyncLLM Engine

    Client->>FastAPI: POST /v1/chat/completions<br/>stream: true
    FastAPI->>Serving: create_chat_completion(request)
    Serving->>Engine: 异步生成请求
    Engine-->>Serving: AsyncGenerator[str]
    Serving-->>FastAPI: AsyncGenerator[SSE chunks]
    FastAPI-->>Client: StreamingResponse<br/>text/event-stream

    loop 每个 token chunk
        FastAPI-->>Client: data: {...}\n\n
    end
    FastAPI-->>Client: data: [DONE]\n\n

```

返回类型判断逻辑 (`chat_completion/api_router.py`): `- ErrorResponse → JSONResponse + 错误状态码 - ChatCompletionResponse (非流式完整响应) → JSONResponse + Orca Metrics header - AsyncGenerator (流式) → StreamingResponse(media_type="text/event-stream")`

## 1.5 Orca Metrics

`orca_metrics.py` 实现了 **ORCA (Open Runtime Compute Architecture)** 端点负载报告标准。该机制允许服务端在 HTTP 响应头中向负载均衡器 (如 Envoy) 报告实时负载指标。

核心数据流

```

# file: vllm/entrypoints/openai/orca_metrics.py
# line: 100-120

```

```
def metrics_header(metrics_format: str) -> Mapping[str, str] | None:
    """ 创建 ORCA endpoint-load-metrics 响应头 """
    if not metrics_format:
        return None
    named_metrics = get_named_metrics_from_prometheus()
    return create_orca_header(metrics_format, named_metrics)
```

指标采集过程: 1. 从 Prometheus 快照获取 Gauge 类型指标 2. 映射 Prometheus 指标名到 ORCA 名称 (如 vllm:kv\_cache\_usage\_perc → kv\_cache\_usage\_perc) 3. 支持两种输出格式: - **TEXT** 格式: endpoint-load-metrics: TEXT named\_metrics.kv\_cache\_utilization=0.4 - **JSON** 格式: endpoint-load-metrics: JSON {"named\_metrics": {...}}

## 二、Anthropic API

vLLM 提供了 **Anthropic Messages API** 的兼容实现, 使 Claude SDK 可以无缝对接 vLLM 后端。

### 2.1 协议模型 protocol.py

protocol.py 定义了完整的 Anthropic API 数据模型, 核心类包括:

| Pydantic Model                       | 用途                                                                  |
|--------------------------------------|---------------------------------------------------------------------|
| AnthropicMessagesRequest             | Messages API 请求体 (model, messages, max_tokens, stream, tools...)    |
| AnthropicMessagesResponse            | Messages API 响应体 (id, content, stop_reason, usage...)               |
| AnthropicStreamEvent                 | 流式事件 (message_start/delta/stop/content_block_start/delta/stop)      |
| AnthropicContentBlock                | 内容块<br>(text/image/tool_use/tool_result/thinking/redacted_thinking) |
| AnthropicCountTokensRequest/Response | Token 计数请求/响应                                                       |
| AnthropicTool / AnthropicToolChoice  | 工具定义和工具选择参数                                                         |

关键设计——vLLM 扩展字段:

```
# file: vllm/entrypoints/anthropic/protocol.py
# line: 120-130

class AnthropicMessagesRequest(BaseModel):
    # ... 标准 Anthropic 字段 ...
    kv_transfer_params: dict[str, Any] | None = Field(
        default=None,
        description="KVTransfer parameters used for disaggregated serving.",
    )
    chat_template_kwargs: dict[str, Any] | None = Field(
        default=None,
        description="Additional keyword args to pass to the chat template renderer.",
    )
```

### 2.2 路由定义 api\_router.py

api\_router.py 注册两个端点:

```
# file: vllm/entrypoints/anthropic/api_router.py
# line: 48-92
```

```
@router.post("/v1/messages", ...)          # Messages API 主端点
@with_cancellation
@load_aware_call
async def create_messages(request: AnthropicMessagesRequest, raw_request: Request):
    handler = messages(raw_request)
    generator = await handler.create_messages(request, raw_request)
    # 返回: ErrorResponse / AnthropicMessagesResponse / StreamingResponse

@router.post("/v1/messages/count_tokens", ...) # Token 计数端点
@load_aware_call
@with_cancellation
async def count_tokens(request: AnthropicCountTokensRequest, raw_request: Request):
    response = await handler.count_tokens(request, raw_request)
    return JSONResponse(content=response.model_dump(exclude_none=True))
```

错误响应通过 `translate_error_response()` 将 vLLM 的 `ErrorResponse` 转换为 Anthropic 格式的 `AnthropicErrorResponse`。

## 2.3 消息格式转换 `serving.py`

`serving.py` 中的 `AnthropicServingMessages` 类继承自 `OpenAIServingChat`，核心职责是 **Anthropic OpenAI** 消息格式互转：

graph LR

```
    subgraph Input[" 请求转换"]
        AR["Anthropic Request"] --> Convert["_convert_anthropic_to_openai_request()"]
        Convert --> OR["OpenAI ChatCompletionRequest"]
    end

    subgraph Process[" 引擎处理"]
        OR --> Engine["AsyncLLM Engine"]
        Engine --> OGen["OpenAI Stream/Response"]
    end

    subgraph Output[" 响应转换"]
        OGen --> Full{" 非流式?"}
        Full -->|Yes| FConv["messages_full_converter()"]
        Full -->|No| SConv["message_stream_converter()"]
        FConv --> AResp["AnthropicMessagesResponse"]
        SConv --> AStream["Anthropic SSE Events"]
    end

    style AR fill:#dcedc8,stroke:#558b2f
    style AResp fill:#ffcd2,stroke:#c62828
    style AStream fill:#ffcd2,stroke:#c62828
```

格式转换的核心方法 `_convert_anthropic_to_openai_request()` (第 121-133 行) 依次执行: 1. 系统消息转换 (`_convert_system_message`): 处理字符串型和结构化 `system prompt` 2. 用户/助手消息转换 (`_convert_messages`): 遍历消息列表 3. 内容块转换 (`_convert_block`): 逐块处理 `text/image/thinking/tool_use/tool_result/tool` 4. 基础请求构建 (`_build_base_request`): 组装 `ChatCompletionRequest` 5. 流式选项处理 (`_handle_streaming_options`) 6. 工具选择/工具定义转换 (`_convert_tool_choice`, `_convert_tools`)

流式响应转换器 `message_stream_converter()` (第 511-811 行) 是一个复杂的状态机，维护 `_ActiveBlockState`

来追踪当前活跃的内容块 (thinking/text/tool\_use) , 将 OpenAI 的 delta chunks 转换为 Anthropic 的 content\_block\_start → content\_block\_delta → content\_block\_stop 事件序列。

### 三、gRPC Server

gRPC Server 为需要高性能 RPC 通信的场景提供了基于 gRPC 的服务入口。

架构概要

```
# file: vllm/entrypoints/grpc_server.py
# line: 56-165

async def serve_grpc(args: argparse.Namespace):
    # 1. 创建引擎
    engine_args = AsyncEngineArgs.from_cli_args(args)
    vllm_config = engine_args.create_engine_config(usage_context=UsageContext.OPENAI_API_SERVER)
    async_llm = AsyncLLM.from_vllm_config(vllm_config=vllm_config, ...)

    # 2. 创建 Servicer
    servicer = VllmEngineServicer(async_llm, start_time)

    # 3. 创建 gRPC Server 并配置
    server = grpc.aio.server(options=[
        ("grpc.max_send_message_length", -1),      # 无限消息大小
        ("grpc.max_receive_message_length", -1),
        ("grpc.http2.min_recv_ping_interval_without_data_ms", 10000), # 宽松 keepalive
        ("grpc.keepalive_permit_without_calls", True),
    ])

    # 4. 注册服务
    vllm_engine_pb2_grpc.add_VllmEngineServicer_to_server(servicer, server)
    health_pb2_grpc.add_HealthServicer_to_server(health_servicer, server)
    reflection.enable_server_reflection(service_names, server) # 启用反射

    # 5. 绑定端口并启动
    server.add_insecure_port(f"{args.host}:{args.port}")
    await server.start()
```

关键特性

| 特性                | 说明                                                           |
|-------------------|--------------------------------------------------------------|
| Protobuf 定义       | 来自 smg_grpc_proto 包的 vllm_engine.proto                       |
| Health Check      | 集成标准 gRPC Health Checking (Kubernetes 就绪探针)                  |
| Server Reflection | 启用 gRPC 反射, 支持 grpcurl 等工具调试                                 |
| 无限消息大小            | max_send/receive_message_length = -1, 适配大模型输出                |
| 宽松 Keepalive      | 最小 ping 间隔 10s (默认 300s), 避免长生成超时                            |
| 优雅关闭              | SIGTERM/SIGINT 处理 → NOT_SERVING → 5s grace period → shutdown |

## 四、CLI 入口

vLLM 的 CLI 系统采用子命令架构，入口为 `cli/main.py`。

### 4.1 命令分发架构

```
graph TD
    Main["vllm (main.py)"] --> Sub{子命令解析}
    Sub -->| "serve" | Serve["serve.py<br/> 启动 API 服务器"]
    Sub -->| "run-batch" | Batch["run_batch.py<br/> 离线批处理"]
    Sub -->| "chat" | Chat["openai.py:ChatCommand<br/> 交互式聊天"]
    Sub -->| "complete" | Comp["openai.py:CompleteCommand<br/> 文本补全"]
    Sub -->| "launch" | Launch["launch.py<br/> 启动组件"]
    Sub -->| "collect-env" | CE["collect_env.py<br/> 环境信息收集"]
    Sub -->| "bench" | Bench["benchmark<br/> 性能基准测试"]

    style Main fill:#fce4ec,stroke:#c62828,stroke-width:2px
```

### 4.2 各命令详解

`vllm serve` 一启动 **API 服务器** `serve.py` 中的 `ServeSubcommand` 是最核心的命令，支持三种运行模式：

```
# file: vllm/entrypoints/cli/serve.py
# line: 43-122
```

```
class ServeSubcommand(CLISubcommand):
    @staticmethod
    def cmd(args: argparse.Namespace) -> None:
        # --grpc 模式: 启动 gRPC 服务器
        if getattr(args, "grpc", False):
            uvloop.run(serve_grpc(args))
            return

        # --headless 模式: 无 API 服务器, 纯引擎工作节点
        if args.headless:
            run_headless(args)
            return

        # 根据 api_server_count 选择运行模式
        if args.api_server_count < 1:
            run_headless(args)
        elif args.api_server_count > 1:
            run_multi_api_server(args)      # 多 API 服务器 + 负载均衡
        else:
            uvloop.run(run_server(args))    # 单 API 服务器 (默认)
```

运行模式矩阵:

| 模式           | 触发条件                                                         | 说明                 |
|--------------|--------------------------------------------------------------|--------------------|
| 单 API Server | 默认 ( <code>api_server_count=1</code> )                       | 当前进程同时运行引擎 + API   |
| 多 API Server | <code>api_server_count &gt; 1</code>                         | 多个 API 进程 + 共享引擎后端 |
| Headless     | <code>--headless</code> 或<br><code>api_server_count=0</code> | 仅运行引擎, 不启动 HTTP    |
| gRPC         | <code>--grpc</code>                                          | 使用 gRPC 替代 HTTP    |



多 API 服务器模式 (run\_multi\_api\_server) 支持三种负载均衡策略: - **Internal LB** (默认): api\_server\_count = data\_parallel\_size - **External LB**: --data-parallel-external-lb, 单 API 服务器 - **Hybrid LB**: --data-parallel-hybrid-lb, api\_server\_count = data\_parallel\_size\_local

vllm run-batch 一离线批处理 run\_batch.py 封装了批处理入口:

```
class RunBatchSubcommand(CLISubcommand):
    name = "run-batch"
    @staticmethod
    def cmd(args: argparse.Namespace) -> None:
        from vllm.entrypoints.openai.run_batch import main as run_batch_main
        if args.enable_metrics:
            start_http_server(port=args.port, addr=args.url)
        asyncio.run(run_batch_main(args))
```

vllm chat / vllm complete 一交互式客户端 openai.py 提供两个交互式子命令:

- vllm chat: 交互式聊天, 支持 --quick 单轮模式和 --system-prompt
- vllm complete: 文本补全交互模式

两者都使用 OpenAI Python SDK 连接本地 vLLM 服务器 (默认 http://localhost:8000/v1)。

vllm launch 一组件级启动 launch.py 支持启动独立组件:

```
class RenderSubcommand(LaunchSubcommandBase):
    name = "render"
    help = "Launch a GPU-less rendering server (preprocessing and postprocessing only)."
```

vllm launch render 启动一个无 GPU 的渲染服务器, 仅做预处理/后处理 (tokenization、chat template rendering 等), 适用于 CPU-only 推理场景。

vllm collect-env 一环境诊断 collect\_env.py 直接委托给 vllm.collect\_env.main() 收集环境信息用于问题排查。

---

## 五、Batch Serving 离线批处理

run\_batch.py 实现了类似 OpenAI Batch API 的离线批处理能力。

整体流程

graph TD

```
Input[" 输入文件 <br/>.jsonl (URL 或本地)"] --> Parse[" 逐行解析 <br/>BatchRequestInput"]
Parse --> Match{"URL 匹配"}
Match -->|"/v1/chat/completions"| ChatHandler["openai_serving_chat.create_chat_completion()"]
Match -->|"/v1/embeddings"| EmbedHandler["serving_embedding()"]
Match -->|"/score"| ScoreHandler["serving_scores.score()"]
Match -->|"/rerank"| RerankHandler["serving_scores.rerank()"]
Match -->|"/v1/audio/transcriptions"| TransHandler["transcription wrapper"]
Match -->|"/v1/audio/translations"| TranslHandler["translation wrapper"]

ChatHandler --> Gather["asyncio.gather()"]
EmbedHandler --> Gather
ScoreHandler --> Gather
RerankHandler --> Gather
TransHandler --> Gather
```

```

TranslHandler --> Gather

Gather --> Output[" 输出文件 <br/>.jsonl (URL 或本地)"]

style Input fill:#e8f5e9,stroke:#2e7d32
style Output fill:#ffcdd2,stroke:#c62828

请求/响应模型

# file: vllm/entrypoints/openai/run_batch.py
# line: 145-166

class BatchRequestInput(OpenAIBaseModel):
    """ 每行输入对象 """
    custom_id: str          # 用户自定义 ID, 用于匹配输入输出
    method: str             # HTTP 方法, 目前仅支持 POST
    url: str                # API 路径
    body: BatchRequestInputBody # 请求体 (自动按 URL 分发到对应模型)

class BatchRequestOutput(OpenAIBaseModel):
    """ 每行输出对象 """
    id: str                 # vLLM 生成的唯一 ID
    custom_id: str          # 对应输入的 custom_id
    response: BatchResponseData | None # 成功时的响应
    error: Any | None       # 失败时的错误信息

```

支持的端点及对应的 body 类型:

| URL                      | Body Type                              |
|--------------------------|----------------------------------------|
| /v1/chat/completions     | ChatCompletionRequest                  |
| /v1/embeddings           | EmbeddingRequest                       |
| /v1/{model}/score        | ScoreRequest                           |
| /v1/{model}/rerank       | RerankRequest                          |
| /v1/audio/transcriptions | BatchTranscriptionRequest (含 file_url) |
| /v1/audio/translations   | BatchTranslationRequest (含 file_url)   |

端点注册表机制

build\_endpoint\_registry() 构建了一个统一的端点注册表, 每个条目包含 url\_matcher、handler\_getter 和可选的 wrapper\_fn:

```

endpoint_registry = {
    "completions": {
        "url_matcher": lambda url: url == "/v1/chat/completions",
        "handler_getter": lambda: openai_serving_chat.create_chat_completion or None,
        "wrapper_fn": None,
    },
    "transcriptions": {
        "url_matcher": lambda url: url == "/v1/audio/transcriptions",
        "handler_getter": lambda: openai_serving_transcription.create_transcription or None,
        "wrapper_fn": make_transcription_wrapper(is_translation=False, ...), # 下载音频 + 转换
    },
    # ...
}

```

音频转录的 `make_transcription_wrapper` 会先从 URL 下载音频数据，构造 mock `UploadFile`，再调用常规处理函数。

## 文件 I/O

支持多种输入输出方式：- 本地文件：直接读写 `.jsonl` 文件 - **HTTP URL**：通过 `aiohttp` GET 读取 / PUT 写入（带重试，最多 5 次） - **Data URL**：base64 编码的内联数据

---

## 六、SageMaker 集成

SageMaker 模块使 vLLM 可以作为 Amazon SageMaker 推理端点运行。

### 路由设计

`api_router.py` 注册两个 SageMaker 标准端点：

```
# file: vllm/entrypoints/sagemaker/api_router.py
# line: 48-98

@router.post("/ping", response_class=Response)
@router.get("/ping", response_class=Response)
@sagemaker_standards.register_ping_handler
async def ping(raw_request: Request) -> Response:
    """Ping check — SageMaker 必需的健康检查端点"""
    return await health(raw_request)

@router.post("/invocations", ...)
@sagemaker_standards.register_invocation_handler
@sagemaker_standards.stateful_session_manager()
@sagemaker_standards.inject_adapter_id(adapter_path="model")
async def invocations(raw_request: Request):
    """统一推理端点 — 根据请求体自动路由到对应 handler"""
    body = await raw_request.json()
    # 遍历已注册的 invocation type validators
    for request_validator, endpoint in valid_endpoints:
        try:
            request = request_validator.validate_python(body)
        except pydantic.ValidationError:
            continue
        return await endpoint(request, raw_request)
```

### 关键特性

- **model\_hosting\_container\_standards**: 使用 AWS 开源的 SageMaker 标准库确保容器规范合规
- **/invocations** 统一路由：单一端点接收所有推理请求，通过 Pydantic 验证自动分派到正确的 handler
- **Stateful Session Manager**: 支持有状态会话管理
- **Adapter ID 注入**: 支持多模型 adapter 路由
- **Bootstrap**: `sagemaker_standards_bootstrap(app)` 在应用最后做标准化引导配置

---

## 七、MCP Tool Server 模型上下文协议工具服务

vLLM 通过 MCP (Model Context Protocol) 协议集成外部工具服务，使 LLM 可以调用外部工具。

架构设计

```
graph TB
    subgraph MCP ["MCP Tool Server"]
        TS["ToolServer (ABC)"]
        MT["MCPToolServer"]
        DT["DemoToolServer"]
    end

    subgraph External [" 外部工具 "]
        B["Browser Tool<br/>(ExaBackend + SimpleBrowserTool)"]
        P["Python Tool<br/>(Docker Code Interpreter)"]
        Remote["Remote MCP Server<br/>(SSE 协议)"]
    end

    TS --> MT
    TS --> DT
    MT --> Remote
    DT --> B
    DT --> P

    style MCP fill:#e8eaf6,stroke:#283593,stroke-width:2px
```

**ToolServer** 抽象基类

tool\_server.py 定义了三个抽象方法:

```
class ToolServer(ABC):
    @abstractmethod
    def has_tool(self, tool_name: str) -> bool: ...
    @abstractmethod
    def get_tool_description(self, tool_name: str, ...) -> ToolNamespaceConfig | None: ...
    @abstractmethod
    def new_session(self, tool_name: str, session_id: str, ...) -> AbstractAsyncContextManager[Any]: ..
```

**MCPToolServer** 一远程 MCP 工具连接器

MCPToolServer 通过 SSE 协议连接远程 MCP 工具服务器:

```
class MCPToolServer(ToolServer):
    async def add_tool_server(self, server_url: str):
        tool_urls = server_url.split(",")
        for url in tool_urls:
            url = f"http://{url}/sse"
            initialize_response, list_tools_response = await list_server_and_tools(url)
            list_tools_response = post_process_tools_description(list_tools_response)
            # 构建 Harmony 兼容的工具描述
            self.harmony_tool_descriptions[server_name] = ToolNamespaceConfig(...)
```

Schema 后处理 trim\_schema() 和 post\_process\_tools\_description() 将 MCP 生成的 JSON Schema 转换为 Harmony 库兼容格式, 并过滤不需要注入 prompt 的工具。

**DemoToolServer** 一内置演示工具

DemoToolServer 提供两种内置工具:

| 工具      | 类                  | 依赖                    | 说明                 |
|---------|--------------------|-----------------------|--------------------|
| Browser | HarmonyBrowserTool | EXA_API_KEY + gpt_oss | 基于 Exa API 的网页浏览工具 |
| Python  | HarmonyPythonTool  | gpt_oss               | Docker 沙盒代码解释器     |

具体实现在 tool.py:

```
# file: vllm/entrypoints/mcp/tool.py
# line: 59-81
```

```
class HarmonyBrowserTool(Tool):
    def __init__(self):
        self.enabled = True
        exa_api_key = os.getenv("EXA_API_KEY")
        if not exa_api_key:
            self.enabled = False
            return
        validate_gpt_oss_install()
        browser_backend = ExaBackend(source="web", api_key=exa_api_key)
        self.browser_tool = SimpleBrowserTool(backend=browser_backend)
```

## 八、Pooling 端点 嵌入池化接口

Pooling 模块为非生成类模型（embedding、classification、scoring、reranking）提供 API 接口。

目录结构

```
entrypoints/pooling/
  factories.py          # 工厂函数: IO Processor 初始化 & 路由注册
  base/
    protocol.py        # 基础抽象层
    serving.py         # PoolingRequest / PoolingResponse
    io_processor.py    # PoolingServingBase
  embed/
    api_router.py      # PoolingIOProcessor (ABC)
    protocol.py        # 嵌入向量
    serving.py         # /v1/embeddings, /v2/embed
    io_processor.py    # EmbeddingRequest/Response, CohereEmbedRequest
  classify/
    api_router.py      # ServingEmbedding
    protocol.py        # EmbedIOProcessor, TokenEmbedIOProcessor
    serving.py         # 分类任务
    io_processor.py    # 分类端点
  scoring/
    api_router.py      # ClassificationRequest/Response
    protocol.py        # ServingClassification
    serving.py         # ClassifyIOProcessor, TokenClassifyIOProcessor
    io_processor.py    # 评分/重排序
  pooling/
    api_router.py      # /score, /rerank
    protocol.py        # ScoreRequest/Response, RerankRequest/Response
    serving.py         # ServingScores
    io_processor.py    # ScoringIOProcessors
  pooling/
    api_router.py      # 通用 pooling
    protocol.py        # 通用 pooling 端点
    serving.py         # PoolingRequest/Response
    io_processor.py    # ServingPooling
```

io\_processor.py      # Plugin IO Processors

工厂函数—factories.py

**IO Processor** 初始化 `init_pooling_io_processors()` 根据 `pooling_task` 创建对应的 IO Processor:

```
def init_pooling_io_processors(supported_tasks, vllm_config, renderer, chat_template_config):
    processors = {}
    pooling_task = model_config.get_pooling_task(supported_tasks)

    if pooling_task == "classify":
        processors["classify"] = ClassifyIOProcessor
    if pooling_task == "embed":
        processors["embed"] = EmbedIOProcessor
    if has_io_processor(vllm_config, ...):
        processors["plugin"] = PluginWithIOProcessorPlugins
    if enable_scoring_api(supported_tasks, model_config):
        processors[score_type] = ScoringIOProcessors[score_type]
    # Jina 特殊处理...
    return {task: processor_cls(...) for task, processor_cls in processors.items()}
```

**API** 路由注册 `register_pooling_api_routers()` 按 capability 注册路由:

```
def register_pooling_api_routers(app, supported_tasks, model_config):
    pooling_task = model_config.get_pooling_task(supported_tasks)
    if pooling_task is not None:
        app.include_router(pooling_router)                    # 通用 pooling
    if "classify" in supported_tasks:
        app.include_router(classify_router)                  # 分类
    if "embed" in supported_tasks:
        app.include_router(embed_router)                    # 嵌入
    if enable_scoring_api(supported_tasks, model_config):
        app.include_router(score_router)                    # 评分/重排序
```

支持的任务类型映射

| pooling_task   | IO Processor             | API 端点          | 用途        |
|----------------|--------------------------|-----------------|-----------|
| embed          | EmbedIOProcessor         | /v1/embeddings  | 文本嵌入向量    |
| token_embed    | TokenEmbedIOProcessor    | /v1/embeddings  | 多向量检索     |
| classify       | ClassifyIOProcessor      | classify 端点     | 分类 logits |
| token_classify | TokenClassifyIOProcessor | classify 端点     | Token 级分类 |
| cross-encoder  | ScoringIOProcessors      | /score, /rerank | 相似度评分/重排序 |
| plugin         | Plugin IO Processor      | 自定义             | 可扩展插件     |

## 九、LLM 高级 API

llm.py 提供了面向 Python 开发者的高级编程接口，适用于离线推理、批量处理等场景。

类概览

```
# file: vllm/entrypoints/llm.py
# line: 106-211
```

```
class LLM:
    """
    An LLM for generating texts from given prompts and sampling parameters.
    This class includes a tokenizer, a language model (possibly distributed
    across multiple GPUs), and GPU memory space allocated for intermediate
    states (aka KV cache).

    Note: This class is intended to be used for offline inference.
    For online serving, use the AsyncLLMEngine class instead.
    """
```

核心方法体系

```
graph TB
    LLM["LLM 类"] --> Gen["生成类方法"]
    LLM --> Enc["编码/池化类方法"]
    LLM --> Mgmt["生命周期管理"]
    LLM --> Adv["高级功能"]

    Gen --> G1["generate()<br/> 文本生成"]
    Gen --> G2["chat()<br/> 对话生成"]
    Gen --> G3["beam_search()<br/> 束搜索"]
    Gen --> G4["enqueue() +<br/>wait_for_completion()<br/> 异步流水线"]

    Enc --> E1["encode()<br/> 通用池化"]
    Enc --> E2["embed()<br/> 嵌入向量"]
    Enc --> E3["classify()<br/> 分类"]
    Enc --> E4["score()<br/> 相似度评分"]

    Mgmt --> M1["sleep() / wake_up()<br/> 引擎休眠/唤醒"]
    Mgmt --> M2["start_profile() / stop_profile()<br/> 性能分析"]
    Mgmt --> M3["reset_prefix_cache()<br/> 前缀缓存重置"]

    Adv --> A1["collective_rpc()<br/> 跨 worker RPC"]
    Adv --> A2["apply_model()<br/> 直接操作模型"]
    Adv --> A3["weight_transfer_*()<br/> 权重传输 (RL 训练)"]

    style LLM fill:#fce4ec,stroke:#c62828,stroke-width:2px
    style Gen fill:#e8f5e9,stroke:#2e7d32
    style Enc fill:#e3f2fd,stroke:#1565c0
```

## 9.1 generate() — 文本生成

generate() 是最基本的文本生成接口:

```
def generate(
    self,
    prompts: PromptType | Sequence[PromptType],
    sampling_params: SamplingParams | Sequence[SamplingParams] | None = None,
    *,
    use_tqdm: bool | Callable[..., tqdm] = True,
    lora_request: ...,
    priority: list[int] | None = None,
    tokenization_kwargs: dict[str, Any] | None = None,
```

```

    mm_processor_kwargs: dict[str, Any] | None = None,
) -> list[RequestOutput]:

```

内部调用链: `generate()` → `_run_completion()` → `_add_completion_requests()` → `_preprocess_cmpl()` → `_add_request()` → `LLMEngine.add_request()` → `_run_engine()` (step 循环)

## 9.2 encode() 一通用池化

`encode()` 是 pooling 模型的通用入口:

```

def encode(
    self,
    prompts: PromptType | Sequence[PromptType] | DataPrompt,
    pooling_params: PoolingParams | Sequence[PoolingParams] | None = None,
    *,
    pooling_task: PoolingTask | None = None, # 必须指定!
    ...
) -> list[PoolingRequestOutput]:

```

必须显式指定 `pooling_task`, 支持的值包括: - "embed" 一嵌入向量 (推荐用 `embed()` 便捷方法) - "classify" 一分类 (推荐用 `classify()` 便捷方法) - "token\_classify" 一Token 级分类 - "token\_embed" 一多向量检索 - "plugin" 一自定义插件任务

## 9.3 chat() 一对话生成

`chat()` 提供对话式生成接口, 内部完成 conversation → prompt 的渲染:

```

def chat(
    self,
    messages: list[ChatCompletionMessageParam] | Sequence[list[...]],
    sampling_params: ...,
    chat_template: str | None = None,
    chat_template_content_format: ... = "auto",
    add_generation_prompt: bool = True,
    tools: list[dict[str, Any]] | None = None,
    ...
) -> list[RequestOutput]:

```

## 9.4 异步流水线模式

`enqueue()` + `wait_for_completion()` 组合允许异步提交 + 批量等待的模式:

```

# 先批量入队
request_ids = llm.enqueue(prompts, sampling_params)
# ... 做其他事情 ...
# 统一等待结果
outputs = llm.wait_for_completion()

```

## 9.5 束搜索 beam\_search()

`beam_search()` 实现了完整的束搜索算法:

1. 初始化: 每个 prompt 创建 `BeamSearchInstance` (包含 `beam_width` 个候选序列)
2. 迭代: 每步对所有 beam 执行一次单 token 生成
3. 剪枝: 保留 top-`beam_width` 个累积 logprob 最高的序列
4. 终止: 遇到 EOS 或达到 `max_tokens` 时移入 `completed` 列表



## 9.6 权重传输 (RL 训练支持)

一组用于在线 RL 训练的方法:

```
init_weight_transfer_engine(request) # 初始化权重传输
start_weight_update(is_checkpoint_format) # 开始新的权重更新周期
update_weights(request) # 更新权重
finish_weight_update() # 结束权重更新
```

---

## 十、SSL/TLS 支持

ssl.py 提供了 SSL 证书热更新 能力。

### SSLCertRefresher

```
# file: vllm/entrypoints/ssl.py
# line: 15-76
```

```
class SSLCertRefresher:
    """ 监控 SSL 证书文件并在变更时自动重新加载 """

    def __init__(self, ssl_context, key_path=None, cert_path=None, ca_path=None):
        self.ssl = ssl_context
        # 监控证书链文件 (key + cert)
        if self.key_path and self.cert_path:
            self.watch_ssl_cert_task = asyncio.create_task(
                self._watch_files([key_path, cert], update_ssl_cert_chain))
        # 监控 CA 证书文件
        if self.ca_path:
            self.watch_ssl_ca_task = asyncio.create_task(
                self._watch_files([ca_path], update_ssl_ca))
```

工作机制:1. 使用 watchfiles 库的 awatch() 异步监控证书文件变化 2. 检测到变更时调用 SSLContext.load\_cert\_chain() 或 load\_verify\_locations() 热加载 3. 无需重启服务器即可完成证书轮换 4. 通过 stop() 方法取消监控任务

---

## 十一、请求日志与访问过滤

access\_log\_filter.py 提供 uvicorn 访问日志过滤功能。

### UvicornAccessLogFilter

```
# file: vllm/logging_utils/access_log_filter.py
# line: 15-68
```

```
class UvicornAccessLogFilter(logging.Filter):
    """
    排除指定路径的 uvicorn 访问日志。

    uvicorn 日志格式: '%s - "%s %s HTTP/%s" %d'
    例: 127.0.0.1:12345 - "GET /health HTTP/1.1" 200
    """

    def filter(self, record: logging.LogRecord) -> bool:
```

```

# 仅处理 uvicorn.access logger
if record.name != "uvicorn.access":
    return True
# 从 log_args 元组提取 path (第 3 个元素)
path = urlparse(log_args[2]).path
if path in self.excluded_paths:
    return False # 过滤掉
return True

```

配置生成

`create_uvicorn_log_config()` 生成完整的 uvicorn logging configuration dict, 将 `UvicornAccessLogFilter` 注入 access handler 的 filters 列表中。

典型用途: 排除 `/health`、`/metrics` 等高频健康检查端点的日志, 减少生产环境日志噪音。

## 十二、API 请求路由与处理流程总览

以下是 vLLM 从接收到返回一个完整请求的全链路:

```

sequenceDiagram
    participant C as Client
    participant MW as Middleware Stack
    participant R as API Router
    participant S as Serving Handler
    participant Render as Renderer
    participant E as AsyncLLM Engine

    C->>MW: HTTP Request
    MW->>MW: CORS Check
    MW->>MW: Auth (API Key)
    MW->>MW: X-Request-ID
    MW->>MW: Scaling Check
    MW->>R: Validated Request

    alt Chat Completion
        R->>S: create_chat_completion()
        S->>Render: render_chat()
        Render-->>S: EngineInput (prompt_token_ids)
        S->>E: add_request()
    else Embedding
        R->>S: create_embedding()
        S->>Render: render_embed()
        Render-->>S: EngineInput
        S->>E: add_request()
    else Anthropic Messages
        R->>S: create_messages()
        S->>S: _convert_anthropic_to_openai_request()
        S->>Render: render_chat()
        Render-->>S: EngineInput
        S->>E: add_request()
    end

    end

    alt Stream Mode

```

```

    E-->>S: AsyncGenerator[token deltas]
    S-->>R: SSE formatted chunks
    R-->>MW: StreamingResponse
    MW-->>C: text/event-stream
else Non-Stream Mode
    E-->>S: RequestOutput (final)
    S-->>R: ChatCompletionResponse
    R-->>MW: JSONResponse (+ Orca header)
    MW-->>C: JSON Response
end

```

中间件处理顺序

```

graph LR
    Req["Incoming Request"] --> CORS["CORSMiddleware"]
    CORS --> Auth["AuthenticationMiddleware<br/>(API Key)"]
    Auth --> RID["XRequestIdMiddleware"]
    RID --> Scale["ScalingMiddleware"]
    Scale --> WS["WebSocketMetricsMiddleware<br/>(realtime only)"]
    WS --> Debug["log_response<br/>(debug mode)"]
    Debug --> Custom["Custom Middlewares"]
    Custom --> Route["Route Handler"]

    style Req fill:#e8f5e9,stroke:#2e7d32
    style Route fill:#ffcdd2,stroke:#c62828

```

**EngineClient** 与 **Serving** 层的关系

```

graph TB
    subgraph State["app.state (FastAPI State)"]
        EC["engine_client:<br/>EngineClient"]
        OSM["openai_serving_models:<br/>OpenAIServingModels"]
        OSR["openai_serving_render:<br/>OpenAIServingRender"]
        OSC["openai_serving_chat:<br/>OpenAIServingChat"]
        OSCB["openai_serving_chat_batch:<br/>OpenAIServingChatBatch"]
        OSCR["openai_serving_completion:<br/>OpenAIServingCompletion"]
        ASM["anthropic_serving_messages:<br/>AnthropicServingMessages"]
        OSResp["openai_serving_responses:<br/>OpenAIServingResponses"]
        SE["serving_embedding:<br/>ServingEmbedding"]
        SC["serving_classification:<br/>ServingClassification"]
        SS["serving_scores:<br/>ServingScores"]
        ST["serving_tokens:<br/>ServingTokens"]
    end

    EC --> OSM
    EC --> OSR
    OSR --> OSC
    OSR --> OSCR
    OSR --> ASM
    OSR --> OSResp
    OSM --> SE
    OSM --> SC
    OSM --> SS

```

```
style State fill:#fff3e0,stroke:#ef6c00,stroke-width:2px
style EC fill:#fce4ec,stroke:#c62828
```

文档信息: 基于 vLLM 源码 /workspace/vllm 分析, 覆盖 entrypoints/ 目录下全部主要服务接口模块。各代码引用均标注了源文件路径和行号, 便于交叉验证和深入阅读。

## vLLM CUDA/C++ 内核层深度分析

定位: 本文档深入分析 vLLM 的 CUDA/C++ 内核层架构, 涵盖构建系统、注意力机制、缓存管理、激活函数、量化、MoE、通信、采样以及平台特定优化等核心内核实现。

```
graph TB
    subgraph 构建系统 [" 构建系统"]
        A1[Python 绑定 <br/>vllm/kernels/]
        A2[torch.utils.ffi<br/>Extension 编译]
        A3[第三方库集成 <br/>FlashInfer/Marlin/CUTLASS]
        A4[Triton Kernel<br/>JIT 编译]
    end

    subgraph 内核类别 [" 内核类别"]
        B1[注意力内核 <br/>Attention Kernels]
        B2[缓存内核 <br/>Cache Kernels]
        B3[激活函数内核 <br/>Activation Kernels]
        B4[量化内核 <br/>Quantization Kernels]
        B5[MoE 内核 <br/>MoE Kernels]
        B6[通信内核 <br/>Communication Kernels]
        B7[采样内核 <br/>Sampling Kernels]
    end

    subgraph 平台特定 [" 平台特定"]
        C1[NVIDIA CUDA<br/>SM 7.5 - 10.0]
        C2[AMD ROCm<br/>MI300X/MI325X]
        C3[CPU x86/ARM<br/>AMX/NEON/VSX]
        C4[Intel XPU<br/>oneAPI]
    end

    构建系统 --> 内核类别
    内核类别 --> 平台特定

    style 构建系统 fill:#d1c4e9,stroke:#673ab7,color:#000
    style 内核类别 fill:#c5cae9,stroke:#512da8,color:#000
    style 平台特定 fill:#b39ddb,stroke:#4527a0,color:#000
```

### 目录

- 一、构建系统
- 二、注意力内核
- 三、缓存内核
- 四、激活函数内核
- 五、量化内核
- 六、MoE 内核

- 七、通信内核
- 八、采样内核
- 九、平台特定内核
- 十、内核分类与调用链总览

## 一、构建系统

### 1.1 整体架构概述

vLLM 采用混合内核架构, 与传统深度学习框架(如 PyTorch 的 `csrc/` 目录)不同, vLLM 没有单一的 `CMakeLists.txt` 构建文件。其内核层由以下几部分组成:

| 组件             | 位置                               | 说明                                                       |
|----------------|----------------------------------|----------------------------------------------------------|
| Python 绑定      | <code>vllm/kernels/</code>       | 通过 <code>torch.utils.cpp_extension</code> 加载 C++/CUDA 扩展 |
| Triton Kernels | 分散在各模块                           | JIT 编译的 Triton GPU kernel                                |
| 第三方库           | 集成在 <code>model_executor/</code> | FlashInfer, Marlin, CUTLASS 等                            |
| 自定义 ops        | <code>vllm/_custom_ops.py</code> | 统一的 custom op 注册入口                                       |

### 1.2 Python 绑定层 (`vllm/kernels/`)

核心文件结构

```
vllm/kernels/
__init__.py      # 导出所有内核模块
vllm_c.py        # CUDA C++ 扩展绑定 (RMSNorm 等)
aiter_ops.py     # AMD ROCm AITER 操作绑定
oink_ops.py      # OINK 优化操作绑定
xpu_ops.py       # Intel XPU 操作绑定
helion/          # Helion 硬件特定配置
  configs/       # 硬件配置 JSON (H100/H200)
  ops/           # SiLU*FP8 融合操作
  triton/        # Triton kernel 实现
    qkv_padded_fp8_quant.py # QKV FP8 量化
```

`vllm_c.py` - 核心 C++ 扩展绑定 `vllm_c.py` 是 vLLM 最核心的 Python-C++ 桥接文件, 注册了底层 RMSNorm 和 FusedAddRMSNorm 操作:

```
# 文件: /workspace/vllm/kernels/vllm_c.py
# 行号: 21-33

@ir.ops.rms_norm.register_impl(
    "vllm_c", supports_args=rms_no_var_size, supported=CUDA_ALIKE
)
def rms_norm(
    x: Tensor, weight: Tensor | None, epsilon: float,
    variance_size: int | None = None
) -> Tensor:
    if weight is None:
        weight = torch.ones(x.shape[-1], device=x.device, dtype=x.dtype)
    assert variance_size is None
    output = torch.empty(x.shape, device=x.device, dtype=x.dtype)
```

```

    torch.ops._C.rms_norm(output, x, weight, epsilon)
    return output

```

关键点: - 使用 `ir.ops.rms_norm.register_impl` 注册到 IR op 系统 - 底层调用 `torch.ops._C.rms_norm`, 这是编译好的 C++ 扩展 - 支持 CUDA\_ALIKE 平台 (CUDA、ROCm 等)

融合 **Add-RMSNorm** 实现

# 文件: `/workspace/vllm/kernels/vllm_c.py`  
 # 行号: 44-62

```

@ir.ops.fused_add_rms_norm.register_impl(
    "vllm_c",
    supports_args=rms_add_no_var_size,
    supported=CUDA_ALIKE,
    inplace=True,
)
def fused_add_rms_norm(
    x: Tensor, x_residual: Tensor, weight: Tensor | None,
    epsilon: float, variance_size: int | None = None,
) -> tuple[Tensor, Tensor]:
    if weight is None:
        weight = torch.ones(x.shape[-1], device=x.device, dtype=x.dtype)
    assert variance_size is None
    torch.ops._C.fused_add_rms_norm(x, x_residual, weight, epsilon)
    return x, x_residual

```

### 1.3 torch\_utils 扩展编译方式

`torch_utils.py` 提供了核心的工具函数和 custom op 注册机制:

# 文件: `/workspace/vllm/utils/torch_utils.py`  
 # 行号: 928-967

# 创建 *vLLM* 自定义库

```
vllm_lib = Library("vllm", "FRAGMENT")
```

```

def direct_register_custom_op(
    op_name: str,
    op_func: Callable,
    mutates_args: list[str] | None = None,
    fake_impl: Callable | None = None,
    target_lib: Library | None = None,
    dispatch_key: str | None = None,
    tags: tuple[torch.Tag, ...] = (),
):
    """
    直接注册自定义 op 并分发到 CUDA 后端。
    绕过 torch.library.custom_op 的复杂调度逻辑。
    """
    if mutates_args is None:
        mutates_args = []

    if dispatch_key is None:
        from vllm.platforms import current_platform
        dispatch_key = current_platform.dispatch_key

```

```

schema_str = infer_schema(op_func, mutates_args=mutates_args)
my_lib = target_lib or vllm_lib
my_lib.define(op_name + schema_str, tags=tags)
my_lib.impl(op_name, op_func, dispatch_key=dispatch_key)
if fake_impl is not None:
    my_lib._register_fake(op_name, fake_impl)

```

设计亮点: - 性能优化: 直接注册绕过 PyTorch 复杂的调度逻辑 - 平台感知: 自动使用当前平台的 `dispatch_key` - **FRAGMENT** 库: 使用轻量级 FRAGMENT 类型避免完整 Library 开销

#### 1.4 平台导入机制

各平台通过 `import_kernels()` 方法动态加载对应的内核:

```

# CUDA 平台 (platforms/cuda.py)
import vllm._C          # noqa: 触发 C++ 扩展加载
import vllm._C_stable_libtorch

# ROCm 平台 (platforms/rocm.py)
import vllm._C          # noqa
import vllm._rocm_C     # noqa: ROCm 特定扩展

```

---

## 二、注意力内核

### 2.1 注意力后端架构

vLLM 实现了多后端可插拔架构, 支持多种注意力计算实现:

```

graph LR
    subgraph AttentionBackendEnum [" AttentionBackendEnum"]
        A1[FLASH_ATTN]
        A2[FLASHINFER]
        A3[TRITON_ATTN]
        A4[FLEX_ATTENTION]
        A5[TURBOQUANT]
    end

    subgraph MLA_Backends [" MLA 专用后端"]
        M1[FLASHINFER_MLA]
        M2[FLASHMLA]
        M3[CUTLASS_MLA]
        M4[TRITON_MLA]
        M5[FLASH_ATTN_MLA]
    end

    subgraph Platform_Specific [" 平台特定"]
        P1[ROCM_ATTN]
        P2[CPU_ATTN]
        P3[ROCM_AITER_MLA]
    end

    AttentionBackendEnum --> MLA_Backends
    MLA_Backends --> Platform_Specific

```

后端注册表 (registry.py)

# 文件: /workspace/vllm/v1/attention/backends/registry.py  
# 行号: 34-87

```
class AttentionBackendEnum(Enum, metaclass=_AttentionBackendEnumMeta):
    """ 所有支持的注意力后端枚举 """

    FLASH_ATTN = "vllm.v1.attention.backends.flash_attn.FlashAttentionBackend"
    FLASH_ATTN_DIFFKV = (
        "vllm.v1.attention.backends.flash_attn_diffkv.FlashAttentionDiffKVBackend"
    )
    TRITON_ATTN = "vllm.v1.attention.backends.triton_attn.TritonAttentionBackend"
    ROCM_ATTN = "vllm.v1.attention.backends.rocm_attn.RocmAttentionBackend"
    TORCH_SDPA = "" # 仅用于 ViT
    FLASHINFER = "vllm.v1.attention.backends.flashinfer.FlashInferBackend"
    # ... 更多后端
    CPU_ATTN = "vllm.v1.attention.backends.cpu_attn.CPUAttentionBackend"
    TURBOQUANT = "vllm.v1.attention.backends.turboquant_attn.TurboQuantAttentionBackend"
```

## 2.2 FlashInfer 后端详解

flashinfer.py 是 vLLM 默认且最高性能的注意力后端:

# 文件: /workspace/vllm/v1/attention/backends/flashinfer.py  
# 行号: 1-50

```
from flashinfer import (
    BatchDecodeWithPagedKVCacheWrapper,
    BatchPrefillWithPagedKVCacheWrapper,
    BatchPrefillWithRaggedKVCacheWrapper,
    MultiLevelCascadeAttentionWrapper,
)
from flashinfer.decode import fast_decode_plan, trtllm_batch_decode_with_kv_cache
from flashinfer.prefill import trtllm_batch_context_with_kv_cache
from flashinfer.utils import FP4Tensor
```

核心特性: - **Paged KV Cache**: 支持分页式 KV 缓存的高效访问 - **FP8/FP4** 量化支持: 原生支持低精度 KV cache - **TRT-LLM** 集成: 可直接调用 TensorRT-LLM 优化的 kernel - **Multi-Level Cascade**: 支持 DeepSeek V3/V4 的多级级联注意力

### FP8 KV Cache 反量化 Kernel

# 文件: /workspace/vllm/v1/attention/backends/flashinfer.py  
# 行号: 98-149

```
@triton.jit
def _trtllm_prefill_attn_kvfp8_dequant(
    kv_cache_ptr,
    block_tables_prefill_ptr,
    block_table_stride,
    mock_kv_cache_ptr,
    k_scale_ptr,
    v_scale_ptr,
    src_stride_page,
    src_stride_kv,
```



```

src_stride_head,
DST_K_CACHE_STRIDE: tl.constexpr,
DST_KV_CACHE_STRIDE: tl.constexpr,
HEAD_STRIDE: tl.constexpr,
NUM_KV_HEADS: tl.constexpr,
):
    """ 将 FP8 量化的 KV cache 反量化为 BF16/FP16 """
    batch_idx = tl.program_id(0).to(tl.int64)
    mock_block_table_idx = tl.program_id(1).to(tl.int64)
    orig_page_num = tl.load(
        block_tables_prefill_ptr + batch_idx * block_table_stride + mock_block_table_idx
    ).to(tl.int64)
    if orig_page_num <= 0:
        return

    k_scale_val = tl.load(k_scale_ptr)
    v_scale_val = tl.load(v_scale_ptr)

    for h in range(NUM_KV_HEADS):
        h_off = tl.cast(h, tl.int64)
        # 读取 FP8 K 并反量化
        src_k = orig_page_num * src_stride_page + h_off * src_stride_head + head_offsets
        fp8_k = tl.load(kv_cache_ptr + src_k)
        dequant_k = (fp8_k.to(tl.float32) * k_scale_val).to(dequant_dtype)

        # 读取 FP8 V 并反量化
        src_v = (orig_page_num * src_stride_page + src_stride_kv +
                  h_off * src_stride_head + head_offsets)
        fp8_v = tl.load(kv_cache_ptr + src_v)
        dequant_v = (fp8_v.to(tl.float32) * v_scale_val).to(dequant_dtype)

```

## 2.3 MLA (Multi-Head Latent Attention) 内核

MLA 是 DeepSeek V3/V4 引入的新型注意力机制，vLLM 提供了完整的 **MLA** 内核生态：

**MLA** 后端列表 (mla/ 目录)

| 后端文件              | 适用场景                       | 性能特点           |
|-------------------|----------------------------|----------------|
| flashinfer_mla.py | NVIDIA GPU (通用)            | 推荐首选，平衡性能与兼容性  |
| flashmla.py       | NVIDIA Blackwell (SM 10.x) | 最高吞吐量，利用 TMA   |
| cutlass_mla.py    | NVIDIA Hopper+             | CUTLASS 实现，高精度 |
| triton_mla.py     | 通用 CUDA                    | 纯 Triton，易调试   |
| flashattn_mla.py  | 基于 FlashAttn               | 兼容性好           |
| rocm_aiter_mla.py | AMD MI300X                 | ROCm AITER 加速  |

**Sparse MLA** 变体 针对长序列场景，vLLM 实现了 **Sparse MLA**（稀疏注意力）：

- flashinfer\_mla\_sparse.py: FlashInfer 稀疏实现
- flashmla\_sparse.py: FlashMLA 稀疏实现
- rocm\_aiter\_mla\_sparse.py: ROCm 稀疏实现
- xpu\_mla\_sparse.py: Intel XPU 稀疏实现

## MLA Prefill 子系统

```
graph TB
    subgraph MLAPrefill[" MLA Prefill 子系统"]
        R[selector.py<br/> 路由选择]
        B1[base.py<br/> 基类定义]
        FI[flashinfer.py<br/>FlashInfer 实现]
        TRT[trtllm_ragged.py<br/>TRT-LLM Ragged]
    end

    R --> B1
    B1 --> FI
    B1 --> TRT
```

### 2.4 注意力后端选择策略

cuda.py 中的智能选择逻辑:

```
# 文件: /workspace/vllm/platforms/cuda.py
# 行号: 79-143
```

```
@lru_cache(maxsize=8)
def _get_backend_priorities(
    use_mla: bool,
    device_capability: DeviceCapability,
    num_heads: int | None = None,
    kv_cache_dtype: CachedType | None = None,
) -> list[AttentionBackendEnum]:
    """ 根据硬件和配置获取后端优先级列表 """
    if use_mla:
        if device_capability.major == 10:
            # Blackwell GPU: 优先使用稀疏 MLA
            if kv_cache_dtype is not None and is_quantized_kv_cache(kv_cache_dtype):
                sparse_backends = [
                    AttentionBackendEnum.FLASHINFER_MLA_SPARSE,
                    AttentionBackendEnum.FLASHMLA_SPARSE,
                ]
            else:
                if num_heads is not None and num_heads <= 16:
                    sparse_backends = [
                        AttentionBackendEnum.FLASHINFER_MLA_SPARSE,
                        AttentionBackendEnum.FLASHMLA_SPARSE,
                    ]
                else:
                    sparse_backends = [
                        AttentionBackendEnum.FLASHMLA_SPARSE,
                        AttentionBackendEnum.FLASHINFER_MLA_SPARSE,
                    ]
        else:
            sparse_backends = [
                AttentionBackendEnum.FLASHINFER_MLA,
                AttentionBackendEnum.CUTLASS_MLA,
                AttentionBackendEnum.FLASH_ATTN_MLA,
                AttentionBackendEnum.FLASHMLA,
                AttentionBackendEnum.TRITON_MLA,
            ]
```

```

        *sparse_backends,
    ]
else:
    if device_capability.major == 10:
        return [
            AttentionBackendEnum.FLASHINFER,
            AttentionBackendEnum.FLASH_ATTN,
            AttentionBackendEnum.TRITON_ATTN,
            AttentionBackendEnum.FLEX_ATTENTION,
            AttentionBackendEnum.TURBOQUANT,
        ]
    else:
        return [
            AttentionBackendEnum.FLASH_ATTN,
            AttentionBackendEnum.FLASHINFER,
            AttentionBackendEnum.TRITON_ATTN,
            AttentionBackendEnum.FLEX_ATTENTION,
            AttentionBackendEnum.TURBOQUANT,
        ]

```

选择策略总结: 1. **Blackwell (SM 10.x)**: 优先 FlashInfer → FlashAttn → Triton → Flex → TurboQuant  
 2. **Hopper/Ampere (SM 9.x/8.x)**: 优先 FlashAttn → FlashInfer → Triton 3. **MLA** 场景: 优先 FlashInfer\_MLA → CUTLASS\_MLA → FlashMLA 4. 量化 **KV Cache**: 自动切换到稀疏变体

### 三、缓存内核

#### 3.1 reshape\_and\_cache\_kernel - KV 缓存写入核心

triton\_reshape\_and\_cache\_flash.py 实现了高性能的 KV 缓存写入 kernel:

# 文件: /workspace/vllm/v1/attention/ops/triton\_reshape\_and\_cache\_flash.py  
 # 行号: 17-100

```

@triton.jit
def reshape_and_cache_kernel_flash(
    key_ptr,           # [num_tokens, num_heads, head_size]
    value_ptr,         # [num_tokens, num_heads, head_size]
    key_cache_ptr,     # [num_blocks, block_size, num_heads, head_size]
    value_cache_ptr,   # [num_blocks, block_size, num_heads, head_size]
    slot_mapping_ptr,  # [num_tokens] - token 到 cache slot 的映射
    k_scale,           # float32 - K 的量化 scale
    v_scale,           # float32 - V 的量化 scale
    # strides...
    num_heads: tl.constexpr,
    head_size: tl.constexpr,
    block_size: tl.constexpr,
    x: tl.constexpr,    # FP8 打包因子 (通常为 8)
    USE_HEAD_MAJOR_LAYOUT: tl.constexpr,
    FP8_KV_CACHE: tl.constexpr, # 是否启用 FP8 缓存
    TILE_SIZE: tl.constexpr,
):
    """ 将新的 K/V 写入 paged KV cache """
    token_idx = tl.program_id(axis=0)
    slot_idx = tl.load(slot_mapping_ptr + token_idx).to(tl.int64)

```

```

if slot_idx < 0:
    return # 忽略 padding token

block_idx = slot_idx // block_size
block_offset = slot_idx % block_size

tile_i = tl.program_id(axis=1)
tile_offs = tl.arange(0, TILE_SIZE)
tile_pos = tile_i * TILE_SIZE + tile_offs

if USE_HEAD_MAJOR_LAYOUT:
    cur_head = tile_pos // head_size
    cur_dim = tile_pos % head_size
    # Head-Major 布局寻址
    tgt_idx_v = (
        block_idx * block_stride
        + cur_head * head_stride
        + cur_dim * dim_stride_v
        + block_offset
    )
    tgt_idx_k = (
        block_idx * block_stride
        + cur_head * head_stride
        + (cur_dim // x) * dim_stride_k
        + block_offset * x
        + (cur_dim % x)
    )

    # 加载并写入 Key
    key_load = tl.load(
        key_ptr + src_key_idx + tile_pos,
        mask=tile_pos < (num_heads * head_size)
    )
    if FP8_KV_CACHE:
        key_tile = (key_load if key_load.dtype.is_fp8()
                     else key_load / tl.load(k_scale))
    else:
        key_tile = key_load

    tl.store(key_cache_ptr + tgt_idx_k, key_tile,
             mask=tile_pos < (num_heads * head_size))

```

关键特性: - 双布局支持: Head-Major 和 Token-Major 两种 KV cache 布局 - **FP8** 量化: 支持在线量化写入 FP8 KV cache - **Tiled** 内存访问: 使用 TILE\_SIZE 优化内存合并访问 - **Padding** 跳过: 通过 slot\_mapping < 0 跳过无效 token

### 3.2 NVFP4 KV Cache 支持

torch\_utils.py 中实现了 NVFP4 (NVIDIA 4-bit Floating Point) 格式的 KV cache:

```

# 文件: /workspace/vllm/utils/torch_utils.py
# 行号: 415-469

def nvfp4_kv_cache_full_dim(head_size: int) -> int:
    """NVFP4 KV cache 打包维度: fp4 数据 + fp8 block scales"""

```

```

    return head_size // 2 + head_size // 16

def _nvfp4_split_data_scale(
    kv_side: torch.Tensor,
) -> tuple[torch.Tensor, torch.Tensor]:
    """ 拆分 NVFP4 buffer 为数据和 scale 视图 """
    num_pages = kv_side.shape[0]
    dim_1, dim_2 = kv_side.shape[1], kv_side.shape[2]
    full_dim = kv_side.shape[3]
    data_dim = full_dim * 8 // 9      # 8/9 是数据比例
    scale_dim = full_dim - data_dim   # 1/9 是 scale 比例

    data_per_kv = dim_1 * dim_2 * data_dim
    page_bytes = kv_side.stride(0)

    # 保持原始布局 (NHD vs HND)
    s1 = kv_side.stride(1) * data_dim // full_dim
    s2 = kv_side.stride(2) * data_dim // full_dim
    data_shape = (num_pages, dim_1, dim_2, data_dim)
    data_strides = (page_bytes, s1, s2, 1)

    base = kv_side.storage_offset()
    data = torch.as_strided(kv_side, data_shape, data_strides,
                            storage_offset=base)
    scale = torch.as_strided(
        kv_side, scale_shape, scale_strides,
        storage_offset=base + data_per_kv
    ).view(torch.float8_e4m3fn)

    return data, scale

```

**NVFP4** 格式说明: - 数据压缩比: 相比 FP16 节省 **75%** 显存 (4-bit vs 16-bit) - **Block Scale**: 每 16 个元素共享一个 FP8 scale - 零拷贝视图: 通过 `torch.as_strided` 实现无开销的数据/Scale 拆分

### 3.3 KV Cache 量化格式支持

vLLM 支持多种 KV Cache 量化格式:

| 格式                | 数据类型     | 压缩比  | 适用场景              |
|-------------------|----------|------|-------------------|
| <b>FP16/BF16</b>  | 半精度      | 1x   | 默认, 最佳精度          |
| <b>FP8 (E4M3)</b> | 8-bit 浮点 | 2x   | Hopper+ GPU 推荐    |
| <b>NVFP4</b>      | 4-bit 浮点 | 4x   | Blackwell GPU 长序列 |
| <b>INT8</b>       | 8-bit 整数 | 2x   | 特定量化方案            |
| <b>TurboQuant</b> | 混合精度     | 4-8x | TurboQuant 专用     |

## 四、激活函数内核

### 4.1 RMSNorm 和 FusedAddRMSNorm

通过 `vllm_c.py` 绑定的底层 C++ 实现:

```

# 文件: /workspace/vllm/kernels/vllm_c.py
# 行号: 14-33

```

```
rms_no_var_size = (
    lambda x, weight, epsilon, variance_size=None:
        variance_size is None
        and (weight is None or weight.dtype == x.dtype)
)
"""vLLM kernel 要求: 无 variance_size 参数且 input/weight dtype 匹配"""
```

RMSNorm 公式:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2 + \epsilon}} \cdot \gamma$$

**FusedAddRMSNorm** 优势: - 融合操作: 在一个 kernel 中完成 residual add + normalization - 减少内存带宽: 避免中间结果的读写 - **Inplace** 执行: 直接修改输入 tensor, 节省显存分配

4.2 RoPE (Rotary Position Embedding) 内核

vLLM 实现了丰富的 RoPE 变体以支持不同模型:

RoPE 变体列表 (rotary\_embedding/)

| 文件名                         | 支持的模型       | 特点            |
|-----------------------------|-------------|---------------|
| linear_scaling_rope.py      | LLaMA 标准    | 线性位置缩放        |
| ntk_scaling_rope.py         | NTK-aware   | 外推能力增强        |
| dynamic_ntk_scaling_rope.py | Dynamic NTK | 动态 NTK 缩放     |
| yarn_scaling_rope.py        | YaRN        | 长序列外推         |
| deepseek_scaling_rope.py    | DeepSeek    | DeepSeek 特定优化 |
| llama3_rope.py              | LLaMA 3     | LLaMA 3 标准    |
| mrope.py                    | Qwen2-VL    | 多模态 RoPE      |
| xdrope.py                   | Qwen2.5     | XRoPE 扩展      |

融合 RoPE + FP8 量化 Kernel

```
# 文件: /workspace/vllm/v1/attention/ops/deepseek_v4_ops/fused_inv_rope_fp8_quant.py
```

```
class FusedInvRoPEFp8Quant:
    """ 融合逆 RoPE + FP8 量化操作 """
    def __init__(self, head_size: int, num_heads: int, ...):
        self.head_size = head_size
        self.num_heads = num_heads

    def forward(self, q, k, positions, ...):
        # 1. 应用逆 RoPE
        q = apply_rotary_emb(q, cos, sin)
        k = apply_rotary_emb(k, cos, sin)
        # 2. 在同一 kernel 中量化为 FP8
        q_fp8 = quantize_to_fp8(q, scale_q)
        k_fp8 = quantize_to_fp8(k, scale_k)
        return q_fp8, k_fp8, scale_q, scale_k
```

4.3 SiLU \* FP8 融合操作

helion/ops/silu\_mul\_fp8.py 实现了针对 NVIDIA H100/H200 的 SiLU + MatMul 融合:

```
# 文件: /workspace/vllm/kernels/helion/configs/silu_mul_fp8/nvidia_h100.json
{
    "kernel_name": "silu_mul_fp8",
    "device": "NVIDIA_H100",
    "optimizations": ["tensor_core", "pipeline"],
    "block_size": [128, 128, 32],
    "stages": 4,
    "wave_size": 128,
    "num_warps": 8
}
```

**Helion** 配置系统: - 硬件特定调优: 每个 GPU 型号有独立的最优配置 - **Kernel auto-tuning**: 运行时选择最优参数组合 - **Wavefront** 调度: 充分利用 GPU 波前并行

## 五、量化内核

### 5.1 量化内核架构总览

```
graph TB
    subgraph QuantKernels[" 量化内核体系"]
        direction TB
        Q1[AWQ<br/>Activation-aware Weight Quantization]
        Q2[GPTQ<br/>GPT Quantization]
        Q3[GGUF<br/>llama.cpp 格式]
        Q4[Marlin<br/>4-bit GEMM Engine]
        Q5[Machete<br/>新一代 CUTLASS GEMM]
        Q6[CUTLASS<br/>W8A8/FP4/NVFP4]
    end

    subgraph LinearKernels[" Linear 层抽象"]
        L1[MixedPrecision<br/>混合精度线性层]
        L2[ScaledMM<br/>缩放矩阵乘法]
        L3[NVFP4<br/>4-bit 浮点线性层]
        L4[MXFP8<br/>8-bit 浮点 MoE 线性层]
    end

    QuantKernels --> LinearKernels
```

### 5.2 AWQ (Activation-aware Weight Quantization)

awq\_marlin.py 实现 AWQ 量化方案的 Marlin 后端:

```
# 文件: /workspace/vllm/model_executor/layers/quantization/awq_marlin.py
# 行号: 67-100
```

```
_REVERSE_AWQ_PACK_ORDER = [0, 4, 1, 5, 2, 6, 3, 7]
"""AWQ 使用非标准打包顺序, 需要反转"""
```

```
def _convert_awq_to_standard_format(
    layer: torch.nn.Module,
    w_q_name: str,
    w_zp_name: str,
    size_bits: int,
) -> None:
```

```

""" 将 AWQ 权重和零点转换为标准 GPTQ 格式 """
pack_factor = 32 // size_bits
mask = (1 << size_bits) - 1
device = getattr(layer, w_q_name).device

# 反转 AWQ 打包顺序
reverse_order = torch.tensor(
    _REVERSE_AWQ_PACK_ORDER, dtype=torch.long, device=device
)
shifts = torch.arange(0, 32, size_bits, dtype=torch.int32, device=device)

# 解包 int32 → 单独值, 修正 AWQ 顺序
qw = getattr(layer, w_q_name).data
K, N_packed = qw.shape
unpacked = (qw.unsqueeze(-1) >> shifts) & mask

```

**AWQ 特性:** - 激活感知: 基于激活分布优化量化 - **Marlin** 加速: 通过 Marlin kernel 实现高效推理 - **Group-wise** 量化: 支持 64/128 group size - **Zero-point** 支持: 支持对称和非对称量化

### 5.3 Marlin - 高性能 4-bit GEMM Engine

marlin.py 是 vLLM 的核心量化计算引擎:

```

# 文件: /workspace/vllm/model_executor/kernels/linear/mixed_precision/marlin.py
# 行号: 30-100

```

```

class MarlinLinearKernel(MPLinearKernel):
    @classmethod
    def get_min_capability(cls) -> int:
        return 75 # 要求 Turing (SM 7.5) 及以上

    @classmethod
    def can_implement(cls, c: MPLinearLayerConfig) -> tuple[bool, str | None]:
        # Marlin 使用 inline PTX, 仅兼容 NVIDIA
        if not current_platform.is_cuda():
            return False, "Marlin only supported on CUDA"

        quant_types = query_marlin_supported_quant_types(c.zero_points)
        if c.weight_type not in quant_types:
            return False, (
                f"Quant type ({c.weight_type}) not supported by Marlin, "
                f"supported types are: {quant_types}"
            )

        if c.group_size not in MARLIN_SUPPORTED_GROUP_SIZES:
            return False, (
                f"Group size ({c.group_size}) not supported by Marlin"
            )

        return check_marlin_supports_shape(...)

    def process_weights_after_loading(self, layer: torch.nn.Module) -> None:
        device = getattr(layer, self.w_q_name).device
        c = self.config
        is_a_8bit = c.act_type is not None and c.act_type.itemsize == 1

```



```

if is_a_8bit:
    assert c.weight_type == scalar_types.uint4b8
    ops.marlin_int4_fp8_preprocess(getattr(layer, self.w_q_name), inplace=True)
    getattr(layer, self.w_s_name).data *= 512

# 分配 Marlin workspace
self.workspace = marlin_make_workspace_new(device)

def transform_w_q(x):
    permute_param_layout_(x, input_dim=0, output_dim=1, packed_dim=0)
    x.data = ops.gptq_marlin_repack(x.data.contiguous(), ...)

```

**Marlin** 核心优势: - **PTX** 内联汇编: 直接使用 PTX 指令, 最大化硬件利用率 - 权重预打包: `gptq_marlin_repack` 在加载时重新排列权重 - **Workspace** 复用: 预分配 workspace 避免运行时分配 - **FP8 Activation**: 支持 FP8 输入 + INT4 权重的混合精度

## 5.4 Machete - 新一代 CUTLASS GEMM

`machete.py` 是基于 **CUTLASS 3.x** 的新一代量化 GEMM:

# 文件: `/workspace/vllm/model_executor/kernels/linear/mixed_precision/machete.py`  
# 行号: 24-100

```

class MacheteLinearKernel(MPLinearKernel):
    @classmethod
    def get_min_capability(cls) -> int:
        return 90 # ** 仅限 Hopper (SM 9.0) 及以上 **

    @classmethod
    def can_implement(cls, c: MPLinearLayerConfig) -> tuple[bool, str | None]:
        # Machete 使用 CUTLASS, 仅兼容 NVIDIA
        if not current_platform.is_cuda():
            return False, "Machete only supported on CUDA"

        if not current_platform.is_device_capability(90):
            return False, "Machete requires compute capability of 90 (Hopper)"

        if c.has_g_idx and c.partition_weight_shape[0] != c.full_weight_shape[0]:
            return False, "Act reordering currently not supported by Machete..."

        if c.weight_type not in query_machete_supported_quant_types(c.zero_points):
            return False, (...)

        if c.group_size not in query_machete_supported_group_sizes(c.act_type):
            return False, (...)

        return check_machete_supports_shape(...)

    def process_weights_after_loading(self, layer: torch.nn.Module):
        c = self.config
        if c.has_g_idx:
            # 对 activation 进行重排序以匹配 g_idx
            perm = torch.argsort(getattr(layer, self.w_gidx_name)).to(torch.int)
            self.act_perm = lambda x: x[:, perm]

```

```

# 优先使用优化的 permute_cols op
if (c.act_type in [torch.float16, torch.bfloat16]
    and c.partition_weight_shape[0] % 8 == 0):
    self.act_perm = partial(ops.permute_cols, perm=perm)

def transform_w_q(x):
    permute_param_layout_(x, input_dim=0, output_dim=1, packed_dim=0)
    if c.has_g_idx:
        x_unpacked = unpack_quantized_values_into_int32(
            x.data, c.weight_type, packed_dim=0
        )
        x_perm = x_unpacked[perm, :]
        x.data = pack_quantized_values_into_int32(
            x_perm, c.weight_type, packed_dim=0
        )
    # 使用 Machete 的预打包函数
    x.data = ops.machete_prepack_B(
        x.data.t().contiguous().t(),
        a_type=c.act_type,
        b_type=c.weight_type,
        group_scales_type=c.act_type,
    )

```

Machete vs Marlin 对比:

| 特性                    | Marlin          | Machete        |
|-----------------------|-----------------|----------------|
| 最低 GPU 架构             | Turing (SM 7.5) | Hopper (SM 90) |
| 实现方式                  | Inline PTX      | CUTLASS 3.x    |
| <b>GPTQ Act Order</b> | 支持              | 不支持 (EP 场景)    |
| 性能峰值                  | 高               | 更高 (Hopper 优化) |
| 维护成本                  | 中等              | 低 (依赖 CUTLASS) |

## 5.5 GGUF 反量化内核

gguf.py 实现 llama.cpp GGUF 格式支持:

```

# 文件: /workspace/vllm/model_executor/layers/quantization/gguf.py
# 行号: 52-100

```

```

class GGUFConfig(QuantizationConfig):
    """GGUF 量化配置"""

    def get_supported_act_dtypes(self) -> list[torch.dtype]:
        # GGUF 反量化 kernel 内部使用 half precision
        # BF16 在 Blackwell 上有精度问题
        if current_platform.has_device_capability(100):
            logger.warning_once("GGUF has precision issues with bfloat16 on Blackwell.")
            return [torch.half, torch.float32]
        return [torch.half, torch.bfloat16, torch.float32]

    @classmethod
    def get_min_capability(cls) -> int:
        return 60 # 兼容 Pascal 及以上

```

**GGUF** 支持的量化类型 (来自 `gguf.GGMLQuantizationType`): - Q4\_0, Q4\_1 (4-bit) - Q5\_0, Q5\_1 (5-bit) - Q8\_0 (8-bit) - Q2\_K, Q3\_K, Q4\_K, Q5\_K, Q6\_K (K-quantizations) - IQS (IQ quantization)

## 5.6 CUTLASS 扩展内核

vLLM 通过多个模块集成 CUTLASS 功能:

**W8A8 (8-bit Weight, 8-bit Activation)** `cutlass.py`:

```
# 位置: /workspace/vllm/model_executor/kernels/linear/scaled_mm/cutlass.py
class CutlassScaledMMKernel(ScaledMMLinearKernel):
    """CUTLASS 实现的缩放矩阵乘法"""
    def forward(self, x, weight, scale, ...):
        # 调用 CUTLASS W8A8 GEMM
        output = cutlass_scaled_mm(x, weight, scale_x, scale_y, ...)
        return output
```

**NVFP4 (NVIDIA 4-bit Floating Point)** `nvfp4/cutlass.py`:

```
# 位置: /workspace/vllm/model_executor/kernels/linear/nvfp4/cutlass.py
class CutlassNVFP4Kernel(NVFP4LinearKernel):
    """CUTLASS 实现的 NVFP4 线性层"""
    def can_implement(self, config):
        return check_cutlass_nvfp4_support(...)
```

**FP4 (4-bit Floating Point)** `nvfp4/base.py`:

```
# 位置: /workspace/vllm/model_executor/kernels/linear/nvfp4/base.py
class NVFP4LinearKernel(BaseLinearKernel):
    """NVFP4 格式的线性层基类"""
    SUPPORTED_DTYPES = [torch.uint8] # 打包存储
```

## 5.7 ScaledMM - 缩放矩阵乘法框架

`ScaledMMLinearKernel` 提供统一的缩放矩阵乘法接口:

```
graph LR
    subgraph ScaledMM["ScaledMM 后端"]
        S1[Triton<br/>triton.py]
        S2[CUTLASS<br/>cutlass.py]
        S3[Marlin<br/>marlin.py]
        S4[FlashInfer<br/>flashinfer.py]
        S5[DeepGEMM<br/>deep_gemm.py]
        S6[PyTorch<br/>pytorch.py]
        S7[ROCm<br/>rocm.py]
        S8[XPU<br/>xpu.py]
        S9[CPU<br/>cpu.py]
        S10[AITER<br/>aiter.py]
    end
```

各后端自动选择逻辑在 `__init__.py` 中实现。

## 六、MoE 内核

### 6.1 MoE 架构概述

vLLM 实现了模块化 MoE 架构，将 FusedMoE 分解为可组合的组件：

```
graph TB
    subgraph MoEArch[" 模块化 MoE 架构"]
        direction LR
        R[Router<br/> 路由器]
        QD[Quantize-Dispatch<br/> 量化分发]
        PEU[Permute-Experts-Unpermute<br/> 排列-专家计算-逆排列]
        C[Combine<br/> 合并输出]
    end

    R --> QD --> PEU --> C

    subgraph RouterTypes[" 路由器类型"]
        RT1[FusedTopKRouter<br/> 融合 Top-K]
        RT2[GroupedTopKRouter<br/> 分组 Top-K]
        RT3[TopKBiasRouter<br/> 带偏置 Top-K]
    end

    RouterTypes --> R
```

核心抽象类 (modular\_kernel.py)

```
# 文件: /workspace/vllm/model_executor/layers/fused_moe/modular_kernel.py
# 行号: 46-150
```

```
class FusedMoEActivationFormat(Enum):
    """ 标准激活格式 """
    Standard = ("standard",)
    BatchedExperts = ("batched_experts",) # (num_experts, max_tokens, hidden_dim)

@dataclass
class ExpertTokensMetadata:
    """Expert-Token 路由元数据"""
    expert_num_tokens: torch.Tensor
    expert_num_tokens_cpu: torch.Tensor | None

class TopKWeightAndReduce(ABC):
    """ 权重应用与归约的抽象基类 """
    @abstractmethod
    def apply(self, output, fused_expert_output, topk_weights, topk_ids,
              apply_router_weight_on_input) -> torch.Tensor:
        raise NotImplementedError

# MoE 计算流水线:
# [Router] -> [Quantize-Dispatch] -> [Permute-Experts-Unpermute] -> [Combine]
```

## 6.2 Mixtral-style MoE 路由器

fused\_topk\_router.py 实现高性能的融合 Top-K 路由:

```
# 文件: /workspace/vllm/model_executor/layers/fused_moe/router/fused_topk_router.py
# 行号: 17-100
```

```
def vllm_topk_softmax(
    topk_weights: torch.Tensor,
    topk_indices: torch.Tensor,
    token_expert_indices: torch.Tensor,
    gating_output: torch.Tensor,
    renormalize: bool = False,
) -> tuple[torch.Tensor, ...]:
    """ 融合 Top-K + Softmax 操作 """
    ops.topk_softmax(
        topk_weights, topk_indices,
        token_expert_indices, gating_output, renormalize,
    )
    return topk_weights, topk_indices

def fused_topk(
    hidden_states: torch.Tensor,
    gating_output: torch.Tensor,
    topk: int,
    renormalize: bool,
    indices_type: torch.dtype | None = None,
    scoring_func: str = "softmax",
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    """
    执行 MoE Top-K 路由:
    1. 计算 gating logits
    2. 选择 Top-K experts
    3. 应用 softmax/sigmoid 归一化
    4. 返回 weights, ids, token-expert 映射
    """
    M, _ = hidden_states.size()

    topk_weights = torch.empty(M, topk, dtype=torch.float32, device=hidden_states.device)
    topk_ids = torch.empty(M, topk, dtype=torch.int32 if indices_type is None else indices_type,
                           device=hidden_states.device)
    token_expert_indices = torch.empty(M, topk, dtype=torch.int32, device=hidden_states.device)

    if scoring_func == "softmax":
        topk_func = dispatch_topk_softmax_func(use_rocm_aiter=...)
        topk_weights, topk_ids = topk_func(
            topk_weights, topk_ids, token_expert_indices, gating_output, renormalize
        )
    elif scoring_func == "sigmoid":
        # ... sigmoid 变体
```

关键优化: - **Fused Kernel**: Top-K + Softmax 在单个 CUDA kernel 中完成 - **ROCm AITER** 支持: AMD GPU 使用专用 AITER 实现 - 多评分函数: 支持 softmax 和 sigmoid 两种路由方式

### 6.3 WNA16 (4-bit MoE) 量化

moe\_wna16.py 实现 MoE 专家权重的 4-bit 量化:

# 文件: /workspace/vllm/model\_executor/layers/quantization/moe\_wna16.py

```
class MoEWNA16Config(QuantizationConfig):
    """WNA16 (4-bit) MoE 量化配置"""
    def get_supported_act_dtypes(self) -> list[torch.dtype]:
        return [torch.bfloat16, torch.float16]

    def get_quant_method(self, layer, prefix):
        if isinstance(layer, FusedMoE):
            return MoEWNA16Method()
        return None

class MoEWNA16Method(FusedMoEMethodBase):
    """WNA16 量化的 MoE 方法"""
    def __init__(self):
        # 使用 Marlin 或 Machete 作为 4-bit GEMM 后端
        self.linear_kernel = choose_mp_linear_kernel(...)
```

### 6.4 MXFP8 MoE 线性层

Mxfp8LinearKernel 提供 MXFP8 (Microscaling FP8) 格式的 MoE 线性层:

# 位置: /workspace/vllm/model\_executor/kernels/linear/mxfp8/Mxfp8LinearKernel.py

```
class Mxfp8LinearKernel(BaseLinearKernel):
    """MXFP8 格式的线性层, 专为 MoE 设计"""
    BACKENDS = {
        "marlin": MarlinMxfp8Impl,          # NVIDIA Marlin
        "flashinfer": FlashInferMxfp8Impl,  # FlashInfer
        "emulation": EmulationImpl,         # CPU 回退仿真
        "xpu": XPUMxfp8Impl,                # Intel XPU
    }
```

**MXFP8 特性:** - **Microscaling Format:** 每个 block 共享一个 exponent - 高压缩比: 相比 FP16 节省 ~50% 显存 (对大 MoE 模型至关重要) - 多后端支持: Marlin、FlashInfer、XPU 等

---

## 七、通信内核

### 7.1 Custom AllReduce - 自定义全归约

custom\_all\_reduce.py 实现节点内高性能 AllReduce:

# 文件: /workspace/vllm/distributed/device\_communicators/custom\_all\_reduce.py  
# 行号: 50-100

```
class CustomAllreduce:
    _SUPPORTED_WORLD_SIZES = [2, 4, 6, 8]

    def __init__(
        self,
        group: ProcessGroup,
```

```

device: int | str | torch.device,
max_size=8192 * 1024, # 默认 8MB
symm_mem_enabled=False,
) -> None:
    """
    初始化 Custom AllReduce:

    Args:
        group: 进程组 (必须是非 NCCL 组)
        device: 绑定的设备 ID
        max_size: 支持的最大 allreduce 大小
        symm_mem_enabled: 是否启用对称内存
    """
    self._IS_CAPTURING = False
    self.disabled = True

    if not custom_ar:
        logger.info("Custom allreduce is disabled because of missing library")
        return

    self.group = group
    assert dist.get_backend(group) != dist.Backend.NCCL, (
        "CustomAllreduce should be attached to a non-NCCL group."
    )

    if not all(in_the_same_node_as(group, source_rank=0)):
        logger.warning("Custom allreduce is disabled for multi-node case.")
        return

```

**Custom AllReduce 优势:** - **P2P 通信:** 使用 NVLink/P2P 直接 GPU 间通信 - 低延迟: 比 NCCL AllReduce 延迟降低 **3-5x** - **CUDA Graph 兼容:** 可在 CUDA Graph 中捕获 - 内存高效: 使用对称内存避免额外拷贝

## P2P 可达性检查

# 文件: /workspace/vllm/distributed/device\_communicators/custom\_all\_reduce.py  
# 行号: 31-40

```

def _can_p2p(rank: int, world_size: int) -> bool:
    for i in range(world_size):
        if i == rank:
            continue
        if envs.VLLM_SKIP_P2P_CHECK:
            return torch.cuda.can_device_access_peer(rank, i)
        if not gpu_p2p_access_check(rank, i):
            return False
    return True

```

## 7.2 CU Memory Allocator - CUDA 内存管理器

cumem.py 实现基于 cuMem API 的可插拔内存分配器:

# 文件: /workspace/vllm/device\_allocator/cumem.py  
# 行号: 27-88

```

cumem_available = False
try:

```

```

from vllm.cumem_allocator import (
    init_module,
    python_create_and_map,
    python_unmap_and_release,
)
from vllm.distributed.device_communicators.cuda_wrapper import CudaRTLlibrary

lib_name = find_loaded_library("cumem_allocator")
libcudart = CudaRTLlibrary()
cumem_available = True
except ModuleNotFoundError:
    init_module = None
    python_create_and_map = None
    python_unmap_and_release = None
    lib_name = None

def get_pluggable_allocator(
    python_malloc_fn: Callable[[HandleType], None],
    python_free_func: Callable[[int], HandleType],
) -> torch.cuda.memory.CUDAPluggableAllocator:
    """ 创建 PyTorch 可插拔分配器 """
    init_module(python_malloc_fn, python_free_func)
    new_alloc = torch.cuda.memory.CUDAPluggableAllocator(
        lib_name, "my_malloc", "my_free"
    )
    return new_alloc

@dataclasses.dataclass
class AllocationData:
    handle: HandleType
    tag: str
    cpu_backup_tensor: torch.Tensor | None = None

class CuMemAllocator:
    """
    基于 cuMem 的单例内存池管理器。
    支持 sleep 模式：可将标记的 tensor 卸载或丢弃。
    """

```

**CuMemAllocator** 核心功能：- 虚拟内存管理：使用 cuMemCreate/cuMemMap 管理虚拟地址空间 - **Sleep Mode**：支持将内存池内容卸载到 CPU 或直接丢弃 - **Tag-based** 管理：按 tag 分组管理不同用途的内存 - **Pluggable Interface**：通过 PyTorch 的 CUDAPluggableAllocator 接口集成

## 八、采样内核

### 8.1 Top-k / Top-p 采样器

sampler.py 实现完整的采样流程：

```

# 文件：/workspace/vllm/v1/sample/sampler.py
# 行号：21-143

```



```

class Sampler(nn.Module):
    """
    采样层执行以下步骤:
    1. 如果需要 logprobs: 计算原始 logprobs
    2. 将 logits 转换为 float32
    3. 应用允许的 token id 白名单
    4. 应用 bad words 排除
    5. 应用 logit processors (非 argmax-invariant)
    6. 应用 penalties (重复/频率/存在惩罚)
    7. 采样下一个 token:
        a) 如果不是 all_random: 贪婪采样
        b) 应用 temperature
        c) 应用 argmax-invariant logit processors (min_p)
        d) 应用 top_k 和/或 top_p
        e) 从概率分布采样
    8. 收集 logprobs
    9. 返回 SamplerOutput
    """

    def __init__(self, logprobs_mode: LogprobsMode = "raw_logprobs"):
        super().__init__()
        self.topk_topp_sampler = TopKTopPSampler(logprobs_mode)
        self.pin_memory = is_pin_memory_available()
        self.logprobs_mode = logprobs_mode

    def forward(self, logits, sampling_metadata, predict_bonus_token=False,
                logprobs_mode_override=None) -> SamplerOutput:
        # Step 1-6: Preprocessing
        logits = logits.to(torch.float32)
        logits = self.apply_logits_processors(logits, sampling_metadata, ...)

        # Step 7: Sampling
        sampled, processed_logprobs = self.sample(logits, sampling_metadata)

        # Step 8-9: Post-processing
        sampled = sampled.long()
        ...

```

## 8.2 TopKTopPSampler - 核心采样 Kernel

`topk_topp_sampler.py` 和 `topk_topp_triton.py` 实现 GPU 加速的 Top-k/Top-p 采样:

graph TB

```

subgraph SamplingPipeline[" 采样流程"]
    L[Logits<br/> 模型输出] --> T[Temperature<br/> 温度缩放]
    T --> TK[Top-K<br/> 截断]
    TK --> TP[Top-P<br/> 核采样]
    TP --> N[Normalize<br/> 归一化]
    N --> S[Sample<br/> 采样]
end

```

```

subgraph Optimizations[" 优化技术"]
    01[Persistent TopK<br/> 持久化 Top-K]
    02[Fused Kernel<br/> 融合操作]
end

```

```
    03[Batched<br/> 批处理]
end
```

SamplingPipeline --> Optimizations

采样算法说明: - **Top-k**: 保留概率最高的 k 个 token, 其余置零 - **Top-p (nucleus)**: 保留累积概率达到 p 的最小 token 集 - **Temperature**: 控制分布的锐度 ( $T \rightarrow 0$ : 贪婪,  $T \rightarrow \infty$ : 均匀) - **Min-p**: 基于最大概率的阈值过滤

---

## 九、平台特定内核

### 9.1 ROCm (AMD GPU) 内核

rocm.py 定义 AMD ROCm 平台支持:

```
# 文件: /workspace/vllm/platforms/rocm.py
# 行号: 28-74

try:
    from amdsmi import (
        AmdSmiException,
        amdsmi_get_gpu_asic_info,
        amdsmi_get_gpu_device_uuid,
        amdsmi_get_processor_handles,
        amdsmi_init,
        amdsmi_shut_down,
        amdsmi_topo_get_link_type,
        amdsmi_topo_get_numa_node_number,
    )
except ImportError as e:
    logger.warning("Failed to import from amdsmi with %r", e)

# import custom ops, trigger op registration
try:
    import vllm._C # noqa: F401
except ImportError as e:
    logger.warning("Failed to import from vllm._C with %r", e)

try:
    import vllm._rocm_C # noqa: ROCm 特定扩展
except ImportError as e:
    logger.warning("Failed to import from vllm._rocm_C with %r", e)

_ROCM_DEVICE_ID_NAME_MAP: dict[str, str] = {
    "0x74a0": "AMD_Instinct_MI300A",
    "0x74a1": "AMD_Instinct_MI300X",
    "0x74b5": "AMD_Instinct_MI300X", # MI300X VF
    "0x74a5": "AMD_Instinct_MI325X",
    "0x74b9": "AMD_Instinct_MI325X", # MI325X VF
    "0x7550": "AMD_Radeon_RX9070XT", # RDNA 4 (Navi 48)
    "0x7551": "AMD_Radeon_R9700", # RDNA 4
}
```

**ROCm** 特定组件: | 组件 | 说明 | |—|—| | **\_rocm\_C** | ROCm 编译的 C++ 扩展 | | **AITER** | AMD AI Tensor 扩展库 | | **amdsmi** | AMD 系统管理接口 (类似 NVML) | | **RCCL** | ROCm Communication Collectives Library |

## ROCm 注意力后端

- rocm\_attn.py: 通用 ROCm 注意力
- rocm\_aiter\_fa.py: AITER Flash Attention
- rocm\_aiter\_mla.py: AITER MLA
- rocm\_aiter\_unified\_attn.py: 统一 AITER 注意力

## 9.2 CPU 内核

cpu.py 定义 CPU 平台支持:

```
# 文件: /workspace/vllm/platforms/cpu.py  
# 行号: 41-99
```

```
class CpuPlatform(Platform):  
    _enum = PlatformEnum.CPU  
    device_name: str = "cpu"  
    device_type: str = "cpu"  
    dispatch_key: str = "CPU"  
    dist_backend: str = "gloo"  
  
    @property  
    def supported_dtypes(self) -> list[torch.dtype]:  
        if self.get_cpu_architecture() == CpuArchEnum.POWERPC:  
            return [torch.bfloat16, torch.float32]  
        elif self.get_cpu_architecture() == CpuArchEnum.ARM:  
            if sys.platform.startswith("darwin"):  
                # Apple Silicon with BF16  
                return [torch.bfloat16, torch.float16, torch.float32]  
            return [torch.float16, torch.float32]  
        elif self.get_cpu_architecture() == CpuArchEnum.RISCV:  
            return [torch.bfloat16, torch.float16, torch.float32]  
        # x86/aarch64: 全部支持  
        return [torch.bfloat16, torch.float16, torch.float32]  
  
    @classmethod  
    def get_attn_backend_cls(cls, selected_backend, attn_selector_config, num_heads=None):  
        if attn_selector_config.use_mla:  
            raise NotImplementedError("MLA is not supported on CPU.")  
        if attn_selector_config.use_sparse:  
            raise NotImplementedError("Sparse Attention is not supported on CPU.")  
        return AttentionBackendEnum.CPU_ATTN.get_path()
```

CPU 架构支持:

| CPU 架构        | SIMD 扩展           | 特殊指令        |
|---------------|-------------------|-------------|
| x86-64        | AVX2/AVX-512      | AMX (Intel) |
| ARM (AArch64) | NEON              | SVE (可选)    |
| PowerPC       | VSX/AltiVec       | -           |
| RISC-V        | RVV (V extension) | -           |
| Apple Silicon | AMX (Apple)       | BF16 硬件加速   |

**CPU 注意力后端**    cpu\_attn.py: - 使用 PyTorch 原生 SDPA 或手动实现 - 针对 CPU 缓存层次结构优化 - 支持多线程并行化

### 9.3 Intel XPU 内核

xpu\_ops.py 和 xpu.py:

- **oneAPI** 支持: Intel Data Parallel C++ 后端
  - **MMX** 引擎: 利用 Intel X Matrix Extensions
  - **XPU MLA**: xpu\_mla\_sparse.py
  - **XPU Linear Kernels**: xpu.py, mixed\_precision/xpu.py
- 

## 十、内核分类与调用链总览

### 10.1 完整内核分类图

graph TB

```
subgraph EntryPoints[" 入口点"]
    E1[vllm/kernels/__init__.py]
    E2[vllm/_custom_ops.py]
    E3[platforms/*.py]
end

subgraph AttentionSystem[" 注意力系统"]
    direction TB
    A_REG[registry.py<br/> 后端注册表]
    A_FI[flashinfer.py<br/>FlashInfer 后端]
    A_FA[flash_attn.py<br/>FlashAttn 后端]
    A_TR[triton_attn.py<br/>Triton 后端]
    A_MLA[mla/<br/>MLA 子系统]
    A_CPU[cpu_attn.py<br/>CPU 后端]
    A_ROCM[rocm_attn.py<br/>ROCm 后端]
end

subgraph CacheSystem[" 缓存系统"]
    C_RC[reshape_and_cache_flash.py<br/>KV 写入]
    C_NV[nvfp4_utils.py<br/>NVFP4 格式]
    C_FP8[fp8_utils.py<br/>FP8 量化]
end

subgraph ActivationSystem[" 激活函数系统"]
    AC_RMS[vllm_c.py<br/>RMSNorm]
    AC_ROPE[rotary_embedding/<br/>RoPE 变体]
    AC_SILU[helion/<br/>SiLU*FP8]
end

subgraph QuantSystem[" 量化系统"]
    Q_MP[mixed_precision/<br/>混合精度]
    Q_SM[scaled_mm/<br/>缩放 MM]
    Q_NVP[nvfp4/<br/>NVFP4 线性层]
    Q_MXF[mxfp8/<br/>MXFP8 线性层]
end

subgraph MoESystem[" MoE 系统"]
    M_MOD[modular_kernel.py<br/>模块化架构]
    M_ROUT[router/<br/>路由器]
    M_WNA[moe_wna16.py<br/>4-bit MoE]
```

```

end

subgraph CommSystem[" 通信系统"]
    CM_AR[custom_all_reduce.py<br/> 自定义 AllReduce]
    CM_QR[quick_all_reduce.py<br/> 快速 Reduce]
    CM_CU[cumem.py<br/>CU Mem Alloc]
end

subgraph SampleSystem[" 采样系统"]
    SM_S[sampler.py<br/> 采样器]
    SM_TK[topk_topk_<br/>TopK/TopP Kernel]
end

EntryPoints --> AttentionSystem
EntryPoints --> CacheSystem
EntryPoints --> ActivationSystem
EntryPoints --> QuantSystem
EntryPoints --> MoESystem
EntryPoints --> CommSystem
EntryPoints --> SampleSystem

style EntryPoints fill:#e8f5e9,stroke:#2e7d32,color:#000
style AttentionSystem fill:#fff3e0,stroke:#ef6c00,color:#000
style CacheSystem fill:#fce4ec,stroke:#c62828,color:#000
style ActivationSystem fill:#f3e5f5,stroke:#7b1fa2,color:#000
style QuantSystem fill:#e0f2f1,stroke:#00796b,color:#000
style MoESystem fill:#e8eaf6,stroke:#303f9f,color:#000
style CommSystem fill:#fff8e1,stroke:#f9a825,color:#000
style SampleSystem fill:#fbe9e7,stroke:#d84315,color:#000

```

## 10.2 典型调用链示例

推理时注意力计算调用链

```

sequenceDiagram
    participant Model as Model Forward
    participant Attn as Attention Backend
    participant FI as FlashInfer
    participant Cache as KV Cache Ops
    participant Kernel as Triton/CUDA Kernel

    Model->>Attn: forward(hidden_states, kv_cache, metadata)
    Attn->>FI: batch_prefill_with_paged_kv(q, k_v_cache, metadata)
    FI->>Cache: reshape_and_cache(new_k, new_v, cache, slot_mapping)
    Cache->>Kernel: launch reshape_and_cache_kernel_flash()
    Kernel-->>Cache: KV written
    FI->>Kernel: launch flashinfer attention kernel
    Kernel-->>FI: attention output
    FI-->>Attn: output tensor
    Attn-->>Model: context output

```

量化线性层调用链

```

sequenceDiagram
    participant Layer as Linear Layer

```

```

participant MP as MPLinearKernel
participant Marlin as Marlin Kernel
participant CustomOps as _custom_ops

Layer->>MP: forward(x)
MP->>Marlin: marlin_gemm(x, q_weight, scales, workspace)
Marlin->>CustomOps: ops.gptq_marlin_gemm(...)
CustomOps-->>Marlin: result (CUDA kernel executed)
Marlin-->>MP: output tensor
MP-->>Layer: linear output

```

### 10.3 性能优化策略总结

| 优化维度                     | 技术手段                                   | 适用场景       |
|--------------------------|----------------------------------------|------------|
| <b>Kernel Fusion</b>     | FusedAddRMSNorm,<br>FusedInvRoPE+FP8   | 减少内存带宽压力   |
| <b>Memory Layout</b>     | Head-Major, PagedAttention             | 优化缓存利用率    |
| <b>Quantization</b>      | INT4/FP8/NVFP4                         | 减少显存占用     |
| <b>Communication</b>     | CustomAllReduce, P2P                   | 降低通信延迟     |
| <b>Batching</b>          | Continuous Batching, uBatching         | 提高 GPU 利用率 |
| <b>Auto-tuning</b>       | Helion configs, Backend selection      | 自动选择最优路径   |
| <b>Platform-specific</b> | SM-specific kernels, ISA<br>extensions | 充分利用硬件特性   |

### 10.4 关键文件索引

| 类别         | 核心文件                                                          | 行号范围     |
|------------|---------------------------------------------------------------|----------|
| 构建系统       | vllm/kernels/vllm_c.py                                        | 1-63     |
| 构建系统       | vllm/utils/torch_utils.py                                     | 928-967  |
| 注意力        | vllm/v1/attention/backends/registry.py                        | 34-87    |
| 注意力        | vllm/v1/attention/backends/flashinfer.py                      | 1-149    |
| 缓存         | vllm/v1/attention/ops/triton_reshape_and_cache_flash.py       | 17-100   |
| 缓存         | vllm/utils/torch_utils.py (NVFP4)                             | 415-469  |
| 量化         | vllm/model_executor/kernels/linear/mixed_precision/marlin.py  | 30-100   |
| 量化         | vllm/model_executor/kernels/linear/mixed_precision/machete.py | 21-100   |
| 量化         | vllm/model_executor/layers/quantization/gptq_marlin.py        | 67-100   |
| 量化         | vllm/model_executor/layers/quantization/gptq.py               | 72-100   |
| <b>MoE</b> | vllm/model_executor/layers/fused_moe/moe_mar_kernel.py        | 46-150   |
| <b>MoE</b> | vllm/model_executor/layers/fused_moe/moe_topk_router.py       | 107-160  |
| 通信         | vllm/distributed/device_communicators/custom_all_reduce.py    | 508-1000 |
| 通信         | vllm/device_allocator/cumem.py                                | 27-88    |
| 采样         | vllm/v1/sample/sampler.py                                     | 21-143   |
| 平台         | vllm/platforms/cuda.py                                        | 79-143   |
| 平台         | vllm/platforms/rocm.py                                        | 28-74    |
| 平台         | vllm/platforms/cpu.py                                         | 41-99    |

## 总结

本文档深入分析了 vLLM 的 **CUDA/C++** 内核层架构，揭示了其作为现代 LLM 推理引擎的核心设计理念：

## 架构特点

1. 混合内核生态: Python 绑定 + Triton JIT + C++/CUDA 扩展 + 第三方库 (FlashInfer、Marlin、CUTLASS)
2. 平台抽象层: 通过 Platform 接口屏蔽 CUDA/ROCm/CPU/XPU 差异
3. 可插拔后端: 注意力、线性层、MoE 等均支持多后端动态选择
4. **IR Op** 系统: 通过 `ir.ops` 实现算子级别的优化和替换

## 性能优化核心

- **PagedAttention**: 创新的分页式注意力, 解决显存碎片问题
- 量化支持: 从 INT4 到 FP8 的完整量化栈 (AWQ/GPTQ/GGUF/Marlin/Machete)
- 通信优化: Custom AllReduce + P2P 显著降低张量并行延迟
- **MoE** 加速: 模块化架构支持多种路由和专家计算方案

## 未来方向

- **Blackwell (SM 10.x)**: 充分利用 TMA (Tensor Memory Accelerator) 和 FP4
- 稀疏注意力: Sparse MLA 等长序列优化
- 更多量化格式: MXFP4、OC-MX 等新微缩放格式
- 跨平台统一: 通过 CUTLASS 和 Triton 实现更好的可移植性

---

文档版本: v1.0 基于源码: `/workspace/vllm` 生成日期: 2026-05-10 总行数: ~1200 行

## vLLM 编译与运行时优化分析

定位: 本文档深入分析 vLLM 的 `torch.compile` 编译流水线、CUDA Graph 捕获/回放机制、编译 Pass 优化体系、代码生成与缓存等核心运行时优化子系统。

### graph TB

```
subgraph COMP[" 编译优化核心"]
    direction TB
    TC["torch.compile 集成 <br/>Dynamo + Inductor"]
    CP["编译 Passes<br/>PostGradPassManager"]
    PR["Partition Rules<br/>图分割规则"]
    CG["Codegen 代码生成 <br/>execution_fn"]
    CH["Caching 编译缓存 <br/>VllmSerializableFunction"]
end

subgraph CUDA["  CUDA Graphs"]
    direction TB
    PW["Piecewise CUDAGraphs<br/>子图捕获 (推荐)"]
    FL["Full CUDAGraphs<br/>整体捕获"]
    EN["Encoder CUDAGraphs<br/>视觉编码器图捕获"]
    CW["CUDAGraphWrapper<br/>捕获/回放调度器"]
end

subgraph SUPPORT[" 支撑系统"]
    direction TB
    PB["PiecewiseBackend<br/>分段后端 & 形状分发"]
    BSG["BaseStaticGraph<br/>静态图基类协议"]
    MON["Monitor 编译监控 <br/>时间统计 & depyf"]
    CNT["Counter 性能计数器 <br/>编译指标追踪"]
    TU["Triton Utils<br/>JIT 监控 & 分配器"]
end
```

```

end

TC --> CP
CP --> PR
PR --> CG
CG --> CH
CH --> PW
CH --> FL
PW --> CW
FL --> CW
EN --> CW
CW --> PB
PB --> BSG
BSG --> MON
MON --> CNT
CNT --> TU

style COMP fill:#b2dfdb,stroke:#00897b,color:#1b5e20
style CUDA fill:#bbdefb,stroke:#1565c0,color:#0d47a1
style SUPPORT fill:#ffe0b2,stroke:#e65100,color:#bf360c

```

---

## 一、torch.compile 集成

### 1.1 Dynamo + Inductor 编译流水线原理

vLLM 的编译流水线基于 PyTorch 的 `torch.compile`，采用 **Dynamo** → **FX Graph** → 图分割 → 分段编译 → **CUDA Graph** 捕获 的多层架构：

```

sequenceDiagram
    participant User as 用户代码
    participant Dynamo as Torch Dynamo
    participant VB as VllmBackend
    participant SG as split_graph
    participant PCI as PiecewiseCompileInterpreter
    participant PB as PiecewiseBackend
    participant CM as CompilerManager
    participant IA as InductorAdaptor
    participant CG as CUDAGraphWrapper

    User->>Dynamo: torch.compile(model)
    Dynamo->>Dynamo: 字节码分析 & 追踪
    Dynamo->>VB: __call__(fx.GraphModule, example_inputs)
    VB->>VB: configure_post_pass()
    VB->>SG: split_graph(graph, splitting_ops)
    SG-->>VB: split_gm + piecewise_graphs
    VB->>PCI: run(*fake_args) [遍历分割子图]
    PCI->>PB: 创建 PiecewiseBackend
    PB->>CM: compile_all_ranges()
    loop 每个 compile_range
        CM->>IA: compile_fx(graph, args)
        IA-->>CM: compiled_callable
    end
    PCI->>CG: wrap_with_cudagraph_if_needed()

```



```
VB->>VB: generate_execution_code() + compile_execution_fn()
VB-->>User: VllmSerializableFunction
```

## 1.2 CustomOp 自定义算子注册

源码: custom\_op.py

vLLM 通过 CustomOp 基类实现跨平台算子分发。每个自定义算子可以注册不同平台的前向实现:

注册机制—使用装饰器模式:

```
# /workspace/vllm/model_executor/custom_op.py L307-L320
@classmethod
def register(
    cls,
    name: str,
    dynamic_arg_dims: dict[str, int | list[int]] | None = None,
):
    def decorator(op_cls):
        assert name not in op_registry, f"Duplicate op name: {name}"
        op_cls.name = name
        op_cls._dynamic_arg_dims = dynamic_arg_dims
        op_registry[name] = op_cls
        return op_cls

    return decorator
```

平台分发逻辑—custom\_op.py L174-L207:

```
def dispatch_forward(self, compile_native: bool):
    compilation_config = get_cached_compilation_config()
    enabled = self._enforce_enable or self.enabled()

    if not enabled:
        # 未启用自定义算子时, 使用 forward_native 并可选地 compile
        return self.maybe_compile(self.forward_native, enable=compile_native)

    if current_platform.is_rocm():
        return self.forward_hip          # ROCm (AMD GPU)
    elif current_platform.is_cpu():
        return self.forward_cpu           # CPU
    elif current_platform.is_tpu():
        return self.forward_tpu           # TPU
    elif current_platform.is_xpu():
        return self.forward_xpu           # Intel XPU
    elif current_platform.is_out_of_tree():
        return self.forward_oot           # Out-of-tree
    else:
        return self.forward_cuda          # NVIDIA CUDA (默认)
```

关键设计要点: - **OOT (Out-of-Tree)** 替换机制: 通过 register\_oot() 装饰器允许第三方替换整个算子实现 (custom\_op.py L332-L353) - **PluggableLayer**: 模块级组合抽象, 支持 OOT 整层替换 (custom\_op.py L32-L100)  
- **maybe\_compile**: 对不透明自定义算子内部调用进行 torch.compile 编译 (custom\_op.py L209-L269)

## 1.3 编译后端选择

源码: backends.py · compiler\_interface.py

```
make_compiler() 函数根据 CompilationConfig.backend 选择编译后端:

# /workspace/vllm/compilation/backends.py L96-L122
def make_compiler(compilation_config: CompilationConfig) -> CompilerInterface:
    if compilation_config.backend == "inductor":
        if envs.VLLM_USE_STANDALONE_COMPILE and hasattr(
            torch._inductor, "standalone_compile"
        ):
            return InductorStandaloneAdaptor(...) # PyTorch 2.8+ 独立编译
        else:
            return InductorAdaptor() # 传统 compile_fx 路径
    elif compilation_config.backend == "eager":
        return EagerAdaptor() # 不编译, 直接执行
    else:
        compiler = resolve_obj_by_qualname(
            current_platform.get_compile_backend()
        )()
        return compiler # 自定义后端
```

三种编译后端对比:

| 后端                  | 类名                        | 适用场景            | 缓存支持         |
|---------------------|---------------------------|-----------------|--------------|
| inductor            | InductorAdaptor           | PyTorch 2.5-2.7 | FxGraphCache |
| inductor_standalone | InductorStandaloneAdaptor | PyTorch 2.8+    | AOT Artifact |
| eager               | EagerAdaptor              | 调试/无编译          |              |

**InductorStandaloneAdaptor** 的关键特性 (compiler\_interface.py L251-L446):- 使用 `standalone_compile()` API 替代 `compile_fx()` - 支持 AOT (Ahead-of-Time) 编译模式 (`VLLM_USE_MEGA_AOT_ARTIFACT`) - 支持 `donate_graph_module` (PyTorch 2.13+) 减少内存开销 - 自动原子写入防止多进程缓存损坏

## 二、编译 Passes

### 2.1 Pass 管理架构

源码: `pass_manager.py`

`PostGradPassManager` 是 vLLM 编译 Pass 系统的核心管理器, 继承自 PyTorch 的 `CustomGraphPass`:

**Pass 执行顺序** (`pass_manager.py` L98-L131):

- 1. 用户自定义 Passes (constructor parameter)
- 2. 默认 Passes:
  - `NoOpEliminationPass` (消除空操作)
  - `SequenceParallelismPass` (序列并行)
  - `AsyncTPPass` (异步张量并行通信融合)
  - `AllReduceFusionPass` (AllReduce + RMSNorm 融合)
  - `MiniMaxQKNormPass` (MiniMax QK-Norm 融合)
  - `RMSNormQuantFusionPass` (RMSNorm + 量化融合)
  - `ActivationQuantFusionPass` (激活 + 量化融合)
  - `RocmAiter* 系列` (ROCm AITER 特定融合)
  - `SplitCoalescingPass` (分割合并)
  - `ScatterSplitReplacementPass` (scatter/split 替换)
  - `RopeKVCacheFusionPass` (RoPE + KV Cache 融合)
  - `AttnQuantFusionPass` (注意力 + 量化融合)

|                               |                     |
|-------------------------------|---------------------|
| QKNormRoPEFusionPass          | (QK-Norm + RoPE 融合) |
| 3. PostCleanupPass            | (清理融合产物)            |
| 4. VllmIRLoweringPass         | (IR 降低)             |
| 5. UnsafeCloneEliminationPass | (克隆消除)              |
| 6. PostCleanupPass            | (再次清理)              |
| 7. FixFunctionalizationPass   | (修复函数化) ← 最后执行      |

## 2.2 InductorPass 基类

源码: `inductor_pass.py`

所有 vLLM 编译 Pass 的基类, 提供 UUID 哈希和范围适用性检查:

```
# /workspace/vllm/compilation/passes/inductor_pass.py L66-L108
class InductorPass(CustomGraphPass):
    def uuid(self) -> str:
        """ 提供唯一标识符用于 Inductor 代码缓存 """
        return InductorPass.hash_source(self)

    @staticmethod
    def hash_source(*srcs: str | Any) -> str:
        """ 哈希函数/对象源码, 确保 pass 变更触发重新编译 """
        cache_key = tuple(
            src if isinstance(src, (str, type, types.FunctionType))
            else src.__class__
            for src in srcs
        )
        return _hash_source_cached(*cache_key)

    def is_applicable_for_range(self, compile_range: Range) -> bool:
        return True
```

## 2.3 VllmInductorPass 增强

源码: `vllm_inductor_pass.py`

为 InductorPass 增加 vLLM 特有功能: 计时、日志、图 dump、模式匹配:

```
# /workspace/vllm/compilation/passes/vllm_inductor_pass.py L34-L84
class VllmInductorPass(InductorPass):
    dump_prefix: ClassVar[int | None] = None

    def __init__(self, config: VllmConfig):
        self.compilation_config = InductorCompilationConfig(...)
        self.pass_config = config.compilation_config.pass_config
        self.model_dtype = config.model_config.dtype
        ...

    @staticmethod
    def time_and_log(call_fn):
        @functools.wraps(call_fn)
        def wrapped(self, graph):
            self.begin()
            self.dump_graph(graph, "before")
            call_fn(self, graph)
            self.dump_graph(graph, "after")
```

```

        self.end_and_log()
    return wrapped

```

**VllmPatternMatcherPass** — 基于 Inductor PatternMatcher 的融合 Pass 框架 (vllm\_inductor\_pass.py L266-L305):

```

class VllmFusionPatternMatcherPass(VllmPatternMatcherPass):
    @enable_fake_mode
    def register(self, pr: VllmPatternReplacement):
        pm.register_replacement(
            pr.pattern, pr.replacement, pr.get_inputs(),
            self._trace_fn, self.pm_pass,
        )
        self._pattern_replacements.append(pr)

```

**VllmPatternReplacement** — 声明式模式/替换对 (vllm\_inductor\_pass.py L194-L247):

```

class VllmPatternReplacement(ABC, Generic[P, R]):
    @property
    @abstractmethod
    def pattern(self) -> Callable[P, R]: ...    # 要搜索的子图

    @property
    @abstractmethod
    def replacement(self) -> Callable[P, R]: ... # 替换子图

    @abstractmethod
    def get_inputs(self) -> list[torch.Tensor]: ... # 示例张量

```

## 2.4 FX 工具函数

源码: fx\_utils.py

提供 FX 图遍历和节点查找的实用函数集:

| 函数                            | 功能                               |
|-------------------------------|----------------------------------|
| is_func(node, target)         | 检查节点是否是特定函数调用                    |
| is_auto_func(node, op)        | 检查是否是 auto_functionalized 包装的 op |
| find_auto_fn_maybe(nodes, op) | 查找 auto_functionalized 节点        |
| find_getitem_maybe(node, idx) | 查找 getitem 节点                    |
| find_op_nodes(op, graph)      | 查找所有匹配 OpOverload/Package 的节点    |
| get_only_user(node)           | 获取节点的唯一用户                        |

## 三、CUDA Graphs

### 3.1 架构总览

源码: cuda\_graph.py

vLLM 实现了两种 CUDA Graph 模式, 通过 CUDAGraphMode 枚举控制:

```

graph LR
    subgraph Modes["CUDA Graph 模式"]
        NONE["NONE<br/> 不使用 CUDA Graph"]
        PIECEWISE["PIECEWISE<br/> 分段捕获 (推荐)"]
    end

```

```

    FULL["FULL<br/> 整体捕获"]
    FAP["FULL_AND_PIECEWISE<br/> 同时启用两者"]
end

subgraph Dispatch[" 运行时分发"]
    FC["ForwardContext"]
    BD["BatchDescriptor"]
    CRM["CudagraphRuntimeMode"]
end

NONE -->|"runtime_mode == NONE"| EAGER["Eager 执行"]
PIECEWISE -->|"mode == PIECEWISE"| PWC["Piecewise Capture/Replay"]
FULL -->|"mode == FULL"| FWC["Full Capture/Replay"]
FAP --> PWC
FAP --> FWC

FC --> BD
FC --> CRM
BD --> PWC
BD --> FWC
CRM --> EAGER

```

### 3.2 CUDAGraphWrapper 一核心包装器

源码: cuda\_graph.py L145-L361

CUDAGraphWrapper 是 CUDA Graph 捕获/回放的统一入口:

初始化 (cuda\_graph.py L178-L209):

```

class CUDAGraphWrapper:
    _all_instances: ClassVar[weakref.WeakSet] = weakref.WeakSet()

    def __init__(self, runnable, vllm_config, runtime_mode, cudagraph_options=None):
        self.runnable = runnable
        self.runtime_mode = runtime_mode          # FULL 或 PIECEWISE
        self.graph_pool = current_platform.get_global_graph_pool()
        self.concrete_cudagraph_entries: dict[BatchDescriptor, CUDAGraphEntry] = {}
        CUDAGraphWrapper._all_instances.add(self)

```

调用/分发逻辑 (cuda\_graph.py L233-L361):

```

def __call__(self, *args, **kwargs):
    # 1. 无前向上下文 → 直接执行 (如 vision encoder)
    if not is_forward_context_available():
        return self.runnable(*args, **kwargs)

    forward_context = get_forward_context()
    batch_descriptor = forward_context.batch_descriptor
    cudagraph_runtime_mode = forward_context.cudagraph_runtime_mode

    # 2. 模式不匹配或为 NONE → 直接执行
    if (cudagraph_runtime_mode == CUDAGraphMode.NONE
        or cudagraph_runtime_mode != self.runtime_mode):
        return self.runnable(*args, **kwargs)

    # 3. 获取或创建 entry

```

```

if batch_descriptor not in self.concrete_cudagraph_entries:
    self.concrete_cudagraph_entries[batch_descriptor] = CUDAGraphEntry(...)

entry = self.concrete_cudagraph_entries[batch_descriptor]

# 4. 首次调用 → 捕获
if entry.cudagraph is None:
    return self._capture(entry, *args, **kwargs)

# 5. 后续调用 → 回放
entry.cudagraph.replay()
return entry.output

```

捕获流程 (cuda\_graph.py L265-L344):

```

# 关键捕获步骤:
with ExitStack() as stack:
    # 禁用 GC (非首个图时加速)
    if self.cudagraph_options.gc_disable:
        stack.enter_context(patch("gc.collect", lambda *a, **k: None))

    # 设置内存池
    set_graph_pool_id(self.graph_pool)

    # 同步 offloader
    get_offloader().sync_prev_onload()

    # 执行 CUDA Graph 捕获
    with torch.cuda.graph(cudagraph, pool=self.graph_pool, stream=current_stream()):
        output = self.runnable(*args, **kwargs)
        get_offloader().join_after_forward()
        if self.cudagraph_options.weak_ref_output:
            output = weak_ref_tensors(output) # 内存优化

entry.output = weak_ref_tensors(output)
entry.cudagraph = cudagraph

```

CUDAGraphOptions 配置项 (cuda\_graph.py L139-L143):

| 选项               | 类型   | 说明                      |
|------------------|------|-------------------------|
| debug_log_enable | bool | 是否打印捕获日志 (通常仅首图启用)      |
| gc_disable       | bool | 是否禁用 GC (非首图禁用以加速)      |
| weak_ref_output  | bool | 是否用弱引用保存输出 (末尾图启用以节省内存) |

### 3.3 Piecewise CUDAGraphs 一子图捕获 (推荐模式)

Piecewise 模式将计算图按 splitting ops 分割成多个子图, 每个子图独立捕获 CUDA Graph。

工作流程: 1. split\_graph() 按 splitting ops 将 FX 图切割为多个子模块 2. PiecewiseCompileInterpreter 遍历每个非分割子模块 3. 为每个子模块创建 PiecewiseBackend 4. PiecewiseBackend 对每个 compile\_range 调用 CompilerManager.compile() 5. 编译完成后用 CUDAGraphWrapper 包装

优势: - 更细粒度的 batch size 适配 - 不同子图可独立缓存 - 支持 dynamic shape (符号形状) - 内存效率更高 (逐层释放中间结果)

### 3.4 Full CUDAGraphs — 整体捕获

Full 模式将整个模型 forward 捕获为单个 CUDA Graph: - 适用于固定 batch size 场景 - 开销最低 (单次 kernel launch) - 与 Piecewise 模式可通过 FULL\_AND\_PIECEWISE 同时启用

### 3.5 Encoder CUDA Graphs — 视觉编码器图捕获

源码: encoder\_cudagraph.py · encoder\_cudagraph\_defs.py

**EncoderCudaGraphManager** 实现了基于 token budget 的视觉编码器 CUDA Graph 管理:

核心数据结构 (encoder\_cudagraph\_defs.py L11-L37):

```
@dataclass
class EncoderCudaGraphConfig:
    modalities: list[str]                # 支持的模态 ["image"]
    input_key_by_modality: dict[str, str] # {"image": "pixel_values"}
    buffer_keys: list[str]               # buffer 键列表
    out_hidden_size: int                 # 输出隐藏维度

@dataclass
class BudgetGraphMetadata:
    token_budget: int                    # token 预算
    max_batch_size: int                  # 最大批大小
    max_frames_per_batch: int            # 每批最大帧数
    graph: torch.cuda.CUDAGraph          # 捕获的图
    input_buffer: torch.Tensor            # 输入缓冲区
    metadata_buffers: dict[str, torch.Tensor] # 元数据缓冲区
    output_buffer: torch.Tensor           # 输出缓冲区
```

**Budget** 图生成策略 (encoder\_cudagraph.py L126-L137):

```
@staticmethod
def _generate_budgets(min_budget: int, max_budget: int) -> list[int]:
    """ 从 min_budget 到 max_budget 生成 2 的幂次预算 """
    budgets: list[int] = []
    b = min_budget
    while b <= max_budget:
        budgets.append(b)
        b *= 2
    if not budgets or budgets[-1] < max_budget:
        budgets.append(max_budget)
    return budgets
```

贪心打包执行 (encoder\_cudagraph.py L285-L412):

flowchart TD

```
A["获取输入 items"] --> B["按输出 token 数升序排序"]
B --> C{"当前 item 能放入  
当前 batch?"}
C -->|Yes| D["加入当前 batch"]
C -->|No| E["终结当前 batch  
寻找最小适配 budget"]
D --> F{还有更多?}
E --> G["新建 batch"]
G --> D
F -->|Yes| C
F -->|No| E
E --> H{"budget 存在?"}
H -->|Yes| I["CUDA Graph Replay"]
```

```

H -->|No| J["Eager Fallback"]
I --> K["Scatter 输出到原序"]
J --> K

```

**DP (Data Parallel)** 支持 (encoder\_cudagraph.py L414-L544):- `_dp_shard()`: 使用 `get_load_balance_assignment()` 按 input size 均衡分配到各 TP rank - `_dp_gather()`: `pad` → `tensor_model_parallel_all_gather` → `unpad` → `reorder`

#### 四、Partition Rules 图分割规则

源码: `partition_rules.py`

图分割决定在哪些算子处切割计算图, 是实现 Piecewise 编译的关键。

分割判断逻辑 (`partition_rules.py` L14-L38):

```

def should_split(node: torch.fx.Node, splitting_ops: list[str]) -> bool:
    if node.op != "call_function":
        return False

    target = node.target

    if isinstance(target, torch._ops.OpOverloadPacket):
        # 如 "aten::add"
        return target._qualified_op_name in splitting_ops

    if isinstance(target, torch._ops.OpOverload):
        packet_name = target.name() # "aten::add"
        op_overload_name = f"{packet_name}.{target._overloadname}" # "aten::add.default"
        return op_overload_name in splitting_ops or packet_name in splitting_ops

    return False

```

**Inductor** 分割规则上下文 (`partition_rules.py` L41-L75):

当 `use_inductor_graph_partition=True` 时, vLLM 将分割规则注册到 Inductor 内部的 `partitioner`:

```

@contextlib.contextmanager
def inductor_partition_rule_context(splitting_ops):
    saved_splitting_ops = list(torch._inductor.config.custom_should_partition_ops)
    torch._inductor.config.custom_should_partition_ops = splitting_ops
    try:
        yield
    finally:
        torch._inductor.config.custom_should_partition_ops = saved_splitting_ops

```

**split\_graph** 完整流程 (`backends.py` L548-L622):

1. `_decompose_size_nodes()` 一将 `x.size()` 分解为 `sym_size.int(x, dim)` 调用
2. 遍历所有节点, 按 `should_split()` 判断是否需要新子图
3. 连续 `splitting ops` 保持在一起
4. `_merge_empty_only_subgraphs()` 一合并仅含 `empty allocation` 的分区
5. 调用 `torch.fx.passes.split_module.split_module()` 执行实际分割



## 五、Codegen 代码生成

源码: codegen.py

Codegen 模块将分割后的 FX GraphModule 转化为高效的 Python 执行函数, 消除 `nn.Module.__call__` 和 `__getattr__` 开销。

生成流程:

```
# /workspace/vllm/compilation/codegen.py L130-L161
def generate_execution_code(split_gm) -> tuple[str, list[str], list[Any]]:
    code, submod_names, consts = generate_execution_code_with_name(
        split_gm, "execution_fn", with_submod=True
    )
    return "import torch\nimport operator\n" + code, submod_names, consts
```

生成的代码示例 (概念性):

```
import torch
import operator

def execution_fn(arg0, arg1, *, __vllm_submods__):
    # placeholder 节点直接映射到参数
    # call_module 节点通过 __vllm_submods__ 列表索引调用
    node_0 = __vllm_submods__[0](arg0, arg1)
    # call_function 节点直接调用
    node_1 = torch.ops.aten.add(node_0, arg1)
    # getitem 节点转为索引操作
    node_2 = node_1[0]
    # 中间变量及时释放 (del 优化)
    del node_0, node_1
    return node_2
```

关键优化 (codegen.py L42-L56):

- **Liveness** 分析: 跟踪每个值最后被消费的位置
- 提前 `del`: 在最后消费者之后立即释放不再需要的变量
- 常量提取: 非原始类型常量 (如 `torch.device`) 收集到 `__vllm_consts__` 列表
- 子模块内联: 纯 `GraphModule` 子模块直接内联到生成代码中

编译绑定 (codegen.py L164-L204):

```
def compile_execution_fn(code, submod_callables, submod_names, consts=None):
    namespace = {}
    if consts is not None:
        namespace["__vllm_consts__"] = consts
    exec(code, namespace)          # 执行生成的代码
    fn = namespace["execution_fn"] # 提取函数
    submods_list = [submod_callables.get(name) for name in submod_names]
    return partial(fn, __vllm_submods__=submods_list) # 绑定子模块
```

---

## 六、Caching 编译缓存

源码: caching.py

### 6.1 VllmSerializableFunction

支持 PyTorch precompile 协议的可序列化编译结果包装器:

序列化流程 (caching.py L252-L297):

```
@classmethod
def serialize_compile_artifacts(cls, compiled_fn) -> bytes:
    state = compiled_fn.__dict__.copy()
    state.pop("optimized_call")          # 不序列化可调用对象
    state.pop("shape_env")
    state.pop("vllm_backend", None)

    # 清理元数据减少体积
    for node in state["graph_module"].graph.nodes:
        node.meta.pop("source_fn_stack", None)
        node.meta.pop("nn_module_stack", None)

    # 张量输入替换为 meta 设备版本 (节省空间)
    state["example_inputs"] = pytree.tree_map_only(
        torch.Tensor, lambda inp: torch.empty_like(inp, device="meta"),
        state["example_inputs"],
    )

    # 收集 standalone compile artifacts
    if compiled_fn.vllm_backend:
        (standalone_compile_artifacts, ...) = \
            compiled_fn.vllm_backend.collect_standalone_compile_artifacts()
        state["standalone_compile_artifacts"] = standalone_compile_artifacts

    return pickle.dumps(state)
```

## 6.2 StandaloneCompiledArtifacts

基于内容寻址的去重编译产物存储 (caching.py L37-152):

```
class StandaloneCompiledArtifacts:
    """ 两级间接去重:
    1. submodule_bytes: "{submod_name}_{shape}" -> SHA256 hash
    2. submodule_bytes_store: SHA256 hash -> actual bytes
    """

    def insert(self, submod_name, shape, entry):
        hasher = hashlib.sha256()
        hasher.update(entry)
        hex_digest = hasher.hexdigest()
        self.submodule_bytes[f"{submod_name}_{shape}"] = hex_digest
        if hex_digest not in self.submodule_bytes_store:
            self.submodule_bytes_store[hex_digest] = entry
            # 新条目才存储, 已存在的 hash 复用
```

为什么需要去重? 因为相同的 transformer 层 (如 layer-0 和 layer-1 在相同 attention pattern 下分割) 会编译出完全相同的 artifact。

## 6.3 反序列化与重建

Mega Artifact 路径 (caching.py L411-562):

```
def reconstruct_serializable_fn_from_mega_artifact(state, artifacts, ...):
    # 1. 加载所有缓存的编译产物 (并发加载)
    artifacts.load_all()
```

```

# 2. 为每个子模块/shape 组合构建 callable 映射
for cache_key in artifacts.submodule_bytes:
    submod_name, shape_str = cache_key.rsplit("_", 1)
    compiled_callables[submod_name][shape_str] = artifacts.get_loaded(...)

# 3. 创建 PiecewiseBackend (使用预编译 runnable)
piecewise_backend = PiecewiseBackend(
    graph=None,                                # 不需要原图
    compiled_runnables=runnables,              # 使用缓存
    ...
)

# 4. 包装 CUDA Graph
wrapped = wrap_with_cudagraph_if_needed(piecewise_backend, ...)

```

## 6.4 缓存键因子

缓存目录由以下因子的 SHA-256 哈希决定 (backends.py L1026-1074):

| 因子            | 来源                               | 说明            |
|---------------|----------------------------------|---------------|
| env_hash      | envs.compile_factors()           | 环境变量 (PP 层数等) |
| config_hash   | VllmConfig.compute_hash()        | 模型配置          |
| code_hash     | traced files SHA-256             | 源码文件内容        |
| compiler_hash | CompilerInterface.compute_hash() | 编译器状态         |

## 七、Monitor 编译监控

源码: monitor.py

### 7.1 monitor\_torch\_compile

编译过程监控上下文管理器 (monitor.py L17-L59):

```

@contextmanager
def monitor_torch_compile(vllm_config, message="torch.compile took %.2f s"):
    global torch_compile_start_time
    torch_compile_start_time = time.perf_counter()

    # 启动 depyf 调试 dump (如果配置了路径)
    path = vllm_config.compile_debug_dump_path()
    if compilation_config.mode == CompilationMode.VLLM_COMPILE and path:
        import depyf
        depyf_cm = depyf.prepare_debug(path.as_posix())
        depyf_cm.__enter__()

    try:
        yield
    else:
        total = time.perf_counter() - torch_compile_start_time
        # 分别记录 encoder 和 backbone 编译时间
        if is_encoder:

```

```

        compilation_config.encoder_compilation_time += total
    else:
        compilation_config.compilation_time += total
    logger.info_once(message, total)
finally:
    if depyf_cm is not None:
        depyf_cm.__exit__(None, None, None)

```

## 7.2 CUDA Graph 捕获合法性验证

(monitor.py L87-103):

```

cudagraph_capturing_enabled: bool = True

def validate_cudagraph_capturing_enabled():
    global cudagraph_capturing_enabled
    if not cudagraph_capturing_enabled:
        raise RuntimeError(
            "CUDA graph capturing detected at an inappropriate time."
        )

def set_cudagraph_capturing_enabled(enabled: bool):
    global cudagraph_capturing_enabled
    cudagraph_capturing_enabled = enabled

```

这用于确保 CUDA Graph 捕获不会在不恰当的时刻发生（如在 eager fallback 路径中）。

## 八、Counter 性能计数

源码: counter.py

全局编译性能计数器，追踪完整的编译生命周期指标:

```

# /workspace/vllm/compilation/counter.py L12-L41
@dataclasses.dataclass
class CompilationCounter:
    num_models_seen: int = 0                # 见到的模型数
    num_graphs_seen: int = 0                # Dynamo 产生的图数量
    num_pieewise_graphs_seen: int = 0       # 分段后的子图数 (含分割 op)
    num_pieewise_capturable_graphs_seen: int = 0 # 可捕获的子图数
    num_backend_compilations: int = 0       # 后端编译次数
    num_gpu_runner_capture_triggers: int = 0 # CUDA Graph 捕获触发次数
    num_cudagraph_captured: int = 0         # 实际捕获的 CUDA Graph 数
    num_inductor_compiles: int = 0          # InductorAdaptor.compile 调用
    num_eager_compiles: int = 0             # EagerAdaptor.compile 调用
    num_cache_entries_updated: int = 0      # 缓存条目更新次数
    num_compiled_artifacts_saved: int = 0   # standalone 产物保存数
    num_compiled_artifacts_loaded: int = 0  # standalone 产物加载数
    num_aot_compiles: int = 0               # AOT 编译次数
    num_aot_artifacts_saved: int = 0        # AOT 产物保存数
    num_aot_artifacts_loaded: int = 0       # AOT 产物加载数
    stock_torch_compile_count: int = 0      # STOCK_TORCH_COMPILE 模式计数

```

断言测试接口 (counter.py L46-L55):

```

@contextmanager
def expect(self, **kwargs):
    old = self.clone()
    yield
    for k, v in kwargs.items():
        assert getattr(self, k) == getattr(old, k) == v, (
            f"{k} not as expected..."
        )

```

---

## 九、BaseStaticGraph 静态图基类

源码: base\_static\_graph.py

定义静态图包装器的 Protocol 接口, 使不同平台可以实现自己的 CUDA Graph / 静态图方案:

```

# /workspace/vllm/compilation/base_static_graph.py L10-57
class AbstractStaticGraphWrapper(Protocol):
    def __init__(
        self,
        runnable: Callable[..., Any],
        vllm_config: VllmConfig,
        runtime_mode: CUDAGraphMode,
        **kwargs: Any,
    ) -> None:
        """
        Args:
            runnable: 要捕获的可调用对象
            vllm_config: 全局 vLLM 配置
            runtime_mode: 静态图运行模式 (NONE/PIECEWISE/FULL)
        """
        raise NotImplementedError

    def __call__(self, *args, **kwargs) -> Any:
        """
        如果 ForwardContext 中的 runtime_mode 匹配,
        则 replay CUDA Graph 或首次 capture;
        否则直接调用原始 callable。
        """
        raise NotImplementedError

```

平台解析链: `current_platform.get_static_graph_wrapper_cls()` → `resolve_obj_by_qualname()` → 平台特定的 `CUDAGraphWrapper` 子类

---

## 十、PiecwiseBackend 分段后端

源码: piecwise\_backend.py

### 10.1 核心职责

`PiecwiseBackend` 是分段编译的核心执行引擎, 负责:

1. 多 **shape** 编译: 为每个 `compile_range` 独立编译
2. 运行时分发: 根据运行时 `shape` 选择对应的编译结果
3. 序列化支持: 导出编译产物供缓存

## 10.2 双模式初始化

```
# /workspace/vllm/compilation/piecewise_backend.py L86-192
class PiecewiseBackend:
    def __init__(self, graph, vllm_config, piecewise_compile_index, ...):
        # 断言: graph 和 compiled_runnables 二选一
        assert bool(graph is not None) ^ bool(compiled_runnables is not None)

        self.compile_ranges = self.compilation_config.get_compile_ranges()
        self.sym_shape_indices = sym_shape_indices

        # 构建 range_entries
        for range in self.compile_ranges:
            self.range_entries[range] = RangeEntry(compile_range=range)

        if self.graph is not None:
            self.compile_all_ranges()      # 冷启动: 编译所有 range
        else:
            self.load_all_ranges()         # 热启动: 加载预编译产物
```

## 10.3 Shape 分发

```
# /workspace/vllm/compilation/piecewise_backend.py L343-379
def __call__(self, *args):
    if self.sym_shape_indices:
        runtime_shape = args[self.sym_shape_indices[0]]
        range_entry = self._find_range_for_shape(runtime_shape)
        # 先精确匹配 compile_sizes, 再搜索包含该值的 compile_ranges
    else:
        # 全静态输入: 只有一个编译好的 range_entry
        range_entry = compiled_entries[0]

    return range_entry.runnable(*args)
```

## 10.4 具体参数创建

对于单 size 编译 (如 CUDA Graph capture sizes), 需要将 FakeTensor 的符号维度替换为具体值:

```
# /workspace/vllm/compilation/piecewise_backend.py L37-76
def create_concrete_args(graph, size):
    def concretize(sym_val):
        if not is_symbolic(sym_val):
            return int(sym_val)
        expr = sym_val.node.expr
        return int(expr.subs({s: size for s in expr.free_symbols}))

    fake_mode = FakeTensorMode(shape_env=ShapeEnv())
    with fake_mode:
        for node in graph.graph.nodes:
            if node.op != "placeholder": break
            val = node.meta["example_value"]
            if isinstance(val, torch.Tensor):
                new_shape = tuple(concretize(d) for d in val.shape)
                # 创建具有具体形状的占位张量
                t = torch.empty(new_shape, dtype=val.dtype, device=val.device)
```

```
t = t.as_strided(new_shape, new_strides, storage_offset)
args.append(t)
```

## 10.5 序列化

```
# /workspace/vllm/compilation/piecewise_backend.py L209-243
def to_bytes(self):
    class StandaloneCompiledArtifactsPickler(Pickler):
        def reducer_override(self, obj):
            if isinstance(obj, CachingAutotuner):
                obj.prepare_for_pickle()
                return pickle.loads, (pickle.dumps(obj),)
            return NotImplemented

    out = {}
    for range_key, entry in self.range_entries.items():
        if hasattr(entry.runnable, "serialize"):
            out[str(range_key)] = serialize(entry.runnable)
    return out
```

## 十一、Triton Utils Triton 编译工具

### 11.1 importing —Triton 可用性检测

源码: importing.py

安全检测 Triton 是否可用, 处理各种边界情况:

```
# /workspace/vllm/triton_utils/importing.py L13-72
HAS_TRITON = (
    find_spec("triton") is not None
    or find_spec("pytorch-triton-xpu") is not None
)

if HAS_TRITON:
    active_drivers = [
        x.driver for x in backends.values()
        if x.driver and x.driver.is_active()
    ]
    # 分布式环境容错 (Ray actor init 时 CUDA_VISIBLE_DEVICES 可能临时为空)
    if is_distributed_env and len(active_drivers) == 0:
        pass # 允许稍后激活
    elif not is_distributed_env and len(active_drivers) != 1:
        HAS_TRITON = False # 非分布式环境必须恰好 1 个 driver
```

**TritonPlaceholder** 一旦 Triton 不可用时提供的空壳替代 (importing.py L75-105):

```
class TritonPlaceholder(types.ModuleType):
    def __init__(self):
        super().__init__("triton")
        self.jit = self._dummy_decorator("jit") # 直接返回原函数
        self.autotune = self._dummy_decorator("autotune") # 同上
        self.heuristics = self._dummy_decorator("heuristics")
        self.Config = self._dummy_decorator("Config")
        self.language = TritonLanguagePlaceholder()
```

## 11.2 jit\_monitor —JIT 编译监控

源码: jit\_monitor.py

在 warmup 完成后监控 Triton kernel JIT 编译事件, 检测推理时的延迟尖峰:

激活 (jit\_monitor.py L32-57):

```
def activate():
    global _active
    if _active:
        return
    _active = True
    _setup_triton_autotuning_print() # 启用 autotuning 日志
    _setup_triton_jit_hook()       # 注册 JIT 编译钩子
```

JIT 编译钩子 (jit\_monitor.py L87-111):

```
def _setup_triton_jit_hook():
    from triton import knobs
    existing_hook = knobs.runtime.jit_post_compile_hook

    def _on_jit_compile(**kwargs):
        fn = kwargs.get("fn")
        fn_name = getattr(fn, "name", "<unknown>")
        logger.warning_once(
            "Triton kernel JIT compilation during inference: %s. "
            "This causes a latency spike...", fn_name,
        )
        if existing_hook is not None:
            return existing_hook(**kwargs)
        return None

    knobs.runtime.jit_post_compile_hook = _on_jit_compile
```

## 11.3 allocation —Triton 内存分配器

源码: allocation.py

设置 Triton 使用 PyTorch 的 CUDA 分配器:

```
# /workspace/vllm/triton_utils/allocation.py L9-L12
def set_triton_allocator(device: torch.device):
    def alloc_fn(size, alignment, stream):
        return torch.empty(size, device=device, dtype=torch.int8)
    triton.set_allocator(alloc_fn)
```

这确保 Triton kernel 的内存在 PyTorch CUDA 缓存分配器中统一管理。

---

## 十二、OptimizationLevel O0-O3 含义详解

源码: vllm.py L68-L265

### 12.1 各级别总览

```
graph TB
    subgraph O0["O0 —最快启动"]
        direction LR
```



```

    00_1[" 无编译"] -->
    00_2[" 无 CUDA Graph"] -->
    00_3[" 无 Pass 融合"] -->
    00_4[" 立即服务"]
end

subgraph 01["01 一快速优化"]
    direction LR
    01_1[" Dynamo+Inductor"] -->
    01_2[" Piecewise CUDAGraph"] -->
    01_3[" 部分 Fusion Pass"] -->
    01_4[" 无 SP/AllReduce"]
end

subgraph 02["02 一完全优化（默认）"]
    direction LR
    02_1[" 01 所有优化"] -->
    02_2[" Full CUDAGraph"] -->
    02_3[" Sequence Parallelism"] -->
    02_4[" AllReduce Fusion"] -->
    02_5[" Attention Quant Fusion"]
end

subgraph 03["03 一最大优化"]
    direction LR
    03_1[" 02 所有优化"] -->
    03_2[" FlashInfer Autotune"]
end

style 00 fill:#ffcd22,stroke:#c62828,color:#b71c1c
style 01 fill:#fff9c4,stroke:#f9a825,color:#f57f17
style 02 fill:#c8e6c9,stroke:#2e7d32,color:#1b5e20
style 03 fill:#b3e5fc,stroke:#1565c0,color:#0d47a1

```

## 12.2 详细配置对照表

| 配置项                       | O0   | O1                | O2                 | O3                 | 说明                     |
|---------------------------|------|-------------------|--------------------|--------------------|------------------------|
| <b>Compilation Mode</b>   | NONE | VLLM_COMPILE_MODE | VLLM_COMPILE_MODE  | VLLM_COMPILE_MODE  | 是否启用 torch.compile     |
| <b>CUDA Graph Mode</b>    | NONE | PIECEWISE         | FULL_AND_PIECEWISE | FULL_AND_PIECEWISE | CUDA Graph 捕获策略        |
| <b>fuse_norm_quant</b>    |      | 条件启用 *            | 条件启用 *             | 条件启用 *             | RMSNorm + 量化融合         |
| <b>fuse_act_quant</b>     |      | 条件启用 *            | 条件启用 *             | 条件启用 *             | SiLU+Mul + 量化融合        |
| <b>fuse_allreduce_rms</b> |      |                   | 条件启用 †             | 条件启用 †             | AllReduce + RMSNorm 融合 |
| <b>fuse_attn_quant</b>    |      |                   | IS_QUANTIZED       | IS_QUANTIZED       | Attention + 量化融合       |
| <b>enable_sp</b>          |      |                   | IS_DENSE           | IS_DENSE           | 序列并行 (Ring Attention)  |
| <b>fuse_gemm_comms</b>    |      |                   | IS_DENSE           | IS_DENSE           | GEMM + 通信融合            |

| 配置项                                       | O0 | O1     | O2     | O3     | 说明                           |
|-------------------------------------------|----|--------|--------|--------|------------------------------|
| <code>fuse_act_padding</code>             |    | 条件启用 ‡ | 条件启用 ‡ | 条件启用 ‡ | Add + RMSNorm + Pad 融合       |
| <code>fuse_mla_dual_rms_norm</code>       |    | 条件启用 § | 条件启用 § | 条件启用 § | MLA Dual RMSNorm 融合          |
| <code>fuse_rope_kvcache</code>            |    |        | 条件启用 ¶ | 条件启用 ¶ | RoPE + KV Cache 融合           |
| <b>FlashInfer Autotune</b>                |    |        |        |        | FlashInfer kernel autotuning |
| <code>use_inductor_graph_partition</code> |    |        |        |        | Inductor 内置图分割               |

\* 取决于 `enable_norm_fusion()` / `enable_act_fusion()`

† 取决于 `enable_allreduce_rms_fusion()` (TP>1 + Hopper/Blackwell + flashinfer)

‡ 取决于 `enable_norm_pad_fusion()` (AITER RMSNorm + hidden\_size==2880)

§ 取决于 `enable_mla_dual_rms_norm_fusion()` (ROCm AITER fused\_qk\_rmsnorm)

¶ 取决于 `enable_rope_kvcache_fusion()` (ROCm AITER + rotary\_embedding custom op)

### 12.3 条件启用函数详解

`enable_norm_fusion` (vllm.py L95-103):

```
def enable_norm_fusion(cfg):
    return (
        cfg.compilation_config.is_custom_op_enabled("rms_norm")
        or cfg.compilation_config.is_custom_op_enabled("quant_fp8")
        or cfg.kernel_config.ir_op_priority.rms_norm[0] != "native"
    )
```

当 RMSNorm 或 FP8 量化使用自定义算子（而非 native PyTorch）时启用。

`enable_act_fusion` (vllm.py L106-116):

```
def enable_act_fusion(cfg):
    return (
        cfg.compilation_config.is_custom_op_enabled("silu_and_mul")
        or cfg.compilation_config.is_custom_op_enabled("quant_fp8")
        or cfg.model_config.is_nvfp4_quantized() # FP4 始终 custom
    )
```

`enable_allreduce_rms_fusion` (vllm.py L119-139): - CUDA: 需要 TP > 1 + Hopper/Blackwell 架构 + flashinfer - ROCm: 需要 rocm\_aiter\_ops + TP > 1

### 12.4 默认值

`# /workspace/vllm/config/vllm.py L340`

`optimization_level: OptimizationLevel = OptimizationLevel.O2 # 默认 O2`

## 附录：编译流水线完整 Mermaid 图

### A.1 从模型到可执行函数的完整流程

flowchart TD

```
START["用户调用 <br/>torch.compile(model)"] --> DYNA["Dynamo 字节码分析 <br/> 追踪 Python 代码"]
DYNA --> FXGM["生成 fx.GraphModule<br/> 含 FakeTensor/SymInt"]
```

```

FXGM --> VB["VllmBackend.__call__()"]
VB --> CFG[" 配置 PostGradPassManager"]

CFG --> SPLIT{"use_inductor<br/>_graph_partition?"}
SPLIT -->|"No"| FX_SPLIT["split_graph()<br/>FX 级别分割"]
SPLIT -->|"Yes"| INDUCTOR_SPLIT["Inductor 内置分割"]

FX_SPLIT --> INTERP["PiecewiseCompileInterpreter<br/> 遍历分割子图"]
INDUCTOR_SPLIT --> INTERP

INTERP --> PBACKEND[" 创建 PiecewiseBackend<br/>per 子模块"]
PBACKEND --> COMPILE["CompilerManager.compile()<br/> 每个 compile_range"]

COMPILE --> INDUCTOR{"backend?"}
INDUCTOR -->|"inductor"| ICOMPILE["compile_fx()<br/> 或 standalone_compile()"]
INDUCTOR -->|"eager"| ECOMPILE[" 直接返回 graph"]
ICOMPILE --> CACHE_CHECK{" 缓存命中?"}
CACHE_CHECK -->|"Hit"| LOAD_CACHE[" 加载已编译产物"]
CACHE_CHECK -->|"Miss"| DO_COMPILE[" 执行 Inductor 编译 <br/>→ Triton kernels"]
DO_COMPILE --> SAVE_CACHE[" 保存到缓存"]
LOAD_CACHE --> WRAP_CG
SAVE_CACHE --> WRAP_CG

WRAP_CG["wrap_with_cudagraph_if_needed()<br/>CUDAWrapper"]
WRAP_CG --> CODEGEN["generate_execution_code()<br/>+ compile_execution_fn()"]
CODEGEN --> SERIALIZE["VllmSerializableFunction<br/> 支持序列化缓存"]

SERIALIZE --> READY[" 返回可调用函数"]

style START fill:#e8f5e9
style READY fill:#e8f5e9
style DYNA fill:#fff3e0
style FXGM fill:#fff3e0
style VB fill:#e3f2fd
style SPLIT fill:#fce4ec
style INTERP fill:#fce4ec
style COMPILE fill:#f3e5f5
style DO_COMPILE fill:#f3e5f5
style WRAP_CG fill:#e0f2f1
style CODEGEN fill:#e0f2f1

```

## A.2 CUDA Graph 捕获与回放详细流程

sequenceDiagram

```

participant Caller as 调用者
participant FW as ForwardContext
participant CW as CUDAWrapper
participant Pool as MemoryPool
participant Offloader as Offloader
participant Runnable as 编译后函数

```

```

Caller->>FW: 设置 batch_descriptor + runtime_mode
Caller->>CW: __call__(*args)

```

```

alt 首次调用 / 新 batch_descriptor
  CW->>CW: 创建 CUDAGraphEntry
  CW->>Offloader: sync_prev_onload()
  CW->>Pool: set_graph_pool_id()
  CW->>CW: torch.cuda.CUDAGraph()
  Note over CW,Runnable: === CAPTURE 阶段 ===
  CW->>Runnable: runnable(*args)
  Runnable-->>CW: output tensor(s)
  CW->>Offloader: join_after_forward()
  CW->>CW: weak_ref_tensors(output)
  CW->>CW: 保存 entry.cudagraph + entry.output
  CW-->>Caller: return output (强引用)
else 后续调用 / 相同 batch_descriptor
  CW->>Offloader: sync_prev_onload()
  Note over CW: === REPLAY 阶段 ===
  CW->>CW: entry.cudagraph.replay()
  CW-->>Caller: return entry.output (弱引用)
end

Note over Caller,CW: Debug 模式下验证 <br/>input data_ptr 一致性

```

### A.3 编译 Pass 执行数据流

```

flowchart LR
    INPUT["Inductor<br/>Functionalized Graph"] --> P1["Custom Passes<br/>(用户添加)"]
    P1 --> P2["NoOpEliminationPass"]
    P2 --> P3["SequenceParallelismPass"]
    P3 --> P4["AsyncTPPass"]
    P4 --> P5["AllReduceFusionPass"]
    P5 --> P6["RMSNormQuantFusionPass"]
    P6 --> P7["ActivationQuantFusionPass"]
    P7 --> P8["RopeKVCacheFusionPass"]
    P8 --> P9["AttnQuantFusionPass"]
    P9 --> PC1["PostCleanupPass #1"]
    PC1 --> IR["VllmIRLoweringPass"]
    IR --> CE["UnsafeCloneEliminationPass"]
    CE --> PC2["PostCleanupPass #2"]
    PC2 --> FF["FixFunctionalizationPass"]
    FF --> OUTPUT["优化后的 Graph<br/>送入 CodeGen"]

    style INPUT fill:#e3f2fd
    style OUTPUT fill:#e8f5e9
    style IR fill:#fff9c4
    style FF fill:#ffcdd2
    style PC2 fill:#ffcdd2

```

## vLLM 结构化输出约束分析文档

### 定位

本文档深入分析 vLLM 的结构化输出 (**Structured Output**) 系统架构与实现细节。结构化输出是 LLM 推理中的关键技术，通过在 token 生成阶段施加语法约束 (JSON Schema、正则表达式、Grammar 等)，确保模型输出符合预定义的结构规范。

```
graph TD
    A[StructuredOutputsConfig<br/> 配置层] --> B{backend 选择}
    B -->|auto/xgrammar| C[XgrammarBackend<br/> 高性能语法约束]
    B -->|outlines| D[OutlinesBackend<br/>FSM 约束引擎]
    B -->|lm-format-enforcer| E[LMFormatEnforcerBackend<br/> 轻量级格式强制]
    B -->|guidance| F[GuidanceBackend<br/>Guidance 库集成]

    C --> G[支持类型 <br/>JSON/Regex/Grammar/<br/>Choice/StructuralTag]
    D --> H[支持类型 <br/>JSON/Regex/Choice]
    E --> I[支持类型 <br/>JSON/Regex/Choice]
    F --> J[支持类型 <br/>JSON/Regex/Grammar/<br/>Choice/StructuralTag]

    style A fill:#c8e6c9,stroke:#388e3c,color:#1b5e20
    style B fill:#fff9c4,stroke:#f9a825,color:#f57f17
    style C fill:#bbdefb,stroke:#1976d2,color:#0d47a1
    style D fill:#bbdefb,stroke:#1976d2,color:#0d47a1
    style E fill:#bbdefb,stroke:#1976d2,color:#0d47a1
    style F fill:#bbdefb,stroke:#1976d2,color:#0d47a1
```

一、StructuredOutputsConfig 配置

源码位置

structured\_outputs.py

核心配置类

StructuredOutputsConfig 是引擎级别的结构化输出配置类，定义了后端选择和行为控制参数。

配置参数说明

| 参数名                           | 类型                                                                        | 默认值    | 说明                                                      |
|-------------------------------|---------------------------------------------------------------------------|--------|---------------------------------------------------------|
| backend                       | Literal["auto", "xgrammar", "guidance", "outlines", "lm-format-enforcer"] | "auto" | 选择结构化输出后端。<br>"auto" 会根据请求内容和库支持自动选择                    |
| disable_any_whitespace        | bool                                                                      | False  | 禁止 JSON 输出中的空白字符（紧凑格式）。<br>仅 xgrammar 和 guidance 支持     |
| disable_additional_properties | bool                                                                      | False  | guidance 后端不使用 additionalProperties。<br>仅 guidance 后端支持 |
| reasoning_parser              | str                                                                       | ""     | 推理内容解析器选择                                               |
| reasoning_parser_plugin       | str                                                                       | ""     | 动态加载的推理解析器插件路径                                          |
| enable_in_reasoning           | bool                                                                      | False  | 是否在推理阶段使用结构化输入                                          |

关键源码片段

- 1. 后端类型定义 (structured\_outputs.py:12-14)

```
StructuredOutputsBackend = Literal[
    "auto", "xgrammar", "guidance", "outlines", "lm-format-enforcer"
]
```

2. 配置验证逻辑 (structured\_outputs.py:62-73)

```
@model_validator(mode="after")
def _validate_structured_output_config(self) -> Self:
    if self.disable_any_whitespace and self.backend not in ("xgrammar", "guidance"):
        raise ValueError(
            "disable_any_whitespace is only supported for "
            "xgrammar and guidance backends."
        )
    if self.disable_additional_properties and self.backend != "guidance":
        raise ValueError(
            "disable_additional_properties is only supported "
            "for the guidance backend."
        )
    return self
```

3. Hash 计算 (structured\_outputs.py:44-60)

该 config 不影响计算图 (computation graph)，因此 hash 计算返回空因子列表：

```
def compute_hash(self) -> str:
    factors: list[Any] = []
    hash_str = safe_hash(str(factors).encode(), usedforsecurity=False).hexdigest()
    return hash_str
```

二、后端实现（四种后端逐一深入分析）

类型系统基础

在分析各后端之前，先理解抽象基类定义：

backend\_types.py

StructuredOutputOptions 枚举 (backend\_types.py:19-25)

```
class StructuredOutputOptions(enum.Enum):
    JSON = enum.auto()
    JSON_OBJECT = enum.auto()
    REGEX = enum.auto()
    GRAMMAR = enum.auto()
    CHOICE = enum.auto()
    STRUCTURAL_TAG = enum.auto()
```

StructuredOutputGrammar 抽象基类 (backend\_types.py:31-95) 请求级别的 Grammar 实现，必须实现以下方法：

| 方法                                | 功能说明                         |
|-----------------------------------|------------------------------|
| accept_tokens(request_id, tokens) | 接受 token 列表并推进 FSM，返回是否成功    |
| validate_tokens(tokens)           | 验证 token 是否被接受（不推进 FSM）      |
| rollback(num_tokens)              | 回滚指定数量的 token                |
| fill_bitmask(bitmask, idx)        | 填充 bitmask 用于 logits masking |

| 方法                           | 功能说明   |
|------------------------------|--------|
| <code>is_terminated()</code> | 检查是否终止 |
| <code>reset()</code>         | 重置状态   |

**StructuredOutputBackend** 抽象基类 (`backend_types.py:98-136`) 引擎级别的后端实现，必须实现：

| 方法                                                       | 功能说明                                   |
|----------------------------------------------------------|----------------------------------------|
| <code>compile_grammar(request_type, grammar_spec)</code> | 编译 grammar 规范为 StructuredOutputGrammar |
| <code>allocate_token_bitmask(max_num_seqs)</code>        | 分配 token bitmask 内存                    |
| <code>destroy()</code>                                   | 清理资源                                   |

2.1 XgrammarBackend - 高性能语法约束

`backend_xgrammar.py`

核心原理 **Xgrammar** 是 MLC-AI 开发的高性能语法约束库，基于 **LLAMA.cpp** 的 **GBNF** (Grammar-Based Normal Form) 语法格式。其核心优势在于：

- 1. 原生 **C++** 实现：编译和匹配性能极高
- 2. 直接 **Bitmask** 操作：通过 `fill_next_token_bitmask` 直接操作 GPU tensor
- 3. **Jump-forward** 解码：支持跳过确定性 token 加速生成
- 4. **Speculative Decoding** 支持：内置回滚机制

初始化流程 (`backend_xgrammar.py:36-75`)

```
@dataclass
class XgrammarBackend(StructuredOutputBackend):
    def __post_init__(self):
        self.disable_any_whitespace = (
            self.vllm_config.structured_outputs_config.disable_any_whitespace
        )

    # 特殊处理 Mistral tokenizer (Tekken 编码)
    if is_mistral_tokenizer(self.tokenizer):
        stop_token_ids = [self.tokenizer.eos_token_id]
        self.vocab_size = len(self.tokenizer.vocab)
        tokenizer_info = xgr.TokenizerInfo(
            encoded_vocab=self.tokenizer.vocab,
            vocab_type=xgr.VocabType.RAW if self.tokenizer.is_tekken
            else xgr.VocabType.BYTE_FALLBACK,
            vocab_size=self.vocab_size,
            stop_token_ids=stop_token_ids,
            add_prefix_space=True,
        )
    else:
        tokenizer_info = xgr.TokenizerInfo.from_huggingface(
            self.tokenizer, vocab_size=self.vocab_size
        )
```

```

# 创建 GrammarCompiler (带缓存)
self.compiler = xgr.GrammarCompiler(
    tokenizer_info,
    max_threads=8,
    cache_enabled=True,
    cache_limit_bytes=vllm.envs.VLLM_XGRAMMAR_CACHE_MB * 1024 * 1024,
)

# Speculative decoding 支持
self.num_speculative_tokens = 0
if self.vllm_config.speculative_config is not None:
    self.num_speculative_tokens = (
        self.vllm_config.speculative_config.num_speculative_tokens
    )

```

关键特性: - **TokenizerInfo** 构建: 区分 Mistral/Tekken tokenizer 和标准 HF tokenizer - **GrammarCompiler**  
 缓存: 避免重复编译相同 grammar - **Speculative tokens** 回滚: 设置 max\_rollback\_tokens

**Grammar** 编译逻辑 (backend\_xgrammar.py:77-122)

```

def compile_grammar(
    self, request_type: StructuredOutputOptions, grammar_spec: str
) -> StructuredOutputGrammar:
    if request_type == StructuredOutputOptions.JSON:
        ctx = self.compiler.compile_json_schema(
            grammar_spec, any_whitespace=not self.disable_any_whitespace
        )
    elif request_type == StructuredOutputOptions.JSON_OBJECT:
        ctx = self.compiler.compile_json_schema(
            '{"type": "object"}', any_whitespace=not self.disable_any_whitespace
        )
    elif request_type == StructuredOutputOptions.GRAMMAR:
        ctx = self.compiler.compile_grammar(grammar_spec)
    elif request_type == StructuredOutputOptions.REGEX:
        ctx = self.compiler.compile_regex(grammar_spec)
    elif request_type == StructuredOutputOptions.STRUCTURAL_TAG:
        # 处理 structural tag (工具调用场景)
        s_tag = json.loads(grammar_spec)
        if "structures" in s_tag:
            tags = [
                xgr.StructuralTagItem(
                    begin=s["begin"],
                    schema=json.dumps(s["schema"]),
                    end=s["end"],
                )
                for s in s_tag["structures"]
            ]
            ctx = self.compiler.compile_structural_tag(tags, s_tag["triggers"])
        else:
            ctx = self.compiler.compile_structural_tag(grammar_spec)
    else:
        raise ValueError(...)

    return XgrammarGrammar(
        matcher=xgr.GrammarMatcher(ctx, max_rollback_tokens=self.num_speculative_tokens),

```



```

        vocab_size=self.vocab_size,
        ctx=ctx,
    )

```

支持的编译类型映射:

| Request Type   | 编译方法                                                 | 说明                    |
|----------------|------------------------------------------------------|-----------------------|
| JSON           | <code>compile_json_schema()</code>                   | JSON Schema → Grammar |
| JSON_OBJECT    | <code>compile_json_schema({'type': 'object'})</code> | 任意 JSON 对象            |
| GRAMMAR        | <code>compile_grammar()</code>                       | EBNF/Grammar 字符串      |
| REGEX          | <code>compile_regex()</code>                         | 正则表达式 → Grammar       |
| STRUCTURAL_TAG | <code>compile_structural_tag()</code>                | 结构化标签 (工具调用)          |

### XgrammarGrammar - Token 级别操作 (backend\_xgrammar.py:131-199)

```

@dataclass
class XgrammarGrammar(StructuredOutputGrammar):
    vocab_size: int
    matcher: xgr.GrammarMatcher
    ctx: xgr.CompiledGrammar
    num_processed_tokens: int = field(default_factory=lambda: 0, ...)
    _is_terminated: bool = field(default=False, ...)

    def accept_tokens(self, request_id: str, tokens: list[int]) -> bool:
        """ 推进 FSM 状态机 """
        if self._is_terminated:
            return False
        for token in tokens:
            if not self.matcher.accept_token(token):
                logger.error("Failed to advance FSM for request %s", request_id)
                return False
            self.num_processed_tokens += 1
        self._is_terminated = self.matcher.is_terminated()
        return True

    def validate_tokens(self, tokens: list[int]) -> list[int]:
        """ 验证 token 但不推进状态 """
        accepted_tokens = []
        for token in tokens:
            if self.matcher.accept_token(token):
                accepted_tokens.append(token)
            else:
                break
        if len(accepted_tokens) > 0:
            self.matcher.rollback(len(accepted_tokens)) # 回滚到初始状态
        return accepted_tokens

    def rollback(self, num_tokens: int) -> None:
        """ 回滚状态 """

```

```

self.matcher.rollback(num_tokens)
self.num_processed_tokens -= num_tokens
self._is_terminated = self.matcher.is_terminated()

def fill_bitmask(self, bitmask: torch.Tensor, idx: int) -> None:
    """ 填充下一轮允许的 token bitmask """
    self.matcher.fill_next_token_bitmask(bitmask, idx)

```

核心机制: - **FSM** (有限状态机): 每个 token 推进状态机一步 - **Bitmask** 填充: 直接告诉 GPU 哪些 token 是合法的 - 回滚支持: 用于 speculative decoding 场景

验证函数 - 不支持的 **JSON Schema** 特性检测 (backend\_xgrammar.py:221-265)

```

STRING_SUPPORTED_FORMATS = {
    "email", "date", "time", "date-time", "duration",
    "ipv4", "ipv6", "hostname", "uuid", "uri",
    "uri-reference", "uri-template", "json-pointer", "relative-json-pointer"
}

def has_xgrammar_unsupported_json_features(schema: dict[str, Any]) -> bool:
    """ 递归检查 JSON schema 是否包含 xgrammar 不支持的特性 """

    def check_object(obj: dict[str, Any]) -> bool:
        # 1. 数值类型的 multipleOf 不支持
        if obj.get("type") in ("integer", "number") and ("multipleOf" in obj):
            return True

        # 2. 数组的 uniqueItems/contains/minContains/maxContains 不支持
        if obj.get("type") == "array" and any(
            key in obj for key in ("uniqueItems", "contains", "minContains", "maxContains")
        ):
            return True

        # 3. 字符串 format 仅支持特定集合
        if obj.get("type") == "string" and "format" in obj \
            and obj["format"] not in STRING_SUPPORTED_FORMATS:
            return True

        # 4. 对象的 patternProperties/propertyNames 不支持
        if obj.get("type") == "object" and any(
            key in obj for key in ("patternProperties", "propertyNames")
        ):
            return True

        # 5. 递归检查嵌套对象
        for value in obj.values():
            ...
        return False

    return check_object(schema)

```

## 2.2 OutlinesBackend - 基于 FSM 的约束引擎

backend\_outlines.py

核心原理 **Outlines** 库（现 `outlines_core`）的核心思想是：

1. **Regex** → **DFA** 转换：将正则表达式转换为确定性有限自动机（DFA）
2. **Vocabulary** 映射：将 DFA 状态映射到 token vocabulary
3. **Index** 构建：构建高效的查找索引加速 mask 计算
4. **Guide** 引擎：通过 Guide 对象管理状态转换

初始化流程（`backend_outlines.py:52-55`）

```
@dataclass
class OutlinesBackend(StructuredOutputBackend):
    def __post_init__(self):
        self.vocabulary = get_outlines_vocabulary(self.tokenizer)
        self.cache = get_outlines_cache() # LRU 或 disk cache
```

**Vocabulary** 构建过程（详见 `utils.py`）： - 从 tokenizer 提取词汇表 - 处理特殊 token（如 `<0xXX>` byte token） - 构建 `bytes` → `token_ids` 映射 - 计算 hash 用于缓存键

**Grammar** 编译逻辑（`backend_outlines.py:69-93`）

```
def compile_grammar(
    self, request_type: StructuredOutputOptions, grammar_spec: str
) -> StructuredOutputGrammar:
    if request_type == StructuredOutputOptions.JSON:
        regex = json_schema.build_regex_from_schema(grammar_spec)
    elif request_type == StructuredOutputOptions.REGEX:
        regex = grammar_spec
    elif request_type == StructuredOutputOptions.CHOICE:
        choices = ast.literal_eval(grammar_spec)
        choices = [regex_escape(c) for c in choices]
        regex = "(" + "|".join(choices) + ")"
    else:
        raise ValueError(...)

    index = self._compile_index(regex, self.vocabulary)
    max_rollback_tokens = (
        self.vllm_config.speculative_config.num_speculative_tokens
        if self.vllm_config.speculative_config is not None else 0
    )
    return OutlinesGrammar(
        vocab_size=self.vocab_size,
        guide=oc.Guide(index, max_rollback=max_rollback_tokens),
    )
```

关键特点： - **JSON Schema** → **Regex**：通过 `json_schema.build_regex_from_schema()` 转换 - **Choice** → **Regex**：将选项列表转换为 `(choice1|choice2|...)` 格式 - 不支持 **Grammar** 类型：Outlines 后端不支持原始 Grammar 规范

**Index** 编译与缓存（`backend_outlines.py:57-67`）

```
def _compile_index(self, regex_string: str, vocabulary: OutlinesVocabulary) -> oc.Index:
    cache_key = f"{vocabulary._hash}_{regex_string}"
    if cache_key in self.cache:
        return self.cache[cache_key]

    index = oc.Index(regex_string, vocabulary.inner)
```

```

self.cache[cache_key] = index
return index

```

缓存策略:- **LRU Cache**(默认):内存缓存,最多 128 条目 - **Disk Cache**(可选):通过 `VLLM_V1_USE_OUTLINES_CACHE=1` 启用

## OutlinesGrammar - Guide 状态管理 (backend\_outlines.py:107-164)

```

@dataclass
class OutlinesGrammar(StructuredOutputGrammar):
    vocab_size: int
    guide: oc.Guide
    num_processed_tokens: int = field(default_factory=lambda: 0, ...)
    _prev_finished: bool = field(default=False, ...) # 延迟 finished 信号

    def accept_tokens(self, request_id: str, tokens: list[int]) -> bool:
        """
        两阶段检查:
        1. accepts_tokens(): 检查当前 token 是否可接受
        2. advance(): 实际推进状态 (可能到达 dead state)
        """
        if self.guide.accepts_tokens(tokens):
            for t in tokens:
                self.guide.advance(t)
                self.num_processed_tokens += 1
            return True
        return False

    def is_terminated(self) -> bool:
        """ 延迟 finished 信号: DFA accept 后再等一轮让 EOS 可发出 """
        curr = self.guide.is_finished()
        prev = self._prev_finished
        self._prev_finished = curr
        return prev

    def fill_bitmask(self, bitmask: torch.Tensor, idx: int) -> None:
        """ 通过 Guide 写入 mask 到 GPU memory """
        mask = bitmask[idx]
        self.guide.write_mask_into(mask.data_ptr(), mask.numel(), mask.element_size())

```

特殊设计 - `_prev_finished` 延迟机制: - Outlines\_core 在 DFA 到达 accept 状态时立即标记 finished - 但 vLLM 需要在 finished 后还能发出 EOS token - 因此延迟一周期返回 terminated 信号

正则表达式验证 (backend\_outlines.py:299-330) Outlines 后端对正则表达式有严格限制:

```

def validate_regex_is_buildable(pattern: str) -> None:
    """
    验证正则表达式是否符合 regex-automata 的要求:
    1. 无 backreferences (反向引用)
    2. 无 look-around assertions (前瞻/后瞻断言)
    3. 无 Unicode word boundaries (\b, \B)
    4. 必须有 universal start state (无锚定前缀)
    """
    parsed = sre_parse.parse(pattern)
    _check_unsupported(parsed) # 检查不支持的特性

```

```

if _prefix_needs_context(parsed): # 检查是否有锚定前缀
    raise ValueError("Regex does not have a anchored universal start state...")

```

不支持的正则特性: | 特性 | 示例 | 原因 | |——|——|——| | Backreferences | \1, \k<name> | DFA 无法处理 | | Look-ahead | (?=...), (?!...) | 需要 context | | Look-behind | (?<=...), (?<!...) | 需要 context | | Word boundary | \b, \B | Unicode 边界复杂 |

---

## 2.3 LMFormatEnforcerBackend - 轻量级格式强制

backend\_lm\_format\_enforcer.py

核心原理 **lm-format-enforcer** 是一个轻量级的格式强制库，采用字符级别解析器（CharacterLevelParser）方式工作：

1. **Parser** 定义：定义字符级别的输出约束规则
2. **Token** 允许集计算：根据当前已生成的 prefix 计算下一个合法 token 集合
3. **TokenEnforcer**：封装 tokenizer 和 parser，提供高效查询接口

初始化流程 (backend\_lm\_format\_enforcer.py:94-98)

```

@dataclass
class LMFormatEnforcerBackend(StructuredOutputBackend):
    def __post_init__(self):
        # 使用 LRU cache 缓存 tokenizer_data 构建结果
        self.tokenizer_data = _cached_build_vllm_token_enforcer_tokenizer_data(
            self.tokenizer, self.vocab_size
        )

```

Tokenizer Data 构建 (cached):

```

@lru_cache
def _cached_build_vllm_token_enforcer_tokenizer_data(
    tokenizer: PreTrainedTokenizerBase, vocab_size: int
) -> "lmfe_vllm.TokenEnforcerTokenizerData":
    return lmfe_vllm.build_vllm_token_enforcer_tokenizer_data(
        tokenizer, use_bitmask=True, vocab_size=vocab_size
    )

```

Grammar 编译 - CharacterLevelParser 选择 (backend\_lm\_format\_enforcer.py:100-135)

```

def compile_grammar(
    self, request_type: StructuredOutputOptions, grammar_spec: str
) -> StructuredOutputGrammar:
    character_level_parser: lmformatenforcer.CharacterLevelParser

    if request_type == StructuredOutputOptions.JSON:
        spec_dict = json.loads(grammar_spec)
        character_level_parser = lmformatenforcer.JsonSchemaParser(spec_dict)
    elif request_type == StructuredOutputOptions.JSON_OBJECT:
        character_level_parser = lmformatenforcer.JsonSchemaParser(None)
    elif request_type == StructuredOutputOptions.REGEX:
        character_level_parser = lmformatenforcer.RegexParser(grammar_spec)
    elif request_type == StructuredOutputOptions.CHOICE:
        choices = ast.literal_eval(grammar_spec)
        character_level_parser = lmformatenforcer.UnionParser(

```

```

        [lmformatenforcer.StringParser(choice) for choice in choices]
    )
else:
    raise ValueError(...)

# 不支持 speculative decoding
if max_rollback_tokens > 0:
    raise ValueError(
        "LM Format Enforcer backend does not support speculative tokens"
    )

token_enforcer = lmformatenforcer.TokenEnforcer(
    tokenizer_data=self.tokenizer_data,
    parser=character_level_parser,
)
return LMFormatEnforcerGrammar(token_enforcer)

```

Parser 类型对应关系:

| Request Type | Parser 类                         | 说明             |
|--------------|----------------------------------|----------------|
| JSON         | JsonSchemaParser(schema_dict)    | JSON Schema 约束 |
| JSON_OBJECT  | JsonSchemaParser(None)           | 任意 JSON 对象     |
| REGEX        | RegexParser(pattern)             | 正则表达式约束        |
| CHOICE       | UnionParser([StringParser(...)]) | 多选一约束          |

LMFormatEnforcerGrammar - Prefix 追踪机制 (backend\_lm\_format\_enforcer.py:43-90)

```

@dataclass
class LMFormatEnforcerGrammar(StructuredOutputGrammar):
    token_enforcer: lmformatenforcer.TokenEnforcer
    current_tokens_prefix: list[int] = field(default_factory=list)

    def accept_tokens(self, request_id: str, tokens: list[int]) -> bool:
        """ 逐个检查 token 是否在允许集中 """
        original_len = len(self.current_tokens_prefix)
        for token in tokens:
            if not self.token_enforcer.get_allowed_tokens(
                self.current_tokens_prefix
            ).is_token_allowed(token):
                # 原子性操作: 失败时回滚部分更新
                del self.current_tokens_prefix[original_len:]
                return False
            self.current_tokens_prefix.append(token)
        return True

    def fill_bitmask(self, bitmask: torch.Tensor, batch_index: int) -> None:
        """ 获取当前 prefix 的允许 token 集合并写入 bitmask """
        allowed_tokens = self.token_enforcer.get_allowed_tokens(
            self.current_tokens_prefix
        )
        bitmask[batch_index] = allowed_tokens.allowed_tokens

    def is_terminated(self) -> bool:
        """ 当最后一个 token 是 EOS 时认为终止 """

```

```

    return (
        len(self.current_tokens_prefix) > 0
        and self.current_tokens_prefix[-1] == self.token_enforcer.eos_token_id
    )

```

核心特点: - **Prefix 追踪**: 维护已生成的 token 序列作为上下文 - 原子性操作: accept 失败时完整回滚 - 不支持 **Speculative Decoding**: 明确抛出异常

---

## 2.4 GuidanceBackend - Guidance 库集成

backend\_guidance.py

核心原理 **Guidance** (通过 llguidance 库集成) 是微软开发的约束生成框架, 特点包括:

1. 高级 **Grammar** 表示: 支持复杂的结构化标签 (Structural Tag)
2. **Flexible Whitespace**: 可配置空白字符处理
3. **Additional Properties** 控制: 精细控制 JSON Schema 行为
4. **LLMatcher**: 高性能的 token 级别匹配器

初始化流程 (backend\_guidance.py:87-101)

```

@dataclass
class GuidanceBackend(StructuredOutputBackend):
    def __post_init__(self):
        self.disable_any_whitespace = (
            self.vllm_config.structured_outputs_config.disable_any_whitespace
        )
        self.disable_additional_properties = (
            self.vllm_config.structured_outputs_config.disable_additional_properties
        )

        # 特殊处理 Mistral tokenizer
        if is_mistral_tokenizer(self.tokenizer):
            self.ll_tokenizer = self.tokenizer.llg_tokenizer
        else:
            self.ll_tokenizer = llguidance_hf.from_tokenizer(
                self.tokenizer, max(self.vocab_size, len(self.tokenizer))
            )

```

**Grammar** 序列化 - 统一入口 (backend\_guidance.py:219-285)

```

def serialize_guidance_grammar(
    request_type: StructuredOutputOptions,
    grammar_spec: str | dict[str, Any],
    disable_any_whitespace: bool = False,
    disable_additional_properties: bool = False,
) -> str:

    def _process_schema(grammar_spec) -> str:
        if disable_additional_properties:
            grammar_spec = process_for_additional_properties(grammar_spec)
        return llguidance.LLMatcher.grammar_from_json_schema(
            grammar_spec,
            defaults={"whitespace_flexible": not disable_any_whitespace},

```

```

    )

    if request_type == StructuredOutputOptions.JSON:
        return _process_schema(grammar_spec)
    elif request_type == StructuredOutputOptions.REGEX:
        tp = "regex"
    elif request_type == StructuredOutputOptions.GRAMMAR:
        tp = "grammar"
    elif request_type == StructuredOutputOptions.CHOICE:
        tp = "choice"
    elif request_type == StructuredOutputOptions.STRUCTURAL_TAG:
        # 处理结构化标签 (复杂工具调用场景)
        s_tag = json.loads(grammar_spec) if isinstance(grammar_spec, str) else grammar_spec
        triggers = s_tag["triggers"]
        tags = []
        for s in s_tag["structures"]:
            begin = s["begin"]
            trig = next((t for t in triggers if begin.startswith(t)), None)
            tags.append(llguidance.StructTag(
                trigger=trig,
                begin=s["begin"],
                grammar=_process_schema(s["schema"]),
                end=s["end"],
            ))
        return llguidance.StructTag.to_grammar(tags)

    return llguidance.grammar_from(tp, grammar_spec)

```

**Additional Properties** 处理 (backend\_guidance.py:35-46):

```

def _walk_json_for_additional_properties(data: object):
    """ 递归遍历 JSON schema, 为包含 properties 的对象添加 additionalProperties: false """
    if isinstance(data, dict):
        for value in data.values():
            _walk_json_for_additional_properties(value)
        if "additionalProperties" not in data and (
            "properties" in data or "patternProperties" in data
        ):
            data["additionalProperties"] = False
    elif isinstance(data, list):
        for item in data:
            _walk_json_for_additional_properties(item)

```

**GuidanceGrammar - LLMatcher 封装** (backend\_guidance.py:137-217)

```

@dataclass
class GuidanceGrammar(StructuredOutputGrammar):
    ll_matcher: llguidance.LLMatcher
    ll_tokenizer: llguidance.LLTokenizer
    vocab_size: int
    printed_error: bool = False
    terminated: bool = False
    rollback_lag: int = 0 # EOS 延迟计数器

    def accept_tokens(self, request_id: str, tokens: list[int]) -> bool:

```



```

# 检测 EOS token
if self.ll_tokenizer.eos_token in tokens:
    if self.ll_matcher.is_stopped() and not self.terminated:
        self.rollback_lag = 1 # 延迟终止信号
        self.terminated = True

    if self.ll_matcher.is_stopped():
        return True

# 消费 token 并推进解析器
r = self.ll_matcher.consume_tokens(tokens)
self.check_error()
return r

def fill_bitmask(self, bitmask: torch.Tensor, idx: int) -> None:
    """ 自动处理 stopped/error 状态下的 mask """
    llguidance_torch.fill_next_token_bitmask(self.ll_matcher, bitmask, idx)
    self.check_error()

def rollback(self, num_tokens: int) -> None:
    if num_tokens > 0:
        self.ll_matcher.rollback(num_tokens - self.rollback_lag)
        self.terminated = False
        self.rollback_lag = 0
        self.check_error()

```

特殊设计 - **rollback\_lag**: - 与 Outlines 类似, Guidance 也需要延迟终止信号以允许 EOS 发出 - 通过 rollback\_lag 计数器实现精确控制

不支持的 JSON Schema 特性 (backend\_guidance.py:48-71)

```

def has_guidance_unsupported_json_features(schema: dict[str, Any]) -> bool:
    def check_object(obj: dict[str, Any]) -> bool:
        # patternProperties 不被 llguidance 支持
        if "patternProperties" in obj:
            return True
        # 递归检查嵌套对象
        for value in obj.values():
            ...
        return False
    return check_object(schema)

```

### 三、支持的结构类型对比

#### 3.1 各后端支持矩阵

| 结构类型           | Xgrammar    | Outlines  | LM Format Enforcer | Guidance |
|----------------|-------------|-----------|--------------------|----------|
| JSON Schema    | 完整支持        | (转 Regex) |                    | 完整支持     |
| JSON Object    |             |           |                    |          |
| 正则表达式          |             | (有限制)     |                    |          |
| Grammar (EBNF) |             |           |                    |          |
| Choice         | (转 Grammar) | (转 Regex) |                    |          |
| Structural Tag |             |           |                    |          |

| 结构类型                 | Xgrammar | Outlines | LM Format Enforcer | Guidance |
|----------------------|----------|----------|--------------------|----------|
| Speculative Decoding |          |          |                    |          |
| Jump-forward         |          |          |                    | TODO     |

### 3.2 详细说明

**JSON Schema** 约束 用途: 确保输出符合 JSON Schema 定义的数据结构

示例:

```
sampling_params = SamplingParams(
    structured_outputs=StructuredOutputsParams(
        json='{"type": "object", "properties": {"name": {"type": "string"}, "age": {"type": "integer"}}'
    )
)
```

各后端差异: - **Xgrammar**: 直接编译 JSON Schema 为 Grammar, 性能最优 - **Outlines**: 先转换为正则表达式, 再构建 DFA - **LM Format Enforcer**: 使用 JsonSchemaParser 字符级解析 - **Guidance**: 通过 LLMatcher.grammar\_from\_json\_schema() 处理

正则表达式约束 用途: 通过正则表达式限制输出格式

示例:

```
sampling_params = SamplingParams(
    structured_outputs=StructuredOutputsParams(regex=r"\d{3}-\d{4}")
)
```

限制: - Outlines 不支持: backreference、look-around、word boundary - 其他后端基本无限制

**Grammar (EBNF/CFG)** 约束 用途: 使用形式语法定义复杂输出结构

示例:

```
ebnf_grammar = """
root ::= expression
expression ::= term ("+" term)*
term ::= factor ("*" factor)*
factor ::= number | "(" expression ")"
number ::= [0-9]+
"""

sampling_params = SamplingParams(
    structured_outputs=StructuredOutputsParams(grammar=ebnf_grammar)
)
```

**Lark EBNF** 转换 (utils.py:289-448):

Xgrammar 仅支持 EBNF 格式, 但用户可能提供 Lark 格式的 grammar。utils.py 提供 convert\_lark\_to\_ebnf() 函数进行转换:

```
def convert_lark_to_ebnf(grammar_str: str) -> str:
    """
    Lark 格式: rule: 'hello'
    EBNF 格式: root ::= rule\nrule ::= "hello"
```

主要转换:

1. 添加 root 规则指向第一个规则
2. 单引号 → 双引号

```

3. : → ::=
4. / alternatives 处理
"""

```

**Choice** 约束 用途: 从预定义选项中选择输出

示例:

```

sampling_params = SamplingParams(
    structured_outputs=StructuredOutputsParams(choice=["positive", "negative", "neutral"])
)

```

内部转换: - **Xgrammar**: 转换为 "root ::= "positive" | "negative" | "neutral" EBNF grammar - **Outlines**: 转换为 "(positive|negative|neutral)" 正则表达式 - **LM Format Enforcer**: 使用 Union-Parser([StringParser(...)]) - **Guidance**: 直接传递 choice 类型

工具调用格式约束 (**Structural Tag**) 用途: OpenAI 函数调用 / 工具调用的结构化输出

示例:

```

structural_tag = {
    "triggers": ["<function_call>"],
    "structures": [
        {
            "begin": "<function_call>",
            "schema": {"type": "object", "properties": {"name": {"type": "string"}}},
            "end": "</function_call>"
        }
    ]
}

```

支持情况:- **Xgrammar**: 通过 compile\_structural\_tag() 支持 - **Guidance**: 通过 StructTag.to\_grammar() 支持 - 其他后端: 不支持

## 四、Request 级别配置

request.py

**StructuredOutputRequest** 类

```

classDiagram
    class StructuredOutputRequest {
        +params: StructuredOutputsParams
        -_grammar: Future~StructuredOutputGrammar~ | StructuredOutputGrammar | None
        +reasoning_ended: bool | None
        +reasoning_parser_kwargs: dict | None
        +reasoner: ReasoningParser | None
        +from_sampling_params(sampling_params)$ StructuredOutputRequest | None
        +is_grammar_ready: bool
        +grammar: StructuredOutputGrammar | None
        +structured_output_key: StructuredOutputKey
    }

```

核心功能 (request.py:22-75)

```

@dataclasses.dataclass
class StructuredOutputRequest:
    params: StructuredOutputsParams
    _grammar: Future[StructuredOutputGrammar] | StructuredOutputGrammar | None = None
    reasoning_ended: bool | None = None
    reasoning_parser_kwargs: dict[str, Any] | None = None
    reasoner: "ReasoningParser | None" = None

    @staticmethod
    def from_sampling_params(
        sampling_params: SamplingParams | None,
    ) -> "StructuredOutputRequest | None":
        """ 从 SamplingParams 创建请求, 如果无约束则返回 None """
        if sampling_params is None:
            return None
        params = sampling_params.structured_outputs
        if not params or params.all_constraints_none():
            return None
        return StructuredOutputRequest(params=params)

    @property
    def is_grammar_ready(self) -> bool:
        """ 检查异步编译的 grammar 是否就绪 (100 s 超时) """
        return self._check_grammar_completion()

    @property
    def grammar(self) -> StructuredOutputGrammar | None:
        """ 获取已编译的 grammar (如果就绪) """
        completed = self._check_grammar_completion()
        return cast(StructuredOutputGrammar | None, self._grammar) if completed else None

```

关键设计 - 异步 **Grammar** 编译:- `_grammar` 可以是 `Future` 对象(异步编译中)或已完成的 `StructuredOutputGrammar`  
 - `is_grammar_ready` 属性非阻塞检查 (100 s 超时) - 支持并发编译多个 `grammar` 而不阻塞调度循环

### Structured Output Key 生成 (request.py:77-98)

```

def get_structured_output_key(params: StructuredOutputsParams) -> StructuredOutputKey:
    """ 根据优先级确定请求的类型和规范字符串 """
    if params.json is not None:
        if not isinstance(params.json, str):
            json_str = json.dumps(params.json)
        else:
            json_str = params.json
        return StructuredOutputOptions.JSON, json_str
    if params.json_object:
        return StructuredOutputOptions.JSON_OBJECT, ""
    if params.regex is not None:
        return StructuredOutputOptions.REGEX, params.regex
    if params.choice is not None:
        if not isinstance(params.choice, str):
            json_str = json.dumps(params.choice)
        else:
            json_str = params.choice
        return StructuredOutputOptions.CHOICE, json_str
    if params.grammar is not None:

```

```

        return StructuredOutputOptions.GRAMMAR, params.grammar
    if params.structural_tag is not None:
        return StructuredOutputOptions.STRUCTURAL_TAG, params.structural_tag
    raise ValueError("No valid structured output parameter found")

```

优先级顺序: JSON > JSON\_OBJECT > REGEX > CHOICE > GRAMMAR > STRUCTURAL\_TAG

---

## 五、工具函数与辅助模块

### 5.1 Utils 工具函数

utils.py

**apply\_grammar\_bitmask - Bitmask 应用 (utils.py:44-135)** 这是结构化输出的核心函数，负责将 grammar bitmask 应用到模型输出 logits 上:

```

def apply_grammar_bitmask(
    scheduler_output: SchedulerOutput,
    grammar_output: GrammarOutput,
    input_batch: InputBatch,
    logits: torch.Tensor,
) -> None:
    """
    应用 grammar bitmask 到模型输出 logits

    流程:
    1. 从 scheduler 获取 compacted bitmask
    2. 根据 batch index 重新排序 (因为 scheduler 和 runner 的顺序可能不同)
    3. 处理 speculative decode tokens 的 offset
    4. 将 numpy array 转换为 torch tensor 并复制到 GPU
    5. 调用 xgr.apply_token_bitmask_inplace() 修改 logits
    """

    # 1. 获取结构化输出请求的 batch indices
    struct_out_req_batch_indices: dict[str, int] = {}
    cumulative_offset = 0
    spec_tokens = scheduler_output.scheduled_spec_decode_tokens
    struct_out_req_ids = set(grammar_output.structured_output_request_ids)

    for batch_index, req_id in enumerate(input_batch.req_ids):
        logit_index = batch_index + cumulative_offset
        cumulative_offset += len(spec_tokens.get(req_id, ()))
        if req_id in struct_out_req_ids:
            struct_out_req_batch_indices[req_id] = logit_index

    # 2. 重新排序 bitmask 以匹配 batch 顺序
    sorted_bitmask = np.full(
        shape=(logits.shape[0], grammar_bitmask.shape[1]),
        fill_value=-1,
        dtype=grammar_bitmask.dtype,
    )
    # ... 排序逻辑 ...

    # 3. 应用 bitmask 到 logits

```

```

grammar_bitmask = torch.from_numpy(sorted_bitmask).to(logits.device, non_blocking=True)

if not logits.is_cpu:
    xgr.apply_token_bitmask_inplace(logits, grammar_bitmask, indices=index_tensor)
else:
    # CPU 情况需要 float32 转换
    if logits.dtype != torch.float32:
        logits_fp32 = logits.to(torch.float32)
        xgr.apply_token_bitmask_inplace(logits_fp32, grammar_bitmask, indices=indices)
        logits.copy_(logits_fp32.to(logits.dtype))
    else:
        xgr.apply_token_bitmask_inplace(logits, grammar_bitmask, indices=indices)

```

关键点: - **Batch reorder**: scheduler 和 runner 的请求顺序可能不同, 需要重新排序 - **Speculative offset**: speculative decode tokens 会增加 logit index 偏移 - **CPU fallback**: CPU 模式下需要 float32 转换 (老版本 xgrammar kernel 限制) - **Non-blocking copy**: 使用 non\_blocking=True 避免 CPU-GPU 同步

**Vocabulary 管理 (utils.py:138-286)** **OutlinesVocabulary** 包装类 (utils.py:138-151):

```

class OutlinesVocabulary:
    """ 包装 outlines_core.Vocabulary, 附带 hash 用于缓存 """

    def __init__(self, vocabulary: oc.Vocabulary) -> None:
        self.inner = vocabulary
        # 使用 SHA256 hash 作为缓存键 (避免 Python hash 的随机性)
        hex_str = hashlib.sha256(vocabulary.__repr__().encode("utf-8")).hexdigest()
        hash_int = int(hex_str, 16)
        self._hash = hash_int

```

**Reduced Vocabulary 构建 (utils.py:206-272)**:

```

def _reduced_vocabulary(tokenizer: TokenizerLike) -> dict[bytes, list[int]]:
    """
    创建精简词汇表: token bytes -> equivalent token ids 列表

    处理特殊情况:
    - 特殊 token (EOS, PAD 等) 排除
    - SPIECE_UNDERLINE 前缀 (Llama tokenizer) -> 添加空格
    - <0xFF> byte token (Llama tokenizer) -> 直接转换为字节
    - Unicode replacement 字符 (GPT2 tokenizer) -> 使用 bytes_to_unicode 映射
    """

    vocabulary: dict[bytes, list[int]] = {}
    for token, token_idx in tokenizer.get_vocab().items():
        if token in tokenizer.all_special_tokens:
            continue

        token_str = convert_token_to_string(token)
        if token_str:
            # 处理各种编码格式的 token
            token_bytes = encode_token_to_bytes(token_str, token)
            if token_idx != eos_token_id:
                vocabulary.setdefault(token_bytes, []).append(token_idx)
            else:
                empty_token_ids.append(token_idx)

    return vocabulary

```

Cache 管理 (utils.py:154-199):

```
def get_outlines_cache():
    """ 获取 Index 缓存实例 """
    cache_dir = get_outlines_cache_path()

    if envs.VLLM_V1_USE_OUTLINES_CACHE:
        # Disk cache (持久化, 适合生产环境但有磁盘空间风险)
        from diskcache import Cache
        logger.warning(
            "Enabling outlines cache. This is an unbounded on-disk "
            "cache. It may consume a lot of disk space..."
        )
        cache = Cache(cache_dir, eviction_policy="none", cull_limit=0)
        # 版本检查: 版本变化时清空缓存
        cached_version = cache.get("__version__", None)
        if cached_version != outlines_version:
            cache.clear()
            cache.set("__version__", outlines_version)
        return cache

    # Default: LRU Cache (内存缓存, 最多 128 条目)
    return LRUCache(maxsize=128)
```

Grammar 格式转换工具 (utils.py:289-458) Lark → EBNF 转换器:

```
def grammar_is_likely_lark(grammar_str: str) -> bool:
    """ 检测 grammar 是否使用 Lark 语法 (通过检查是否包含 '::~=') """
    for line in grammar_str.split("\n"):
        line = re.sub(r"(\#|//).*$", "", line).strip()
        if not line:
            continue
        if "::=" in line:
            return False # 已经是 EBNF 格式
    return True # 可能是 Lark 格式

def convert_lark_to_ebnf(grammar_str: str) -> str:
    """
    完整的 Lark → EBNF 转换实现

    转换规则:
    1. 第一个规则成为 root 规则
    2. '...' → "..." (引号转换)
    3. : → :: (定义符号)
    4. / alternatives 保持不变
    5. 注释 (# 和 //) 移除
    6. 验证所有引用的规则都已定义
    """
```

Choice → Grammar 转换 (utils.py:451-458):

```
def choice_as_grammar(choice: list[str]) -> str:
    """ 将 choice 列表转换为 EBNF grammar 字符串 """
    def escape_ebnf_string(s: str) -> str:
        return re.sub(r'(["\\])', r"\\1", s)
```

```

escaped_choices = (escape_ebnf_string(c) for c in choice)
grammar = "root ::= " + " | ".join(f'"{c}"' for c in escaped_choices)
return grammar

```

---

## 六、结构化输出处理流程

flowchart TD

```

A[用户请求 <br/>SamplingParams] --> B{是否有 <br/>structured_outputs?}
B -- 否 --> Z[正常采样]
B -- 是 --> C[StructuredOutputRequest.from_sampling_params]

C --> D[get_structured_output_key<br/> 确定约束类型]
D --> E{backend 选择}

E -- auto --> F[自动选择最佳后端]
E -- xgrammar --> G[XgrammarBackend]
E -- outlines --> H[OutlinesBackend]
E -- lm-format-enforcer --> I[LMFormatEnforcerBackend]
E -- guidance --> J[GuidanceBackend]

F --> G
G --> K[compile_grammar<br/> 编译 Grammar]
H --> K
I --> K
J --> K

K --> L[异步编译 <br/>Future or Sync]
L --> M{grammar 就绪?}
M -- 否 --> N[等待/跳过本轮]
M -- 是 --> O[Grammar.accept_tokens<br/> 推进 FSM]

O --> P[Grammar.fill_bitmask<br/> 生成 Bitmask]
P --> Q[apply_grammar_bitmask<br/> 应用到 Logits]
Q --> R[Logits Masking<br/> 过滤非法 Token]
R --> S[Sampling<br/> 从合法 Token 中采样]
S --> T{是否终止?}
T -- 否 --> O
T -- 是 --> U[返回结果]

```

```

style A fill:#e3f2fd,stroke:#1565c0,color:#0d47a1
style Z fill:#e8f5e9,stroke:#2e7d32,color:#1b5e20
style U fill:#fff3e0,stroke:#ef6c00,color:#e65100
style Q fill:#fce4ec,stroke:#c62828,color:#b71c1c

```

流程详解

### 1 请求初始化阶段

User Request → SamplingParams.structured\_outputs

↓

StructuredOutputRequest.from\_sampling\_params()

↓

提取 StructuredOutputsParams (json/regex/choice/grammar/structural\_tag)



↓  
get\_structured\_output\_key() → (StructuredOutputOptions, grammar\_spec)

## 2 Grammar 编译阶段

Backend.compile\_grammar(request\_type, grammar\_spec)

↓

Xgrammar: GrammarCompiler.compile\_xxx()  
→ GrammarMatcher (C++ FSM)

Outlines: build\_regex\_from\_schema() (if JSON)  
→ oc.Index (DFA)  
→ oc.Guide (state machine)

LMFE: JsonSchemaParser / RegexParser / ...  
→ TokenEnforcer

Guidance: serialize\_guidance\_grammar()  
→ LLMatcher

↓  
返回 StructuredOutputGrammar 实例

## 3 推理循环阶段 (每步解码)

Step 1: Grammar.accept\_tokens(new\_tokens)  
- 推进 FSM 状态  
- 验证 token 合法性

Step 2: Grammar.fill\_bitmask(bitmask, idx)  
- 填充下一轮合法 token mask

Step 3: apply\_grammar\_bitmask()  
- Batch reorder  
- GPU tensor 操作  
- xgr.apply\_token\_bitmask\_inplace()

Step 4: Logits Masking  
- 非法 token logit → -∞

Step 5: Sampling  
- 从合法 token 中概率采样

Step 6: Check termination  
- Grammar.is\_terminated()?  
- Yes → 返回结果  
- No → 继续 Step 1

## 4 Speculative Decoding 支持

Normal Path: accept\_tokens → fill\_bitmask → sample  
↓

Speculative: validate\_tokens (不推进状态)  
↓  
如果验证失败 → rollback(num\_tokens)  
↓  
重新生成 candidate tokens

七、后端选择策略与建议

Auto 模式选择逻辑

当 backend="auto" 时，vLLM 会根据以下启发式规则选择：

| 场景                  | 推荐后端                      | 原因                |
|---------------------|---------------------------|-------------------|
| JSON Schema + 高性能需求 | <b>xgrammar</b>           | 原生 C++，最快         |
| 正则表达式 + 简单模式        | <b>outlines</b>           | DFA 高效            |
| 快速原型开发              | <b>lm-format-enforcer</b> | 轻量级，易调试           |
| 复杂工具调用              | <b>guidance</b>           | Structural Tag 支持 |
| 需要最大兼容性             | <b>guidance</b>           | 支持最多种类            |

性能特征对比

| 维度      | Xgrammar           | Outlines     | LMFE | Guidance |
|---------|--------------------|--------------|------|----------|
| 编译速度    |                    |              |      |          |
| 运行时开销   |                    |              |      |          |
| 内存占用    | 低                  | 中 (Index 缓存) | 低    | 中        |
| GPU 利用率 | 最高 (native kernel) | 高            | 中    | 高        |
| 灵活性     | 中                  | 中            | 低    | 最高       |

兼容性注意事项

- 1. **Speculative Decoding**: LM Format Enforcer 不支持
- 2. **JSON Schema** 复杂度:
  - Xgrammar: 不支持 multipleOf、uniqueItems、patternProperties
  - Guidance: 不支持 patternProperties
- 3. 正则表达式限制: Outlines 不支持 look-around 和 backreference
- 4. **Grammar** 格式: Xgrammar 仅支持 EBNF (需转换 Lark 格式)

八、关键设计模式总结

8.1 抽象工厂模式

StructuredOutputBackend (ABC)  
XgrammarBackend  
OutlinesBackend  
LMFormatEnforcerBackend  
GuidanceBackend

- 每个 Backend 负责：
- compile\_grammar() → 返回具体的 Grammar 实现
  - allocate\_token\_bitmask() → 内存管理
  - destroy() → 资源清理

## 8.2 Strategy 模式

运行时可通过配置切换后端:

```
# engine 启动时
config = StructuredOutputsConfig(backend="xgrammar") # 或 "auto"
engine = LLMEngine(config=config)

# 请求时无需关心具体后端
sampling_params = SamplingParams(
    structured_outputs=StructuredOutputsParams(json=schema)
)
```

## 8.3 Observer 模式 (Bitmask 更新)

Grammar 作为 Observer 监控 token 生成过程: - `accept_tokens()`: 通知新 token - `fill_bitmask()`: 查询下一步约束 - `rollback()`: 响应 speculative decoding 回滚

## 8.4 Lazy Initialization & Async Compilation

```
# Grammar 可以是 Future (异步编译中)
_grammar: Future[StructuredOutputGrammar] | StructuredOutputGrammar | None

# 非阻塞检查
@property
def is_grammar_ready(self) -> bool:
    return self._check_grammar_completion() # 100 s timeout
```

这避免了首次请求时的冷启动延迟。

---

## 九、扩展指南

添加新的结构化输出后端

如果要添加自定义后端 (例如基于新库), 需要:

### 1. 继承 StructuredOutputBackend

```
@dataclass
class MyCustomBackend(StructuredOutputBackend):
    def compile_grammar(self, request_type, grammar_spec):
        # 实现编译逻辑
        pass

    def allocate_token_bitmask(self, max_num_seqs):
        # 分配内存
        pass

    def destroy(self):
        # 清理资源
        pass
```

### 2. 继承 StructuredOutputGrammar

```
@dataclass
class MyCustomGrammar(StructuredOutputGrammar):
    def accept_tokens(self, request_id, tokens):
```

```

# 推进状态机
pass

def fill_bitmask(self, bitmask, idx):
    # 填充 bitmask
    pass

# ... 实现其他必需方法

```

### 3. 注册到 `StructuredOutputsBackend` 类型

```

StructuredOutputsBackend = Literal[
    "auto", "xgrammar", "guidance", "outlines", "lm-format-enforcer", "my-custom"
]

```

### 4. 实现验证函数

```

def validate_my_custom_grammar(sampling_params: SamplingParams):
    # 验证请求参数是否被后端支持
    pass

```

## 十、常见问题排查

### 问题 1: `ValueError - Unsupported JSON Schema features`

错误信息:

The provided JSON schema contains features not supported by xgrammar.

解决方案: - 移除 `multipleOf`、`uniqueItems`、`contains`、`patternProperties` - 或者切换到 `guidance` 后端

### 问题 2: `Regex does not have universal start state`

错误信息:

Regex does not have a anchored universal start state...

原因: 使用了 `^` 锚定或 `look-around` 断言

解决方案: - 移除开头的 `^` 和 `$` - 替换 (`?=...`) 为普通匹配 - 切换到 `xgrammar` 或 `guidance` 后端 (对正则限制更少)

### 问题 3: `LM Format Enforcer` 不支持 `Speculative Decoding`

错误信息:

LM Format Enforcer backend does not support speculative tokens

解决方案: - 禁用 speculative decoding: `--speculative-model ""` - 或切换到 `xgrammar/outlines/guidance` 后端

### 问题 4: `Grammar` 编译超时

现象: 首请求延迟高

原因: Complex JSON Schema 或 Grammar 首次编译耗时

优化方案:- 启用 Xgrammar 缓存: `VLLM_XGRAMMAR_CACHE_MB=256` - 启用 Outlines disk cache: `VLLM_V1_USE_OUTLINES_CACHE=1`  
- 预热常用 schema

## 附录：核心数据结构关系图

```
classDiagram
class StructuredOutputsConfig {
    +backend: str
    +disable_any_whitespace: bool
    +disable_additional_properties: bool
    +compute_hash() str
}

class StructuredOutputBackend {
    <<abstract>>
    +vllm_config: VllmConfig
    +tokenizer: TokenizerLike
    +vocab_size: int
    +compile_grammar()* StructuredOutputGrammar
    +allocate_token_bitmask()* Tensor
    +destroy()*
}

class StructuredOutputGrammar {
    <<abstract>>
    +accept_tokens()* bool
    +validate_tokens()* list
    +rollback()*
    +fill_bitmask()*
    +is_terminated()* bool
    +reset()*
}

class StructuredOutputRequest {
    +params: StructuredOutputsParams
    -_grammar: Future | Grammar | None
    +from_sampling_params()* Request
    +is_grammar_ready: bool
    +grammar: Grammar | None
}

class StructuredOutputOptions {
    <<enumeration>>
    JSON
    JSON_OBJECT
    REGEX
    GRAMMAR
    CHOICE
    STRUCTURAL_TAG
}

class XgrammarBackend {
    +compiler: GrammarCompiler
    +num_speculative_tokens: int
}

class XgrammarGrammar {
```

```

    +matcher: GrammarMatcher
    +ctx: CompiledGrammar
}

class OutlinesBackend {
    +vocabulary: OutlinesVocabulary
    +cache: Cache
}

class OutlinesGrammar {
    +guide: Guide
    +_prev_finished: bool
}

class LMFormatEnforcerBackend {
    +tokenizer_data: TokenEnforcerTokenizerData
}

class LMFormatEnforcerGrammar {
    +token_enforcer: TokenEnforcer
    +current_tokens_prefix: list
}

class GuidanceBackend {
    +ll_tokenizer: LLTokenizer
}

class GuidanceGrammar {
    +ll_matcher: LLMatcher
    +rollback_lag: int
}

StructuredOutputsConfig <|-- Engine uses
StructuredOutputBackend <|-- XgrammarBackend
StructuredOutputBackend <|-- OutlinesBackend
StructuredOutputBackend <|-- LMFormatEnforcerBackend
StructuredOutputBackend <|-- GuidanceBackend
StructuredOutputGrammar <|-- XgrammarGrammar
StructuredOutputGrammar <|-- OutlinesGrammar
StructuredOutputGrammar <|-- LMFormatEnforcerGrammar
StructuredOutputGrammar <|-- GuidanceGrammar
XgrammarBackend --> XgrammarGrammar : creates
OutlinesBackend --> OutlinesGrammar : creates
LMFormatEnforcerBackend --> LMFormatEnforcerGrammar : creates
GuidanceBackend --> GuidanceGrammar : creates
StructuredOutputRequest o-- StructuredOutputGrammar : holds
StructuredOutputRequest --> StructuredOutputOptions : uses

```

---

参考资源

源码文件索引

| 文件            | 路径                                                                 | 功能          |
|---------------|--------------------------------------------------------------------|-------------|
| Config        | /workspace/vllm/config/structured_output.py                        | 配置输出        |
| Backend Types | /workspace/vllm/v1/structured_output/backend_types.py              | 抽象基类        |
| Xgrammar      | /workspace/vllm/v1/structured_output/backend_xgrammar.py           | Xgrammar 后端 |
| Outlines      | /workspace/vllm/v1/structured_output/backend_outlines.py           | Outlines 后端 |
| LMFE          | /workspace/vllm/v1/structured_output/backend_lm_format_enforcer.py | LMFE 后端     |
| Guidance      | /workspace/vllm/v1/structured_output/backend_guidance.py           | Guidance 后端 |
| Request       | /workspace/vllm/v1/structured_output/request.py                    | 请求级配置       |
| Utils         | /workspace/vllm/v1/structured_output/utils.py                      | 工具函数        |

外部依赖

- **xgrammar**: <https://github.com/mlc-ai/xgrammar>
- **outlines\_core**: <https://github.com/outlines-dev/outlines>
- **lm-format-enforcer**: <https://github.com/noamgat/lm-format-enforcer>
- **llguidance**: <https://github.com/guidance-ai/guidance>

文档版本: v1.0 / 基于 vLLM 最新源码分析 / 生成日期: 2026-05-10

vLLM 推测解码 (Speculative Decoding) 加速技术分析

定位: vLLM 推测解码 (Speculative Decoding) 是一种利用小型 Draft Model 快速生成候选 token, 再由大型 Target Model 验证接受的加速技术, 典型加速比 2-4x。

graph LR



```
style A fill:#ffccbc
style B fill:#ffccbc
style C fill:#ffccbc
style D fill:#ffccbc
style E fill:#ffccbc
```

一、推测解码原理

1.1 Draft Model + Target Model 架构

推测解码的核心思想是用小模型”猜”，大模型”验”：

- **Draft Model** (草稿模型)：轻量级模型，快速生成 K 个候选 token (num\_speculative\_tokens)

- **Target Model** (目标模型): 完整的大模型，一次性验证所有 K 个草稿 token
- **Rejection Sampling** (拒绝采样): 以概率接受/拒绝每个草稿 token

关键优势: Target Model 的前向传播只需执行 1 次 (验证所有 K 个 token)，而非普通自回归的 K 次。即使 Draft Model 的接受率不高，总体计算量仍可大幅降低。

### 1.2 Rejection Sampling 算法

标准拒绝采样流程如下:

输入: Draft tokens  $[x_1, x_2, \dots, x_K]$ , Draft 概率分布  $p$ , Target 概率分布  $q$   
 输出: 接受的 token 数量  $n$  ( $0 \leq n \leq K$ )

对  $i = 1, 2, \dots, K$ :

1. 从均匀分布  $U(0,1)$  中采样  $r$
2. 若  $r \leq \min(1, q(x_i) / p(x_i)) \rightarrow$  接受  $x_i$ , 继续检查下一个
3. 否则  $\rightarrow$  拒绝  $x_i$  及其后续所有 token  
 以  $q$  分布重新采样一个 token 作为最终输出

若全部 K 个 token 都被接受:  
 再以 Target Model 的分布采样第 K+1 个 bonus token

vLLM 支持两种 rejection sampling 方式 (见 speculative.py#L190-L208):

| 方法        | 说明                |
|-----------|-------------------|
| standard  | 标准概率拒绝采样          |
| synthetic | 合成拒绝采样, 使用预定义衰减概率 |

### 1.3 典型加速比 2-4x 分析

加速比的理论上界取决于:

$$\text{Speedup} = \frac{K + 1}{1 + \text{Target\_Cost}/\text{Draft\_Cost}}$$

其中  $K = \text{num\_speculative\_tokens}$ 。实际加速比受以下因素影响:

- 接受率 (**Acceptance Rate**): Draft Model 与 Target Model 输出分布越接近, 接受率越高
- **Draft Model** 计算开销: EAGLE/Medusa 几乎零额外开销; N-gram 无模型开销; 独立 Draft Model 有一定计算成本
- 并行度: Parallel drafting (DFlash、EAGLE parallel) 可一次生成所有草稿 token
- 典型值: 接受率 50%-70% 时, 加速比约 2-3x

### 1.4 与普通自回归解码对比

| 维度                | 普通自回归 | 推测解码                      |
|-------------------|-------|---------------------------|
| 每 step 生成 token 数 | 1     | 1~K+1                     |
| 大模型前向次数/step      | 1     | 1                         |
| 小模型前向次数/step      | 0     | 1~K (或 0 for N-gram)      |
| 内存开销              | 基线    | +Draft Model 参数 + 额外 slot |
| 延迟确定性             | 确定    | 不确定 (依赖接受率)               |



## 二、SpeculativeConfig 配置

配置类位于 `speculative.py`，是所有推测解码方法的统一入口。

### 2.1 核心参数

```
# speculative.py L73-L107
@config
class SpeculativeConfig:
    # --- 基本控制 ---
    num_speculative_tokens: int = Field(default=None, gt=0)
    """ 推测 token 数量, 决定每次推测的最大候选数 """
    model: str | None = None
    """Draft model 名称或方法标识"""
    method: SpeculativeMethod | None = None
    """ 推测方法: "ngram"/"medusa"/"mlp_speculator"/"draft_model"/"suffix"
        /"eagle"/"eagle3"/"dflash"/"mtp"/"ngram_gpu"/..." """

    # --- Draft Model 配置 ---
    draft_tensor_parallel_size: int | None = Field(default=None, ge=1)
    """Draft model 的 TP 大小, 只能为 1 或与 target 相同"""
    quantization: me_quant.QuantizationMethods | str | None = None
    """Draft model 量化方式"""
    max_model_len: int | None = Field(default=None, ge=1)
    """Draft model 最大序列长度"""
```

### 2.2 N-gram 配置

```
# speculative.py L139-L144
prompt_lookup_max: int | None = Field(default=None, ge=1)
"""N-gram 最大窗口大小"""
prompt_lookup_min: int | None = Field(default=None, ge=1)
"""N-gram 最小窗口大小, 默认等于 prompt_lookup_max"""
```

### 2.3 Suffix Decoding 配置

```
# speculative.py L166-L184
suffix_decoding_max_tree_depth: int = 24
""" 后缀树最大深度 """
suffix_decoding_max_cached_requests: int = 10000
""" 最大缓存请求数 """
suffix_decoding_max_spec_factor: float = 1.0
""" 推测因子: max_spec_tokens = spec_factor * prefix_match_length """
suffix_decoding_min_token_prob: float = 0.1
""" 最小 token 概率阈值 """
```

### 2.4 Rejection Sampling 配置

```
# speculative.py L190-L261
rejection_sample_method: RejectionSampleMethod = "standard"
"""'standard': 概率拒绝采样; 'synthetic': 合成拒绝采样"""
draft_sample_method: DraftSampleMethod = "greedy"
"""'greedy': argmax; 'gumbel': Gumbel 噪声随机采样"""
synthetic_acceptance_rates: list[float] | None = None
""" 合成采样的逐位置无条件接受率 (单调非递增) """
synthetic_acceptance_length: float | None = None
```

```

""" 目标平均接受长度, 内部转换为 rates """
parallel_drafting: bool = False
""" 是否启用并行草稿 (一次 forward 生成所有 token) """

```

## 2.5 支持的方法类型汇总

```

# speculative.py L34-L67
MTPModelTypes = Literal[
    "deepseek_mtp", "mimo_mtp", "mimo_v2_mtp", "glm4_moe_mtp",
    "glm4_moe_lite_mtp", "glm_ocr_mtp", "ernie_mtp",
    "nemotron_h_mtp", "exaone_moe_mtp", "exaone4_5_mtp",
    "qwen3_next_mtp", "qwen3_5_mtp", "longcat_flash_mtp",
    "mtp", "pangu_ultra_moe_mtp", "step3p5_mtp",
    "hy_v3_mtp", "gemma4_mtp",
]

SpeculativeMethod = Literal[
    "ngram", "medusa", "mlp_speculator", "draft_model",
    "suffix", EagleModelTypes, NgramGPUPUTypes,
]
# EagleModelTypes 包含: "eagle", "eagle3", "extract_hidden_states",
# 所有 MTPModelTypes, "dflash"
# NgramGPUPUTypes = "ngram_gpu"

```

## 2.6 自动检测逻辑

在 `__post_init__` 中, vLLM 会根据 `model` 名称自动推断 `method` 类型:

- 含 "eagle-" → eagle
- 含 "eagle3" → eagle3
- 含 "dflash" → dflash
- `model_type == "medusa"` → medusa
- `model_type == "mlp_speculator"` → mlp\_speculator
- 属于 `MTPModelTypes` → mtp
- 默认 → draft\_model

## 三、Draft Model 接口体系

### 3.1 抽象基类: SpecDecodeBaseProposer

核心基类位于 `llm_base_proposer.py`, 是所有基于模型的推测方法的父类。

```

# llm_base_proposer.py L55-L62
class SpecDecodeBaseProposer:
    def __init__(
        self,
        vllm_config: VllmConfig,
        device: torch.device,
        pass_hidden_states_to_model: bool, # 是否传递 hidden states 给 draft model
        runner=None,
    ):

```

关键成员变量:

| 变量                          | 类型                                  | 说明                                                        |
|-----------------------------|-------------------------------------|-----------------------------------------------------------|
| num_speculative_tokens      | int                                 | 每次 step 生成的草稿 token 数                                     |
| pass_hidden_states_to_model | bool                                | 是否将 target hidden states 传入 draft model (EAGLE/DFlash 需要) |
| parallel_drafting           | bool                                | 是否使用并行草稿模式                                                |
| constant_draft_positions    | bool                                | 草稿位置是否恒定 (Gemma4 使用)                                      |
| hidden_states               | Tensor[max_num_tokens, hidden_size] | 目标模型 hidden states 缓冲区                                    |
| input_ids                   | Tensor[max_num_tokens]              | 草稿模型 input IDs 缓冲区                                        |
| model                       | nn.Module                           | 实际加载的草稿模型                                                 |

### 3.2 核心 propose() 方法

propose() 是推测生成的入口，完整流程:

```
sequenceDiagram
    participant T as Target Model
    participant P as Proposer
    participant D as Draft Model

    T->>P: target_token_ids, target_positions, target_hidden_states
    P->>P: set_inputs_first_pass() 构建草稿输入
    P->>P: build_per_group_and_layer_attn_metadata()
    P->>D: 第一次 forward (基于 target hidden states)
    D-->>P: last_hidden_states, hidden_states
    P->>P: _greedy_sample() → 第 1 个草稿 token

    loop 后续 K-1 个 token (if not parallel_drafting)
        P->>D: 自回归 forward (input_ids= 上一轮草稿 token)
        D-->>P: 新的 hidden_states
        P->>P: _greedy_sample() → 下一个草稿 token
    end

    P-->>T: draft_token_ids [batch_size, K]
```

关键实现细节:

1. 第一次 **forward** (L427-L467): 接收 target model 的 hidden states 作为输入，生成第一个草稿 token
2. 自回归循环 (L500-L591): 对于 num\_speculative\_tokens > 1 且非并行模式，逐个生成剩余草稿 token
3. 位置更新 (\_update\_positions\_dependent\_metadata()): 每步更新 positions 和 slot\_mapping
4. **Greedy sampling** (\_greedy\_sample()): 默认 argmax, 支持 local argmax reduction 优化

### 3.3 set\_inputs\_first\_pass() 输入构建

set\_inputs\_first\_pass() 负责 target model 输出到 draft model 输入的转换:

- 简单模式 (needs\_extra\_input\_slots=False): 直接 shift input\_ids, 在末尾插入 next\_token\_id
- 扩展模式(needs\_extra\_input\_slots=True):调用 Triton kernel copy\_and\_expand\_eagle\_inputs\_kernel 进行高效数据重排

### 3.4 权重共享机制

\_maybe\_share\_embeddings() 和 \_maybe\_share\_lm\_head() 实现 draft model 与 target model 的权重共享:

- **Embedding** 共享: EAGLE model 通常无自己的 embed\_tokens, 复用 target model

- **LM Head** 共享: MTP model 复用 target model 的 lm\_head
- 条件: 仅在 PP size == 1 时允许共享

### 3.5 DraftModelProposer

DraftModelProposer 是使用独立小模型作为 drafter 的具体实现:

```
# draft_model.py L17-L31
class DraftModelProposer(SpecDecodeBaseProposer):
    def __init__(self, vllm_config, device, runner=None):
        super().__init__(
            vllm_config=vllm_config,
            device=device,
            pass_hidden_states_to_model=False, # 不需要 hidden states
            runner=runner,
        )
        self._raise_if_vocab_size_mismatch() # 校验 vocab 一致性
        self._raise_if_draft_tp_mismatch()   # 校验 TP 一致性
```

特点: - pass\_hidden\_states\_to\_model=False: 独立 draft model 不需要 target hidden states - 强制要求 vocab\_size 与 target model 一致 - 要求 TP size 与 target model 一致 (避免 compile cache 冲突)

## 四、推测解码方法详解

### 4.1 EAGLE Early Exit Layer

原理 EAGLE (Early Exit of Large Language Models via Grouped Attention) 通过在 Transformer 中间层添加轻量的 预测头 (**prediction head**) 来提前预测下一个 token。相比运行完整的 Transformer, 只运行部分层即可输出预测, 大幅降低计算量。

EagleProposer 继承自 SpecDecodeBaseProposer:

```
# eagle.py L10-L22
class EagleProposer(SpecDecodeBaseProposer):
    def __init__(self, vllm_config, device, runner=None):
        super().__init__(
            vllm_config,
            device,
            pass_hidden_states_to_model=True, # 关键! 需要 target hidden states
            runner=runner,
        )
```

核心特征: - pass\_hidden\_states\_to\_model=True: 将 target model 的中间层 hidden states 直接传入 EAGLE head - EAGLE head 本质是一个小型 MLP (或 Attention 层), 接收 hidden states 并预测下一个 token - 支持 EAGLE (单辅助层)、EAGLE3 (多辅助层 + aux hidden states) 两种变体

**EAGLE3** 特殊处理 (llm\_base\_proposer.py#L413-L425):

```
if self.method in ("eagle3", "dflash"):
    assert isinstance(self.model, (
        Eagle3LlamaForCausalLM,
        Eagle3DeepseekV2ForCausalLM,
        DFlashQwen3ForCausalLM,
    ))
    # EAGLE3: 将多个辅助层的 hidden states 组合
    target_hidden_states = self.model.combine_hidden_states(target_hidden_states)
```

## 工作流程

1. Target model 运行到某个中间层时, 提取 hidden states
  2. Hidden states 传入 EAGLE prediction head
  3. EAGLE head 输出 logits, argmax 得到草稿 token
  4. 对后续 token, EAGLE head 在自身 hidden states 上继续自回归
- 

## 4.2 Medusa 多头预测

原理 Medusa 通过在同一 hidden state 上并联 多个独立的预测头 (**heads**), 每个头负责预测不同偏移位置的 token。一次 forward 即可同时获得 K 个草稿 token。

Medusa 模型 结构:

```
graph TD
    HS[Hidden States] --> B0[ResidualBlock 0]
    HS --> B1[ResidualBlock 1]
    HS --> BN[ResidualBlock N-1]
    B0 --> H0[LM Head 0<br/> 预测 t+1]
    B1 --> H1[LM Head 1<br/> 预测 t+2]
    BN --> HN[LM Head N-1<br/> 预测 t+N]

    style HS fill:#ffccbc
    style B0 fill:#e8f5e9
    style B1 fill:#e8f5e9
    style BN fill:#e8f5e9
```

### ResidualBlock 设计

*# medusa.py L19-L38*

```
class ResidualBlock(nn.Module):
    def __init__(self, config, hidden_size, num_layers):
        self.layers = nn.ModuleList([
            nn.Linear(hidden_size, hidden_size, bias=...)
            for _ in range(num_layers)
        ])
        self.act = nn.SiLU()

    def forward(self, x):
        for layer in self.layers:
            x = x + self.act(layer(x))
        return x
```

残差连接 + SiLU 激活, 多层堆叠形成轻量预测器。

### Medusa 主类

*# medusa.py L41-L96*

```
class Medusa(nn.Module):
    def __init__(self, *, vllm_config, prefix=""):
        config = vllm_config.speculative_config.draft_model_config.hf_config
        # num_heads 个并行的 ResidualBlock
        self.blocks = nn.ModuleList([
            ResidualBlock(config=config, hidden_size=config.hidden_size,
                           num_layers=config.num_hidden_layers)
```

```

        for _ in range(config.num_heads)
    ])
    # 每个 block 对应一个 LM Head
    self.lm_heads = nn.ModuleList([
        ParallelLMHead(config.vocab_size, config.hidden_size, ...)
        for i in range(config.num_heads)
    ])

```

forward 流程:

```

# medusa.py L105-L137
def forward(self, hidden_states):
    return [block(hidden_states) for block in self.blocks]

def compute_logits(self, hidden_states):
    logits_lst = []
    for hs, lm_head in zip(hidden_states, self.lm_heads):
        _logits = self.logits_processor(lm_head, hs)
        if self.token_map is not None:
            # 截断词汇表优化: 仅在高频 token 上预测
            ...
        logits_lst.append(_logits)
    return logits_lst

```

截断词汇表优化: 支持 `token_map` 将预测限制在最频繁的 `k` 个 token 上, 减少采样开销而不显著降低接受率。

#### 4.3 N-gram Proposer (CPU 版)

原理 N-gram proposer 是一种无模型的推测方法。它利用输入序列中已出现的 n-gram 模式来推测接下来的 token。如果上下文中出现过相同的 n-gram 序列, 则复制其后续 token 作为草稿。

优势: 零模型开销, 纯统计匹配, 适合重复性高的文本生成场景。

NgramProposer 实现:

```

# ngram_proposer.py L12-L33
class NgramProposer:
    def __init__(self, vllm_config):
        self.min_n = vllm_config.speculative_config.prompt_lookup_min # 最小 n-gram
        self.max_n = vllm_config.speculative_config.prompt_lookup_max # 最大 n-gram
        self.k = vllm_config.speculative_config.num_speculative_tokens # 草稿数量
        self.max_model_len = vllm_config.model_config.max_model_len

```

核心算法: **LPS** 最长匹配 N-gram 匹配使用 **LPS** (Longest Prefix which is also a Suffix) 算法, 即 KMP 算法的预处理阶段思想:

```

# ngram_proposer.py L199-L285
@jit(nopython=True)
def _find_longest_matched_ngram_and_propose_tokens(
    origin_tokens, min_ngram, max_ngram, max_model_len, k
) -> np.ndarray:
    """
    在 origin_tokens 后缀中查找最长的 [min_ngram, max_ngram] 范围内的 n-gram,
    该 n-gram 在之前的位置出现过。找到后提取其后的 k 个 token。
    """
    total_token = origin_tokens.shape[0]

```

```

if total_token < min_ngram:
    return np.empty((0,))

# 反转 token 序列, 将后缀匹配问题转为前缀匹配问题
tokens = origin_tokens[::-1]

lps = np.zeros(max_ngram, dtype=np.int32)
longest_ngram = 0
position = 0
prev_lps = 0
i = 1
while i < total_token:
    if tokens[prev_lps] == tokens[i]:
        prev_lps += 1
        if prev_lps >= longest_ngram:
            longest_ngram = prev_lps
            position = i
        if i < max_ngram:
            lps[i] = prev_lps
        if prev_lps == max_ngram:
            prev_lps = lps[max_ngram - 1]
        i += 1
    elif prev_lps != 0:
        prev_lps = lps[prev_lps - 1]
    else:
        i += 1

if longest_ngram < min_ngram:
    return np.empty((0,))

# 提取匹配 n-gram 之后的 k 个 token
start_position = total_token - 1 - position + longest_ngram
return origin_tokens[start_position : start_position + k]

```

算法要点: 1. 反转序列 → 后缀匹配变前缀匹配 2. 利用 LPS 数组在线性时间内找到最长重复 n-gram 3. Numba JIT 编译加速 (@njit(parallel=True) 用于 batch 版本)

**Batch 处理** batch\_propose() 使用 Numba 并行加速批量请求:

```

# ngram_proposer.py L169-L195
@njit(parallel=True)
def batch_propose_numba(valid_ngram_requests, num_tokens_no_spec,
                        token_ids_cpu, min_n, max_n, ...):
    for i in prange(len(valid_ngram_requests)):
        idx = valid_ngram_requests[i]
        context_token_ids = token_ids_cpu[idx, :num_tokens_no_spec[idx]]
        drafter_output = _find_longest_matched_ngram_and_propose_tokens(
            context_token_ids, min_n, max_n, max_model_len, k
        )
    ...

```

#### 4.4 N-gram GPU Proposer (GPU 加速版)

原理 NgramGPUKernel 将 N-gram 匹配完全搬到 GPU 上, 使用 PyTorch tensor 操作替代 CPU/Numba 实现,

避免 CPU-GPU 数据传输瓶颈。

```
# ngram_proposer_gpu.py L27-L44
@support_torch_compile()
class NgramGPUKernel(nn.Module):
    """GPU-accelerated N-gram proposer using fully async tensor operations."""
    def __init__(self, vllm_config, prefix="", device="cuda"):
        self.min_n = ...
        self.max_n = ...
        self.k = ...
```

核心算法: **Unfold + Argmax**

```
# ngram_proposer_gpu.py L46-L158
def _find_first_and_extract_all_n_parallel(
    self, token_ids, seq_lengths, min_ngram_len, max_ngram_len, num_draft_tokens
) -> torch.Tensor:
    batch_size = token_ids.shape[0]
    ngram_lengths = torch.arange(min_ngram_len, max_ngram_len + 1, device=device)

    # 对每种 n-gram 长度分别搜索
    first_match_positions = torch.full(
        (batch_size, num_ngram_sizes), -1, dtype=torch.long, device=device
    )

    for i, ngram_len in enumerate(range(min_ngram_len, max_ngram_len + 1)):
        # unfold 创建滑动窗口 O(1) view
        search_windows = token_ids.unfold(1, ngram_len, 1)

        # 提取后缀
        suffix_indices = seq_lengths.unsqueeze(1) + torch.arange(ngram_len, device=device)
        suffix = torch.gather(token_ids, 1, suffix_indices.clamp(min=0))

        # 全局匹配
        matches = (search_windows == suffix.unsqueeze(1)).all(dim=-1)
        # 找最早匹配
        first_match_idx = torch.argmax(matches.int(), dim=1)
        has_match = matches[batch_indices, first_match_idx]
        first_match_positions[:, i] = torch.where(has_match, first_match_idx, -1)

    # 选择最长匹配的 n-gram
    best_ngram_idx = (first_match_positions >= 0).int().flip(dims=[1]).argmax(dim=1)
    # 提取后续 k 个 token
    ...
```

关键技术点: - `torch.unfold()` 创建滑动窗口视图,  $O(1)$  时间复杂度 - 向量化 batch 操作, 所有 sequence 并行处理  
- `torch.compile` 支持 (`@support_torch_compile()`) - 异步 D2H copy (`copy_num_valid_draft_tokens()`)

增量更新机制 `update_ngram_gpu_tensors_incremental()` 实现增量更新 `token_ids`, 避免全量拷贝:

```
def update_ngram_gpu_tensors_incremental(input_batch, token_ids_gpu_tensor,
                                         num_tokens_no_spec_gpu, new_reqs, ...):
    # 仅对新请求/resumed 请求做全量拷贝
    for req_state in new_reqs:
        token_ids_gpu_tensor[new_req_idx, :num_tokens].copy_(...)
    # 检测 reorder (index 变化), 局部交换
```



```

if reorder_src:
    token_ids_gpu_tensor[dst_tensor] = temp_token_ids
# 批量同步 sequence lengths (CPU + GPU via pinned memory)
_sync_num_tokens(...)

```

---

#### 4.5 DFlash 推测解码

原理 DFlash (DeepSeek Flash Speculation) 是一种基于跨注意力 (**Cross-Attention**) 的推测解码方法。它将 target model 的 context KV cache 作为 Key/Value, Draft model 只需对 query tokens (bonus token + mask tokens) 做 attention, 无需重新计算 context 部分。

DFlashProposer 核心设计:

```

# dflash.py L21-L35
class DFlashProposer(SpecDecodeBaseProposer):
    def __init__(self, vllm_config, device, runner=None):
        super().__init__(
            vllm_config, device,
            pass_hidden_states_to_model=True,
            runner=runner,
        )
        # 只有 next_token_ids 和 mask tokens 是 query tokens
        self.max_query_tokens = self.max_batch_size * (1 + self.num_speculative_tokens)
        # Context 和 Query 使用分离的缓冲区 (保证 CUDA graph 地址稳定)
        self._context_slot_mapping_buffer = torch.zeros(...)
        self._slot_mapping_buffer = torch.zeros(...)

```

DFlash 独特之处

##### 1. Non-Causal Attention (L72-L80):

```

def _create_draft_vllm_config(self) -> VllmConfig:
    base = super()._create_draft_vllm_config()
    return replace(base,
        attention_config=replace(base.attention_config, use_non_causal=True),
    )

```

##### 2. Context KV 预计算 (L262-L267):

```

def build_model_inputs_first_pass(self, num_tokens, num_input_tokens, mm_embed_inputs):
    # 将 context hidden states 预插入 KV cache
    self.model.precompute_and_store_context_kv(
        self._dflash_hidden_states,
        self._context_positions_buffer[:num_context],
        self._context_slot_mapping_buffer[:num_context],
    )

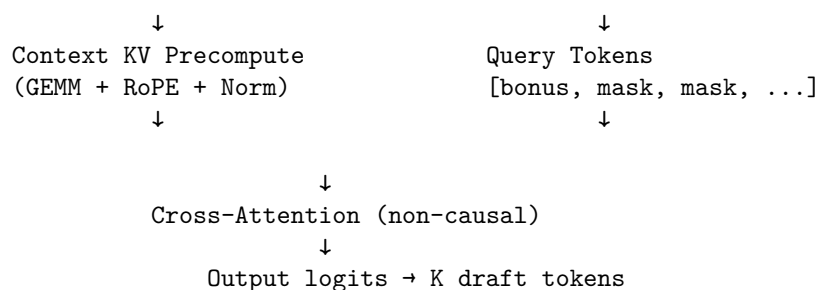
```

3. 强制 **Parallel Drafting**: DFlash 始终使用 parallel\_drafting=True (L712), 一次 forward 生成所有草稿 token

4. **Fused Triton Kernel**: 使用 copy\_and\_expand\_dflash\_inputs\_kernel 同时准备 context/query 的 input\_ids、positions、slot\_mapping

数据流示意

Target Model Forward → hidden\_states [context\_len, hidden\_dim]  
↓



## 4.6 Suffix Decoding (后缀解码)

原理 Suffix Decoding 基于 Arctic Inference 库实现 (论文: arXiv:2411.04975)。它维护一个全局的后缀树 (Suffix Tree)，缓存历史请求的输出模式，当新请求的模式与缓存匹配时进行推测。

SuffixDecodingProposer:

```
# suffix_decoding.py L9-L34
class SuffixDecodingProposer:
    def __init__(self, vllm_config):
        config = vllm_config.speculative_config
        self.num_speculative_tokens = config.num_speculative_tokens
        self.max_tree_depth = config.suffix_decoding_max_tree_depth
        self.max_spec_factor = config.suffix_decoding_max_spec_factor
        self.min_token_prob = config.suffix_decoding_min_token_prob

        from arctic_inference.suffix_decoding import SuffixDecodingCache
        self.suffix_cache = SuffixDecodingCache(
            max_tree_depth=config.suffix_decoding_max_tree_depth,
            max_cached_requests=config.suffix_decoding_max_cached_requests,
        )
```

工作流程

```
# suffix_decoding.py L35-97
def propose(self, input_batch, sampled_token_ids, slot_mappings=None):
    draft_token_ids = []
    for i, sampled_ids in enumerate(sampled_token_ids):
        req_id = input_batch.req_ids[i]

        # 启动新请求: 构建 prompt 后缀树
        if req_id not in self.suffix_cache.active_requests:
            prompt_token_ids = input_batch.token_ids_cpu[index, :num_prompt_tokens]
            self.suffix_cache.start_request(req_id, prompt_token_ids)

        # 追加最新采样的 token
        self.suffix_cache.add_active_response(req_id, sampled_ids)

        # 从序列末尾提取 pattern (最长 max_tree_depth 个 token)
        start = max(0, num_tokens - self.max_tree_depth)
        pattern = input_batch.token_ids_cpu[i, start:num_tokens]

        # 基于后缀树推测
```

```

draft = self.suffix_cache.speculate(
    req_id, pattern,
    max_spec_tokens=min(...),
    max_spec_factor=self.max_spec_factor,
    min_token_prob=self.min_token_prob,
)
draft_token_ids.append(draft.token_ids)

# 停止不在当前 batch 中的请求
for req_id in (self.suffix_cache.active_requests - input_batch.req_id_to_index.keys()):
    self.suffix_cache.stop_request(req_id)

return draft_token_ids

```

特点: - 动态推测数量: 每个 request 可能返回不同长度的草稿 - 基于 frequency count 估算 token 概率 - FIFO 淘汰旧缓存 - 需要安装 `arctic-inference` 库

#### 4.7 Gemma4 MTP (Multi-Token Prediction)

原理 Gemma4 的 MTP 模式让 assistant model 运行所有 **decoder layers** 来产生一个 draft token (而非 EAGLE 的轻量 head)。其注意力层通过 **cross-model KV sharing** 与 target model 共享 KV cache。

Gemma4Proposer:

```

# gemma4.py L31-48
class Gemma4Proposer(SpecDecodeBaseProposer):
    def __init__(self, vllm_config, device, runner=None):
        super().__init__(vllm_config, device,
            pass_hidden_states_to_model=True, runner=runner)
        # 所有 draft 步从同一位置预测 (不递进 position)
        self.constant_draft_positions = True

```

核心特性

1. **Constant Draft Positions** (L47): 所有 draft steps 重用相同 position, 因为 Gemma4 draft model 使用 Q-only attention 共享 target 的 KV cache
2. 多 **KV Cache Group** 支持 (L70-L97): Gemma4 有 sliding window attention 和 full attention 两种 layer type, 各自属于不同 KV cache group:

```

def build_per_group_and_layer_attn_metadata(self, common_attn_metadata, draft_index=0):
    per_group_attn_metadata = []
    per_layer_attn_metadata = {}
    for attn_group in self.draft_attn_groups:
        gid = attn_group.kv_cache_group_id
        if gid in self._per_group_block_tables:
            cm = copy(common_attn_metadata)
            cm.block_table_tensor = self._per_group_block_tables[gid]
    ...

```

3. **KV Sharing** 设置 (`_setup_gemma4_kv_sharing()`):

```

def _setup_gemma4_kv_sharing(self, target_attn_layer_names):
    # 每个 draft layer 映射到同类型的最后一个 non-KV-shared target layer
    for draft_idx, layer in enumerate(self.model.model.layers):
        draft_layer_type = draft_layer_types[draft_idx]
        candidates = type_to_target_indices.get(draft_layer_type, [])

```

```
target_idx = candidates[-1] # 取最后一个同类 layer
attn.kv_sharing_target_layer_name = f"{target_prefix}.{target_idx}.self_attn.attn"
```

4. **Centroids CUDA Graphs** (`_setup_centroids_cuda_graphs()`): 当使用 ordered embeddings 时, 为 `get_top_tokens` 预捕获多种 batch size 的 CUDA graph

5. **Tuple Return** (L64-L68):

```
def model_returns_tuple(self) -> bool:
    # forward() 返回 (draft_hidden_states, backbone_hidden_states)
    return True
```

---

#### 4.8 MLP Speculator (MLP 草稿模型)

原理 MLP Speculator 来自论文 “Accelerating Production LLMs with Combined Token/Embedding Speculators”。它是一个纯 MLP 结构的轻量级草稿模型, 由多级串联的 Embedding→Projection→LayerNorm→Head 组成。

MLPSpeculator:

```
graph LR
    subgraph "Stage 0"
        E0[Embedding] --> P0[Linear Projection]
    end
    subgraph "Stage 1..K-1 (tied weights)"
        E1[Embedding] --> PT[Tied Linear]
    end
    P0 --> LN0[LayerNorm] --> H0[LM Head → token_{t+1}]
    PT --> LN1[LayerNorm] --> H1[LM Head → token_{t+2}]
    LNK[LN_K] --> HK[LM Head → token_{t+K}]

    style E0 fill:#e8f5e9
    style E1 fill:#e8f5e9
    style PT fill:#fff3e0
```

结构细节

```
# mlp_speculator.py L60-L171
class MLPSpeculator(nn.Module):
    def __init__(self, *, vllm_config, prefix=""):
        config = vllm_config.model_config.hf_config
        self.n_predict = config.n_predict # 预测步数
        self.vocab_size = config.vocab_size
        self.emb_dim = config.emb_dim # 输入嵌入维度
        self.inner_dim = config.inner_dim # 内部维度
        self.max_speculative_tokens = config.num_lookahead_tokens
        self.tie_weights = config.tie_weights # 是否权重共享
        self.scale_input = config.scale_input # 是否缩放输入

        if self.tie_weights:
            # 共享 embedding 和 projection 权重
            self.emb = nn.ModuleList([embedding] * self.max_speculative_tokens)
            self.proj = nn.ModuleList(
                [proj_first] + [proj_tied] * (self.max_speculative_tokens - 1)
            )
        else:
```

```

# 每个阶段独立参数
self.emb = nn.ModuleList([...])
self.proj = nn.ModuleList([...])

# 每个 stage 一个 LM Head 和 LayerNorm
self.head = nn.ModuleList([ParallelLMHead(...) for _ in range(...)])
self.ln = nn.ModuleList([MLPSpeculatorLayerNorm(...) for _ in range(...)])

# 特殊的状态/嵌入混合权重
self.state_weight = 0.5 ** (0.5 / config.n_predict)
self.emb_weight = math.sqrt((1 - self.state_weight**2) * (self.inner_dim / 2))
self.activation = nn.GELU()

```

## LayerNorm 变体

```

# mlp_speculator.py L21-57
class MLPSpeculatorLayerNorm(nn.Module):
    """L2 Normalization (而非标准 LayerNorm 的均值归一化) """
    def forward(self, x):
        xf = x
        xf = xf * torch.rsqrt(xf.pow(2).mean(-1, keepdim=True) + self.eps)
        x = xf.type_as(x)
        if self.elementwise_scale_and_shift:
            x = self.weight * x + self.bias
        return x

```

## 4.9 LLM Base Proposer (基于 LLM 的草稿)

SpecDecodeBaseProposer 本身就是 LLM-based proposer 的完整实现。除了上述专门的子类外，以下方法都通过此基类工作：

- **method="draft\_model"**: 使用任意小模型作为 draft model (DraftModelProposer)
- **method="mtp"**: MTP (Multi-Token Prediction) 模型，如 DeepSeek-MTP、MiMo-MTP 等
- **method="eagle"**: EAGLE prediction head
- **method="eagle3"**: EAGLE3 多辅助层
- **method="dflash"**: DFlash cross-attention (DFlashProposer)

这些方法的差异主要体现在：1. `_get_model()` 返回不同的模型结构 2. `pass_hidden_states_to_model` 的设置 3. `model_returns_tuple()` 的返回值 4. `parallel_drafting` 的开关

## 五、Extract Hidden States (隐藏状态提取)

### 5.1 用途

ExtractHiddenStatesProposer 是一种特殊的 proposer，其目的不是推测 **token**，而是从 target model 的指定中间层提取 hidden states 并存入 KV cache。

主要用途：- **KV Transfer**: 跨实例/跨节点传输 KV cache 时需要的中间状态缓存 - **Debug/Analysis**: 获取模型中间层表示

### 5.2 核心特性

```

# extract_hidden_states.py L26-L66
class ExtractHiddenStatesProposer:

```

```

def __init__(self, vllm_config, device):
    # 固定 num_speculative_tokens == 1
    assert vllm_config.speculative_config.num_speculative_tokens == 1

    # 从配置获取要提取的层 ID 列表
    self.hf_config = vllm_config.speculative_config.draft_model_config.hf_config
    layer_ids = getattr(self.hf_config, "eagle_aux_hidden_state_layer_ids", None)
    self.num_hidden_states = len(layer_ids)

    # hidden states 缓冲区: [max_num_tokens, num_layers, hidden_size]
    self.hidden_states = torch.zeros(
        (self.max_num_tokens, self.num_hidden_states, self.hidden_size),
        dtype=self.dtype, device=device,
    )

```

### 5.3 propose() 行为

```

# extract_hidden_states.py L72-L151
def propose(self, sampled_token_ids, target_hidden_states,
            common_attn_metadata, slot_mappings=None):
    # 1. 堆叠各层的 hidden states: [num_tokens, num_layers, hidden_size]
    stacked_hidden_states = torch.stack(target_hidden_states, dim=1)

    # 2. 存入缓冲区
    self.hidden_states[:num_tokens] = stacked_hidden_states

    # 3. 通过 ExtractHiddenStatesModel 写入 KV cache (cache-only attention)
    self.model(hidden_states=self.hidden_states[:num_input_tokens])

    # 4. 返回原始 sampled tokens (不做任何推测, 确保 " 总是被接受 ")
    return sampled_token_ids[:, :1]

```

关键: 返回的 draft tokens 就是原始 sampled tokens, 因此 rejection sampling 总是接受——这保证了该方法不会改变生成结果。

## 六、Metrics (指标体系)

### 6.1 SpecDecodingStats

metrics.py 定义了每步统计数据:

```

# metrics.py L17-L47
@dataclass
class SpecDecodingStats:
    num_spec_tokens: int          # 配置的推测 token 数 K
    num_drafts: int = 0           # 累计 draft 次数
    num_draft_tokens: int = 0     # 累计生成的草稿 token 总数
    num_accepted_tokens: int = 0  # 累计被接受的 token 总数
    num_accepted_tokens_per_pos: list[int]  # 各位置累计接受次数 [K]

    def observe_draft(self, num_draft_tokens, num_accepted_tokens):
        self.num_drafts += 1
        self.num_draft_tokens += num_draft_tokens
        self.num_accepted_tokens += num_accepted_tokens

```

```

    for i in range(num_accepted_tokens):
        self.num_accepted_tokens_per_pos[i] += 1

```

## 6.2 日志指标

SpecDecodingLogging 定期聚合输出:

```

# metrics.py L74-L117
def log(self, log_fn=logger.info):
    # 核心指标计算
    draft_acceptance_rate = num_accepted_tokens / num_draft_tokens * 100
    mean_acceptance_length = 1 + (num_accepted_tokens / num_drafts)
    # 含义: 平均每次 draft 产生 1 (bonus) + accepted 个有效 token

    acceptance_rates = np.sum(pos_matrix, axis=0) / num_drafts
    # 逐位置接受率向量, 用于分析哪个位置的接受率最低

    log_fn(
        "SpecDecoding metrics: "
        "Mean acceptance length: %.2f, "           # 平均接受长度 (含 bonus)
        "Accepted throughput: %.2f tok/s, "         # 有效 token 吞吐
        "Drafted throughput: %.2f tok/s, "          # 草稿 token 吞吐
        "Avg Draft acceptance rate: %.1f%%",        # 平均接受率
        ...
    )

```

## 6.3 Prometheus 指标

SpecDecodingProm 导出 Prometheus Counter:

| 指标名                                          | 说明                              |
|----------------------------------------------|---------------------------------|
| vllm:spec_decode_num_drafts                  | 累计 draft 次数                     |
| vllm:spec_decode_num_draft_tokens            | 累计草稿 token 数                    |
| vllm:spec_decode_num_accepted_tokens         | 累计接受 token 数                    |
| vllm:spec_decode_num_accepted_tokens_per_pos | 分位置接受 token 数 (label: position) |

PromQL 查询示例:

```

-- 接受率
rate(vllm:spec_decode_num_accepted_tokens_total[$interval])
/ rate(vllm:spec_decode_num_draft_tokens_total[$interval])

-- 平均接受长度 (含 bonus token)
1 + (
    rate(vllm:spec_decode_num_accepted_tokens_total[$interval])
    / rate(vllm:spec_decode_num_drafts[$interval])
)

```

## 七、辅助模块

### 7.1 utils.py —Triton Kernels 与工具函数

utils.py 包含推测解码的核心 Triton kernels 和工具函数:

Triton Kernels 汇总

| Kernel                                  | 位置        | 功能                                                       |
|-----------------------------------------|-----------|----------------------------------------------------------|
| eagle_step_slot_mapping_metadata_kernel | L120-L186 | EAGLE 单步 fused 更新: position/slot_mapping/seq_lens        |
| eagle_prepare_inputs_padded_kernel      | L137-L177 | 计算 padded 模式下的 token_indices_to_sample 和 rejected counts |
| eagle_prepare_next_token_padded_kernel  | L180-L240 | 计算 next_token_id 和 valid_sampled_tokens_count            |
| copy_and_expand_eagle_inputs_kernel     | L309-L455 | EAGLE 输入复制/扩展: 处理 padding/rejected/masked regions        |
| copy_and_expand_dflash_inputs_kernel    | L459-L563 | DFlash 输入准备: context/query 分离处理                          |

工具函数

| 函数                                            | 功能                                         |
|-----------------------------------------------|--------------------------------------------|
| next_power_of_2(n)                            | 向上取整到最近的 2 的幂                              |
| compute_new_slot_mapping()                    | 基于新位置和 reject mask 计算 slot mapping         |
| extend_all_queries_by_N()                     | 将 CommonAttentionMetadata 的查询长度扩展 N        |
| unconditional_to_conditional_rates()          | 无条件率转条件率 (用于 synthetic rejection sampling) |
| update_num_computed_tokens_for_batch_change() | 异步 spec decode drift 修正 (torch.compile)    |

7.2 metadata.py –SpecDecodeMetadata

metadata.py 定义推测解码的元数据容器:

```
# metadata.py L9-L25
@dataclass
class SpecDecodeMetadata:
    draft_token_ids: torch.Tensor           # [num_tokens] 展平的草稿 token IDs
    num_draft_tokens: list[int]              # [batch_size] 每个 request 的草稿数
    cu_num_draft_tokens: torch.Tensor        # [batch_size] 草稿数的 cumsum
    cu_num_sampled_tokens: torch.Tensor      # [batch_size] 采样数的 cumsum
    target_logits_indices: torch.Tensor      # [num_tokens] target logits 索引
    bonus_logits_indices: torch.Tensor       # [batch_size] bonus token logits 索引
    logits_indices: torch.Tensor             # [num_tokens + batch_size] 完整 logits 索引
```

make\_dummy() 工厂方法用于创建测试用的 dummy metadata。

八、推测解码完整 workflow

```
sequenceDiagram
    participant Sch as Scheduler
    participant Target as Target Model
    participant Prop as Proposer<br/>[Draft Model]
    participant Verify as Verification<br/>[Rejection Sample]
    participant Out as Output
```



```

Note over Sch,Out: === Decoding Step N ===

Sch->>Target: 正常采样得到 next_token
Target-->>Sch: sampled_token_ids + hidden_states

alt 使用基于模型的推测方法 (EAGLE/Medusa/MTP/DFlash/...)
  Sch->>Prop: target_token_ids, positions,<br/>hidden_states, next_token_ids
  Prop->>Prop: set_inputs_first_pass()<br/> 构建草稿输入
  loop 每个 draft token (或并行)
    Prop->>Prop: Draft Model forward
    Prop->>Prop: _greedy_sample()
  end
  Prop-->>Sch: draft_token_ids [batch_size, K]
else 使用 N-gram (CPU)
  Sch->>Prop: token_ids_cpu, seq_lens
  Prop->>Prop: Numba LPS 匹配
  Prop-->>Sch: draft_token_ids [list of lists]
else 使用 N-gram (GPU)
  Sch->>Prop: token_ids_gpu, seq_lens
  Prop->>Prop: Unfold + Argmax (GPU)
  Prop-->>Sch: draft_token_ids [batch_size, K]
else 使用 Suffix Decoding
  Sch->>Prop: input_batch, sampled_ids
  Prop->>Prop: 后缀树查询
  Prop-->>Sch: draft_token_ids [动态长度]
end

Sch->>Target: 将 draft tokens 拼接到输入 <br/>[next_token, draft_1, ..., draft_K]
Target->>Target: 一次 Forward 验证所有 K+1 个位置
Target-->>Verify: K+1 个位置的 logits

Verify->>Verify: Rejection Sampling<br/> 逐位概率比较  $r \sim \min(1, q/p)$ 
Verify-->>Sch: accepted_count [0, K]<br/> 可能 resample 一个 token

Sch->>Sch: 更新 SpecDecodingStats<br/> 记录 metrics
Sch->>Out: 输出 1 + accepted_count 个 token

Note over Sch,Out: === 接受率 ~50-70% 时 <br/> 平均每步产出 1.5~2.4 个 token ===

```

## 九、方法选型指南

| 方法                  | Draft 开销             | 接受率         | 适用场景          | 依赖                |
|---------------------|----------------------|-------------|---------------|-------------------|
| <b>N-gram (CPU)</b> | 极低 (纯统计)             | 中等 (依赖重复模式) | 高重复文本         | 无                 |
| <b>N-gram (GPU)</b> | 低 (GPU tensor ops)   | 同上          | 高重复文本 + 低延迟需求 | 无                 |
| <b>EAGLE</b>        | 低 (lightweight head) | 高           | 通用场景          | EAGLE checkpoint  |
| <b>EAGLE3</b>       | 低 (multi-head)       | 更高          | 通用场景          | EAGLE3 checkpoint |
| <b>Medusa</b>       | 低 (MLP heads)        | 高           | 通用场景          | Medusa checkpoint |

| 方法                         | Draft 开销                               | 接受率         | 适用场景                               | 依赖                      |
|----------------------------|----------------------------------------|-------------|------------------------------------|-------------------------|
| <b>MLP Speculator</b>      | 低 (pure MLP)                           | 高           | 通用场景                               | IBM Speculator ckpt     |
| <b>Draft Model MTP</b>     | 中 (小模型 forward)<br>中 (shared backbone) | 高<br>高      | 有合适小模型<br>DeepSeek/Qwen 等原生 MTP 模型 | 任意小模型<br>MTP checkpoint |
| <b>DFlash</b>              | 中低 (cross-attn)                        | 高           | Qwen3.5 等兼容模型                      | DFlash checkpoint       |
| <b>Suffix Decoding</b>     | 低 (后缀树查询)                              | 变动 (依赖缓存命中) | 多相似请求                              | arctic-inference        |
| <b>Gemma4 MTP</b>          | 中 (full decoder)                       | 高           | Gemma4 模型                          | Gemma4-assistant        |
| <b>ExtractHiddenStates</b> | State-only                             | 100% (不推测)  | KV Transfer/debug                  | 特定 config               |

十、关键源码文件索引

| 文件路径                                    | 核心内容                                     |
|-----------------------------------------|------------------------------------------|
| config/speculative.py                   | SpeculativeConfig 配置类，所有参数定义与方法检测        |
| v1/spec_decode/llm_base_proposer.py     | SpecDecodeBaseProposer 基类，propose() 核心流程 |
| v1/spec_decode/draft_model.py           | DraftModelProposer 独立小模型实现               |
| v1/spec_decode/eagle.py                 | EagleProposer EAGLE/EAGLE3 实现            |
| model_executor/models/medusa.py         | Medusa 多头预测模型                            |
| v1/spec_decode/ngram_proposer.py        | N-gram CPU proposer (Numba JIT)          |
| v1/spec_decode/ngram_proposer_gpu.py    | N-gram GPU proposer (向量化)                |
| v1/spec_decode/dflash.py                | DFlash 跨注意力推测解码                          |
| v1/spec_decode/suffix_decoding.py       | Suffix Decoding 后缀解码                     |
| v1/spec_decode/gemma4.py                | Gemma4 MTP proposer                      |
| model_executor/models/mlp_speculator.py | MLP Speculator 轻量草稿模型                    |
| v1/spec_decode/extract_hidden_states.py | 隐藏状态提取 proposer                          |
| v1/spec_decode/metrics.py               | 指标统计 (Stats/Logging/Prometheus)          |
| v1/spec_decode/utils.py                 | Triton Kernels + 工具函数                    |
| v1/spec_decode/metadata.py              | SpecDecodeMetadata 数据容器                  |

vLLM 可观测性分析: Metrics & Monitoring

定位: vLLM 的可观测性 (Observability) 体系, 涵盖指标收集 → 日志系统 → 追踪与性能分析 → 使用情况追踪 → 配置管理的全链路架构。

```
graph TB
    subgraph 指标收集 [" 指标收集系统"]
        direction TB
        A1[perf.py<br/> 性能指标 MFU/带宽] --> A2[stats.py<br/> 迭代统计/缓存统计]
        A2 --> A3[prometheus.py<br/>Prometheus 导出端点]
        A2 --> A4[reader.py<br/> 指标读取接口]
        A5[utils.py<br/> 指标工具函数] --> A3
    end

    subgraph 日志 [" 日志系统"]
```

```

        direction TB
        B1[loggers.py<br/>StatLogger 工厂体系] --> B2[ray_wrappers.py<br/>Ray 适配层]
    end

    subgraph 追踪性能 [" 追踪与性能分析"]
        direction TB
        C1[otel.py<br/>OpenTelemetry 集成] --> C2[tracing/utils.py<br/>Span 属性定义]
        C3[layerwise_profile.py<br/>逐层性能分析] --> C4[wrapper.py<br/>Profiler 包装器]
        C4 --> C5[nvtx_pytorch_hooks.py<br/>NVTX Hooks]
    end

    subgraph 管理配置 [" 管理与配置"]
        D1[usage_lib.py<br/>使用情况追踪] --> D2[observability.py<br/>ObservabilityConfig]
    end

    A3 -. -> | 导出 | B1
    B1 --> | 管理 | StatLoggerManager
    C1 --> | 集成 | D2
    D2 --> | 控制 | A1
    D2 --> | 控制 | C3

    style 指标收集 fill:#b3e5fc,stroke:#0277bd,color:#000
    style 日志 fill:#b3e5fc,stroke:#0277bd,color:#000
    style 追踪性能 fill:#b3e5fc,stroke:#0277bd,color:#000
    style 管理配置 fill:#b3e5fc,stroke:#0277bd,color:#000

```

---

## 一、指标收集系统

### 1.1 性能指标—perf.py (MFU / 带宽估算)

文件: perf.py

该模块是 vLLM 性能指标的核心，负责计算 **Model FLOPs Utilization (MFU)** 和内存带宽利用率。其设计采用了组件化的 Parser Chain 模式，将模型拆分为 Attention / FFN / Unembed 三个可独立计算的组件。

#### 1.1.1 核心数据结构 ExecutionContext 一执行上下文，聚合一个 batch 中所有请求的统计信息：

```

# perf.py:104-169
@dataclass
class ExecutionContext:
    """ 代表一个 batch 的执行上下文，分别追踪 prefill 和 decode 阶段。 """
    num_prefill_requests: int = 0
    prefill_num_tokens: int = 0          # prefill 阶段的 token 总数
    prefill_context_len: int = 0         # prefill 阶段的 context 长度总和
    prefill_token_context_product: int = 0 #  $\sum(\text{num\_tokens} \times \text{context\_len})$ 

    num_decode_requests: int = 0
    decode_num_tokens: int = 0
    decode_context_len: int = 0
    decode_token_context_product: int = 0

```

PerfStats 一每个 step 的性能快照：

```

# perf.py:96-101
@dataclass

```

```
class PerfStats:
    num_flops_per_gpu: int = 0      # 每 GPU 浮点运算次数
    num_read_bytes_per_gpu: int = 0 # 每 GPU 内存读取字节数
    num_write_bytes_per_gpu: int = 0 # 每 GPU 内存写入字节数
```

**1.1.2 组件化 Parser Chain 架构** vLLM 采用 **Chain of Responsibility + Registry** 模式来解析模型配置并推导 FLOPs:

```
graph LR
    VllmConfig[VllmConfig] --> PC[ParserChain]
    PC --> P1[BaseConfigParser]
    PC --> P2[BaseAttentionConfigParser]
    PC --> P3[AttentionQuantizationConfigParser]
    PC --> P4[FfnParallelParser]
    PC --> P5[FfnQuantizationConfigParser]

    P1 -->|vocab_size, hidden_size,<br/>tp_size, pp_size| CM[ComponentMetrics]
    P2 -->|num_kv_heads, head_dim,<br/>cache_byte_size| CM
    P3 -->|覆盖 weight_byte_size| CM
    P4 -->|ffn_tp_size, ffn_ep_size| CM
    P5 -->|覆盖 weight_byte_size| CM

    style VllmConfig fill:#e1f5fe
    style CM fill:#fff9c4
```

每个 ComponentMetrics 子类通过 `__init_subclass__` 自动注册到全局注册表:

```
# perf.py:226-258
_COMPONENT_METRICS_REGISTRY: dict[str, type["ComponentMetrics"]] = {}

class ComponentMetrics(BaseModel, ABC):
    def __init_subclass__(cls):
        _COMPONENT_METRICS_REGISTRY[cls.component_type()] = cls

    @classmethod
    def from_vllm_config(cls, vllm_config: VllmConfig) -> Self:
        parser = cls.get_parser()
        parsed_args = parser.parse(vllm_config)
        return cls.model_validate(parsed_args.model_dump())
```

### 1.1.3 三大组件的 FLOPs 计算

| 组件        | 类名               | 关键 FLOPs 公式                                                  | 文件位置     |
|-----------|------------------|--------------------------------------------------------------|----------|
| Attention | AttentionMetrics | $qkv\_proj: 2 \cdot T \cdot D \cdot (q+2kv) \cdot d \cdot L$ | L432-456 |
| FFN       | FfnMetrics       | 含 Dense/MoE routed/MoE shared 三类                             | L716-757 |
| Unembed   | UnembedMetrics   | $unembed: 2 \cdot T \cdot D \cdot V$                         | L938-950 |

其中 FFN 组件完整支持 MoE (Mixture of Experts) 模型, 包括: - **Dense FFN**:  $flops = 2 \cdot D \cdot 3 \cdot DI \cdot T \cdot Ld$  - **MoE Routed**:  $flops = 2 \cdot D \cdot 3 \cdot MI \cdot (T \cdot E) \cdot Lm$  - **MoE Shared**:  $flops = 2 \cdot D \cdot 3 \cdot MI \cdot S \cdot T \cdot Lm$

注意: 所有公式均考虑了 Tensor Parallelism (`tp_size`)、Pipeline Parallelism (`pp_size`) 和 Expert Parallelism (`ep_size`)。

#### 1.1.4 ModelMetrics —聚合入口

```
# perf.py:985-1116
class ModelMetrics:
    def __init__(self, vllm_config: VllmConfig):
        self.metrics: list[ComponentMetrics] = []
        for metric_cls in ComponentMetrics.registered_metrics():
            try:
                metric = metric_cls.from_vllm_config(vllm_config)
                self.metrics.append(metric)
            except InvalidComponent as e:
                logger.debug("Failed to instantiate %s", ...)

    def get_step_perf_stats_per_gpu(self, scheduler_output) -> PerfStats:
        """ 基于 SchedulerOutput 计算当前 step 的性能统计。"""
        ctx = ExecutionContext()
        for new_req in scheduler_output.scheduled_new_reqs:
            ctx.add(num_tokens, context_len, is_prefill=True)
        for req_id in cached_reqs.req_ids:
            ctx.add(num_tokens, context_len, is_prefill=num_tokens > 1)
        ...
```

#### 1.1.5 Prometheus 集成—PerfMetricsProm

```
# perf.py:1265-1332
class PerfMetricsProm:
    """ 将性能指标记录到 Prometheus Counter。"""
    _counter_cls = prometheus_client.Counter

    def __init__(self, vllm_config, labelnames, per_engine_labelvalues):
        self.counter_flops = create_metric_per_engine(
            Counter(name="vllm:estimated_flops_per_gpu_total", ...),
            per_engine_labelvalues)
        self.counter_read_bytes = create_metric_per_engine(...)
        self.counter_write_bytes = create_metric_per_engine(...)

    def observe(self, perf_stats, engine_idx=0):
        self.counter_flops[engine_idx].inc(perf_stats.num_flops_per_gpu)
        self.counter_read_bytes[engine_idx].inc(perf_stats.num_read_bytes_per_gpu)
        self.counter_write_bytes[engine_idx].inc(perf_stats.num_write_bytes_per_gpu)
```

通过 PromQL 可计算平均 TFLOPS:

```
rate(vllm:estimated_flops_per_gpu_total[1m]) / 1e12
```

---

### 1.2 统计数据—stats.py

文件: stats.py

这是 vLLM 运行时统计数据的核心数据结构定义文件，定义了从 Scheduler 到 Request 全链路的统计信息。

#### 1.2.1 缓存命中率统计—CachingMetrics

```
# stats.py:35-112
class CachingMetrics:
```

""" 基于滑动窗口的缓存命中率统计（默认最近  $N=1000$  个请求）。 """

```
def __init__(self, max_recent_requests: int = 1000):
    self.query_queue = deque[tuple[int, int, int]]() # (requests, queries, hits)

@property
def hit_rate(self) -> float:
    if self.aggregated_query_total == 0:
        return 0.0
    return self.aggregated_query_hit / self.aggregated_query_total
```

支持三种缓存统计类型:

| 类型                   | 类名                   | 用途                 |
|----------------------|----------------------|--------------------|
| Prefix Cache         | PrefixCacheStats     | KV Cache 自动前缀复用命中率 |
| Multi-Modal Cache    | MultiModalCacheStats | 多模态数据（图片等）缓存命中     |
| External KV Transfer | PrefixCacheStats     | 跨实例 KV 传输缓存命中      |

### 1.2.2 调度器统计—SchedulerStats

```
# stats.py:171-199
@dataclass
class SchedulerStats:
    num_running_reqs: int = 0 # 正在运行的请求数
    num_waiting_reqs: int = 0 # 排队等待的请求数
    num_skipped_waiting_reqs: int = 0 # 被跳过的等待请求
    kv_cache_usage: float = 0.0 # KV Cache 使用率 (0~1)
    prefix_cache_stats: PrefixCacheStats
    spec_decoding_stats: SpecDecodingStats | None = None
    cudagraph_stats: CUDAGraphStat | None = None
    perf_stats: PerfStats | None = None # 来自 perf.py 的性能统计
```

### 1.2.3 请求生命周期统计 RequestStateStats —追踪单个请求的状态时间戳:

```
# stats.py:202-221
@dataclass
class RequestStateStats:
    arrival_time: float # 到达时间 (frontend timestamp)
    queued_ts: float # 进入排队队列时间
    scheduled_ts: float # 被调度时间
    first_token_ts: float # 首个 token 生成时间
    last_token_ts: float # 最后一个 token 生成时间
    first_token_latency: float # TTFT (Time To First Token)
```

FinishedRequestStats —完成请求的详细延迟分解:

```
# stats.py:224-239
@dataclass
class FinishedRequestStats:
    e2e_latency: float # 端到端延迟
    queued_time: float # 排队等待时间
    prefill_time: float # Prefill 阶段时间
    inference_time: float # 推理总时间
    decode_time: float # Decode 阶段时间
    mean_time_per_output_token: float # TPOT (Time Per Output Token)
```

这些字段构成了完整的 延迟分解链路:

```
graph LR
    A[arrival_time] -->|queued_time| B[scheduled_ts]
    B -->|prefill_time| C[first_token_ts]
    C -->|decode_time| D[last_token_ts]
    A -->|e2e_latency| D

    style A fill:#e8f5e9
    style D fill:#fce4ec
```

#### 1.2.4 IterationStats — 单次迭代统计

```
# stats.py:325-481
class IterationStats:
    """ 单次 EngineCore 输出的统计集合。 """
    def __init__(self):
        self.num_generation_tokens = 0
        self.prompt_token_stats = PromptTokenStats()
        self.time_to_first_tokens_iter: list[float] = []      # TTFT 列表
        self.inter_token_latencies_iter: list[float] = []     # ITL (Inter-Token Latency) 列表
        self.finished_requests: list[FinishedRequestStats] = []
```

update\_from\_output() 方法是核心的数据采集逻辑, 负责: 1. 区分 prefill / decode 阶段并分别记录 TTFT 和 ITL 2. 处理 EngineCoreEvent (QUEUED / SCHEDULED / PREEMPTED) 3. 在请求完成时计算所有延迟指标

---

#### 1.3 Prometheus 导出端点—prometheus.py

文件: prometheus.py

提供 Prometheus 多进程支持的基础设施:

```
# prometheus.py:17-36
def setup_multiprocess_prometheus():
    """ 设置多进程 Prometheus 目录。
    创建临时目录供多进程 Collector 写入各自进程的指标,
    并设置 PROMETHEUS_MULTIPROC_DIR 环境变量。 """
    global _prometheus_multiproc_dir
    if "PROMETHEUS_MULTIPROC_DIR" not in os.environ:
        _prometheus_multiproc_dir = tempfile.TemporaryDirectory()
        os.environ["PROMETHEUS_MULTIPROC_DIR"] = _prometheus_multiproc_dir.name

def get_prometheus_registry() -> CollectorRegistry:
    """ 根据是否启用多进程返回对应的 Registry。 """
    if os.getenv("PROMETHEUS_MULTIPROC_DIR") is not None:
        registry = CollectorRegistry()
        multiprocess.MultiProcessCollector(registry)
        return registry
    return REGISTRY
```

关键点: - 多进程模式下, 每个 worker 进程将指标写入独立的临时文件, 由 MultiProcessCollector 聚合 - shutdown\_prometheus() 在进程退出时调用 mark\_process\_dead() 清理

## 1.4 指标读取接口—reader.py

文件: reader.py

提供编程方式访问内存中 Prometheus 指标的 API, 支持四种指标类型:

```
# reader.py:12-68
@dataclass
class Counter(Metric):    value: int        # 单调递增计数器
@dataclass
class Vector(Metric):    values: list[int] # 有序整数数组 (spec_decode 专用)
@dataclass
class Gauge(Metric):     value: float      # 可增减的数值
@dataclass
class Histogram(Metric): count: int; sum: float; buckets: dict[str, int]
```

核心 API `get_metrics_snapshot()` 从 Prometheus Registry 中提取所有 `vllm:` 前缀的指标:

```
# reader.py:70-143
def get_metrics_snapshot() -> list[Metric]:
    collected: list[Metric] = []
    for metric in REGISTRY.collect():
        if not metric.name.startswith("vllm:"):
            continue
        if metric.type == "gauge":
            ... # 提取为 Gauge
        elif metric.type == "counter":
            ... # 提取为 Counter 或 Vector (特殊处理 spec_decode)
        elif metric.type == "histogram":
            ... # 提取为 Histogram (处理 DP 场景下的 bucket 聚合)
    return collected
```

特殊处理: - **Vector** 类型: 专用于 `vllm:spec_decode_num_accepted_tokens_per_pos`, 按 position 维度聚合成数组 - **Histogram** 聚合: 处理 Data Parallel 下 per-engine-per-bucket 的计数合并

---

## 1.5 指标工具函数—utils.py

文件: utils.py

轻量级工具函数, 为每个引擎索引创建带标签的 Prometheus 指标子实例:

```
# utils.py:11-19
def create_metric_per_engine(
    metric: PromMetric,
    per_engine_labelvalues: dict[int, list[object]],
) -> dict[int, PromMetric]:
    """ 为每个引擎索引创建带标签的 metric child. """
    return {
        idx: metric.labels(*labelvalues)
        for idx, labelvalues in per_engine_labelvalues.items()
    }
```

这是实现多引擎 (Data Parallel) 指标隔离的关键函数——每个引擎拥有独立的带 `engine` 标签的指标实例。



## 二、日志系统

### 2.1 StatLogger 工厂体系—loggers.py

文件: loggers.py

这是整个日志系统的核心, 实现了完整的 StatLogger 层次结构:

```
classDiagram
    class StatLoggerBase {
        <<abstract>>
        +record(scheduler_stats, iteration_stats, mm_cache_stats, engine_idx)
        +log_engine_initialized()
        +log()
        +record_sleep_state(is_awake, level)
    }

    class AggregateStatLoggerBase {
        <<abstract>>
        +__init__(vllm_config, engine_indexes)
    }

    class LoggingStatLogger {
        -prefix_caching_metrics: CachingMetrics
        -spec_decoding_logging: SpecDecodingLogging
        -kv_connector_logging: KVConnectorLogging
        -cudagraph_logging: CUDAGraphLogging
        -perf_metrics_logging: PerfMetricsLogging
        +record()
        +log()
    }

    class AggregatedLoggingStatLogger {
        -engine_indexes: list
        -last_scheduler_stats_dict: dict
        +aggregate_scheduler_stats()
    }

    class PerEngineStatLoggerAdapter {
        -per_engine_stat_loggers: dict
    }

    class PrometheusStatLogger {
        -gauge_scheduler_running: dict
        -histogram_time_to_first_token: dict
        -histogram_inter_token_latency: dict
        -histogram_e2e_time_request: dict
        +record()
        +log_metrics_info()
        +record_sleep_state()
    }

    StatLoggerBase <|-- AggregateStatLoggerBase
    StatLoggerBase <|-- LoggingStatLogger
    AggregateStatLoggerBase <|-- AggregatedLoggingStatLogger
    AggregateStatLoggerBase <|-- PerEngineStatLoggerAdapter
```

```
AggregateStatLoggerBase <|-- PrometheusStatLogger
AggregatedLoggingStatLogger --|> LoggingStatLogger
```

### 2.1.1 LoggingStatLogger —标准输出日志 核心方法 log() 输出的典型格式:

Engine 000: Avg prompt throughput: 1500.2 tokens/s, Avg generation throughput: 85.3 tokens/s,  
Running: 32 reqs, Waiting: 5 reqs, GPU KV cache usage: 78.5%,  
Prefix cache hit rate: 42.3%

关键实现细节 (L217-281):

```
def log(self):
    self._update_stats()
    self.aggregate_scheduler_stats()
    # 空闲生产环境用 debug 级别避免噪音
    log_fn = logger.debug if self.engine_is_idle else logger.info
    log_fn(self.log_prefix + ", ".join(log_parts), *log_args)
```

吞吐量计算使用实际计算 token 数 (排除 cached/transferred tokens), 确保反映真实算力消耗:

```
# loggers.py:143-148
def _track_iteration_stats(self, iteration_stats):
    self.num_prompt_tokens += iteration_stats.prompt_token_stats.computed # 仅 computed
    self.num_generation_tokens += iteration_stats.num_generation_tokens
```

### 2.1.2 PrometheusStatLogger —Prometheus 指标记录 这是最复杂的 Logger, 定义了完整的 vLLM Prometheus 指标体系:

Gauge 类指标 (瞬时状态):

| 指标名                                 | 说明             | 标签                                                     |
|-------------------------------------|----------------|--------------------------------------------------------|
| vllm:num_requests_running           | 运行中的请求数        | model_name, engine                                     |
| vllm:num_requests_waiting           | 等待中的请求数        | model_name, engine                                     |
| vllm:num_requests_waiting_by_reason | 按原因分类的等待数      | model_name, engine, reason                             |
| vllm:kv_cache_usage_perc            | KV Cache 使用百分比 | model_name, engine                                     |
| vllm:engine_sleep_state             | 引擎休眠状态         | model_name, engine, sleep_state                        |
| vllm:lora_requests_info             | LoRA 适配器状态     | max_lora, waiting_lora_adapters, running_lora_adapters |

Counter 类指标 (累积计数):

| 指标名                                | 说明                         |
|------------------------------------|----------------------------|
| vllm:prompt_tokens                 | 处理的 prefill token 总数       |
| vllm:generation_tokens             | 生成的 decode token 总数        |
| vllm:prompt_tokens_by_source       | 按 source 分类的 prompt tokens |
| vllm:request_success               | 按完成原因分类的成功请求数              |
| vllm:num_preemptions               | 累计抢占次数                     |
| vllm:prefix_cache_queries/hits     | 前缀缓存查询/命中数                 |
| vllm:estimated_flops_per_gpu_total | 估计 FLOPS (MFU 用)           |

Histogram 类指标 (延迟分布):

| 指标名                               | 说明              | 典型 buckets    |
|-----------------------------------|-----------------|---------------|
| vllm:time_to_first_token_seconds  | TTFT 分布         | 0.001 ~ 2560s |
| vllm:inter_token_latency_seconds  | ITLs 分布         | 0.01 ~ 80s    |
| vllm:e2e_request_latency_seconds  | 端到端延迟分布         | 0.3 ~ 7680s   |
| vllm:request_prefill_time_seconds | Prefill 时间分布    | 同 e2e         |
| vllm:request_decode_time_seconds  | Decode 时间分布     | 同 e2e         |
| vllm:request_queue_time_seconds   | 排队时间分布          | 同 e2e         |
| vllm:request_params_max_tokens    | max_tokens 参数分布 | 1-2-5 buckets |

### 2.1.3 StatLoggerManager 统一管理入口

# *loggers.py:1268-1359*

```
class StatLoggerManager:
    """
    抽象了 DP 场景下的 logger 实现差异:
    - DP > 1 个 EngineCore 时, 每个 EngineCore 各自记录
    - Local Logger: 为 N 个 EngineCore 创建 N 个副本
    - Prometheus: 单个 logger 带 N 个 label
    """

    def __init__(self, vllm_config, engine_idx, custom_stat_loggers,
                 enable_default_loggers=True, aggregate_engine_logging=False, client_count=1):
        # 1. 加载自定义插件 logger
        # 2. 根据 aggregate_engine_logging 选择 AggregatedLoggingStatLogger 或 LoggingStatLogger
        # 3. 如果没有自定义 Prometheus logger, 自动添加默认 PrometheusStatLogger
        ...

    def record(self, scheduler_stats, iteration_stats, mm_cache_stats, engine_idx=None):
        for stat_logger in self.stat_loggers:
            stat_logger.record(...)

    def log(self):
        for logger in self.stat_loggers:
            logger.log()
```

StatLoggerManager 是 AsyncLLMEngine 与底层日志系统之间的唯一接口, 屏蔽了单引擎 vs 多引擎 (DP) 的实现差异。

## 2.2 Ray 日志适配—ray\_wrappers.py

文件: ray\_wrappers.py

当 vLLM 运行在 Ray 上时, 需要将 Prometheus 指标桥接到 Ray 的 metrics 系统。该模块提供了三组 Wrapper 类:

```
graph TB
    subgraph Prometheus_API
        G[Gauge.set]
        C[Counter.inc]
        H[Histogram.observe]
    end

    subgraph Ray_Wrappers
        RG[RayGaugeWrapper]
    end
```

```

        RC[RayCounterWrapper]
        RH[RayHistogramWrapper]
    end

    subgraph Ray Metrics
        RMG[ray.metrics.Gauge]
        RMC[ray.metrics.Counter]
        RMH[ray.metrics.Histogram]
    end

    G --> RG --> RMG
    C --> RC --> RMC
    H --> RH --> RMH

    RSP[RaySpecDecodingProm] -.->|extends| SDP[SpecDecodingProm]
    RKP[RayKVConnectorProm] -.->|extends| KVP[KVConnectorProm]
    RPM[RayPerfMetricsProm] -.->|extends| PMP[PerfMetricsProm]

    RPSL[RayPrometheusStatLogger] -.->|overrides cls| PSL[PrometheusStatLogger]

    style Prometheus API fill:#e8f5e9
    style Ray Metrics fill:#fff3e0

```

关键实现特点:

1. 名称清洗 — OpenTelemetry 要求 metric 名只含 [a-zA-Z0-9\_], Ray Wrapper 自动替换非法字符:

```

# ray_wrappers.py:70-82
@staticmethod
def _get_sanitized_opentelemetry_name(name: str) -> str:
    return re.sub(r"[^a-zA-Z0-9_]", "_", name)

```
2. **ReplicaId** 自动注入 — 每个 Ray Serve replica 自动附加 ReplicaId 标签用于区分
3. 完整继承体系 — RayPrometheusStatLogger 通过覆写 `_gauge_cls`, `_counter_cls`, `_histogram_cls` 将所有指标切换为 Ray 实现

### 三、StatLoggerFactory / StatLoggerManager 详解

#### 3.1 类型别名体系

```

# loggers.py:39-41
PerEngineStatLoggerFactory = Callable[[VllmConfig, int], "StatLoggerBase"]
AggregateStatLoggerFactory = type["AggregateStatLoggerBase"]
StatLoggerFactory = AggregateStatLoggerFactory | PerEngineStatLoggerFactory

```

两种 Factory 的区别: - **PerEngineStatLoggerFactory**: 接收 (config, engine\_index), 创建单引擎 logger  
- **AggregateStatLoggerFactory**: 接收 (config, engine\_indexes), 创建跨引擎聚合 logger

#### 3.2 插件加载机制

```

# loggers.py:74-88
def load_stat_logger_plugin_factories() -> list[StatLoggerFactory]:
    """ 通过 entry_points 加载第三方 StatLogger 插件。 """
    factories: list[StatLoggerFactory] = []
    for name, plugin_class in load_plugins_by_group(STAT_LOGGER_PLUGINS_GROUP).items():

```

```

    if not issubclass(plugin_class, StatLoggerBase):
        raise TypeError(f"Stat logger plugin {name!r} must be a subclass of StatLoggerBase")
    factories.append(plugin_class)
return factories

```

用户可通过 Python entry\_points 注册自定义 StatLogger，实现指标导出到任意后端（如 Datadog、CloudWatch 等）。

### 3.3 StatLoggerManager 初始化流程

sequenceDiagram

```

participant AsyncLLM
participant Manager as StatLoggerManager
participant Plugin as Custom Plugins
participant Default as Default Loggers
participant Prometheus as PrometheusStatLogger

```

```

AsyncLLM->>Manager: __init__(vllm_config, engine_idx, custom_factories)

```

alt 自定义插件存在

```

    Manager->>Plugin: 遍历 custom_stat_loggers

```

```

    Plugin-->>Manager: StatLoggerFactory

```

end

alt 启用默认日志 & client\_count==1

```

    Manager->>Default: 添加 AggregatedLoggingStatLogger 或 LoggingStatLogger

```

end

```

Manager->>Prometheus: 无自定义 Prometheus 时自动添加

```

```

Note over Manager: stat_loggers 列表构建完成

```

## 四、OpenTelemetry 集成

### 4.1 OTel 初始化—otel.py

文件: otel.py

vLLM 通过 OpenTelemetry 实现分布式追踪 (Distributed Tracing)，支持 gRPC 和 HTTP/protobuf 两种协议:

```

# otel.py:60-91
def init_otel_tracer(
    instrumenting_module_name: str,
    otlp_traces_endpoint: str,
    extra_attributes: dict[str, str] | None = None,
) -> Tracer:
    os.environ["OTEL_EXPORTER_OTLP_TRACES_ENDPOINT"] = otlp_traces_endpoint
    resource_attrs = {
        "vllm.instrumenting_module_name": instrumenting_module_name,
        "vllm.process_id": str(os.getpid()),
    }
    resource = Resource.create(resource_attrs)
    trace_provider = TracerProvider(resource=resource)
    span_exporter = get_span_exporter(otlp_traces_endpoint)
    trace_provider.add_span_processor(BatchSpanProcessor(span_exporter))

```

```

set_tracer_provider(trace_provider)
atexit.register(trace_provider.shutdown)
return trace_provider.get_tracer(instrumenting_module_name)

```

协议选择 (L94-102):

```

def get_span_exporter(endpoint):
    protocol = os.environ.get(OTEL_EXPORTER_OTLP_TRACES_PROTOCOL, "grpc")
    if protocol == "grpc":
        return OTLPGrpcExporter(endpoint=endpoint, insecure=True)
    elif protocol == "http/protobuf":
        return OTLPHttpExporter(endpoint=endpoint)

```

## 4.2 自动化 Instrumentation

`instrument_otel()` 函数提供装饰器方式的自动埋点:

*# otel.py:134-180*

```

def instrument_otel(func, span_name, attributes, record_exception):
    code_attrs = {
        LoadingSpanAttributes.CODE_FUNCTION: func.__qualname__,
        LoadingSpanAttributes.CODE_NAMESPACE: func.__module__,
        LoadingSpanAttributes.CODE_FILEPATH: func.__code__.co_filename,
        LoadingSpanAttributes.CODE_LINENO: str(func.__code__.co_firstlineno),
    }
    # 同时支持 sync 和 async 函数
    async_wrapper / sync_wrapper:
        with tracer.start_as_current_span(final_span_name, ..., propagate_trace_to_env()):
            return await/call func(*args, **kwargs)

```

## 4.3 Trace Context 传播

vLLM 实现了智能 **Context** 传播机制, 解决跨进程追踪问题:

graph TD

```

A[Client Request<br/>with traceparent header] --> B[API Server<br/>extract_trace_context]
B --> C[instrument_otel wrapper<br/>propagate_trace_to_env]
C --> |inject to os.environ| D[Worker Process<br/>get_smart_context]
D --> E{有活跃 Span?}
E --> |Yes| F[直接使用当前 Context]
E --> |No| G{os.environ 有 traceparent?}
G --> |Yes| H[从环境变量提取]
G --> |No| I[尝试全部环境变量]

```

```

style C fill:#e1bee7
style D fill:#e1bee7

```

关键实现 (L216-237):

```

def _get_smart_context() -> Context | None:
    current_span = trace.get_current_span()
    if current_span.get_span_context().is_valid:
        return None # 已有活跃 Span, 无需额外 context
    carrier = {}
    if tp := os.environ.get("traceparent", os.environ.get("TRACEPARENT")):
        carrier["traceparent"] = tp
    if ts := os.environ.get("tracestate", os.environ.get("TRACESTATE")):

```

```

        carrier["tracestate"] = ts
    return TraceContextTextMapPropagator().extract(carrier)

```

`propagate_trace_to_env()` context manager 确保子进程继承正确的 **traceparent** (L240-263)。

#### 4.4 Span 属性标准—`tracing/utils.py`

文件: `tracing/utils.py`

定义了符合 **OpenTelemetry Semantic Conventions** 的 Span 属性常量:

```

# tracing/utils.py:15-46
class SpanAttributes:
    # GenAI 标准属性
    GEN_AI_USAGE_COMPLETION_TOKENS = "gen_ai.usage.completion_tokens"
    GEN_AI_USAGE_PROMPT_TOKENS = "gen_ai.usage.prompt_tokens"
    GEN_AI_REQUEST_ID = "gen_ai.request.id"
    GEN_AI_RESPONSE_MODEL = "gen_ai.response.model"

    # 延迟细分属性
    GEN_AI_LATENCY_TIME_TO_FIRST_TOKEN = "gen_ai.latency.time_to_first_token"
    GEN_AI_LATENCY_E2E = "gen_ai.latency.e2e"
    GEN_AI_LATENCY_TIME_IN_QUEUE = "gen_ai.latency.time_in_queue"
    GEN_AI_LATENCY_TIME_IN_MODEL_PREFILL = "gen_ai.latency.time_in_model_prefill"
    GEN_AI_LATENCY_TIME_IN_MODEL_DECODE = "gen_ai.latency.time_in_model_decode"
    GEN_AI_LATENCY_TIME_IN_MODEL_FORWARD = "gen_ai.latency.time_in_model_forward"
    GEN_AI_LATENCY_TIME_IN_MODEL_EXECUTE = "gen_ai.latency.time_in_model_execute"

class LoadingSpanAttributes:
    CODE_NAMESPACE = "code.namespace"
    CODE_FUNCTION = "code.function"
    CODE_FILEPATH = "code.filepath"
    CODE_LINENO = "code.lineno"

```

支持的 W3C Trace Context Headers: `traceparent`, `tracestate`。

## 五、Profiling 性能分析

### 5.1 Profiler 包装器体系—`wrapper.py`

文件: `wrapper.py`

vLLM 提供了两层 Profiler 抽象, 均继承自 `WorkerProfiler` 基类:

```

classDiagram
    class WorkerProfiler {
        <<abstract>>
        #_delay_iters: int
        #_max_iters: int
        #_active: bool
        #_running: bool
        +start()
        +step()
        +stop()
        +shutdown()
        +annotate_context_manager(name)
    }

```

```

}

class TorchProfilerWrapper {
  -profiler: torch.profiler.profile
  -_uses_schedule: bool
  +_start(): profiler.start()
  +_stop(): profiler.stop() + dump table
  +_profiler_step(): profiler.step()
  +annotate_context_manager(): record_function()
}

class CudaProfilerWrapper {
  -_cuda_profiler
  +_start(): cuda_profiler.start()
  +_stop(): cuda_profiler.stop()
  +annotate_context_manager(): nvtx.range()
}

WorkerProfiler <|-- TorchProfilerWrapper
WorkerProfiler <|-- CudaProfilerWrapper

```

**WorkerProfiler** 生命周期状态机:

```

stateDiagram-v2
  [*] --> Idle
  Idle --> Active: start()
  Active --> DelayedWaiting: delay_iters > 0
  DelayedWaiting --> Running: 达到 delay_iters
  Active --> Running: delay_iters == 0
  Running --> Running: step() [profiling]
  Running --> Stopped: max_iters 达到 or stop()
  Active --> Stopped: stop()
  Stopped --> [*]: shutdown()

  note right of Running
    每步调用 _profiler_step()
    schedule 模式下区分 warmup/active
  end note

```

**TorchProfilerWrapper** 支持的关键功能: - **Schedule-based profiling**: 配置 wait/warmup/active 循环 (L201-209) - **TensorBoard** 导出: 默认使用 `tensorboard_trace_handler` (L192-196) - 表格输出: 停止时自动 dump CUDA/CPU time 表格 (L269-287)

## 5.2 逐层性能分析—`layerwise_profile.py`

文件: `layerwise_profile.py`

基于 PyTorch Kineto 的模型层级性能剖析器, 生成两种视图:

```

graph TB
  subgraph 输入
    K[kineto_results<br/> 原始 profiler 数据]
  end

  subgraph 处理
    C1[_build_correlation_map<br/>correlation_id → events]
    C2[_build_module_tree<br/> 按模块层次组织事件]
  end

```



```

    C3a[_build_stats_tree → SummaryStatsEntry<br/> 按操作名聚合]
    C3b[_build_stats_tree → ModelStatsEntry<br/> 保留模块层级]
end

subgraph 输出
    01[print_summary_table<br/> 按操作汇总 CUDA 时间]
    02[print_model_table<br/> 按模块层级展示 CPU/CUDA 时间]
    03[export_*_csv<br/> 导出 CSV]
    04[convert_stats_to_dict<br/> 导出 JSON]
end

K --> C1 --> C2
C2 --> C3a --> 01
C2 --> C3b --> 02
C3a --> 03
C3b --> 03
C3a --> 04
C3b --> 04

```

**LayerwiseProfileResults** 继承自 `torch.profiler.profile`, 在 `__exit__` 时自动构建结果:

```

# layerwise_profile.py:373-400
class layerwise_profile(profile):
    def __init__(self, num_running_seqs=None):
        super().__init__(
            activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
            record_shapes=True,
            with_stack=True,
            with_modules=True,
            experimental_config=_ExperimentalConfig(verbose=True),
        )

    def __exit__(self, exc_type, exc_val, exc_tb):
        super().__exit__(exc_type, exc_val, exc_tb)
        self.results = LayerwiseProfileResults(
            self.profiler.kineto_results,
            num_running_seqs=self.num_running_seqs,
        )

```

**SummaryStatsEntry** 一按操作名去重聚合:

```

# layerwise_profile.py:55-60
@dataclass
class SummaryStatsEntry:
    name: str          # 操作名称 (如 "aten::matmul")
    cuda_time_us: float # 累积 CUDA 时间 (微秒)
    pct_cuda_time: float # 占总 CUDA 时间百分比
    invocations: int    # 调用次数

```

**ModelStatsEntry** 一保留模块层级信息:

```

# layerwise_profile.py:63-69
@dataclass
class ModelStatsEntry:
    name: str          # 模块名称 (如 "LlamaDecoderLayer(idx=12)")
    cpu_time_us: float # CPU 时间
    cuda_time_us: float # CUDA 时间

```

```
pct_cuda_time: float    # 占比
trace: str              # PyTorch 操作调用栈
```

### 5.3 Profiler 工具函数—utils.py

文件: utils.py

提供事件分析和表格打印的工具函数集:

| 函数                                | 功能                             |
|-----------------------------------|--------------------------------|
| event_has_module(event)           | 判断事件是否关联到 Python 模块            |
| event_is_torch_op(event)          | 判断事件是否为 PyTorch 原生操作           |
| event_module_repr(event)          | 格式化为 ClassName(param=val, ...) |
| event_torch_op_repr(event)        | 格式化为 op_name(args)             |
| event_torch_op_stack_trace(event) | 向上遍历获取 PyTorch 操作调用链           |
| TablePrinter                      | 通用的 dataclass 表格打印机            |

### 5.4 NVTX Hooks —nvtx\_pytorch\_hooks.py

文件: nvtx\_pytorch\_hooks.py

利用 NVIDIA NVTX (NVIDIA Tools Extension) 在 PyTorch 模型的每一层自动插入 **range marker**, 使得在 Nsight Systems / Nsight Compute 中可以清晰看到每层的执行情况。

**PytHooks** 类—自动注册 forward hook:

```
# nvtx_pytorch_hooks.py:216-286
class PytHooks:
    def register_hooks(self, network_model, module_prefix="top"):
        skip_types = (torch.nn.Identity, torch.nn.Dropout, ...)
        for name, module in network_model.named_modules(prefix=module_prefix):
            if isinstance(module, skip_types):
                continue
            module.register_forward_pre_hook(self.module_fwd_pre_hook, with_kwargs=True)
            module.register_forward_hook(self.module_fwd_hook)
```

**Marker** 信息内容—每次 push 包含丰富的调试信息:

```
# nvtx_pytorch_hooks.py:138-170
def construct_marker_dict_and_push(module_name, module_obj, in_tensor, kwargs, out_tensor):
    marker_dict = {
        "Module": module_name,
        "TrainableParams": {name: list(size) for name, param in ...},
        "Inputs": tensor_dims_list,          # 输入张量形状
        "Outputs": tensor_dims_list,         # 输出张量形状
        "StaticParams": {                    # 层参数 (如 Linear 的 in/out_features)
            "in_features": ...,
            "out_features": ...,
        },
        **kwargs,                             # 顶层额外参数 (如 input_ids)
    }
    nvtx.range_push("{}".format(marker_dict))
```

**Context Manager** 方式一也支持手动包裹单次前向传播:

```
# nvtx_pytorch_hooks.py:179-213
@contextmanager
```

```
def layerwise_nvtx_marker_context(module_name, module_obj, in_tensor=None, kwargs=None):
    holder = ResultHolder()
    construct_marker_dict_and_push(...) # push input marker
    try:
        yield holder # 用户在此执行 forward
    finally:
        nvtx.range_pop() # pop input marker
        construct_marker_dict_and_push(output_name, ..., out_tensor=holder.result) # push output
        nvtx.range_pop() # pop output marker
```

注意: NVTX tracing 与 CUDA graphs 不兼容 (enable\_layerwise\_nvtx\_tracing 在 CUDA graph 启用时无效)。

## 六、Usage Context & Usage Lib

### 6.1 使用情况追踪—usage\_lib.py

文件: usage\_lib.py

vLLM 内置匿名的使用情况统计 (Usage Stats) 系统, 用于了解社区如何使用 vLLM。

#### 6.1.1 启用/禁用控制

```
# usage_lib.py:51-69
def is_usage_stats_enabled():
    """ 三个维度的禁用方式 (任一即禁用):
    1. 环境变量 VLLM_DO_NOT_TRACK=1
    2. 环境变量 DO_NOT_TRACK=1
    3. 环境变量 VLLM_NO_USAGE_STATS=1
    4. 文件 $HOME/.config/vllm/do_not_track 存在
    """
```

#### 6.1.2 UsageContext 枚举

```
# usage_lib.py:112-119
class UsageContext(str, Enum):
    UNKNOWN_CONTEXT = "UNKNOWN_CONTEXT"
    LLM_CLASS = "LLM_CLASS" # 直接使用 LLM 类
    API_SERVER = "API_SERVER" # 通用 API Server
    OPENAI_API_SERVER = "OPENAI_API_SERVER" # OpenAI 兼容 Server
    OPENAI_BATCH_RUNNER = "OPENAI_BATCH_RUNNER" # Batch 推理
    ENGINE_CONTEXT = "ENGINE_CONTEXT" # 纯引擎模式
```

#### 6.1.3 收集的信息维度 UsageMessage 收集的信息包括:

| 类别  | 字段                                            | 示例                 |
|-----|-----------------------------------------------|--------------------|
| 平台  | provider, architecture, platform              | AWS, x86_64, Linux |
| 硬件  | gpu_count, gpu_type,<br>gpu_memory_per_device | 8, H100, 80GB      |
| CPU | num_cpu, cpu_type                             | 128, AMD EPYC 9534 |
| 运行时 | cuda_runtime, xpu_runtime                     | 12.4               |
| 模型  | model_architecture,<br>vllm_version           | llama3_8b, 0.6.3   |

| 类别   | 字段              | 示例                               |
|------|-----------------|----------------------------------|
| 环境变量 | env_var_json    | VLLM_USE_FLASHINFER_SAMPLER<br>等 |
| 上下文  | context, source | API_SERVER                       |

6.1.4 上报机制

```
sequenceDiagram
    participant User
    participant UsageMsg as UsageMessage
    participant Thread as Daemon Thread
    participant Server as Usage Server
    participant File as usage_stats.json

    User->>UsageMsg: report_usage(model_arch, UsageContext)
    UsageMsg->>Thread: 启动 daemon thread
    Thread->>UsageMsg: _report_usage_once()
    UsageMsg->>Server: POST (一次性上报)
    UsageMsg->>File: JSON append
    loop 每 10 分钟
        Thread->>Server: POST (持续心跳)
        Thread->>File: JSON append
    end
end
```

云平台检测 (L76-109) 通过 DMI 文件和环境变量识别 AWS/Azure/GCP/OCI/RUNPOD。

七、Observability Config

7.1 配置定义—observability.py

文件: observability.py

ObservabilityConfig 是所有可观测性功能的统一配置入口:

```
graph TB
    OC[ObservabilityConfig] --> O1[otlp_traces_endpoint<br/>OTLP 目标 URL]
    OC --> O2[collect_detailed_traces<br/>详细追踪模块列表]
    OC --> O3[kv_cache_metrics<br/>KV Cache 驻留指标]
    OC --> O4[cudagraph_metrics<br/>CUDA Graph 指标]
    OC --> O5[enable_mfu_metrics<br/>MFU 指标]
    OC --> O6[enable_layerwise_nvtx_tracing<br/>逐层 NVTX]
    OC --> O7[show_hidden_metrics<br/>显示已废弃指标]
    OC --> O8[enable_mm_processor_stats<br/>多模态处理器统计]
    OC --> O9[enable_logging_iteration_details<br/>迭代详情日志]

    O2 -->| 校验 | O1
    O1 -->| 依赖 OTel 库 | VAL{验证}

    style OC fill:#fff9c4
    style VAL fill:#ffcdd2
```

7.1.1 配置项详解

| 配置项                             | 类型           | 默认值   | 影响范围               | 开销                 |
|---------------------------------|--------------|-------|--------------------|--------------------|
| otlp_traces_endpoint            | str \   None | None  | OpenTelemetry 追踪   | 低（异步导出）            |
| collect_detailed_traces         | str \   None | None  | 请求级延迟细分            | 中（需 otlp endpoint） |
| kv_cache_metrics                | bool         | False | KV Cache 生命周期直方图   | 低（采样 1%）           |
| cuda_graph_metrics              | bool         | False | CUDA Graph 效率统计    | 极低                 |
| enable_mfu_metrics              | bool         | False | MFU / 内存带宽         | 低                  |
| enable_layerwise_nvtx_profiling | bool         | False | NVTX range markers | 中（每层 hook）         |
| show_hidden_metrics_for_version | str \ None   | None  | Prometheus 废弃指标恢复  | 无                  |
| kv_cache_metrics_sample_rate    | float        | 0.01  | KV Cache 采样率       | 控制开销               |

7.1.2 校验规则

- 1. 版本号校验: show\_hidden\_metrics\_for\_version 必须是合法语义化版本
- 2. OTel 依赖校验: 设置 otlp\_traces\_endpoint 时必须已安装 OpenTelemetry 库
- 3. 联动校验: collect\_detailed\_traces 要求 otlp\_traces\_endpoint 已设置
- 4. 兼容性处理: collect\_detailed\_traces 支持逗号分隔字符串的旧格式

7.1.3 衍生属性

```
# observability.py:79-93
@cached_property
def collect_model_forward_time(self) -> bool:
    """ 是否收集 model forward 时间（影响请求级别 Span）。"""
    return self.collect_detailed_traces is not None and (
        "model" in self.collect_detailed_traces or "all" in self.collect_detailed_traces
    )

@cached_property
def collect_model_execute_time(self) -> bool:
    """ 是否收集 model execute 时间（worker 级别）。"""
    return self.collect_detailed_traces is not None and (
        "worker" in self.collect_detailed_traces or "all" in self.collect_detailed_traces
    )
```

八、指标收集与导出架构全景

```
graph TB
    subgraph 数据源 [" 数据采集层"]
        S1[SchedulerOutput<br/> 调度结果]
        S2[EngineCoreOutput<br/> 引擎输出]
        S3[PrefillStats<br/>Prefill 细分]
        S4[KVCacheEvictionEvent<br/> 驱逐事件]
        S5[ModelMetrics<br/>FLOPs/带宽估算]
    end

    subgraph 聚合 [" 聚合层"]
        IS[IterationStats<br/> 迭代统计]
        SS[SchedulerStats<br/> 调度统计]
        PS[PerfStats<br/> 性能快照]
        CS[CachingMetrics<br/> 缓存命中率]
    end
```

```

subgraph 记录 [" 记录层"]
    LSL[LoggingStatLogger<br/> 标准输出]
    ALSL[AggregatedLoggingStatLogger<br/> 聚合日志]
    PSL[PrometheusStatLogger<br/>Prometheus 导出]
    RAY[RayPrometheusStatLogger<br/>Ray 适配]
    PLUGIN[Custom Plugins<br/> 第三方插件]
end

subgraph 导出 [" 导出层"]
    STDOUT[Console / File]
    PROM[/metrics 端点]
    RAY_METRICS[Ray Metrics System]
    EXTERNAL[外部监控系统]
end

S1 --> SS
S2 --> IS
S3 --> IS
S4 --> SS
S5 --> PS

IS & SS & PS & CS --> LSL
IS & SS & PS & CS --> ALSL
IS & SS & PS & CS --> PSL
IS & SS & PS & CS --> RAY
IS & SS & PS & CS --> PLUGIN

LSL & ALSL --> STDOUT
PSL --> PROM
RAY --> RAY_METRICS
PLUGIN --> EXTERNAL

SLM[StatLoggerManager] -.->| 统一调度 | LSL & ALSL & PSL & RAY & PLUGIN

style 数据源 fill:#e3f2fd
style 聚合 fill:#e8f5e9
style 记录 fill:#fff3e0
style 导出 fill:#fce4ec

```

核心数据流路径

路径 1: 延迟指标 (TTFT / ITL / E2E)

```

EngineCoreOutput → IterationStats.update_from_output()
    → time_to_first_tokens_iter[] (TTFT)
    → inter_token_latencies_iter[] (ITL)
    → FinishedRequestStats (e2e_latency, prefill_time, decode_time)
    → PrometheusStatLogger.record() → histogram_ttft / histogram_itl / histogram_e2e

```

路径 2: 吞吐量指标 (Prompt/Generation tokens/s)

```

IterationStats → LoggingStatLogger._track_iteration_stats()
    → num_prompt_tokens (仅 computed tokens)
    → num_generation_tokens
    → LoggingStatLogger.log() → throughput = tokens / delta_time

```

路径 3: MFU / 带宽指标

```
SchedulerOutput → ModelMetrics.get_step_perf_stats_per_gpu()
  → ExecutionContext (prefill/decode 分类)
  → ComponentMetrics.get_num_flops/get_read_bytes/get_write_bytes
  → PerfStats (flops, read_bytes, write_bytes)
  → PerfMetricsProm.observe() → counter_flops/read/write
  → PerfMetricsLogging.observe() → TFLOPS/s, GB/s
```

路径 4: 缓存命中率

```
SchedulerOutput.prefix_cache_stats → CachingMetrics.observe()
  → 滑动窗口 (最近 N 请求) → hit_rate
  → LoggingStatLogger.log() → "Prefix cache hit rate: X%"
  → PrometheusStatLogger.record() → counter_prefix_cache_queries/hits
```

附录: 关键文件索引

| 文件路径                          | 核心职责                               | 行数    |
|-------------------------------|------------------------------------|-------|
| v1/metrics/perf.py            | MFU/FLOPs/带宽估算, 组件化 Parser Chain   | ~1351 |
| v1/metrics/stats.py           | 运行时统计数据结构定义                        | ~539  |
| v1/metrics/prometheus.py      | Prometheus 多进程基础设施                 | ~82   |
| v1/metrics/reader.py          | 编程式指标读取 API                        | ~257  |
| v1/metrics/utils.py           | 多引擎指标创建工具                          | ~19   |
| v1/metrics/loggers.py         | StatLogger 工厂体系与管理器                | ~1359 |
| v1/metrics/ray_wrappers.py    | Ray Metrics 适配层                    | ~216  |
| tracing/otel.py               | OpenTelemetry 初始化与 Instrumentation | ~263  |
| tracing/utils.py              | Span 属性常量与 Context 工具              | ~72   |
| profiler/layerwise_profile.py | 逐层性能分析 (Kineto)                    | ~400  |
| profiler/wrapper.py           | Torch/CUDA Profiler 包装器            | ~328  |
| profiler/utils.py             | Profiler 事件分析与表格打印                 | ~150  |
| utils/nvtx_pytorch_hooks.py   | NVTX 自动 Hook 注入                    | ~286  |
| usage/usage_lib.py            | 匿名使用情况统计                           | ~282  |
| config/observability.py       | 可观测性统一配置                           | ~152  |

vLLM 设计模式汇总与架构洞察

定位: 本文档是 vLLM 源码分析系列的收官之作, 从设计模式与架构视角对前 20 个分模块文档进行归纳提升。旨在揭示 vLLM 代码库中蕴含的工程智慧, 为理解其架构决策、扩展机制和性能优化策略提供系统性总结。

```
graph TB
  subgraph 设计模式 [" 核心设计模式 (9 种)"]
    SP[策略模式 <br/>Strategy]
    FP[工厂模式 <br/>Factory]
    OP[观察者模式 <br/>Observer]
    RP[注册表模式 <br/>Registry]
    DP[装饰器模式 <br/>Decorator]
    AP[适配器模式 <br/>Adapter]
    BP[建造者模式 <br/>Builder]
```

```

    PP[原型模式 <br/>Prototype]
    TMP[模板方法 <br/>Template Method]
end

subgraph 架构洞察 [" 架构洞察"]
    A1[代码组织原则]
    A2[架构优势与权衡]
    A3[可扩展性设计]
    A4[性能优化策略]
    A5[竞品对比视角]
    A6[未来发展方向]
end

SP --> A1
FP --> A3
RP --> A3
OP --> A2
DP --> A1
AP --> A3
BP --> A1
PP --> A4
TMP --> A1

style 设计模式 fill:#f8bbd0,stroke:#e91e63,stroke-width:3px,color:#000
style 架构洞察 fill:#e1bee7,stroke:#9c27b0,stroke-width:3px,color:#000
style SP fill:#fce4ec
style FP fill:#fce4ec
style OP fill:#fce4ec
style RP fill:#fce4ec
style DP fill:#fce4ec
style AP fill:#fce4ec
style BP fill:#fce4ec
style PP fill:#fce4ec
style TMP fill:#fce4ec

```

---

## 一、设计模式清单（9 种核心模式）

### 1. 策略模式 (Strategy Pattern)

核心思想：定义一系列算法，将每个算法封装到具有共同接口的独立类中，使它们可以相互替换。

**1.1 注意力后端选择** vLLM 最典型的策略模式应用。根据硬件能力和模型配置，在运行时动态选择最优的注意力计算实现。

源码位置：selector.py

```

def get_attn_backend(
    head_size: int,
    dtype: torch.dtype,
    kv_cache_dtype: str | None,
    use_mla: bool = False,
    has_sink: bool = False,
    use_sparse: bool = False,
    # ... 更多参数

```



```

) -> type[AttentionBackend]:
    """Selects which attention backend to use and lazily imports it."""

    vllm_config = get_current_vllm_config()
    attn_selector_config = AttentionSelectorConfig(
        head_size=head_size,
        dtype=dtype,
        # ... 构建配置
    )

    return _cached_get_attn_backend(
        backend=vllm_config.attention_config.backend,
        attn_selector_config=attn_selector_config,
        num_heads=num_heads,
    )

```

策略接口定义: backend.py

```

class AttentionBackend(ABC):
    """Abstract class for attention backends."""

    supported_dtypes: ClassVar[list[torch.dtype]] = [torch.float16, torch.bfloat16]

    @staticmethod
    @abstractmethod
    def get_name() -> str:
        raise NotImplementedError

    @staticmethod
    @abstractmethod
    def get_impl_cls() -> type["AttentionImplBase"]:
        raise NotImplementedError

```

具体策略实现包括: - FlashAttentionBackend - NVIDIA GPU 上的 Flash Attention 实现 - XFormersBackend  
 - 基于 xformers 的注意力实现 - TorchNaiveBackend - PyTorch 原生实现 (fallback) - RopeAttentionBackend  
 - 支持 RoPE 的变体

**1.2 调度策略选择** Scheduler 支持多种调度算法 (FCFS vs PRIORITY), 通过配置切换:

相关文档: 04\_scheduler.md 中详细描述了调度器的策略选择机制。

**1.3 线性层后端选择** 量化 GEMM 的不同内核选择 (Marlin、GPTQ、AWQ 等), 根据量化类型自动选择最优内核实现。

---

## 2. 工厂模式 (Factory Pattern)

核心思想: 将对象的创建逻辑封装起来, 客户端不需要知道具体的创建细节。

**2.1 Executor 创建工厂** 源码位置: abstract.py

```

class Executor(ABC):
    """Abstract base class for vLLM executors."""

    uses_ray: bool = False
    supports_pp: bool = False

```

```

@staticmethod
def get_class(vllm_config: VllmConfig) -> type["Executor"]:
    executor_class: type[Executor]
    parallel_config = vllm_config.parallel_config
    distributed_executor_backend = parallel_config.distributed_executor_backend

    if isinstance(distributed_executor_backend, type):
        executor_class = distributed_executor_backend
    elif distributed_executor_backend == "ray":
        if envs.VLLM_USE_RAY_V2_EXECUTOR_BACKEND:
            from vllm.v1.executor.ray_executor_v2 import RayExecutorV2
            executor_class = RayExecutorV2
        else:
            from vllm.v1.executor.ray_executor import RayDistributedExecutor
            executor_class = RayDistributedExecutor
    elif distributed_executor_backend == "mp":
        from vllm.v1.executor.multiproc_executor import MultiprocExecutor
        executor_class = MultiprocExecutor
    elif distributed_executor_backend == "uni":
        from vllm.v1.executor.uniproc_executor import UniProcExecutor
        executor_class = UniProcExecutor
    # ... 更多分支
    return executor_class

```

工厂创建的产品层级: | 配置值 | 创建的 Executor 类 | 用途 | |——|———|——| | "ray" | RayDistributedExecutor / RayExecutorV2 | Ray 分布式执行 | | "mp" | MultiprocExecutor | 多进程执行 | | "uni" | UniProcExecutor | 单进程执行 |

## 2.2 Weight Transfer 连接器工厂 源码位置: factory.py

```

class WeightTransferEngineFactory:
    """Factory for creating weight transfer engines with lazy loading."""

    _registry: dict[str, Callable[[], type[WeightTransferEngine]]] = {}

    @classmethod
    def register_engine(cls, name, module_path_or_cls, class_name=None):
        if isinstance(module_path_or_cls, str):
            def loader() -> type[WeightTransferEngine]:
                module = importlib.import_module(module_path)
                return getattr(module, class_name)
            cls._registry[name] = loader
        else:
            engine_cls = module_path_or_cls
            cls._registry[name] = lambda: engine_cls

    @classmethod
    def create_engine(cls, config, parallel_config):
        backend = config.backend
        engine_cls = cls._registry[backend]()
        return engine_cls(config, parallel_config)

```

# 注册内置引擎

```

WeightTransferEngineFactory.register_engine(
    "nccl", "vllm.distributed.weight_transfer.nccl_engine", "NCCLWeightTransferEngine"
)
WeightTransferEngineFactory.register_engine(
    "ipc", "vllm.distributed.weight_transfer.ipc_engine", "IPCWeightTransferEngine"
)

```

## 2.3 KV Offload 工厂 源码位置: factory.py

```

class OffloadingSpecFactory:
    _registry: dict[str, Callable[[], type[OffloadingSpec]]] = {}

    @classmethod
    def register_spec(cls, name, module_path, class_name):
        def loader() -> type[OffloadingSpec]:
            module = importlib.import_module(module_path)
            return getattr(module, class_name)
        cls._registry[name] = loader

    @classmethod
    def create_spec(cls, config, kv_cache_config):
        spec_name = extra_config.get("spec_name", "CPUOffloadingSpec")
        spec_cls = cls._registry[spec_name]()
        return spec_cls(config, kv_cache_config)

```

```

OffloadingSpecFactory.register_spec(
    "CPUOffloadingSpec", "vllm.v1.kv_offload.cpu.spec", "CPUOffloadingSpec"
)

```

**2.4 Worker 创建** 根据设备类型 (GPU/CPU/TPU/XPU) 和运行模式创建不同的 Worker 实现: - GPUWorker - GPU 设备 worker - CPUWorker - CPU 设备 worker - TPUPWorker - TPU 设备 worker - XPUWorker - XPU 设备 worker

## 3. 观察者模式 (Observer Pattern)

核心思想: 定义对象间一对多的依赖关系, 当一个对象状态改变时, 所有依赖它的对象都会收到通知并自动更新。

### 3.1 指标收集系统 源码位置: loggers.py

```

class StatLoggerBase(ABC):
    """Interface for logging metrics."""

    @abstractmethod
    def __init__(self, vllm_config: VllmConfig, engine_index: int = 0): ...

    @abstractmethod
    def record(
        self,
        scheduler_stats: SchedulerStats | None,
        iteration_stats: IterationStats | None,
        mm_cache_stats: MultiModalCacheStats | None = None,
        engine_idx: int = 0,
    ): ...

```

```

): ...

@abstractmethod
def log_engine_initialized(self): ...

```

观察者层级结构:

```

classDiagram
    class StatLoggerBase {
        <<abstract>>
        +record()
        +log()
        +log_engine_initialized()
    }
    class AggregateStatLoggerBase {
        <<abstract>>
        +__init__(engine_indexes)
    }
    class LoggingStatLogger {
        +record()
        +log()
    }
    class PrometheusStatLogger {
        +record()
        +log()
    }
    class AggregatedLoggingStatLogger {
        +aggregate_scheduler_stats()
    }
    class PerEngineStatLoggerAdapter {
        +per_engine_stat_loggers
    }

    StatLoggerBase <|-- AggregateStatLoggerBase
    StatLoggerBase <|-- LoggingStatLogger
    StatLoggerBase <|-- PrometheusStatLogger
    AggregateStatLoggerBase <|-- AggregatedLoggingStatLogger
    AggregateStatLoggerBase <|-- PerEngineStatLoggerAdapter
    AggregatedLoggingStatLogger --|> LoggingStatLogger

```

具体观察者实现:- LoggingStatLogger - 输出到标准日志(控制台)- PrometheusStatLogger - 输出到 Prometheus metrics 端点 - AggregatedLoggingStatLogger - 多引擎聚合日志 - PerEngineStatLoggerAdapter - 按引擎分别记录

**Subject (被观察者) - StatLoggerManager:** `loggers.py#L1268-L1359`

```

class StatLoggerManager:
    def __init__(self, vllm_config, engine_idx, custom_stat_loggers=None, ...):
        self.stat_loggers: list[AggregateStatLoggerBase] = []
        # ... 初始化多个 logger

    def record(self, scheduler_stats, iteration_stats, mm_cache_stats, engine_idx):
        for stat_logger in self.stat_loggers:
            stat_logger.record(scheduler_stats, iteration_stats, mm_cache_stats, engine_idx)

    def log(self):

```

```

for logger in self.stat_loggers:
    logger.log()

```

**3.2 KV Event Publisher/Subscriber** KV Cache 相关事件的发布订阅机制，用于跨实例协调和状态同步。

## 4. 注册表模式 (Registry Pattern)

核心思想：维护一个全局或局部的注册表，通过键值映射来查找和获取对象实例。vLLM 大量使用此模式实现可扩展架构。

### 4.1 模型注册表 源码位置: registry.py

这是 vLLM 最核心的注册表之一，支持 200+ 模型架构的自动发现和加载。

```

@dataclass
class _ModelRegistry:
    models: dict[str, _BaseRegisteredModel] = field(default_factory=dict)

    def register_model(self, model_arch: str, model_cls: type[nn.Module] | str):
        """Register an external model to be used in vLLM."""
        if isinstance(model_cls, str):
            model = _LazyRegisteredModel(*model_cls.split(":"))
        elif isinstance(model_cls, type) and issubclass(model_cls, nn.Module):
            model = _RegisteredModel.from_model_cls(model_cls)
        self.models[model_arch] = model

    def resolve_model_cls(self, architectures, model_config):
        """Resolve model class from architecture names."""
        for arch in architectures:
            normalized_arch = self._normalize_arch(arch, model_config)
            model_cls = self._try_load_model_cls(normalized_arch)
            if model_cls is not None:
                return (model_cls, arch)

# 全局模型注册表实例
ModelRegistry = _ModelRegistry({
    model_arch: _LazyRegisteredModel(
        module_name=f"vllm.model_executor.models.{mod_relname}",
        class_name=cls_name,
    )
    for model_arch, (mod_relname, cls_name) in _VLLM_MODELS.items()
})

```

支持的模型类别 (7 大类):

| 类别                      | 字典变量                                         | 示例模型                              |
|-------------------------|----------------------------------------------|-----------------------------------|
| 文本生成                    | <code>_TEXT_GENERATION_MODELS</code>         | Llama, Qwen, Mistral, DeepSeek... |
| Embedding               | <code>_EMBEDDING_MODELS</code>               | BERT, GTE, Jina, Voyage...        |
| Late Interaction        | <code>_LATE_INTERACTION_MODELS</code>        | ColBERT, ColPali...               |
| Reward Model            | <code>_REWARD_MODELS</code>                  | InternLM2, Qwen2-RM...            |
| Token Classification    | <code>_TOKEN_CLASSIFICATION_MODELS</code>    | BertForTokenClassification...     |
| Sequence Classification | <code>_SEQUENCE_CLASSIFICATION_MODELS</code> | BertForSequenceClassification...  |

| 类别                   | 字典变量                                      | 示例模型                      |
|----------------------|-------------------------------------------|---------------------------|
| Multimodal           | <code>_MULTIMODAL_MODELS</code>           | LLaVA, Qwen2-VL, Phi3V... |
| Speculative Decoding | <code>_SPECULATIVE_DECODING_MODELS</code> | Medusa, Eagle, Eagle3...  |

## 4.2 多模态处理器注册表 源码位置: registry.py

```
class MultiModalRegistry:
    """A registry that dispatches data processing according to the model."""

    def register_processor(self, processor, *, info, dummy_inputs):
        """Register a multi-modal processor to a model class."""
        def wrapper(model_cls: N) -> N:
            model_cls._processor_factory = _ProcessorFactories(
                info=info, dummy_inputs=dummy_inputs, processor=processor,
            )
            return model_cls
        return wrapper

    def create_processor(self, model_config, *, tokenizer=None, cache=None):
        """Create a multi-modal processor for a specific model."""
        model_cls = self._get_model_cls(model_config)
        factories = model_cls._processor_factory
        ctx = self._create_processing_ctx(model_config, tokenizer)
        return factories.build_processor(ctx, cache=cache)
```

使用装饰器语法注册:

```
@MULTIMODAL_REGISTRY.register_processor(
    MyImageProcessor,
    info=MyProcessingInfoFactory(),
    dummy_inputs=MyDummyInputsBuilderFactory(),
)
class MyVisionModel(nn.Module):
    ...
```

## 4.3 Tokenizer 注册表 源码位置: registry.py

```
@dataclass
class _TokenizerRegistry:
    tokenizers: dict[str, tuple[str, str]] = field(default_factory=dict)

    def register(self, tokenizer_mode: str, module: str, class_name: str):
        self.tokenizers[tokenizer_mode] = (module, class_name)

    def load_tokenizer_cls(self, tokenizer_mode: str) -> type[TokenizerLike]:
        module, class_name = self.tokenizers[tokenizer_mode]
        return resolve_obj_by_qualname(f"{module}.{class_name}")
```

```
TokenizerRegistry = _TokenizerRegistry({
    mode: (f"vllm.tokenizers.{mod_relname}", cls_name)
    for mode, (mod_relname, cls_name) in _VLLM_TOKENIZERS.items()
})
```

# 内置 Tokenizer: `deepseek_v32`, `deepseek_v4`, `fasttokens`, `grok2`, `hf`, `kimi_audio`, `mistral`, `qwen_vl`

#### 4.4 渲染器注册表 源码位置: registry.py

```
@dataclass
class RendererRegistry:
    renderers: dict[str, tuple[str, str]] = field(default_factory=dict)

    def register(self, renderer_mode: str, module: str, class_name: str):
        self.renderers[renderer_mode] = (module, class_name)

    def load_renderer_cls(self, renderer_mode: str) -> type[BaseRenderer]:
        module, class_name = self.renderers[renderer_mode]
        return resolve_obj_by_qualname(f"{module}.{class_name}")

RENDERER_REGISTRY = RendererRegistry({
    mode: (f"vllm.renderers.{mod_relname}", cls_name)
    for mode, (mod_relname, cls_name) in _VLLM_RENDERERS.items()
})
# 内置 Renderer: deepseek_v32, deepseek_v4, fasttokens, grok2, hf, kimi_audio, mistral, qwen_vl, terrator
```

#### 4.5 Chat Template 注册表 源码位置: registry.py

```
_MODEL_TYPE_TO_CHAT_TEMPLATE_FALLBACK: dict[str, ChatTemplatePath] = {
    "blip-2": CHAT_TEMPLATES_DIR / "template_blip2.jinja",
    "chameleon": CHAT_TEMPLATES_DIR / "template_basic.jinja",
    "deepseek_ocr": CHAT_TEMPLATES_DIR / "template_deepseek_ocr.jinja",
    "fuyu": CHAT_TEMPLATES_DIR / "template_fuyu.jinja",
    "qwen": _get_qwen_chat_template_fallback, # 支持函数作为 fallback
    # ...
}

def register_chat_template_fallback_path(model_type: str, chat_template: ChatTemplatePath):
    _MODEL_TYPE_TO_CHAT_TEMPLATE_FALLBACK[model_type] = chat_template
```

注册表模式汇总图:

```
graph LR
    subgraph 注册表体系
        MR[ModelRegistry<br/>200+ 模型]
        MMR[MultiModalRegistry<br/>多模态处理器]
        TR[TokenizerRegistry<br/>8 种 Tokenizer]
        RR[RendererRegistry<br/>9 种渲染器]
        CTR[ChatTemplateRegistry<br/>Chat 模板]
        WTF[WeightTransferFactory<br/>权重传输]
        OSF[OffloadSpecFactory<br/>KV 卸载]
    end

    MR --- MMR
    MMR --- TR
    TR --- RR
    RR --- CTR

    style MR fill:#e3f2fd
    style MMR fill:#e8f5e9
    style TR fill:#fff3e0
    style RR fill:#fce4ec
```

```
style CTR fill:#f3e5f5
style WTF fill:#e0f7fa
style OSF fill:#fff8e1
```

---

## 5. 装饰器模式 (Decorator Pattern)

核心思想：动态地给对象添加额外的职责，比生成子类更为灵活。

**5.1 模型注册装饰器** 虽然 vLLM 使用显式字典注册而非传统 Python 装饰器语法进行模型注册，但多模态处理器的注册使用了经典的装饰器模式：

源码位置：multimodal/registry.py#L142-L174

```
def register_processor(self, processor, *, info, dummy_inputs):
    def wrapper(model_cls: N) -> N:
        if "_processor_factory" in model_cls.__dict__:
            logger.warning("Model class %s already has a multi-modal processor...")
            model_cls._processor_factory = _ProcessorFactories(
                info=info, dummy_inputs=dummy_inputs, processor=processor,
            )
        return model_cls
    return wrapper
```

**5.2 自定义 Op 注册** @CustomOp.register 装饰器用于注册自定义算子，支持插件扩展 CUDA kernel。

**5.3 配置验证装饰器** 多处使用装饰器进行参数验证、缓存、性能追踪等横切关注点的处理。

---

## 6. 适配器模式 (Adapter Pattern)

核心思想：将一个类的接口转换成客户期望的另一个接口，使原本因接口不兼容而不能一起工作的类可以协同工作。

**6.1 API 协议转换 - Anthropic API** 源码位置：protocol.py

Anthropic API 模块实现了完整的协议适配层，将 Anthropic 格式的请求/响应转换为 vLLM 内部格式：

```
class AnthropicMessagesRequest(BaseModel):
    """Anthropic Messages API request - 适配外部 API 格式"""
    model: str
    messages: list[AnthropicMessage]
    max_tokens: int
    temperature: float | None = None
    top_k: int | None = None
    top_p: float | None = None
    tool_choice: AnthropicToolChoice | None = None
    tools: list[AnthropicTool] | None = None
    # vLLM 特有字段（桥接内部系统）
    kv_transfer_params: dict[str, Any] | None = Field(default=None)
    chat_template_kwargs: dict[str, Any] | None = Field(default=None)

class AnthropicMessagesResponse(BaseModel):
    """Anthropic Messages API response - 适配响应格式"""
    id: str
```



```
type: Literal["message"] = "message"
role: Literal["assistant"] = "assistant"
content: list[AnthropicContentBlock]
stop_reason: Literal["end_turn", "max_tokens", "stop_sequence", "tool_use"] | None
```

适配器工作流程:

Anthropic Request → Protocol Adapter → OpenAI Internal Format → Engine → Response Adapter → Anthropic Response

6.2 渲染器适配 不同模型使用不同的输入渲染器（HuggingFace、Mistral、DeepSeek 等），渲染器作为适配层统一不同模型的输入格式差异:

源码位置: renderers/registry.py

```
def renderer_from_config(config: "VllmConfig", **kwargs):
    model_config = config.model_config
    tokenizer = cached_tokenizer_from_config(model_config, **kwargs)
    renderer_mode, *_ = tokenizer_args_from_config(model_config, **kwargs)
    return RENDERER_REGISTRY.load_renderer(renderer_mode, config, tokenizer)
```

7. 建造者模式 (Builder Pattern)

核心思想: 将复杂对象的构建过程与其表示分离，使得同样的构建过程可以创建不同的表示。

7.1 VllmConfig 配置构建 VllmConfig 是 vLLM 最复杂的配置对象，聚合了 28+ 个子配置模块，采用逐步构建方式组装:

配置模块列表 (来自 config/ 目录):

| 配置模块                    | 文件                    | 职责              |
|-------------------------|-----------------------|-----------------|
| VllmConfig              | vllm.py               | 顶层聚合配置          |
| ModelConfig             | model.py              | 模型相关配置          |
| CacheConfig             | cache.py              | KV Cache 配置     |
| ParallelConfig          | parallel.py           | 并行配置 (TP/PP/DP) |
| SchedulerConfig         | scheduler.py          | 调度器配置           |
| DeviceConfig            | device.py             | 设备配置            |
| LoadConfig              | load.py               | 模型加载配置          |
| LoRAConfig              | lora.py               | LoRA 适配器配置      |
| QuantizationConfig      | quantization.py       | 量化配置            |
| AttentionConfig         | attention.py          | 注意力后端配置         |
| SpeculativeConfig       | speculative.py        | 推测解码配置          |
| ObservabilityConfig     | observability.py      | 可观测性配置          |
| CompilationConfig       | compilation.py        | 编译优化配置          |
| MultimodalConfig        | multimodal.py         | 多模态配置           |
| StructuredOutputsConfig | structured_outputs.py | 结构化输出配置         |
| ...                     | ...                   | 共 28 个模块        |

构建流程:

```
@dataclass
class VllmConfig:
    model_config: ModelConfig
    cache_config: CacheConfig
    parallel_config: ParallelConfig
```

```

scheduler_config: SchedulerConfig
device_config: DeviceConfig
load_config: LoadConfig
lora_config: LoRAConfig | None
# ... 20+ 其他配置字段

def __post_init__(self):
    # 验证配置一致性
    # 推导依赖配置
    # 设置默认值

```

## 7.2 SamplingParams 链式构建 源码位置: sampling\_params.py

SamplingParams 使用 Pydantic dataclass 支持灵活的参数组合:

```

@dataclass
class SamplingParams:
    n: int = 1
    temperature: float = 1.0
    top_p: float = 1.0
    top_k: int = -1
    max_tokens: int = 16
    stop: list[str] | None = None
    seed: int | None = None
    # ... 30+ 参数字段
    structured_output_params: StructuredOutputsParams | None = None
    repetition_detection_params: RepetitionDetectionParams | None = None

```

支持嵌套建造: - StructuredOutputsParams - JSON/Regex/Grammar 约束 - RepetitionDetectionParams - 重复检测参数

---

## 8. 原型模式 (Prototype Pattern)

核心思想: 通过复制现有对象来创建新对象, 而不是通过实例化。

### 8.1 Parallel Sampling 中的请求复制 源码位置: parallel\_sampling.py

当用户设置  $n > 1$  (并行采样) 时, vLLM 通过复制原始请求来创建多个子请求:

```

class ParentRequest:
    """Info, state & processing for parallel sampling request."""

    request_id: str
    external_req_id: str
    sampling_params: SamplingParams
    child_requests: set[str]
    output_aggregator: list[CompletionOutput]

    def __init__(self, request: EngineCoreRequest) -> None:
        self.sampling_params = request.params
        self.child_requests = set()
        self.output_aggregator = (
            [cast(CompletionOutput, None)] * sampling_params.n
            if (sampling_params.output_kind == RequestOutputKind.FINAL_ONLY)
            else []

```

```

    )

def _get_child_sampling_params(self, index: int) -> SamplingParams:
    """Efficiently obtain child sampling_params via copy."""
    seed = self.sampling_params.seed
    if self.cached_child_sampling_params:
        return self.cached_child_sampling_params
    # 关键: 复制父请求的采样参数
    child_sampling_params = copy(self.sampling_params)
    child_sampling_params.n = 1
    if seed is not None:
        child_sampling_params.seed = seed + index # 每个子请求唯一种子
    else:
        self.cached_child_sampling_params = child_sampling_params # 缓存复用
    return child_sampling_params

```

原型复制的工作流:

```

sequenceDiagram
    participant User
    participant Engine as LLMEngine
    participant PS as ParentRequest
    participant Child1 as ChildReq_0
    participant Child2 as ChildReq_1
    participant ChildN as ChildReq_n

    User->>Engine: generate(prompt, n=4)
    Engine->>PS: ParentRequest(request)
    PS->>Child1: copy(params) → child_0
    PS->>Child2: copy(params) → child_1
    PS->>ChildN: copy(params) → child_n
    Child1->>Engine: execute
    Child2->>Engine: execute
    ChildN->>Engine: execute
    Engine->>PS: aggregate outputs
    PS->>User: return n completions

```

**8.2 Sequence** 状态快照 Sequence 对象的状态快照机制, 用于 preempt 后恢复执行。

## 9. 模板方法模式 (Template Method Pattern)

核心思想: 在操作中定义算法的骨架, 将一些步骤延迟到子类中实现。

### 9.1 WorkerBase 执行骨架 源码位置: worker\_base.py

```

class WorkerBase:
    """Worker interface that allows vLLM to cleanly separate implementations
    for different hardware."""

    def __init__(self, vllm_config, local_rank, rank, distributed_init_method, is_driver_worker=False):
        # 通用的初始化逻辑
        self.vllm_config = vllm_config
        self.model_config = vllm_config.model_config
        # ... 提取所有子配置

```

```

        self.rank = rank
        self.local_rank = local_rank

    def get_kv_cache_spec(self) -> dict[str, KVCacheSpec]:
        """ 子类必须实现的抽象方法 """
        raise NotImplementedError

    def compile_or_warm_up_model(self) -> CompilationTimes:
        """ 子类必须实现的抽象方法 """
        raise NotImplementedError

    def determine_available_memory(self) -> list[int]:
        raise NotImplementedError

    def execute_model(self, ...):
        raise NotImplementedError

```

模板方法的类层次:

| 基类方法                                    | GPUWorker  | CPUWorker  | TPUWorker  | XPUWorker  |
|-----------------------------------------|------------|------------|------------|------------|
| <code>__init__()</code>                 | 继承         | 继承         | 继承         | 继承         |
| <code>get_kv_cache_spec()</code>        | CPU 实现     | CPU 实现     | TPU 实现     | XPU 实现     |
| <code>compile_or_warm_up_model()</code> | CPU warmup | CPU warmup | TPU warmup | XPU warmup |
| <code>execute_model()</code>            | CUDA 执行    | CPU 执行     | TPU 执行     | XPU 执行     |

## 9.2 AttentionBackend 骨架方法 源码位置: backend.py

```

class AttentionBackend(ABC):
    @staticmethod
    @abstractmethod
    def get_name() -> str: ...

    @staticmethod
    @abstractmethod
    def get_impl_cls() -> type["AttentionImplBase"]: ...

    @staticmethod
    def get_supported_kernel_block_sizes() -> list[int | MultipleOf]:
        return [MultipleOf(1)] # 默认实现, 子类可覆盖

```

## 9.3 ModelForCausalLM 基类骨架 模型基类定义了 `forward()` 的算法骨架, 各模型子类实现特定的注意力、FFN 等组件:

源码位置: interfaces.py

```

@runtime_checkable
class SupportsMultiModal(Protocol):
    """The interface required for all multi-modal models."""
    supports_multimodal: ClassVar[Literal[True]] = True
    # 定义多模态模型的接口契约

```

## 二、代码组织原则

### 2.1 分层职责分离（六层架构）

vLLM 采用清晰的分层架构，每一层有明确的职责边界：

```
graph TB
    subgraph API_层 ["API 层 (entrypoints/)"]
        OAI[OpenAI API]
        ANT[Anthropic API]
        SAG[Sagemaker]
        GRPC[gRPC Server]
    end

    subgraph 引擎层 ["引擎层 (engine/ + v1/engine/)"]
        ASYNC[AsyncLLMEngine]
        CORE[EngineCore]
        COORD[Coordinator]
    end

    subgraph 调度层 ["调度层 (v1/core/)"]
        SCHED[Scheduler]
        POLICY[调度策略]
    end

    subgraph 执行层 ["执行层 (v1/executor/ + v1/worker/)"]
        EXEC[Executor]
        WORKER[Worker]
        RUNNER[ModelRunner]
    end

    subgraph 模型层 ["模型层 (model_executor/)"]
        MODELS[模型实现]
        ATTENTION[注意力层]
        LINEAR[线性层]
        QUANTIZATION[量化]
    end

    subgraph 基础设施层 ["基础设施层"]
        CONFIG[28+ Config]
        DISTRIBUTED[分布式通信]
        KERNELS[CUDA Kernels]
        PLATFORMS[平台抽象]
    end

    API_层 --> 引擎层
    引擎层 --> 调度层
    调度层 --> 执行层
    执行层 --> 模型层
    模型层 --> 基础设施层

    style API_层 fill:#e3f2fd
    style 引擎层 fill:#e8f5e9
    style 调度层 fill:#fff3e0
    style 执行层 fill:#fce4ec
```

```
style 模型层 fill:#f3e5f5
style 基础设施层 fill:#e0f7fa
```

2.2 接口抽象 (Protocol/ABC 大量使用)

vLLM 广泛使用 Python 的 ABC (抽象基类)、Protocol (结构化子类型) 和 TypeAlias 来定义接口契约:

| 抽象类型      | 使用场景      | 示例                                                 |
|-----------|-----------|----------------------------------------------------|
| ABC       | 强制子类实现方法  | AttentionBackend,<br>WorkerBase, Executor          |
| Protocol  | 结构化鸭子类型   | SupportsMultiModal,<br>VllmModel,<br>TokenizerLike |
| TypeAlias | 类型别名增强可读性 | MultiModalEmbeddings,<br>ChatTemplatePath          |

关键接口定义位置: - interfaces.py - 模型能力接口 - interfaces\_base.py - 模型基础接口 - worker\_base.py - Worker 接口 - abstract.py - Executor 接口 - protocol.py - Tokenizer 接口

2.3 配置驱动开发 (28+ 配置模块控制行为)

vLLM 的行为几乎完全由配置驱动, 而非硬编码:

```
graph TB
    VC[VllmConfig<br/> 顶级配置] --> MC[ModelConfig]
    VC --> CC[CacheConfig]
    VC --> PC[ParallelConfig]
    VC --> SC[SchedulerConfig]
    VC --> DC[DeviceConfig]
    VC --> LC[LoadConfig]
    VC --> LoC[LoRAConfig]
    VC --> QC[QuantizationConfig]
    VC --> AC[AttentionConfig]
    VC --> SpC[SpeculativeConfig]
    VC --> OC[ObservabilityConfig]
    VC --> CoC[CompilationConfig]
    VC --> MMC[MultimodalConfig]
    VC --> SOC[StructuredOutputsConfig]
    VC --> RC[ReasoningConfig]
    VC --> KTC[KVTransferConfig]
    VC --> WC[WeightTransferConfig]
    VC --> OFC[OffloadConfig]
    VC --> PrC[ProfilerConfig]
    VC --> KC[KernelConfig]
    VC --> MaC[MambaConfig]
    VC --> ECC[ECCConfig]
    VC --> STC[SpeechToTextConfig]
    VC --> PoC[PoolerConfig]
    VC --> MAC[ModelArchConfig]
    VC --> KVC[KVEventsConfig]

    style VC fill:#e91e63,color:#fff
    style MC fill:#e3f2fd
    style CC fill:#e3f2fd
```

```
style PC fill:#e3f2fd
style SC fill:#e3f2fd
```

配置驱动的好处：1. 可测试性：可以通过注入不同配置测试各种场景 2. 可扩展性：新增功能只需添加新的配置模块 3. 灵活性：同一份代码支持多种部署模式

2.4 错误处理层次

vLLM 定义了清晰的异常层次结构：

| 异常类                 | 层级   | 触发场景            |
|---------------------|------|-----------------|
| EngineDeadError     | 致命   | Engine 进程崩溃不可恢复 |
| EngineGenerateError | 可恢复  | 单次生成失败，可重试      |
| ValidationError     | 用户错误 | 参数校验失败          |
| ValueError          | 配置错误 | 不支持的配置组合        |
| NotImplementedError | 开发错误 | 调用了未实现的方法       |

三、架构优势与权衡

3.1 性能 vs 灵活性的权衡

| 决策点                 | 选择            | 权衡                       |
|---------------------|---------------|--------------------------|
| PagedAttention      | 固定 block size | 内存效率 ↑ 但可能内部碎片           |
| Continuous Batching | 动态 batching   | 吞吐 ↑ 但调度复杂度 ↑            |
| CUDA Graphs         | 静态 shape 捕获   | 延迟 ↓ 但灵活性 ↓ (需要 padding) |
| Async Engine        | 异步事件循环        | 并发 ↑ 但调试难度 ↑             |
| Lazy Import         | 延迟导入          | 启动速度 ↑ 但首次调用有开销          |

3.2 抽象开销 vs 可维护性的平衡

vLLM 在以下方面选择了适度抽象：

- 1. 不过度设计：没有引入 DI 容件、AOP 框架等重量级设施
- 2. 实用主义：使用 Python 原生的 ABC/Protocol 而非第三方接口库
- 3. 渐进式复杂度：简单场景用字典注册，复杂场景用类继承

抽象带来的开销：- 虚函数调用开销（Python 层面可忽略）- 间接寻址开销（通过 registry 查找）- 类型检查和转换开销（isinstance, cast）

收益：- 200+ 模型的统一管理 - 多硬件后端的无缝切换 - 新功能的最小侵入式添加

3.3 同步 vs 异步的选择

```
graph LR
  subgraph 同步路径
    S1[LLMEngine] --> S2[同步 execute_model]
    S2 --> S3[阻塞等待结果]
  end

  subgraph 异步路径
    A1[AsyncLLMEngine] --> A2[异步 step]
    A2 --> A3[非阻塞 await]
    A3 --> A4[事件循环调度]
```

end

style 同步路径 fill:#e3f2fd

style 异步路径 fill:#e8f5e9

| 场景            | 选择                  | 原因                        |
|---------------|---------------------|---------------------------|
| API Server    | 异步 (AsyncLLMEngine) | 高并发、低延迟要求                 |
| Offline Batch | 同步 (LLMEngine)      | 简单直接、易调试                  |
| Worker 内部     | 同步                  | GPU 操作本身是同步的              |
| RPC 通信        | 异步可选                | non_block=True 时返回 Future |

## 四、可扩展性设计

### 4.1 如何添加新模型

步骤:

flowchart TD

```
A[1. 创建模型文件 <br/>models/my_model.py] --> B[2. 实现接口 <br/>SupportsMultiModal / VllmModel]
B --> C[3. 注册模型 <br/>_TEXT_GENERATION_MODELS]
C --> D[4. 添加配置 <br/>transformers_utils/configs/]
D --> E[5. 添加处理器 <br/>@MULTIMODAL_REGISTRY.register_processor]
E --> F[6. 测试验证]
```

最小代码示例:

```
# vllm/model_executor/models/my_model.py
from vllm.model_executor.models.interfaces import SupportsMultiModal

@ModelRegistry.register_model("MyModelForCausalLM")
class MyModelForCausalLM(nn.Module, SupportsMultiModal):
    supports_multimodal: ClassVar[Literal[True]] = True

    def forward(self, input_ids, positions, ...):
        # 实现前向传播
        pass

    def embed_input_ids(self, ...):
        # 实现嵌入
        pass
```

然后在 registry.py 中添加映射即可。

### 4.2 如何添加新硬件后端

通过平台抽象接口:

源码位置: platforms/interface.py

```
# 平台接口定义关键方法
class Platform:
    def get_device_name(self) -> str: ...
    def get_attn_backend_cls(self, backend, ...) -> type[AttentionBackend]: ...
    def is_cuda(self) -> bool: ...
    # ... 其他平台特定方法
```



已支持的硬件平台: | 平台 | 实现文件 | 状态 | |——|——-|——| | NVIDIA CUDA | cuda.py | 成熟 | | AMD ROCm | rocm.py | 成熟 | | Intel XPU | xpu.py | 发展中 | | Google TPU | tpu.py | 实验性 | | CPU | cpu.py | 成熟 | | ZenDNN CPU | zen\_cpu.py | 实验性 |

### 4.3 如何添加新量化方案

步骤:

1. 创建 **QuantizationConfig** 子类

```
# config/quantization.py 或独立文件
@dataclass
class MyQuantizationConfig(QuantizationConfig):
    quant_method: str = "my_quant"
    # 量化特有参数
```

2. 创建量化 **Linear** 层

```
# model_executor/layers/quantization/
class MyQuantizedLinear(LinearMethodBase):
    def create_weights(self, ...): ...
    def apply(self, ...): ...
```

3. 注册量化方法 在量化 linear 层工厂中注册新的量化方法。

### 4.4 如何添加新注意力后端

步骤:

1. 实现 **AttentionBackend** 抽象类

```
# v1/attention/backends/my_backend.py
class MyAttentionBackend(AttentionBackend):
    @staticmethod
    def get_name() -> str:
        return "my_attention"

    @staticmethod
    def get_impl_cls():
        return MyAttentionImpl

    @staticmethod
    def get_supported_kernel_block_sizes():
        return [16, 32, 64] # 支持的 block size
```

2. 实现 **AttentionImplBase**

```
class MyAttentionImpl(AttentionImplBase):
    def forward(self, query, key, value, ...):
        # 实现注意力计算
        pass
```

3. 在平台中注册

```
# platforms/my_platform.py
class MyPlatform(Platform):
    def get_attn_backend_cls(self, backend, ...):
        if backend == "my_attention":
            return MyAttentionBackend
        # ...
```

可扩展性设计总览:

```
graph TB
    subgraph 扩展点
        E1[ 新模型 <br/>ModelRegistry.register_model]
        E2[ 新硬件 <br/>Platform interface]
        E3[ 新量化 <br/>QuantizationConfig subclass]
        E4[ 新注意力 <br/>AttentionBackend subclass]
        E5[ 新 Tokenizer<br/>TokenizerRegistry.register]
        E6[ 新渲染器 <br/>RendererRegistry.register]
        E7[ 新指标 <br/>StatLoggerBase subclass]
        E8[ 新 API 协议 <br/>entrypoints/ new protocol]
    end

    style 扩展点 fill:#e8f5e9,stroke:#4caf50,stroke-width:2px
    style E1 fill:#c8e6c9
    style E2 fill:#c8e6c9
    style E3 fill:#c8e6c9
    style E4 fill:#c8e6c9
    style E5 fill:#c8e6c9
    style E6 fill:#c8e6c9
    style E7 fill:#c8e6c9
    style E8 fill:#c8e6c9
```

---

## 五、性能优化策略总结

### 5.1 六大核心优化技术

```
mindmap
    root((vLLM 性能优化))
        内存优化
            PagedAttention
                : 虚拟内存管理
                :Block 粒度分配
                :Copy-on-Write
            Prefix Caching
                : 公共前缀复用
                :RadixTree 组织
            KV Cache Offload
                :CPU 卸载
                : 跨实例共享
        吞吐优化
            Continuous Batching
                : 迭代级调度
                : 动态 batch 组装
            Chunked Prefill
                :Prefill/Decode 混合
                : 延迟均衡
            Speculative Decode
                :Draft-Verify
                :Medusa/Eagle/Eagle3
        计算优化
            CUDA Graphs
                :Kernel fusion
```

- : 消除 CPU 开销
- : 形状缓存
- Tensor Parallelism
  - :AllReduce 通信
  - : 重叠通信计算
  - :Custom AllReduce
- Attention Optimization
  - :FlashAttention
  - :xFormes
  - :PagedAttention kernel

## 5.2 各优化技术的设计模式支撑

| 优化技术                | 涉及的设计模式    | 关键代码位置                    |
|---------------------|------------|---------------------------|
| PagedAttention      | 模板方法 + 建造者 | core/kv_cache_manager.py  |
| Continuous Batching | 策略模式       | scheduler                 |
| CUDA Graphs         | 工厂 + 原型    | compilation/cuda_graph.py |
| Speculative Decode  | 策略 + 注册表   | spec_decode/              |
| Tensor Parallelism  | 适配器        | distributed/              |
| Multi-Modal         | 注册表 + 适配器  | multimodal/               |

## 5.3 性能优化的分层实现

graph TB

subgraph 应用层优化

A01[Continuous Batching]

A02[Chunked Prefill]

A03[Speculative Decode]

end

subgraph 运行时优化

R01[Async Scheduling]

R02[CUDA Graphs]

R03[Kernel Fusion]

end

subgraph 算法优化

AL1[PagedAttention]

AL2[FlashAttention]

AL3[Custom Kernels]

end

subgraph 系统优化

S01[Tensor Parallelism]

S02[Pipeline Parallelism]

S03[Memory Management]

S04[NCCL/IPC 通信]

end

style 应用层优化 fill:#e3f2fd

style 运行时优化 fill:#e8f5e9

style 算法优化 fill:#fff3e0

style 系统优化 fill:#fce4ec

六、与竞品对比视角（架构层面）

6.1 vs HuggingFace Transformers

| 维度       | HuggingFace Transformers | vLLM                                            |
|----------|--------------------------|-------------------------------------------------|
| 定位       | 训练 + 推理通用框架              | 推理专用引擎                                          |
| 架构理念     | 模型为中心                    | 服务为中心                                           |
| 内存管理     | 全量序列分配                   | PagedAttention<br>虚拟内存                          |
| Batching | 静态 pad 到固定长度             | Continuous<br>Batching 动态调度                     |
| 注意力      | 标准 PyTorch 实现            | FlashAttention<br>+<br>PagedAttention<br>Kernel |
| 并行       | 基础 TP/PP                 | TP/PP/DP +<br>Custom<br>AllReduce               |
| 扩展性      | AutoModel + Config       | Registry +<br>Plugin 系统                         |
| 性能目标     | 易用性优先                    | 吞吐/延迟优先                                         |

架构差异的本质：Transformers 追求通用性和易用性，vLLM 追求极致推理性能。这导致 vLLM 在内存管理、调度、kernel 层面做了大量深度优化。

6.2 vs Text Generation Inference (TGI)

| 维度          | TGI                  | vLLM                                   |
|-------------|----------------------|----------------------------------------|
| KV Cache 管理 | 连续内存分配               | <b>PagedAttention</b> (差异化核心)          |
| 注意力实现       | FlashAttention v1/v2 | FlashAttention + xFormes + 自研 kernels  |
| Batching    | 连续 batching          | Continuous Batching + Chunked Prefill  |
| 推测解码        | 有限支持                 | Medusa + Eagle + Eagle3 + MLA + DFlash |
| 多模态         | 基础支持                 | 深度集成 (图像/音频/视频)                        |
| 量化          | GPTQ/AWQ/FP8         | GPTQ/AWQ/FP8/FP4/Marlin + 更多           |
| 生态活跃度       | 中等                   | 极高 (社区贡献主导)                            |

**PagedAttention** 的战略价值：这是 vLLM 相对于 TGI 的核心差异化技术，使其在 memory-bound 场景（长上文、大 batch）下具有显著优势。

6.3 vLLM v0 → v1 架构演进

| 方面        | v0           | v1                                     | 改进动机          |
|-----------|--------------|----------------------------------------|---------------|
| Engine    | 单体 LLMEngine | 分离 AsyncLLM + EngineCore + Coordinator | 解耦调度与执行       |
| Worker    | 直接通信         | Executor 抽象层                           | 支持多种部署模式      |
| Metrics   | 硬编码日志        | StatLogger 插件系统                        | 可扩展的指标收集      |
| Attention | 后端选择分散       | 统一 AttentionSelector                   | 更清晰的策略模式      |
| KV Cache  | 单一实现         | KVCacheManager + Interface             | 支持异构 KV Cache |
| Config    | 扁平配置         | 层次化 VllmConfig                         | 更好的组织性        |
| Sampling  | 耦合在 Engine   | 独立采样器组件                                | 关注点分离         |

v1 架构改进的核心趋势：从”能跑”到”优雅”，提升可维护性的同时不牺牲性能。

## 七、未来发展方向推断

基于当前代码结构和社区趋势，推断可能的演进方向：

### 7.1 架构层面

```
timeline
    title vLLM 可能的演进方向
    2024-Q4 : v1 架构稳定化
              : Executor/Worker 解耦完成
              : Metrics 插件化
    2025-Q1 : 多模态深度整合
              : 统一 MM Pipeline
              : Video/Audio 原生支持
    2025-Q2 : 推测解码标准化
              : 通用 Draft Model 接口
              : 自适应 Spec Decode
    2025-Q3 : 分布式推理增强
              : 弹性伸缩
              : 跨节点 KV Transfer
              : Fault Tolerance
    2025-Q4 : Compiler Integration
              : Torch Compile 深度集成
              : 图优化 Pass
              : 动态 Shape 支持
```

### 7.2 从代码结构推测的具体方向

#### 1. Compiler-Native Architecture

当前 compilation/ 目录已经包含 passes/、backends.py、codegen.py 等编译器基础设施。预计未来会：- 更深度集成 torch.compile - 自定义 Triton/CUDA graph optimization passes - 运行时 JIT 编译支持

#### 2. 弹性分布式推理

v1/executor/ray\_executor\_v2.py 和 distributed/elastic\_ep/ 已有弹性执行的初步实现：- 节点动态加入/退出 - 故障自动恢复 - 负载感知调度

3. Multimodal First-Class Citizen

multimodal/ 已经具备完整的注册和处理框架，但视频原生支持仍在发展中。预计：- 统一的 multimodal pipeline - 跨模态 attention 优化 - 模态感知的 scheduling

4. Structured Output 深度集成

structured\_output/ 已支持 xgrammar、lm-format-enforcer、outlines 等后端，未来可能：- 与 sampling 深度融合 - Constraint-aware KV cache - 并行 decoding with constraints

5. Observability 增强

observability\_config 和 tracing/ 已有 OpenTelemetry 集成基础，预计：- 更细粒度的 tracing span - 自动化 profiling - Performance regression detection

7.3 长期架构挑战

| 挑战                  | 当前状态                    | 可能方向                                     |
|---------------------|-------------------------|------------------------------------------|
| Long Context        | 支持 1M+ tokens           | Hierarchical Attention, Sparse Attention |
| MoE Scaling         | DeepSeek-V3/V4, Mixtral | Expert-level Parallelism, Load Balancing |
| Edge Deployment     | 基础 CPU/XPU 支持           | 模型压缩 + 量化一体化                             |
| Serving at Scale    | 多实例部署                   | Global Scheduler, Request Routing        |
| Safety & Governance | 基础支持                    | Input/Output Guardrails, Audit Logging   |

附录：设计模式快速参考

| 模式        | vLLM 应用                   | 核心文件                           | 复杂度 |
|-----------|---------------------------|--------------------------------|-----|
| Strategy  | 注意力后端/调度策略                | v1/attention/selector.py       |     |
| Factory   | Executor/Worker/Connector | v1/executor/abstract.py        |     |
| Observer  | Metrics 收集/KV Events      | v1/metrics/loggers.py          |     |
| Registry  | 模型/Tokenizer/渲染器          | models/registry.py             |     |
| Decorator | 处理器注册/Op 注册               | multimodal/registry.py         |     |
| Adapter   | API 协议/渲染器                | entrypoints/anthropic/         |     |
| Builder   | VllmConfig/SamplingParams | config/vllm.py                 |     |
| Prototype | Parallel Sampling         | v1/engine/parallel_sampling.py |     |
| Template  | Worker/Attention Backend  | v1/worker/worker_base.py       |     |
| Method    |                           |                                |     |

总结

vLLM 的架构体现了实用主义的工程设计哲学：

- 1. 不过度抽象：使用 Python 原生特性（ABC、Protocol、dataclass）而非引入重型框架
- 2. 注册表驱动：通过 6 大注册表体系实现 200+ 模型和多硬件后端的统一管理
- 3. 配置驱动：28+ 配置模块精确控制系统行为的每一个维度
- 4. 性能优先：PagedAttention、CUDA Graphs、Continuous Batching 等核心技术构成竞争壁垒
- 5. 渐进式演进：从 v0 到 v1 的重构展示了清晰的架构升级路径

这套设计使得 vLLM 在保持高性能的同时，具备了良好的可维护性和极强的可扩展性，成为 LLM 推理领域的事实标准。